Alex Goessmann

# Artificial Intelligence in the Tensor Formalism

October 12, 2026

Springer Nature

# Contents

**Notation** . . . . . xiii
1.1 Tensors . . . . . xiii
1.2 Tensor networks and contractions . . . . . xiv
1.3 Generic contractions . . . . . xv
1.4 Normalizations . . . . . xvi
1.5 Function encoding and Computation-Activation Networks . . . . . xvi

**Part I Foundations of Artificial Intelligence**

**2 Propositional Logic** . . . . . 3
2.1 Propositonal Formulas . . . . . 3
2.1.1 Knowledge bases . . . . . 4
2.2 Entailment . . . . . 4
2.2.1 Deciding Entailment by Contractions . . . . . 5
2.2.2 Deciding Entailment by Partial Ordering . . . . . 5
2.2.3 Redundancy of Entailed Formulas . . . . . 6
2.2.4 Contraction Knowledge Base . . . . . 6
2.3 Syntactical decompositions . . . . . 7
2.3.1 Encoding of Booleans . . . . . 8
2.3.2 Basis encodings . . . . . 9
2.3.3 Circuit decomposition . . . . . 11
2.3.4 Atomic Formulas . . . . . 11
2.3.5 Syntactical Composition of Formulas . . . . . 12
2.3.6 Minterm and maxterm decompositions . . . . . 15
2.3.7 Outlook . . . . . 17
2.4 Inference algorithms . . . . . 17
2.4.1 Monotonicity of Entailment . . . . . 17
2.4.2 Syntactical inference algorithms . . . . . 18
2.4.3 Backward Chaining in Horn Logics . . . . . 18

**3 Constraint Satisfaction Problems** . . . . . 21
3.1 Representation . . . . . 21
3.1.1 Manipulation of constraint tensor networks . . . . . 21
3.1.2 Knowledge bases as constraint networks . . . . . 22
3.1.3 Exactly-one and almost-one constraint . . . . . 23
3.2 Inference . . . . . 27
3.2.1 Solution by disentanglement . . . . . 27
3.2.2 Generlized unit propagation . . . . . 27
3.2.3 Constraint Propagation . . . . . 28
3.2.4 Applications . . . . . 31

3.2.5 Backtracking search . . . 31
3.2.6 Discussion . . . 32

**4 First-Order Logic** . . . 33
4.1 Syntax and Semantics of first-order logic . . . 33
4.1.1 Syntax . . . 33
4.1.2 Tensor Representation of Semantics . . . 33
4.1.3 Worlds . . . 34
4.1.4 Two Tensor Structures . . . 34
4.2 Substitution Structure . . . 35
4.2.1 Grounding Tensors . . . 35
4.2.2 Basis encodings of terms . . . 35
4.2.3 Atomic sentences . . . 36
4.2.4 Formula Synthesis by Connectives . . . 36
4.2.5 Quantifiers . . . 37
4.2.6 Universally quantified formulas as contraction equations . . . 39
4.2.7 Discussion . . . 39
4.3 Tensor storage of Groundings . . . 39
4.3.1 Storage in Basis CP Decomposition . . . 39
4.3.2 Storage in Tensor Network Formats . . . 40
4.3.3 Example: Relational Databases . . . 41
4.3.4 Sparsification . . . 42
4.4 Representation of Knowledge Graphs . . . 42
4.4.1 Representation as Unary and Binary Predicates . . . 42
4.4.2 Representation as Ternary Predicate . . . 42
4.4.3 SPARQL Queries . . . 43
4.5 Semantic structure . . . 45
4.5.1 Database semantics . . . 46
4.5.2 World Enumerating Variables . . . 46
4.5.3 Representation of Terms and Formulas . . . 46
4.5.4 Case of Propositional Logic . . . 47
4.5.5 One-hot Encoding of Worlds . . . 47

**5 Probability Theory** . . . 49
5.1 Tensor Formalism in Probability Theory . . . 49
5.1.1 Joint Probability Distributions . . . 49
5.1.2 Base Measures . . . 50
5.1.3 Marginal Distribution . . . 51
5.1.4 Conditional Probabilities . . . 52
5.1.5 Bayes Theorem and the Chain Rule . . . 54
5.2 Independence as a Decomposition Mechanism . . . 54
5.3 Graphical Models . . . 58
5.3.1 Markov Networks . . . 58
5.3.2 Bayesian Networks . . . 60
5.3.3 Bayesian Networks as Markov Networks . . . 62
5.4 Factorization based on sufficient statistics . . . 64
5.5 Efficient contractions by message passing . . . 66
5.6 Queries . . . 68
5.6.1 Querying by Functions . . . 68
5.6.2 Mode Queries . . . 69
5.6.3 Energy Representations . . . 70
5.7 Sampling . . . 70
5.7.1 Forward Sampling . . . 71
5.7.2 Gibbs Sampling . . . 71

5.7.3 Simulated Annealing . . . 72
5.8 Maximum Likelihood Estimation . . . 73
5.8.1 Empirical Distributions . . . 73
5.8.2 Likelihood Loss . . . 75
5.8.3 Entropic Interpretation . . . 76

**6 Causality** . . . 79
6.1 Association . . . 79
6.1.1 Simplification using directed contractions . . . 80
6.1.2 Statistical influence . . . 80
6.2 Intervention . . . 81
6.2.1 Do calculus . . . 82
6.2.2 Intervention Queries . . . 83
6.2.3 Simplifications . . . 84
6.3 Counterfactuals . . . 87
6.3.1 Response Variables . . . 88
6.3.2 Counterfactual Twinned Network . . . 90
6.4 Learning . . . 91

**Part II Contraction Calculus**

**7 Basis calculus** . . . 95
7.1 Basis Encoding of Subsets . . . 95
7.1.1 Binary Relations . . . 96
7.1.2 Higher-Order Relations . . . 96
7.2 Basis Encoding of Functions . . . 98
7.2.1 Definition . . . 98
7.2.2 Function Evaluation . . . 99
7.3 Circuits . . . 99
7.3.1 Function decomposition . . . 100
7.3.2 Function evaluation by message passing . . . 103
7.4 Hybrid Basis and Coordinate Calculus . . . 105

**8 Message Passing Schemes** . . . 109
8.1 Semiring contraction . . . 109
8.1.1 Foundations . . . 109
8.1.2 Examples . . . 110
8.1.3 Tensor network formats . . . 112
8.1.4 Basis calculus . . . 113
8.2 Contraction of tree tensor networks . . . 113
8.2.1 Tree message passing . . . 113
8.2.2 Analysis . . . 113
8.3 Boolean message passing . . . 114
8.3.1 Properties of the Boolean semiring . . . 115
8.3.2 Application on sum-product contractions . . . 115
8.3.3 Generic support tensors . . . 115
8.4 Hypergraph manipulations . . . 116
8.4.1 Bethe hypergraphs . . . 116
8.4.2 Coarse graining . . . 117

**9 Coordinate calculus** . . . 121
9.1 Group fields . . . 121
9.2 Fourier transform . . . 121
9.2.1 Unfolding . . . 121
9.2.2 Tensor network decomposition . . . 122
9.2.3 Fast Fourier Transform . . . 123
9.2.4 Generic groups . . . 124

**10 Sparse Calculus** . . . 127
10.1 CP Decomposition . . . 127
10.1.1 Directed Leg Cores . . . 128
10.1.2 Basis CP Decompositions and the $\ell_0$-norm . . . 129
10.1.3 Basis+ CP Decompositions and Polynomials . . . 130
10.2 Constructive Bounds on CP Ranks . . . 133
10.2.1 Cascade of Ranks . . . 133
10.2.2 Operations on CP Decompositions . . . 133
10.3 Sparse Encoding of Functions . . . 136
10.4 Reed-Muller expansions . . . 138
10.4.1 Rank bound . . . 139

**11 Languages and Automata** . . . 141
11.1 Languages . . . 141
11.2 Deterministic Finite Automata . . . 141
11.2.1 Outlook . . . 142
11.3 Nondeterministic Finite Automata . . . 142
11.3.1 Stochastic Finite Automata . . . 142

**12 Quantum circuits** . . . 143
12.1 Foundations . . . 143
12.1.1 Quantum and Tensor Parallelism . . . 143
12.1.2 Quantum states and quantum circuits . . . 143
12.1.3 Controlled single qubit gates . . . 144
12.1.4 Measurements and Contractions . . . 144
12.2 Directed Circuits . . . 147
12.2.1 Definition . . . 147
12.2.2 Preparation of q-samples to Bayesian Networks . . . 148
12.2.3 Equivalence with Bayesian Networks . . . 150
12.2.4 Ancilla Augmentation . . . 151
12.3 Computation circuits . . . 158
12.3.1 Relation with basis encodings . . . 159
12.3.2 Circuit decomposition sparsity . . . 159
12.3.3 Circuit polynomial sparsity . . . 161
12.3.4 Function Encoding . . . 163
12.4 Quantum algorithms . . . 165
12.4.1 Quantum Fourier transform . . . 165
12.4.2 Quantum Phase Estimation . . . 168
12.4.3 Grover operator . . . 169
12.4.4 Moment Estimation . . . 172
12.5 Conclusion . . . 174

**Part III Computation-Activation Networks**

**13 Elementary Computation-Activation Networks** . . . 177
13.1 Exponential Families . . . 177
13.1.1 Definition . . . 177
13.1.2 Tensor Network Representation . . . 180
13.1.3 Markov Networks as Exponential Families . . . 182
13.1.4 Representation of Generic Distributions . . . 184

**14 Mean Polytopes and Maximum Entropy Distributions** . . . 185
14.1 Maximum entropy problem . . . 185
14.2 The mean polytope . . . 186
14.2.1 Faces . . . 186
14.2.2 Representation in the CP format . . . 189
14.2.3 Examples of elementarily representable faces . . . 190
14.2.4 Partition into relative interiors of faces . . . 191
14.3 Exponential families . . . 192
14.3.1 Representation as elementary Computation-Activation Networks . . . 192
14.3.2 Maximum entropy distributions . . . 194
14.4 Generic maximum entropy distributions . . . 195
14.4.1 Main result . . . 195
14.4.2 Family of maximum entropy distributions . . . 196
14.5 Boolean statistics . . . 197
14.5.1 Characterization of elementary representable faces . . . 197
14.5.2 Interpretation by propositional formulas . . . 199
14.5.3 Faces activated in the TT format . . . 202
14.5.4 Construction of Boolean statistics to polytopes . . . 203
14.5.5 Examples in statistical physics and coding theory . . . 204
14.6 Mean as a Statistic . . . 205
14.7 Face Computation . . . 206
14.7.1 Examples . . . 206
14.7.2 Generic Construction . . . 207
14.7.3 Relation with Facet Complexity . . . 208

**15 Variational Inference** . . . 209
15.1 Foundational inference problems . . . 209
15.1.1 Forward mapping . . . 209
15.1.2 Backward mapping . . . 209
15.2 Face estimation . . . 210
15.2.1 Cube-like faces . . . 210
15.2.2 Face refinement . . . 211
15.3 Parameter estimation . . . 211
15.3.1 Moment matching . . . 211
15.3.2 Issues with erroneous faces . . . 212

**16 Inductive Reasoning** . . . 213

**17 Probabilistic Guarantees** . . . 215
17.1 Fluctuations . . . 215
17.1.1 Widths of the noise vector . . . 215
17.1.2 Recovery guarantees given width bounds . . . 215
17.2 Face recovery . . . 215

**18 Quantum inference** . . . 217
18.1 Introduction . . . 217
18.1.1 Circuit Preparation schemes . . . 217
18.1.2 Related Works . . . 218
18.2 Preparation of q-samples to CompActNets . . . 218
18.2.1 Sampling from Computation-Activation Networks . . . 219
18.3 Quantum Rejection Sampling . . . 220
18.3.1 Acceptance Amplification . . . 221
18.3.2 Optimal rotation numbers . . . 221
18.3.3 Applications on CompActNets . . . 222
18.3.4 Examples . . . 222
18.4 Moment Amplification . . . 222
18.4.1 Interpretation of Grover rotations . . . 222
18.4.2 Moment Matching . . . 224
18.5 Extension: Sampling from proposal distributions . . . 225

**19 Probabilistic Relational Models** . . . 227
19.1 Ideas . . . 227
19.2 Inference in FOL . . . 227
19.2.1 Manipulation of contraction equations . . . 227
19.2.2 Unification . . . 228
19.2.3 Modus ponens . . . 228
19.2.4 Contraction equations . . . 230
19.3 Probabilistic Relational Models . . . 231
19.3.1 Maximum entropy distributions on counted formulas . . . 231
19.3.2 Hybrid First-Order Logic Networks . . . 232
19.3.3 Propositionalization . . . 232
19.3.4 Conditioning on an Importance Formula . . . 233
19.3.5 Decomposition of the Log Likelihood . . . 234
19.3.6 Reduction to Propositional Logic . . . 234
19.3.7 Sample Extraction from First-Order Logic Worlds . . . 237
19.4 Generation of First-Order Logic Worlds . . . 240
19.4.1 Samples by Single Objects . . . 241
19.4.2 Samples by Tuples of Objects . . . 242
19.5 Discussion . . . 243
References . . . 245

# Acronyms

| | |
|---|---|
| TT | Tensor-Train format |
| CP | Canonical Polyadic format |
| CSP | Constraint Satisfaction Problem |
| CAN | Computation-Activation Network |
| HLN | Hybrid Logic Network, Computation-Activation Network in case of binary variables |
| PRM | Probabilistic Relational Model |

# Chapter 1
# Notation

**Abstract** We introduce the notation used in this book.

In this section, we introduce the hypergraph-based tensor network formalism and define the most general tensor network architecture of CompActNets based on this formalism.

## 1.1 Tensors

Tensors are multiway arrays and a generalization of vectors and matrices to higher orders. We first provide a formal definition as real maps from index sets enumerating the coordinates of vectors, matrices, and higher-order tensors. To ease the notation, we abbreviate sets as $[d] = \{0, \ldots, d-1\}$, tuples of state indices by $x_{[d]} = (x_0, \ldots, x_{d-1})$ and tuples of variables by $X_{[d]} = (X_0, \ldots, X_{d-1})$.

**Definition 1.1 (Tensor)** For $k \in [d]$, let $m_k \in \mathbb{N}$ and let $X_k$ be categorical variables taking values in $[m_k]$. A tensor $\tau\,[X_0, \ldots, X_{d-1}]$ of order $d$ and leg dimensions $m_0, \ldots, m_{d-1}$ is defined through its coordinates

$$\tau\left[X_{[d]} = x_{[d]}\right] = \tau\,[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}] \in \mathbb{R}$$

for index tuples

$$x_{[d]} = (x_0, \ldots, x_{d-1}) \in \bigtimes_{k \in [d]} [m_k]\,.$$

Tensors $\tau\left[X_{[d]}\right]$ are elements of the tensor space

$$\bigotimes_{k \in [d]} \mathbb{R}^{m_k}\,,$$

which is a linear space, enriched with the operations of coordinate-wise summation and scalar multiplication. We call a tensor $\tau\left[X_{[d]}\right]$ boolean, when all coordinates are in $\{0, 1\}$, and positive, when all coordinates are greater than 0.

We introduced tensors here in a non-canonical way based on categorical variables assigned to their axes. While this may look like syntactic sugar at this point, it allows us to define contractions without further specification of axes, based on comparisons of shared variables. We occasionally also allow for variables $X$ taking values in infinite sets such as $\mathbb{R}$, in which case we denote the set of values to a variable by $\mathrm{val}\,(X)$.

*Example 1.1 (Delta tensor)* Given a tuple of variables $X_{[d]} = (X_0, \ldots, X_{d-1})$ with identical dimension $m$, where $d \geq 1$, the delta tensor is the element

$$\delta^{[d],m}\left[X_{[d]}\right] \in \bigotimes_{k \in [d]} \mathbb{R}^m$$

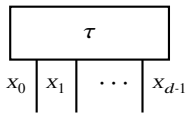
with coordinates

$$\delta^{[d],m}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if} \quad x_0 = \ldots = x_{d-1} \\ 0 & \text{else} \end{cases}.$$

We depict this tensor by black dots, which sometimes appears as auxiliary elements in tensor network diagrams (see e.g. Figure 1.3). For $d = 1$, the delta tensor is the trivial vector, whose coordinates are constantly 1, which we denote by $\mathbb{I}[X]$.

## 1.2 Tensor networks and contractions

We use a standard visualization of tensors (dating back to [66]) by blocks with lines depicting the axes of the tensor. Additionally, we assign to each axis of the tensor the corresponding variable $X_k$:




We now associate categorical variables with nodes of a hypergraph and tensors with hyperedges, which are arbitrary subsets of nodes. Based on this association we continue with the definition of tensor networks.

**Definition 1.2 (Tensor network)** Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a hypergraph, let $X_v$ for $v \in \mathcal{V}$ be categorical variables with dimensions $m_v \in \mathbb{N}$, and let

$$\tau^e\left[X_e\right] \in \bigotimes_{v \in e} \mathbb{R}^{m_v}$$

be tensors for $e \in \mathcal{E}$, where we denote by $X_e$ the set of categorical variables $X_v$ with $v \in e$. Then, we call the set

$$\tau^{\mathcal{G}}\left[X_{\mathcal{V}}\right] = \left\{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\right\}$$

the tensor network of the decorated hypergraph $\mathcal{G}$. The set of tensor networks on $\mathcal{G}$ such that all tensors have non-negative coordinates is denoted by $\mathcal{T}^{\mathcal{G}}$.

As examples we now present the CP and the TT formats in our hypergraph notation.

*Example 1.2 (The* CP *format)* The Candecomp-Parafac (CP) tensor format (see [39]) corresponds in our notation to a hypergraph (see Figure 1.1) defined by

- nodes $X_{[d]}$ and a single hidden variable $I$, decorated by dimensions $m_{[d]}$ and the CP-rank $n$, respectively
- edges $\left\{e_k = \{X_k, I\} \,:\, k \in [d]\right\}$ each decorated by a matrix $\tau^{e_k}\left[X_k, I\right] \in \mathbb{R}^{m_k \times n}$.

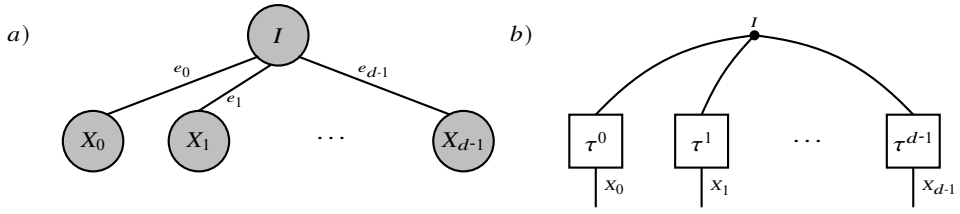



**Fig. 1.1** Hypergraph to a CP format (see Example 1.2). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

*Example 1.3 (The* TT *format)* The Tensor-Train (TT) format (see [61]) corresponds in our notation to a hypergraph (see Figure 1.2) defined by

- nodes $X_{[d]}$ and hidden variables $I_{[d-1]}$, each decorated by a dimension $m_{[d]}$ and $n_{[d-1]}$,
- edges

$$\big\{e_0 = \{X_0, I_0\}\big\} \cup \big\{e_k = \{I_{k-1}, X_k, I_k\} \,:\, k \in \{1, \dots, d-2\}\big\} \cup \big\{e_{d-1} = \{I_{d-2}, X_{d-1}\}\big\}$$

each decorated by a tensor of order 3 (respectively 2 for $k \in \{0, d-1\}$).

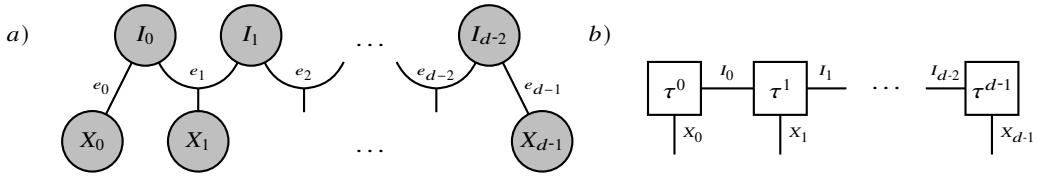



**Fig. 1.2** Hypergraph to a TT format (see Example 1.3). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

## 1.3 Generic contractions

Let us now exploit our graphical approach to tensor networks in the definition of contractions.

**Definition 1.3** Let $\tau^{\mathcal{G}}$ be a tensor network on a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. For any subset $\mathcal{U} \subset \mathcal{V}$ we define the contraction of $\tau^{\mathcal{G}}$ with open variables $X_{\mathcal{U}}$ to be the tensor (for an example see Figure 1.3)

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]} \in \bigotimes_{v \in \mathcal{U}} \mathbb{R}^{m_v}$$

with coordinates at indices $x_{\mathcal{U}} \in \bigtimes_{v \in \mathcal{U}} [m_v]$ by

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \sum_{x_{\mathcal{V} \setminus \mathcal{U}} \in \bigtimes_{v \in \mathcal{V} \setminus \mathcal{U}} [m_v]} \left( \prod_{e \in \mathcal{E}} \tau^e \left[ X_e = x_e \right] \right).$$

When an open variable $X$ does not appear in any tensor in a contraction, we define the contraction as a tensor product with the trivial tensor $\mathbb{I}[X]$ (see Example 1.1). To ease notation, we often omit the set notation by brackets $\{\cdot\}$.

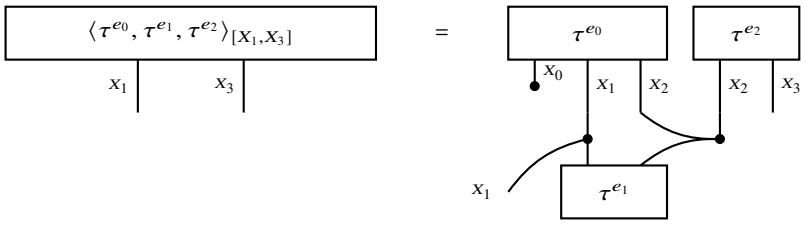



**Fig. 1.3** Graphical depiction of a tensor network contraction with the open variables $X_1, X_3$. Open variables are depicted by those without a dot at the end of the line.

*Example 1.4 (Tensor product)* The simplest contraction is the tensor product, which maps a pair of two tensors with distinct variables onto a third tensor and has an interpretation by coordinate-wise products. Such a contraction corresponds with a tensor network of two tensors with disjoint variables. Let there be two tensors

$$\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k} \quad \text{and} \quad \tilde{\tau}\left[Y_{[p]}\right] \in \bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}$$

with disjoint tuples of categorical variables assigned to their axes. Then their tensor product is the tensor

$$\left\langle \tau\left[X_{[d]}\right], \tilde{\tau}\left[Y_{[p]}\right]\right\rangle_{[X_{[d]},Y_{[p]}]} \in \left(\bigotimes_{k\in[d]} \mathbb{R}^{m_k}\right) \otimes \left(\bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}\right)$$

with coordinates to tuples of $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $y_{[p]} \in \bigtimes_{\ell\in[p]}[n_\ell]$ as

$$\begin{aligned}&\left\langle \tau\left[X_{[d]}\right], \tilde{\tau}\left[Y_{[p]}\right]\right\rangle_{[X_{[d]}=x_{[d]},Y_{[p]}=y_{[p]}]}\\ &\quad := \tau\left[X_{[d]} = x_{[d]}\right] \cdot \tilde{\tau}\left[Y_{[p]} = y_{[p]}\right] .\end{aligned}$$

## 1.4 Normalizations

Based on generic contractions, we now introduce the normalization of tensors, which introduces certain contraints on tensors to be depicted by directed hyperedges.

**Definition 1.4** The normalization of a tensor $\tau\left[X_{\mathcal{V}}\right]$ on incoming nodes $\mathcal{V}^{\text{in}} \subset \mathcal{V}$ and outgoing nodes $\mathcal{V}^{\text{out}} \subset \mathcal{V}\backslash\mathcal{V}^{\text{in}}$ is the tensor $\langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}^{\text{out}}}|X_{\mathcal{V}^{\text{in}}}\right]}$ defined for $x_{\mathcal{V}^{\text{in}}}$ as

$$\langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}^{\text{out}}}|X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]} = \begin{cases} \frac{\langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]}}{\langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]}} & \text{if} \quad \langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]} \neq 0 \\ \frac{1}{\prod_{v\in\mathcal{V}^{\text{out}}} m_v}\mathbb{I}\left[X_{\mathcal{V}^{\text{out}}}\right] & \text{else} \end{cases} .$$

We say that $\tau\left[X_{\mathcal{V}}\right]$ is normalized with incoming nodes $\mathcal{V}^{\text{in}} \subset \mathcal{V}$, if

$$\tau\left[X_{\mathcal{V}}\right] = \langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}\backslash\mathcal{V}^{\text{in}}}|X_{\mathcal{V}^{\text{in}}}\right]} .$$

In our graphical tensor notation, we depict normalized tensors by directed hyperedges (a), which are decorated by directed tensors (b), for example when $X_{\mathcal{V}^{\text{in}}} = (X_2, X_3)$ and $X_{\mathcal{V}\backslash\mathcal{V}^{\text{in}}} = (X_0, X_1)$:




## 1.5 Function encoding and Computation-Activation Networks

Towards presenting the function encoding schemes, we define one-hot encodings mapping the states of variables to basis tensors.

**Definition 1.5 (One-hot encoding)** To any variable $X$ taking values in $[m]$, the one-hot encoding of any state $x \in [m]$ is the vector with coordinates

$$\epsilon_x\left[X = \tilde{x}\right] := \begin{cases} 1 & \text{if} \quad x = \tilde{x} \\ 0 & \text{else} . \end{cases}$$

To any tuple $X_{[d]}$ of variables taking values in $\bigtimes_{k\in[d]}[m_k]$, the one-hot encoding of a state tuple $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}}\left[X_{[d]}\right] := \bigotimes_{k\in[d]} \epsilon_{x_k}\left[X_k\right] .$$

We now use one-hot encodings to encode functions between state sets.

**Definition 1.6 (Basis encoding of maps between state sets)** Let there be two sets of variables $X_{[d]}$ and $Y_{[p]}$, and let there be a map

$$q : \bigtimes_{k\in[d]} [m_k] \to \bigtimes_{\ell\in[p]} [n_\ell]$$

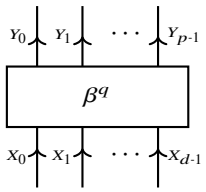
between their state sets. Then, the basis encoding of $q$ is a tensor

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] \in \left(\bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}\right) \otimes \left(\bigotimes_{k\in[d]} \mathbb{R}^{m_k}\right)$$

defined by

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \epsilon_{q(x_{[d]})}\left[Y_{[p]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] .$$

Basis encodings are normalized tensors and are thus depicted as decorations of directed edges in hypergraphs:




We further generalize basis encodings to arbitrary functions between finite sets by the use of bijective image enumeration maps. Given an arbitrary set $\mathcal{U}$, we say a map

$$I : \bigtimes_{k\in[d]} [m_k] \to \mathcal{U}$$

is an enumeration map of $\mathcal{U}$ by $d$ variables $X_k$, taking values in $m_k$. Given a function $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ between arbitrary sets and enumerating maps $I_{\text{in}}$ and $I_{\text{out}}$ for both sets, we define the basis encoding of $q$ as

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \sum_{u\in\mathcal{U}^{\text{in}}} \epsilon_{I_{\text{out}}^{-1}(q(u))}\left[Y_{[p]}\right] \otimes \epsilon_{I_{\text{in}}^{-1}(u)}\left[X_{[d]}\right] ,$$

where $X_{[d]}, Y_{[p]}$ are variables taking values in $[\left|\mathcal{U}^{\text{in}}\right|]$ and $[\left|\mathcal{U}^{\text{out}}\right|]$. In Example 7.2 we present index enumeration maps for summations in $m$-adic integer representations. Based on these concepts, we define the most general tensor network architecture to be applied in the rest of this work.

**Definition 1.7 (Computation-Activation Network (CompActNets))** Let there be a function $t : \bigtimes_{k\in[d]}[m_k] \to \mathbb{R}^p$ with basis encoding $\beta^t\left[Y_{[p]}, X_{[d]}\right]$, where $Y_{[p]}$ is a tuple of variables to an enumeration map of the image of $t$. Let there further be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathcal{V}$ containing $[p]$. We define the by $t$ computable and by $\mathcal{G}$ activated family of distributions by

$$\Lambda^{t,\mathcal{G}} = \left\{ \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \langle\xi\rangle_{\left[Y_{[p]}\right]} \right\rangle_{\left[X_{[d]}|\varnothing\right]} : \xi\left[Y_{\mathcal{V}}\right] \in \mathcal{T}^{\mathcal{G}} \right\} .$$

We refer to any member $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{t,\mathcal{G}}$ as a *Computation-Activation Network* (or shorter as a *CompActNet*). We call $\beta^t\left[Y_{[p]}, X_{[d]}\right]$ (and any decomposition of it) the *computation network* and $\xi\left[Y_{\mathcal{V}}\right]$ the *activation network*.

The elementary activated networks are representable by an elementary activation tensor with respect to the graph

$$\mathrm{EL} = \left(\mathcal{V}, \{\{v\} \,:\, v \in \mathcal{V}\}\right)$$

and we denote such networks by $\Lambda^{t,\mathrm{EL}}$. Any CompActNet is representable with respect to the maximal hypergraph

$$\mathrm{MAX} = \left(\mathcal{V}, \{\mathcal{V}\}\right).$$

We therefore have for any graph that $\Lambda^{t,\mathcal{G}} \subset \Lambda^{t,\mathrm{MAX}}$.

# Part I
# Foundations of Artificial Intelligence

Many concepts of Artificial Intelligence can be captured by the formalism of tensors, tensor networks and their contractions. We dedicate this part to anchoring the formalism in the probabilistic and logical foundations of artificial intelligence.

The two major claims we want to explain are:

- **Tensor network decompositions** represent sparsity principles in AI
- **Tensor network contractions** represent inference tasks

# Chapter 2
# Propositional Logic

**Abstract** We present the representation and the reasoning mechansims of propositional logics.
Call $X$ semantic variable instead of categorical/Boolean.

## 2.1 Propositonal Formulas

We choose a semantic-central definition and define propositional formulas as maps from states to The epistemological commitments of propositional logics are whether the state is True or False reflected by coordinates 1 or 0. Intuitively these maps mark, whether a specific world can be the state of a factored system.

#### 2.1.0.1 Formulas as maps

We now choose a semantic centric approach to propositional logic, by defining formulas as boolean tensors. Then we investigate the corresponding syntax of formulas as specification of a tensor network decomposition of the basis encoding of formulas.

Logics is especially useful in interpreting boolean tensors representing Propositional Knowledge Bases, based on connections with abstract human thinking. To make this more precise, we associate each such tensor with a formula $f$ being a composition of the atomic variables with logical connectives as we proof next.

**Definition 2.1** A propositional formula $f\left[X_{[d]}\right]$ depending on $d$ atoms $X_k$ is a boolean-valued tensor

$$f\left[X_{[d]}\right] : \bigtimes_{k\in[d]} [2] \to \{0,1\} \subset \mathbb{R}\,.$$

We call a state $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ a model of a propositional formula $f$, if

$$f\left[X_{[d]} = x_{[d]}\right] = 1\,.$$

If there is a model to a propositional formula, we say the formula is satisfiable.

The propositional formulas coincide therefore with the boolean tensors (see Def. **??**).

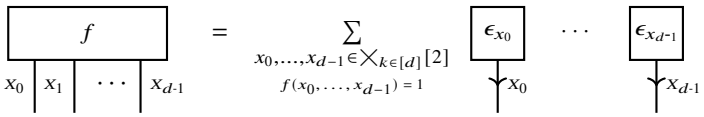
Since propositional formulas are binary valued tensors, the generic decomposition of Lem. **??** simplifies to

$$f\left[X_{[d]}\right] = \sum_{x_0,\ldots,x_{d-1}\in\bigtimes_{k\in[d]}[2]} f\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right] \tag{2.1}$$

$$= \sum_{x_0,\ldots,x_{d-1}\in\bigtimes_{k\in[d]}[2] \,:\, f[X_0=x_0,\ldots,X_{d-1}=x_{d-1}]=1} \epsilon_{x_0,\ldots,x_{d-1}}\left[X_{[d]}\right]\,. \tag{2.2}$$

Thus, any propositional formula is the sum over the one-hot encodings of its models. This is equal to the encoding of the set of models, which will be introduced in Chapter 7 (see Def. 7.1).

We depict this decomposition in the diagrammatic notation by




We here chose a semantic approach to propositional logic in contrary to the standard syntactical approach. Instead of defining formulas by connectives acting on atomic formulas, we define them here as binary valued functions of the states of a factored system. They are interpreted by marking possible states as models, given the knowledge of $f$. The syntactical side will then be introduced later by studying decompositions of formulas.

### 2.1.1 Knowledge bases

Review the notation:

- $\mathcal{KB}$ as a set of formulas, i.e. a tensor network
- $\mathcal{KB}\left[X_{[d]}\right]$ as the contraction of the tensor network

Knowledge bases of propositional formulas are our first examples of tensor network decompositions.

**Definition 2.2** A propositional knowledge base is a set $\mathcal{KB} = \{f_\ell\left[X_{e_\ell}\right] : \ell \in [p]\}$ of propositional formulas dependent on variables $X_{e_\ell}$. The corresponding formula $\mathcal{KB}\left[X_\mathcal{V}\right]$ depending on $X_\mathcal{V} = \bigcup_{\ell\in[p]} X_{e_\ell}$ has coordinates to $x_\mathcal{V} \in \bigtimes_{v\in\mathcal{V}}[m_v]$

$$\mathcal{KB}\left[X_\mathcal{V} = x_\mathcal{V}\right] = \begin{cases} 1 & \text{if} \quad \forall \ell \in [p] : f_\ell\left[X_{e_\ell} = x_{e_\ell}\right] = 1 \\ 0 & \text{else}\,. \end{cases}$$

We will later investigate syntactical decompositions and denote

$$\mathcal{KB}\left[X_\mathcal{V}\right] = \bigvee_{\ell\in[p]} f_\ell\left[X_{e_\ell}\right]\,.$$

**Theorem 2.1 (Tensor network representation of knowledge base)** *Let there be a propositional knowledge base* $\mathcal{KB} = \{f_\ell : \ell \in [p]\}$. *Then the*

$$\mathcal{KB}\left[X_\mathcal{V}\right] = \left\langle f\left[X_{e_\ell}\right] : \ell \in [p]\right\rangle_{[X_\mathcal{V}]}\,.$$

*Proof.* We have for any $x_\mathcal{V}$ that

$$\mathcal{KB}\left[X_\mathcal{V} = x_\mathcal{V}\right] = \left\langle f\left[X_{e_\ell}\right] : \ell \in [p]\right\rangle_{[X_\mathcal{V}=x_\mathcal{V}]} = \prod_{\ell\in[p]} f\left[X_{e_\ell} = x_{e_\ell}\right]$$

where by $x_{e_\ell}$ we denote the reduction of the tuple $x_\mathcal{V}$ to the indices $e_\ell$. It follows that $\mathcal{KB}\left[X_\mathcal{V} = x_\mathcal{V}\right] = 1$ if and only if $\forall \ell \in [p] : f_\ell\left[X_{e_\ell} = x_{e_\ell}\right] = 1$. □

## 2.2 Entailment

We here discuss entailment in propositinal logics based on contractions of formulas.

Entailment is the central consequence relation among logical formulas. Let us define this relation first based on the models of a knowledge base and a test formula.

**Definition 2.3 (Entailment of propositional formulas)** Given two propositional formulas $\mathcal{KB}$ and $f$ we say that $\mathcal{KB}$ entails $f$, denoted by $\mathcal{KB} \models f$, if any model of $\mathcal{KB}$ is also a model of $f$, that is

$$\forall x_{[d]} \in \bigtimes_{k\in[d]} [2] : \left(\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1\right) \Rightarrow \left(f\left[X_{[d]} = x_{[d]}\right] = 1\right).$$

If $\mathcal{KB} \models \neg f$ holds, we say that $\mathcal{KB}$ contradicts $f$.

To use the tensor network formalism for the decision of entailment, we will in the following develop three equivalent criteria for entailment.

### 2.2.1 Deciding Entailment by Contractions

First of all, we can decide entailment based on vanishing contractions with the negated test formula.

**Theorem 2.2 (Contraction Criterion of Entailment)** *We have $\mathcal{KB} \models f$ if and only if*

$$\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} = 0 .$$

*Proof.* "⇐": If for a $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have $\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1$ but not $\left(f\left[X_{[d]} = x_{[d]}\right] = 1\right)$, we would have $\left(\neg f\left[X_{[d]} = x_{[d]}\right] = 1\right)$ and

$$\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[2]} \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] \cdot f\left[X_{[d]} = x_{[d]}\right] > 1 .$$

Thus, whenever the contraction vanishes, we have

$$\forall x_{[d]} \in \bigtimes_{k\in[d]} [2] : \left(\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1\right) \Rightarrow \left(f\left[X_{[d]} = x_{[d]}\right] = 1\right).$$

"⇒": Conversely, if the contraction $\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]}$ does not vanish, we would find $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ with $\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1$ and $\neg f\left[X_{[d]} = x_{[d]}\right] = 1$, therefore $f\left[X_{[d]} = x_{[d]}\right] = 0$. It follows that $\mathcal{KB} \models f$ does not hold. □

The contraction criterion can be extended to the decision of contradiction as well, since $\mathcal{KB} \models \neg f$ is equivalent to $\langle \mathcal{KB}, f \rangle_{[\varnothing]} = 0$. Therefore, entailment and contradiction can be decided simultaneously by a single contraction, as we state next.

**Theorem 2.3** *Given propositional formulas $\mathcal{KB}$ and $f$ we build*

$$\tau\left[Y_f\right] = \left\langle \mathcal{KB}\left[X_{[d]}\right], \beta^f\left[Y_f, X_{[d]}\right] \right\rangle_{[Y_f]} .$$

*Then $\mathcal{KB} \models f$ is equivalent to $\tau\left[Y_f = 0\right] = 0$, and $\mathcal{KB} \models \neg f$ is equivalent to $\tau\left[Y_f = 1\right] = 0$ .*

*Proof.* This follows from Thm. 2.2 using that

$$\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} = \tau\left[Y_f = 0\right]$$

and

$$\langle \mathcal{KB}, f \rangle_{[\varnothing]} = \tau\left[Y_f = 1\right] .\square$$

### 2.2.2 Deciding Entailment by Partial Ordering

Logical entailment can be understood by subset relations of the models of the respective formulas. This perspective can be applied with subset encodings in Chapter 7. The subset relation corresponds with partial ordering of its encoded

tensors, as will be shown in Thm. 7.1. For two propositional formulas, we denote to this end $f \prec h$ (see Def. 7.2), if and only if for all $x_{[d]} \in \bigtimes_{k\in[d]}[2]$

$$f\left[X_{[d]} = x_{[d]}\right] \leq h\left[x_{[d]}\right] .$$

**Theorem 2.4 (Partial Ordering Criterion of Entailment)** *We have $\mathcal{KB} \models f$ if and only if $\mathcal{KB}\left[X_{[d]}\right] \prec f\left[X_{[d]}\right]$.*

*Proof.* Since both $\mathcal{KB}$ and $f$ are boolean tensors, we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ that

$$\mathcal{KB}\left[X_{[d]} = x_{[d]}\right], f\left[X_{[d]} = x_{[d]}\right] \in \{0,1\} .$$

Thus,

$$\forall x_{[d]} \in \bigtimes_{k\in[d]}[2] : \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] \leq f\left[X_{[d]} = x_{[d]}\right]$$

is equivalent to

$$\forall x_{[d]} \in \bigtimes_{k\in[d]}[2] : \left(\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1\right) \Rightarrow \left(f\left[X_{[d]} = x_{[d]}\right] = 1\right) .$$

This states that $\mathcal{KB}\left[X_{[d]}\right] \prec f\left[X_{[d]}\right]$ is equivalent to $\mathcal{KB} \models f$. □

### 2.2.3 Redundancy of Entailed Formulas

Another interpretation of entailment is by redundancy of a formula in a Knowledge Base. This is especially interesting for the sparse representation of Knowledge Bases.

**Theorem 2.5 (Redundancy Criterion of Entailment)** *If and only if $\mathcal{KB} \models f$ we have*

$$\mathcal{KB}\left[X_{[d]}\right] = \langle \mathcal{KB}, f \rangle_{\left[X_{[d]}\right]} .$$

*Proof.* For any formula $f$ we have

$$\mathbb{I}\left[X_{[d]}\right] = f\left[X_{[d]}\right] + \neg f\left[X_{[d]}\right]$$

and thus

$$\begin{aligned}\mathcal{KB}\left[X_{[d]}\right] &= \left\langle \mathcal{KB}\left[X_{[d]}\right], \mathbb{I}\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} \\ &= \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} + \left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} .\end{aligned}$$

Now, by Thm. 2.2 we have $\mathcal{KB} \models f$, if and only if $\left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = 0$, which is thus equal to

$$\mathcal{KB}\left[X_{[d]}\right] = \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} .\square$$

### 2.2.4 Contraction Knowledge Base

We exploit the contraction and redundancy criteria of entailment to sketch an implementation of a propositional Knowledge Base in Algorithm 1. Here the function ASK($f$) returns, whether a formula $f$ is entailed or contradicted by a Knowledge Base. If the formula is neither entailed or contradicted, we say it is contingent. If it is both, we have $\mathcal{KB}\left[X_{[d]}\right] = 0$ and thus an inconsistent Knowledge Base. Exploiting Thm. 2.3 we decide these situations based on a single contraction.

The function TELL($f$) incorporates an additional formula $f$ into a Knowledge Base $\mathcal{KB}$. Here we exploit Thm. 2.5 and do not add a formula, which is entailed in order to maintain a sparse representation. The function further refuses to add a formula, which would make the Knowledge Base inconsistent (returns Refused) and only changes the Knowledge Base in case of a contingent formula (returns Added).

| $\mathcal{KB} \models \neg f$ \ $\mathcal{KB} \models f$ | False | True |
|---|---|---|
| False | *"Contingent"* | *"Entailed"* |
| True | *"Contracticted"* | *"Inconsistent"* |

**Fig. 2.1** Table of possible logical relations between a knowledge base $\mathcal{KB}$ and $f$, based on whether the knowledge base entails the formula ($\mathcal{KB} \models f$) and its negation ($\mathcal{KB} \models \neg f$).

**Algorithm 1** Contraction Knowledge Base with operations ASK and TELL

**ASK**($\mathcal{KB}$, $f$)

**Require:** Knowledge base $\mathcal{KB}$, query formula $f$
**Ensure:** Decision which relation between $\mathcal{KB}$ and $f$ holds (see Figure 2.1)

$\tau\left[Y_f\right] \leftarrow \left\langle \{h\left[X_{[d]}\right] : h \in \mathcal{KB}\}, \beta^f\left[Y_f, X_{[d]}\right]\right\rangle_{[Y_f]}$
**if** $\tau\left[Y_f = 0\right] = 0$ and $\tau\left[Y_f = 1\right] = 0$ **then**
  **return** *"Inconsistent"*
**else if** $\tau\left[Y_f = 0\right] = 0$ **then**
  **return** *"Entailed"*
**else if** $\tau\left[Y_f = 1\right] = 0$ **then**
  **return** *"Contradicted"*
**else**
  **return** *"Contingent"*
**end if**

**TELL**($\mathcal{KB}$, $f$)

**Require:** Knowledge base $\mathcal{KB}$, query formula $f$
**Ensure:** Decision whether a formula is added to the knowledge base (*"Added"*), or an exception in (*"Inconsistent"*, *"Redundant"* or *"Refused"*) is raised.

answer $\leftarrow$ ASK($f$)
**if** answer is *"Inconsistent"*: **then**
  **return** *"Inconsistent"*
**else if** answer is *"Entailed"*: **then**
  **return** *"Redundant"*
**else if** answer is *"Contradicted"*: **then**
  **return** *"Refused"*
**else if** answer is *"Contingent"*: **then**
  $\mathcal{KB} \leftarrow \mathcal{KB} \cup \{f\}$
  **return** *"Added"*
**end if**

## 2.3 Syntactical decompositions

When syntax is available for formulas, we find tensor network decompositions. This is a first example of basis calculus.

### 2.3.1 Encoding of Booleans

Before investigating schemes for syntactical decompositions, we revisit the encoding of Booleans. Boolean variables are a basic data structure, valued in {False, True}.

#### 2.3.1.1 Representation by Coordinates

To represent booleans by categorical variables $X$ with two states we use the index interpretation function (see Def. **??** and for more details Def. 7.1 in Part **??**)

$$I \,:\, [2] \to \{\text{False}, \text{True}\}$$

defined as

$$I\,[1] = \text{True} \quad \text{and} \quad I\,[0] = \text{False}\,.$$

One motivation for this particular choice of the interpretation function $I$ is the effective execution of the conjunction as we show in the next Lemma.

**Lemma 2.1** *$I$ is a homomorphism between the groups*

$$\big(\{0,1\},\cdot\big) \quad \textit{and} \quad \big(\{\text{False}, \text{True}\},\wedge\big)\,.$$

*Proof.* It suffices to notice, that for arbitrary $z_0, z_1 \in \{0,1\}$ we have

$$I\,[z_0 \cdot z_1] = I\,[z_0] \wedge I\,[z_1]\ .\square$$

Based on this homomorphism, contractions of boolean tensors, in which all variables are kept open, can be regarded as parallel calculations of the conjunction $\wedge$ encoded by $I$. This homomorphism is further applied in type conversion in dynamically-typed languages (e.g. in python [26]).

Operations like the negation fail to be linear and are only affine linear, since for $z \in \{\text{False}, \text{True}\}$ we have

$$I^{-1}(\neg z) = 1 - I^{-1}(z)\,. \tag{2.3}$$

Since any logical connective can be represented as a composition of conjunctions and negations, any logical connective corresponds with an affine linear function on the interpreted truth values. Direct applications of this insight to execute logical calculus will be discussed later in Sect. 7.4. For our purposes here, we would like to execute logical connective based on single contractions and avoid summations over them. This is why we call the negation representation as in (2.3) the affine representation problem, which we aim to resolve in the following.

While in this work, we will always encode boolean states by $I$, other index interpretation functions could be chosen. For example, the interpretation

$$I_\vee \,:\, \{0,1\} \to \{\text{False}, \text{True}\}$$

defined as

$$I_\vee\,[0] = \text{True} \quad \text{and} \quad I_\vee\,[1] = \text{False}$$

results is a homomorphism between the groups

$$\big(\{0,1\},\cdot\big) \quad \text{and} \quad \big(\{\text{False}, \text{True}\},\vee\big)\,.$$

While placing the disjunction $\vee$ as the logical connective effectively executed by contractions, the negation will for arbitrary interpretations mapping onto $\{0,1\}$ remain the function

$$I_\vee^{-1}(\neg z) = 1 - I_\vee^{-1}(z)\,.$$

Thus, the problem of affine linear operations cannot be resolved by a clever choice of an interpretation function with image in $\{0, 1\}$.

#### 2.3.1.2 Representation by Basis Vectors

While contractions can just perform conjunctions, we need a representation trick to extend the contraction expressivity to arbitrary connectives and resolve the affine representation problem. To this end we now compose $I$ with the one-hot encoding $\epsilon$ and get an encoding

$$\epsilon \circ I^{-1} : \{\text{False}, \text{True}\} \to \{\epsilon_0 [X], \epsilon_1 [X]\},$$

where $X$ is a categorical variable with $m = 2$. For any $z \in \{\text{False}, \text{True}\}$ we have

$$\epsilon \circ I^{-1}(z) = \begin{bmatrix} I^{-1}(\neg z) \\ I^{-1}(z) \end{bmatrix}.$$

Performing the negation now amounts to switching the coordinates of the encoded vector, which can be performed by contraction with a transposition matrix

$$\beta^{\neg} [Y_{\neg}, X] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} [Y_{\neg}, X],$$

where in this notation we always understand the first variable $X$ as the row index selector and the second variable $Y_{\neg}$ as the column index selector. We then have

$$\epsilon \circ I^{-1}(\neg z)[Y_{\neg}] = \left\langle \beta^{\neg} [Y_{\neg}, X], \epsilon \circ I^{-1}(z)[X] \right\rangle_{[Y_{\neg}]}.$$

We therefore arrived at our aim to resolve the affine representation problem and have found a procedure to represent logical negations by a contraction, which is a linear operation. Besides negations, we will show in this chapter, that arbitrary logical formulas can be represented by contractions.

#### 2.3.1.3 Coordinate and Basis Calculus

Our findings on the encoding of booleans hint towards more general schemes to encode information into boolean tensors, which will be explored in more detail in Chapter **??** and Chapter 7. When each coordinate in a boolean tensor represents one in $\{0, 1\}$ interpreted boolean we call the scheme coordinate calculus. In basis calculus on the other hand, booleans are represented by elements of $\{\epsilon_0 [X], \epsilon_1 [X]\}$. In that scheme, there are pairs of two coordinates (building slice vectors of the tensors), which are restricted to be different from each other. This amounts to posing a global directionality constraint on the boolean tensor, as will be shown in Thm. 7.3.

### 2.3.2 Basis encodings

Add entailment by basis calculus diagram.

#### 2.3.2.1 Definition

**Definition 2.4**

#### 2.3.2.2 Formula representation by Computation-Activation Networks

We apply the coordinate and the basis encoding schemes to represent propositional formulas as tensors. In the coordinate encoding scheme, which we use by default, we understand $[2]$ as subset of $\mathbb{R}$ and interpreting the formula directly as a tensor (as in Def. 2.1). To apply the basis encoding scheme, we understand $[2]$ as the states of a system with factored representation by a single categorical variable. Although it comes with an additional boolean variable compared with coordinate encoding scheme, we in this work use the basis encoding scheme to find sparse tensor network decompositions based on syntactical formula decompositions. Furthermore, the basis encoding connects with the formalism of Computation-Activation Networks developed in Chapter **??**, as we show next.

**Theorem 2.6 (Formula representation as Computation-Activation Network)** *For each formula $f$ we have*

$$f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_1\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]}$$

*and*

$$\neg f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]} ,$$

*which are Computation-Activation Networks to the statistic $\{f\}$ and the hard activation tensor $\epsilon_1\left[Y_f\right]$, respectively $\epsilon_0\left[Y_f\right]$.*

*Proof.* Given a factored system with $d$ atoms $X_{[d]}$ and a propositional formula $f$, the basis encoding of $f$ (see Def. 1.6 and Figure 2.2) is the tensor

$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[2]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{f\left[X_{[d]}=x_{[d]}\right]}\left[Y_f\right] .$$

We reorder the summation into

$$\begin{aligned}\beta^f\left[Y_f, X_{[d]}\right] =& \epsilon_0\left[Y_f\right] \otimes \left(\sum_{x_{[d]} : f\left[x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right) \\ &+ \epsilon_1\left[Y_f\right] \otimes \left(\sum_{x_{[d]} : f\left[x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right)\end{aligned}$$

where the second term sums up the models of $f$ and the first one the models of $\neg f$. Using linearity of contractions we get

$$\begin{aligned}\left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_1\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]} &= \left\langle \epsilon_0\left[Y_f\right], \epsilon_1\left[Y_f\right]\right\rangle_{[\varnothing]} \cdot \left(\sum_{x_{[d]} : f\left[x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right) \\ &\quad + \left\langle \epsilon_1\left[Y_f\right], \epsilon_1\left[Y_f\right]\right\rangle_{[\varnothing]} \cdot \left(\sum_{x_{[d]} : f\left[x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right) \\ = \sum_{x_{[d]} : f\left[x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right] &= f\left[X_{[d]}\right] .\end{aligned}$$

Analogously we have

$$\left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]} = \sum_{x_{[d]} : f\left[x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right] = \neg f\left[X_{[d]}\right] . \square$$

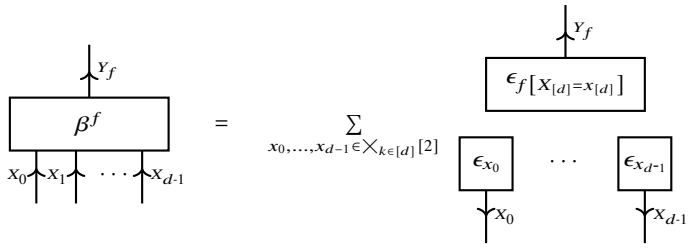



**Fig. 2.2** Basis encoding of a propositional formula. The encoding is a sum of the one hot encodings of all states of the factored system in a tensor product with basis vectors, which encode whether the state is a model of the formula. The tensor is directed, since any contraction with an encoded state results in the basis vector evaluating the formula, which we called basis calculus.

Thm. 2.6 shows that besides the formula itself, we can represent the negation of the formula as a Computation-Activation Network with the same statistic $\{f\}$. In general, any boolean tensor which has a representation as a Computation-Activation Network needs to have a boolean activation tensor. In the case of the single-dimensional statistic $\{f\}$ we have three boolean activation vectors $\{\mathbb{I}\left[Y_f\right], \epsilon_1\left[Y_f\right], \epsilon_0\left[Y_f\right], 0\left[Y_f\right]\}$. While the Computation-Activation Networks with the activation cores $\epsilon_1\left[Y_f\right], \epsilon_0\left[Y_f\right]$ are by Thm. 2.6 the formula and its negation, the activation core $\mathbb{I}\left[Y_f\right]$ and $0\left[Y_f\right]$ represent the trivial $\mathbb{I}\left[X_{[d]}\right]$ and the vanishing tensor $0\left[X_{[d]}\right]$.

The contraction of the basis encoding with one-hot encodings to worlds $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ is

$$\left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right\rangle_{[Y_f]} = \begin{cases} \epsilon_1\left[Y_f\right] & \text{if the world } x_{[d]} \text{ is a model of } f \\ \epsilon_0 Y_f & \text{else .} \end{cases}$$

We thus interpret the contractions as calculating in basis vectors, whether the world $x_{[d]}$ is a model of the formula $f$. This is the basis calculus scheme and will be investigated it in more detail in Chapter 7 (see Thm. 7.4).

### 2.3.3 Circuit decomposition

Syntax provides a description of a formula as a circuit of connective formulas.

Basis encodings of propositional formulas are especially useful when representing function compositions by the representation of their components (see Thm. **??**). In propositional logics, the syntax of defining propositional formulas is oriented on compositions of formulas by connectives. In this section, we investigate the decomposition schemes of basis encodings into tensor networks of component encodings for binary tensors following propositional logic syntax.

### 2.3.4 Atomic Formulas

We call atomic formulas the most granular formulas, which are not split into compositions of other formulas. Our syntactic decomposition of propositional formulas will then investigate, how any propositional formula can be represented by these.

**Definition 2.5** The tensors $f_k\left[X_{[d]}\right]$ defined for $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ as

$$f_k\left[X_{[d]} = x_{[d]}\right] = x_k$$

are called atomic formulas.

Atomic formulas and their basis encodings have an especially compelling representation.

**Lemma 2.2** *Any atomic formula* $f^k\left[X_{[d]}\right]$ *is represented as*

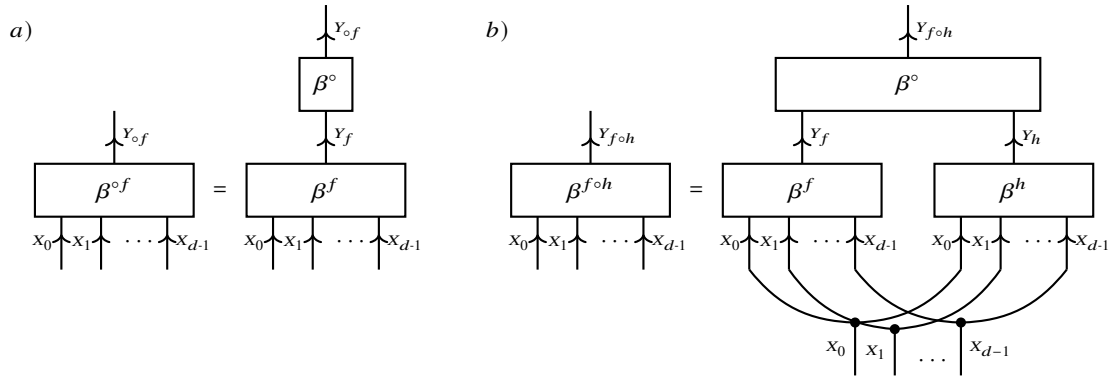


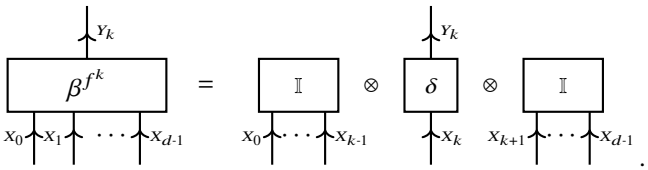

**Fig. 2.3** Representation of syntactical compositions of formulas by basis encodings. a) Unary connective $\circ$ acting on a formula $f$. b) Binary connective $\circ$ composing formulas $f, h$. In more generality, the basis encoding of arbitrary syntactical compositions is a contraction of the basis encoding of the components.

$$f^k\left[X_{[d]} = x_{[d]}\right] = \langle \epsilon_1\left[X_k\right]\rangle_{\left[X_{[d]}\right]} = \epsilon_1\left[X_k\right] \otimes \mathbb{I}\left[X_{[d]/\{k\}}\right] .$$

*and has a basis encoding*

$$\beta^{f^k}\left[Y_d, X_{[d]}\right] = \langle \delta\left[Y_k, X_k\right]\rangle_{\left[X_{[d]}\right]} = \delta\left[Y_k, X_k\right] \otimes \mathbb{I}\left[X_{[d]/\{k\}}\right] .$$

*The decomposition is depicted in a network diagram as*



*Proof.* We have by definition

$$\begin{aligned}\beta^{f^k}\left[Y_k, X_{[d]}\right] &= \sum_{x_0,\ldots,x_{d-1}\in\bigtimes_{k\in[d]}[2]} \epsilon_{x_0,\ldots,x_{d-1}}\left[X_{[d]}\right] \otimes \epsilon_{f_k[X_0=x_0,\ldots,X_{d-1}=x_{d-1}]}\left[Y_k\right] \\ &= \left(\epsilon_{0,0}\left[X_k, Y_k\right] + \epsilon_{1,1}\left[X_k, Y_k\right]\right) \otimes \mathbb{I}\left[X_l \,:\, l \neq k\right] \\ &= \langle \delta\left[X_k, Y_k\right]\rangle_{\left[X_{[d]}, Y_k\right]} .\square\end{aligned}$$

### 2.3.5 Syntactical Composition of Formulas

Propositional formulas are elements of tensor spaces with $d$ axis. The number of coordinates thus grows exponentially with the number of atoms, which is

$$\dim\left(\bigotimes_{k\in[d]} \mathbb{R}^2\right) = 2^d .$$

When the number of atoms is large, the naive representation of formula tensors will be thus intractable. In contrast, typical logical formulas appearing in practical knowledge bases are sparse in the sense that they have short representations in a logical syntax. In logical syntax formulas are described by atomic formulas recursively combined via connectives, which are itself formulas of typically low order $d$. We now exploit such syntactical decompositions of formulas to derive tensor network decompositions.

Let there be formulas $f$ and $h$ depending on categorical variables $X_{[d]}$ and a binary connective

$$\circ : [2] \times [2] \to [2] .$$

Then we have (see Figure 2.3b)

$$\beta^{f\circ h}\left[X_{[d]}, X_{f\circ h}\right] = \left\langle \beta^{\circ}\left[Y_f, Y_h, X_{f\circ h}\right], \beta^{f}\left[X_{[d]}, Y_f\right], \beta^{h}\left[X_{[d]}, Y_h\right]\right\rangle_{\left[X_{[d]}, X_{f\circ h}\right]} .$$

For any unary connective $\circ : [2] \to [2]$ we analogously have (see Figure 2.3a)

$$\beta^{\circ f}\left[Y_{\circ f}, X_{[d]}\right] = \left\langle \beta^{\circ}\left[Y_{\circ f}, Y_f\right], \beta^{f}\left[Y_f, X_{[d]}\right]\right\rangle_{\left[Y_{\circ f}, X_{[d]}\right]} .$$

**Lemma 2.3** *Let there be a $r$-ary connective*

$$\circ : \bigtimes_{l\in[r]} [2] \to [2]$$

*and for each $l \in [r]$ a formula $f_l\left[X_{[d]}\right]$. For the composition formula $f\left[X_{[d]}\right]$ defined by*

$$f\left[X_{[d]} = x_{[d]}\right] = \circ\left(f_0\left[X_{[d]} = x_{[d]}\right], \ldots, f_{r-1}\left[X_{[d]} = x_{[d]}\right]\right)$$

*we have*

$$\beta^{f\left[X_{[d]}\right]}\left[Y_f, X_{[d]}\right] = \left\langle \{\beta^{\circ}\left[Y_{f\left[X_{[d]}\right]}, Y_f, Y_{[r]}\right]\} \cup \{\beta^{f_l}\left[Y_l, X_{[d]}\right] : l \in [r]\}\right\rangle_{\left[Y_{f\left[X_{[d]}\right]}, X_{[d]}\right]} .$$

*Proof.* Here we used the main property of basis encodings, that basis encodings of composition are equal to contracted basis encodings of the components. We will show this property in more generality in Thm. **??**. □

These properties motivate a hypergraph-based definition of syntactical compositions of a propositional formula.

**Definition 2.6** A syntactical circuit is a directed acyclic hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Def. **??**) such that

- each hyperedge $e = (e^{\text{in}}, e^{\text{out}})$ has exactly one outgoing node, i.e. $\left|e^{\text{out}}\right| = 1$
- each node $v \in \mathcal{V}$ carries a boolean variable $Y_v$ and appears at most once as the outgoing node of a hyperedge
- each hyperedge $(e^{\text{in}}, \{v\})$ with $e^{\text{in}} \neq \emptyset$ is decorated by a logical connective

$$\circ_v\left[Y_{e^{\text{in}}}\right] : \bigtimes_{v\in e^{\text{in}}} [2] \to [2]$$

- the node not appearing as an outgoing node are labeled by $[d]$

We say that the syntactical circuit is single-rooted, if exactly one node $\tilde{v}$ does not appear as an incoming node of a hyperedge. In this case this unique node is called the root node. We assign atomic formulas to the nodes $[d]$ and recursively assign to each further node $v$ a node formula

$$f_v\left[X_{[d]} = x_{[d]}\right] = \circ_v\left[\left[f_{\bar{v}}\left[X_{[d]} = x_{[d]}\right] : \bar{v} \in e^{\text{in}}\right]\right] \quad \forall x_{[d]} \in \bigtimes_{k\in[d]} [m_k] ,$$

where $e^{\text{in}}$ are the incoming nodes in the unique hyperedge with outgoing nodes $\{v\}$. We call the formula $f\left[X_{[d]}\right] := f_{\tilde{v}}\left[X_{[d]}\right]$ to the root note $\tilde{v}$ the syntactical composition of $\mathcal{G}$ and $\mathcal{G}$ is a syntactical decomposition of $f$.

We are now prepared to show the main result of this chapter, the representation of formulas as contracted basis encoding of their decomposition.

**Theorem 2.7** *For any syntactical circuit $\mathcal{G}$ with composition $f$ we have*

$$f\left[X_{[d]}\right] = \left\langle \left\{\beta^{\circ_v}\left[Y_v, Y_{e^{\text{in}}}\right] : (e^{\text{in}}, \{v\}) \in \mathcal{E}\right\} \cup \{\delta\left[Y_k, X_k\right] : k \in [d]\} \cup \{\epsilon_1\left[Y_{\tilde{v}}\right]\}\right\rangle_{\left[X_{[d]}\right]} .$$

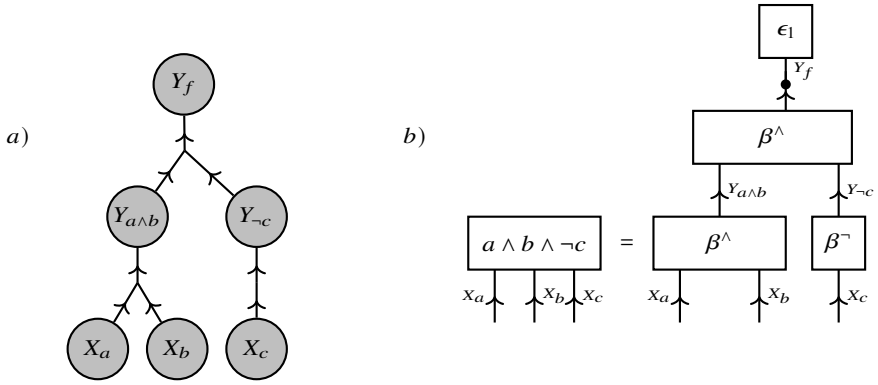



**Fig. 2.4** Decomposition of the formula tensor to $f = a \wedge b \wedge \neg c$ into unary (matrix) and binary (third order tensor) cores. a) Syntactical decomposition of $f$ as a hypergraph $\mathcal{G}$ according to Def. 2.6. b) Tensor network of basis encodings of the connectives decorating the hyperedges, which represents by Thm. 2.7 the formula $f$.

*Proof.* We show inductively over the nodes, that

$$\beta^{f_v}\left[Y_v, X_{[d]}\right] = \Big\langle \left\{\beta^{\circ_{\bar{v}}}\left[Y_{\bar{v}}, Y_{e^{\text{in}}}\right] \,:\, \bar{v} \prec v,\, (e^{\text{in}}, \{\bar{v}\}) \in \mathcal{E}\right\} \cup \{\beta^{\circ_v}\left[Y_v, Y_{e^{\text{in}}}\right]\} \cup \tag{2.4}$$

$$\{\delta\left[Y_k, X_k\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}\right]}. \tag{2.5}$$

By $\prec$ we denote the partial ordering of nodes by directed paths in $\mathcal{G}$, i.e. $\bar{v} \prec v$ holds if there is a directed path from $\bar{v}$ to $v$ in the hypergraph $\mathcal{G}$. We start the induction at the nodes $[d]$ not appearing as outgoing nodes, where (2.4) holds by Lem. 2.2.

If for a node $v \in \mathcal{V}$ (2.4) holds at all $\bar{v} \in e^{\text{in}}$, we apply Lem. 2.3 to the connective $\circ_v$ and the formulas $f_{\bar{v}}\left[X_{[d]}\right]$ of the parents $\bar{v} \in e^{\text{in}}$ and get

$$\begin{aligned}
\beta^{f_v}\left[Y_v, X_{[d]}\right] &= \left\langle \{\beta^{\circ_v}\left[Y_{f_v}, Y_f, Y_{[r]}\right]\} \cup \{\beta^{f_l}\left[Y_l, X_{[d]}\right] \,:\, l \in [r]\}\right\rangle_{\left[Y_f\left[X_{[d]}\right], X_{[d]}\right]} \\
&= \Big\langle \left\{\beta^{\circ_{\bar{v}}}\left[Y_{\bar{v}}, Y_{e^{\text{in}}}\right] \,:\, \bar{v} \prec v,\, (e^{\text{in}}, \{\bar{v}\}) \in \mathcal{E}\right\} \cup \{\beta^{\circ_v}\left[Y_v, Y_{e^{\text{in}}}\right]\} \cup \\
&\qquad \{\delta\left[Y_k, X_k\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}\right]}.
\end{aligned}$$

Thus (2.4) also holds for $v$. Since the hypergraph is acyclic the induction reached all nodes and (2.4) holds in particular for the root node $\tilde{v}$. The claim then follows with Thm. 2.6 applied on $f = f_{\tilde{v}}$. □

An example of a syntactical decomposition is provided in Figure 2.4a, and the corresponding tensor network representation in Figure 2.4b. Examples of unary ($d = 1$) and binary ($d = 2$) connectives are further provided in Example 2.1.

*Example 2.1* We use the following connectives:

- negation $\neg : [2] \to [2]$ by the vector

$$\neg[Y_f] = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- conjunctions $\wedge : [2] \times [2] \to [2]$

$$\wedge[Y_f, Y_h] = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

- disjunctions $\vee : [2] \times [2] \to [2]$

$$\vee[Y_f, Y_h] = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

- exact disjunction $\oplus : [2] \times [2] \to [2]$

$$\oplus[Y_f, Y_h] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- implications $\Rightarrow$: $[2] \times [2] \to [2]$

$$\Rightarrow [Y_f, Y_h] = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

- biimplication $\Leftrightarrow$: $[2] \times [2] \to [2]$

$$\Leftrightarrow [Y_f, Y_h] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

*Remark 2.1 (d-ary connectives such as $\wedge$ and $\vee$)* Since the decomposition of basis encoding can be applied to generic function compositions (see Thm. **??**), we can also allow for $d$-ary connectives

$$\circ : \bigtimes_{k \in [d]} [2] \to [2] \,.$$

The connectives $\wedge$ and $\vee$ satisfy associativity and have thus straightforward generalizations to the $d$-ary case. This is because associativity can be exploited to represent the basis encoding by any tree-structured composition of binary $\wedge$ and $\vee$ connectives.

### 2.3.6 Minterm and maxterm decompositions

We have seen how syntactical decompositions of formulas into connectives acting can be exploited to find effective representations by tensor networks. We now provide a generic scheme to find syntactical decompositions to arbitrary propositional formulas.

**Definition 2.7 (Terms and Clauses)** Given a subset $\mathcal{U} \subset [d]$ and $y_{\mathcal{U}} \in \bigtimes_{k \in \mathcal{U}} [2]$, the corresponding term is the formula

$$Z^{\wedge}_{\mathcal{U}} \left[ X_{[d]} \right] = \left( \bigwedge_{k \in \mathcal{U} \,:\, y_k = 0} \neg f^k \left[ X_k \right] \right) \wedge \left( \bigwedge_{k \in \mathcal{U} \,:\, y_k = 1} f^k \left[ X_k \right] \right)$$

and the corresponding clause is the formula

$$Z^{\vee}_{\mathcal{U}} \left[ X_{[d]} \right] = \left( \bigvee_{k \in \mathcal{U} \,:\, y_k = 0} f^k \left[ X_k \right] \right) \vee \left( \bigvee_{k \in \mathcal{U} \,:\, y_k = 1} \neg f^k \left[ X_k \right] \right)$$

$$Z^{\vee}_{\mathcal{V}^0} \left[ \mathcal{V}^1 \right] X_{[d]} = \left( \bigvee_{k \in \mathcal{V}^0} f^k \right) \vee \left( \bigvee_{k \in \mathcal{V}^1} \neg f^k \right) ,$$

We call the term a minterm and the clause a maxterm, if $\mathcal{U} = [d]$.

Use the $(\mathcal{U}, y_{\mathcal{U}})$ parametrization instead of the nodes 0 1 parametrization in the following.
Terms and Clauses have for any index tuple $x_{[d]}$ a polynomial representation by

$$Z^{\wedge}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]} = x_{[d]}] = \left(\prod_{k\in\mathcal{V}^0}(1-x_k)\right)\left(\prod_{k\in\mathcal{V}^1}x_k\right)$$

and

$$Z^{\vee}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]} = x_{[d]}] = 1-\left(\prod_{k\in\mathcal{V}^0}(1-x_k)\right)\left(\prod_{k\in\mathcal{V}^1}x_k\right).$$

**Lemma 2.4** *Terms are contractions of one-hot encodings, that is for any disjoint subsets $\mathcal{V}^0, \mathcal{V}^1 \subset [d]$ we have*

$$Z^{\wedge}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]}] = \left\langle \epsilon_{\{x_k=0:k\in\mathcal{V}^0\}\cup\{x_k=1:k\in\mathcal{V}^1\}}\right\rangle_{[X_{[d]}]}.$$

*Clauses are subtractions of one-hot encodings from the trivial tensor, that is for any disjoint subsets $\mathcal{V}^0, \mathcal{V}^1 \subset [d]$ we have*

$$Z^{\vee}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]}] = \mathbb{I}\left[X_{[d]}\right] - \left\langle \epsilon_{\{x_k=0:k\in\mathcal{V}^0\}\cup\{x_k=1:k\in\mathcal{V}^1\}}\right\rangle_{[X_{[d]}]}.$$

The reference of the formulas in the case $\mathcal{V}^0\dot\cup\mathcal{V}^1 = [d]$ as minterms and maxterms is due to the fact, that minterms are formulas with unique models and maxterms are formulas with a unique world not satisfying the formula. We use this insight and enumerate maxterms and minterms by the index $x \in \bigtimes_{k\in[d]}[2]$ of the unique world where the minterm is satisfied, respectively the maxterm is not satisfied. For any $\mathcal{V}^0\dot\cup\mathcal{V}^1 = [d]$ we take the index tuple $x_0, \ldots, x_{d-1}$ where $x_k = 0$ if $k \in \mathcal{V}^0$ and $x_k = 1$ if $k \in \mathcal{V}^1$ and define

$$Z^{\vee}_{x_0,\ldots,x_{d-1}} = Z^{\vee}_{\mathcal{V}^0}\mathcal{V}^1 \quad \text{and} \quad Z^{\wedge}_{x_0,\ldots,x_{d-1}} = Z^{\wedge}_{\mathcal{V}^0}\mathcal{V}^1.$$

**Corollary 2.1** *Minterms are basis elements of the tensor space, that is for any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have*

$$Z^{\wedge}_{x_{[d]}} = \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

*Maxterms are subtraction of basis elements from the trivial tensor, that is for any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have*

$$Z^{\vee}_{x_{[d]}} = \mathbb{I}\left[X_{[d]}\right] - \epsilon_{x_{[d]}}\left[X_{[d]}\right].$$

*Proof.* Follows from Lem. 2.4, since when $\mathcal{V}^0 \cup \mathcal{V}^1 = [d]$ the contraction of the one-hot encodings coincides with the one-hot encoding of a fully specified state. □

Based on this insight, we can decompose any propositional formula into a conjunction of maxterms or a disjunction of minterms as we show next.

**Theorem 2.8** *For any boolean tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]}\mathbb{R}^2$ we have*

$$\tau\left[X_{[d]}\right] = \left(\bigvee_{x_{[d]}\,:\,\tau\left[X_{[d]}=x_{[d]}\right]=1} Z^{\wedge}_{\{k\,:\,x_k=0\}}\{k\,:\,x_k=1\}\right)[X_{[d]}]$$

*and*

$$\tau\left[X_{[d]}\right] = \left(\bigwedge_{x_{[d]}\,:\,\tau\left[X_{[d]}=x_{[d]}\right]=0} Z^{\vee}_{\{k\,:\,x_k=0\}}\{k\,:\,x_k=1\}\right)[X_{[d]}].$$

*Proof.* To show the representation by minterms we use the decomposition

$$\tau\left[X_{[d]}\right] = \sum_{x_{[d]}\,:\,\tau\left[X_{[d]}=x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

and notice that each term in the disjunction modifies the formula by adding respective world $x_{[d]}$ to the models of the formula. To show the representation by maxterms we use the decomposition

$$\tau\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right] \quad - \sum_{x_{[d]} : \tau\left[X_{[d]}=x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

and notice that each term in the conjunction modifies the formula by removing the respective world $x_{[d]}$ from the models of the formula. Thus, both decompositions are propositional formulas with the same set of models as the formula $\tau$ and are thus identical to $\tau$. □

The decompositions found in Thm. 2.8 are also called canonical normal forms to propositional formulas $\tau\left[X_{[d]}\right]$.

*Remark 2.2 (Efficient Representation in Propositional Syntax)* The decomposition in Thm. 2.8 is a basis CP decomposition of the binary tensor and will further be investigated in Chapter **??**. The formulas constructed in the proof of Thm. 2.8 are however just one possibility to represent a formula tensor in propositional syntax. Typically there are much sparser representations for many formula tensors, in the sense that less connectives and atomic symbols are required. Having such a sparser syntactical description of a propositional formula can be exploited to find a shorter conjunctive normal form of the formula and construct a sparse polynomial based on similar ideas as in Thm. 2.8. We will provide such constructions in Chapter **??**, where we show that dropping the demand of directionality and investigating binary CP Decompositions will improve the sparsity of the polynomial formula representation.

### 2.3.7 Outlook

While we in this chapter investigated representation schemes for single propositional formulas, we will further study the representation of knowledge bases consisting in multiple formulas in Sect. **??**. Further, we will build hybrid models bridging the concepts of probability distributions and propositional logics in Sect. **??**. Propositional formulas will therein serve as features and base measures for exponential families.

## 2.4 Inference algorithms

Contractions of sub-networks are the local inference steps. We review algorithms like AC-3 in that perspective and give with knowledge propagation a generic inference scheme.

When having a Constraint Satisfaction Problem on a large number of variables, which are densely connected by constraint tensors, direct exploitation of the global entailment criterion in Thm. 2.2 will be infeasible. An alternative to deciding entailment by global operations is the use of local operations. Here we interpret a part of the network (for example a single core) as an own knowledge base (with atomic formulas being the roots of the directed subgraph, that is potentially differing with the atoms in the global perspective) and perform entailment with respect to that.

### 2.4.1 Monotonicity of Entailment

Remove markov networks for monotonocity statements. Understand as the proof of the rule $i$.

Vanishing local contractions provide sufficient but not necessary criterion to decide entailment, as we show in the next theorem.

**Theorem 2.9 (Monotonicity of Entailment)** *For any Markov Network $\mathbb{P}^{\mathcal{G}}$ on the decorated hypergraph $\mathcal{G}$ and any subgraph $\tilde{\mathcal{G}}$, we have for any formula that $\mathbb{P}^{\mathcal{G}} \models f$ if $\mathbb{P}^{\tilde{\mathcal{G}}} \models f$.*

To prove the theorem, we first establish the following lemma that states if a contraction of non-negative tensors vanishes, the vanishing of a contraction over a subset of these tensors is a sufficient criterion.

**Lemma 2.5** *For any non-negative tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G}$ and $\tilde{\mathcal{E}} \subset \mathcal{E}$ we have the following. For $\tilde{\mathcal{G}} = (\mathcal{U}, \tilde{\mathcal{E}})$ with $\mathcal{U} = \cup_{e\in\tilde{\mathcal{E}}} e$ and the tensor network $\tau^{\tilde{\mathcal{G}}}$ with tensors coinciding on $\tilde{\mathcal{E}}$ with those in $\tau^{\mathcal{G}}$ we have*

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[\varnothing]} = 0$$

*if* $\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[\varnothing]} = 0$.

*Proof.* Since the tensor network $\tau^{\tilde{\mathcal{G}}}$ is non-negative, we have whenever $\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[\varnothing]} = 0$ that

$$\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[X_{\mathcal{U}}]} = 0\,[X_{\mathcal{U}}]\ .$$

It follows with the commutation of contractions (see Thm. **??** in Chapter 8), that

$$\begin{aligned}\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}}]} &= \left\langle \{\tau^e \ :\ e \in \mathcal{E}/\tilde{\mathcal{E}}\} \cup \{\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[X_{\mathcal{U}}]}\} \right\rangle_{[X_{\mathcal{V}}]} \\ &= \left\langle \{\tau^e \ :\ e \in \mathcal{E}/\tilde{\mathcal{E}}\} \cup \{0\,[X_{\mathcal{U}}]\} \right\rangle_{[X_{\mathcal{V}}]} \\ &= 0\end{aligned}$$

Thus, also the contraction of $\tau^{\mathcal{G}}$ vanishes in this case. □

*Proof of Thm. 2.9.* We use Lem. 2.5 on the subset $\tau^{\tilde{\mathcal{G}}}$ of the cores $\tau^{\mathcal{G}}$ to the Markov Network $\mathbb{P}^{\mathcal{G}}$, which itself defines the Markov Network $\mathbb{P}^{\tilde{\mathcal{G}}}$. Whenever $\mathbb{P}^{\tilde{\mathcal{G}}} \models f$ for a formula $f$, then we have by Thm. 2.2

$$\left\langle \tau^{\tilde{\mathcal{G}}} \cup \{\neg f\} \right\rangle_{[\varnothing]} = 0\,.$$

It follows with Lem. 2.5 that also

$$\left\langle \tau^{\mathcal{G}} \cup \{\neg f\} \right\rangle_{[\varnothing]} = 0\,.$$

and therefore $\mathbb{P}^{\mathcal{G}} \models f$. □

*Remark 2.3* To make use of Thm. 2.9 we can exploit any entailment criterion. However, there is no general statement about entailment possible, when the local entailment does not hold. Thm. 2.9 therefore just provides a sufficient but not necessary criterion of entailment with respect to $\mathbb{P}^{\mathcal{G}}$.

### 2.4.2 Syntactical inference algorithms

Clauses are tensors, which either shrink or trivialize when contracted with basis vectors.

#### 2.4.2.1 Forward Chaining in Horn Logics

When all clauses definite and of length at most 2, then both the $1_{\mathcal{V}}$ and $0_{\mathcal{V}}$ satisfy the constraints. Forward chaining as copying length 1 clauses to other clauses and contract them. When terminated all clauses are of lenght at most 2, and definite if they initially where definite (then any clause of length 1 is a fact).

### 2.4.3 Backward Chaining in Horn Logics

Backward chaining as adding hypthesis and systematically removing it by other cores and possibly further hypothesis, when having local entailment.

#### 2.4.3.1 Resolution

**Theorem 2.10 (Grand resolution theorem)** *When a constraint tensor network of clauses is unsatisfiable, then the empty clause is in the closure under resolution.*

*Proof.* We give a standard proof [74] with the perspective of contractions. Having a clause tensor network we add basis vectors to get a elementary tensor network, which is satisfiable when we started with the closure under resolution. □

#### 2.4.3.2 Generic inference rules

More generally, whenever

$$\bigwedge_{f \in \mathcal{F}} f \models h$$

then the support of the sub contraction of $\mathcal{F}$ leaving the variables of $h$ open produces a formula entailing $h$.

# Chapter 3
# Constraint Satisfaction Problems

**Abstract** We generalize propositional logics to arbitrary variable dimensions and study generalized entailment as Constraint Satisfaction Problems.

## 3.1 Representation

Let us now explore a more general class of logical inference problems and discuss probabilistic entailment within that class. We then provide further examples based on categorical constraints. Following Chapter 5 in [74], we now define Constraint Satisfaction Problems.

**Definition 3.1** Let there be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and $\tau^{\mathcal{G}}$ be a tensor network of boolean constraint tensors $\tau^e\,[X_e]$ to each $e \in \mathcal{E}$, that is

$$\tau^{\mathcal{G}} = \{\tau^e\,[X_e] \,:\, e \in \mathcal{E}\}\,.$$

We then call $\tau^{\mathcal{G}}$ a *constraint network* and the tensor

$$\tau^{\mathcal{G}}\,[X_{\mathcal{V}}] = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}}]}$$

its contraction. The *constraint satisfaction problem (CSP)* to the constraint network $\tau^{\mathcal{G}}$ is the decision whether there is a state $x_{\mathcal{V}}$ such that

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}} = x_{\mathcal{V}}]} = 1\,.$$

We say the CSP is satisfiable, when there is such a state, and unsatisfiable if not.

*Example 3.1* Knowledge base satisfaction as a constraint satisfaction problem Given a knowledge base $\mathcal{KB} = \{f_\ell \,:\, \ell \in [p]\}$ deciding whether the knowledge base is satisfiable is a constraint satisfaction problem. Here the by the formula $f_\ell$ affected variables $X_{e_\ell}$ build the hyperedges, which are decorated by the constraint tensors coinciding with the formulas $f\left[X_{e_\ell}\right]$.

### 3.1.1 Manipulation of constraint tensor networks

We study basic manipulations of constraint tensor networks, which are leaving the contracted network invariant and which are leading to monotonically decreasing solution sets.

#### 3.1.1.1 Invariances

Decide whether tensor networks should be denoted hyperedge-based and not graph based.

The monotonicity of logic is generalized in the following theorem.

**Theorem 3.1 (Invariance under adding subcontractions, Rule** $(i)$**)** *Let there be a constraint network* $\tau^{\mathcal{E}}$ *on the hypergraph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ *and choose an arbitrary subset* $\tilde{\mathcal{E}} \subset \mathcal{E}$ *and* $\mathcal{U} \subset \mathcal{V}$*. We further choose a reduced subset* $\mathcal{E}^{\text{reduced}} \subset \mathcal{E}$ *with* $\mathcal{E} \backslash \tilde{\mathcal{E}} \subset \mathcal{E}^{\text{reduced}}$ *and for any* $e \in \mathcal{E} \backslash \mathcal{E}^{\text{reduced}}$ *we have* $e \subset \mathcal{U}$*. Then we have*

$$\left\langle \tau^{\mathcal{E}} \right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{E}^{\text{reduced}}} \cup \{ \mathbb{I}_{\neq 0} \circ \left\langle \tau^{\tilde{\mathcal{E}}} \right\rangle_{[\mathcal{U}]} \} \right\rangle_{[X_{\mathcal{V}}]} .$$

*Proof.* The theorem can be shown coordinatewise: When the coordinate to a index tuple $x_{\mathcal{V}}$ does not vanish, then also not in the sub-contraction $\left\langle \tau^{\tilde{\mathcal{E}}} \right\rangle_{[\mathcal{U}]}$ and the coordinate stays invariant. If it vanishes, then the coordinate of the augmented network also vanishes, independently of the sub-contraction. □

**Theorem 3.2 (Invariance under splitting disentangled edges, Rule** $(ii)$**)** *Let there be a constraint network* $\tau^{\mathcal{E}}$ *and let for some* $e \in \mathcal{E}$ *with* $e = e_0 \dot{\cup} e_1$ *exist tensors* $\tau^{e_0}\left[X_{e_0}\right]$ *and* $\tau^{e_1}\left[X_{e_1}\right]$ *obeying*

$$\tau^{e}\left[X_e\right] = \tau^{e_0}\left[X_{e_0}\right] \otimes \tau^{e_1}\left[X_{e_1}\right] .$$

*Then*

$$\left\langle \tau^{\mathcal{E}} \right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{(\mathcal{E} \cup \{e_0, e_1\}) \backslash \{e\}} \right\rangle_{[X_{\mathcal{V}}]} .$$

*Proof.* Trivial. □

### 3.1.2 Knowledge bases as constraint networks

We now investigate examples of constraint networks and their manipulations.

#### 3.1.2.1 Clauses

An interesting class of constraint networks are those where each constraint is a clause. Many logical inference algorithms are understood as manipulations of the constraint networks [9]. Before presenting the basic syntactical inference rules, let us derive contraction properties of clauses.

**Lemma 3.1** *For any clause* $Z^{\vee}_{(\mathcal{U}, y_{\mathcal{U}})}\left[X_{\mathcal{U}}\right]$ *and any* $k \in \mathcal{U}$ *and* $\tilde{y}_k \in [2]$ *we have*

$$\left\langle Z^{\vee}_{(\mathcal{U}, y_{\mathcal{U}})}\left[X_{\mathcal{U}}\right], \epsilon_{y_k}\left[X_k\right] \right\rangle_{[X_{\mathcal{U}}]} = \begin{cases} Z^{\vee}_{(\mathcal{U}, y_{\mathcal{U}})} X_{\mathcal{U} \backslash \{k\}} \otimes \epsilon_{y_k}\left[X_k\right] & \text{if} \quad y_k = \tilde{y}_k \\ \mathbb{I}\left[X_{\mathcal{U} \backslash \{k\}}\right] \otimes \epsilon_{y_k}\left[X_k\right] & \text{if} \quad y_k \neq \tilde{y}_k \end{cases} .$$

*Proof.* Using the tensor representation of clauses, by the $\mathbb{I}\left[X_{\mathcal{U}}\right] - \bigotimes_{k \in \mathcal{U}} \epsilon_{y_k}\left[X_k\right]$. □

*Example 3.2 (Modus ponens)* When there is an edge

$$e = \mathcal{V}^{\text{in}} \cup \mathcal{V}^{\text{out}}$$

with

$$\tau^{e}\left[X_e\right] = \left( \bigwedge_{v \in \mathcal{V}^{\text{in}}} X_v \right) \Rightarrow \left( \bigwedge_{v \in \mathcal{V}^{\text{out}}} X_v \right)$$

and for each $v \in \mathcal{V}^{\text{in}}$ an edge $\{v\}$ with $\tau^{\{v\}}\,[X_v] = \epsilon_1\,[X_v]$, then:

- Choose $\tilde{\mathcal{G}} = \left(e, \{e\} \cup \bigcup_{v\in\mathcal{V}^{\text{in}}}\{v\}\right), \mathcal{U} = e$
- Apply *i* rule to replace $\tilde{\mathcal{G}}$ with $\tau^e\,[X_e] = \bigtimes_{v\in e}\epsilon_1\,[X_v]$
- Apply *ii* rule to split into edges $\{v\}$ for $v \in e$ with $\epsilon_1\,[X_v]$

Notice, that any basis vector is also a clause. We further derive a contraction property of arbitrary clauses with negated literals.

**Lemma 3.2** *Let there be two clauses* $Z^{\vee}_{(\mathcal{U}^0, y^0_{\mathcal{U}^0})}\left[X_{\mathcal{U}^0}\right]$, $Z^{\vee}_{(\mathcal{U}^1, y^1_{\mathcal{U}^1})}\left[X_{\mathcal{U}^1}\right]$ *and let us assume there is* $k \in (\mathcal{U}^0 \cap \mathcal{U}^1)$ *with*

$$y^0_k \neq y^1_k\,.$$

*Then*

$$\mathbb{I}_{\neq 0} \circ \left\langle Z^{\vee}_{(\mathcal{U}^0, y^0_{\mathcal{U}^0})}\left[X_{\mathcal{U}^0}\right], Z^{\vee}_{(\mathcal{U}^1, y^1_{\mathcal{U}^1})}\left[X_{\mathcal{U}^1}\right]\right\rangle_{\left[X_{(\mathcal{U}^0\cup\mathcal{U}^1)\setminus\{k\}}\right]}$$
$$= \begin{cases} \mathbb{I}\left[X_{(\mathcal{U}^0\cup\mathcal{U}^1)\setminus\{k\}}\right] & \text{if} \quad \exists \tilde{k} \in (\mathcal{U}^0 \cup \mathcal{U}^1)\setminus\{k\} : y^0_{\tilde{k}} \neq y^1_{\tilde{k}} \\ Z^{\vee}_{(\mathcal{U}^0\cup\mathcal{U}^1)\setminus\{k\}, y_{(\mathcal{U}^0\cup\mathcal{U}^1)\setminus\{k\}}}\left[X_{(\mathcal{U}^0\cup\mathcal{U}^1)\setminus\{k\}}\right] & \text{else}\,. \end{cases}$$

*Proof.* Since contraction is the sum of the contractions with $\epsilon_1\,[X_k]$ and $\epsilon_1\,[X_k]$, shrinking the right and the left clause. Can argue with the unique assignments to the left and right shrinked variables such that vanishing coordinate is possible. Together, if they agree, they build the unique assignment where the resolvent is vanishing. □

*Example 3.3 (Resolution)* When there are two clauses with a complementary literal can apply *i* with the complementary literal variable left out of the open variables and get the resolvent clause. Notice that the old clauses cannot always be dropped, since they are no subsets of $\mathcal{U}$. This is different to the modus ponens inference rule.

### 3.1.3 Exactly-one and almost-one constraint

We further investigate Sudoku and the *n*-queens problem as examples of constraint networks, which are constructed using exactly-one constraint.

Generalize to arbitrary summation constraints?

**Definition 3.2** Given a list $X_{[d]}$ of boolean variables and a categorical variable $X$ with dimension $d$ the exactly-one constraintis the tensor $\tau^{=1}[X_{[m]}, X]$ with coordinates

$$\tau^{=1}[X_{[d]} = x_{[d]}] = \begin{cases} 1 & \text{if} \quad \sum_{k\in[d]} x_k = 1 \\ 0 & \text{else}\,. \end{cases}$$

The almost one constraint is the tensor $\tau^{\leq 1}[X_{[d]}]$

$$\tau^{\leq 1}[X_{[d]} = x_{[d]}] = \begin{cases} 1 & \text{if} \quad \sum_{k\in[d]} x_k \leq 1 \\ 0 & \text{else}\,. \end{cases}$$

We extend the exactly-one constraintby a decomposition variable $I$ of dimension $m$, which indicates which of the boolean variables is in the 1 state.

We notice that the categorical constraint tensor is the basis encoding of the one-hot map on $[m]$. The restriction to the image coordinate $k$ of the one-hot map is the function

$$\epsilon_k\left[\tilde{k}\right] = \begin{cases} 1 & \text{if} \quad \tilde{k} = k \\ 0 & \text{else}\,. \end{cases}$$

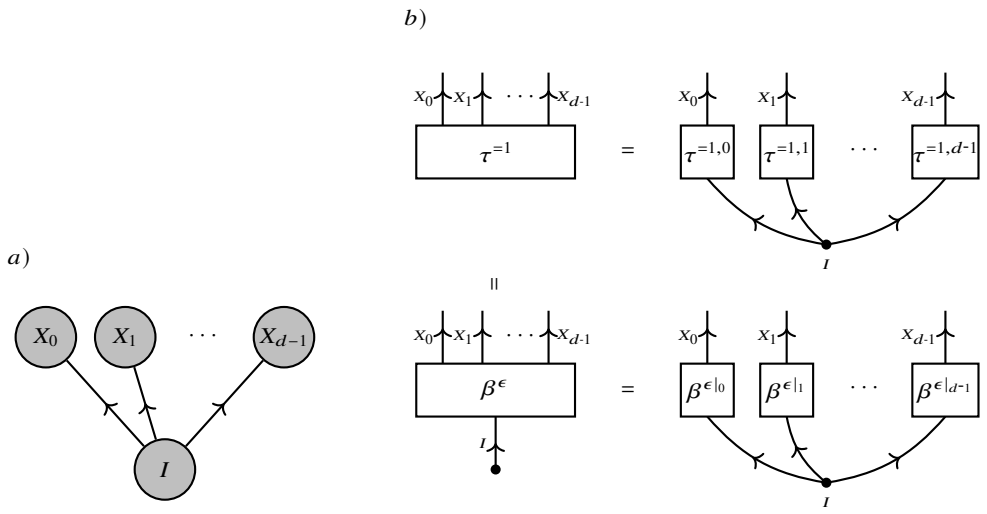



**Fig. 3.1** Representation of an exactly-one constraintin a CP Format tensor network.

To find a decomposition of the exactly-one constraintwe define cores

$$\tau^{=1,k}[X_k, I] = \epsilon_1[X_k] \otimes \epsilon_k[I] + \epsilon_0[X_k] \otimes \left(\mathbb{I}[I] - \epsilon_k[I]\right),$$

which are the basis encodings to the coordinate restrictions of the one-hot encoding map, that is

$$\tau^{=1,k}[X_k, I] = \beta^{\epsilon|_k}[X_k, I] \ .$$

We then have (see Figure 3.1)

$$\tau^{=1}[X_{[d]}] = \left\langle \left\{ \tau^{=1,k}[X_k, I] \ : \ k \in [d] \right\} \right\rangle_{[X_{[d]}]} .$$

This follows from the property of basis encodings of maps with multiple target coordinates, which are the contraction of the basis encodings to the target restriction maps (see Theorem XXX).

*Example 3.4 (Sudoku representation)* We recall the Sudoku knowledge base Example **??** and now aim to find efficient tensor network representation.

First of all, we notice that the formulas $\bigoplus^{(1)}[X_{[n^2]}]$ coincide with the exactly-one constraint.

We apply the decomposition of that constraint and get, that the knowledge base $\mathcal{KB}^n$ of Sudoku rules is a tensor network of the $4 \cdot n^4$ constraint formulas using the $n^2$-ary connective $\bigoplus^{(1)}$, and the evidence $E^{\text{start}}$ can be encoded by vectors $\epsilon_1\left[X_{(r_0,r_1,c_0,c_1,i)}\right]$. To get a representation by matrices instead of tensors of order $n^2$, we introduce a hidden variable $I$ taking values in $[n^2]$ for each of the constraints. With the usage of matrices

$$\tau^k[X_k, I] = \epsilon_0[X_k] \otimes \mathbb{I}[I] + (\epsilon_1[X_k] - \epsilon_0[X_k]) \otimes \epsilon_k[I]$$

we have the decomposition

$$\bigoplus^{(1)}[X_{[n^2]}] = \left\langle \{\tau^k[X_k, I] \ : \ k \in [n^2]\} \right\rangle_{\left[X_{[n^2]}\right]} ,$$

which is a CP decomposition (see Example 1.2) depicted in Figure 3.2 a).

Alternatively there is a TT decomposition (see Example 1.3) of the constraint $\bigoplus^{(1)}$, which we depict in Figure 3.2 b). We introduce for $k \in [d-1]$ hidden variables $I^k$ of dimension 2, which are interpreted as the indicator, whether one of the variables $X_{[k]}$ is true. Following this interpretation we introduce TT cores

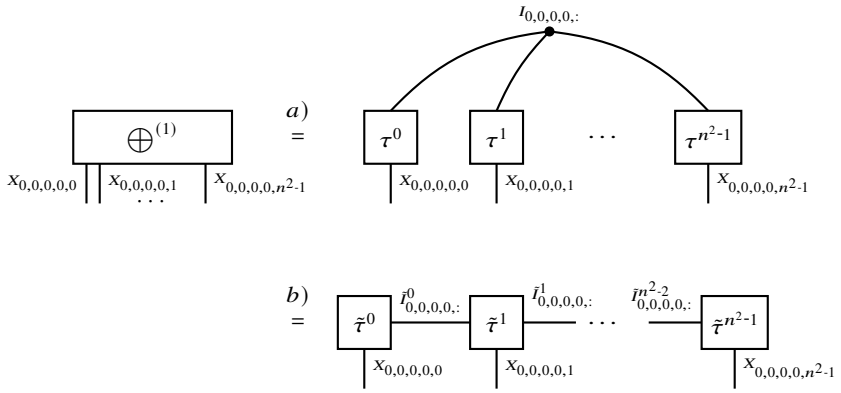



**Fig. 3.2** Decomposition of the position constraint $\bigoplus^{(1)}$ at position $(r0, r1, c0, c1) = (0, 0, 0, 0)$ into a) a CP decomposition with hidden variable $I_{0,0,0,0,:}$ and b) a TT decomposition with $d - 1$ hidden variables $I^k_{0,0,0,0,:}$, $k \in [d-1]$.

$$\tilde{\tau}^0\left[X_0, \tilde{I}^0\right] = \epsilon_1\left[X_0\right] \otimes \epsilon_1\left[\tilde{I}^0\right] + \epsilon_0\left[X_0\right] \otimes \epsilon_0\left[\tilde{I}^0\right]$$
$$\tilde{\tau}^{d-1}\left[\tilde{I}^{d-2}, X_{d-1}\right] = \epsilon_0\left[\tilde{I}^{d-2}\right] \otimes \epsilon_1\left[X_{d-1}\right] + \epsilon_1\left[\tilde{I}^{d-2}\right] \otimes \epsilon_0\left[X_{d-1}\right]$$

and for $k \in \{1, \ldots, d-2\}$

$$\tilde{\tau}^k\left[\tilde{I}^{k-1}, X_k, \tilde{I}^k\right] = \delta\left[\tilde{I}^{k-1}, \tilde{I}^k\right] \otimes \epsilon_0\left[X_k\right] + \epsilon_0\left[\tilde{I}^{k-1}\right] \otimes \epsilon_1\left[X_k\right] \otimes \epsilon_1\left[\tilde{I}^{k-1}\right] .$$

We notice, that the TT decomposition of the constraint $\bigoplus^{(1)}$ introduces $d - 1$ many hidden variables of dimension 2, whereas the CP decomposition introduces a single hidden variable of dimension $d$. However, in the following we will further apply the CP decomposition.

Given evidence $E^{\text{start}}$ we denote the Sudoku knowledge base $\mathcal{KB}^{n,E^{\text{start}}}$. We model the Sudoku knowledge base $\mathcal{KB}^{n,E^{\text{start}}}$ as a tensor network on a hypergraph $\mathcal{G}^{\text{Sudoku},n}$ consistent in

- $n^6 + 4 \cdot n^4$ nodes by $n^6$ categorical variables $X_{(r0,r1,c0,c1,i)}$ and by $4 \cdot n^4$ decomposition variables to the constraints
- $5 \cdot n^6$ edges

$$\mathcal{E} = \bigcup_{r0,r1,c0,c1 \in [n]} \Big\{ \{X_{(r0,r1,c0,c1,i)}\}, \{X_{(r0,r1,c0,c1,i)}, I_{r0,r1,c0,c1,:}\}, \{X_{(r0,r1,c0,c1,i)}, I_{r0,r1,:,:,i}\},$$
$$\{X_{(r0,r1,c0,c1,i)}, I_{:,:,c0,c1,i}\}, \{X_{(r0,r1,c0,c1,i)}, I_{r0,:,c0,:,i}\} \Big\}$$

We denote the decomposition variables to the position, row, column and square constraints by $I_{r0,r1,c0,c1,:}$, $I_{r0,r1,:,:,i}$, $I_{:,:,c0,c1,i}$ and $I_{r0,:,c0,:,i}$.

Each edge containing a decomposition variable is decorated by a matrix $\tau^k[X, I]$ corresponding to a core in the CP decomposition of a constraint. Here $k$ is determined by the tuple $(r0, r1, c0, c1, i)$ and the type of the constraint (for example, for the variable $X_{(0,1,1,2,1)}$ and the row constraint $I_{(0,1,:,:,1)}$ we have $k = 1 \cdot n + 2$. We further assign to each edge containing a single variable $\{X_{(r0,r1,c0,c1,i)}\}$ either the vector $\epsilon_1\left[X_{(r0,r1,c0,c1,i)}\right]$ if $(r0, r1, c0, c1, i) \in E^{\text{start}}$ or the trivial vector $\mathbb{I}\left[X_{(r0,r1,c0,c1,i)}\right]$.

For a more concrete example, let $n = 2$ and

$$E^{\text{start}} = \{(0,0,0,0,0), (0,0,1,0,2), (0,0,1,1,1), (0,1,0,1,1),$$
$$(1,0,1,0,3), (1,1,0,0,3), (1,1,0,1,2)\} .$$

We visualize this evidence by writing $i + 1$ in a grid cell $(r0, r1, c0, c1)$ to indicate that $(r0, r1, c0, c1, i) \in E^{\text{start}}$:

| | | | |
|---|---|---|---|
| 1 | | 3 | 2 |
| | 2 | | |
| | | 4 | |
| 4 | 3 | | |

*Example 3.5 (Exact cover)*

In general, constraint networks of the exactly-one constrainton boolean variables are interpreted as exact cover problems.

Here each constraint is understood as an element of a set, and each boolean variable is understood as the subset of constraints, which affect the variable.

Satisfying the exactly one constraints then amounts to choosing a collection of the subsets being an exact cover of all elements (i.e. their disjoint union is the set of constraints).

This generalizes the Sudoku problem Example 3.4, where we have in the boolean parametrization $4 \cdot n * *4$ elements (constraints) and $n * *6$ sets (variables). Further examples are visualizable by polyominoe tilings.

The dancing links algorithm is a well-established way to solve them [44].

By extending the dimension of the decomposition variable $I$ to $m+1$ we get a decomposition of the at-most one constraint. That is for the matrices

$$\tau^{\leq 1,k}[X_k, I] = \epsilon_1[X_k] \otimes \epsilon_k[I] + \epsilon_0[X_k] \otimes (\mathbb{I}[I] - \epsilon_k[I])$$

where $k \in [d]$ we have

$$\tau^{\leq 1}[X_{[d]}] = \left\langle \{\tau^{\leq 1,k}[X_k, I] \,:\, k \in [d]\} \right\rangle_{[X_{[d]}]} .$$

*Example 3.6 (Queens assignment game)* For arbitrary $n_0, n_1 \in \mathbb{N}$ we have a chess board of $n_0 \times n_1$ positions with $n_0$ columns and $n_1$ rows. We want to assign $r$ queens onto positions of the chess board, such that each column and each row contains at most one queen.

We assign to each position $(i,j)$ of the chess board a boolean variable $X_{i,j} \in \{0,1\}$, which indicates whether there is a queen at position $(i,j)$. The constraints that each row $i$ and each column $j$ contains at most one queen can then be expressed by the constraints $\tau^{\text{row},i}\left[X_{i\times[n_1]}\right]$ defined by coordinates

$$\tau^{\text{row},i}\left[X_{i\times[n_1]} = x_{i\times[n_1]}\right] = \begin{cases} 1 & \text{if} \quad \sum_{j\in[n_1]} x_{i,j} \leq 1 \\ 0 & \text{else} \end{cases}$$

$$\tau^{\text{col},j}\left[X_{[n_0]\times j} = x_{[n_0]\times j}\right] = \begin{cases} 1 & \text{if} \quad \sum_{i\in[n_0]} x_{i,j} \leq 1 \\ 0 & \text{else} . \end{cases}$$

In addition, we have diagonal constraints.

We introduce decomposition variables $I_{\text{row},i}$ and $I_{\text{col},i}$ of dimension $n_{\text{row},i} = n_1 + 1$ and $n_{\text{col},i} = n_0 + 1$. The first $n_1$ (respectively $n_0$) states shall be interpreted by a queen in the row (repectively column) at the corresponding position. The last state is interpreted by a missing queen in that row (respectively column). This interpretation is enforced by the matrices

$$\tau^{\text{row},i,j}\left[X_{i,j}, I_{\text{row},i}\right] = \epsilon_1\left[X_{i,j}\right] \otimes \epsilon_j\left[I_{\text{row},i}\right] + \epsilon_0\left[X_{i,j}\right] \otimes \left(\mathbb{I}\left[I_{\text{row},i}\right] - \epsilon_j\left[I_{\text{row},i}\right]\right)$$

and we have the property

$$\tau^{\text{row},i}\left[X_{i\times[n_1]}\right] = \left\langle \{\tau^{\text{row},i,j}\left[X_{i,j}, I_{\text{row},i}\right] \,:\, j \in [n_1]\} \right\rangle_{[X_{i\times[n_1]}]} .$$

The tensor of possible assignments to the queens game

$$\left\langle \{\tau^{\text{row},i,j}\left[X_{i,j}, I_{\text{row},i}\right] \,:\, i \in [n_0],\, j \in [n_1]\} \cup \{\tau^{\text{col},i,j}\left[X_{i,j}, I_{\text{col},i}\right] \,:\, i \in [n_0],\, j \in [n_1]\} \right\rangle_{\left[X_{[n_0]\times[n_1]}\right]} .$$

The number of possible assignments of queens to a $n_0 \times n_1$ board is the contraction without open variables.

#### 3.1.3.1 All-different constraint

The all-different constraint of order $d$ and dimenstions $m_{[d]}$ is the tensor with coordinates

$$\tau^{\mathrm{dif}}[X_{[d]} = x_{[d]}] = \begin{cases} 1 & \text{if} \quad \forall k, \tilde{k} \in [d] : (k = \tilde{k}) \vee (x_k \neq x_{\tilde{k}}) \\ 0 & \text{else}\,. \end{cases}$$

The all-different constraint in case of $d = 2$ is a matrix of rank $m$ and has therefore no tensor network decomposition with a variable of dimension less than $m$.

*Example 3.7 (Graph coloring)*
Graph coloring is the CSP where each edge of a graph has the add-different constraint assigned.

## 3.2 Inference

Based on the inference algorithms in propositional logics we now derive generalized versions for CSPs.

### 3.2.1 Solution by disentanglement

Let us now use the manipulation approaches to derive solution approaches to CSPs.

#### 3.2.1.1 Elementarization

Constraint Satisfaction Problems on elementary constraint networks have solutions by cartesian products of the local supports.

We want to find elementary constraint networks which support is included in the support of the constraint network.

We notice that finding a single solution assignment to a constraint network $\tau^{\mathcal{G}}$ is finding a one-hot encoding with

$$\epsilon_{x_{\mathcal{V}}}[X_{\mathcal{V}}] \prec \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}}]}\,.$$

One-hot encodings are in particular non-vanishing elementary tensors.

Let us generalize this approach to solving CSPs by elementarization:

- Find a non-vanishing elementary tensor network, which entails the

Modus ponens is an elementarization approach: The implication edges are split into elementary vectors.

#### 3.2.1.2 Vanishing sub-contraction

When a sub-contraction of the constraint network is vanishing, we know that the CSP has no solution.

This is a typical approach in logics, e.g. the aim of resolution.

### 3.2.2 Generlized unit propagation

We generalize forward chaining to arbitrary constraint networks. Main elements:

- Constraint vectors are contracted with constraint tensors containing the corresponding node.
- If the split successes and results in a new constraint vector, we continue step 1 with the new constraint vector.

**Algorithm 2** Generalized Forward Chaining

**Require:** Constraint tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, message edges $\mathcal{E}^k$ and a set $\mathcal{U}$ of subsets of $\mathcal{E}$ for local propagation

**Ensure:**

Initialize a queue $Q$ by all nodes with constraint vectors
**while** $Q$ is not empty **do**
  Choose a node variable $v$ from $Q$
  **for** All $e \in \mathcal{E}^k$ with $v \in e$ **do**
    Contract

$$\tau^e\left[X_e\right] = \left\langle \tau^e\left[X_e\right], \tau^{\{v\}}\left[X_v\right]\right\rangle_{[X_e]}$$

    Try to split $\tau^e\left[X_e\right]$ into $\{v\}$ and $e\backslash\{v\}$
    **if** Splitting successful and $|e\backslash\{v\}| = 1$ **then**
      Add $e\backslash\{v\}$ to the queue
    **end if**
  **end for**
**end while**
**return** $\{\tau^e\left[X_e\right] : e \in \mathcal{E}^k\}$

### 3.2.3 Constraint Propagation

Understand constraint propagation as an iterative application of the $(i)$ manipulation rule where messages are computed and contracted into memory cores. (Each step applies the rule twice: New message core computed and fused into old message core.) Call $\kappa$ memory core, which is updated by a message.

To store preliminary conclusions, we define auxiliary message cores storing constraints on variables $e \subset \mathcal{V}$. They are understood as logical formulas to the atomization variables $(X_e = x_e)$ of the respective formulas

$$\kappa^e\left[X_e\right] = \bigvee_{x_e \,:\, \kappa^e[X_e = x_e]} \bigwedge_{v \in e} \left(X_e = x_e\right) .$$

**Definition 3.3** Let $\tau^{\mathcal{G}}$ be a constraint satisfaction problem. We say that a message core $\kappa^e\left[X_e\right]$ is sound for $\tau^{\mathcal{G}}$, if

$$\mathbb{I}_{\neq 0} \circ \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_e]} \prec \kappa^e\left[X_e\right]$$

and complete for $\tau^{\mathcal{G}}$ if in addition

$$\mathbb{I}_{\neq 0} \circ \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_e]} = \kappa^e\left[X_e\right] .$$

We now provide a solution algorithm for constraint satisfaction problems by propagating local contractions. The dynamic programming paradigm is implemented by the storage of partial entailment results in message cores. We then iterate over local entailment checks, where we recursively add further entailment checks to be redone due to additional knowledge. This local entailment scheme is called Knowledge Propagation and described in a generic way in Algorithm 3.

We understand the support of local contractions as messages, which are tensors $\kappa^e\left[X_e\right]$ associated with message edges $e$. These message cores are propagated through the network, and summarize neighboring constraints.

Adjust constraint propagation to match tnreason message passing schemes: Queue containing $\tilde{\mathcal{G}}$ and open nodes $\mathcal{U}$.

**Algorithm 3** Constraint Propagation

**Require:** Constraint tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, message edges $\mathcal{E}^k$ and a set $\mathcal{U}$ of subsets of $\mathcal{E}$ for local propagation
**Ensure:** Message cores $\kappa^e [X_e]$ for $e \in \mathcal{E}^k$ with $\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_e]} \prec \kappa^e [X_e]$

Initialize for all $e \in \mathcal{E}^k$:

$$\kappa^e [X_e] = \mathbb{I} [X_e]$$

Initialize a queue

$$Q = \left\{ (\mathcal{E}^{inference}, e_m) \,:\, \mathcal{E}^{inference} \in \mathcal{U},\, e_m \in \mathcal{E}^k,\, e_m \cap \Big( \bigcup_{e \in \mathcal{E}^{inference}} e \Big) \neq \emptyset \right\}$$

**while** $Q$ is not empty **do**
Choose a set of edges from the queue

$$\mathcal{E}^{\text{inference}}, e_m \leftarrow Q.\text{pop}()$$

Contract

$$\tau \left[ X_{e_m} \right] = \mathbb{I}_{\neq 0} \circ \left\langle \{ \tau^{\tilde{e}} [X_{\tilde{e}}] \,:\, \tilde{e} \in \mathcal{E}^{\text{inference}} \} \cup \{ \kappa^e [X_e] \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{\tilde{e} \in \tilde{\mathcal{E}}} \tilde{e} \neq \emptyset \} \right\rangle_{[X_{em}]}$$

**if** $\tau [X_e] \neq \kappa^e [X_e]$ **then**

$$\kappa^e [X_e] \leftarrow \tau [X_e]$$

**for all** $\tilde{\mathcal{E}} \in \mathcal{U}$ with $e \cap \bigcup_{\tilde{e} \in \tilde{\mathcal{E}}} \tilde{e} \neq \emptyset$ **do**

$$Q.\text{push}(\tilde{\mathcal{E}})$$

**end for**
**end if**
**end while**
**return** $\{ \kappa^e [X_e] \,:\, e \in \mathcal{E}^k \}$

Each chosen subset $\tilde{\mathcal{E}} \in \mathcal{U}$ is understood as a local knowledge base, which is then applied for local entailment. The message cores are understood as messages, which propagate information from different regions of a tensor network (see Chapter 8).

There are different ways of implementing Algorithm 3, by choosing the set $\mathcal{U}$ of constraint sets $\tilde{\mathcal{E}}$ and domain $\mathcal{E}^k$.

*Example 3.8 (AC-3 as a Constraint propagation scheme)* The AC-3 algorithm (see [52]) is a specific instance, where message cores are assigned to single variables and propagation is performed on single constraint cores. We assume that each edge has cardinality 2, that is $\mathcal{G}$ is a graph. Here the inference clusters are

$$\mathcal{U} = \left\{ \{e\} \,:\, e \in \mathcal{E} \right\}.$$

The message edges are

$$\mathcal{E}^k = \left\{ \{v\} \,:\, v \in \mathcal{V} \right\}.$$

At each local inference step to the edge $e = \{v_0, v_1\}$ we compute

$$\left\langle \kappa^{\{v_0\}} \left[ X_{v_0} \right], \tau^e \left[ X_{v_0, v_1} \right], \kappa^{\{v_0\}} \left[ X_{v_1} \right] \right\rangle_{[X_{v_0}]}.$$

**Theorem 3.3** *At any state of the constraint propagation Algorithm 3, we have that each message core $\kappa^{\tilde{e}}$ is sound for $\tau^{\mathcal{G}}$. After each update in Algorithm 3, $\kappa^{\tilde{e}}$ is further monotonically decreasing with respect to the partial ordering.*

*Proof.* This follows from soundness of the *i* manipulation rule. We show the first claim by induction over the update steps in Algorithm 3. At the start, where $\kappa^e [X_e] = \mathbb{I} [X_e]$, we trivially have

$$\left\langle \tau^{\mathcal{G}} \cup \{\kappa^e\,[X_e] \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_\mathcal{V}]} = \left\langle \tau^{\mathcal{G}} \cup \{\mathbb{I}\,[X_e] \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_\mathcal{V}]} = \left\langle \tau^{\mathcal{G}}\right\rangle_{[X_\mathcal{V}]} .$$

Let us now assume, that for a state of cores $\{\kappa^e \,:\, e \in \mathcal{E}^k\}$ the first claim holds and let $\tilde{\mathcal{E}} \subset \mathcal{E}$ be chosen for the update of $\kappa^{\tilde{e}}$. By the invariance under adding the support of subcontractions, which we will proof in more detail as Thm. **??** in Chapter 8, we have for the update

$$\tilde{\kappa}^{\tilde{e}}[X_{\tilde{e}}] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{\tilde{e}\in\tilde{\mathcal{E}}} \tilde{e} \neq \emptyset\}\right\rangle_{[X_{\tilde{e}}]}$$

that

$$\left\langle \tau^{\mathcal{G}} \cup \{\kappa^e\,[X_e] \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_\mathcal{V}]} = \left\langle \tau^{\mathcal{G}} \cup \{\kappa^e\,[X_e] \,:\, e \in \mathcal{E}^k\} \cup \{\tilde{\kappa}^{\tilde{e}}[X_{\tilde{e}}]\}\right\rangle_{[X_\mathcal{V}]} .$$

Thus, the first claim holds also after the update of the core to $\tilde{e}$.

We further have with the monotonicity of boolean contraction (see Thm. **??**) that for any update of $\kappa^{\tilde{e}}$ by $\tilde{\kappa}^{\tilde{e}}$

$$\tilde{\kappa}^{\tilde{e}}[X_{\tilde{e}}] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{e\in\tilde{\mathcal{E}}} e \neq \emptyset\}\right\rangle_{[X_{\tilde{e}}]} \prec \kappa^{\tilde{e}}\,[X_{\tilde{e}}] .$$

Thus, each message core is monotonously decreasing at each update, with respect to the partial tensor ordering.

From the first claim we further have for any $\tilde{e} \in \tilde{\mathcal{E}}$

$$\left\langle \{\kappa^{\tilde{e}}\,[X_{\tilde{e}}]\} \cup \tau^{\mathcal{G}} \cup \left\{\kappa^e \,:\, e \in \mathcal{E}^k/\{\tilde{e}\}\right\}\right\rangle_{[X_{\tilde{e}}]} = \left\langle \tau^{\mathcal{G}}\right\rangle_{[X_{\tilde{e}}]}$$

And thus in combination with the monotonicity of boolean contraction (see Thm. **??**) that

$$\mathbb{I}_{\neq 0}\left(\left\langle \tau^{\mathcal{G}}\right\rangle_{[X_{\tilde{e}}]}\right) \prec \kappa^{\tilde{e}}\,[X_{\tilde{e}}] \;.\square$$

Let us now show that constraint propagation always terminates. We can further characterize the message cores at termination.

**Definition 3.4** We say that a set of message cores $\{\kappa^e \,:\, e \in \mathcal{E}^k\}$ is consistent with a set $\{\tau^e \,:\, e \in \tilde{\mathcal{E}}\}$, if for any $e \in \mathcal{E}^k$

$$\kappa^e\,[X_e] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_e]} .$$

This property is similar to the completeness of a message core, when interpreting the other message cores and the constraints $\{\tau^e \,:\, e \in \tilde{\mathcal{E}}\}$ as posing a Constraint Satisfaction Problem.

**Theorem 3.4** *The Constraint propagation Algorithm 3 always terminates. At termination we further for each $\tilde{\mathcal{E}} \in \mathcal{U}$ and $e \in \mathcal{E}^k$ with $e \cap \bigcup_{e\in\tilde{\mathcal{E}}} e \neq \emptyset$, that the message cores $\{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{e\in\tilde{\mathcal{E}}} e \neq \emptyset\}$ are consistent with $\{\tau^e \,:\, e \in \tilde{\mathcal{E}}\}$.*

*Proof.* For each message core, there are finitely many boolean tensor precessing it with respect to the partial order. Therefore, since they are monotonously decreasing, each message core can only be varied finitely many times during the algorithm. In total the algorithm can run only finitely many times in the second for loop, where new sets of edges are pushed into the queue. Therefore the while loop will always terminate.

When after a single pass through the while loop with chosen $\tilde{\mathcal{E}} \in \mathcal{U}$, the set $\tilde{\mathcal{E}}$ is not pushed back into $\mathcal{Q}$, we have for any $e \in \mathcal{E}^k$ with $e \cap \bigcup_{e\in\tilde{\mathcal{E}}} e \neq \emptyset$ that

$$\kappa^e\,[X_e] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{e\in\tilde{\mathcal{E}}} e \neq \emptyset\}\right\rangle_{[X_e]} .$$

Whenever the contraction on the right hand side changes during the algorithm, the set $\tilde{\mathcal{E}}$ is pushed into $\mathcal{Q}$. At termination of the algorithm, $\mathcal{Q}$ is empty, and the claimed consistency therefore has to hold. □

We can exploit the constraint propagation Algorithm 3 for the solution of Constraint Satisfaction Problems, by taking $\tau^{\mathcal{G}}$ as the tensor network of constraint tensors. Whenever a message core vanishes, we can conclude that the Constraint Satisfaction Problems is not satisfiable, as we show next.

**Corollary 3.1** *Let us for a Constraint Satisfaction Problem encoded by $\tau^{\mathcal{G}}$ run constraint propagation Algorithm 3. Whenever for any $\tilde{e} \in \mathcal{E}$ we have $\kappa^{\tilde{e}}\left[X_{\tilde{e}}\right] = 0\left[X_{\tilde{e}}\right]$, then the Constraint Satisfaction Problem is not satisfiable.*

*Proof.* Whenever $\kappa^{\tilde{e}}\left[X_{\tilde{e}}\right] = 0\left[X_{\tilde{e}}\right]$, then we have by Thm. 3.3

$$\mathbb{I}_{\neq 0}\left(\left\langle \tau^{\mathcal{G}}\right\rangle_{\left[X_{\tilde{e}}\right]}\right) \prec 0\left[X_{\tilde{e}}\right]$$

and therefore

$$\left\langle \tau^{\mathcal{G}}\right\rangle_{[\varnothing]} = 0 \,. \square$$

### 3.2.4 Applications

Let us exemplify the usage of constraint propagation on Constraint Satisfaction Problems posed by entailment queries on Markov Networks.

**Corollary 3.2** *Let Algorithm 3 be run on the cores $\tau^{\mathcal{G}} \cup \{\beta^f\}$ with an arbitrary design of $\tilde{\mathcal{E}}$. Whenever for a formula $f\left[X_{\mathcal{U}}\right]$ and a $\kappa^e$ we have*

$$\left\langle \kappa^e, \beta^f\right\rangle_{\left[Y_f=0\right]} = 0$$

*then the Markov Network $\tau^{\mathcal{G}}$ probabilistically entails $f$. If on the contrary*

$$\left\langle \kappa^e, \beta^f\right\rangle_{\left[Y_f=1\right]} = 0$$

*then the Markov Network $\tau^{\mathcal{G}}$ probabilistically entails $\neg f$, that is probabilistically contradicts $f$.*

*Proof.* This follows from Thm. 3.3 ensuring the soundness of constraint propagation and the sufficiency of local entailment. □

*Example 3.9 (Batch decision of entailment)* Let $\mathcal{F}$ be a set of formulas and $\mathbb{P}^{\mathcal{G}}\left[X_{[d]}\right]$ a Markov Network, for which it shall be decided, which formulas in $\mathcal{F}$ are entailed, contradicted or contingent. We can in addition to the cores of the Markov Network create the cores $\{\beta^f\left[Y_f, X_{[d]}\right] : f \in \mathcal{F}\}$ and prepare the message cores

$$\kappa^{\{f\}}\left[Y_f\right] \,.$$

To decide entailment batchwise, constraint propagation Algorithm 3 can be run. Whenever during the algorithm we have that for a $f$, then Cor. 3.2 implies that if

$$\kappa^{\{f\}}\left[Y_f\right] = \begin{cases} \epsilon_1\left[Y_f\right] & \text{then, the formula is entailed by } \mathbb{P}^{\mathcal{G}} \,. \\ \epsilon_0\left[Y_f\right] & \text{then, the formula is contradicted by } \mathbb{P}^{\mathcal{G}} \,. \\ \mathbb{I}\left[Y_f\right] & \text{then no conclusion can be drawn.} \end{cases}$$

Note, that $\kappa^{\{f\}}\left[Y_f\right] = 0\left[Y_f\right]$ can not happen, since this would mean that $\mathbb{I}_{\neq 0}\left(\mathbb{P}^{\mathcal{G}}\right)$ is inconsistent. Thus, at any stage of Algorithm 3, one of the three holds.

### 3.2.5 Backtracking search

Understand as adding basis vectors which are locally possible (i.e. can satisfy affected constraints).

When the constraint propagation Algorithm 3 converges in a given implementation and no message core vanishes, we can however not conclude that the Constraint Satisfaction Problem is not satisfiable. However, for any index tuple $x_{\mathcal{V}}$ to be a solution of the CSP to $\tau^{\mathcal{G}}$, we have the necessary condition

$$\forall \tilde{e} \in \tilde{\mathcal{E}} : \kappa^e \left[ X_{\tilde{e}} = x_{\mathcal{V}}|_{\tilde{\mathcal{E}}} \right] = 1 \,,$$

where by $x_{\mathcal{V}}|_{\tilde{\mathcal{E}}}$ we denote the restriction of the index tuple $x_{\mathcal{V}}$ to the variables included in $\tilde{\mathcal{E}}$. One can use this insight as a starting point for backtracking search, where the assignments to variables $X_{\mathcal{U}}$ are iteratively guessed, based on the restriction that each constraint is locally satisfialbe, i.e. .

$$\forall \tilde{e} \in \tilde{\mathcal{E}} : \left\langle \kappa^e \left[ X_{\tilde{e} \cap \mathcal{U}} = x_{\mathcal{V}}|_{\tilde{e} \cap \mathcal{U}}, X_{\tilde{e}/\mathcal{U}} \right] \right\rangle_{[\varnothing]} \neq 0 \,.$$

One can understand the guess of an assignment $x_v$ to a variable $X_v$, as it is done during backtracking search, as an inclusion of a constraint

$$\kappa^{\{v\}} \left[ X_v \right] = \epsilon_{x_v} \left[ X_v \right] \,.$$

Therefore, constraint propagation Algorithm 3 can be integrated with backtracking search, with iterations between propagations of knowledge and guessing of additional variables.

### 3.2.6 Discussion

In our perspective, the number of possible sub-networks to be contracted is the bottleneck, instead of the demand to perform such contractions. The choice and the scheduling of these sub-contractions is most important for the design of efficient inference schemes.

# Chapter 4
# First-Order Logic

**Abstract** To introduce the syntax and semantics of first-order logic, we in this chapter introduce the substitution tensor structure. This allows to capture structured representations of systems, beyond the factored representations introduced in the first two chapters.

## 4.1 Syntax and Semantics of first-order logic

The framework of first-order logic generalizes propositional logic analogously to the generalization of factored system representations towards structured system representations [74]. This more expressive framework is designed to reason about relations and functions between objects. To capture this framework by tensors we here first define the syntax and then investigate tensor representations of the semantics.

### 4.1.1 Syntax

A first-order logic theory consists in a finite set of constant symbols $\mathcal{D}$, a set of function symbols $\{v_l : l \in [r]\}$, a set of predicate symbols $\{g_k : k \in [d]\}$ and an arity map assigning the arity $n_l$ to the function $v_l$ and the arity $n_k$ to the predicate $g_k$.

### 4.1.2 Tensor Representation of Semantics

We here follow the model-theoretic semantics of first-order logic, where sets of possible worlds are considered. We only treat in this work database semantics, where we assume that the domain $\mathcal{D}$ of each world has a one-to-one mapping to the constants of the theory and is therefore constant among the worlds. Database semantics is central to combine the semantic structure with the substitution structure. Under this assumption, the dimension of the substitution structure does not depend on the specific world, and we can combine both structures in a single tensor space to be defined in the next sections.

To each world there is a world domain $\mathcal{D}$ of objects, which we assume to be finite. We exploit the set-encoding formalism discussed in more detail in Chapter 7 and use bijective index interpretation maps

$$I : [r] \to \mathcal{D} .$$

A so-called term variable $O$ takes states $o \in [r]$, which represent objects

$$I[o] \in \mathcal{D} .$$

The relations between objects are described by $n$-ary predicates $g$. Given a specific world $x_W$ the truth of relations is represented by boolean tensors

$$g|_{x_W} : \bigtimes_{l \in [n]} [r] \to \{0, 1\} .$$

Given a tuple $o_0, \ldots, o_{n-1} \in \bigtimes_{l \in [n]} [r]$ the boolean

$$g|_{x_W} [O_0 = o_0, \ldots, O_{n-1} = o_{n-1}] \in \{0, 1\}$$

is called a grounding and encodes, whether the relation $g$ is satisfied in the world $x_W$ for the objects $I [o_0], \ldots, I [o_{n-1}]$.

Functions in first-order logic are object-valued maps

$$v|_{x_W} : \mathcal{D}^r \to \mathcal{D} .$$

While relations are represented by their coordinate encodings, functions are represented by directed basis encodings

$$\beta^{v|_{x_W}} \left[O_v, O_{[n]}\right]$$

where to each object tuple $I [o_0], \ldots, I [o_{n-1}]$ the vector $\beta^{v|_{x_W}} \left[O_v, O_{[n]} = o_{[n]}\right]$ is the one-hot encoding of the object, that is

$$\beta^{v|_{x_W}} \left[O_v, O_{[n]} = o_{[n]}\right] = \epsilon_{v|_{x_W}[I[o_0],\ldots,I[o_{n-1}]]} [O_v] .$$

### 4.1.3 Worlds

Worlds in First-Order Logics contain varying numbers of objects. We use domain variables $O$ to enumerate the sets of objects.

Worlds are represented by three sets of tensors:

- One-hot encodings of constants
- Coordinate encodings for each predicate
- Basis encoding of each function

### 4.1.4 Two Tensor Structures

In comparison with propositional logic, first-order logic bears two natural tensor structures.

- **Semantic structure:** As in propositional logic, we enumerate possible worlds by a collection of variables $X$. This is a representation of the model-theoretic semantics, where subsets of worlds are represented by boolean tensors.
- **Substitution structure:** Different to propositional logic, a formula can have variables and the evaluation of a formula on a world is represented by its grounding tensor. Given a world which contains objects in the domain $\mathcal{D}$, we can substitute variables by objects in the domain. In this way, each predicate with $r$ variables is represented in that world as a boolean tensor of order $r$.

While the semantic structure appears already in factored representations of systems, the substitution structure arises in more generality when treating structured representations of systems [74].

## 4.2 Substitution Structure

We here investigate the structure of worlds in first-order logics and how they can be represented by tensors. As we already argued, groundings of sentences can be aligned in boolean tensors, to be called grounding tensors. We here investigate sparse representation schemes for this substitution structure. In particular we apply basis calculus to find sparse decompositions of the grounding tensors

The semantic structure, an attempt to represent in tensors multiple worlds at once, is to be investigated in Chapter 19.

### 4.2.1 Grounding Tensors

Given a first-order logic world $x_W$, arbitrary formulas are interpreted in terms of the satisfactions of their groundings. We define their semantic first, and then relate their syntactical decomposition to tensor networks, similar to our approach to propositional logic in Chapter 2.

**Definition 4.1 (Grounding of a first-order formula given a world)** Given a specific world $x_W$, with a domain $\mathcal{D}$ enumerated by $[r]$, the grounding of a formula $q$ with variables $O_q$ is the tensor

$$q|_{x_W}\left[O_q\right] : \bigtimes_{l \in [O_q]} [r] \to \{0,1\} .$$

Each coordinate represents thereby the boolean, whether the substitution of the variables in the formula is satisfied given a world $x_W$, that is

$$q|_{x_W}\left[O_q = o_q\right] = 1$$

if and only if the substitution of $q$ with the variables $O_q$ replaced by the objects $I\left[o_l\right]$ is satisfied on the world $x_W$.

We in the following investigate how these tensors are represented by tensor networks involving the world defining tensors.

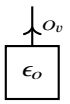
### 4.2.2 Basis encodings of terms

Terms are object valued functions, which $r_v$ different arguments are also object valued and called variables. We represent terms by the corresponding basis encodings

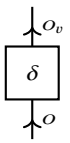
$$\beta^v\left[O_v, O_{[r_v]}\right]$$

Basic blocks:

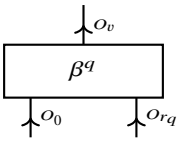
- Constant




- Variable

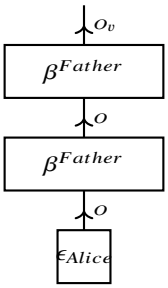



- Function of terms




Contractions of those terms along directed acyclic graphs with single root nodes represent more complex terms. As such, basis encodings of functions can be contracted with constants, variables or other functions to build up basis encodings of more complex terms.

Example: Father of the father of alice is represented by the contraction

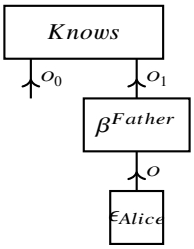



We notice, that the basis encoding of each term is decomposed into a network of world defining tensors, namely constant and function defining tensors.

### 4.2.3 Atomic sentences

Atomic sentences are contractions of coordinate encodings of predicates with basis encodings of terms.

Example: Knowing the father of alice




Complex sentences are built by

- Quantifier free sentences are contractions of basis encodings to atomic sentences with basis encodings of the connectives.
- Quantifiers act as non-linear transforms of tensors.

### 4.2.4 Formula Synthesis by Connectives

In order to have a sound semantic, the grounding of first-order logic formulas is determined by the syntax of the formula, i.e. a decomposition of the formula into connectives and quantifiers acting on atomic formulas.

Quantifier-free formulas are connectives acting on atomic formulas. We can describe them as in the case of propositional logic in the $\beta$-formalism. While the atomic formulas where delta tensors copying states, they are more involved here.

**Theorem 4.1** *For any connective $\circ$ and formulas $q_1$ and $q_2$ we have*

$$(q_1 \circ q_2)|_{x_W} \left[O_{q_1} \cup O_{q_2}\right] \tag{4.1}$$
$$= \left\langle \beta^{q_1|_{x_W}} \left[Y_{q_1}, O_{q_1}\right], \beta^{q_2|_{x_W}} \left[Y_{q_2}, O_{q_2}\right], \beta^{\circ} \left[Y_{q_1 \circ q_2}, Y_{q_1}, Y_{q_2}\right], \epsilon_1 \left[Y_{q_1 \circ q_2}\right] \right\rangle_{[O_{[n]}]} . \tag{4.2}$$

*Proof.* This directly follows from Thm. **??**. □

Here, variables can be shared by the connected formulas, therefore the variables in the combined formula are unions of the possible not disjoint variables of the connected formulas.

When interpreting the head variables of relational encoded atomic formulas as the atoms of a propositional theory, we find a propositional formula $f$ associated with any decomposable first-order logic formula.

**Definition 4.2** Given a formula $q$ in first-order logic, we say that a propositional formula $f\left[X_{[d]}\right]$ is the propositional equivalent to $q$ given atomic formulas $g_k$ in first-order logic, when for any world $x_W$ we have

$$q|_{x_W}\left[O_q\right] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{g_k}\right] : k \in [d]\} \cup \{f\left[X_{[d]}\right]\} \right\rangle_{[O_q]} .$$

We here denote the head variables of the basis encoding to $\beta^{g_k|_{x_W}}$ by $X_k$ to highlight their interpretation as propositional atoms.

We depict the relation of a grounding tensor to a propositional formula in Figure 4.1.

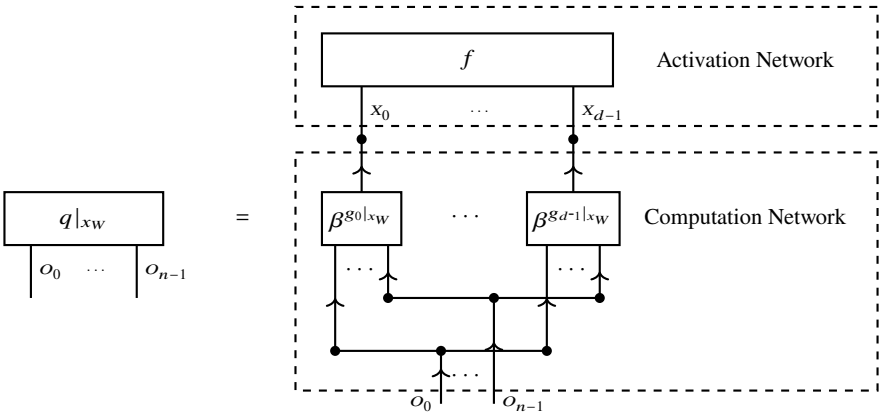



**Fig. 4.1** Grounding of a quantifier-free first-order logic formula $q$ is a Computation-Activation Network: The atomic formulas $g_k$ are computed and activated by a propositional formula $f\left[X_{[d]}\right]$. The computation network therefore consists of the of the (possibly substituted) predicate groundings, while the activation network consists of the (possibly decomposed) propositional formula $f\left[X_{[d]}\right]$.

### 4.2.5 Quantifiers

Existential and universal quantifiers appear in generic first-order logic and are besides substitutions further means to reduce the number of variables in a formula.

The semantics of existential quantification consists in a formula being true, if at least one state of the quantified variable is true, as we define next.

**Definition 4.3** Given a grounding tensor

$$q|_{x_W}\left[O_0, \ldots, O_{n-1}\right]$$

the existential and universal quantification with respect to the first variable are the tensors

$$\left(\exists_{o_0} q\right)|_{x_W}\left[O_1, \ldots, O_{n-1}\right] \quad \text{and} \quad \left(\forall_{o_0} q\right)|_{x_W}\left[O_1, \ldots, O_{n-1}\right]$$

with coordinates as follows. For an assignment $o_1, \dots, o$ to the non-quantified variables we have

$$(\exists_{o_0} q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1$$

if and only if there is an assignment $o_0 \in [r_0]$ such that

$$q|_{x_W} [O_0 = o_0, O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1 \,.$$

Conversely, we have for the universal quantification that

$$(\forall_{o_0} q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1$$

if and only if for any assignment $o_0 \in [r_0]$ we have

$$q|_{x_W} [O_0 = o_0, O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1 \,.$$

Let us now show, that existential and universal quantification are coordinatewise transforms (see Def. **??**) of contracted grounding tensors. To this end, let us introduce the greater-$z$ indicator $\mathbb{I}_{>z}$, where $z \in \mathbb{R}$, as the function

$$\mathbb{I}_{>z} : \mathbb{R} \to \{0, 1\} \quad , \quad \mathbb{I}_{>z}(x) = \begin{cases} 1 & \text{if} \quad x > z \\ 0 & \text{else} \end{cases} \,.$$

**Theorem 4.2** *For any formula $q$ with variables $O_{[n]}$ we have*

$$(\exists o_0 q)|_{x_W} [O_1, \dots, O_{n-1}] = \mathbb{I}_{>0} \left( \langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1, \dots, O_{n-1}]$$

*and*

$$(\forall o_0 q)|_{x_W} [O_1, \dots, O_{n-1}] = \mathbb{I}_{>r-1} \left( \langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1, \dots, O_{n-1}]$$

*Proof.* We proof the claimed equalities to arbitrary slices of the remaining variables, which amount to arbitrary substitutions of the formulas. For any indices $o_1 \in [r_1], \dots, o_{n-1} \in [r_{n-1}]$ We notice that

$$\begin{aligned} \langle q|_{x_W} \rangle_{[O_1 = o_1, \dots, O_{n-1} = o_{n-1}]} &= \sum_{o_0 \in [r_0]} q|_{x_W} [O_0 = o_0, \dots, O_{n-1} = o_{n-1}] \\ &= \left| o_0 \in [r_0] \,:\, q|_{x_W} [O_0 = o_0, \dots, O_{n-1} = o_{n-1}] = 1 \right| \,. \end{aligned}$$

We can thus understand the contracted grounding tensor as storing in its coordinates the count of the coordinate extensions to the zeroth variable, such that the grounding tensor is satisfied. This is analogous to our interpretation of contracted propositional formulas as world counts. From this it is obvious, that the existential quantification is satisfied, if the count is different from zero, which is captured by the coordinatewise transform with $\mathbb{I}_{>0}$. We therefore arrive at

$$(\exists_{o_0} q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = \mathbb{I}_{>0} \left( \langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] \,.$$

The first claim follows, since the assignment to the non-quantified variables was arbitrary. The universal quantification is satisfied, when all extensions are satisfied, and the count is $r$. Since $r$ is the maximal count, this is captured by the coordinatewise transform with $\mathbb{I}_{>r-1}$ and we get

$$(\forall o_0 q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = \mathbb{I}_{>r-1} \left( \langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] \,.$$

With the same argument, the second claim is established. □

### 4.2.6 Universally quantified formulas as contraction equations

When restricting to universally quantified formulas, we can understand them as contraction equations

FOL knowledge bases without existential quantifiers are understood as contraction equations. That is, exactly those worlds are models, for which

$$\left\langle \{ q_\ell \left[ O_{[r_{q_\ell}]} \right] \,:\, \ell \in [p] \} \right\rangle_{[O_\mathcal{V}]} = \mathbb{I}\,[O_\mathcal{V}] \ .$$

### 4.2.7 Discussion

We can extend this discussion towards more generic counting quantifiers, of which the existential and the universal quantifier are extreme cases. One can define quantifiers by demanding that at least $z \in \mathbb{N}$ compatible groundings are satisfied, and show that they amount to coordinatewise transforms with $\mathbb{I}_{>z}$. What is more, quantifiers demanding that at most $z \in \mathbb{N}$ are satisfied would be representable by transforms with an analogously defined function $\mathbb{I}_{\leq z}$. Such customized quantifiers appear for example in the OWL 2 standard of description logics (see [73] and Sect. 4.4).

As will be discussed in Chapter 7, any coordinatewise transform can be performed by a contraction of a basis encoding of the tensor with a head vector prepared by the transform function (see Thm. **??**). In the case here, a direct implementation would require a dimension of these head variables by $r$, which can be infeasible when having large object sets.

To summarize, let us assume a formula is in its prenex normal form, that is a collection of quantifiers are acting on a quantifier free part. We can represent its grounding tensor in three steps:

- Instantiations of the atom groundings with the assigned variables, as contractions of the basis encoding of the world tensor with atom selecting tensors.
- Propositional formula acting on the head variables of the predicate instantiations, representing the connectives combining the formula.
- Quantifiers as a composition of contractions closing the quantified variable and coordinatewise transforms with the respective greater-than indicators.

## 4.3 Tensor storage of Groundings

A typical storage of a grounding is the enlistment of all non-zero coordinates. We understand this as a basis CP decomposition of the grounding tensor. We then propose more efficient storage tensor formats, which are respecting the structure of the grounded formula.

### 4.3.1 Storage in Basis CP Decomposition

In many situations, grounding cores are sparse and representations as single tensor cores comes with a drastic overhead. We often encounter sparse grounding tensors, where the number of non-zero coordinates (to be investigated by basis CP ranks in Chapter **??**) satisfies

$$\ell_0 \left( q|_{x_W} \right) << r^{|O_q|} \ .$$

In this case, since most coordinates vanish, the basis CP decomposition (see Sect. 10.1.2) enables a representation of the grounding with significantly lower storage demand, see Thm. 10.1. This is particularly useful for representing large relational databases, where each object has only a few relations with others, while the majority of possible relations remains unsatisfied. We depict such CP decomposition of a formula grounding in Thm. 4.2.

Most logical syntaxes exploit $\ell_0$-sparsity, explicitly storing only known assertions. The interpretation of unspecified assertions depends on the underlying assumptions. Under the Closed World Assumption, all unspecified assertions are assumed to be false.

*Example 4.1* Consider the binary relation $Knows$ on the set of individuals $\{Alice, Bob, Charlie\}$ defined by the following pairs:

| **Individual 0** | **Individual 1** |
|---|---|
| Alice | Bob |
| Bob | Alice |
| Charlie | Bob |
| Bob | Charlie |

$$Knows[O_0, O_1] = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} [O_0, O_1] \ .$$

with a CP representation with the leg cores

$$\rho^0 [O_0, I] = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} [O_0, I] \quad \text{and} \quad \rho^1 [O_1, I] = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} [O_1, I] \ .$$

Consider now an extension

| **Individual 0** | **Individual 1** |
|---|---|
| Alice | Bob |
| Bob | Alice |
| Charlie | Bob |
| Bob | Charlie |
| John | Alice |
| John | Bob |
| John | John |
| Alice | John |
| Bob | John |
| Charlie | John |

Enumerating them in the order $[Alice, Bob, Charlie, John]$ this is the matrix

$$Knows[O_0, O_1] = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} [O_0, O_1] \ .$$

This has basis rank of 11. Notice, that allowing for bas+ representation, we would reduce to 5, by encoding that John knows everyone, and everyone knows John.

$$\rho^0 [O_0, I] = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} [O_0, I] \quad \text{and} \quad \rho^1 [O_1, I] = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} [O_1, I] \ .$$

### 4.3.2 Storage in Tensor Network Formats

We now investigate, whether the basis CP storage of a formula is optimal.

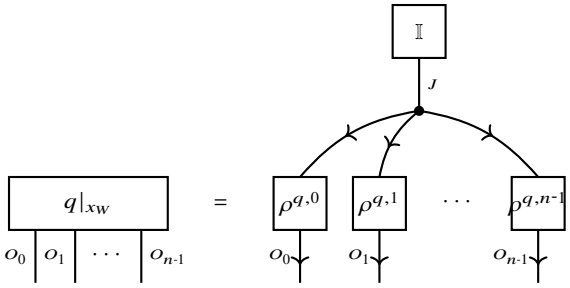



**Fig. 4.2** Basis CP Decomposition of the grounding of $q$, following the scheme of Thm. 10.1. Instead of direct storage of the grounding tensor $q|_{x_W}$, the non-zero coordinates are enumerated by a variable $J$ and the corresponding coordinates stored in leg-matrices $\rho^{q,l}$.

- Advantage of contracted storage: Redundancies by vanishing slices removed. We could try to remove redundancies by message passing algorithms, processing constraints on the support of the tensor.
- Advantage of decomposed store: Repetition effects of partial indices are respected.

*Example 4.2 (Conjunction chain of random predicates)*

Let us consider a conjunction chain of predicates, i.e. the $k$ predicate shares with the $k+1$ and $k-1$ predicate one term variable, but with no other predicate. The formula would be a TT-like tensor network. Now let us consider an random world, where each complete substitution of each predicate is satisfied independently with probability $p$. The expectation of the groundings is the tensor $p^d \cdot \mathbb{I}\left[O_{[d+1]}\right]$ and the expectation of the contraction is

$$p^d \cdot r^(d+1) .$$

The expected basis CP rank of the grounding grows exponentially in $d$, while in the TT-like decomposition the expectation of the storage demand is

$$p \cdot d \cdot r^2$$

and thus linear in $d$.

More general, when any random network of predicates is considered, the expected contraction is

$$p^{|\mathcal{E}|} \cdot r^n$$

and the storage demand

$$\sum_{e \in \mathcal{E}} p \cdot r^{|e|} .$$

### 4.3.3 Example: Relational Databases

A database is understood as a specific first-order logic world, and are operations on such a single world. Queries are described by a formula $p$, which are asked against a specific world $x_W$ to retrieve the grounding $p|_{x_W}$. The variables of such formulas are called projection variables. The answer $p|_{x_W}$ of a query is most conveniently represented as a list of solution mappings from the projection variables to objects in the world, such that the query formula is satisfied. Answering a query by solution mappings corresponds with finding the basis CP Decomposition (see Sect. 10.1.2) of $p|_{x_W}$. We can understand these solution mappings as stored in the leg-matrices $\rho^{q,l}$ (see Figure 4.2).

Let us give with the outer join an example of a popular operation to define queries, which efficient execution and storage can be improved based on considerations in the tensor network formalism.

**Definition 4.4 (Outer join)** Let there be a world $x_W$ and formulas $g_\ell$ depending on variables $O_{\mathcal{V}^\ell}$, which have grounding tensors by

$$g_\ell|_{x_W}\,[O_v]\;:\;\bigtimes_{v\in\mathcal{V}^\ell}[r_v]\to\{0,1\}\,.$$

Then their (outer) JOIN is defined as the grounding of their conjunctions, as

$$\mathrm{JOIN}\,(g_0,\ldots,g_{p-1})|_{x_W}\left[\bigcup_{\ell\in[p]}O_{\mathcal{V}^\ell}\right]=\left\langle g_\ell|_{x_W}\left[O_{\mathcal{V}^\ell}\right]\;:\;\ell\in[p]\right\rangle_{\left[\bigcup_{l\in[p]}O_{\mathcal{V}^\ell}\right]}\,.$$

We can understand the JOIN of groundings by a factor graph, where each grounding tensor decorates the hyperedge to the node set $\mathcal{V}^\ell$. The projection variable assignment to each formula combined in a JOIN operation provide a basic tensor network format to store the output of the operation. There are thus situations, in which the solution map storage corresponding with a CP Decomposition comes with unnecessary overheads compared with other formats.

### 4.3.4 Sparsification

The grounding of a formula when represented as a tensor network often comes with redundancies to be removed for sparser representation. Some assignments to variable subsets, which satisfy local formulas, can never be extended to a global assignments, which is supported by the overall grounding. Iterative message passing algorithms can be constructed to remove the redundancy, analogously to solution algorithms of CSPs (see Sect. 8.3).

## 4.4 Representation of Knowledge Graphs

Knowledge graphs as a specific fragment of FOL. Here we focus on A-Box representation by sparse tensors.

Let us now represent a specific fragment of first-order logic, namely Description Logics which Knowledge Bases are often referred to as Knowledge Graphs. We here use the OWL 2 standard, which encodes the syntax of the description logic $\mathcal{SROIQ}(\mathcal{D})$ [73]. It is further applied as a foundation of semantic web [2]. Knowledge bases in this fragment are further called Knowledge Graphs [40], due to a graphical interpretation. Note that the relation to tensor representations is here through extended adjacency matrices, and not decorations of the graphs by tensors as in the formalism here.

### 4.4.1 Representation as Unary and Binary Predicates

Predicates in knowledge graphs are binary (owl:ObjectProperties) and unary (owl:Class). We enumerate the predicates by $[d]$, the objects in the domain $\mathcal{D}$ by $[r]$, and extend the unary predicates to binaries by tensor product with $\epsilon_0\,[O_1]$. A Knowledge Graph on the set $\mathcal{D}$ of constants (owl:NamedIndividuals) is then the tensor

$$\mathrm{KG}|_{x_W}\,[L,O_0,O_1]:[d]\times[r]\times[r]\to\{0,1\}\,.$$

### 4.4.2 Representation as Ternary Predicate

It has been particularly convenient to represent a Knowledge Graph instead as a grounding of a single ternary predicate RDF. To this end, the predicates $g_k$ and another object rdftype are added to a domain $\mathcal{D}$, by extending the $r$ and the index interpretation function accordingly.

Following our notation we understand a Knowledge Graph as a grounding of the rdf triple relation RDF (being a formula of order 3) on a specific world $\mathrm{KG}|_{x_W}$ with individuals $\mathcal{D}$

We then construct a grounding tensor $\mathrm{RDF}|_{x_W}$ out of the world $\mathrm{KG}|_{x_W}\,[L,O_0,O_1]$ by

$$\mathrm{RDF}|_{x_W} : [r] \times [r] \times [r] \to \{0, 1\}$$

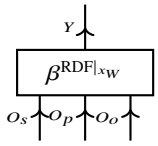
where

$$\mathrm{RDF}|_{x_W}\left[O_s = o_s, O_p = o_p, O_o = o_o\right] = \begin{cases} \mathrm{KG}|_{x_W}\left[L = o_s, O_0 = o_o, O_1 = 0\right] & \text{if} \quad o_p = I^{-1}(\text{rdftype}) \\ \mathrm{KG}|_{x_W}\left[L = o_p, O_0 = o_s, O_1 = o_o\right] & \text{if} \quad o_p = I^{-1}(g_k) \quad \text{for some} \quad k \\ 0 & \text{else} \end{cases} .$$

Slicing the tensor $\mathrm{RDF}|_{x_W}$ along the predicate axis retrieves specific information about roles and can be efficiently be performed on these formats. The role rdftype has a specific meaning, since it contains from a DL perspective classifications (memberships of named concepts). Further slicing the tensor along object axis therefore results in membership lists for specific classes (concepts). One can thus regard rdftype as a placeholder for unitary formulas in a space of binary formulas.

Exploiting the $\ell_0$-sparsity now leads to a so-called triple store, where $\mathrm{RDF}|_{x_W}$ is stored by a listing of those triples $o_s, o_p, o_o$ such that $\mathrm{RDF}|_{x_W}\left[O_s = o_s, O_p = o_p, O_o = o_o\right] = 1$ A recent implementation of a triple store exploiting these intuitions is TENTRIS, see [7]. In this work, such decompositions are generalized into more generic CP formats, see Chapter **??**. Approximations of grounding tensors by decompositions leads to embeddings of the individuals such as Tucker, ComplEx and RESCAL (see [59]).

For our purposes of evaluating logical formulas such as SPARQL queries we use the basis encoding of the groundings, which are depicted by




### 4.4.3 SPARQL Queries

The SPARQL query language is a syntax to express first-order logic formulas $q$ and intended to be evaluated given a Knowledge Graph. We here consider tensor network representations of the WHERE· block. Given a specific knowledge graph $\mathrm{RDF}|_{x_W}$, the execution of query is the interpretation $q|_{x_W}$, typical represented in a sparse basis CP format where each slice represents a solution mapping.

#### 4.4.3.1 Triple Patterns

Central to SPARQL queries are triple patterns, which we understand as slicings of the tensor $\mathrm{RDF}|_{x_W}$. To each so-called triple pattern we build a corresponding creation tensor. The triple pattern is then evaluated by contraction of the atom creating tensor with $\mathrm{RDF}|_{x_W}$.

Let us now provide examples of such pattern tensors. A unary triple patterns contains a single projection variable, typically related with the subject variable $O_s$ of $\mathrm{RDF}|_{x_W}$. The corresponding pattern tensor is then

$$\psi_{\langle Z,\text{rdftype},g_k\rangle}\left[O_s, O_p, O_o, Z\right] = \delta\left[O_s, Z\right] \otimes \epsilon_{I^{-1}(\text{rdftype})}\left[O_p\right] \otimes \epsilon_{I^{-1}(g_k)}\left[O_o\right] .$$

Binary triple patterns come with two projection variables, typically related with the subject and the object variables $O_s$ and $O_o$. The pattern tensor to the $k$-th predicate is then

$$\psi_{\langle Z_0,g_k,Z_1\rangle}\left[O_s, O_p, O_o, Z_0, Z_1\right] = \delta\left[O_s, Z_0\right] \otimes \epsilon_{I^{-1}(g_k)}\left[O_p\right] \otimes \delta\left[O_o, Z_1\right] .$$

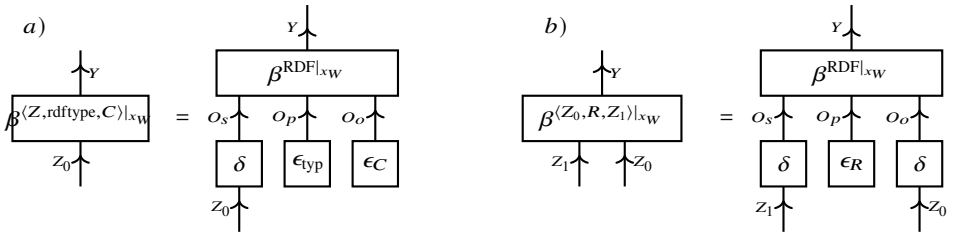



**Fig. 4.3** Triple patterns of SPARQL as tensor networks. a) Example of unary triple pattern $\langle Z, \text{rdftype}, C\rangle$ specifying whether an individual $I_{o_1}$ is a member of class $C$. b) Example of a binary triple pattern $\langle Z_0, R, Z_1\rangle$ specifying whether individuals $I_{o_1}$ and $I_{o_2}$ have a relation $R$. By $\epsilon_{\text{typ}}$, $\epsilon_C$, $\epsilon_R$ we denote the one-hot encodings of the enumeration of the resources $rdf:type$, $C$ and $R$.

Contraction with these pattern tensor evaluated the specific triple pattern, and outputs in a boolean tensor the indicator, which objects are members of a specific class (for unary patterns) or which pair of objects are related by a specific relation. Again, the output of such contractions is a subset encodings of the set of solutions (see Def. 7.1).

Examples of triple patterns, drawn in Figure 4.3 are

- Unary triple pattern with one variable, representing a formula with a single projection variable. For the example $\langle Z, \text{rdftype}, C\rangle$ see Figure 4.3a.

$$\psi_{\langle Z,\text{rdftype},g_k\rangle}\left[O_s, O_p, O_o, Z\right] = \delta\left[O_s, Z\right] \otimes \epsilon_{I^{-1}(\text{rdftype})}\left[O_p\right] \otimes \epsilon_{I^{-1}(C)}\left[O_o\right]$$

  If and only if the output slice is $\epsilon_1$, then the corresponding object encoded by the input indices is of class $C$.
- Binary triple pattern with two variables, representing a formula with two projection variables. For the example $\langle Z_0, R, Z_1\rangle$ see Figure 4.3b. If and only if the output slice is $\epsilon_1$, then the corresponding object tuple encoded by the input indices has a relation $R$.

The composition $\psi(\psi^T)$ of the matrification of the tensor $\psi$ is an orthogonal projection. That means that applying $\psi(\psi^T)$ is the same map as applying once.

#### 4.4.3.2 Basic Graph Patterns

Generic SPARQL queries are compositions of triple patterns by logical connectives. These triple patterns possibly share projection variables. Statements in SPARQL can be translated into propositional logic combining the triple patterns:

| SPARQL | **Propositional logic** | **Tensor Representation** |
|---|---|---|
| $\{f_1, f_2\}$ | $f_1 \wedge f_2$ | $\beta^{\wedge}\left[Y_{f_1\wedge f_2}, Y_{f_1}, Y_{f_2}\right]$ |
| UNION$\{f_1, f_2\}$ | $f_1 \vee f_2$ | $\beta^{\vee}\left[Y_{f_1\vee f_2}, Y_{f_1}, Y_{f_2}\right]$ |
| FILTER NOT EXISTS$\{f\}$ | $\neg f$ | $\beta^{\neg}\left[Y_{\neg f}, Y_f\right]$ |

If a SPARQL query consists of these keywords, we find a straight forward corresponding network of triple patterns and encoded logical connectives, by applying our findings of Sect. 4.2.4. To this end, we prepare for each appearing triple pattern the corresponding pattern tensor, and a copy of $\text{RDF}|_{x_W}$. Here we also copy the term variables $O_s, O_p$ and $O_o$, to ensure that each copy of $\text{RDF}|_{x_W}$ shares variables with a single pattern tensor. Projection variables are not copied, since we need to keep track of them shared among triple patterns. Then we prepare the basis encoding of logical connectives according to the hierarchy specified in the SPARQL query. Finally we add a $\epsilon_1$-vector to the final head variable representing the complete SPARQL query, to restrict the support to coordiantes corresponding with solution mappings. We then contract the resulting tensor network, leaving all projection variables open.

If a projection variable is not appearing in the SELECT statement in front of the WHERE$\{\cdot\}$-block, we simply exclude it from the open variables of the described contraction. Note that in that case, the coordinates contain solution counts, i.e. how many assignments to the dropped variable have been a 1 coordinate. We can drop this additional information simply by performing a coordinatewise transform with the greater zero indicator $\mathbb{I}_{>0}$.

Here we represented a SPARQL query $p$ consistent of multiple triple pattern by instantiating a head variables to each triple pattern. Alternatively, the more direct hybrid calculus developed in Sect. 7.4 can be applied and the additional

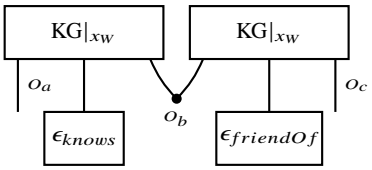
head variables avoided. This is especially compelling, when the WHERE{·}-block does not contain further keywords, i.e. it is the conjunction of all triple patterns. In that case, we avoid the instantiation of head variables (i.e. close the head variables separately by $\epsilon_1$-vectors) and represent the query by a contraction of all triple pattern tensors.

We further notice, that any propositional formula acting on the head variables of the triple patterns can be expressed by a hierarchical combination of the key words in the above table. To find the expression, one can transform a given formula into its conjunctive or disjunctive normal form and apply the statements according to the apperaing operations $\wedge$, $\vee$ and $\neg$.

*Example 4.3 (SPARQL query in a tensor network)* Let us consider the example query:

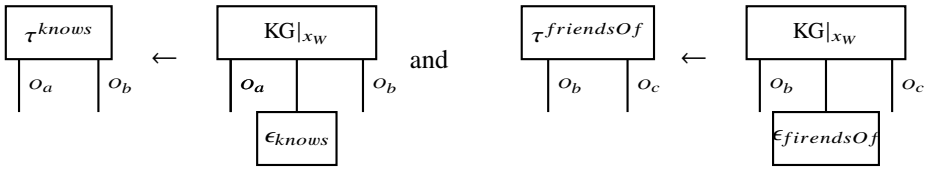
```
PREFIX foaf: <http://xmlns.com>
SELECT ?a ?c
WHERE{
    ?a foaf:knows ?b;
    ?b foaf:friendOf ?c;
}
```

As a tensor network the query is represented as:




We can now apply sparsification techniques developed in Sect. 4.3 to provide an efficient storage.

- Local contraction with the basis vectors:




- Message passing of the support:

$$\tilde{\tau}^{knows}\left[O_a, O_b\right] \leftarrow \mathbb{I}_{\neq 0} \circ \left\langle \tau^{knows}\left[O_a, O_b\right], \left\langle \tau^{friendsOf}\left[O_b, O_c\right]\right\rangle_{[O_b]}\right\rangle_{[O_a, O_b]}$$

and

$$\tilde{\tau}^{friendsOf}\left[O_b, O_c\right] \leftarrow \mathbb{I}_{\neq 0} \circ \left\langle \tau^{friendsOf}\left[O_b, O_c\right], \left\langle \tau^{knows}\left[O_a, O_b\right]\right\rangle_{[O_b]}\right\rangle_{[O_b, O_c]}$$

The resulting sparsified storage of the query is then the tensor network

$$\{\tilde{\tau}^{knows}\left[O_a, O_b\right], \tilde{\tau}^{friendsOf}\left[O_b, O_c\right]\}.$$

For any locally supported index, we find an extension to an global index which is supported by the tensor network (see Chapter 8). For example, if for a tuple $o_a, o_b$ we have $\tilde{\tau}^{knows}\left[O_a = o_a, O_b = o_b\right] = 1$ then there is $o_c$ with $\tilde{\tau}^{friendsOf}\left[O_b = o_b, O_c = o_c\right] = 1$, which means that $(o_a, o_b, o_c)$ is a solution of the query.

## 4.5 Semantic structure

We now investigate the semantic structure of a first-order logic theory, when restricting to database semantics. This involves the representation of collections of worlds, where each has a substitution structure as discussed above.

### 4.5.1 Database semantics

We want to enumerate all models in a first-order logic theory. However, without restricting assumptions, there are infinitely many worlds. Therefore we apply database semantics, where the objects appearing in the models are fixed sets. We can then enumerated all possible worlds by semantic variables $X$.

### 4.5.2 World Enumerating Variables

Given database semantics, we have a finite set of worlds, which we enumerate by tuples of variables $X$.

**Definition 4.5 (World enumerating variables)** Given a first-order logic theory with constant symbols $\mathcal{D}$, function symbols $\{v_l : l \in [r]\}$ and predicate symbols $\{g_k : k \in [d]\}$, we enumerate the world under database semantics by a tuple

$$X_W = \left( \bigtimes_{\tilde{k} \in [d]} \bigtimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} X_{\tilde{k}, o_{\tilde{k}}} \right) \times \left( \bigtimes_{k \in [d]} \bigtimes_{o_k \in \in [r]^{n_k}} X_{k, o_k} \right)$$

of boolean variables. In the world indexed by $x_W$, the grounding tensor of the function $v_l$ is

$$\beta^{v_l|_{x_W}} \left[ O_l, O_{[r_l]} \right] = \sum_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1} : o_{\tilde{k}}=1} \epsilon_{o_{\tilde{k}}} \left[ O_{\tilde{k}} \right]$$

and of the predicate $g_k$

$$g_k|_{x_W} \left[ O_{[r_k]} \right] = \sum_{o_{\tilde{k}} \in [r]^{n_k} : o_k=1} \epsilon_{o_k} \left[ O_k \right] .$$

We further have the restriction that each basis encoded function is a directed tensor. To restrict to those worlds, where this is true, we further introduce the validation base measure $\nu \left[ X_W \right]$ with coordinates

$$\nu \left[ X_W = x_W \right] = \begin{cases} 1 & \text{if} \quad \forall l \in [r] : \sum_{o_l \in [r]^{n_{\tilde{k}}+1}} = 1 \\ 0 & \text{else} \end{cases} .$$

We notice that there are

$$\left( \sum_{k \in [d]} r^{n_k} \right) + \left( \sum_{\tilde{k} \in [d]} r^{n_{\tilde{k}}} \right)$$

many semantic variables, and the number of states is exponential in that number.

### 4.5.3 Representation of Terms and Formulas

Combining its substitution structure and semantic structure we represent a first-order logic term $v$ with arity $r_v$ as a tensor

$$\beta^v \left[ O_v, O_{[r_v]}, X_W \right] = \sum_{x_W : \nu[X_W = x_W]=1} \beta^{v|_{x_W}} \left[ O_v, O_{[r_v]} \right] \otimes \epsilon_{x_W} \left[ X_W \right] .$$

and a formula as the tensor

$$q\left[O_q, X_W\right] = \sum_{x_W \,:\, \nu[X_W = x_W]=1} q|_{x_W}\left[O_q\right] \otimes \epsilon_{x_W}\left[X_W\right] .$$

From Def. 4.5 the representation of predicate and function symbols are directly derived from the $X_W$, that is

$$g_k\left[O_{[r_k]}, X_W = x_W\right] = \sum_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1} \,:\, o_{\tilde{k}}=1} \epsilon_{o_{\tilde{k}}}\left[O_{\tilde{k}}\right]$$

and

$$\beta^{v_l}\left[O_l, O_{[r_l]}, X_W = x_W\right] = \sum_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1} \,:\, o_{\tilde{k}}=1} \epsilon_{o_{\tilde{k}}}\left[O_{\tilde{k}}\right]$$

### 4.5.4 Case of Propositional Logic

Propositional logic as discussed in (see Chapter 2) is the special case of first-order logic with empty sets of constant and function symbols, and $n_k = 0$ for the predicate symbols. With the convention $[]^0 = [2]$ the worlds are enumerated by the tuple

$$X_W = \bigtimes_{k \in [d]} X_k$$

which we have in previous chapters abbreviated by $x_{[d]}$.

To represent logical formulas as sets of possible worlds, and distributions of worlds, we applied in Part **??** one-hot encodings of possible worlds. For the case of propositional logic, this is

$$\epsilon_{x_W}\left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k}\left[X_k\right] .$$

### 4.5.5 One-hot Encoding of Worlds

Let us now generalize the one-hot encodings of propositional logic worlds to worlds of first-order logic. To encode the boolean tensors $x_W$ describing first-order logics as basis elements of a tensor space, we take the one-hot encoding

$$\epsilon : \left(\bigtimes_{\tilde{k} \in [d]} \bigtimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} [2]\right) \times \left(\bigtimes_{k \in [d]} \bigtimes_{o_k \in [r]^{n_k}} [2]\right) \to \left(\bigotimes_{\tilde{k} \in [d]} \bigotimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} \mathbb{R}^2\right) \otimes \left(\bigotimes_{k \in [d]} \bigotimes_{o_k \in [r]^{n_k}} \mathbb{R}^2\right)$$

defined by

$$\epsilon_{x_W}\left[X_W\right] = \left(\bigotimes_{\tilde{k} \in [d]} \bigotimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} \epsilon_{x_{\tilde{k},o_{\tilde{k}}}}\left[X_{\tilde{k},o_{\tilde{k}}}\right]\right) \otimes \left(\bigotimes_{k \in [d]} \bigotimes_{o_k \in [r]^{n_k}} \epsilon_{x_{k,o_k}}\left[X_{k,o_k}\right]\right)$$

This is a tensor of order $d \cdot r^n$, in a tensor space of dimension $2^{(d \cdot r^n)}$. Storage of such tensors in naive formats would not be possible. However, the basis CP format discussed in Chapter **??** still provides storage with demand linear in the order $d \cdot r^n$.

# Chapter 5
# Probability Theory

**Abstract** In this chapter, we will establish relations between the formalism of tensor networks and basic concepts of probability theory. We will first introduce joint distributions by tensors and connect their marginalizations and conditionings to basic tensor operations. Sparse representations of these tensors by tensor networks will be motivated based on independence properties. These mechanisms lead towards graphical models, for which we provide tensor network representations.

Topics:

- Chapter I.2 except for exponential families (later in book II.0)
- Sufficient statistics from sketches

## 5.1 Tensor Formalism in Probability Theory

Let us relate the main terminology of probability theory with the tensor network formalism. As such, any joint distribution of finitely valued variables is a tensor. Further, the operations of marginalization and conditioning of distributions are equal to contractions and normalizations of tensors.

### 5.1.1 Joint Probability Distributions

Let there be uncertainties of the assignments $x_k$ to the categorical variables $X_k$ of a system. In this scenario we understand each $X_k$ as random variables, which together have a joint distribution defined by the uncertainties of the state assignments. The uncertainty of a state $x_{[d]}$ is then quantified by probabilities

$$\mathbb{P}\left[X_{[d]} = x_{[d]}\right] \in [0, 1] \,.$$

Joint probability distributions are therefore naturally represented by tensors, whose coordinates store the probabilities to each state. These tensors have additional properties as we state in the next definition.

**Definition 5.1 (Joint Probability Distribution)** Let there be for each $k \in [d]$ a categorical variable $X_k$ taking values in $[m_k]$. A joint probability distribution of these categorical variables is a function

$$\mathbb{P}\left[X_{[d]}\right] \,:\, \bigtimes_{k\in[d]} [m_k] \to \mathbb{R}$$

which is non-negative, that is for any $x_{[d]} \in \bigtimes_{k\in[d]} [m_k]$

$$\mathbb{P}\left[X_{[d]} = x_{[d]}\right] \geq 0 \,,$$

and which is normalized, that is

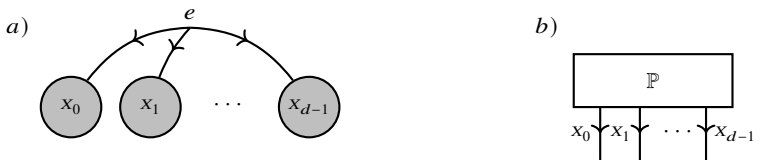



**Fig. 5.1** Probability distributions of variables $X_0, \ldots, X_{d-1}$, sketched a) by a directed edge $e$ with all variables outgoing, which is decorated b) by a directed tensor $\mathbb{P}\left[X_{[d]}\right]$.

$$\left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 1\,.$$

In a slight abuse of notation, we directly applied the coordinate encoding scheme (see Def. **??** and for more details Chapter **??**) to represent a probability distribution as a tensor. Along this scheme, any probability distribution is a weighted sum of one-hot encodings (see Lem. **??**) by

$$\mathbb{P}\left[X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right]\,,$$

where we understand $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$ as the probability of the categorical variables to take the state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$. The normalization condition $1 = \left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$ has a more convenient equivalence by the coordinate sum

$$1 = \left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right]\,,$$

and thus ensures that all probabilities sum to 1, which is necessary for the probabilistic interpretation. While the assumptions of non-negative coordinates in Def. 5.1 reflects the first probability axiom of Kolmogorov, the assumption of normalization, that is a contraction 1, implements the second axiom (see for example [22]). Since probability distributions contract to 1, they are directed (see Def. **??**) with all distributed variables outgoing and no incoming variables (see Figure 5.1).

### 5.1.2 Base Measures

From a measure theoretic perspective, probabilities are measurable functions called probability densities, which integrals are 1 (see for example [22]). In our case of finite dimensional state spaces of factored systems, we implicitly used the trivial tensor $\mathbb{I}\left[X_{[d]}\right]$ as a base measure, which measures subsets of states by their cardinality and is therefore referred to as state counting base measure. The distributions $\mathbb{P}\left[X_{[d]}\right]$ can then be understood as probability densities with respect to this state counting base measure. We in this work will also consider more general base measures $\nu\left[X_{[d]}\right]$, which we restrict to be boolean, that is $\nu\left[X_{[d]} = x_{[d]}\right] \in \{0,1\}$ for all states $x_{[d]}$. When understanding $\mathbb{P}\left[X_{[d]}\right]$ as a probability density with respect to $\nu\left[X_{[d]}\right]$, any probabilistic interpretation will be through the contraction $\left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}$ and the normalization condition reads as

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 1\,.$$

Since we restrict to boolean base measures, the contraction effectively manipulates the tensor $\mathbb{P}$ by setting the coordinates $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$ to zero, when $\nu\left[X_{[d]} = x_{[d]}\right] = 0$. Therefore, multiple tensors $\mathbb{P}$ will have the same probabilistic interpretation, when $\nu\left[X_{[d]}\right] \neq \mathbb{I}\left[X_{[d]}\right]$. To avoid this ambiguity, we introduce the notation of representability with respect to a base measure $\nu$, by demanding that such coordinates are zero.

**Definition 5.2** We say that a probability distribution $\mathbb{P}$ is representable with respect to a boolean base measure $\nu$, if for all $x_{[d]}$ with $\nu\left[X_{[d]} = x_{[d]}\right] = 0$ we have $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] = 0$. We denote the set of by $\nu$ representable distributions by $\Lambda^{\delta,\mathrm{MAX},\nu}$.

When a probability distribution $\mathbb{P}$ is representable with respect to a boolean base measure $\nu$, we have the invariance

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

and can therefore safely ignore the base measures. This enables the characterization of by $\nu$ representable distributions by

$$\Lambda^{\delta,\mathrm{MAX},\nu} = \Big\{\mathbb{P}\left[X_{[d]}\right] : \Big(\forall x_{[d]} \in \bigtimes_{k\in[d]} [m_k] : \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \geq 0\Big), \\ \left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = \mathbb{P}\left[X_{[d]}\right]\Big\}.$$

Starting with Chapter 2 we will further investigate boolean tensors and relate them with propositional formulas. In Chapter **??** we will connect the representation and positivity with respect to boolean base measures with the formalism of entailment. The notation $\Lambda^{\delta,\mathrm{MAX},\nu}$ of by $\nu$ representable distributions will later in this chapter relate with distributions computable given a statistic.

We now investigate, which base measures $\nu$ can be chosen for a probability distribution $\mathbb{P}$, such that $\mathbb{P}$ is representable by $\nu$. Here we want to find a $\nu$, which is in a sense to be defined minimal amount the base measures, such that $\mathbb{P}$ is representable with respect to them. For this minimality criterion we will develop in Chapter **??** orders based on entailment and show the minimality in Thm. **??**. Here, we just introduce the minimality criterion as positivity of a distribution with respect to a base measure.

**Definition 5.3** We say that a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ is positive with respect to a boolean base measure $\nu\left[X_{[d]}\right]$, if the distribution is representable by $\nu$ (i.e. $\langle \mathbb{P}, \nu\rangle_{[\varnothing]} = 1$) and for all $x_{[d]}$ with $\nu\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] = 1$ we have $\mathbb{P}\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] > 0$. We further say that a distribution is positive, if it is positive with respect to the trivial base measure $\mathbb{I}\left[X_{[d]}\right]$.

### 5.1.3 Marginal Distribution

Contractions of probability distributions are related to marginalizations as we introduce next.

**Definition 5.4 (Marginal Probability)** Given a distribution $\mathbb{P}\left[X_0, X_1\right]$ of the categorical variables $X_0$ and $X_1$ the marginal distribution of the categorical variable $X_0$ is the tensor

$$\mathbb{P}\left[X_0\right] \in \mathbb{R}^{m_0}$$

defined by the contraction

$$\mathbb{P}\left[X_0\right] = \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[X_0]} .$$

To connect with a more standard defining equation of marginal distributions, let us notice that for any $x_0 \in [m_0]$

$$\mathbb{P}\left[X_0 = x_0\right] = \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[X_0 = x_0]} = \sum_{x_1 \in [m_1]} \mathbb{P}\left[X_0 = x_0, X_1 = x_1\right] .$$

Thus, each coordinate of the marginal distribution is the sum of the joint probability of compatible states. We say that the variable $X_1$ is marginalized out, when building the marginal distribution $\mathbb{P}\left[X_0\right]$ of $X_0$. Let us now justify this terminology and show, that any marginal distribution is a probability distribution as introduced in Def. 5.1.

**Theorem 5.1** *Any marginal distribution is a probability distribution.*

*Proof.* We further have that any marginal distribution is normalized, since by the commutativity of contractions (see for more details Thm. **??** in Part **??**)

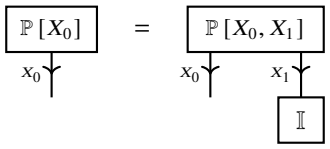
$$\langle \mathbb{P}\left[X_0\right]\rangle_{[\varnothing]} = \left\langle \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[X_0]}\right\rangle_{[\varnothing]} = \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[\varnothing]} = 1 .$$




**Fig. 5.2** Depiction of conditional probability distributions a) by an edge with the incoming variable $X_1$ and the outgoing variable $X_1$, which is decorated by b) the directed tensor $\mathbb{P}[X_0|X_1]$.

Further any coordinate is non-negative, since it is a sum of non-negative coordinates. It follows from Def. 5.1, that any marginal distribution is a probability distribution. □

In a tensor network diagram we often represent variables $X_1$ not appearing as open variables of a contraction as contracted with the trivial tensor $\mathbb{I}[X_1]$. Following this notation, we depict the marginal distribution in Def. 5.4 by




Since we have shown, that marginal distributions are themself probability distributions, they inherit the outgoing directionality in tensor network diagrams.

We notice that Def. 5.4 generalizes to marginalizations of arbitrary sets of variables, when having a distribution $\mathbb{P}\left[X_{[d]}\right]$ of an arbitrary number of categorical variables. It suffices for this to interpret $X_0$ and $X_1$ as collections of variables, which indices take the states of the respective factored systems.

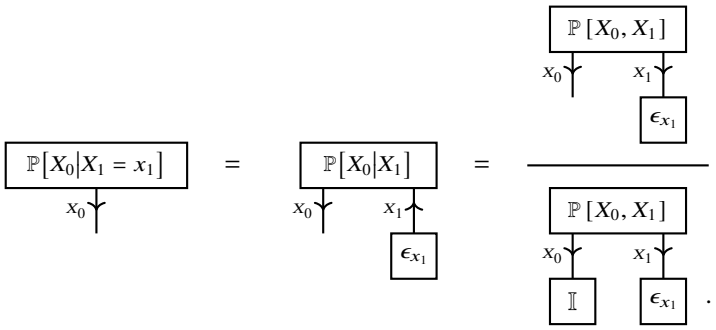
### 5.1.4 Conditional Probabilities

Normalizations of probability distributions result in conditional distributions as we define next.

**Definition 5.5 (Conditional Probability)** Let $\mathbb{P}[X_0, X_1]$ be a distribution of the categorical variables $X_0$ and $X_1$, such that $\mathbb{P}[X_0, X_1]$ is normalizable on $\{X_1\}$. Then the distribution of $X_0$ conditioned on $X_1$ is defined by

$$\mathbb{P}[X_0|X_1] = \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0|X_1]} \ .$$

Since conditional probabilities are normalizations of probability tensors, they are directed and therefore depicted by directed hyperedges (see Figure 5.2). For any $x_1 \in [m_1]$ we depict the slice $\mathbb{P}[X_0|X_1 = x_1]$ defined by a normalization operation as




As we have done before for marginal distribution, we relate Def. 5.5 with a more convenient coordinatewise definition of conditional probabilities. For any indices $x_0 \in [m_0]$ and $x_1 \in [m_1]$ we have

$$\mathbb{P}[X_0 = x_0|X_1 = x_1] = \frac{\mathbb{P}[X_0 = x_0, X_1 = x_1]}{\langle \mathbb{P}[X_0, X_1] \rangle_{[X_1 = x_1]}} = \frac{\mathbb{P}[X_0 = x_0, X_1 = x_1]}{\sum_{x_0 \in [m_0]} \mathbb{P}[X_0 = x_0, X_1 = x_1]} \ .$$

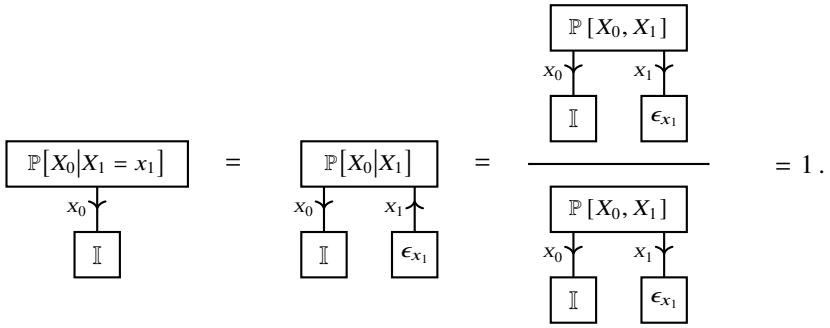
The distribution of $X_0$ conditioned on $X_1$ is the normalized collection of slice of the probability distribution $\mathbb{P}[X_0, X_1]$. Each slice of the conditioned distribution with respect to incoming variables is a probability distribution itself, as we show next.

**Theorem 5.2** *For any $x_1 \in [m_1]$ the tensor $\mathbb{P}\big[X_0 \big| X_1 = x_1\big]$ is a probability tensor.*

*Proof.* As a normalization of a non-negative tensor, the conditional probability $\mathbb{P}\big[X_0 \big| X_1 = x_1\big]$ and any of its slices is also a non-negative tensor. Further, we have for any $x_1 \in [m_1]$

$$\begin{aligned}\Big\langle \mathbb{P}\big[X_0 \big| X_1 = x_1\big] \Big\rangle_{[\varnothing]} &= \sum_{x_0 \in [m_0]} \mathbb{P}\big[X_0 = x_0 \big| X_1 = x_1\big] \\ &= \frac{\sum_{x_0 \in [m_0]} \mathbb{P}[X_0 = x_0, X_1 = x_1]}{\sum_{x_0 \in [m_0]} \mathbb{P}[X_0 = x_0, X_1 = x_1]} \\ &= 1\,,\end{aligned}$$

and therefore each slice is normalized. We can visualize this calculation exploiting our diagramatic notation as




Since for any $x_1 \in [m_1]$ the slice $\mathbb{P}\big[X_0 \big| X_1 = x_1\big]$ is non-negative and contracts to 1, we conclude that it is a probability distribution. □

We further show, that exactly the directed tensors with non-negative coordinates are conditional probability tensors.

**Theorem 5.3** *Any tensor with non-negative coordinates is a conditional distribution, if and only if it is directed with the conditioned variables incoming and the other outgoing.*

*Proof.* "⇒": By Thm. 5.2 a conditional probability tensor $\mathbb{P}\big[X_0 \big| X_1\big]$ is the normalization of a tensor and by Thm. **??** a directed tensor. Since probability tensors have only non-negative coordinates, their contractions with one-hot encodings also have only non-negative coordinates and also their normalizations.

"⇐": Conversely, let $\tau[X_{\mathcal{V}}]$ be a directed tensor with $\mathcal{V}^{\text{in}}$ incoming and $\mathcal{V}^{\text{out}}$ outgoing and non-negative coordinates. Then

$$\mathbb{P}[X_{\mathcal{V}}] = \frac{1}{\prod_{v \in \mathcal{V}^{\text{in}}} m_v} \cdot \tau[X_{\mathcal{V}}] \tag{5.1}$$

is a probability tensor, since

$$\sum_{x_{\mathcal{V}^{\text{in}}}} \sum_{x_{\mathcal{V}^{\text{out}}}} \mathbb{P}[X_{\mathcal{V}} = x_{\mathcal{V}}] = \sum_{x_{\mathcal{V}^{\text{in}}}} \sum_{x_{\mathcal{V}^{\text{out}}}} \frac{1}{\prod_{v \in \mathcal{V}^{\text{in}}} m_v} \cdot \tau[X_{\mathcal{V}} = x_{\mathcal{V}}] = \sum_{x_{\mathcal{V}^{\text{in}}}} \frac{1}{\prod_{v \in \mathcal{V}^{\text{in}}} m_v} = 1\,.$$

The conditional probability $\mathbb{P}\big[X_{\mathcal{V}^{\text{out}}} \big| X_{\mathcal{V}^{\text{in}}}\big]$ coincides with $\tau$, since

$$\begin{aligned}\mathbb{P}\big[X_{\mathcal{V}^{\text{out}}} \big| X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\big] &= \frac{\mathbb{P}\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]}{\sum_{x_{\mathcal{V}^{\text{out}}}} \mathbb{P}\left[X_{\mathcal{V}^{\text{out}}} = x_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]} \\ &= \frac{\tau\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]}{\sum_{x_{\mathcal{V}^{\text{out}}}} \tau\left[X_{\mathcal{V}^{\text{out}}} = x_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]} = \tau\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]\,,\end{aligned}$$

where in the last equation we used that the denominator is by definition trivial since $\tau$ is normalized. □

Thm. 5.3 specifies a broad class of tensors to represent conditional probabilities. In combination with Thm. 7.3, which states that basis encodings are directed, we get that any basis encoding of a function is a conditional probability tensor.

### 5.1.5 Bayes Theorem and the Chain Rule

So far, we have connected concepts of probability theory such as marginal and conditional probabilities with contractions and normalizations of tensors. We will now proceed to show that basic theorems of probability theory translate into more general contraction equations.

**Theorem 5.4 (Bayes Theorem)** *For any probability distribution $\mathbb{P}[X_0, X_1]$ with positive $\mathbb{P}[X_1]$ we have*

$$\mathbb{P}[X_0, X_1] = \left\langle \mathbb{P}\left[X_0 \middle| X_1\right], \mathbb{P}[X_1] \right\rangle_{[X_0, X_1]} .$$

*Proof.* This theorem follows from the more generic contraction equation Thm. **??** to be shown in Chapter **??**. We note that by positivity of $\mathbb{P}[X_1]$, the tensor network $\mathbb{P}$ is normalizable with respect to $X_1$. Thm. **??** therefore implies choosing $\mathcal{V} = \{0, 1\}$, $\mathcal{V}^{\text{in}} = \{1\}$ and $\mathcal{V}^{\text{out}} = \{0\}$, that For our tensor

$$\begin{aligned}\mathbb{P}[X_0, X_1] &= \left\langle \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0|X_1]}, \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]} \right\rangle_{[X_0, X_1]} \\ &= \left\langle \mathbb{P}\left[X_0 \middle| X_1\right], \mathbb{P}[X_1] \right\rangle_{[X_0, X_1]} .\square\end{aligned}$$

Following the insight of the Bayes Thm. 5.4, probability distributions of arbitrary numbers of variables can be decomposed as a contraction of conditional probabilities, as we show in the next theorem.

**Theorem 5.5 (Chain Rule)** *For any probability distribution $\mathbb{P}\left[X_{[d]}\right]$ we have*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}[X_0]\} \cup \left\{ \mathbb{P}\left[X_k \middle| X_0, \ldots, X_{k-1}\right] \,:\, k \in [d],\, k \geq 1 \right\} \right\rangle_{\left[X_{[d]}\right]} ,$$

*provided that all conditional probability distributions exist.*

*Proof.* The claim can be derived by an iterative application of the Bayes Thm. 5.4 theorem. We will proof this statement in more generality in Chapter **??** as Thm. **??**, deducing it from the generalization of the Bayes Thm. 5.4 by Thm. **??**. The claim here then follows from Thm. **??** using $\mathcal{V} = [d]$ and $\tau[X_{\mathcal{V}}] = \mathbb{P}\left[X_{[d]}\right]$, since

$$\begin{aligned}\mathbb{P}\left[X_{[d]}\right] &= \left\langle \{\mathbb{P}[X_0]\} \cup \{ \langle \mathbb{P}\left[X_{[d]}\right] \rangle_{[X_k|X_0,\ldots,X_{k-1}]} \,:\, k \in [d],\, k \geq 1 \} \right\rangle_{\left[X_{[d]}\right]} \\ &= \left\langle \{\mathbb{P}[X_0]\} \cup \left\{ \mathbb{P}\left[X_k \middle| X_0, \ldots, X_{k-1}\right] \,:\, k \in [d],\, k \geq 1 \right\} \right\rangle_{\left[X_{[d]}\right]} .\square\end{aligned}$$

We observe, that the chain rule provides a generic decomposition scheme of probability distributions into conditional distributions. The conditional distribution to $k = d - 1$, which appears in the chain decomposition, is in the same tensor space as the decomposed distribution $\mathbb{P}\left[X_{[d]}\right]$. To achieve our main goal of tensor network decompositions, which is an efficient storage format of the decomposed tensor, we need to further sparsify the appearing conditional probabilities (to be more precise, we aim at basis+ CP decompositions, to be introduced in Chapter **??**). These simplification require additional assumptions on independencies in the distribution, which we will introduce in the next section.

## 5.2 Independence as a Decomposition Mechanism

The number of coordinates in a tensor representation of probability distributions is the product

$$\prod_{k\in[d]} m_{X_k} ,$$

and therefore scales exponentially in the number of coordinates. To find efficient representation schemes of probability distributions by tensor networks, we need to exploit additional properties of the distribution. In this section, we introduce the independence and the computation mechanisms towards tensor network decompositions of probability distributions.

Independence leads to severe sparsifications of conditional probabilities and is therefore the key assumption to gain sparse decompositions of probability distributions. Before showing such decomposition schemes, we first provide a coordinatewise definition of independent variables.

**Definition 5.6 (Independence)** We say that $X_0$ is independent of $X_1$ with respect to a distribution $\mathbb{P}[X_0, X_1]$, if for any values $x_0 \in [m_0]$ and $x_1 \in [m_1]$ the distribution satisfies

$$\mathbb{P}[X_0 = x_0, X_1 = x_1] = \mathbb{P}[X_0 = x_0] \cdot \mathbb{P}[X_1 = x_1] .$$

In this case we denote $(X_0 \perp X_1)$.

We state next an equivalent independence criterion based on a contraction equation of probability distributions.

**Theorem 5.6 (Independence Criterion as a Contraction Equation)** *The variable $X_0$ is independent from $X_1$ with respect to a probability distribution $\mathbb{P}[X_0, X_1]$, if and only if*

$$\mathbb{P}[X_0, X_1] = \left\langle \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0]} , \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]} \right\rangle_{[X_0, X_1]} .$$

*Proof.* By Thm. 5.1 we know that marginal probabilities are equivalent to contracted probability distributions, i.e. $\mathbb{P}[X_0] = \langle \{\mathbb{P}\} \rangle_{[X_0]}$. By orthogonality of one-hot encodings we have that

$$\forall x_0 \in [m_0], x_1 \in [m_1] : \mathbb{P}[X_0 = x_0, X_1 = x_1] = \mathbb{P}[X_0 = x_0] \cdot \mathbb{P}[X_1 = x_1]$$

is equivalent to

$$\begin{aligned} &\sum_{x_0\in[m_0]} \sum_{x_1\in[m_1]} \mathbb{P}[X_0 = x_0, X_1 = x_1] \cdot \epsilon_{x_0}[X_0]\, \epsilon_{x_1}[X_1] \\ &\quad = \sum_{x_0\in[m_0]} \sum_{x_1\in[m_1]} \mathbb{P}[X_0 = x_0] \cdot \mathbb{P}[X_1 = x_1] \cdot \epsilon_{x_0}[X_0]\, \epsilon_{x_1}[X_1] . \end{aligned}$$

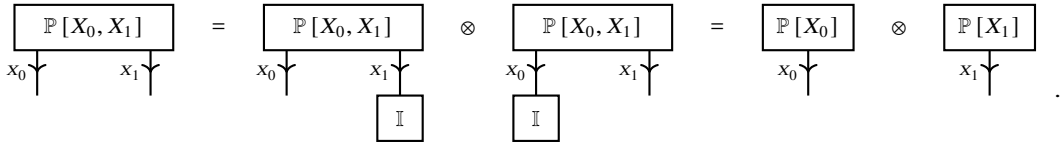
We reorder the summations and arrive at

$$\begin{aligned} &\sum_{x_0\in[m_0]} \sum_{x_1\in[m_1]} \mathbb{P}[X_0 = x_0, X_1 = x_1] \cdot \epsilon_{x_0,x_1}[X_0, X_1] \\ &\quad = \left( \sum_{x_0\in[m_0]} \mathbb{P}[X_0 = x_0]\, \epsilon_{x_0}[X_0] \right) \cdot \left( \sum_{x_1\in[m_1]} \mathbb{P}[X_1 = x_1] \cdot \epsilon_{x_1}[X_1] \right) \end{aligned}$$

which is by Lem. **??** equal to the claim

$$\mathbb{P}[X_0, X_1] = \left\langle \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0]} , \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]} \right\rangle_{[X_0, X_1]} . \square$$

Two jointly distributed variables are by Thm. 5.6 independent, if and only if their joint distribution $\mathbb{P}[X_0, X_1]$ is the tensor product of marginal probabilities. Using tensor network diagrams we depict this property by




Let us notice, that the assumption of independence reduces the degrees of freedom from $m_0 \cdot m_1 - 1$ to $(m_0 - 1) + (m_1 - 1)$. The decomposition into marginal distributions furthermore exploits this reduced freedom and provides an efficient

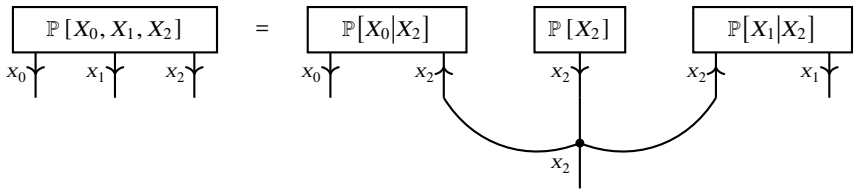



**Fig. 5.3** Diagrammatic visualization of the contraction equation in Cor. 5.1. Conditional independence of $X_0$ and $X_1$ given $X_2$ holds if the contraction on the right side is equal to the probability tensor on the left side.

storage. Having a joint distribution of multiple variables, which disjoint subsets are independent, we can iteratively apply the decomposition scheme. As a result, the degrees of freedom scaling exponential in the number of distributed variables would be reduced to a linear scaling, by the assumption of independence.

Independence is, as we observed, a strong assumption, which is often too restrictive. It is furthermore an undesired property, when in a supervised learning scenario a target variable has to be predicted based on known feature variables. Conditional independence instead is a less demanding assumption, which still implies efficient tensor network decompositions schemes. We introduce conditional independence as independence of variables with respect to conditional distributions.

**Definition 5.7 (Conditional Independence)** Given a joint distribution of variables $X_0$, $X_1$ and $X_2$, such that $\mathbb{P}[X_2]$ is positive. We say that $X_0$ is independent of $X_1$ conditioned on $X_2$ if for any states $x_0 \in [m_0], x_1 \in [m_1]$ and $x_2 \in [m_2]$

$$\mathbb{P}\big[X_0 = x_0, X_1 = x_1 \big| X_2 = x_2\big] = \mathbb{P}\big[X_0 = x_0 \big| X_2 = x_2\big] \cdot \mathbb{P}\big[X_1 = x_1 \big| X_2 = x_2\big] .$$

In this case we denote $(X_0 \perp X_1) \mid X_2$.

Conditional independence stated in Def. 5.7 has a close connection with independence stated in Def. 5.6. To be more precise, $X_0$ is independent of $X_1$ conditioned on $X_2$, if and only if $X_0$ is independent of $X_1$ with respect to any slice $\mathbb{P}\big[X_0, X_1 \big| X_2 = x_2\big]$ of the conditional distribution $\mathbb{P}\big[X_0, X_1 \big| X_2\big]$. Analogously to Thm. 5.6 for independence, we further find a decomposition criterion for conditional independence. Since conditional independence can be regarded as a property of conditional probabilities, this decomposition criterion also involves conditional probabilities.

**Theorem 5.7 (Conditional Independence as a Contraction Equation)** *Given a distribution $\mathbb{P}$ of variables $X_0$, $X_1$ and $X_2$, the variable $X_0$ is independent of $X_1$ conditioned on $X_2$, if and only if the equation*

$$\mathbb{P}\big[X_0, X_1 \big| X_2\big] = \big\langle \mathbb{P}\big[X_0 \big| X_2\big], \mathbb{P}\big[X_1 \big| X_2\big] \big\rangle_{[X_0, X_1, X_2]}$$

*holds.*

*Proof.* With the same argumentation as in the proof of Thm. 5.6, we notice that the contraction equation holds, if and only if for any $x_0 \in [m_0], x_1 \in [m_1]$ and $x_2 \in [m_2]$

$$\mathbb{P}\big[X_0 = x_0, X_1 = x_1 \big| X_2 = x_2\big] = \mathbb{P}\big[X_0 = x_0 \big| X_2 = x_2\big] \cdot \mathbb{P}\big[X_1 = x_1 \big| X_2 = x_2\big] .$$

This is equivalent to conditional independence by Def. 5.7. □

We can further exploit conditional independence to find tensor network decompositions of probabilities, as we show as the next corollary.

**Corollary 5.1** *If and only if $X_0$ is independent of $X_1$ conditioned on $X_2$ the probability distribution $\mathbb{P}$ satisfies (see Figure 5.3)*

$$\mathbb{P}[X_0, X_1, X_2] = \big\langle \mathbb{P}\big[X_0 \big| X_2\big], \mathbb{P}\big[X_1 \big| X_2\big], \mathbb{P}[X_2] \big\rangle_{[X_0, X_1, X_2]} .$$

*Proof.* With the Bayes Thm. 5.4 it holds that

$$\mathbb{P}[X_0, X_1, X_2] = \big\langle \mathbb{P}\big[X_0, X_1 \big| X_2\big], \mathbb{P}[X_2] \big\rangle_{[X_0, X_1, X_2]} .$$

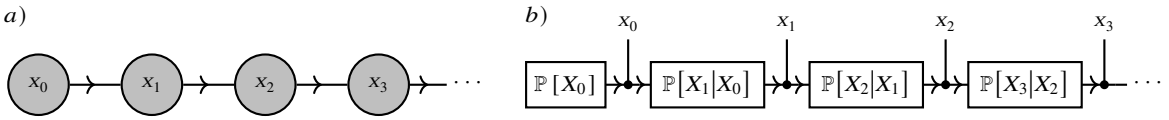



**Fig. 5.4** Depiction of a Markov Chain Decomposition by a a) hypergraph with the nodes $\mathcal{V} = [d]$ and edges $\mathcal{E} = \big\{\{0\} \cup \{\{k, k+1\} : k \in [d],\ k > 1\}\big\}$ and b) a decorating Tensor Network representing the sparsified conditional probabilities.

Decomposing the first tensor in the contraction, Thm. 5.7 implies, that $X_0$ is independent of $X_1$ conditioned on $X_2$, if and only if

$$\mathbb{P}[X_0, X_1, X_2] = \big\langle \mathbb{P}\big[X_0\big|X_2\big], \mathbb{P}\big[X_1\big|X_2\big], \mathbb{P}[X_2]\big\rangle_{[X_0,X_1,X_2]} \ .\square$$

Let us now recall our motivation of the study of conditional independence, namely to find sparsifications of conditional probabilities as those appearing in chain decompositions Thm. 5.5. As we state as the next theorem, such sparsifications follow from conditional independence.

**Theorem 5.8** *Whenever $X_0$ is independent of $X_1$ given $X_2$, we have for any $x_1 \in [m_1]$*

$$\mathbb{P}\big[X_0\big|X_1 = x_1, X_2\big] = \mathbb{P}\big[X_0\big|X_2\big] \ .$$

*Proof.* By the Bayes Thm. 5.4 we have for any indices to the variables

$$\mathbb{P}\big[X_0 = x_0\big|X_1 = x_1, X_2 = x_2\big] = \frac{\mathbb{P}\big[X_0 = x_0, X_1 = x_1\big|X_2 = x_2\big]}{\big\langle \mathbb{P}\big[X_0, X_1 = x_1\big|X_2 = x_2\big]\big\rangle_{[\varnothing]}}$$

If $X_0$ is independent of $X_1$ given $X_2$ it follows that

$$\begin{aligned}\mathbb{P}\big[X_0 = x_0\big|X_1 = x_1, X_2 = x_2\big] &= \frac{\mathbb{P}\big[X_0 = x_0\big|X_2 = x_2\big] \cdot \mathbb{P}\big[X_1 = x_1\big|X_2 = x_2\big]}{\big\langle \mathbb{P}\big[X_0, X_1 = x_1\big|X_2 = x_2\big]\big\rangle_{[\varnothing]}} \\ &= \mathbb{P}\big[X_0 = x_0\big|X_2 = x_2\big] \ .\square\end{aligned}$$

Following our motivation of sparse decompositions, we now combine this result with the generic chain rule, to show Markov Chain decompositions.

**Theorem 5.9 (Markov Chain)** *Let there be a set of variables $X_k$ where $k \in [d]$, and let us denote for $k \in [d]$ by $X_{[k]}$ the collection of variables $X_0, \ldots, X_{k-1}$. Let us assume, that for any $k \in [d]$ with $k \geq 2$ the variable $X_k$ is independent of $X_{[k-1]}$ conditioned on $X_{k-1}$, then*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}[X_0]\} \cup \{\mathbb{P}\big[X_k\big|X_{k-1}\big] \ : \ k \in [d],\ k \geq 1\}\right\rangle_{\left[X_{[d]}\right]}$$

*We depict this decomposition in Figure 5.4.*

*Proof.* By the chain rule shown in Thm. 5.5 we have

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}\big[X_k\big|X_{[k]}\big] \ : \ k \in [d]\}\right\rangle_{\left[X_{[d]}\right]}$$

Using that $X_k$ is conditionally independent of $X_{[k-1]}$ conditioned on $X_{k-1}$ we further have by Thm. 5.8

$$\mathbb{P}\big[X_k\big|X_{[k]}\big] = \mathbb{P}\big[X_k\big|X_{k-1}\big] \otimes \mathbb{I}\left[X_{[k-1]}\right] \ .$$

Composing both equalities and omitting the trivial tensors shows the claim. □

The assumption of $X_k$ being independent of $X_{[k-1]}$ conditioned on $X_{k-1}$ is called the Markov property and the corresponding collection of random variables is called a Markov Chain. Thm. 5.9 states an efficient decomposition of

the probability distribution into a concatenated product of matrices representing conditional probability distributions. Marginal distributions of Markov Chains can therefore consecutively be computed by matrix-vector products, that is for $k \in [d]$ with $k \geq 1$

$$\mathbb{P}\left[X_k\right] = \left\langle \mathbb{P}\left[X_k \middle| X_{k-1}\right], \mathbb{P}\left[X_{k-1}\right] \right\rangle_{[X_k]} .$$

The conditional probability matrices are therefore called stochastic transition matrices.

We notice that the decomposition scheme of Thm. 5.9 hints at an efficient representation of $\mathbb{P}\left[X_{[d]}\right]$ based on transition matrices. While $\mathbb{P}\left[X_{[d]}\right]$ is a tensor in a space of dimension

$$\prod_{k\in[d]} m_k ,$$

the sum of the dimension of the transition matrices is

$$m_0 + \sum_{k\in[d]\,:\,k\geq 1} m_k \cdot m_{k-1} .$$

We therefore observe a linear increase of the storage demand of the transition matrices in the order $d$, whereas a naive storage of $\mathbb{P}\left[X_{[d]}\right]$ by its coordinates would have an exponentially demand.

The Markov Chain serves as a toy example drawing on a restrictive chain arrangement of conditional independencies. In the following section, we will investigate decomposition schemes, which relax this assumption and draw on more general collections of conditional independencies. The computation of marginal distribution by consecutive transition matrix multiplications will then be replaced by more general tensor network contractions.

## 5.3 Graphical Models

Graphical models are specific instances of exponential families [82, 55]. They combine both the independence approach and the computation approach to tensor network representations of probability distributions. Graphical models provide a generic framework to relate conditional dependency assumptions on a distribution with tensor network decompositions. Following the tensor network formalism we in this section introduce graphical models based on hypergraphs. First, we study Markov Networks in most generality and then connect with conditional probabilities in the discussion of Bayesian Networks.

### 5.3.1 Markov Networks

We now define Markov Networks based on hypergraphs, to establish a direct connection with tensor network decorating the hypergraph. In a more canonical way, Markov Networks are instead defined by graphs, where instead of the edges the cliques are decorated by factor tensors (see for example [45]).

**Definition 5.8 (Markov Network)** Let $\tau^{\mathcal{G}}$ be a tensor network of non-negative tensors decorating a hypergraph $\mathcal{G}$. Then the Markov Network $\mathbb{P}^{\mathcal{G}}$ to $\tau^{\mathcal{G}}$ is the probability distribution of $X_v$ defined by the tensor

$$\mathbb{P}^{\mathcal{G}}\left[X_{\mathcal{V}}\right] = \frac{\langle\{\tau^e \,:\, e \in \mathcal{E}\}\rangle_{[X_{\mathcal{V}}]}}{\langle\{\tau^e \,:\, e \in \mathcal{E}\}\rangle_{[\varnothing]}} = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}}|\varnothing]} .$$

We call the denominator

$$\mathcal{Z}\left(\tau^{\mathcal{G}}\right) = \langle\{\tau^e \,:\, e \in \mathcal{E}\}\rangle_{[\varnothing]}$$

the partition function of the tensor network $\tau^{\mathcal{G}}$.

The marginalization of a Markov Network to $\tau^{\mathcal{G}}$ on subsets of variables $X_{\mathcal{U}}$ is

$$\mathbb{P}^{\mathcal{G}}\left[X_{\mathcal{U}}\right] = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}|\varnothing]} .$$

This can be derived from Thm. **??**, which established an equivalence of contractions with sequences of consecutive contractions.

Further, the distribution of $X_{\mathcal{U}}$ conditioned on $X_{\mathcal{W}}$, where $\mathcal{U}, \mathcal{W}$ are disjoint subsets of $\mathcal{V}$, is

$$\mathbb{P}^{\mathcal{G}}\left[X_{\mathcal{U}}|X_{\mathcal{W}}\right] = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}|X_{\mathcal{W}}]} .$$

While we have directly defined Markov Networks as decomposed probability distributions, we now want to derive assumptions on a distribution assuring that such decompositions exist. As we will see, the sets of conditional independencies encoded by a hypergraph are captured by its seperation properties, as we define next.

**Definition 5.9 (Separation of Hypergraph)** A path in a hypergraph is a sequence of nodes $v_k$ for $k \in [d]$, such that for any $k \in [d-1]$ we find a hyperedge $e \in \mathcal{E}$ such that $(v_k, v_{k+1}) \subset e$. Given disjoint subsets $A$, $B$, $C$ of nodes in a hypergraph $\mathcal{G}$ we say that $C$ separates $A$ and $B$ with respect to $\mathcal{G}$, when any path starting at a node in $A$ and ending in a node in $B$ contains a node in $C$.

To characterize Markov Networks in terms of conditional independencies we need to further define the property of clique-capturing. This property of clique-capturing established a correspondence of hyperedges with maximal cliques in the more canonical graph-based definition of Markov Networks [45].

**Definition 5.10 (Clique-Capturing Hypergraph)** We call a hypergraph $\mathcal{G}$ clique-capturing, when each subset $\mathcal{U} \subset \mathcal{V}$ is contained in a hyperedge, if for any $a, b \in \mathcal{U}$ there is a hyperedge $e \in \mathcal{E}$ with $a, b \in e$.

Let us now show a characterization of Markov Networks in terms of conditional independencies, which is analogous to Thm. 5.11.

**Theorem 5.10 (Hammersley-Clifford)** *Given a clique-capturing hypergraph $\mathcal{G}$, the set of positive Markov Networks on the hypergraph coincides with the set of positive probability distributions, such that each for each disjoint subsets of variables A, B, C we have $X_A$ is independent of $X_B$ conditioned on $X_C$, when C separates A and B in the hypergraph.*

*Proof.* "⇒": Let there be a hypergraph $\mathcal{G}$, a Markov Network $\tau^{\mathcal{G}}$ on $\mathcal{G}$ and nodes $A, B, C \subset \mathcal{V}$, such that $C$ separates $A$ from $B$. Let us denote by $\mathcal{V}_0$ the nodes with paths to $A$, which do not contain a node in $C$, and by $\mathcal{V}_1$ the nodes with paths to $B$, which do not contain a node in $C$. Further, we denote by $\mathcal{E}_0$ the hyperedges which contain a node in $\mathcal{V}_0$ and by $\mathcal{E}_1$ the hyperedges which contain a node in $\mathcal{V}_1$. By assumption of separability, both sets $\mathcal{E}_0$ and $\mathcal{E}_1$ are disjoint and no node in $A$ is in a hyperedge in $\mathcal{E}_1$, respectively no node in $B$ is in a hyperedge in $\mathcal{E}_0$, . We then have

$$\begin{aligned}
\langle\{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}\rangle_{[X_A, X_B|X_C = x_C]} &= \left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\} \cup \{\epsilon_{x_C}\} \right\rangle_{[X_A, X_B|\varnothing]} \\
&= \left\langle \{\tau^e \,:\, e \in \mathcal{E}_0\} \cup \{\epsilon_{x_C}\} \right\rangle_{[X_A|\varnothing]} \\
&\quad \otimes \left\langle \{\tau^e \,:\, e \in \mathcal{E}_1\} \cup \{\epsilon_{x_C}\} \right\rangle_{[X_B|\varnothing]} .
\end{aligned}$$

By Thm. 5.7, it now follows that $X_A$ is independent of $X_B$ conditioned on $X_C$.

"⇐": The converse direction, i.e. that positive distributions respecting the conditional independence assumptions are representable as Markov Networks, is known as the Hammersley-Clifford Theorem (see [16]), which we will proof later in Sect. **??** of Chapter **??**. □

From the proof of Thm. 5.10 Markov Networks with zero coordinates still satisfy the conditional independence assumption. However, the reverse is not true, that is there are distributions with vanishing coordinates, which satisfy the conditional independence assumptions, but cannot be represented as a Markov Network (see Example 4.4 in [45]).

*Example 5.1 (Independent boolean variables)* Let there be $d$ boolean variables $X_{[d]}$, which are i.i.d. drawn from a positive distribution $\mathbb{P}\left[X\right]$. From the pairwise independencies of $X_k$ it follows with the Hammersley-Clifford Factorization Thm. **??** that the distribution is representable by an elementary tensor network, that is

$$\mathbb{P}\left[X_{[d]}\right] = \bigotimes_{k\in[d]} \mathbb{P}^k\left[X_k\right] .$$

The corresponding hypergraph is the elementary graph, with respect to which any two disjoint subsets of nodes are separated (see Figure 5.5).

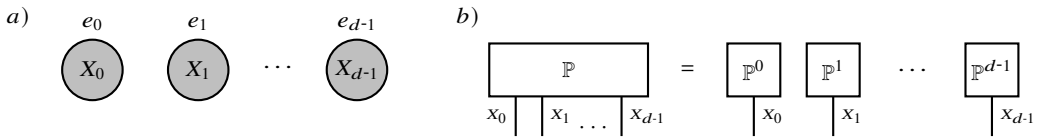



**Fig. 5.5** Decomposition of a probability distribution with independent variables (see Example 5.1). The independencies are captured by the elementary hypergraph a), whose edges contain single nodes. The corresponding tensor $\mathbb{P}\left[X_{[d]}\right]$ is then represented by a Markov network on the elementary hypergraph, where each factor is the marginal distribution of the corresponding variable as visualized in b).

*Example 5.2* We consider a classical example of a graphical model (see [45, Example 4.3]): A student of intelligence ($X_I$) and SAT score ($X_S$) is assigned a test of difficulty ($X_D$), for which he gets a grade ($X_G$) depending on which he gets a recommendation letter ($X_L$) by its teacher. We make the following modelling assumptions:

- "The SAT score depends only on the students intelligence": $\left(X_S \perp X_{\{D,G,L\}}\right) \Big| X_I$.
- "The recommendation letter depends only on the grade": $\left(X_L \perp X_{\{D,I,S\}}\right) \Big| X_G$.

The associated hypergraph capturing these conditional independencies is drawn in Figure 5.6 a).

### 5.3.2 Bayesian Networks

Compared to Markov Networks, Bayesian Networks impose further conditions on tensor networks representing a distribution. They assume a directed hypergraph and each tensor decorating the edges to be normalized according to the direction. We will observe, that if the hypergraph is in addition acyclic, then each tensor core coincides with the conditional distribution of the underlying Markov Network. To introduce Bayesian Networks, we extend Def. **??** by introducing the property of acyclicity for hypergraphs.

**Definition 5.11** A directed path is a sequence $v_0, \dots v_r$ such that for any $l \in [r]$ there is an hyperedge $e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E}$ such that $v_l \in e^{\text{in}}$ and $v_{l+1} \in e^{\text{out}}$. We call the hypergraph $\mathcal{G}$ acyclic, if there is no path with $r > 0$ such that $v_0 = v_r$. Given a directed hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ we define for any node $v \in \mathcal{V}$ its parents by

$$\text{Pa}(v) = \left\{\tilde{v} \,:\, \left(\exists e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} : \tilde{v} \in e^{\text{in}},\, v \in e^{\text{out}}\right)\right\}$$

and its non-descendants $\text{NonDes}(v)$ as the set of nodes $\tilde{v}$, such that there is no directed path from $v$ to $\tilde{v}$.

Based on these additional graphical properties, we now define Bayesian Networks.

**Definition 5.12 (Bayesian Network)** Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed acyclic hypergraph with edges of the form

$$\mathcal{E} = \{(\text{Pa}(v), \{v\}) \,:\, v \in \mathcal{V}\} .$$

A *Bayesian Network* is a decoration of each edge $(\text{Pa}(v), \{v\})$ by a conditional probability distribution

$$\mathbb{P}\left[X_v \middle| X_{\text{Pa}(v)}\right]$$

which represents the probability distribution

$$\mathbb{P}\left[X_{\mathcal{V}}\right] = \left\langle \{\mathbb{P}\left[X_v \middle| X_{\text{Pa}(v)}\right] \,:\, v \in \mathcal{V}\} \right\rangle_{[X_{\mathcal{V}}]} .$$

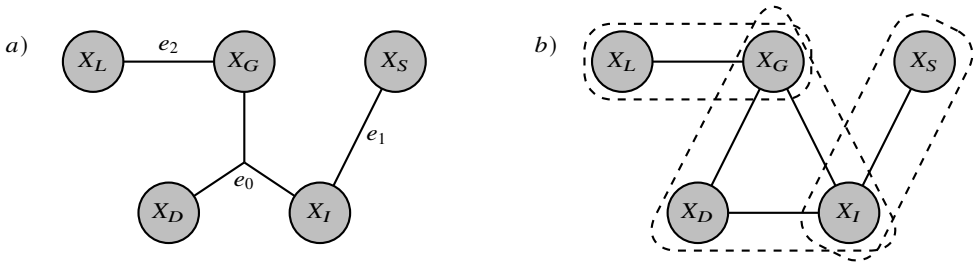



**Fig. 5.6** Hypergraph a) capturing the conditional independencies of the student example. The cliques of the node adjacency graph are highlighted in b) and coincide with hyperedges of the hypergraph. The hypergraph is therefore clique-capturing (see Def. 5.10).

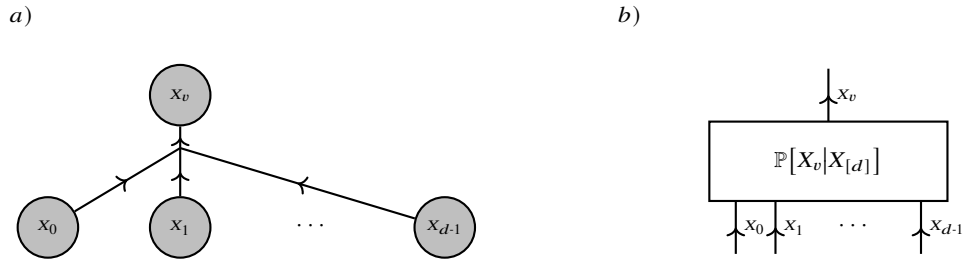



**Fig. 5.7** Example of a Factor of a Bayesian Network to the node $X_v$ with parents $X_0, \dots, X_{d-1}$, as an directed edge a) which is decorated by a directed tensor b).

By definition each tensor decorating a hyperedge is directed with $X_{\mathrm{Pa}(v)}$ incoming and $X_v$ outgoing. Thus, the directionality of the hypergraph is reflected in each tensor decorating a directed hyperedge. This allows us to verify with Thm. **??** that their contraction defines a probability distribution.

Marginalization of a Bayesian Network are still Bayesian Networks on a graph where the edges directing to variables, which are not marginalized over, are replaced by directed edges to the children. Conditioned Bayesian Network do not have a simple Bayesian Network representation, which is why we will treat them as Markov Networks to be introduced next.

**Theorem 5.11 (Independence Characterization of Bayesian Networks)** *A probability distribution $\mathbb{P}[X_{\mathcal{V}}]$ has a representation by a Bayesian Network on a directed acyclic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, if and only if for any $v \in \mathcal{V}$ the variables $X_v$ are independent on* $\mathrm{NonDes}(v)$ *conditioned on* $\mathrm{Pa}(v)$.

*Proof.* We choose a topological order $\prec$ on the nodes of $\mathcal{G}$, which exists since $\mathcal{G}$ is acyclic.

"⇒": Let us assume, that the conditional independencies are satisfied and apply the chain rule with respect to that ordering to get

$$\mathbb{P}[X_{\mathcal{V}}] = \left\langle \mathbb{P}\big[X_v \big| X_{\tilde{v}} \,:\, \tilde{v} \prec v\big] \right\rangle_{[X_{\mathcal{V}}]} .$$

Since $\prec$ is a topological ordering we have

$$\mathrm{Pa}(v) \subset \{\tilde{v} \,:\, \tilde{v} \prec v\}$$

We apply the assumed conditional independence with Thm. 5.8 and get

$$\mathbb{P}[X_{\mathcal{V}}] = \left\langle \mathbb{P}\big[X_v \big| X_{\mathrm{Pa}(v)}\big] \right\rangle_{[X_{\mathcal{V}}]} .$$

"⇐": To show the converse direction, let there be a Bayesian Network $\mathbb{P}[X_{\mathcal{V}}]$ on $\mathcal{G}$. To show for any node $v$, that $X_v$ is independent of $\mathrm{NonDes}(v)$ conditioned on $\mathrm{Pa}(v)$, we reorder the tensors in the contraction

$$
\begin{aligned}
&\mathbb{P}\big[X_v, X_{\mathrm{NonDes}(v)}\big|X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\big] \\
&\quad= \left\langle \{\mathbb{P}\big[X_{\tilde{v}}\big|X_{\mathrm{Pa}(\tilde{v})}\big] \,:\, \tilde{v} \in \mathcal{V}\}\right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)}|X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}\right]} \\
&\quad= \left\langle \{\mathbb{P}\big[X_{\tilde{v}}\big|X_{\mathrm{Pa}(\tilde{v})}\big] \,:\, \tilde{v} \in \mathcal{V}\} \cup \{\epsilon_{x_{\mathrm{Pa}(v)}}\}\right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)}|\varnothing\right]} \\
&\quad= \left\langle \{\mathbb{P}\big[X_{\tilde{v}}\big|X_{\mathrm{Pa}(\tilde{v})}\big] \,:\, \tilde{v} \in \mathrm{NonDes}(v)\} \cup \{\epsilon_{x_{\mathrm{Pa}(v)}}, \mathbb{P}\big[X_v\big|X_{\mathrm{Pa}(v)}\big]\}\right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)}|\varnothing\right]} \\
&\quad= \left\langle \{\mathbb{P}\big[X_{\tilde{v}}\big|X_{\mathrm{Pa}(\tilde{v})}\big] \,:\, \tilde{v} \in \mathrm{NonDes}(v)\} \cup \{\epsilon_{x_{\mathrm{Pa}(v)}}\}\right\rangle_{\left[X_{\mathrm{NonDes}(v)}|\varnothing\right]} \\
&\qquad\cdot \left\langle \{\mathbb{P}\big[X_v\big|X_{\mathrm{Pa}(v)}\big], \epsilon_{x_{\mathrm{Pa}(v)}}\}\right\rangle_{[X_v|\varnothing]} \\
&\quad= \left\langle \{\mathbb{P}\big[X_{\mathrm{NonDes}(v)}\big|X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\big], \mathbb{P}\big[X_v\big|X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\big]\}\right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)}\right]}
\end{aligned}
$$

Here we have dropped in the third equation all tensors to the descendants, since their marginalization is trivial (which can be shown by a leaf-stripping argument). In the fourth equation we made use of the fact, that any directed path between the non-descendants and the node is through the parents of the node. By Thm. 5.7, it now follows that $X_v$ is independent of $\mathrm{NonDes}(v)$ conditioned on $\mathrm{Pa}(v)$. □

### 5.3.3 Bayesian Networks as Markov Networks

Markov Networks are more flexible compared with Bayesian Networks, since any Bayesian Network is a Markov Network by ignoring the directionality of the hypergraph and understanding the conditional distributions as generic tensor cores. In the next theorem we provide the conditions for the interpretation of a Markov Network as a Bayesian Network.

**Theorem 5.12** *Let $\tau^{\mathcal{G}}$ be a tensor network on a directed acyclic hypergraph, such that the edges are of the structure*

$$\mathcal{E} = \{(\mathrm{Pa}(v), \{v\}) \,:\, v \in \mathcal{V}\}$$

*and each tensor $\tau^e$ respects the directionality of the graph, that is each $\tau^{(\mathrm{Pa}(v),\{v\})}$ is directed with the variables to* $\mathrm{Pa}(v)$ *incoming and $v$ outgoing. Then $\mathcal{Z}\left(\tau^{\mathcal{G}}\right) = 1$ and for each $v \in \mathcal{V}$ we have*

$$\tau^{(\mathrm{Pa}(v),\{v\})} = \left\langle \tau^{\mathcal{G}}\right\rangle_{\left[X_v|X_{\mathrm{Pa}(v)}\right]} .$$

*In particular, $\tau^{\mathcal{G}}$ is a Bayesian Network.*

*Proof.* We show the claim by induction over the cardinality of $\mathcal{V}$.

$|\mathcal{V}| = 1$: In this case we find a unique node $v \in \mathcal{V}$ and have $\mathcal{E} = \{(\varnothing, \{v\})\}$. The tensor $\tau^{(\varnothing,\{v\})}$ is then normalized with no incoming variables and we thus have

$$\mathcal{Z}\left(\tau^{\mathcal{G}}\right) = \left\langle \tau^{\mathcal{G}}\right\rangle_{[\varnothing]} = \left\langle \tau^{(\varnothing,\{v\})}\right\rangle_{[\varnothing]} = 1$$

and

$$\left\langle \tau^{\mathcal{G}}\right\rangle_{[X_v|\varnothing]} = \tau^{(\varnothing,\{v\})} .$$

$|\mathcal{V}| - 1 \to |\mathcal{V}|$: Let there now be a directed hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and let us now assume, that the theorem holds for any tensor networks with node cardinality $|\mathcal{V}| - 1$. Since the hypergraph is acyclic, we find a root $v \in \mathcal{V}$ such that $v \notin \mathrm{Pa}(\tilde{v})$ for $\tilde{v} \in \mathcal{V}$. We denote $\tau^{\tilde{\mathcal{G}}}$ the tensor network on the hypergraph $\tilde{\mathcal{G}} = \{\mathcal{V}/\{v\}, \mathcal{E}/\{(\mathrm{Pa}(v), \{v\})\}\}$ with decorations inherited from $\tau^{\mathcal{G}}$. With Thm. **??**, the directionality of $\tau^{(\mathrm{Pa}(v),\{v\})}$ and the induction assumption on $\tau^{\tilde{\mathcal{G}}}$ we have

$$\begin{aligned}\left\langle \tau^{\tilde{\mathcal{G}}} \cup \left\{ \tau^{(\mathrm{Pa}(v),\{v\})} \right\} \right\rangle_{[\varnothing]} &= \left\langle \tau^{\tilde{\mathcal{G}}} \cup \left\{ \left\langle \tau^{(\mathrm{Pa}(v),\{v\})} \right\rangle_{[X_{\mathrm{Pa}(v)}]} \right\} \right\rangle_{[\varnothing]} \\ &= \left\langle \tau^{\tilde{\mathcal{G}}} \cup \left\{ \mathbb{I}\left[X_{\mathrm{Pa}(v)}\right] \right\} \right\rangle_{[\varnothing]} \\ &= 1\end{aligned}$$

and thus a trivial partition function. Since $v$ does not appear in $\tilde{\mathcal{G}}$, we have for any index $x_{\mathrm{Pa}(\mathcal{V})}$

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_v, X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}]} = \left\langle \tau^{(\mathrm{Pa}(v),\{v\})} \right\rangle_{[X_v, X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}]} \cdot \left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}]}$$

and thus, since $\tau^{(\mathrm{Pa}(v),\{v\})}$ is directed, that

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_v|X_{\mathrm{Pa}(v)}]} = \tau^{(\mathrm{Pa}(v),\{v\})} .\square$$

Thm. 5.12 states that Bayesian Networks are a subset of Markov Networks. While Markov Network allow generic tensor cores, Bayesian Networks impose a local directionality condition on each tensor core by demanding it to be a conditional probability tensor. In our diagrammatic notation, the local normalization of Bayesian Networks is highlighted by the directionality of the hypergraph. Generic Markov Networks are on undirected hypergraphs, where in general no local directionality condition is assumed. As a consequence, tasks such as the determination of the partition functions or calculation of conditional distributions involve global contractions.

*Example 5.3 (Hidden Markov Models)* Hidden Markov Models are examples of Bayesian Networks, constructed as follows. Let us recall Markov Chains as investigated in Thm. 5.9 and extend them by observation variables $E_k$ for $k \in [d]$, representing limited observations of the state variables $X_k$. To be more precise, we assume the following conditional independencies:

- As for Markov Chains, we assume that for $k \in [d]$ with $k \geq 1$ the variable $X_k$ is independent of $X_{[k-1}$ and $E_{[k-1]}$ conditioned on $X_{k-1}$
- In addition, for we assume that for $k \in [d]$ the observation variable $E_k$ is independent of $X_{[k}$ and $E_{[k]}$ conditioned on $X_k$

From this conditional independence assumption, we apply the Chain Rule Thm. 5.5 given the order of variables

$$X_0, E_0, X_1, E_1, \ldots, X_{d-1}, E_{d-1}$$

and get

$$\begin{aligned}\mathbb{P}\left[X_{[d]}, E_{[d]}\right] = \Big\langle &\{\mathbb{P}\left[X_0\right], \mathbb{P}\left[E_0 \middle| X_0\right]\} \\ &\cup \{\mathbb{P}\left[X_k \middle| X_{[k]}, E_{[d]}\right] \,:\, k \in [d]\} \\ &\cup \{\mathbb{P}\left[E_k \middle| X_{[k+1]}, E_{[d]}\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}, E_{[d]}\right]} .\end{aligned}$$

We now apply the conditional independence assumptions to sparsify the appearing conditional distributions by application of Thm. 5.8. This results in the decomposition (see Figure 5.8b)

$$\begin{aligned}\mathbb{P}\left[X_{[d]}, E_{[d]}\right] = \Big\langle &\{\mathbb{P}\left[X_0\right], \mathbb{P}\left[E_0 \middle| X_0\right]\} \\ &\cup \{\mathbb{P}\left[X_k \middle| X_{k-1}\right] \,:\, k \in [d]\} \\ &\cup \{\mathbb{P}\left[E_k \middle| X_k\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}, E_{[d]}\right]} .\end{aligned}$$

In addition to the stochastic transition matrices $\mathbb{P}\left[X_k \middle| X_{k-1}\right]$ appearing in Markov Chains, we further have stochastic observation matrices $\mathbb{P}\left[E_k \middle| X_k\right]$ for $k \in [d]$. Their contraction with marginal distribution of the respective state varibles delivers the marginal distribution of the observation matrix by

$$\mathbb{P}\left[E_k\right] = \left\langle \mathbb{P}\left[E_k \middle| X_k\right], \mathbb{P}\left[X_k\right] \right\rangle_{[E_k]}$$

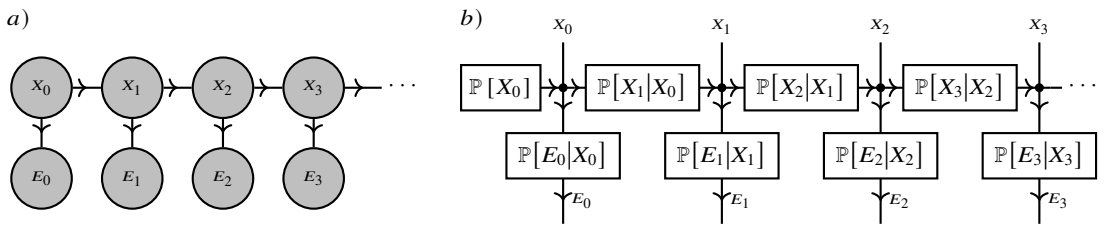



**Fig. 5.8** Decomposition of a probability distribution in the Hidden Markov Model, consistent of state variables $X_k$ and observation variables $E_k$. Given the models conditional independence assumptions, the distribution is a Bayesian Network on the directed hypergraph a). The hypergraph is decorated by the network of conditional probability tensors b), which are interpreted as stochastic transition matrices $\mathbb{P}[X_k|X_{k-1}]$ and stochastic observation matrices $\mathbb{P}[E_k|X_k]$.

We notice that this is a Bayesian Network on a directed acyclic hypergraph $\mathcal{G}$ (see Figure 5.8a) consistent in nodes $\{X_{[d]}\} \cup \{E_{[d]}\}$ to each state and observation variables, and the directed hyperedges by

- $(\varnothing, \{X_0\})$, decorated by the intial marginal distribution of $X_0$
- $(\{X_{k-1}\}, \{X_k\})$ for $k \in [d]$ with $k \geq 1$, decorated by stochastic transition matrices
- $(\{X_k\}, \{E_k\})$ for $k \in [d]$, decorated stochastic observation matrices

While we have derived this directed graph structure directly based on the chain rule decomposition with sparsified conditional distributions, it also follows from the more generic hypergraph characterization of Bayesian Networks through separability by Thm. 5.11.

## 5.4 Factorization based on sufficient statistics

Missing: Definition of family of distributions

Let us now introduce sufficient statistics towards studying further tensor network decompositions of probability distributions.

**Definition 5.13** Let $\mathbb{P}[X, Z]$ be a joint distribution of the $m$-dimensional variable $X$ and the $n$-dimensional variable $Z$ and let

$$t : [m] \to [n]$$

be a statistic. We are interested in the distribution $\mathbb{P}[X, Z, Y_t] = \langle \mathbb{P}[X, Z], \beta^t [Y_t, X] \rangle_{[X,Z,Y_t]}$. We say that $t$ is a sufficient statistic for $Z$ if $X$ is independent of $Z$ conditioned on $Y_t$.

*Example 5.4 (Sufficient statistics for the probability)* Let $Z$ be the value $\mathbb{P}[X_{[d]} = x_{[d]}]$ when drawing $X_{[d]}$ from $\mathbb{P}[X_{[d]}]$. Then $t$ is a sufficient statistic for $Z = \mathbb{P}[X_{[d]}]$ if for all $y$ in the image of $t$ we have

$$\mathbb{P}[X_{[d]} = x_{[d]} | Y_t = y] = \begin{cases} \frac{1}{|\{x_{[d]} : t(x_{[d]}) = y\}|} & \text{if} \quad t(x_{[d]}) = y \\ 0 & \text{else} \end{cases}.$$

When knowing the value $t(x_{[d]})$ of the sufficient statistic at a given index $x_{[d]}$, we then also know the probability $\mathbb{P}[X_{[d]} = x_{[d]}]$. The function $t$ is thus a sufficient statistic for $Z = \mathbb{P}[X_{[d]}]$ if and only if there is a tensor $\xi[Y_{[p]}]$ with

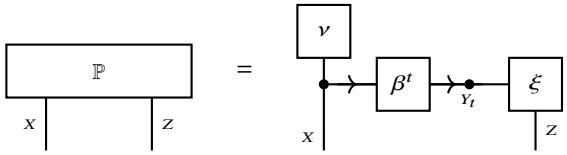
$$\mathbb{P}[X_{[d]}] = \langle \beta^t [Y_{[p]}, X_{[d]}], \xi[Y_{[p]}] \rangle_{[X_{[d]}]}.$$

Example 5.4 hints at a connection between sufficient statistics and decompositions into CompActNets. More generally, such decompositions are provided by the Fisher-Neyman Factorization Theorem.

**Theorem 5.13 (Fisher-Neyman Factorization Theorem)** *Let $\mathbb{P}$ be a joint distribution of variables $X, Z$ with values* $\mathrm{val}(X)$, $\mathrm{val}(Z)$. *Let there further be a finite set* $\mathrm{val}(Y_t)$, *which is enumerated with a variable $Y_t$. Then $t : \mathrm{val}(X) \to \mathrm{val}(Y_t)$ is a sufficient statistic for $Z$ if and only if there are tensors $\nu[X]$ and $\xi[Y_t, Z]$ such that*

$$\mathbb{P}[X, Z] = \left\langle \xi[Y_t, Z], \beta^t[Y_t, X], \nu[X] \right\rangle_{[X,Z]} .$$

*We depict this equation diagrammatically by*




*Proof.* Shown in the appendix, see Thm. **??**. □

Note that the definition of sufficient statistic does not make use of the marginal distribution $\mathbb{P}[Z]$. We therefore can define sufficient statistics also for families of distributions $\mathbb{P}\left[X \middle| Z\right]$ with respect to arbitrary non-degenerate marginal distributions $\mathbb{P}[Z]$. We then use Thm. 5.13 to embed such families in CompActNets.

**Corollary 5.2** *Let $\mathbb{P}\left[X_{[d]} \middle| Z\right]$ be an arbitrary family of distributions of $X_{[d]}$ and $t$ a sufficient statistic for $Z$. Then, there is a tensor $\nu\left[X_{[d]}\right]$ and an activation tensor $\xi\left[Y_{[p]}, Z\right]$ such that for any $z \in \mathrm{val}(Z)$ we have*

$$\mathbb{P}\left[X_{[d]} \middle| Z = z\right] \in \Lambda^{t,\mathrm{MAX},\xi} .$$

The Factorization Theorem of Fisher-Neyman provides the fundamental motivation for the CompActNets architecture. Any decomposition of $\beta^t\left[Y_{[p]}, X_{[d]}\right]$ is called a *computation network* and common to all members of a family with sufficient statistic $t$. The activation tensor $\xi\left[Y_{[p]}, Z = z\right]$, whose decomposition is called the *activation network*, is specific to each member of the family. We now show in two examples how families of distributions can be represented in CompActNets by sufficient statistics.

*Example 5.5 (Order statistic for boolean variables)* Let there be $d$ boolean variables $X_{[d]}$ and a family $\mathbb{P}\left[X_{[d]} \middle| Z\right]$ of distributions. The order statistic assigns the ordered tuple to each tuple $x_{[d]}$. The ordered tuple effectively counts the number of 1 coordinates in the tuple $x_{[d]}$, that is the statistic

$$t^+ : \bigtimes_{k \in [d]} [m_k] \to [p], \quad t^+(x_{[d]}) = |\{k : x_k = 1\}| .$$

When the order statistic is sufficient for $Z$, the detailed order of the outcomes is uninformative about the member $z$ from which the random variables have been drawn. By the Fisher-Neyman Factorization Thm. 5.13, $t^+$ is a sufficient statistic if and only if there are tensors $\nu\left[X_{[d]}\right]$ and $\xi[Y_+, \Theta]$ such that for each $\theta \in \Theta$

$$\mathbb{P}\left[X_{[d]} \middle| Z = z\right] = \left\langle \xi[Y_+, Z = z], \beta^{t^+}\left[Y_+, X_{[d]}\right], \nu\left[X_{[d]}\right] \right\rangle_{\left[X_{[d]}\right]} .$$

For each member $z$ of the family, the probability of each sequence $x_{[d]}$ is thus the product of a base measure factor $\nu\left[X_{[d]} = x_{[d]}\right]$ and a factor $\xi\left[Y_+ = +(x_{[d]}), Z = z\right]$ depending only on the count $+(x_{[d]})$ of 1 coordinates in $x_{[d]}$. We later continue this example in Example 13.1, where further interpretations to the case of i.i.d. variables are provided.

*Example 5.6 (Graphical models as a special case of CompActNets)* For graphical models we take the *identity statistic*

$$\delta(x_{[d]}) = x_{[d]}$$

so that the image coordinates coincide with the variables and there are no non-trivial computation cores. The associated basis encoding is just the identity tensor

$$\beta^\delta\left[Y_{[d]}, X_{[d]}\right] = \delta\left[X_{[d]}, Y_{[d]}\right] .$$

Therefore, for any activation tensor $\xi\left[Y_{[p]}\right]$ we obtain

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \xi\left[Y_{[p]}\right], \beta^{\delta}\left[Y_{[d]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} = \left\langle \xi\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} .$$

Put differently, in the graphical–model case the activation tensor coincides with the joint distribution tensor. In this setting, structural properties of the distribution such as (conditional) independences can be read off as algebraic factorization patterns of the activation (and hence joint) tensor.

## 5.5 Efficient contractions by message passing

Contractions of tensor networks are generally hard to solve. Here, we investigate message passing algorithms, which decompose global contractions into a sequence of local contractions, whose results are passed as messages through the tensor network. The resulting algorithm is called the Tree Belief Propagation (see Algorithm 8). While various scheduling strategies for the message passing exist, we focus on the case of tree hypergraphs, for which exactness and efficiency can be shown. We denote $\mathcal{E}^{\rightarrow}$ to be all tuples $(e_0, e_1)$ of hyperedges $e_0, e_1 \in \mathcal{E}$ such that $e_0 \neq e_1$ and $e_0 \cap e_1 \neq \varnothing$. For our purposes we call a hypergraph $\mathcal{G}$ a tree when the graph with nodes by the hyperedges $\mathcal{E}$ and edges by $\mathcal{E}^{\rightarrow}$ is a tree (for an example see Figure 5.9).

**Algorithm 4** Tree Belief Propagation

**Require:** Tensor network $\tau^{\mathcal{G}}$ on a hypergraph $\mathcal{G}$
**Ensure:** Messages $\{\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

Initialize a message scheduler $S = \{(e_0, e_1) \in \mathcal{E}^{\rightarrow} : e_0 \text{ a leaf in the tree } (\mathcal{E}, \mathcal{E}^{\rightarrow})\}$]
**while** $S$ not empty **do**
Pop a $(e_0, e_1)$ pair from $S$
Compute the message

$$\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2\rightarrow e_0}\left[X_{e_2\cap e_0}\right] : (e_2, e_0) \in \mathcal{E}^{\rightarrow},\ e_2 \neq e_1\}\right\rangle_{\left[X_{e_0\cap e_1}\right]}$$

Update $S$ by all messages $(e_1, e_3)$ which have not yet been sent, if all messages $(e_2, e_1)$ with $e_2 \neq e_3$ have been sent.
**end while**
**return** Messages $\{\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

For an implementation of Algorithm 8 in the python package tnreason, see Sect. **??**.

The following theorem states that the contraction of a whole tensor network can be replaced by local contractions with messages. Since contracting the whole network can be infeasible, this shows that calculating the messages with the Algorithm 8 can be advantageous.

**Theorem 5.14** *Let $\tau^{\mathcal{G}}$ be a tensor network on a tree hypergraph $\mathcal{G}$ (i.e. the graph $(\mathcal{E}, \mathcal{E}^{\rightarrow})$ is a tree). The messages in the tree belief propagation Algorithm 8 are contracted to local marginals, meaning that for each $e_0 \in \mathcal{E}$ we have*

$$\left\langle \tau^{\mathcal{G}}\right\rangle_{\left[X_{e_0}\right]} = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2\rightarrow e_0}\left[X_{e_2\cap e_0}\right] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\}\right\rangle_{\left[X_{e_0}\right]} .$$

We show Thm. 5.14 based on the following lemma. We denote for each pair $(e_0, e_1)$ the subset $\mathcal{E}^{\rightarrow(e_0,e_1)} \subset \mathcal{E}$ as the subset of edges $e \in \mathcal{E}$, for which each path in $(\mathcal{E}, \mathcal{E}^{\rightarrow})$ to $e_1$ passes through $e_0$. The tree hypergraph property makes this definition equivalent to an existing path through $e_0$, which is used in the proof of the following lemma. Note that by construction $e_0 \in \mathcal{E}^{\rightarrow(e_0,e_1)}$.

**Lemma 5.1** *For any tensor network on a tree hypergraph, Algorithm 8 terminates in the tree-based implementation and returns final messages*

$$\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] = \left\langle \{\tau^{e}\left[X_e\right] : e \in \mathcal{E}^{\rightarrow(e_0,e_1)}\}\right\rangle_{\left[X_{e_0\cap e_1}\right]} .$$

*Proof.* We show this property by induction over the size of the edge sets $\mathcal{E}^{\to(e_0,e_1)}$ for pairs $(e_0, e_1) \in \mathcal{E}^{\to}$, such that $\left|\mathcal{E}^{\to(e_0,e_1)}\right| \leq n$. Note that since always $e_0 \in \mathcal{E}^{\to(e_0,e_1)}$ we have that $n \geq 1$.

$n = 1$: In this case we have $\mathcal{E}^{\to(e_0,e_1)} = \{e_0\}$ and $e_0$ is a leaf of the tree-hypergraph $\mathcal{G}$. The claimed message property holds thus by definition.

$n \to n+1$: Assume that the message obeys the claimed property for edge sets with cardinality up to $n$. If there is no edge set with cardinality $n+1$, the property holds also for those with cardinality up to $n+1$. If there is an edge set $\mathcal{E}^{\to(e_0,e_1)}$ with size $n+1$, we have

$$\mathcal{E}^{\to(e_0,e_1)} = \{e_0\} \cup \left( \bigcup_{e_2 \in \mathcal{E}^{\to}} \mathcal{E}^{\to(e_2,e_0)} \right) .$$

The message $\chi_{e_0 \to e_1}$ is sent once all messages $\chi_{e_2 \to e_0}$ to $(e_2, e_0) \in \mathcal{E}^{\to(e_0,e_1)}$ arrived. By definition we have that

$$\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to},\, e_2 \neq e_1\} \right\rangle_{\left[X_{e_0 \cap e_1}\right]} .$$

Now we use the induction assumption on $\mathcal{E}^{\to(e_2,e_0)}$ (since its cardinality is at most $n$) and get

$$\begin{aligned}
\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] &= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \left( \bigcup_{(e_2,e_0) \in \mathcal{E}^{\to},\, e_2 \neq e_1} \left\langle \{\tau^{e_3}\left[X_{e_3}\right] \,:\, e_3 \in \mathcal{E}^{\to(e_2,e_0)}\} \right\rangle_{\left[X_{e_2 \cap e_0}\right]} \right) \right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \left( \bigcup_{(e_2,e_0) \in \mathcal{E}^{\to},\, e_2 \neq e_1} \{\tau^{e_3}\left[X_{e_3}\right] \,:\, e_3 \in \mathcal{E}^{\to(e_2,e_0)}\} \right) \right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \left\langle \{\tau^{e}\left[X_{e}\right] \,:\, e \in \mathcal{E}^{\to(e_0,e_1)}\} \right\rangle_{\left[X_{e_0 \cap e_1}\right]} .
\end{aligned}$$

Here, we used the commutation of contraction property in the second equation, which is justified by the assumed tree property of the hypergraph. Thus, the message property holds also for any edge sets of size $n+1$.

By induction, the claimed message property therefore holds for all final messages. □

*Proof of Thm. 5.14.* Since the hypergraph is by assumption a tree, we can partition $\mathcal{E}$ into disjoint subsets $\{e_0\}$ and $\mathcal{E}^{\to(e_2,e_0)}$ for $(e_2, e_0) \in \mathcal{E}^{\to}$. We then have

$$\begin{aligned}
\left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_{e_0}\right]} &= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \left\{ \left\langle \tau^{e}\left[X_e\right] \,:\, e \in \mathcal{E}^{\to(e_2,e_0)} \right\rangle_{\left[X_{e \cap e_2}\right]} \,:\, (e_2, e_0) \in \mathcal{E}^{\to} \right\} \right\rangle_{\left[X_{e_0}\right]} \\
&= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to}\} \right\rangle_{\left[X_{e_0}\right]} ,
\end{aligned}$$

where we used Lem. 5.1 in the second equation. □

We illustrate the usage of Algorithm 8 on the Markov network of Example 5.2.

*Example 5.7 (Continuation of Example 5.2)* We exemplify the Belief Propagation Algorithm 8 on the Markov network in the student example (see Example 5.2). The directions of the messages result from the hyperedge overlaps (see Figure 5.9 a) and the resulting directions $\mathcal{E}^{\to}$ are sketched in Figure 5.9 b). The messages to $\{(e_2, e_0), (e_0, e_2)\}$ are vectors of $X_G$ and the messages $\{(e_0, e_1), (e_1, e_0)\}$ are vectors of $X_I$.

Since the hyperedges are minimally connected, we can implement Algorithm 8 by a tree scheduler $S$:

- The scheduler is initialized with messages from leafs, in our example $\{(e_2, e_0), (e_1, e_0)\}$.
- Each message is placed exactly once on $S$, when at a hyperedge all but the reverse message have been received. In our example, after execution of $(e_2, e_0)$ the message $(e_0, e_1)$ is placed on $S$ and after execution of $(e_1, e_0)$ the message $(e_0, e_2)$.

In this implementation, Algorithm 8 terminates after $|\mathcal{E}^{\to}| = 4$ iterations of the while loop. The exact marginals of the edge variables are then

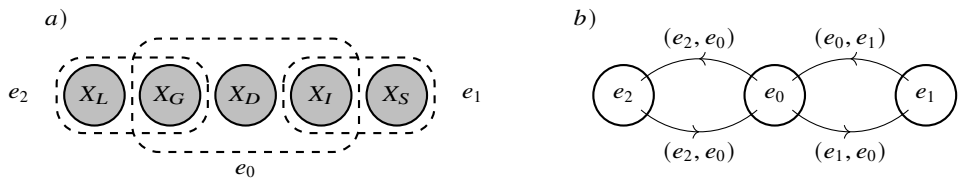



**Fig. 5.9** a) Sketch of the overlap of the edges, resulting in the message directions b) $\mathcal{E}^{\rightarrow} = \{(e_2, e_0), (e_0, e_2), (e_0, e_1), (e_1, e_0)\}$.

$$\begin{aligned}
\mathbb{P}\left[X_L, X_G\right] &= \left\langle \tau^{e_2}\left[X_L, X_G\right], \chi_{e_0 \to e_2}\left[X_G\right]\right\rangle_{[X_L, X_G|\varnothing]}, \\
\mathbb{P}\left[X_G, X_D, X_I\right] &= \left\langle \tau^{e_0}\left[X_G, X_D, X_I\right], \chi_{e_2 \to e_0}\left[X_G\right], \chi_{e_1 \to e_0}\left[X_I\right]\right\rangle_{[X_G, X_D, X_I|\varnothing]}, \\
\mathbb{P}\left[X_I, X_S\right] &= \left\langle \tau^{e_1}\left[X_I, X_S\right], \chi_{e_0 \to e_1}\left[X_I\right]\right\rangle_{[X_I, X_S|\varnothing]} .
\end{aligned}$$

## 5.6 Queries

The efficient retrieval of information stored in probability distributions has to exploit the available decomposition schemes. To avoid the instantiation of a distribution based on its decomposition, we directly define deductive reasoning schemes by contractions, which can be executed using the available decomposition.

### 5.6.1 Querying by Functions

We now formalize queries by retrieving expectations of functions given a distribution specified by probability tensors. We exploit basis calculus in defining categorical variables $Y_q$ to tensors $q$, which are enumerating the set $\mathrm{im}\,(q)$. More details on this scheme are provided in Chapter 7, see Def. 7.5 therein.

**Definition 5.14** The marginal query of a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ by a query statistic

$$q : \bigtimes_{k \in [d]} [m_k] \to \mathcal{U}^q$$

is the vector $\mathbb{P}\left[Y_q\right]$, where $Y_q$ is an image enumerating variable (see Chapter 7 for more details), defined as the contraction

$$\mathbb{P}\left[Y_q\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \beta^q\left[Y_q, X_{[d]}\right]\right\rangle_{[Y_q]} .$$

If $\mathcal{U}^q \subset \mathbb{R}$, and the statistic $q$ is therefore a tensor, we further define the expectation query of $\mathbb{P}\left[X_{[d]}\right]$ by $q$ as

$$\mathbb{E}\left[q\right] = \left\langle q(X_{[d]}), \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .$$

Given another query statistic $g : \bigtimes_{k \in [d]} [m_k] \to \mathcal{U}^g$, which image is enumerated by a variable $Y_g$, the conditional query of the probability distribution $\mathbb{P}\left[X_{[d]}\right]$ by the statistic $q$ conditioned on $g$ is the matrix $\mathbb{P}\left[Y_q | Y_g\right] \in \mathbb{R}^{|\mathrm{im}(q)|} \otimes \mathbb{R}^{|\mathrm{im}(g)|}$ defined as the normalization

$$\mathbb{P}\left[Y_q | Y_g\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \beta^q\left[Y_q, X_{[d]}\right], \beta^g\left[Y_g, X_{[d]}\right]\right\rangle_{[Y_q | Y_g]} .$$

We notice that marginal and conditional queries generalize the schemes of marginalization and conditioning on the distributed variables. As such, marginal distributions are marginal queries with respect to a identity query statistic acting

on the respective variables. Conditional distributions can be similarly retrieved by two identity statistics, representing the incoming and outgoing variables.

Expectation queries return the expectation of a real-valued feature $q : \bigtimes_{k\in[d]}[m_k] \to \mathbb{R}$. When understanding $q$ as a random variable given the probability measure $\mathbb{P}\left[X_{[d]}\right]$, the expectation query returns the expectation of that random variable. Expectation queries are further contractions of marginal queries with the identity function restricted on the image of $q$, since

$$\mathbb{E}\left[q\right] = \left\langle \mathbb{P}\left[Y_q\right], \mathrm{Id}|_{\mathrm{im}(q)} Y_q \right\rangle_{[\varnothing]} .$$

This contraction equation follows from the more general Cor. **??**, which will be shown in Chapter 7.

Expectation queries compute mean parameters to a distribution, see Def. **??**. Given a statistic $t$ and a distribution $\mathbb{P}\left[X_{[d]}\right]$, each feature $t_\ell$ for $\ell \in [p]$ corresponds with a mean parameter

$$\mathbb{E}\left[q\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], t_\ell \right\rangle_{[\varnothing]} = \mu\left[L = \ell\right] .$$

### 5.6.2 Mode Queries

A different kind of queries are mode queries, which we formalize by the searches of the indices to maximal coordinates in a tensor.

**Definition 5.15** Given a tensor $\tau\left[X_{[d]}\right]$ the mode query is the problem

$$\mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \tau\left[X_{[d]} = x_{[d]}\right] . \tag{5.2}$$

We can pose mode queries as convex optimization problems. To this end we recall, that the set of all probability distributions is a convex hull of the one-hot encoded states, that is

$$\Lambda^{\delta,\mathrm{MAX},\mathbb{I}} = \mathrm{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \,:\, x_{[d]} \in \bigtimes_{k\in[d]}[m_k]\right) .$$

With this we have

$$\begin{aligned}\max_{x_{[d]}} \tau\left[X_{[d]} = x_{[d]}\right] &= \max_{x_{[d]}} \left\langle \tau\left[X_{[d]}\right], \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\ &= \max_{\mu\left[X_{[d]}\right]\in\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}} \left\langle \tau\left[X_{[d]}\right], \mu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .\end{aligned}$$

We note that the maximization over $\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}$ is a convex optimization problem and the maxima are taken at

$$\mathrm{argmax}_{\mu\left[X_{[d]}\right]\in\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}} \left\langle \tau\left[X_{[d]}\right], \mu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \mathrm{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \,:\, x_{[d]} \in \mathrm{argmax}_{x_{[d]}} \tau\left[X_{[d]} = x_{[d]}\right]\right) .$$

We will further apply this generic trick to approach mode queries when studying inference problems for exponential families in the following sections.

Let us note, that we can also approach modified mode queries, where we restrict to $\mathcal{U} \subset \bigtimes_{k\in[d]}[m_k]$, namely

$$\mathrm{argmax}_{x_{[d]}\in\mathcal{U}} \tau\left[X_{[d]} = x_{[d]}\right] .$$

We can choose the base measure by the subset encoding (see Chapter 7) of $\mathcal{U}$, namely

$$\nu\left[X_{[d]}\right] = \sum_{x_{[d]}\in\mathcal{U}} \epsilon_{x_{[d]}}\left[X_{[d]}\right] ,$$

and conclude that

$$\operatorname{argmax}_{\mu\left[X_{[d]}\right]\in\Lambda^{\delta,\mathrm{MAX},\nu}}\left\langle\tau\left[X_{[d]}\right],\mu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}=\operatorname{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right]\,:\,x_{[d]}\in\operatorname{argmax}_{x_{[d]}\in\mathcal{U}}\tau\left[X_{[d]}=x_{[d]}\right]\right)\,.$$

### 5.6.3 Energy Representations

For exponential families (see Sect. 13.1) we have observed, that often energy tensors have feasible tensor network representations, whereas the corresponding probability distributions can get infeasible. We therefore investigate here schemes to answer queries based on the energy tensor instead of the distribution.

**Theorem 5.15** *Let $\phi\left[X_{[d]}\right]$ be an energy tensor and $\mathbb{P}\left[X_{[d]}\right]=\left\langle\exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$ the corresponding distribution. For disjoint subsets $A,B\subset[d]$ with $A\cup B=[d]$ and any $x_B$ we have*

$$\mathbb{P}\left[X_A\middle|X_B=x_B\right]=\left\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\right\rangle_{[X_A|\varnothing]}\,.$$

*Proof.* To show the theorem, we use a generic simplification property of coordinatewise transforms, which we will show as Lem. **??** in Chapter **??** and get

$$\left\langle\exp\left[\phi\left[X_A,X_B\right]\right]\right\rangle_{[X_A,X_B=x_B]}=\left\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\right\rangle_{[X_A]}$$

Based on this we get

$$\begin{aligned}\mathbb{P}\left[X_A\middle|X_B=x_B\right]&=\left\langle\exp\left[\phi\left[X_A,X_B\right]\right]\right\rangle_{[X_A|X_B=x_B]}\\&=\frac{\left\langle\exp\left[\phi\left[X_A,X_B\right]\right]\right\rangle_{[X_A,X_B=x_B]}}{\left\langle\exp\left[\phi\left[X_A,X_B\right]\right]\right\rangle_{[X_B=x_B]}}\\&=\frac{\left\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\right\rangle_{[X_A]}}{\left\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\right\rangle_{[\varnothing]}}\\&=\left\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\right\rangle_{[X_A|\varnothing]}\,.\square\end{aligned}$$

Importantly, Thm. 5.15 does not generalize to situations, where $A\cup B\neq[d]$, since summation over the indices of the variables $[d]/A\cup B$ and contraction do not commute. In this more generic situation, we would need to sum over exponentiated coordinates, that is

$$\mathbb{P}\left[X_A\middle|X_B=x_B\right]=\left\langle\sum_{x_{[d]/A\cup B}\in[m_{[d]/A\cup B}]}\exp\left[\phi\left[X_A,X_B=x_B,X_{[d]/A\cup B}=x_{[d]/A\cup B}\right]\right]\right\rangle_{[X_A|\varnothing]}\,.$$

Mode queries on probability distributions in an energy representation can always be reduces to mode queries on the energy tensor. This is due to the monotonicity of the exponential function, which implies

$$\begin{aligned}\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]}=x_{[d]}\right]&=\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\left\langle\exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}=x_{[d]}|\varnothing\right]}\\&=\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\exp\left[\phi\left[X_{[d]}=x_{[d]}\right]\right]\\&=\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\phi\left[X_{[d]}=x_{[d]}\right]\,.\end{aligned}$$

Since we are only interested in identifying the index of the maximum coordinate, and not its value, we have further dropped the normalization term by partition functions. When one instead need to get the value of the maximal, the partition function cannot be ignored.

## 5.7 Sampling

Let us here investigate how to draw samples from a probability distribution, based on queries on it. Naive methods, such as drawing a random number in $[0,1]$, adding iteratively the coordinates and stopping when the sum exceeds the random

variables, are infeasible when having large tensor orders causing exponential increases of the coordinate number. We recall, that the number of coordinates of $\mathbb{P}\left[X_{[d]}\right]$ is $\prod_{k\in[d]} m_k$, which increases exponentially in the number $d$ of the variables. Efficient methods instead have to exploit tensor network decompositions of the decompositions.

### 5.7.1 Forward Sampling

The first insight to derive efficient sampling algorithms is to sample a single variable in each step. Forward sampling (see Algorithm 5) exploits to this end the generic chain decomposition (see Thm. 5.5) of a probability distribution, namely

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}\left[X_0\right]\} \cup \left\{\mathbb{P}\left[X_k \middle| X_0, \dots, X_{k-1}\right] \,:\, k \in [d],\, k \geq 1\right\}\right\rangle_{\left[X_{[d]}\right]},$$

It then samples iteratively a state $x_k$ for the variable $X_k$ conditioned on the previously sampled states, that is from the conditonal distribution

$$\mathbb{P}\left[X_k \middle| X_{[k]} = x_{[k]}\right].$$

The generic chain decomposition thereby ensures that probability of getting a state $x_{[d]}$ by this procedure coincides with $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$.

**Algorithm 5** Forward Sampling

**Require:** Probability distribution $\mathbb{P}$
**Ensure:** Exact sample $x_0, \dots, x_{d-1}$ of $\mathbb{P}$

**for all** $k \in [d]$ **do**
Draw $x_k \in [m_k]$ from the conditional distribution

$$\mathbb{P}\left[X_k \middle| X_{[k]} = x_{[k]}\right]$$

**end for**
**return** $x_0, \dots, x_{d-1}$

Forward Sampling is especially efficient, when sampling from a Bayesian Network respecting the topological order of its nodes. More technically, when the parents $\mathrm{Pa}(k)$ of a node $k$ are contained in the preceding variables $[k]$, we apply the conditional independence assumption (more precisely Thm. 5.8 in combination with Thm. 5.11) to get

$$\mathbb{P}\left[X_k \middle| X_{[k]} = x_{[k]}\right] = \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right].$$

Since this conditional probability coincides with a local tensor in the Bayesian Network, we can avoid to contract the network for preparing the conditional distribution. Different to more general Markov Networks, forward sampling from Bayesian Network can therefore be done efficiently by reduction to conditional queries answerable using local tensors. We note that it is important to sample in the topological order induced by the underlying directed hypergraph, since the computation of generic conditional distributions is also for Bayesian Networks NP-hard (see Chapter 13 in [45]). Sampling along the topological variable order requires only tractable to answer conditional queries on the Bayesian Network.

### 5.7.2 Gibbs Sampling

While we have seen that forward sampling can be performed efficiently on Bayesian Networks, Gibbs sampling can be also performed efficiently for Markov Networks. Gibbs sampling Algorithm 6 overcomes the intractability problems of sampling steps during forward sampling at the expense of repetitions of the sampling step. When performing finite repetitions, Gibbs sampling in general samples from an approximate distribution to the one desired. It can be shown,

that these approximate distribution tend to one desired in the asymptotic limit of infinite repetitions of the sampling step (see Chapter 12 in [45]).

**Algorithm 6** Gibbs Sampling

**Require:** Probability distribution $\mathbb{P}$
**Ensure:** Approximative sample $x_0, \ldots, x_{d-1}$ of $\mathbb{P}$

**for all** $k \in [d]$ **do**
  Draw $x_k$ from an initialization distribution.
**end for**
**while** Stopping criterion is not met **do**
  **for all** $k \in [d]$ **do**
    Draw $x_k$ from the conditional distribution

$$\mathbb{P}\left[X_k \middle| X_{[d]/\{k\}} = x_{[d]/\{k\}}\right]$$

  **end for**
**end while**
**return** $x_0, \ldots, x_{d-1}$

The central problem of forward sampling on Markov Networks has been the need for global contractions to answer the required conditional queries, which originates from large numbers of variables to be marginalized out. When avoiding the marginalization of variables, and conditioning on them instead, global contractions can be avoided. To be more precise, for any tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G} = ([d], \mathcal{E})$ and any $k \in [d]$ we have

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_k, X_{[d]/\{k\}}]} = \left\langle \{\tau^e \left[X_k, X_{e/\{k\}} = x_{e/\{k\}}\right] \,:\, e \in \mathcal{E},\, k \in e\} \right\rangle_{[X_k]} \cdot \prod_{e \in \mathcal{E},\, k \notin e} \tau^e \left[X_e = x_e\right] .$$

As a consequence, we get for the Markov Network $\mathbb{P} = \mathbb{P}^{\mathcal{G}}$ to $\tau^{\mathcal{G}}$, that

$$\begin{aligned} \mathbb{P}\left[X_k \middle| X_{[d]/\{k\}} = x_{[d]/\{k\}}\right] &= \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_k, X_{[d]/\{k\}} | \varnothing]} \\ &= \left\langle \{\tau^e \left[X_k, X_{e/\{k\}} = x_{e/\{k\}}\right] \,:\, e \in \mathcal{E},\, k \in e\} \right\rangle_{[X_k|\varnothing]} . \end{aligned}$$

The conditional queries on a Markov Network asked in Gibbs Sampling can therefore be answered by contractions only of those tensors containing the variable $X_k$. To find further locally answerable conditional queries, we need to condition only on the neighbored variables, referred to as Markov blanket, such that the other variables are conditionally independent. This follows from the characterization of the conditional independences in Markov Networks, which has been shown in Thm. 5.10, and can be used to design further tractable sampling schemes for Markov Networks.

We can further answer these conditional queries efficiently, when we perform Gibbs sampling on a probability distribution in an energy representation, that is $\mathbb{P}\left[X_{[d]}\right] = \left\langle \exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$. Using Thm. 5.15, we have

$$\mathbb{P}\left[X_k \middle| X_{[d]/\{k\}} = x_{[d]/\{k\}}\right] = \left\langle \exp\left[\phi\left[X_k, X_{[d]/\{k\}} = x_{[d]/\{k\}}\right]\right]\right\rangle_{[X_A|\varnothing]}$$

We note, that the main property of the conditional query exploited here, is that all variables but the one sampled appear as a condition and none is marginalized out. In the scheme of forward sampling, where most of the variables are marginalized out in many queries, we cannot apply this trick and would have to perform sums over exponentiated coordinates to the variables marginalized out.

### 5.7.3 Simulated Annealing

Simulated annealing is an adapted sampling scheme targeting mode queries rather than generating representative samples from a distribution. It employs an annealing process that gradually transforms the probability distribution by increasingly favoring high-likelihood configurations, thereby improving the chances of sampling a solution to a mode query. To

be more precise, let there be a distribution in energy representation, that is $\mathbb{P}\left[X_{[d]}\right] = \left\langle \exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$. We introduce a parameter $\beta \in \mathbb{R}$, which we understand as the inverse temperature, and anneal the distribution through scaling its energy by this parameter. In the limit of $\beta \to \infty$, for each state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ the annealed distribution behaves as

$$\left\langle \exp\left[\beta\cdot\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}=x_{[d]}|\varnothing\right]} \to \left\langle \mathbb{I}_{\mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\phi\left[X_{[d]}=x_{[d]}\right]}\right\rangle_{\left[X_{[d]}=x_{[d]}|\varnothing\right]} .$$

In this limit, the annealed distribution tends to the uniform distribution of the maximal coordinates, that is the uniform distribution of the set

$$\mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\phi\left[X_{[d]} = x_{[d]}\right] = \mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]} = x_{[d]}\right] .$$

i To integrate annealing into Gibbs sampling, one chooses a parameter $\beta$ for each repetition of a sampling step and sample from the conditioned annealed distribution $\left\langle \exp\left[\beta\cdot\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$. Through increasing $\beta$ during the algorithm, the samples are drawn towards states with larger coordinates in $\mathbb{P}\left[X_{[d]}\right]$. However, when $\beta$ is large, the sampling procedure can get stuck in local maxima, whereas small $\beta$ are in favor of overcoming such. The inverse temperature is thus understood as a tradeoff parameter between the exploration of new regions of the state space and increasing the coordinate of the sample by local coordinate optimization. It is therefore typically chosen low in the beginning of the sampling algorithm and then sequentially increased to find maximal coordinates. Due to this typical increasement of the inverse temperature strategy, the algorithm is referred to as simulated annealing.

## 5.8 Maximum Likelihood Estimation

So far we have been concerned with deductive reasoning task, that is retrieve information from a given distribution or drawing a sample. We now turn to inductive reasoning tasks, where a probability distribution is estimated given data. To present the generic framework of maximum likelihood estimation in the tensor network contraction formalism, we introduce the likelihood loss exploiting the structure of empirical distribution, and then provide interpretations in terms of entropies.

### 5.8.1 Empirical Distributions

To prepare for reasoning on data, we now derive tensor network representation for empirical distributions, which are defined based on observed states $\left((x_0^j,\ldots,x_{d-1}^j) \,:\, j\in[m]\right)$ of a factored system.

**Definition 5.16** Given a dataset $\left((x_0^j,\ldots,x_{d-1}^j) \,:\, j\in[m]\right)$ of samples of the factored system we define the sample selector map

$$D \,:\, [m] \to \bigtimes_{k\in[d]}[m_k]$$

elementwise by

$$D(j) = \left(x_0^j,\ldots,x_{d-1}^j\right) .$$

The empirical distribution to the sample selector map $D$ is the probability distribution

$$\mathbb{P}^D\left[X_{[d]}\right] := \left\langle \beta^D\left[X_{[d]},J\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} ,$$

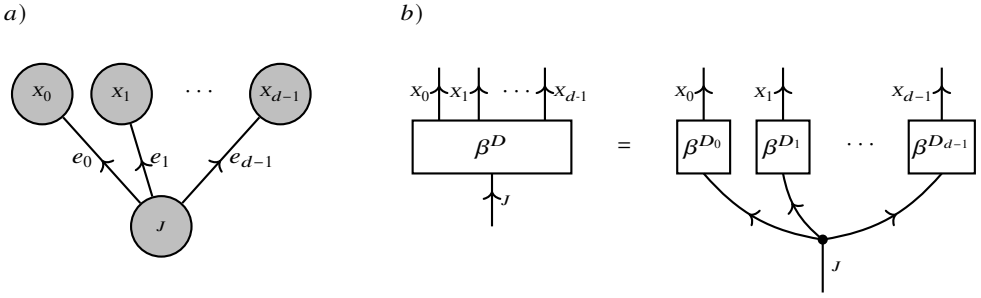



**Fig. 5.10** Decomposition of the basis encoding of a sample selector map to a dataset $\left((x_0^j, \ldots, x_{d-1}^j) : j \in [m]\right)$. a) Interpretation as a sample selection variable $J$ selecting states for the variables $X_{[d]}$ according to the enumerated dataset. b) Corresponding decomposition of the basis encoding $\beta^D$ into a tensor network in the basis CP Format (see Sect. 10.1.2), where $\tau^{e_k} = \beta^{D_k}$.

where we introduced as single incoming for the basis encoding of the sample selector map the sample selecting variable $J$ taking values in $[m]$.

The basis encoding of the sample selector map has a decomposition by

$$\beta^D \left[X_{[d]}, J\right] = \sum_{j \in [m]} \epsilon_{x_0^j, \ldots, x_{d-1}^j} \left[X_{[d]}\right] \otimes \epsilon_j \left[J\right] .$$

Each coordinate $x_{[d]}$ of the empirical distribution can thus be calculated by

$$\begin{aligned} \mathbb{P}^D \left[X_{[d]} = x_{[d]}\right] &= \frac{1}{\left\langle \beta^D \right\rangle_{[\varnothing]}} \left( \sum_{j \in [m]} \epsilon_{x_0^j, \ldots, x_{d-1}^j} \left[X_{[d]} = x_{[d]}\right] \right) \\ &= \frac{\left| \left\{ j \in [m] : (x_0^j, \ldots, x_{d-1}^j) = (x_0, \ldots, x_{d-1}) \right\} \right|}{m} . \end{aligned}$$

We can therefore interpret each coordinate of the empirical distribution as the relative frequency of the corresponding state in the observed data.

The basis encoding of the sample selector map is a sum of one-hot encodings of the data indices and the corresponding sample states. Such sums of basis tensors will be further investigated in Sect. 10.1.2 as basis CP decompositions. We now exploit this structure to find efficient tensor network decompositions (see Figure 5.10) based on matrices encoding its variables.

**Theorem 5.16** *Given a data map $D : [m] \to \bigtimes_{k \in [d]} [m_k]$ we define for $k \in [d]$ its coordinate maps*

$$D_k : [m] \to [m_k]$$

*by*

$$D_k(j) = x_k^j .$$

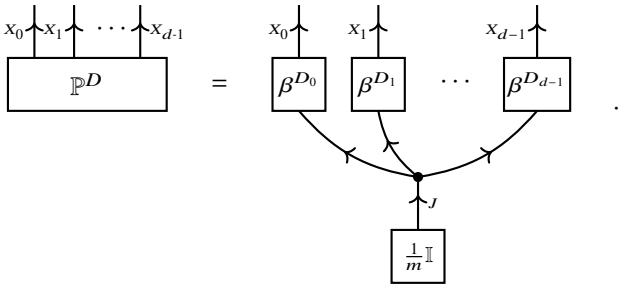
*We then have*

$$\beta^D \left[X_{[d]}, J\right] = \left\langle \{\beta^{D^k} \left[X_k, J\right] : k \in [d]\} \right\rangle_{\left[X_{[d]}, J\right]}$$

*and*

$$\mathbb{P}^D \left[X_{[d]}\right] = \left\langle \beta^D \left[X_{[d]}, J\right], \frac{1}{m} \mathbb{I} \left[J\right] \right\rangle_{\left[X_{[d]}\right]} = \left\langle \beta^{D_0} [X_0, J], \ldots, \beta^{D_{d-1}} [X_{d-1}, J], \frac{1}{m} \mathbb{I} \left[J\right] \right\rangle_{\left[X_{[d]}\right]} .$$

*In a contraction diagram this decomposition is represented as*




*Proof.* The first claim is a special case of Thm. 10.9, to be shown in Chapter 7. To show the second claim we notice

$$\left\langle \beta^D \right\rangle_{[\varnothing]} = \sum_{j \in [m]} \left\langle \beta^D \left[ X_{[d]}, J = j \right] \right\rangle_{[\varnothing]} = m \,.$$

With the first claim it now follows that

$$\begin{aligned} \mathbb{P}^D \left[ X_{[d]} \right] = \left\langle \beta^D \right\rangle_{[X_{[d]}|\varnothing]} &= \frac{\left\langle \beta^D \right\rangle_{[X_{[d]}]}}{\left\langle \beta^D \right\rangle_{[\varnothing]}} \\ &= \left\langle \left\{ \beta^{D^k} \left[ X_k, J \right] \,:\, k \in [d] \right\} \cup \left\{ \frac{1}{m} \mathbb{I} \left[ J \right] \right\} \right\rangle_{[X_{[d]}]} . \square \end{aligned}$$

The cores $\beta^{D_k}$ are matrices storing the value of the categorical variable $X_k$ in the sample world indexed by $j$.

From the proof of Thm. 5.16 we notice that the scalar $\frac{1}{m}$ could be assigned with any core in a representation of $\mathbb{P}^D$, and the core $\mathbb{I}\left[J\right]$ is thus redundant in the contraction representation. However, creating the core $\frac{1}{m}\mathbb{I}\left[J\right]$ provides us with a simple interpretation of the empirical distribution. We can understand $\frac{1}{m}\mathbb{I}\left[J\right]$ as the uniform probability distribution over the samples, which is by the map $D$ forwarded to a distribution over $\bigtimes_{k\in[d]}[m_k]$. The one-hot encoding of each sample is itself a probability distribution, which is understood as conditioned on the respective state of the sample selection variable $J$. The conditional distribution $\beta^D$ therefore forwards the uniform distribution of the samples to a distribution of the variables $X_{[d]}$. In the perspective of a Bayesian Network (see Figure 5.10), the variable $J$ serves as single parent for each categorical variable $X_k$.

### 5.8.2 Likelihood Loss

The likelihood of a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ to produce an observed sample is

$$\mathbb{P}\left[X_{[d]} = D(j)\right] \,.$$

We further introduce the likelihood of $\mathbb{P}\left[X_{[d]}\right]$ with respect to a dataset as

$$\mathbb{P}\left[\left((x_0^j, \ldots, x_{d-1}^j) \,:\, j \in [m]\right)\right] := \prod_{j\in[m]} \mathbb{P}\left[X_{[d]} = D(j)\right] \,.$$

The likelihood draws on the assumption, that each datapoint in the dataset has been drawn independently from the same distribution. When this generating distribution coincides with $\mathbb{P}\left[X_{[d]}\right]$, then the probability of generating a dataset by this scheme is the likelihood. In inductive reasoning, the true distribution $\mathbb{P}\left[X_{[d]}\right]$ is unknown and needs to be approximatively estimated based on data and a learning hypothesis. We will therefore compute and compare the likelihood of distributions, which in general do not coincide with the distribution generating the data. It is thus important to not understand the likelihood as a probability, which is only true for the generating distribution, as pointed out in Chapter 2 in [51].

Let us now transform the likelihood to find an representation involving the empirical distribution (see Def. 5.16), for which efficient tensor network decompositions have been derived in Thm. 5.16. Applying a scaled logarithm we get

$$\begin{aligned}\frac{1}{m}\cdot\ln\left[\mathbb{P}\left[\left((x_0^j,\ldots,x_{d-1}^j)\ :\ j\in[m]\right)\right]\right] &= \frac{1}{m}\cdot\ln\left[\prod_{j\in[m]}\mathbb{P}\left[X_{[d]}=D(j)\right]\right]\\ &= \frac{1}{m}\sum_{j\in[m]}\ln\left[\mathbb{P}\left[X_{[d]}=D(j)\right]\right]\\ &= \left\langle\ln\left[\mathbb{P}\left[X_{[d]}\right]\right],\mathbb{P}^D\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\ .\end{aligned}$$

Let us notice, that this transform of the likelihood is monotoneous and therefore does not influence the position of the maximum, when optimizing the likelihood. Motivated by this property, we use the transformed form to define the loss-likelihood loss and maximum likelihood estimation.

**Definition 5.17** The log-likelihood loss of a distribution $\mathbb{P}$ given a dataset

$$\left((x_0^j,\ldots,x_{d-1}^j)\ :\ j\in[m]\right)$$

is the functional

$$\mathcal{L}_D\left(\mathbb{P}\right)=-\left\langle\ln\left[\mathbb{P}\left[X_{[d]}\right]\right],\mathbb{P}^D\left[X_{[d]}\right]\right\rangle_{[\,]}\ .$$

Having a hypothesis $\Gamma\subset\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}$, that is a set of probability distributions, the maximum likelihood estimation is the problem

$$\mathrm{argmin}_{\mathbb{P}\in\Gamma}\mathcal{L}_D\left(\mathbb{P}\right)\ . \tag{$\mathrm{P}^{\mathrm{M}}_{\Gamma,\mathbb{P}^D}$}$$

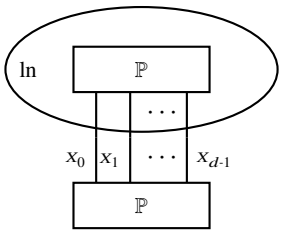
### 5.8.3 Entropic Interpretation

The Shannon entropy, which has been introduced in the seminal paper [79], is a foundational concept in various research fields beyond statistical learning, such as information theory or statistical physics. While a detailed discussion is out of the scope of this work, we here only provide computation schemes of the entropy based on contractions of distributions.

**Definition 5.18 (Shannon entropy)** The Shannon entropy of a distribution $\mathbb{P}\left[X_{[d]}\right]$ is the quantity

$$\mathbb{H}\left[\mathbb{P}\right]=\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]}=x_{[d]}\right]\cdot\left(-\ln\left[\mathbb{P}\left[X_{[d]}=x_{[d]}\right]\right]\right)=\left\langle\mathbb{P},-\ln\left[\mathbb{P}\right]\right\rangle_{[\varnothing]}\ .$$

We represent the Shannon entropy by the tensor network diagram

$\mathbb{H}\left[\mathbb{P}\right]\ =$ 


where we denote a coordinatewise transform by the logarithm as an ellipsis (see Sect. **??**).

We here make the convention $\ln\left[0\right]=-\infty$ and $0\cdot\ln\left[0\right]=0$, to have the Shannon entropy well-defined for distributions with non-trivial support.

Among the distributions in the same tensor space, the uniform distribution maximizes the Shannon entropy

$$\mathbb{H}\left[\langle\mathbb{I}\rangle_{[X_{[d]}|\varnothing]}\right]=\sum_{k\in[d]}\ln\left[m_k\right]$$

and the one-hot encodings to states $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ minimize the Shannon entropy

$$\mathbb{H}\left[\epsilon_{x_{[d]}}\left[X_{[d]}\right]\right] = 0\,.$$

The Shannon entropy measures the information content of a distribution and is therefore a central tool for regularization in statistical learning (see for an introduction Chapter 2 in [51]). We therefore exploit this information content as a regularizer to identify a distribution among those coinciding in the answer to a collection of expectation queries. To be more precise, let there be for $\ell \in [p]$ query tensors $q^\ell$ (see Def. 5.14) and $\mathbb{P}^D$ an empirical distribution. The problem of maximal entropy with respect to coinciding expectation queries with $\mathbb{P}^D$ is then posed as

$$\text{argmax}_{\mathbb{P}\in\Lambda^{\delta,\text{MAX},\mathbb{I}}}\mathbb{H}\left[\mathbb{P}\right] \quad \text{subject to} \quad \forall \ell \in [p] : \left\langle \mathbb{P}, q^\ell \right\rangle_{[\varnothing]} = \left\langle \mathbb{P}^D, q^\ell \right\rangle_{[\varnothing]}$$

where $\Lambda^{\delta,\text{MAX},\mathbb{I}}$ is the set of probability distributions given a factored system. We study instances of this maximal entropy problem later in Sect. **??**, where we show that its solution is a member of the exponential family, which statistic is build by the query tensors. We will further provide connections between the problems of maximal entropy and maximum likelihood estimation.

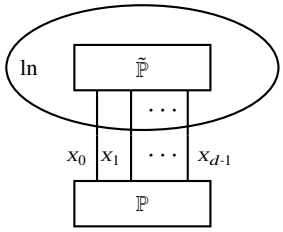
While the Shannon entropy is a property of a single distribution, the cross-entropy is a straight forward generalization towards pairs of distributions. We first introduce this quantity and then interpret the log-likelihood loss based on the cross-entropy.

**Definition 5.19 (Cross entropy and Kullback Leibler divergence)** The cross-entropy between two distributions $\mathbb{P}\left[X_{[d]}\right]$ and $\tilde{\mathbb{P}}[X_{[d]}]$ defined with respect to the same factored system is the quantity

$$\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] = \sum_{x_0,\dots,x_{d-1}} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \left(-\ln\left[\tilde{\mathbb{P}}[X_{[d]} = x_{[d]}]\right]\right) = \left\langle \mathbb{P}, -\ln\left[\tilde{\mathbb{P}}\right]\right\rangle_{[\varnothing]}\,.$$

The cross-entropy is captured by the tensor network diagram

$\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right]$ =




The Kullback-Leiber divergence or relative entropy between $\mathbb{P}\left[X_{[d]}\right]$ and $\tilde{\mathbb{P}}[X_{[d]}]$ is the quantity

$$\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}||\tilde{\mathbb{P}}\right] = \mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] - \mathbb{H}\left[\mathbb{P}\right]\,.$$

Let us notice, that we have $\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] = \infty$ if and only if there is a state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ such that $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] > 0$ and $\tilde{\mathbb{P}}[X_{[d]} = x_{[d]}] = 0$.

The Gibbs inequality (for a proof see for example Chapter 2 in [18]) states that for any distributions $\mathbb{P}\left[X_{[d]}\right]$ and $\tilde{\mathbb{P}}[X_{[d]}]$ we have

$$\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] \geq \mathbb{H}\left[\mathbb{P}\right]\,,$$

where equality holds if any only if $\mathbb{P} = \tilde{\mathbb{P}}$ This ensures, that the Kullback-Leiber Divergence between any distributions is positive and vanishes if and only if both distributions coincide.

We in the next lemma provide an entropic interpretation of maximum likehlihood estimation as defined in Def. 5.17.

**Lemma 5.2** *The maximum likelihood estimation* $\mathrm{P}^{\mathrm{M}}_{\Gamma,\mathbb{P}^D}$ *is equivalent to the minimization of cross-entropy and Kullback-Leibler divergence, that is*

$$\text{argmin}_{\mathbb{P}\in\Gamma}\mathcal{L}_D\left(\mathbb{P}\right) = \text{argmin}_{\mathbb{P}\in\Gamma}\mathbb{H}\left[\mathbb{P}^D, \mathbb{P}\right] = \text{argmin}_{\mathbb{P}\in\Gamma}\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}^D||\mathbb{P}\right]\,.$$

*Proof.* Comparing the log-likelihood loss in Def. 5.17 with the cross-entropy in Def. 5.19, we get

$$\mathcal{L}_D\left(\mathbb{P}\right) = \mathbb{H}\left[\mathbb{P}^D, \mathbb{P}\right]$$

which established the equivalence of maximum likelihood estimation and cross-entropy minimization. Further, since

$$\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}^D || \mathbb{P}\right] = \mathbb{H}\left[\mathbb{P}^D, \mathbb{P}\right] - \mathbb{H}\left[\mathbb{P}^D\right]$$

and $\mathbb{H}\left[\mathbb{P}^D\right]$ is a constant offset in the objective, maximum likelihood estimation is equivalent to the minimization of the Kullback-Leibler divergence. □

More general than Maximum Likelihood Estimation, we define the moment projection of an arbitrary distribution $\mathbb{P}^*$ onto a set $\Gamma$ of probability distributions as the problem

$$\mathrm{argmin}_{\mathbb{P}\in\Gamma}\mathbb{H}\left[\mathbb{P}^*, \mathbb{P}\right] . \tag{$\mathrm{P}^{\mathrm{M}}_{\Gamma,\mathbb{P}^*}$}$$

With Lem. 5.2 we have established, that maximum likelihood estimation is the moment projection of the empirical distribution onto a set $\Gamma$.

We further define the information projection an arbitrary distribution $\mathbb{P}^*$ onto a set $\Gamma$ of probability distributions as the problem

$$\mathrm{argmin}_{\mathbb{P}\in\Gamma}\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}||\mathbb{P}^*\right] . \tag{$\mathrm{P}^{\mathrm{I}}_{\Gamma,\mathbb{P}^*}$}$$

The relative entropy is not symmetric, thus the information projection ($\mathrm{P}^{\mathrm{I}}_{\Gamma,\mathbb{P}^*}$) and the moment projections do not coincide in general. The differences of both are discussed in Chapter 8 in [45].

# Chapter 6
# Causality

**Abstract** We here describe how to represent Causal AI models in the tnreason formalism. In particular, we will extend the Bayesian network formalism to capture intervention and counterfactual queries.

We find tensor network models for the different rungs of the *ladder of causation* [65]. The notes are organized to follow the three rungs of the ladder:

- **Rung 1: Association** – captured by Bayesian Networks
- **Rung 2: Intervention** – captured by do-calculus, for which we introduce intervention variables
- **Rung 3: Counterfactuals** – captured by functional causal models (see Chapter 21 in [45]) involving response variables

Following the formalism of Chapter 21 in [45]:

- Mutilation of the Bayesian network is captured by contractions with one-hot encodings to do-variables $D_k$.
- Intervention queries are formalized by contractions with one-hot encodings to all do-variables
- Response variables are formalized by selection variables
- Counterfactual queries are modeled as intervention queries on counterfactual twin networks (see [45, Chapter 21])

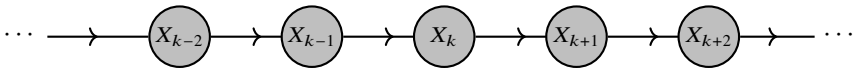
## 6.1 Association

See Chapter 5: Bayesian Networks are tensor networks and probabilistic queries can be answered by their contractions. We here provide more theory on query simplification and d-separation, to prepare for the study of intervention mechanisms.

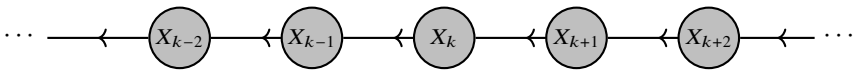
*Example 6.1 (Markov chain)*

Markov chains can be parametrized in two directions:

a) Past-to-future representation:




b) Future-to-past representation:




Both parametrizations represent the same distribution, and encode the same conditional independence statements. However, intuitively the future is influenced by the past, so we would prefer the parametrization $a$). We will observe in the following (see Example 6.3) how expanding the model through intervention mechanisms will provide us a method to distinguish between both parametrizations.

### 6.1.1 Simplification using directed contractions

**Lemma 6.1** *Let $X_{[d]}$ be an enumeration of the variables respecting a topological order on the directed acyclic hypergraph $\mathcal{G}$. For any $\tilde{k} < k$ we build the sets $[k]\backslash[\tilde{k}] = \{\tilde{k}, \tilde{k}+1, \dots, k-1\}$ and $\mathcal{U} = \left(\bigcup_{l\in[k]\backslash[\tilde{k}]} \mathrm{Pa}(l)\right) \cap [\tilde{k}]$. Then*

$$\mathbb{P}\left[X_{[k]\backslash[\tilde{k}]}\middle|X_{\mathcal{U}}\right] = \left\langle \{\mathbb{P}\left[X_l\middle|X_{\mathrm{Pa}(l)}\right] \,:\, l \in [k]\backslash[\tilde{k}]\}\right\rangle_{\left[X_{[k]\backslash[\tilde{k}]}, X_{\mathcal{U}}\right]}$$

*Proof.* By construction we have for $l \in [k]\backslash[\tilde{k}]$ the independencies

$$\left(X_l \perp X_{[l]\backslash([\tilde{k}]\cup\mathrm{Pa}(l))}, X_{\mathcal{U}\backslash\mathrm{Pa}(l)}\right) \,\middle|\, X_{\mathrm{Pa}(l)}$$

and therefore

$$\mathbb{P}\left[X_l\middle|X_{\mathrm{Pa}(l)}\right] \otimes \mathbb{I}\left[X_{[l]\backslash([\tilde{k}]\cup\mathrm{Pa}(l))}\right] \otimes \mathbb{I}\left[X_{\mathcal{U}\backslash\mathrm{Pa}(l)}\right] = \mathbb{P}\left[X_l\middle|X_{([l]\backslash[\tilde{k}])\cup\mathcal{U}^l}\right].$$

It follows that

$$\left\langle \{\mathbb{P}\left[X_l\middle|X_{\mathrm{Pa}(l)}\right] \,:\, l \in [k]\backslash[\tilde{k}]\}\right\rangle_{\left[X_{[k]\backslash[\tilde{k}]}, X_{\mathcal{U}}\right]} = \left\langle \{\mathbb{P}\left[X_l\middle|X_{([l]\backslash[\tilde{k}])\cup\mathcal{U}^l}\right] \,:\, l \in [k]\backslash[\tilde{k}]\}\right\rangle_{\left[X_{[k]\backslash[\tilde{k}]}, X_{\mathcal{U}}\right]} = \mathbb{P}\left[X_{[k]\backslash[\tilde{k}]}\middle|X_{\mathcal{U}}\right]$$

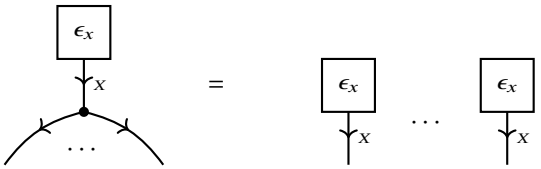
Here we used in the last equation the chain decomposition on the conditioned distribution $\mathbb{P}\left[X_{[k]\backslash[\tilde{k}]}\middle|X_{\mathcal{U}}\right]$. □

### 6.1.2 Statistical influence

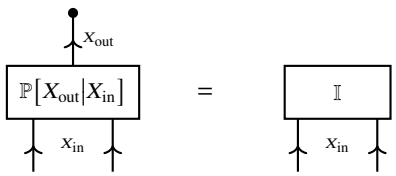
Let us investigate, how the directionality in a Bayesian network hypergraph can be interpreted as statistical influence between variables. To be more precise, we study which cores need to be contracted to answer probabilistic queries, and in such way contribute a statistical influence to a query.

When conditioning on some variables, and marginalizing others, we can simplify the Bayesian network. There are two simplification mechanisms:

(i) When having a basis vector to a closed variable, we can remove the variable and replace all tensors affected by the variable with contractions with a copy of the basis vector. That is, we apply the contraction identity:

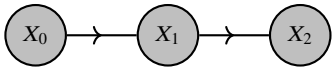



(ii)When a closed variables appears only as an outgoing variable, we can replace the corresponding tensor with the trivial tensor. This corresponds with the contraction identity:



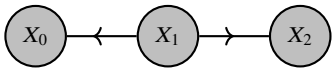

When having a path through the directed hypergraph (which itself does not necessarily respect the directionality), we investigate whether it will be a path in the simplified hypergraph. Consiering a node $X_1$ and its neighbors $X_0$, $X_1$ in a path, there are three different situations [63]:

- **Chains:** $X_1$ appears as outgoing and incoming variable.



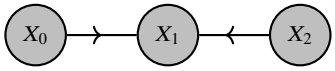

When conditioned on $X_1$, the path closes by rule $(i)$.

- **Forks:** $X_1$ appears only as outgoing variable.




When conditioned on $X_1$, the path also closes by rule $(i)$.

- **Colliders:** $X_1$ appears only as incoming variable.




When $X_1$ is marginalized out and not conditioned on, simplification rule $(ii)$ disconnects the path. When $X_1$ is conditioned on, the path is still active by the tensor $\mathbb{P}\big[X_1 = x_1 \big| X_0, X_2, X_{\mathrm{Pa}(1)/\{0,2\}}\big]$

We use this intuition to define active trails and d-separation, oriented on [45, Def. 3.6 and Def. 3.7].

**Definition 6.1 (Active trails and d-separation)** A trail in a directed hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a enumerated subset of variables $X_{[r]}$ such that for each $s \in [r-1]$ there is a hyperedge $e = (e^{\mathrm{in}}, e^{\mathrm{out}})$ with $X_s \in e^{\mathrm{in}}$ and $X_s \in e^{\mathrm{out}}$ or $X_s \in e^{\mathrm{out}}$ and $X_s \in e^{\mathrm{in}}$. We say that a trail in $\mathcal{G}$ is blocked giving a set $C$, if any collider has a node in $C$ or in $\mathrm{Desc}(C)$ in the middle, and no chain or fork has a node in $C$ in the middle. Two subsets $A$ and $B$ are said to be d-separated given $C$, if any trail from $A$ to $B$ is blocked by $C$.

To show the utility of this definition, let us consider two sets of variables $A, B$ d-separated by $3.C$.

**Theorem 6.1** *If and only if $A, B$ d-separated by $C$ then for any Bayesian network on $\mathcal{G}$*

$$(X_A \perp X_B) \mid X_C \,.$$

*Proof.* Let us first show, that whenever we have $A, B$ which are d-separated by $C$ and a Bayesian network on $\mathcal{G}$, then $(X_A \perp X_B) \mid X_C$ with respect to that Bayesian network. By performing the simplification rules iteratively on the contraction

$$\mathbb{P}\big[X_{A\cup B}\big|X_C = x_C\big]$$

and observing that the tensor network decomposes into a tensor product

$$\mathbb{P}\big[X_{A\cup B}\big|X_C = x_C\big] = \mathbb{P}\big[X_A\big|X_C = x_C\big] \otimes \mathbb{P}\big[X_B\big|X_C = x_C\big] \,.$$

To show the contrary, it is enough construct a Bayesian network not satisfying this independence statement once $A, B$ are not d-separated by $C$. We refer to Thm. 3.4 in [45] for such a construction. □

We already characterized in Chapter 5 Bayesian networks on a hypergraph as those distributions satisfying the independencies

$$\big(X_k \perp X_{[k]\setminus \mathrm{Pa}(k)}\big) \,\big|\, X_{\mathrm{Pa}(k)}$$

with respect to a topological order on $\mathcal{G}$.

## 6.2 Intervention

So far, there are several hypergraphs to model a probability distributions by a Bayesian Network. We now want to understand the directed hyperedges as causal relationships between the variables, beyond encoding of conditional independencies (see Example 6.2). To this end, we model the effects of interventions, which will distinguish between multiple Bayesian Networks modeling the same joint distribution.

*Example 6.2 (Patient treatment outcome)* We model a relationship about the treatment of a sick patient and the outcome of the treatment. Whether a treatment is conducted is modelled by the binary variable $X_T$ and whether the patient improves after the treatment by the variable $X_0$.

There are in general two possibilities of representing the joint distribution of $X_T$ and $X_O$ by a Bayesian Network:

- Treatment directs to outcome:

$$\mathcal{G} = (\{X_T, X_O\}, \{(\emptyset, X_T), (\{X_T\}, \{X_O\})\})$$

- Outcome directs to treatment:

$$\mathcal{G} = (\{X_T, X_O\}, \{(\emptyset, X_T), (\{X_O\}, \{X_T\})\})$$

In the typical intuition, we expect the treatment to have a causal effect on the outcome and not the other way round. From a Bayesian Network perspective, both models are however equivalent and represent the same joint distribution.

Interventions on Bayesian Networks are modeled by the do-calculus [63], which we capture here based on do-variables.

### 6.2.1 Do calculus

We introduce the do-variables $D_k$ taking values in $[m_k + 1]$ for each variable $X_k$ of dimension $m_k$, which model the effects of interventions. The states are interpreted as

- $d_k \in [m_k]$: An intervention setting $X_k$ to the value $d_k$
- $d_k = m_k$: No intervention on $X_k$

**Definition 6.2** The causally augmented conditional probability core is defined as

$$\begin{aligned}\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] =& \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] \otimes \epsilon_{m_k}\left[D_k\right] \\ &+ \sum_{d_k \in [m_k]} \epsilon_{d_k}\left[X_k\right] \otimes \epsilon_{d_k}\left[D_k\right] \otimes \mathbb{I}\left[X_{\mathrm{Pa}(k)}\right] .\end{aligned}$$

The causally augmented Bayesian Network is the tensor network

$$\left\langle \{\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d]\} \right\rangle_{[X_{[d]}, D_{[d]}]} .$$

The Bayesian Networks investigated so far are retrieved by contractions with $\epsilon_{m_k}\left[D_k\right]$. This corresponds with the situation, where no interventions are performed on any variable.

**Lemma 6.2** *For any Bayesian Network $\mathbb{P}\left[X_{[d]}\right]$ on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ we have*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d]\} \cup \{\epsilon_{m_k}\left[D_k\right] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}\right]} .$$

*Proof.* By construction we have

$$\mathbb{P}^c\left[X_k \middle| D_k = m_k, X_{\mathrm{Pa}(k)}\right] = \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] .$$

Performing this on each causally augmented conditional distribution retrieves thus the original Bayesian Network. □

Adding the do-variables comes with a moderate increase of the bas+ rank. We provide the following bound.

**Lemma 6.3** *For any conditional probability distribution we have*

$$\mathrm{rank}^{\mathrm{bas}}\left(\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right]\right) \leq \mathbb{P}\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right]$$

*Example 6.3 (Intervention on a Markov Chain)*

Let us recall the ambiguity of parameterizing a Markov Chain in Example 6.1, where we observed that both past to future and future to past representations result in Bayesian Networks of similar sparsity. Adding do variables to any of these parametrization will result in different interventions (see Figure 6.1) and thus resolves the ambiguity.

Intervening on a variable $X_k$ and putting it into the state $x_k$ is done by contracting with

$$\epsilon_{x_k}\left[D_k\right] .$$

We use that since $x_k \in \mathbb{R}^{m_k}$

$$\left\langle \mathbb{P}^c\left[X_k \middle| X_{k-1}, D_k\right], \epsilon_{x_k}\left[D_k\right]\right\rangle_{[X_k, X_{k-1}]} = \mathbb{I}\left[X_{k-1}\right] \otimes \epsilon_{x_k}\left[X_k\right]$$

and have that the intervention effectively restarts the Markov Chain at the variable $X_k$ (see Figure 6.3a). When we choose the causal augmentation of the future-to-past representation instead, the effect would be differently since the restart would be towards the past (see Figure 6.3b).

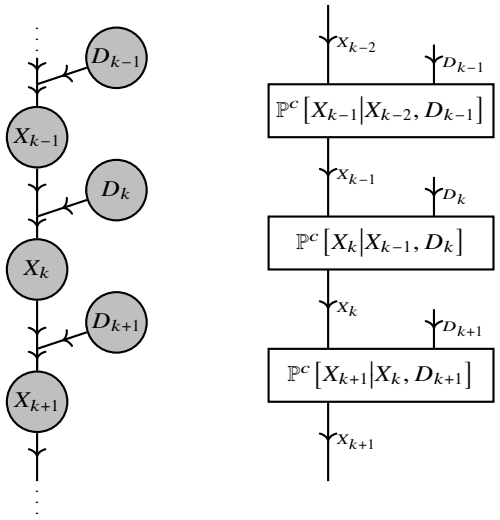



**Fig. 6.1** Extension of a Markov Chain in the past-to-future representation by do variables.

### 6.2.2 Intervention Queries

In a Bayesian Network, when intervening on a variable $X_v$, the corresponding conditional probability gets trivialized. The probability tensor is then captured by

$$\mathbb{P}\left[X_{\mathcal{U}} \middle| \mathrm{do}\left(X_{\mathcal{W}} = x_{\mathcal{W}}\right)\right] = \left\langle \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, v \notin \mathcal{W}\} \cup \{\frac{1}{m_v}\mathbb{I}\left[X_v, X_{\mathrm{Pa}(v)}\right] \,:\, v \in \mathcal{W}\} \cup \{\epsilon_{x_v}\left[X_v\right] \,:\, v \in \mathcal{W}\}\right\rangle_{[X_{\mathcal{U}}]} .$$

Since $\frac{1}{m_v}\mathbb{I}\left[X_v, X_{\mathrm{Pa}(v)}\right]$ is directed with $X_{\mathrm{Pa}(v)}$ incoming and $X_v$ outgoing, the partition function of the network stays 1 and the normalization is captured by the contraction.

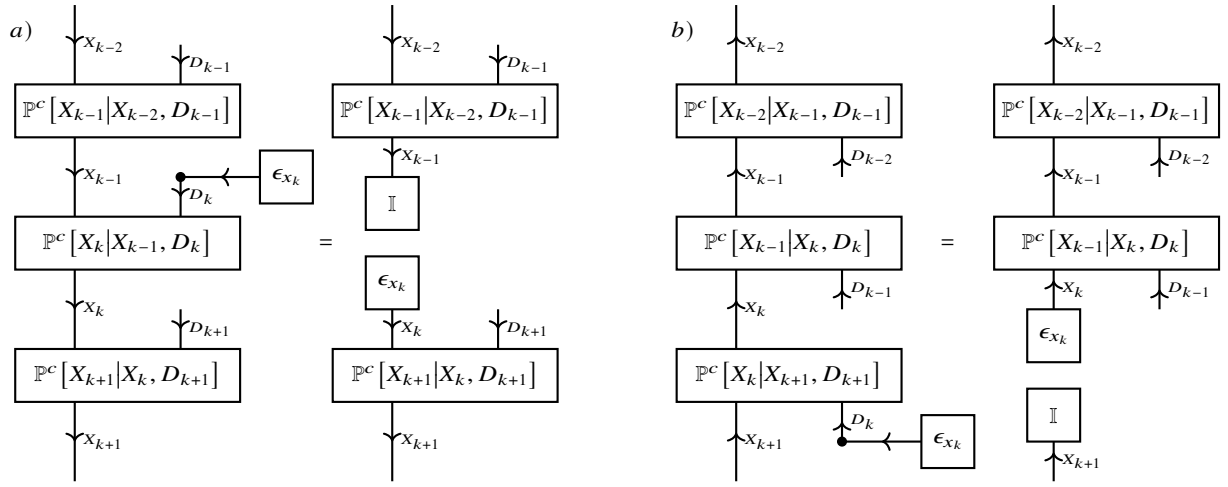



**Fig. 6.2** Effect of intervention on a variable $D_k$: Restarting the Markov chain at the variable $X_k$ in state $x_k$. The effect differs between the past-to-future parameterization a) and the future-to-past parameterization b).

### 6.2.3 Simplifications

In general, when all variables are observable, intervention queries can be expressed by a collection of conditional queries (since any cpd of a Bayesian network is a conditional query, and we contract all except those with interventions on the head variables). In realistic scenarios, however, not all variables are observable (for example, the smoking gene has been only assumed, but no observations have been made). We are interested in situations, where intervention queries can be answered by conditional queries of observable variables. We can proof them in the tensor network formalism based on network separations, which contribution to contractions are scalar multiplications, which therefore are dropped in normalizations.

An example for a simplification is given by the randomized controlled trial.

*Example 6.4 (Randomized controlled trial)* The randomized controlled trial is a joint distribution of variables

- Confounding variable $X_C$ summarizing other factors affecting the outcome.
- Treatment assignment $X_T$ with values in $\{0, 1\}$. Here, 1 indicates assignment to the treatment group and 0 to the control group. The experiment is conducted such that the treatment variable is independent of the confounding variable $X_C$.
- Outcome $X_O$ .

We want to estimate the causal effect of the treatment on the outcome, namely calculate the intervention query

$$\mathbb{P}\left[X_O|D_T = 1\right].$$

By design of the randomized control, the confounding variable has no causal effect on $X_T$. Thus, we have

$$\mathbb{P}\left[X_O|D_T = 1\right] = \mathbb{P}\left[X_O|X_T = 1\right]$$

that is, the intervention query is equal to the conditional probability (see Figure 6.3).

If the treatment variable would depend on the confounder (see Figure 6.4), the effect of the intervention can not be identified based on conditional probabilities, when not measuring the confounder $X_C$.

Compared to the conditional queries studied in Sect. 6.1, interventions create further simplification possibilities.

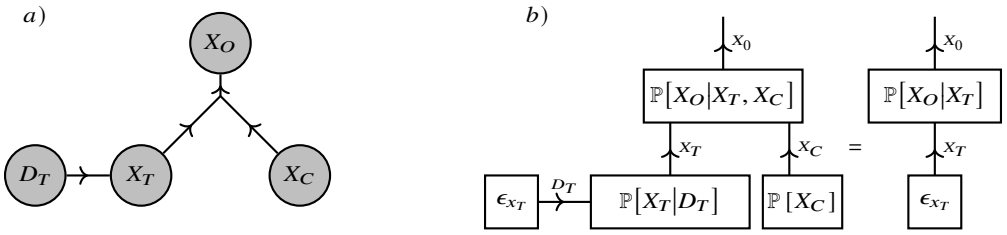



**Fig. 6.3** Model of a Randomized Controlled Trial: The confounding variable $X_C$ summarizes all influenced on the outcome $X_O$ other than the treatment variable $X_T$. a) b) Effect of intervening on the treatement variable $x_T \in \{0, 1\}$

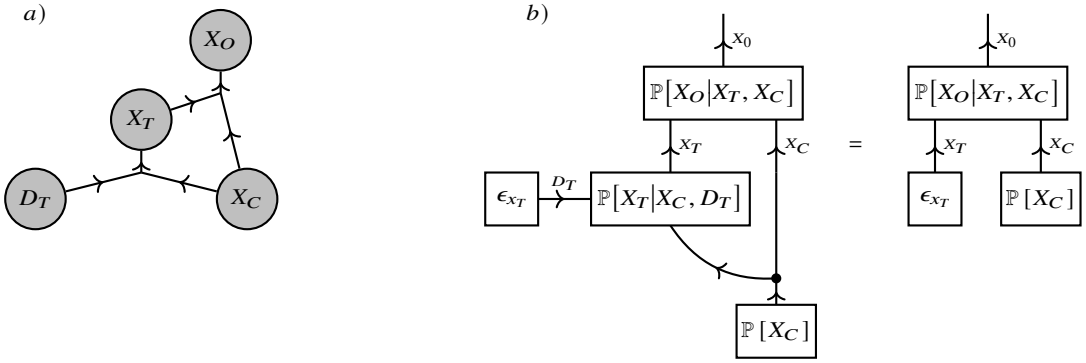



**Fig. 6.4** Effect when the confounder has a causal influence on the treatment: The causal effect is not identifiable when not measuring the confounding variable.

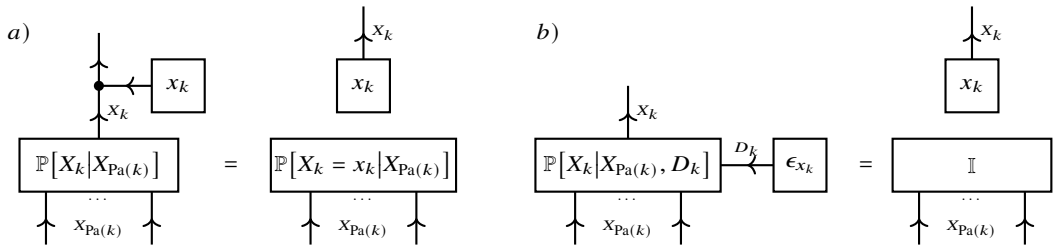



**Fig. 6.5** Comparison of conditioning $a)$ and intervention $b)$ on a variable $X_k$. When intervening, a trivial tensor over the parent variables is left, which can be exploited for further query simplifications.

#### 6.2.3.1 Backdoor Criterion

More general than in the randomized controlled trial we have to deal with confounding variables. The Backdoor Criterion and the Frontdoor Criterion exploit mediator variables $X_M$, which are also observable and block certain causal paths to unobserved confounding variables $X_C$.

**Definition 6.3** Let there be a directed acyclic hypergraph, such that $X_T$ is topologically ordered before $X_O$. A backdoor path is a path through the hypergraph, such that the first and the second node in the path are in the outgoing and the incoming nodes of a hyperedge. If every backdoor path from $X_T$ to $X_O$ is blocked by a variable in $X_M$, and $X_M$ does not contain a descendant of $X_T$, then we say $X_M$ satisfies the backdoor criterion with respect to $X_T$ and $X_O$.

See Figure 6.6 for a typical hypergraph where $X_M$ satisfies the backdoor criterion.

**Lemma 6.4** *When $X_M$ satisfies the backdoor criterion for $X_T$ and $X_O$, and for some $x_M \in [m_M], x_T \in [m_T]$ we have $\mathbb{P}[X_T = x_T, X_M = x_M \neq 0]$ then*

$$\mathbb{P}\left[X_O \middle| X_T = x_T, X_M = x_M\right] = \mathbb{P}^c\left[X_O \middle| D_T = x_T, X_M = x_M\right].$$

*Proof.* We denote $\mathcal{U}$ as the set of nodes appearing in backdoor trails from $X_T$ to $X_M$ and those variables having a variable in $X_T$ as a descendant. Since any backdoor path is blocked by $X_M$, we can isolate the contraction of the nodes in $\mathcal{U}$ by simplifying the contraction as

$$\begin{aligned}\mathbb{P}^c\left[X_O, D_T = x_T, X_M = x_M\right] = &\left\langle \{\epsilon_{x_M}\left[X_M\right]\} \cup \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, e = (\{v\}, e^{\mathrm{in}}),\, v \in \mathcal{U},\, v \notin T\}\right\rangle_{[\varnothing]} \\ &\cdot \left\langle \{\epsilon_{x_T}\left[X_T\right], \epsilon_{x_M}\left[X_M\right]\} \cup \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, e = (\{v\}, e^{\mathrm{in}}),\, v \notin \mathcal{U},\, v \notin T\}\right\rangle_{[X_O]} .\end{aligned}$$

We compare this contraction with the analogous decomposition

$$\begin{aligned}\mathbb{P}^c\left[X_O, D_T = x_T, X_M = x_M\right] = &\left\langle \{\mathbb{P}\left[X_T = x_T \middle| X_{\mathrm{Pa}(T)}\right], \epsilon_{x_M}\left[X_M\right]\} \cup \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, e = (\{v\}, e^{\mathrm{in}}),\, v \in \mathcal{U},\, v \notin T\}\right\rangle_{[\varnothing]} \\ &\cdot \left\langle \{\epsilon_{x_T}\left[X_T\right], \epsilon_{x_M}\left[X_M\right]\} \cup \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, e = (\{v\}, e^{\mathrm{in}}),\, v \notin \mathcal{U},\, v \notin T\}\right\rangle_{[X_O]} .\end{aligned}$$

and notice that both differ only in a factor. Whenever $\mathbb{P}\left[X_T = x_T, X_M = x_M\right] \neq 0$ this factor does not vanish. After normalization with $X_T$ and $X_M$ incoming, the tensors are therefore equivalent. □

**Lemma 6.5** *When $X_M$ does not contain a descendant of $X_T$ then*

$$\mathbb{P}^c\left[X_M \middle| D_T\right] = \mathbb{P}\left[X_M\right] \otimes \mathbb{I}\left[D_T\right] .$$

*Proof.* We causally augment the Bayesian network by the do variable $D_T$, and notice that also with respect to the causally augmented hypergraph, $X_M$ does not contain a descendant of $X_T$ nor $D_T$. We then investigate for arbitrary $d_T \in [m_T + 1]$ the contraction

$$\left\langle \{\mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d],\, k \neq 0\} \cup \{\mathbb{P}\left[X_T \middle| X_{\mathrm{Pa}(T)}\right], \epsilon_{d_T}\left[D_T\right]\}\right\rangle_{[X_M]} .$$

We enumerate the nodes by a topological order, such that any node having $X_M$ as a descendant is ordered before those nodes ot having $X_M$ as a descendant. For some $k \in [d]$ we then have that $X_{[k]}$ are the variables having $X_M$ as a descendant and $X_M$ itself. Lem. 6.1 implies for $\tilde{k} = 0$ and $\mathcal{U} = \varnothing$ that

$$\left\langle \{\mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d],\, k \neq 0\} \cup \{\mathbb{P}\left[X_T \middle| X_{\mathrm{Pa}(T)}\right], \epsilon_{d_T}\left[D_T\right]\}\right\rangle_{[X_M]} = \left\langle \mathbb{P}\left[X_{[k]} \middle| \varnothing\right]\right\rangle_{[X_M]} = \mathbb{P}\left[X_M\right] .$$

□

**Theorem 6.2 (Backdoor Criterion)** *Let $X_M$ satisfy the backdoor criterion with respect to $X_T$ and $X_O$, no node in $X_M$ be a descendant of $X_T$, and let $x_T \in [m_T]$ be such that for any $x_M \in [m_M]$ we have $\mathbb{P}\left[X_T = x_T, X_M = x_M\right] \neq 0$. Then we have*

$$\mathbb{P}^c\left[X_O \middle| D_T = x_T\right] = \left\langle \mathbb{P}\left[X_O \middle| X_T = x_T, X_M\right], \mathbb{P}\left[X_M\right]\right\rangle_{[X_O]} .$$

*Proof.* We have by the Bayes rule

$$\mathbb{P}^c\left[X_O \middle| D_T = x_T\right] = \left\langle \mathbb{P}^c\left[X_O \middle| X_M, D_T = x_T\right], \mathbb{P}^c\left[X_M \middle| D_T = x_T\right]\right\rangle_{[X_O]} .$$

We apply Lem. 6.4 for arbitrary $x_M \in [m_M]$ and get

$$\mathbb{P}^c\left[X_O \middle| X_M = x_M, D_T = x_T\right] = \mathbb{P}\left[X_O \middle| X_M = x_M, D_T = d_T\right] .$$

By assumption of the backdoor criterion, $X_M$ is not a descendant of $X_T$. Lem. 6.5 thus implies

$$\mathbb{P}^c\left[X_M \middle| D_T = x_T\right] = \mathbb{P}\left[X_M\right] .$$

We conclude that

$$\mathbb{P}^c\left[X_O \middle| D_T = x_T\right] = \left\langle \mathbb{P}\left[X_O \middle| X_T = x_T, X_M\right], \mathbb{P}\left[X_M\right]\right\rangle_{[X_O]} .$$

□

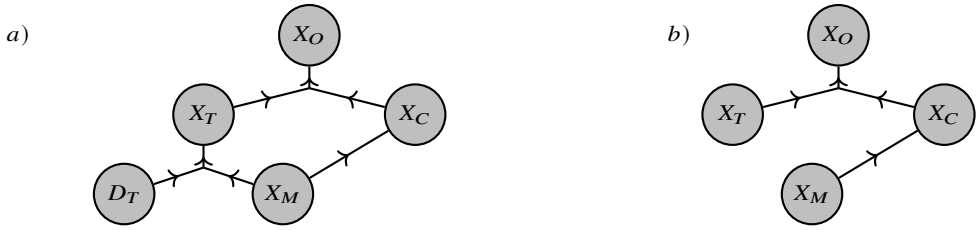



**Fig. 6.6** Causal graph for the Backdoor Criterion (see Def. 6.3). a) The mediator variable $X_M$ blocks any backdoor path from $X_T$ to $X_O$, potentially through a confounding variable $X_C$. b) The effect of the intervention on $X_T$ reduces the Bayesian network effectively to the shown subgraph.

#### 6.2.3.2 Frontdoor Criterion

We now state the frontdoor criterion, see [64].

**Definition 6.4** A frontdoor path from $X_T$ to $X_O$ is any directed path from $X_T$ to $X_O$. A set $X_M$ of variables satisfies the frontdoor criterion with respect to $X_T$ and $X_O$, if the following hold:

- Any frontdoor path from $X_T$ to $X_O$ contains a node in $X_M$.
- There is no backdoor path from $X_T$ to $X_M$.
- All backdoor paths from $X_M$ to $X_O$ are blocked by $X_T$.

**Lemma 6.6 (Frontdoor Criterion)** *When $X_M$ satisfies the frontdoor criterion with respect to $X_T$ and $X_O$, then*

$$\mathbb{P}^c\left[X_O \middle| D_T = x_T\right] = \left\langle \mathbb{P}\left[X_M \middle| X_T = x_T\right], \left\langle \mathbb{P}\left[X_O \middle| X_M, X_T\right], \mathbb{P}\left[X_T\right]\right\rangle_{[X_O, X_M]}\right\rangle_{[X_O]} .$$

*Note that the inner contraction has $X_T$ as a closed variable.*

*Proof.* We expand

$$\mathbb{P}^c\left[X_O \middle| D_T = x_T\right] = \left\langle \mathbb{P}^c\left[X_M \middle| D_T = x_T\right], \mathbb{P}^c\left[X_O \middle| X_M, D_T = x_T\right]\right\rangle_{[X_O]}$$

Since there are no backdoor trails from $X_T$ to $X_M$, we apply the backdoor simplification by Thm. 6.2 to get

$$\mathbb{P}^c\left[X_M \middle| D_T = x_T\right] = \mathbb{P}\left[X_M \middle| X_T = x_T\right] .$$

Since $X_M$ blocks all directed paths from $X_T$ to $X_O$ we have for any $x_M \in [m_M]$

$$\mathbb{P}^c\left[X_O \middle| X_M = x_M, D_T = x_T\right] = \mathbb{P}^c\left[X_O \middle| D_M = x_M\right] .$$

By assumption, $X_T$ further satisfies the backdoor criterion with respect to $X_M$ and $X_O$. We can therefore apply the backdoor simplification by Thm. 6.2 again and get

$$\mathbb{P}^c\left[X_O \middle| D_M = x_M\right] = \left\langle \mathbb{P}\left[X_O \middle| X_M, X_T\right], \mathbb{P}\left[X_T\right]\right\rangle_{[X_O]} .$$

Combining these contraction identities, we arrive at the claim. □

## 6.3 Counterfactuals

Following the formalism of Chapter 21 in [45], counterfactual queries are modeled as intervention queries on counterfactual twin networks. Counterfactual twin networks are two copies of the original causal model, one representing the factual world and one representing the counterfactual world. They further share response variables, which capture

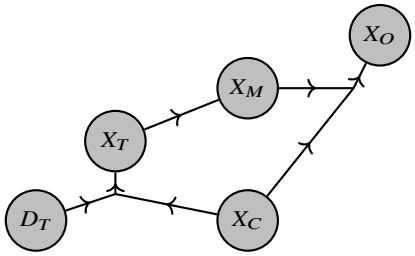



**Fig. 6.7** Typical hypergraph, where $X_M$ satisfies the frontdoor criterion with respect to $X_T$ and $X_O$.

the functional relationships between variables in the causal model. These response variables are modeled as selection variables in the tnreason formalism.

*Example 6.5 (Patient counterfactual query)* Let us assume, that we know whether a patient did not get assigned to the treatment group, and we know whether he did improve. We want to ask, whether he would have improved, if he had been assigned to the treatment group. This is not an intervention query, but a counterfactual query, since we reason about a different outcome, although we already know that the patient has not been treated.

The causally augmented Bayesian Network is not rich enough to answer such queries. Let us consider a situation where $X_T$ is chosen uniformly from $[2]$. There might be situations, where the probabilistic (and intervention) queries are the same, but the counterfactual queries differ:

(i) The outcome is independent of the treatment.
(ii)With probability 0.5, the treatment always leads to improvement and missing the treatment not. And also with probability 0.5, the treatment never leads to improvement, but missing the treatment does.

Intuitively, in the situation $(i)$, the counterfactual query should return probability $1/2$ for improvement under treatment. In the situation $(ii)$, the counterfactual query returns the same value with probability 1 as observed in the real world. Both cases are however indistinguishable by causal models itself, since in both cases the causally augmented conditional probabilities are

$$\mathbb{P}^c\left[X_T \middle| D_T\right] = \frac{1}{2}\mathbb{I}\left[X_T\right] \otimes \epsilon_2\left[D_T\right] + \left(\sum_{d_T \in [2]} \epsilon_{d_T}\left[D_T\right] \otimes \epsilon_{d_T}\left[X_T\right]\right)$$

and

$$\mathbb{P}^c\left[X_O \middle| D_O, X_T\right] = \frac{1}{2}\mathbb{I}\left[X_O, X_T\right] \otimes \epsilon_2\left[D_O\right] + \left(\sum_{d_O \in [2]} \epsilon_{d_O}\left[D_O\right] \otimes \epsilon_{d_O}\left[X_O\right] \otimes \mathbb{I}\left[X_T\right]\right).$$

To answer counterfactual queries, we therefore have to also reason about the mechanisms behind the causal relationships, which we do by introducing response variables in the following.

### 6.3.1 Response Variables

We introduce response variables $L_k$, such that $X_k$ has a deterministic dependence on $L_k$ and the parents of a variable to the variable itself. The dependence is captured by a function

$$q_k \,:\, [p_k] \times \left(\bigtimes_{\tilde{k} \in \mathrm{Pa}(k)} [m_{\tilde{k}}]\right) \to [m_k]\,.$$

The response augmented conditional probability core is defined as (see Figure 6.8):




**Fig. 6.8** Introduction of response variables: a) as a Bayesian network, b) decoration by the basis encoding of the corresponding function.

$$\mathbb{P}^f\left[X_k \middle| L_k, X_{\mathrm{Pa}(k)}\right] = \beta^{q_k}\left[X_k, L_k, X_{\mathrm{Pa}(k)}\right]$$

*Example 6.6 (Response variables for a random walk)*

Let us investigate a specific example of a Markov chain, namely a random walk on the set $[m]$. We assume that with a probability $p \in [0, 0.5]$ we move to the right neighbor, also with $p$ to the left neighbor and with $(1-2p)$ stay at the current state.

$$\begin{aligned}\mathbb{P}\left[X_k \middle| X_{k-1}\right] =& p \cdot \sum_{x\in[m]} \epsilon_{x-1}\left[X_k\right] \otimes \epsilon_x\left[X_{k-1}\right] \\ &+ (1-2p) \cdot \sum_{x\in[m]} \epsilon_x\left[X_k\right] \otimes \epsilon_x\left[X_{k-1}\right] \\ &+ p \cdot \sum_{x\in[m]} \epsilon_{x+1}\left[X_k\right] \otimes \epsilon_x\left[X_{k-1}\right]\end{aligned}$$

Here we use the summation modulo $m$, that is $0-1 = m-1$ and $m-1+1 = 0$.

We introduce to each $k \in [d]$ a response variable $L_k$ with dimension 3

$$\mathbb{P}\left[L_k\right] = \begin{pmatrix} p \\ 1-2p \\ p \end{pmatrix}\left[L_k\right]$$

where the zeroth coordinate is interpreted as the move to the next lower state, the first as staying at the position and the second as moving to the next larger state.

This interpretation is encoded in the function

$$q_k(x_{k-1}, \ell_k) = \begin{cases} x_{k-1} - 1 & \text{if} \quad \ell_k = 0 \\ x_{k-1} & \text{if} \quad \ell_k = 1 \\ x_{k-1} + 1 & \text{if} \quad \ell_k = 2 \end{cases}$$

which has the basis encoding

$$\begin{aligned}\beta^{q_k}\left[X_k | X_{k-1}, L_k\right] =& \left(\sum_{x\in[m]} \epsilon_{x-1}\left[X_k\right] \otimes \epsilon_x\left[X_{k-1}\right]\right) \otimes \epsilon_0\left[L_k\right] \\ &+ \left(\sum_{x\in[m]} \epsilon_x\left[X_k\right] \otimes \epsilon_x\left[X_{k-1}\right]\right) \otimes \epsilon_1\left[L_k\right] \\ &+ \left(\sum_{x\in[m]} \epsilon_{x+1}\left[X_k\right] \otimes \epsilon_x\left[X_{k-1}\right]\right) \otimes \epsilon_2\left[L_k\right] .\end{aligned}$$

By construction we have the decomposition

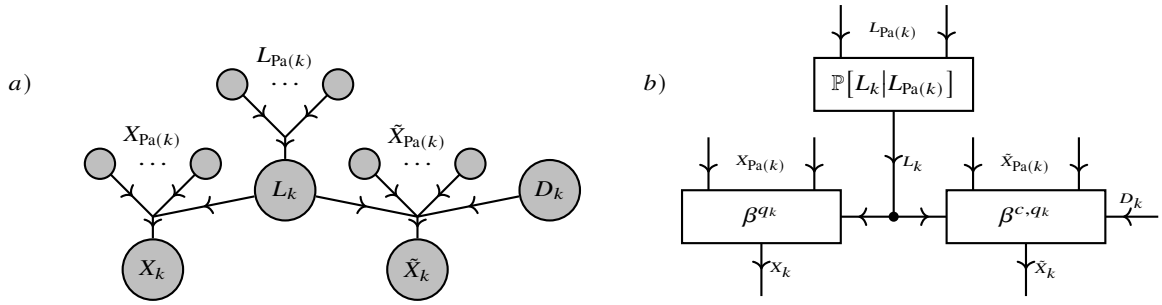



**Fig. 6.9** Counterfactual twinned network: a) as a Bayesian network. b) Decorations by causal augmented basis encodings by a do-variable for $X_k$, a conditional probability distribution for $L_k$ and a basis encoding for the counterfactual variable $\tilde{X}_k$.

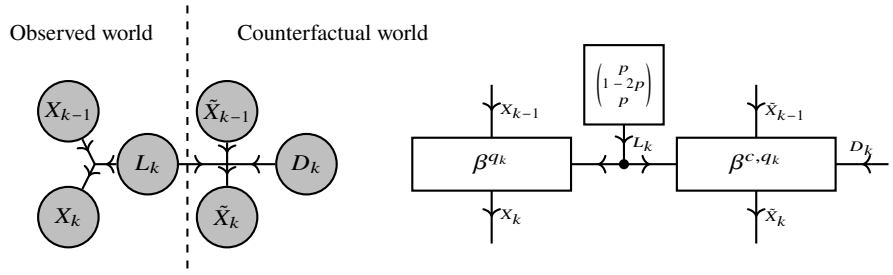
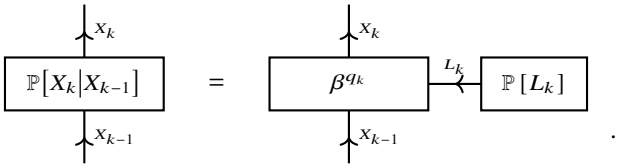



**Fig. 6.10** Part of the counterfactual twinned network for the random walk (see Example 6.6 and Example 6.7).




## 6.3.2 Counterfactual Twinned Network

To answer counterfactual queries, we now construct the counterfactual twinned network (see Figure 6.9).

**Definition 6.5** Let there be a response augmented Bayesian Network on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ with response variables $L_{[d]}$. Further another hypergraph $\tilde{\mathcal{G}}$ and a Bayesian Network on $\tilde{\mathcal{G}}$ modelling the joint distribution of the response variables $L_{[d]}$. Be build counterfactual copies $\tilde{X}_k$ of each variable $X_k$, which are controlled by the same response variables $L_k$ as the factual variables $X_k$. The counterfactual twinned network is then the tensor network consists of the causal and response augmented conditional probability cores to $X_{[d]}$, the conditional probability cores of the response variables $L_{[d]}$ and the response augmented conditional probability cores to the counterfactual variables $\tilde{X}_{[d]}$.

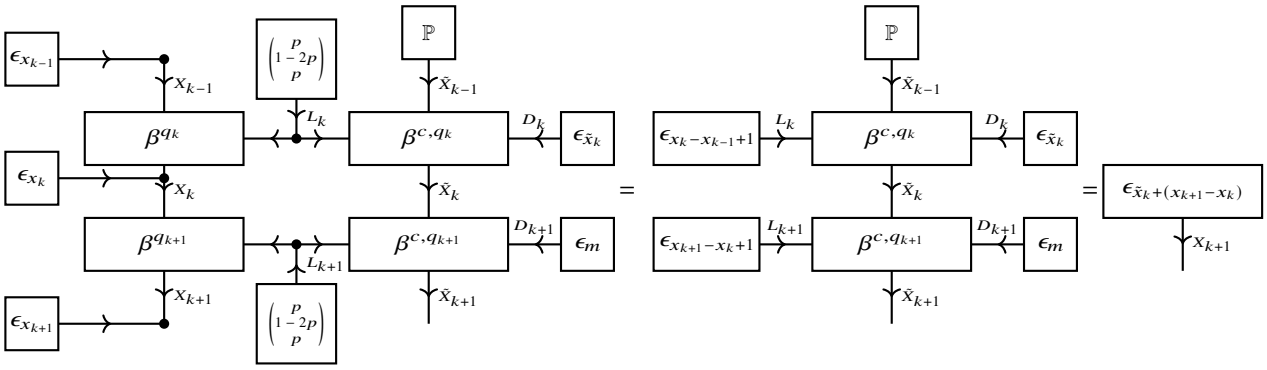
*Example 6.7 (Counterfactual query for a random walk)*

We build a counterfactual twinned network (see Figure 6.10), by adding counterfactual variables $\tilde{X}_{[d]}$ linked to the same response variables. Further, we include do variables to the counterfactual world, to answer counterfactual queries.

Setup:

- The variables $X_{k-1}, X_k, X_{k+1}$ have been observed.
- The counterfactual variable $X_k$ is intervened on.

This counterfactual query is answered by the contraction




Let us consider an arbitrary set $\mathcal{U} \subset [d]$ of variables intervened on. For each $\tilde{k}$ we denote $k$ as the previous variable which has been intervened on.

The counterfactual world is by the above contraction only supported at the indices

$$\tilde{x}_{\tilde{k}} = \tilde{x}_k + \left(x_{\tilde{k}} - x_k\right) \ .$$

If no intervention has been done before $\tilde{k}$, we define $\tilde{x}_k = x_k$ and have $\tilde{x}_{\tilde{k}} = x_{\tilde{k}}$.

Thus, the same steps (left, right neighbor or keeping positions) are done in the counterfactual world. Given the observed world, the distribution of the counterfactual world variables is therefore deterministic.

## 6.4 Learning

Given a parametrization of hypothesis distribution by the states of selection variables, we can include the intervention variables. The cores to be trivialized by intervention can depend on the state of the selection variables. For example, a selection variable might select whether the cores direction is from $X_0$ to $X_1$ or in the other direction. In both cases only one of the to intervention variables $D_0$ and $D_1$ influences the core.

# Part II
# Contraction Calculus

We observed that tensor network contractions are generic inference tasks in AI. In this part we take a broader perspective and study principles and techniques of performing computations with tensor networks. Specific interest will be dedicated to encoding schemes of computation tasks and their efficient execution.

# Chapter 7
# Basis calculus

**Abstract** In contrast to coordinate calculus we sum in basis calculus over basis elements, and in such way restrict the tensor coordinates to be boolean.

Start with set encodings, relations, funcitons then circuit decompositions.

## 7.1 Basis Encoding of Subsets

Based on the concept of one-hot encodings of states we in this chapter develop the construction of encodings to sets, relations and functions. We start with the definition of subset encodings, which represent set memberships in their boolean coordinates.

**Definition 7.1 (Basis encoding of subsets)** We say that an arbitrary finite set $\mathcal{U}$ is enumerated by an enumeration variable $O_{\mathcal{U}}$ taking values in $[r_{\mathcal{U}}]$, when $r_{\mathcal{U}} = |\mathcal{U}|$ and there is a bijective index interpretation function

$$I : [r_{\mathcal{U}}] \to \mathcal{U} .$$

Given an set $\mathcal{U}$ enumerated by the variable $O_{\mathcal{U}}$, any subset $\mathcal{V} \subset \mathcal{U}$ is encoded by the tensor $\epsilon_{\mathcal{V}}[O]$ defined for $o \in [|\mathcal{U}|]$ as

$$\epsilon_{\mathcal{V}}\left[O = o\right] = \begin{cases} 1 & \text{if} \quad I\left[o\right] \in \mathcal{V} \\ 0 & \text{else} \end{cases} .$$

In a one-hot basis decomposition we have

$$\epsilon_{\mathcal{V}}\left[O\right] := \sum_{o \in [r_{\mathcal{U}}] : I[o] \in \mathcal{V}} \epsilon_o\left[O\right] .$$

The inclusion of subsets is represented by the partial ordering of tensors. Let us first define this property for arbitrary tensors.

**Definition 7.2 (Partial ordering of tensors)** We say that two tensors $f\left[X_{[d]}\right]$ and $h\left[X_{[d]}\right]$ attached with the same variables are partially ordered, denoted by

$$f \prec h ,$$

if for all $x_{[d]} \in \bigtimes_{k \in [d]} [m_k]$

$$f\left[X_{[d]} = x_{[d]}\right] \leq h\left[x_{[d]}\right] .$$

For boolean tensors, the partially ordering is equal to a subset relation of the coordinates with value 1, as we show next.

**Theorem 7.1** *Let $\mathcal{U}$ be an arbitrary set enumerated by the variable $O$ and index interpretation function $I$. For two subsets $\mathcal{U}^0, \mathcal{U}^1$ of $\mathcal{U}$ we have*

$$\mathcal{U}^0 \subset \mathcal{U}^1$$

*if and only if*

$$\epsilon_{\mathcal{U}^0}[O] \prec \epsilon_{\mathcal{U}^1}[O] \ .$$

*Proof.* We have $\mathcal{U}^0 \subset \mathcal{U}^1$ if and only if

$$\forall o \in [|\mathcal{U}|]\left(I[o] \in \mathcal{U}^0\right) \Rightarrow \left(I[o] \in \mathcal{U}^1\right),$$

which is equal to

$$\forall o \in [|\mathcal{U}|]\left(\epsilon_{\mathcal{U}^0}[O = o] = 1\right) \Rightarrow \left(\epsilon_{\mathcal{U}^0}[O = o] = 1\right).$$

Since subset encodings are boolean tensors, this is equivalent to

$$\epsilon_{\mathcal{U}^0}[O] \prec \epsilon_{\mathcal{U}^1}[O] \ . \square$$

### 7.1.1 Binary Relations

Since relations are subsets of cartesian products between two sets, their encoding is a straightforward generalization of Def. 7.1.

**Definition 7.3 (Basis encoding of binary relations)** A relation between two finite sets $\mathcal{U}^{\text{in}}$ and $\mathcal{U}^{\text{out}}$ is a subset of their cartesian product

$$\mathcal{R} \subset \mathcal{U}^{\text{in}} \times \mathcal{U}^{\text{out}}.$$

Given an enumeration of $\mathcal{U}^{\text{in}}$ and $\mathcal{U}^{\text{out}}$ by the categorical variables $O_{\text{in}}$ and $O_{\text{out}}$ and interpretation maps $I_{\text{in}}$, $I_{\text{out}}$, we define the basis encoding of this subset as the tensor $\epsilon_{\mathcal{R}}[O_{\text{in}}, O_{\text{out}}]$ with the coordinates

$$\epsilon_{\mathcal{R}}[O_{\text{in}} = o_{\text{in}}, O_{\text{out}} = o_{\text{out}}] = \begin{cases} 1 & \text{if} \quad (I_{\text{in}}[o_{\text{in}}], I_{\text{out}}[o_{\text{out}}]) \in \mathcal{R} \\ 0 & \text{else} \end{cases}.$$

The basis encoding has a decomposition into one-hot encodings as

$$\epsilon_{\mathcal{R}}[O_{\text{in}}, O_{\text{out}}] = \sum_{o_{\text{in}}, o_{\text{out}} \,:\, (I_{\text{in}}[o_{\text{in}}], I_{\text{out}}[o_{\text{out}}]) \in \mathcal{R}} \epsilon_{o_{\text{in}}}[O_{\text{in}}] \otimes \epsilon_{o_{\text{out}}}[O_{\text{out}}] \ .$$

basis encodings have a matrix structure by the cartesian product, which can be further folded to tensors, when the sets itself are cartesian products. The basis encoding is a bijection between the relations of two sets and the boolean tensors with their enumeration variables.

### 7.1.2 Higher-Order Relations

We can extend this contraction to relations of higher order, and arrive at encoding schemes usable for relational databases.

**Definition 7.4 (Basis encoding of $d$-ary relations)** Given sets $\mathcal{U}^k$ for $k \in [d]$, a $d$-ary relation is a subset of a their cartesian product, that is

$$\mathcal{R} \subset \bigtimes_{k \in [d]} \mathcal{U}^k \,.$$

Given an enumeration of each set $\mathcal{U}^k$ by a variable $O_k$ and an interpretation map $I_k$, we define the basis encoding of the relation as the tensor $\epsilon_{\mathcal{R}}[O_{[d]}]$ with coordinates

$$\epsilon_{\mathcal{R}}\left[O_{[d]} = o_{[d]}\right] = \begin{cases} 1 & \text{if} \quad (I_0\,[o_0], \ldots, I_{d-1}\,[o_{d-1}]) \in \mathcal{R} \\ 0 & \text{else} \end{cases} .$$

Let there be for $k \in [d]$ sets $\mathcal{U}^k$ of truth assignments to the $k$-th atom, which are all enumerated by [2]. A propositional formula then corresponds with a $d$-ary relation and we directly defined them in Def. 2.1 by their basis encoding.

**Theorem 7.2** *The encoding of any $d$-ary relation*

$$\mathcal{R} = \{x^i_{[d]} \,:\, i \in [n]\} \subset \bigtimes_{k \in [d]} \{\text{False}, \text{True}\}$$

*where the objects in* {False, True} *are enumerated by $X_k$ with the standard index interpretation function (see Sect. 2.3.1)*

$$I\,[\text{True}] = 1 \quad \textit{and} \quad I\,[\text{False}] = 0\,,$$

*coincides with the propositional formula*

$$f\left[X_{[d]}\right] = \bigvee_{i \in [n]} Z^{\wedge}_{x^i_{[d]}} \,.$$

*Proof.* By definition, the encoding $\epsilon_{\mathcal{R}}$ is decomposed as

$$\epsilon_{\mathcal{R}}\left[X_{[d]}\right] = \sum_{i \in [n]} \epsilon_{x^i_{[d]}}\left[X_{[d]}\right] \,.$$

By Thm. 2.8 this is equal to

$$\tau\left[X_{[d]}\right] = \left( \bigvee_{x_{[d]} \,:\, \tau\left[X_{[d]} = x_{[d]}\right] = 1} Z^{\wedge}_{\{k \,:\, x_k = 0\}} \{k \,:\, x_k = 1\} \right) [X_{[d]}] = \bigvee_{i \in [n]} Z^{\wedge}_{x^i_{[d]}} \,. \square$$

*Example 7.1 (Relational Databases)* Relational Databases can be encoded as tensors using the relation encoding scheme. Each column is thereby understood as an enumeration variable, which values form the sets $\mathcal{U}^k$.

Let us notice, that the dimensionality of the tensor space used for representing a relation is

$$\prod_{k \in [d]} |\mathcal{U}^k|$$

and therefore growing exponentially with the number of variables. Relations are however often sparse, in the sense that

$$|\mathcal{R}| << \prod_{k \in [d]} \left|\mathcal{U}^k\right| \,.$$

It is therefore often beneficially to choose sparse encoding schemes, for example by restricted CP formats (see Chapter **??**) to represent $\epsilon_{\mathcal{R}}$.

## 7.2 Basis Encoding of Functions

Let us now restrict to relations, which have an expression by functions. We in this section then show, how contractions of their encodings can be exploited in function evaluation.

### 7.2.1 Definition

**Definition 7.5 (Basis encoding of maps)** Any map

$$q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$$

can be represented by a relation

$$\mathcal{R}^q := \left\{ \left(x, q(x)\right) : x \in \mathcal{U}^{\text{in}} \right\} \subset \mathcal{U}^{\text{in}} \times \mathcal{U}^{\text{out}} .$$

Given a enumeration of the sets by $O_{\text{in}}$ and $O_{\text{out}}$ we define the basis encoding of $q$ as the tensor

$$\beta^q \left[O_{\text{out}}, O_{\text{in}}\right] = \epsilon_{\mathcal{R}^q} \left[O_{\text{in}}, O_{\text{out}}\right] .$$

*Remark 7.1 (Reduction to images)* When $q$ maps into a set of infinite cardinality, we restrict $\mathcal{U}^{\text{out}}$ to the image of $q$ and enumerate the image by a variable $O_q$. This scheme is applied, when $q$ is itself a tensor, i.e. $\mathcal{U}^{\text{out}} = \mathbb{R}$. While the variable $O_q$ can in general be of the same cardinality as the domain set $\mathcal{U}^{\text{in}}$, it will be valued in $[2]$ when considering boolean tensors.

We notice that any basis representation of a function is also a directed tensor with incoming variables to the domain and outgoing variables to the image. It furthermore holds, that the set of directed and boolean tensors is characterized by the basis encoding of functions. This is shown in the next theorem, by the claim that any boolean tensor which is directed is the basis representation of a function.

**Theorem 7.3** *Let $\mathcal{U}^{\text{in}}, \mathcal{U}^{\text{out}}$ be sets and $\mathcal{R} \subset \mathcal{U}^{\text{in}} \times \mathcal{U}^{\text{out}}$ a relation. If and only if there exists a map $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ such that $\mathcal{R} = \mathcal{R}^q$, the basis encoding $\beta^q$ is a directed tensor with $O_{\text{in}}$ incoming and $O_{\text{out}}$ outgoing.*

*Proof.* "⇒": When $q$ is a function, we have for any $o_{\text{in}} \in [r_{\text{in}}]$

$$\sum_{o_{\text{out}} \in [r_{\text{out}}]} \beta^q \left[O_{\text{out}} = o_{\text{out}}, O_{\text{in}} = o_{\text{in}}\right] = \beta^q \left[O_{\text{out}} = I_{\text{out}}^{-1}(q(I_{\text{in}}\left[o_{\text{in}}\right])), O_{\text{in}} = o_{\text{in}}\right] = 1 .$$

Thus, $\beta^q \left[O_{\text{out}}, O_{\text{in}}\right]$ is a directed tensor with variables $O_{\text{in}}$ incoming and $O_{\text{out}}$ outgoing.

"⇐": Conversely let there be a relation $\mathcal{R}$, such that $\beta^{\mathcal{R}}$ is directed. To this end, we observe that for any $o_{\text{in}} \in [r_{\text{in}}]$ the tensor

$$\epsilon_{\mathcal{R}} \left[O_{\text{in}} = o_{\text{in}}, O_{\text{out}}\right]$$

is a boolean tensor with coordinate sum one and therefore a basis vector. It follows that the function $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ defined for $x \in \mathcal{U}^{\text{in}}$ as

$$q(x) = I_{\text{out}} \left[\epsilon^{-1}(\epsilon_{\mathcal{R}} \left[O_{\text{in}} = I_{\text{in}}^{-1}(x), O_{\text{out}}\right])\right]$$

is well-defined. We then have by construction

$$\begin{aligned}
\beta^q \left[O_{\text{out}}, O_{\text{in}}\right] &= \sum_{o_{\text{in}} \in [r_{\text{in}}]} \epsilon_{q(o_{\text{in}})} \left[O_{\text{out}}\right] \otimes \epsilon_{o_{\text{in}}} \left[O_{\text{in}}\right] \\
&= \sum_{o_{\text{in}} \in [r_{\text{in}}]} \epsilon_{\mathcal{R}} \left[O_{\text{in}} = o_{\text{in}}, O_{\text{out}}\right] \otimes \epsilon_{o_{\text{in}}} \left[O_{\text{in}}\right] \\
&= \epsilon_{\mathcal{R}} \left[O_{\text{out}}, O_{\text{in}}\right]
\end{aligned}$$

and therefore by Def. 7.5 $\mathcal{R} = \mathcal{R}^q$. □

We are specially interested in sets of states of a factored system, which amounts to the case in Def. 1.6. Those state sets have a decomposition into a cartesian product of $d$ sets

$$\mathcal{U} = \bigtimes_{k\in[d]} [m_k] \,.$$

The most obvious enumeration of the set $\mathcal{U}$ is therefore by the collection of state variables $\{X_k : k \in [d]\}$. Functions between states of factored systems with $d_{\text{in}}$ and $d_{\text{out}}$ state variables can be represented by $d_{\text{in}} + d_{\text{out}}$-ary relations and Def. 7.5 has an obvious generalization to this case with multiple enumeration variables.

### 7.2.2 Function Evaluation

We now justify the nomenclature of basis calculus, by showing that contraction with basis elements produce the one-hot encoded function evaluation.

**Theorem 7.4 (Function evaluation in Basis Calculus)** *Retrieving the value of the function $q$ at a specific state is then the contraction of the tensor representation with the one-hot encoded state. For any $u \in \mathcal{U}^{\text{in}}$ we have*

$$\epsilon_{I_{\text{out}}^{-1}(q(u))}\,[O_{\text{out}}] = \left\langle \beta^f\,[O_{\text{out}}, O_{\text{in}}]\,, \epsilon_{I_{\text{in}}^{-1}(u)}\,[O_{\text{in}}]\right\rangle_{[O_{\text{out}}]} \,.$$

*Thus, we can retrieve the function evaluation by the inverse one-hot mapping as*

$$q(u) = \epsilon^{-1}(\left\langle \beta^f\,[O_{\text{out}}, O_{\text{in}}]\,, \epsilon_{I_{\text{in}}^{-1}(u)}\,[O_{\text{in}}]\right\rangle_{[O_{\text{out}}]}) \,.$$

*Proof.* From the representation

$$\beta^q\,[O_{\text{out}}, O_{\text{in}}] = \sum_{o_{\text{in}}\in[r_{\text{in}}]} \epsilon_{(I_{\text{out}}^{-1}\circ q\circ I_{\text{in}})o_{\text{in}}}\,[O_{\text{in}}] \otimes \epsilon_{o_{\text{in}}}\,[O_{\text{in}}]$$

and the orthonormality of the one-hot encodings of the input enumeration we get

$$\left\langle \beta^f\,[O_{\text{out}}, O_{\text{in}}]\,, \epsilon_{I_{\text{in}}^{-1}(u)}\,[O_{\text{in}}]\right\rangle_{[O_{\text{out}}]} = \epsilon_{I_{\text{out}}^{-1}(q(u))}\,[O_{\text{out}}] \;.\square$$

In comparison with the Coordinate Calculus scheme (see Thm. **??**), the Basis Calculus produces basis vectors of a functions evaluation instead of scalars. While this seems to produce unnecessary redundancy in representing a function, we will see in the following section, that this scheme is efficient in representing compositions of functions.

## 7.3 Circuits

The neural paradigm of artificial intelligence exploits the decomposition of functions into neurons, which are aligned in a directed acyclic graph. We show in this section how functions decomposable into neurons can be represented by tensor networks. To this end, we formalize discrete neural models by decomposition graphs and formally prove the corresponding decomposition of their basis encodings.

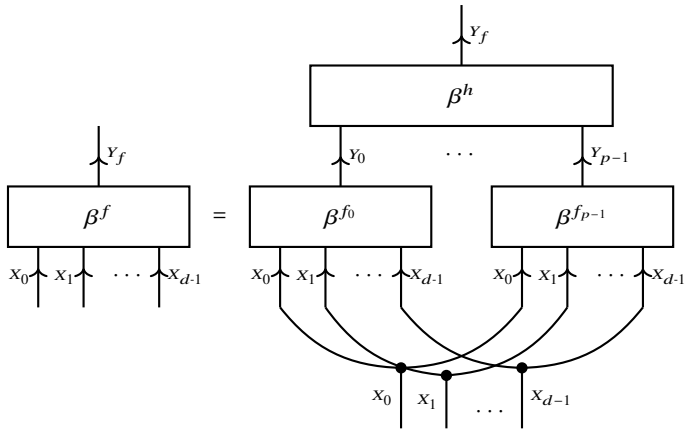



**Fig. 7.1** Tensor network decomposition of the basis encoding of a function $f$, which is the composition of the functions $f_0, \dots, f_{p-1}$ with a function $h$.

### 7.3.1 Function decomposition

As a main principle of tensor decompositions, we now show that basis encodings of composition functions are contractions of the basis encodings of their components.

**Lemma 7.1** *Let $f\left[X_{[d]}\right]$ be a composition of a $p$-ary connective function $h$ and functions $f_\ell\left[X_{[d]}\right]$, where $\ell \in [p]$, that is for $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have*

$$f(x_{[d]}) = h\left(f_0\left[x_{[d]}\right], \dots, f_{p-1}\left[x_{[d]}\right]\right) .$$

*Then, we have (see Figure 7.1)*

$$\beta^f\left[Y_f, X_{[d]}\right] = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f, X_{[d]}]} .$$

*Proof.* For any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have

$$\begin{aligned}
&\left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f, X_{[d]}=x_{[d]}]} \\
&\quad = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]} = x_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f]} \\
&\quad = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\epsilon_{f_\ell(x_{[d]})}\left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f]} \\
&\quad = \epsilon_{f(x_{[d]})}\left[Y_f\right] \\
&\quad = \beta^f\left[Y_f, X_{[d]} = x_{[d]}\right] .
\end{aligned}$$

Thus, the tensors on both sides of the equation coincide in all slices to $X_{[d]}$ and are thus equal. □

We now define a more generic decomposition of discrete functions.

**Definition 7.6** A *circuit* is a directed acyclic hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ such that the following holds.

- Each node $v \in \mathcal{V}$ is decorated by a set $\mathcal{U}^v$ of finite cardinality $m_v$, a variable $X_v$, and an index interpretation function

$$I_v \,:\, [m_v] \to \mathcal{U}^v .$$

- Each directed hyperedge $(e^{\text{in}}, e^{\text{out}})$ has at least one outgoing node, that is $e^{\text{out}} \neq \emptyset$, and is decorated by an activation function

$$g^e : \bigtimes_{v \in e^{\text{in}}} \mathcal{U}^v \to \bigtimes_{v \in e^{\text{out}}} \mathcal{U}^v .$$

- Each node $v \in \mathcal{V}$ appears at most once as an outgoing node.
- The nodes not appearing as an outgoing node are enumerated by $v^{\text{in}}_{[d]}$. We abbreviate the corresponding variables by $X_{v^{\text{in}}_{[d]}} = X_{[d]}$.
- The nodes not appearing as an incoming node are enumerated by $v^{\text{out}}_{[p]}$. We abbreviate the variables by $X_{v^{\text{out}}_{[\ell]}} = Y_{[p]}$.

We assign for each $k \in [d]$ restriction functions

$$q_{v^{\text{in}}_k} : \bigtimes_{\tilde{k} \in [d]} \mathcal{U}^{v^{\text{in}}_{\tilde{k}}} \to \mathcal{U}^{v^{\text{in}}_k} \quad , \quad x_{[d]}|_k = x_k$$

to the nodes $v^{\text{in}}_{[d]}$ and recursively assign to each further node $v$ a node function

$$q_v : \bigtimes_{k \in [d]} \mathcal{U}^{v^{\text{in}}_k} \to \mathcal{U}^v \quad , \quad q_v(x_{[d]}) = g^{e_v}\left( \bigtimes_{\tilde{v} \in e^{\text{in}}} q_{\tilde{v}}(x_{[d]}) \right)\Big|_v ,$$

where $e_v$ is to each $v \in e^{\text{in}}$ the unique hyperedge with outgoing nodes $\{v\}$. We then call the function

$$q_{\mathcal{G}} : \bigtimes_{k \in [d]} \mathcal{U}^{v^{\text{in}}_k} \to \bigtimes_{\ell \in [p]} \mathcal{U}^{v^{\text{out}}_\ell} \quad , \quad q_{\mathcal{G}} = \bigtimes_{\ell \in [p]} q_{v^{\text{out}}_\ell}$$

the composition function to the circuit $\mathcal{G}$.

The neural paradigm in AI can be modeled by the existence of circuits for functions on large sets. We now show how circuits enable the sparse representation of composition functions by tensor networks.

**Theorem 7.5** *For any circuit $\mathcal{G}$ with composition formula $q_{\mathcal{G}}$, we have*

$$\beta^{q_{\mathcal{G}}}\left[Y_{[p]}, X_{[d]}\right] = \left\langle \left\{ \beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} .$$

*Proof.* We show by induction over the number of edges in $\mathcal{G}$ that, for any $x_{[d]} \in \bigtimes_{k \in [d]} [m_k]$ and $v \in \mathcal{V}$, we have

$$\left\langle \left\{ \beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{\left[X_{\mathcal{V} \setminus v^{\text{in}}_{[d]}}, X_{[d]} = x_{[d]}\right]} = \bigotimes_{v \in \mathcal{V} \setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] . \tag{7.1}$$

$|\mathcal{E}| = 1$: If there is a single edge $e = (e^{\text{in}}, e^{\text{out}})$ in $\mathcal{E}$, we have $X_{[d]} = X_{e^{\text{in}}}$ and $\mathcal{V} \setminus v^{\text{in}}_{[d]} = e^{\text{out}}$. In this case (7.1) holds since

$$\beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}} = x_{e^{\text{in}}}\right] = \bigotimes_{v \in \mathcal{V} \setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .$$

$(|\mathcal{E}| = n) \Rightarrow (|\mathcal{E}| = n + 1)$: Let us now assume that (7.1) holds for all graphs with $|\mathcal{E}| \leq n$ and let $\mathcal{G}$ be a graph with $|\mathcal{E}| = n + 1$. We find an edge $\tilde{e} = (e^{\text{in}}, e^{\text{out}})$ such that all nodes in $e^{\text{in}}$ are only appearing as outgoing nodes in other edges.

$$
\begin{aligned}
&\left\langle \left\{ \beta^{g^e}\left[X_e\right] \,:\, e \in \mathcal{E} \right\} \right\rangle_{\left[X_{\mathcal{V}\setminus v^{\text{in}}_{[d]}}, X_{[d]}=x_{[d]}\right]} \\
&= \left\langle \beta^{\tilde{e}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right], \left\langle \left\{ \beta^{g^e}\left[X_e\right] \,:\, e \in \mathcal{E}\setminus\{\tilde{e}\} \right\} \right\rangle_{\left[X_{\mathcal{V}\setminus\{v^{\text{in}}_{[d]}\cup e^{\text{out}}\}}, X_{[d]}=x_{[d]}\right]} \right\rangle_{\left[X_{\mathcal{V}\setminus v^{\text{in}}_{[d]}}, X_{[d]}=x_{[d]}\right]} \\
&= \left\langle \beta^{\tilde{e}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right], \bigotimes_{v\in\mathcal{V}\setminus\{v^{\text{in}}_{[d]}\cup e^{\text{out}}\}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \right\rangle_{\left[X_{\mathcal{V}\setminus v^{\text{in}}_{[d]}}, X_{[d]}=x_{[d]}\right]} \\
&= \bigotimes_{v\in\mathcal{V}\setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .
\end{aligned}
$$

Here, in the second equation we used the induction hypothesis on the subgraph $(\mathcal{V}, \mathcal{E}\setminus\{\tilde{e}\})$ with $|\mathcal{E}\setminus\{\tilde{e}\}| = n$ and a generic contraction property of basis encodings in the third equation.

Thus, (7.1) holds always and we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ that

$$
\begin{aligned}
\left\langle \left\{ \beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{\left[Y_{[p]}, X_{[d]}=x_{[d]}\right]} &= \left\langle \bigotimes_{v\in\mathcal{V}\setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} \\
&= \bigotimes_{v\in v^{\text{out}}_{[p]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \\
&= \epsilon_{q_{\mathcal{G}}}\left[Y_{[p]}\right] \\
&= \beta^{q_{\mathcal{G}}}\left[Y_{[p]}, X_{[d]} = x_{[d]}\right] .
\end{aligned}
$$

Keeping $X_{[d]}$ open, the claim is established. □

When neurons have tunable parameters, we can discretize those by sets $\mathcal{U}^k$ and understand them as additional input variables.

*Example 7.2 (Sum of integers in m-adic representation)* We develop a tensor network representation of integer summations on the set $[m^d] = \{0, \ldots, m^d - 1\}$, where $m, d \in \mathbb{N}$,

$$
+ \,:\, [m^d] \times [m^d] \to [m^{d+1}] \quad , \quad +(i, \tilde{i}) = i + \tilde{i} ,
$$

which have a $m$-adic representation of length $d$. We define an index interpretation map

$$
I \,:\, \bigtimes_{k\in[d]} [m] \to [m^d] \quad , \quad I\left[x_{[d]}\right] = \sum_{k\in[d]} x_k \cdot m^k ,
$$

which enables the parameterization of $[m^d]$ as the states of $d$ categorical variables $X_{[d]}$ of dimension $m$. We analogously represent a second set $[m^d]$ by variables $\tilde{X}_{[d]}$ and the set $[m^{d+1}]$ of possible sums by $Y_{[d+1]}$. The basis encoding of the sum is then

$$
\beta^{+}\left[Y_{[d+1]}, X_{[d]}, Y_{[d]}\right] = \sum_{x_{[d]}, \tilde{x}_{[d]}} \epsilon_{I^{-1}(I\left[x_{[d]}\right] + I\left[\tilde{x}_{[d]}\right])}\left[Y_{[d+1]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{\tilde{x}_{[d]}}\left[\tilde{X}_{[d]}\right] .
$$

Note that the tensor space of $\beta^{+}$ is of dimension $m^{3\cdot d+1}$ increasing exponentially in $d$. Feasible representation of this tensor for large $d$ therefore requires tensor network decompositions, which we now provide based on a decomposition hypergraph. The targeted function to be decomposed is the representation of the integer sum by

$$
+^m \,:\, \left(\bigtimes_{k\in[d]} [m]\right) \times \left(\bigtimes_{k\in[d]} [m]\right) \to \bigtimes_{k\in[d+1]} [m] \quad , \quad +^m(x_{[d]}, \tilde{x}_{[d]}) = I^{-1}(I\left[x_{[d]}\right] + I\left[\tilde{x}_{[d]}\right]) .
$$

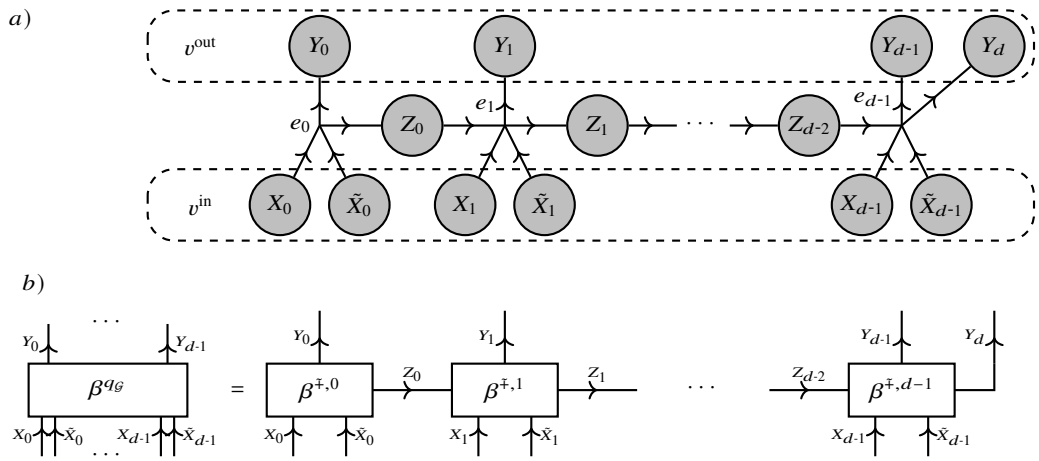



**Fig. 7.2** Example of a decomposition hypergraph to the sum of integers (see Example 7.2). a) Hypergraph of directed edges $e_k$ for $k \in [d]$, each decorated by an integer summation + preparing an index $Y_k$ of the resulting sum. b) Corresponding tensor network decomposition of the basis encoded composition function, which is the sum of integers in $m$-adic representation.

We build a decomposition hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Def. **??**) consisting of $4 \cdot d$ nodes (see Figure 7.2a) . The nodes carry the $(3 \cdot d + 1)$ variables $X_{[d]}, Y_{[d]}, Y_{[d+1]}$ of dimension $m$ constructed above and $d - 1$ auxiliary variables $Z_{[d-1]}$ of dimension 2 representing carry bits. The directed hyperedges of $\mathcal{G}$ are

$$\begin{aligned}\mathcal{E} = &\left\{(\{X_0, \tilde{X}_0\}, \{Y_0, Z_0\})\right\} \cup \left\{(\{Z_{k-1}, X_k, \tilde{X}_k\}, \{Y_k, Z_k\}) \,:\, k \in \{1, \ldots, d-2\}\right\} \\ &\cup \left\{(\{Z_{d-2}, X_{d-1}, \tilde{X}_{d-1}\}, \{Y_{d-1}, Y_d\})\right\}\end{aligned}$$

and are decorated by local summation functions

$$\tilde{+} \,:\, [2] \times [m] \times [m] \to [m] \times [2] \quad , \quad \tilde{+}(z, x, \tilde{x}) = \left( (z + x + \tilde{x}) \mod m, \left\lfloor \frac{z + x + \tilde{x}}{m} \right\rfloor \right) .$$

Since to the first hyperedge we do not have a carry bit, the decorating function is the restriction of the first argument to 0.

It is known that the composition of the local summations $\tilde{+}$ is the global summation $+^m$ of integers in $m$-adic representation. Thus, the composition function $q_{\mathcal{G}}$ is $+^m$. By Thm. 7.5 we have a decomposition of the basis encoding to $q_{\mathcal{G}}$ (see Figure 7.2b) as

$$\begin{aligned}\beta^{+^m}\left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}\right] = \big\langle &\{\beta^{\tilde{+},0}\left[Y_0, Z_0, X_0, \tilde{X}_0\right]\} \cup \\ &\{\beta^{\tilde{+},k}\left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}\right] \,:\, k \in \{1, \ldots, d-2\}\} \cup \\ &\{\beta^{\tilde{+},d-2}\left[Y_{d-1}, Y_d, X_{d-1}, \tilde{X}_{d-1}, Z_{d-2}\right]\}\big\rangle_{\left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}\right]} .\end{aligned}$$

### 7.3.2 Function evaluation by message passing

We are now concerned with an efficient inference algorithm based on tensor network contractions. To evaluate a function given as a tensor network decomposition of its basis encoding, the whole network has to be contracted. As this can be infeasible for large networks, a message passing algorithm based on local contractions can be applied, compare Algorithm 8 for a message passing algorithm for tensor networks on a on tree hypergraph.

**Algorithm 7** Directed Belief Propagation

**Require:** Tensor network $\tau^{\mathcal{G}}$ on a directed hypergraph $\mathcal{G}$
**Ensure:** Messages $\{\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] : (e_0,e_1)\in\mathcal{E}^{\to}\}$

Prepare directed message directions

$$\mathcal{E}^{\to}=\left\{\left((e_0^{\text{in}},e_0^{\text{out}}),(e_1^{\text{in}},e_1^{\text{out}})\right) : e_0^{\text{in}}\cap(e_1^{\text{in}},e_1^{\text{out}})=\emptyset,\ e_1^{\text{out}}\cap(e_0^{\text{in}},e_0^{\text{out}})=\emptyset,\ e_0^{\text{out}}\cap e_1^{\text{in}}\neq\emptyset\right\}$$

Initialize a message queue $S=\{(e_2,e_0) : e_2 \text{ has empty incoming nodes}\}$
**while** $S$ not empty **do**
  Pop a $(e_0,e_1)$ pair from $S$
  Update the message

$$\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right]=\left\langle\{\tau^{e_0}\left[X_{e_0}\right]\}\cup\{\chi_{e_2\to e_0}\left[X_{e_2\cap e_0}\right] : (e_2,e_0)\in\mathcal{E}^{\to},\ e_2\neq e_1\}\right\rangle_{[X_{e_0\cap e_1}]}$$

  Update $S$ by all messages $(e_1,e_3)$ which have not yet been sent, if all messages $(e_2,e_1)$ have been sent.
**end while**
**return** Messages $\{\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] : (e_2,e_0)\in\mathcal{E}^{\to}\}$

We apply the Directed Belief Propagation, Algorithm Algorithm 7, on a circuit, where we add hyperedges to each leaf node and assign one-hot encodings of input states. We then show that the messages are the one-hot encodings to the evaluations of the node functions.

**Theorem 7.6** *Let $\mathcal{G}$ be a decomposition graph and let us add hyperedges containing single input nodes, which are decorated by one-hot encodings. Then the messages computed in Algorithm 7 are characterized by*

$$\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right]=\bigotimes_{v\in e_0\cap e_1}\epsilon_{q_v(x_{[d]})}\left[X_v\right] .$$

*Proof.* We show the theorem inductively over the messages computed in Algorithm 7. The first message is sent from an input edge $\{[k]\}$ to an edge $e$ of the decomposition graph and is by assumption the one-hot encoding of an input state $\epsilon_{x_k}\left[X_k\right]$.

We now assume that all previous messages satisfy the claimed equation at an arbitrary stage of the algorithm. The message computed in the while loop is then a contraction of one-hot encodings with basis encodings and

$$\begin{aligned}\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right]&=\left\langle\{\beta^{g^{e_0}}\left[X_{e^{\text{out}}},X_{e^{\text{in}}}\right]\}\cup\{\chi_{e_2\to e_0}\left[X_{e_2\cap e_0}\right] : (e_2,e_0)\in\mathcal{E}^{\to}\}\right\rangle_{[X_{e_0\cap e_1}]}\\&=\left\langle\{\beta^{g^{e_0}}\left[X_{e^{\text{out}}},X_{e^{\text{in}}}\right]\}\cup\{\epsilon_{q_v(x_{[d]})}\left[X_v\right] : v\in e^{\text{in}}\}\right\rangle_{[X_{e_0\cap e_1}]}\\&=\bigotimes_{v\in e_0\cap e_1}\epsilon_{q_v(x_{[d]})}\left[X_v\right] .\end{aligned}$$

Thus, the new message is also the tensor product of the one-hot encodings of the evaluated node functions. By induction, the property is therefore true for all messages. □

We notice that we can interpret any directed acyclic hypergraph for which each node appears exactly once as an outgoing node and which is decorated by boolean and directed tensors $\tau^{\mathcal{G}}$. Edges with empty incoming sets are carrying one-hot encodings of input states and all further edges carry functions.

*Example 7.3 (Continuation of Example 7.2)* We now show how Algorithm 7 can be exploited to compute an efficient message passing algorithm for the digits of the $m$-adic sum. Given two numbers in $m$-adic representation by the tuples $x_{[d]}$ and $\tilde{x}_{[d]}$, we add the hyperedges with empty incoming nodes and single outgoing node

$$\bigcup_{k\in[d]}\left\{(\emptyset,\{X_k\}),(\emptyset,\{\tilde{X}_k\})\right\}$$

to the hypergraph of Example 7.2 and decorate them by the digit one-hot encodings $\epsilon_{x_k}\left[X_k\right]$ and $\epsilon_{\tilde{x}_k}\left[\tilde{X}_k\right]$ (see Figure 7.3). We then apply the Directed Belief Propagation Algorithm 7. The initial messages queue then consists of the messages from the digit encoding. As sketched in Figure 7.3, to each digit there are three messages (with the exception

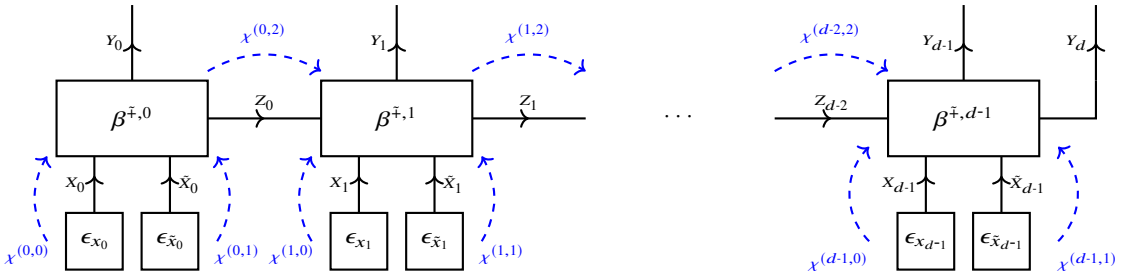



**Fig. 7.3** Computation of the integer sum in $m$-adic representation by the Directed Belief Propagation Algorithm 7 (see Example 7.3). The summands are represented by one-hot encodings of the digits $x_{[d]}$ and $\tilde{x}_{[d]}$, from which the messages start. The $k$-th digit (for $k \in \{0, \ldots, d-1\}$) of the sum is computed based on the first messages of the epoch labeled by $\chi^{(k,[2])}$, The third message $\chi^{(k,2)}$ in each epoch communicates the carry bit to the next digit summation core. In the last message epoch the digit $d-1$ and $d$ are computed based.

of the first being two), which can be scheduled in consecutive epochs $\chi^{(k,[3])}$. We then have by Thm. 7.6 for $k \in [d-1]$ that

$$\left\langle \beta^{\tilde{+},k} \left[ Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1} \right], \chi^{(k-1,2)}[Z_{k-1}], \chi^{(k,0)}[X_k], \chi^{(k,1)}[\tilde{X}_k] \right\rangle_{[Z_k]} = \epsilon_{z_k}[Z_k] \ ,$$

where $z_k$ is the value of the $k$-th carry bit. The $k$-th digit of the sum $y_k$ can further be obtained by the contraction

$$\left\langle \beta^{\tilde{+},k} \left[ Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1} \right], \chi^{(k-1,2)}[Z_{k-1}], \chi^{(k,0)}[X_k], \chi^{(k,1)}[\tilde{X}_k] \right\rangle_{[Y_k]} = \epsilon_{y_k}[Y_k] \ .$$

Note that the hypergraph representing this instance is a tree and by Thm. 5.14 also the message passing scheme of Algorithm 8 is guaranteed to produce the exact contractions. We can exploit this fact for example in the efficient computation of averages of the summation digits, when we have an elementary distribution of input digits. We emphasize that the directed belief propagation Algorithm 8 is exact even if the hypergraph fails to be a tree, provided that we have directed and boolean tensors..

## 7.4 Hybrid Basis and Coordinate Calculus

In some situations, we can perform basis calculus more effectively by avoiding image enumeration variables, and instead apply coordinatewise transforms on tensors (see Def. **??**). As we show here, these include conjunctions, which correspond with coordinatewise multiplication, and negation, which correspond with coordinatewise subtraction from the trivial tensor. Such schemes are applied for example in [81] in batchwise logical inference.

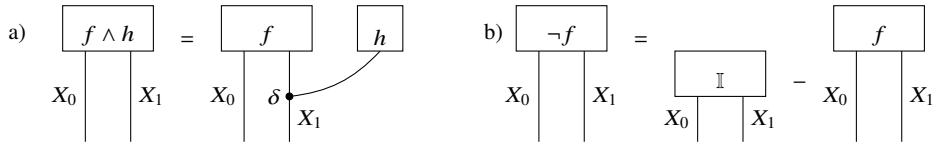



**Fig. 7.4** Decomposition schemes by hybrid calculus, using coordinatewise transforms of tensors (see Def. **??**). a) Conjunction performed by coordinatewise multiplications. b) Negations performed by coordinatewise subtraction from one.

**Theorem 7.7** *For any formulas $f, h$ we have*

$$\left\langle \beta^{\wedge} \left[ Y_{f\wedge h}, Y_f, Y_h \right], \epsilon_1 \left[ Y_{f\wedge h} \right] \right\rangle_{[Y_f, Y_h]} = \epsilon_1 \left[ Y_f \right] \otimes \epsilon_1 \left[ Y_h \right] \ .$$

*In particular, it holds that (see Figure 7.4a)*

$$(f \wedge h)[X_{[d]}] = \langle f, h \rangle_{[X_{[d]}]} \ .$$

*Proof.* We decompose

$$\beta^{\wedge}\left[Y_{f\wedge h}, Y_f, Y_h\right] = \epsilon_1\left[Y_{f\wedge h}\right] \otimes \epsilon_1\left[Y_f\right] \otimes \epsilon_1\left[Y_h\right] + \epsilon_0\left[Y_{f\wedge h}\right]\left(\mathbb{I}\left[Y_f, Y_h\right] - \epsilon_1\left[Y_f\right] \otimes \epsilon_1\left[Y_h\right]\right)$$

and get the first claim as

$$\begin{aligned}\left\langle \beta^{\wedge}\left[Y_{f\wedge h}, Y_f, Y_h\right], \epsilon_1\left[Y_{f\wedge h}\right]\right\rangle_{[Y_f, Y_h]} &= \left\langle \epsilon_1\left[Y_{f\wedge h}\right] \otimes \epsilon_1\left[Y_f\right] \otimes \epsilon_1\left[Y_h\right], \epsilon_1\left[Y_{f\wedge h}\right]\right\rangle_{[Y_f, Y_h]} \\ &= \epsilon_1\left[Y_f\right] \otimes \epsilon_1\left[Y_h\right] \ .\end{aligned}$$

To show the second claim we use

$$\begin{aligned}(f \wedge h)[X_{[d]}] &= \left\langle \beta^f\left[Y_f, X_{[d]}\right], \beta^h\left[Y_h, X_{[d]}\right], \beta^{\wedge}\left[Y_{f\wedge h}, Y_f, Y_h\right], \epsilon_1\left[Y_{f\wedge h}\right]\right\rangle_{[X_{[d]}]} \\ &= \left\langle \beta^f\left[Y_f, X_{[d]}\right], \beta^h\left[Y_h, X_{[d]}\right], \left(\epsilon_1\left[Y_f\right] \otimes \epsilon_1\left[Y_h\right]\right)\right\rangle_{[X_{[d]}]} \\ &= \langle f, h \rangle_{[X_{[d]}]} \ .\square\end{aligned}$$

A similar decomposition holds for negations, as we show next.

**Theorem 7.8** *For any formula $f$ we have*

$$\left\langle \beta^{\neg}\left[Y_{\neg f}, Y_f\right], \epsilon_1\left[Y_{\neg f}\right]\right\rangle_{[Y_f]} = \epsilon_0\left[Y_f\right] = \mathbb{I}\left[Y_f\right] - \epsilon_1\left[Y_f\right] \ .$$

*and*

$$\left\langle \beta^{\neg}\left[Y_f, Y_{\neg f}\right], \epsilon_0\left[Y_{\neg f}\right]\right\rangle_{[Y_f]} = \epsilon_1\left[Y_f\right] \ .$$

*In particular, it holds that (see Figure 7.4b)*

$$(\neg f)[X_{[d]}] = \mathbb{I}\left[X_{[d]}\right] - f\left[X_{[d]}\right] \ .$$

*Proof.* We have

$$\beta^{\neg}\left[Y_{\neg f}, Y_f\right] = \epsilon_1\left[Y_{\neg f}\right] \otimes \epsilon_0\left[Y_f\right] + \epsilon_0\left[Y_{\neg f}\right] \otimes \epsilon_1\left[Y_f\right]$$

and therefore get the second claim by contraction with $\epsilon_0\left[Y_{\neg f}\right]$

$$\begin{aligned}\left\langle \beta^{\neg}\left[Y_f, Y_{\neg f}\right], \epsilon_0\left[Y_{\neg f}\right]\right\rangle_{[Y_f]} &= \left\langle \epsilon_1\left[Y_{\neg f}\right] \otimes \epsilon_0\left[Y_f\right], \epsilon_0\left[Y_{\neg f}\right]\right\rangle_{[Y_f]} \\ &\quad + \left\langle \epsilon_0\left[Y_{\neg f}\right] \otimes \epsilon_1\left[Y_f\right], \epsilon_0\left[Y_{\neg f}\right]\right\rangle_{[Y_f]} \\ &= \epsilon_1\left[Y_f\right] \ .\end{aligned}$$

The first equation of the first claim follows similarly by contraction with $\epsilon_1\left[Y_{\neg f}\right]$ as

$$\left\langle \beta^{\neg}\left[Y_{\neg f}, Y_f\right], \epsilon_1\left[Y_{\neg f}\right]\right\rangle_{[Y_f]} = \epsilon_0\left[Y_f\right]$$

and the second equation using that $\mathbb{I}[X] = \epsilon_0[X] + \epsilon_1[X]$ and hence

$$\epsilon_0\left[Y_f\right] = \mathbb{I}\left[Y_f\right] - \epsilon_1\left[Y_f\right] \ .$$

To show the third claim, we contract the computation tensor $\beta^f\left[Y_f, X_{[d]}\right]$ to the formula $f$ on both sides of the first claim and get

$$\begin{aligned}&\left\langle \beta^{f}\left[Y_f, X_{[d]}\right], \beta^{\neg}\left[Y_{\neg f}, Y_f\right], \epsilon_1\left[Y_{\neg f}\right]\right\rangle_{[Y_f]}\\ &\quad = \left\langle \beta^{f}\left[Y_f, X_{[d]}\right], \mathbb{I}\left[Y_f\right]\right\rangle_{[Y_f]} - \left\langle \beta^{f}\left[Y_f, X_{[d]}\right], \epsilon_1\left[Y_f\right]\right\rangle_{[Y_f]} .\end{aligned}$$

We simplify this equation using the trivial head contraction identity of directed tensors of Cor. **??** stating that

$$f\left[X_{[d]}\right] = \left\langle \beta^{f}\left[Y_f, X_{[d]}\right], \epsilon_1\left[Y_f\right]\right\rangle_{[X_{[d]}]} \quad \text{and} \quad \neg f\left[X_{[d]}\right] = \left\langle \beta^{f}\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{[X_{[d]}]}$$

and arrive at the third claim

$$(\neg f)[X_{[d]}] = \mathbb{I}\left[X_{[d]}\right] - f\left[X_{[d]}\right] .\square$$

These theorems provide a mean to represent logical formulas by sums of one-hot encodings. Since any propositional formula can be represented by compositions of negations and conjunctions, they are universal. We further notice, that the resulting decomposition is a basis+ CP format, as further discussed in Chapter **??**. In Figure 7.5 we provide an example of this decomposition.

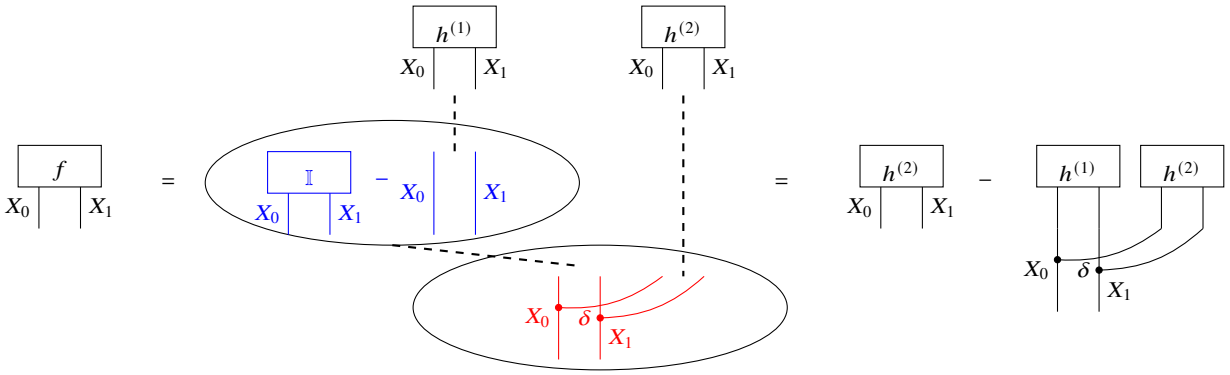



**Fig. 7.5** Example of a decomposition by hybrid calculus of a formula $f\left[X_1, X_2\right] = \neg h^{(1)}\left[X_1, X_2\right] \wedge h^{(2)}\left[X_1, X_2\right]$ into a sum of contractions.

# Chapter 8
# Message Passing Schemes

**Abstract** Message Passing schemes are effective ways to evaluate contractions. We first introduce based on semirings a more generic definitions of tensor network contraction. We then show that any such contraction can be splitted based on hypergraph cuts, which enables message-passing schemes to efficiently perform contractions. In the special cases of Boolean and basis encoding tensor networks, we further show even more efficient algorithms.

## 8.1 Semiring contraction

We here introduce with commutative semirings the algebraic structure required for the main separability property of contractions, and therefore for message-passing algorithms.

See [48, 49] for applications and [32] for algebraic and graph-theoretic foundations.

In an even broader perspective, contractions can be defined based on sequential compositions of categories [**?**].

### 8.1.1 Foundations

**Definition 8.1** A commutative semiring is a tuple $\mathbf{S} = (S, \oplus, \otimes, \mathbf{0}, \mathbf{1})$, where $S$ is a set, $\mathbf{0}, \mathbf{1} \in S$, and

$$\oplus : S \times S \to S \quad , \quad \otimes : S \times S \to S ,$$

such that the following properties hold:

(i) $(S, \oplus, \mathbf{0})$ is a commutative monoid[1].

(ii) $(S, \otimes, \mathbf{1})$ is a commutative monoid.

(iii) $\otimes$ distributes over $\oplus$, that is for any $s_0, s_1, s_2 \in S$

$$s_0 \otimes (s_1 \oplus s_2) = (s_0 \otimes s_1) \oplus (s_0 \otimes s_2) .$$

(iv) $\mathbf{0}$ annihilates $\otimes$, that is for any $s_0 \in S$

$$\mathbf{0} \otimes s_0 = \mathbf{0} .$$

Notice, that by these properties, the group operations $\oplus$ and $\otimes$ are specified on elements $\{0, 1\}$, except for $1 \oplus 1$, that is the following holds for any semiring:

| $\oplus$ | **0** | **1** |
|---|---|---|
| **0** | **0** | **1** |
| **1** | **1** | * |

| $\otimes$ | **0** | **1** |
|---|---|---|
| **0** | **0** | **0** |
| **1** | **0** | **1** |

[1] That is associativity, commutativity of $\oplus$ and identity of $s \oplus \mathbf{0} = s$ for all $s \in S$.

The Galois semiring and the Boolean semiring are the smallest semirings containing only the elements $\mathbf{0}, \mathbf{1}$ and differing only in $\mathbf{1} \oplus \mathbf{1}$.

We understand $S$ as the set of possible coordinates of a tensor.

**Definition 8.2** A tensor with coordinates in $S$ of order $d \in \mathbb{N}$ and leg dimensions $m_{[d]} = (m_0, \dots, m_{d-1})$, where $m_k \in \mathbb{N}$ for $k \in [d]$, is a map

$$\tau : \bigtimes_{k \in [d]} [m_k] \to S .$$

A tensor network with coordinates in $S$ is a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node is decorated by a dimension $m_v$ and each edge by a tensor of order $|e|$ and leg dimensions $m_e$.

**Definition 8.3** Given a semiring $\mathbf{S}$ and a tensor network $\{\tau^e [X_e] : e \in \mathcal{E}\}$ with coordinates in $S$, the contraction of the tensor network keeping the variables to a subset $\mathcal{U} \subset \mathcal{V}$ open is the tensor

$$\langle \{\tau^e [X_e] : e \in \mathcal{E}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{U}}]}$$

with coordinates at indices $x_{\mathcal{U}}$ by

$$\langle \{\tau^e [X_e] : e \in \mathcal{E}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{U}}]} = \bigoplus_{x_{\mathcal{V} \setminus \mathcal{U}}} \bigotimes_{e \in \mathcal{E}} \tau^e [X_e = x_e] .$$

The distributive property $(iii)$ of the semiring ensures, that we can compute contractions by sub-contractions.

**Theorem 8.1** *Given a disjoint partition of $\mathcal{E}$ into $\mathcal{E}^0, \mathcal{E}^1$, and a subset $\mathcal{U} \subset \mathcal{V}$, we define the sets $\mathcal{V}^0$ and $\mathcal{V}^1$ as the nodes appearing in $\mathcal{E}^0$, respectively $\mathcal{E}^1$. We then have*

$$\langle \{\tau^e [X_e] : e \in \mathcal{E}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{U}}]} = \langle \langle \{\tau^e [X_e] : e \in \mathcal{E}^0\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}^0 \cap (\mathcal{U} \cup \mathcal{V}^1)}]}, \langle \{\tau^e [X_e] : e \in \mathcal{E}^1\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}^1 \cap (\mathcal{U} \cup \mathcal{V}^0)}]} \rangle^{\mathbf{S}}_{[X_{\mathcal{U}}]} .$$

*Proof.* We use the associativity and commutativity of $\otimes$ to get for any index $x_{\mathcal{V}}$

$$\bigotimes_{e \in \mathcal{E}} \tau^e [X_e = x_e] = \left( \bigotimes_{e \in \mathcal{E}^0} \tau^e [X_e = x_e] \right) \otimes \left( \bigotimes_{e \in \mathcal{E}^1} \tau^e [X_e = x_e] \right) .$$

Let us now keep $x_{\mathcal{U}}$ fixed and investigate the $\oplus$ operation over the indices $x_{\mathcal{V} \setminus \mathcal{U}}$. Distributivity of $\otimes$ over $\oplus$ implies

$$\sum_{x_{\mathcal{V} \setminus \mathcal{U}}} \bigotimes_{e \in \mathcal{E}} \tau^e [X_e = x_e] = \sum_{x_{(\mathcal{V}^0 \cap \mathcal{V}^1) \setminus \mathcal{U}}} \left( \sum_{x_{\mathcal{V}^0 \setminus (\mathcal{U} \cup \mathcal{V}^1)}} \bigotimes_{e \in \mathcal{E}^0} \tau^e [X_e = x_e] \right) \otimes \left( \sum_{x_{\mathcal{V}^1 \setminus (\mathcal{U} \cup \mathcal{V}^0)}} \bigotimes_{e \in \mathcal{E}^1} \tau^e [X_e = x_e] \right)$$

which is equivalent to

$$\langle \{\tau^e [X_e] : e \in \mathcal{E}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \langle \langle \{\tau^e [X_e] : e \in \mathcal{E}^0\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}^0 \cap (\mathcal{U} \cup \mathcal{V}^1)}]}, \langle \{\tau^e [X_e] : e \in \mathcal{E}^1\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}^1 \cap (\mathcal{U} \cup \mathcal{V}^0)}]} \rangle^{\mathbf{S}}_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} .$$

Since therefore both tensors of the claimed contraction equation coincide on all coordinates, they are equal. □

As a consequence, message passing schemes such as variable elimination generalize to semiring tensor network contractions.

### 8.1.2 Examples

#### 8.1.2.1 Sum-product TN

Standard TN use the semiring

$$(\mathbb{R}, +, \cdot, 0, 1) .$$

Variants are:

- Restriction of $\mathbb{R}$ to non-negative numbers $[0, \infty)$: Ensuring the probabilistic interpretation
- Generalization of $\mathbb{R}$ to complex numbers $\mathbb{C}$: Used in quantum computing

#### 8.1.2.2 Boolean semiring

We take $S = \{0, 1\}$ and interpret 0 as False, 1 as True. We define the disjunction and conjunction operations by

$$\vee [X_0, X_1] = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} [X_0, X_1]$$

and

$$\wedge [X_0, X_1] = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} [X_0, X_1] .$$

Tensors over $\{0, 1\}$ are interpreted as constraints, and their tensor networks as constraint networks.

The satisfiability of a constraint network $\{\tau^e [X_e] : e \in \mathcal{E}\}$ is the contraction

$$\bigvee_{x_\mathcal{V}} \bigwedge_{e \in \mathcal{E}} \tau^e [X_e = x_e] = \langle \{\tau^e [X_e] : e \in \mathcal{E}\} \rangle_\emptyset^{(\{0,1\}, \vee, \wedge, 0, 1)} .$$

As an additional property, we have invariance of adding subcontractions to the network. In general tensor networks, this holds only for the support of subcontractions (being the operator mapping 0 to 0 and any other $s \in S$ to 1).

The relation with the standard semiring is for boolean tensor networks

$$\langle \tau^\mathcal{G} \rangle_{[X_\mathcal{U}]}^{(\{0,1\}, \vee, \wedge, 0, 1)} \mathbb{I}_{\neq 0} \left( \langle \tau^\mathcal{G} \rangle_{[X_\mathcal{U}]} \right)$$

In this way, the message passing based algorithms in Chapter 2 and Chapter 3 simplify when chosing the satisfaction semiring instead of the default semiring.

We can use the Boolean semiring to represent first-oder logic formulas with existential quantifications as contraction equations. A formula

$$\forall x_\mathcal{U} : \exists x_\mathcal{W} : f [X_\mathcal{U} = x_\mathcal{U}, X_\mathcal{W} = x_\mathcal{W}]$$

is the contraction equation

$$\langle f [X_{\mathcal{U} \cup \mathcal{W}}] \rangle_{[X_\mathcal{U}]}^{(\{0,1\}, \vee, \wedge, 0, 1)} = \mathbb{I} [X_\mathcal{U}] .$$

#### 8.1.2.3 Maximum calculus

The maximum semiring is the tuple

$$(\mathbb{R}^+, \max, \cdot, 0, 1) .$$

The full contraction is the maximal coordinate in the tensor

$$\langle \tau \left[ X_{[d]} \right] \rangle_\emptyset^{(\mathbb{R}^+, \max, \cdot, 0, 1)} = \max_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} \tau \left[ X_{[d]} = x_{[d]} \right] .$$

When leaving variables open in the contraction, we get the maximal coordinate in the specific slices of the tensor, namely

$$\langle \tau \left[ X_{[d]} \right] \rangle_{[X_{\mathcal{U}} = x_{\mathcal{U}}]}^{(\mathbb{R}^+, \max, \cdot, 0, 1)} = \max_{x_{\mathcal{V} \setminus \mathcal{U}}} \tau \left[ X_{[d]} = x_{[d]} \right] .$$

Analogous properties has the minimal semiring

$$(\mathbb{R}^+, \min, \cdot, \infty, 1) .$$

These schemes only work, when taking the minimal coordinate keeping all variables open. When searching for the maximal/minimal coordinate of a contraction with some variables closed, we need to first coarse grain the tensor network.

These contractions are used in encoding-decoding schemes [57, 51]. Message passing schemes involving the maximum semiring are known as max-product algorithms [45], and generalizations of the Viterbi algorithm.

#### 8.1.2.4 Tropical semiring

The tropical semiring is

$$\mathbb{T} = (\mathbb{R} \cup \{\infty\}, \min, +, \infty, 0) .$$

The contraction of a tensor gives the minimum

$$\langle \tau \left[ X_{[d]} \right] \rangle_{\varnothing}^{\mathbb{T}} = \min_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} \tau \left[ X_{[d]} \right] .$$

The tensor product in the tropical semiring sums the leg vector coordinates, that is

$$\langle \{ \rho^k \left[ X_k \right] \, : \, k \in [d] \} \rangle_{[X_{[d]} = x_{[d]}]}^{\mathbb{T}} = \sum_{k \in [d]} \rho^k \left[ X_k = x_k \right] .$$

Further application in [42] and on spin glasses [49].

#### 8.1.2.5 Galois field of two elements

The Galois field of two elements is the semiring

$$\mathbb{F}_2 := (\{0, 1\}, \oplus, \cdot, 0, 1) ,$$

where $\oplus$ denotes the sum $\mod 2$ and $\cdot$ the standard multiplication.

*Example 8.1* Contractions in the semiring $\mathbb{F}_2$ While $\left\langle f \left[ X_{[d]} \right] \right\rangle_{[\varnothing]}$ is the model count, the contraction with respect to $\mathbb{F}_2$ decides whether the number of models is odd or even:

$$\begin{aligned} \langle f \left[ X_{[d]} \right] \rangle_{\varnothing}^{\mathbb{F}_2} &= \bigoplus_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} f \left[ X_{[d]} = x_{[d]} \right] \\ &= \left| \{ x_{[d]} \, : \, f \left[ X_{[d]} = x_{[d]} \right] = 1 \} \right| \mod 2 . \end{aligned}$$

We will investigate that semiring further in Sect. 10.4, where we relate CP decompositions with respect to that semiring to Reed-Muller expansions.

### 8.1.3 Tensor network formats

Based on the semiring contraction, we can define tensor network formats for arbitrary semirings.

- EL depends only on the multiplicative structure of the semiring. Therefore equivalent for all the semirings with the standard product.
- CP format is the semiring sum of EL. Examples in the galois field semiring in Sect. 10.4.

### 8.1.4 Basis calculus

Basis calculus can be generalized to arbitrary semirings.

**Definition 8.4** Let $\mathbf{S}$ be a commutative semiring and $q : \bigtimes_{k\in[d]}[m_k] \to \bigtimes_{\ell\in[p]}[n_\ell]$ a function. Then the basis encoding of $q$ with respect to the $\mathbf{S}$ is the tensor $\beta^q\left[Y_{[p]}, X_{[d]}\right]$ with coordinates

$$\beta^{q,\mathbf{S}}[Y_{[p]} = y_{[p]}, X_{[d]} = x_{[d]}] = \begin{cases} \mathbf{1} & \text{if} \quad q(x_{[d]}) = y_{[p]} \\ \mathbf{0} & \text{else}\,. \end{cases}$$

Based on the restricted group operation tables we show the decomposition property of generic basis encodings.

**Theorem 8.2** *Let $\mathbf{S}$ be a commutative semiring and $q : \bigtimes_{k\in[d]}[m_k] \to [n]$ and $g : [n] \to \bigtimes_{\ell\in[p]}[n_\ell]$ functions. Then we have*

$$\langle\beta^{q,\mathbf{S}}[Z, X_{[d]}], \beta^{g,\mathbf{S}}[Y_{[p]}, Z]\rangle^{\mathbf{S}}_{[X_{[d]},Y_{[p]}]} = \beta^{g\circ q,\mathbf{S}}[Y_{[p]}, X_{[d]}]$$

*Proof.* For any indices $x_{[d]}$ and $y_{[p]}$ we have

$$\begin{aligned}&\langle\beta^{q,\mathbf{S}}[Z, X_{[d]}], \beta^{g,\mathbf{S}}[Y_{[p]}, Z]\rangle^{\mathbf{S}}_{[X_{[d]}=x_{[d]},Y_{[p]}=y_{[p]}]} \\ &\quad= \bigoplus_{z\in[n]} \beta^{q,\mathbf{S}}[Z = z, X_{[d]} = x_{[d]}] \otimes \beta^{g,\mathbf{S}}[Y_{[p]} = y_{[p]}, Z = z]\end{aligned}$$

Now, for any $z$ for which either $q(x_{[d]}) \neq z$ or $g(z) \neq y_{[p]}$, the summand is the neutral element $\mathbf{0}$ ($\mathbf{0}\otimes\mathbf{0}$, $\mathbf{0}\otimes\mathbf{1}$ or $\mathbf{1}\otimes\mathbf{0}$). If and only if $g \circ q(x_{[d]}) = y_{[p]}$, there is one summand $\mathbf{1}\otimes\mathbf{1} = \mathbf{1}$, and the contraction coordinate of the contraction is thus $\mathbf{1}$. The coordinates therefore coincide with those of $\beta^{g\circ q,\mathbf{S}}[Y_{[p]}, X_{[d]}]$. □

## 8.2 Contraction of tree tensor networks

We first define tree hypergraphs.

**Definition 8.5** A hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is called connected, if for any two nodes $v, \tilde{v} \in \mathcal{V}$ we find a sequence $v_{[n]}$ of nodes, such that $v_0 = v, v_{n-1} = \tilde{v}$ and for any $k \in [n-1]$ there is $e \in \mathcal{E}$ with $v_k, v_{k+1} \in e$. A $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is called a tree, if it is connected and if for any $e \in \mathcal{E}$ the hypergraph $(\mathcal{V}, \mathcal{E}\backslash\{e\})$ is not connected.

### 8.2.1 Tree message passing

### 8.2.2 Analysis

We denote for each pair $(e_0, e_1)$ the subset $\mathcal{E}^{\to(e_0,e_1)} \subset \mathcal{E}$ as the subset of edges $e \in \mathcal{E}$, for which each path to $e_1$ passes through $e_0$. Note, that by construction $e_0 \in \mathcal{E}^{\to(e_0,e_1)}$.

**Theorem 8.3** *For any tensor network on a tree hypergraph, Algorithm* **??** *terminates in the tree-based implementation and returns final messages*

$$\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] = \langle\{\tau^e\left[X_e\right] : e \in \mathcal{E}^{\to(e_0,e_1)}\}\rangle^{\mathbf{S}}_{[X_{e_0\cap e_1}]}$$

**Algorithm 8** Tree Belief Propagation

**Require:** Tensor network $\tau^{\mathcal{G}}$ on a hypergraph $\mathcal{G}$ with respect to a semiring $\mathbf{S}$
**Ensure:** Messages $\{\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] : (e_0,e_1)\in\mathcal{E}^{\to}\}$

Initialize a message scheduler $S = \{(e_0,e_1)\in\mathcal{E}^{\to} : e_0 \text{ a leaf in the tree } (\mathcal{E},\mathcal{E}^{\to})\}]$
**while** $S$ not empty **do**
Pop a $(e_0,e_1)$ pair from $S$
Compute the message

$$\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] = \langle\{\tau^{e_0}\left[X_{e_0}\right]\}\cup\{\chi_{e_2\to e_0}\left[X_{e_2\cap e_0}\right] : (e_2,e_0)\in\mathcal{E}^{\to},\ e_2\neq e_1\}\rangle^{\mathbf{S}}_{[X_{e_0\cap e_1}]}$$

Update $S$ by all messages $(e_1,e_3)$ which have not yet been sent, if all messages $(e_2,e_1)$ with $e_2\neq e_3$ have been sent.
**end while**
**return** Messages $\{\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] : (e_0,e_1)\in\mathcal{E}^{\to}\}$

*Proof.* We show this property by induction over the edge sets $\mathcal{E}^{\to(e_0,e_1)}$ to pairs $(e_0,e_1)\in\mathcal{E}^{\to}$, such that $\left|\mathcal{E}^{\to(e_0,e_1)}\right|\leq n$. Notice, that since always $e_0\in\mathcal{E}^{\to(e_0,e_1)}$ we have $n\geq 1$.

$n=1$: In this case we have $\mathcal{E}^{\to(e_0,e_1)}=\{e_0\}$ and $e_0$ is a leaf of the tree-hypergraph $\mathcal{G}$. The claimed message property holds thus by definition.

$n\to n+1$: Let us assume, that the message obeys the claimed property for edge sets with cardinality up to $n$. If there is no edge set with cardinality $n+1$, the property holds also for those with cardinality up to $n+1$. If there is an edge set $\mathcal{E}^{\to(e_0,e_1)}$ with size $n+1$, we have

$$\mathcal{E}^{\to(e_0,e_1)} = \{e_0\}\cup\left(\bigcup_{e_2\in\mathcal{E}^{\to}}\mathcal{E}^{\to(e_2,e_0)}\right).$$

The message $\chi_{e_0\to e_1}$ is sent, once all messages $\chi_{e_2\to e_0}$ to $(e_2,e_0)\in\mathcal{E}^{\to}\{(e_1,e_0)\}$ arrived. By definition we have

$$\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] = \langle\{\tau^{e_0}\left[X_{e_0}\right]\}\cup\{\chi_{e_2\to e_0}\left[X_{e_2\cap e_0}\right] : (e_2,e_0)\in\mathcal{E}^{\to},\ e_2\neq e_1\}\rangle^{\mathbf{S}}_{[X_{e_0\cap e_1}]}$$

Now we use the induction assumption on $\mathcal{E}^{\to(e_2,e_0)}$ (since its cardinality is at most $n$) and get

$$\begin{aligned}
\chi_{e_0\to e_1}\left[X_{e_0\cap e_1}\right] &= \langle\{\tau^{e_0}\left[X_{e_0}\right]\}\cup\left(\bigcup_{(e_2,e_0)\in\mathcal{E}^{\to},\,e_2\neq e_1}\langle\{\tau^{e_3}\left[X_{e_3}\right] : e_3\in\mathcal{E}^{\to(e_2,e_0)}\}\rangle^{\mathbf{S}}_{[X_{e_2\cap e_0}]}\right)\rangle^{\mathbf{S}}_{[X_{e_0\cap e_1}]}\\
&= \langle\{\tau^{e_0}\left[X_{e_0}\right]\}\cup\left(\bigcup_{(e_2,e_0)\in\mathcal{E}^{\to},\,e_2\neq e_1}\{\tau^{e_3}\left[X_{e_3}\right] : e_3\in\mathcal{E}^{\to(e_2,e_0)}\}\right)\rangle^{\mathbf{S}}_{[X_{e_0\cap e_1}]}\\
&= \langle\{\tau^{e}\left[X_{e}\right] : e\in\mathcal{E}^{\to(e_0,e_1)}\}\rangle^{\mathbf{S}}_{[X_{e_0\cap e_1}]}
\end{aligned}$$

Here we used the commutation of contraction property in the second equation, which is justified by the assumed tree property of the hypergraph. Thus, the message property holds also for any edge sets of size $n+1$.

By induction, the claimed message property therefore holds for all final messages. □

## 8.3 Boolean message passing

We investigate message passing schemes with respect to the Boolean semiring $(\{0,1\},\vee,\wedge,0,1)$. To this end, we derive the schemes based on properties of the Boolean semiring and then show their application to sparsify generic contractions.

### 8.3.1 Properties of the Boolean semiring

The main property of contractions in the Boolean semiring is the invariance under adding subcontractions, which we state in the next theorem.

**Theorem 8.4** *Let $\tau^{\mathcal{G}}$ be an tensor network on an arbitrary hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with respect to the Boolean semiring. For any subsets $\mathcal{U} \subset \mathcal{V}$ and $\tilde{\mathcal{E}} \subset \mathcal{E}$ we have*

$$\langle \tau^{\mathcal{G}} \cup \{\langle\{\tau^e\left[X_e\right] \,:\, e \in \tilde{\mathcal{E}}\}\rangle^{(\{0,1\},\vee,\wedge,0,1)}_{[X_{\mathcal{U}}]}\}\rangle^{(\{0,1\},\vee,\wedge,0,1)}_{[X_{\mathcal{V}}]} = \langle \tau^{\mathcal{G}}\rangle^{(\{0,1\},\vee,\wedge,0,1)}_{[X_{\mathcal{V}}]}\,.$$

*Proof.* Coordinatewise: Only invariant, if subcontraction evaluates to 0. But then by monotonocity of subcontractions also the whole contraction evaluates to 0, therefore invariant. □

### 8.3.2 Application on sum-product contractions

Thm. 8.4 can be generalized to contractions on more general semirings sharing the elements 0, 1 with $(\{0,1\}, \vee, \wedge, 0, 1)$. In such way we can switch to message passing schemes in the Boolean semiring to sparsify more generic tensor networks.

### 8.3.3 Generic support tensors

Even more generic, we define the support operator on a generic tensor with respect to a semiring $\mathbf{S}$ as

$$\mathbb{I}_{\neq \mathbf{0}} \circ \tau\left[X_{[d]} = x_{[d]}\right] = \begin{cases} \mathbf{0} & \text{if} \quad \tau\left[X_{[d]} = x_{[d]}\right] = \mathbf{0} \\ \mathbf{1} & \text{else}\,. \end{cases}$$

We notice that the support operator leaves any tensor over semirings with $S = \{\mathbf{0}, \mathbf{1}\}$ invariant. This is the case for the Boolean semiring $(\{0,1\}, \vee, \wedge, 0, 1)$ and the Galois field $\mathbb{F}_2$.

**Lemma 8.1** *Any tensor is invariant under contraction with its support tensor, that is*

$$\tau\left[X_{[d]}\right] = \langle \tau\left[X_{[d]}\right], \mathbb{I}_{\neq \mathbf{0}} \circ \tau\left[X_{[d]}\right]\rangle^{\mathbf{S}}_{[X_{[d]}]}\,.$$

*Proof.* For any coordinate $s \neq \mathbf{0}$ the support operator returns $\mathbf{1}$, and by the multiplicative invariance property we have $s \otimes \mathbf{1} = s$. If $s = \mathbf{0}$ the support operator returns $\mathbf{0}$ and we have $\mathbf{0} \otimes \mathbf{0} = \mathbf{0}$. □

**Definition 8.6** A semiring $\mathbf{S}$ is called *inverse-free*, if for any $s_0, s_1 \in S$ with $s_0 \oplus s_1 = \mathbf{0}$ we have $s_0 = s_1 = \mathbf{0}$.

Note that $([0,\infty), +, \cdot, 0, 1)$ is inverse-free, while $(\mathbb{R}, +, \cdot, 0, 1)$ is not. The Boolean semiring $(\{0,1\}, \vee, \wedge, 0, 1)$ is inverse-free, but the Galois field $\mathbb{F}_2$ is not.

**Lemma 8.2** *The support of a contraction obeys*

$$\mathbb{I}_{\neq \mathbf{0}} \circ \langle \tau^{\mathcal{G}}\rangle^{\mathbf{S}}_{[X_{\mathcal{U}}]} \prec \langle\{\mathbb{I}_{\neq \mathbf{0}} \circ \tau^e\left[X_e \,:\, e \in \mathcal{E}\right]\}\rangle^{(\{0,1\},\vee,\wedge,0,1)}_{[X_{\mathcal{U}}]}\,.$$

*Here the Boolean semiring is defined with the neutral elements $\mathbf{0}, \mathbf{1}$ of the semiring $\mathbf{S}$, and the inequality denotes partial ordering of Boolean tensors. If the semiring is inverse free, then the above inequality holds straight.*

*Proof.* To show the first claim, let $x_{\mathcal{U}}$ be an index such that

$$\langle\{\mathbb{I}_{\neq \mathbf{0}} \circ \tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}\rangle^{(\{0,1\},\vee,\wedge,0,1)}_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \mathbf{0}\,.$$

It suffices to show that for this index we have $\langle \tau^{\mathcal{G}}\rangle^{\mathbf{S}}_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \mathbf{0}$. Since the Boolean semiring is inverse-free we have for any extension $x_{\mathcal{V}}$ of $x_{\mathcal{U}}$ to $\mathcal{V}$ that

$$\langle\{\mathbb{I}_{\neq \mathbf{0}} \circ \tau^e\left[X_e \,:\, e \in \mathcal{E}\right]\}\rangle^{(\{0,1\},\vee,\wedge,0,1)}_{[X_\mathcal{V}=x_\mathcal{V}]} = \mathbf{0}\,.$$

We therefore find $e \in \mathcal{E}$ such that for the restriction $x_e$ of $x_\mathcal{V}$ to $e$ we have

$$\tau^e\left[X_e = x_e\right] = \mathbf{0}$$

and thus

$$\langle\tau^\mathcal{G}\rangle^{\mathbf{S}}_{[X_\mathcal{V}=x_\mathcal{V}]} = \mathbf{0}\,.$$

It follows that also

$$\langle\tau^\mathcal{G}\rangle^{\mathbf{S}}_{[X_\mathcal{U}=x_\mathcal{U}]} = \mathbf{0}\,.$$

The second claim, namely the vise versa inequality, can be shown analogously using that $\mathbf{S}$ is inverse-free. □

As an example, where the inequality in Lem. 8.2 does not hold straight, consider the contraction of

$$\tau^0\left[X_0\right]\begin{pmatrix}1\\-1\end{pmatrix}\left[X_0\right] \quad \text{and} \quad \tau^1\left[X_0,X_1\right]\begin{pmatrix}1&1\\1&1\end{pmatrix}\left[X_0,X_1\right]\,.$$

The support is the trivial tensor in both cases, but $\left\langle\tau^0\left[X_0\right],\tau^1\left[X_0,X_1\right]\right\rangle_{[X_1]} = \mathbb{I}\left[X_1\right]$. When we restrict to the non-negative real numbers in the sum-product semiring, the semiring is inverse free and Lem. 8.2 would guarantee equality.

Boolean message passing can therefore be applied to sparsify generic contractions to arbitrary semirings:

- Add the support of the contraction (Lem. 8.1)
- Decompose the support into local network (Lem. 8.2)
- Perform Boolean message passing to simplify the support

## 8.4 Hypergraph manipulations

### 8.4.1 Bethe hypergraphs

**Definition 8.7 (Delta tensor on generic semirings)** Given a semiring $\mathbf{S} = (S, \oplus, \otimes, \mathbf{0}, \mathbf{1})$ and a collection of variables $X_e$ of same dimension, the corresponding delta tensor $\delta\left[X_e\right]$

$$\delta\left[X_e = x_e\right] = \begin{cases}\mathbf{1} & \text{if} \quad \forall v,\tilde{v} \in e : x_v = x_{\tilde{v}}\\ \mathbf{0} & \text{else}\end{cases}\,.$$

The main manipulation operation with delta tensor is shown in the next lemma.

**Lemma 8.3** *Given a tensor network $\tau^\mathcal{G}\left[X_{[d]}\right]$ with respect to a semiring $\mathbf{S} = (S, \oplus, \otimes, \mathbf{0}, \mathbf{1})$. For any set of edges $\tilde{\mathcal{E}} \subset \mathcal{E}$ each containing a variable $v$ define cores $\tilde{\tau}^e\left[X_{e\setminus\{v\}}, X_{v_e}\right]$ with coordinates*

$$\tilde{\tau}^e\left[X_{e\setminus\{v\}} = x_{e\setminus\{v\}}, X_{v_e} = x_v\right] = \tau^e\left[X_e = x_e\right]\,.$$

*Then we have*

$$\tau^\mathcal{G}\left[X_{[d]}\right] = \langle\{\tau^e\left[X_e\right] \,:\, e \notin \tilde{\mathcal{E}}\} \cup \{\tilde{\tau}^e\left[X_{e\setminus\{v\}}, X_{v_e}\right] \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\delta\left[X_v, X_{v_e} \,:\, e \in \tilde{\mathcal{E}}\right]\}\rangle^{\mathbf{S}}_{[X_\mathcal{V}]}\,.$$

*Proof.* Using the neutrality properties of $\mathbf{1}$ and $\mathbf{0}$. □

**Definition 8.8** Given a tensor network $\tau^\mathcal{G}$ on a decorated hypergraph $\mathcal{G}$, we define the Bethe hypergraph $\tilde{\mathcal{G}}$ as $(\mathcal{U}, \tilde{\mathcal{E}} \cup \Delta^\mathcal{G})$ where we have

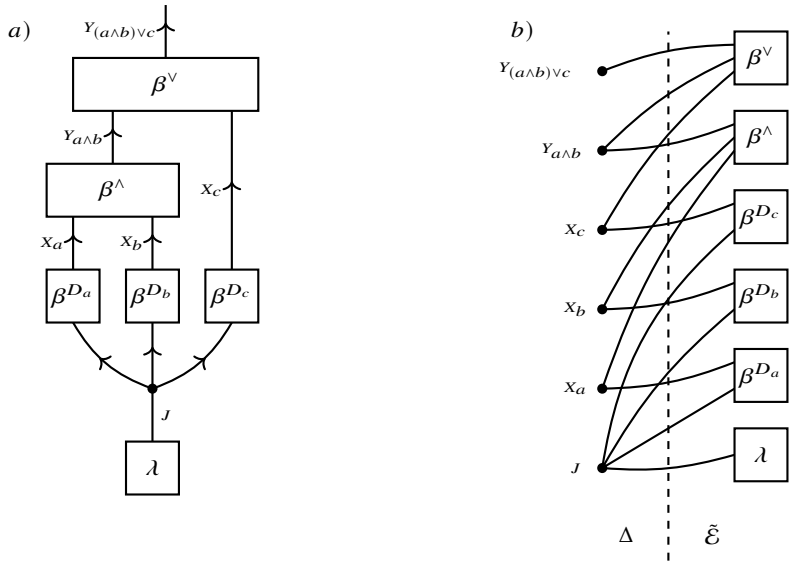



**Fig. 8.1** Example of a Bethe Cluster Graph. a) Example of a Tensor Network $\tau^{\mathcal{G}}$, which represents the by $\lambda$ averaged evaluation of the formula $(a \wedge b) \vee c$ on data $D$. b) Corresponding Bethe Cluster Hypergraph, which dual is bipartite by the sets $\Delta$ and $\tilde{\mathcal{E}}$.

- Recolored Edges $\tilde{\mathcal{E}} = \{\tilde{e} \,:\, e \in \mathcal{E}\}$ where $\tilde{e} = \{v^e \,:\, v \in e\}$, which decoration tensor has same coordinates as $\tau^e$
- Nodes $\mathcal{U} = \mathcal{V} \cup (\bigcup_{e\in\mathcal{E}} \tilde{e})$
- Delta Edges $\Delta^{\mathcal{G}} = \left\{\{v\} \cup \{v^e \,:\, e \ni v\} \,:\, v \in \mathcal{V}\right\}$, each of which decorated by a delta tensor $\delta^{\{v^e : e\ni v\}}$

The resulting overlap graph is bipartite, since any variable appears exactly in one cluster in $\tilde{\mathcal{E}}$ and in one cluster of $\Delta^{\mathcal{G}}$. This further makes the dual of the Bethe Cluster Hypergraph a proper graph (i.e. edges consistent of node pairs).

We effectively copy each node $v$ by $v^e$. Let us now show, that when assigning dirac delta tensors the contraction reproduces the original contraction.

By adding delta tensors to each node $v \in \mathcal{V}$ and defining its leg variables by $v^e$ for $e \in \mathcal{E}$. We mark each such delta tensor by a cluster in $\Delta^{\mathcal{G}}$, as defined in the following (see also Figure 8.1).

By Lem. 8.4 this construction does not change contractions.

**Lemma 8.4 (Bethe Invariance)** *Given a tensor network $\tau^{\mathcal{G}}\left[X_{[d]}\right]$ on a hypergraph $\mathcal{G}$. We define a tensor network on its bethe hypergraph $\tilde{\mathcal{G}}$ by the tensors for $e \in \mathcal{E}$*

$$\tilde{\tau}^{\tilde{e}}\left[X_{\tilde{e}}\right] = \left\langle \tau^{e}\left[X_e\right], \delta\left[X_e, X_{\tilde{e}}\right]\right\rangle_{[X_{\tilde{e}}]}$$

*and the tensors for $\tilde{e} \in \Delta^{\mathcal{G}}$*

$$\tilde{\tau}^{\tilde{e}}\left[X_{\tilde{e}}, X_v\right] = \delta\left[X_{\tilde{e}}, X_v\right] .$$

*Then we have*

$$\tau^{\mathcal{G}}\left[X_{[d]}\right] = \left\langle \{\tilde{\tau}^{\tilde{e}}\left[X_{\tilde{e}}\right] \,:\, \tilde{e} \in \tilde{\mathcal{E}} \cup \Delta^{\mathcal{G}}\}\right\rangle_{[X_{\mathcal{V}}]}$$

*Proof.* By invariance of adding dirac deltas. □

Equal constructions:

- BP on factor graphs in [57]
- Bethe Cluster graphs in [45]

### 8.4.2 Coarse graining

We often account for tensor networks, which are not defined on a tree hypergraph. In that case, one can perform graph manipulations, which we call coarse graining schemes, to reformulate the contraction on an equivalent tree hypergraph.

**Definition 8.9** Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and $\tilde{\mathcal{G}} = (v, \tilde{\mathcal{E}})$ be two hypergraphs sharing their node set and $c : \mathcal{E} \to \tilde{\mathcal{E}}$ a map. We say that $c$ is a coarse graining map and $\tilde{\mathcal{G}}$, if for all $e \in \mathcal{E}$

$$e \subset c(e) .$$

For any tensor network $\tau^{\mathcal{G}}[X_{\mathcal{V}}]$ on $\mathcal{G}$ we further define a coarse grained tensor network $\tilde{\tau}^{\tilde{\mathcal{G}}}[X_{\mathcal{V}}]$ on $\tilde{\mathcal{G}}$ with hypercores to $\tilde{e} \in \tilde{\mathcal{E}}$ by

$$\tilde{\tau}^{\tilde{e}}[X_{\tilde{e}}] = \langle \{\tau^{e}[X_e] \,:\, \tilde{e} = c(e)\} \rangle^{\mathbf{S}}_{[X_{\tilde{e}}]} .$$

In the literature such coarse graining procedures appear as Cluster Graphs.
We notice that coarse graining of tensor networks preserves their contractions, since

$$\begin{aligned}
\tau^{\mathcal{G}}[X_{\mathcal{V}}] &= \langle \{\tau^{e}[X_e] \,:\, e \in \mathcal{E}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}}]} \\
&= \langle \bigcup_{\tilde{e} \in \tilde{\mathcal{E}}} \{\tau^{e}[X_e] \,:\, c(e) = \tilde{e}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}}]} \\
&= \langle \{ \langle \{\tau^{e}[X_e] \,:\, c(e) = \tilde{e}\} \rangle^{\mathbf{S}}_{[X_e]} \,:\, \tilde{e} \in \tilde{\mathcal{E}} \} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}}]} \\
&= \langle \{\tilde{\tau}^{\tilde{e}}[X_{\tilde{e}}] \,:\, \tilde{e} \in \tilde{\mathcal{E}}\} \rangle^{\mathbf{S}}_{[X_{\mathcal{V}}]} \\
&= \tilde{\tau}^{\tilde{\mathcal{G}}}[X_{\mathcal{V}}] .
\end{aligned}$$

#### 8.4.2.1 Variable Elimination

Given an order of the nodes the variable elimination procedure provides a coarse graining scheme of an augmented graph.

The variable elimination algorithm is simultaneously defining an augmentation of a tensor network with delta cores and defining a coarse graining mechanism of the augmentation.

**Lemma 8.5** *The Algorithm 9 returns a coarse graining map $c$ towards a tree hypergraph.*

*Proof.* By construction we have $e \subset c(e)$ for any $e \in \mathcal{E}$, thus $c$ is a coarse graining map.

□

#### 8.4.2.2 Junction tree algorithm

Classically the junction tree algorithm is formulated on the extended graph (we define extended graphs here by two nodes adjacent if and only if they are contained in a hypergraph) [46], where one exploits that a graph has a junction tree if and only if it is triangulated [47]. The junction tree algorithm therefore constructs a triangulation of the graph by adding edges (also called chordal graph [45]), and gets a tree-hypergraph by its cliques. The coarse graining map to a tree hypergraph is thus the selection of the maximal clique in the triangulated extended graph, which contains the nodes of the hyperedge. Note that finding a minimal triangulation is NP-hard.

**Algorithm 9** Variable Elimination

**Require:** Tensor network $\{\tau^e\,[X_e]\,:\,e\in\mathcal{E}\}$ with total order of the nodes
**Ensure:** Coarse graining map $c$ towards a tree hypergraph $(\mathcal{V},\tilde{\mathcal{E}})$

$\mathcal{C}\leftarrow\emptyset$ of cores already contracted
$c:\emptyset\to\emptyset$
$\mathcal{M}$ auxiliary edges (messages)
**for** $v\in\mathcal{V}$ following the total order **do**
**Step 1:** Plan new message $e_v$ given the current ones $\mathcal{M}$

- Choose all edges from $\mathcal{E}$ containing $v$, which have not been contracted before and update the contracted ones

$$\mathcal{E}^v\leftarrow\{e\,:\,e\in\mathcal{E}\backslash\mathcal{C},\,v\in e\}$$
$$\mathcal{C}\leftarrow\mathcal{C}\cup\mathcal{E}^v$$

- Choose and remove all message edges from $\mathcal{M}$ containing $v$

$$\mathcal{M}^v\leftarrow\{e_{\tilde{v}}\,:\,e_{\tilde{v}}\in\mathcal{M},\,v_{\tilde{v}}\in e_{\tilde{v}}\}$$
$$\mathcal{M}\leftarrow\mathcal{M}\backslash\mathcal{M}^v$$

- Build a coarse grained edge and add it to the messages

$$e_v\leftarrow\{v\}\cup\bigcup_{e\in\mathcal{E}^v}\{\bar{v}^v\,:\,\bar{v}\in e\backslash\{v\}\}\cup\bigcup_{e_{\tilde{v}}\in\mathcal{M}^v}\{\bar{v}^{\tilde{v}}\,:\,\bar{v}^{\tilde{v}}\in e_{\tilde{v}}\backslash\{\tilde{v}\}\}$$
$$\mathcal{M}\leftarrow\mathcal{M}\cup\{e_v\}$$

**Step 2**: Build manipulated cores of an augmented network, which are coarse grained to $e_v$

- Rename the variables at each $e\in\mathcal{E}^v$, keeping only $v$

$$\tilde{\tau}^e\left[X_v,X_{\tilde{v}_v}\,:\,\tilde{v}\in e\backslash\{v\}\right]\leftarrow\langle\tau^e\,[X_e]\cup\{\delta\left[X_{\tilde{v}},X_{\tilde{v}_v}\right]\,:\,\tilde{v}\in e\backslash\{v\}\}\rangle^{\mathbf{S}}_{[X_v,X_{\tilde{v}_v}\,:\,\tilde{v}\in e\backslash\{v\}]}$$

- Add delta core to current variable

$$\delta\left[X_v,X_{v_{\tilde{v}}}\,:\,e_{\tilde{v}}\in\mathcal{M}^v\right]$$
$$c(\{v\}\cup\{v_{\tilde{v}}\,:\,e_{\tilde{v}}\in\mathcal{M}^v\})=e_v$$

- Add delta cores to any other variable $\bar{v}^v\in e_v\{v\}$

$$\delta\,[X_{\bar{v}^v},X_{\tilde{v}^{\bar{v}}}\,:\,e_{\tilde{v}}\in\mathcal{M}^v,\,\bar{v}\in\tilde{v}]\,c(\{\bar{v}^v\}\cup\{\bar{v}^{\tilde{v}}\,:\,e_{\tilde{v}}\in\mathcal{M}^v,\,\bar{v}\in\tilde{v}\})=e_v$$

**end for**
**return** $c,\tilde{\mathcal{E}}$

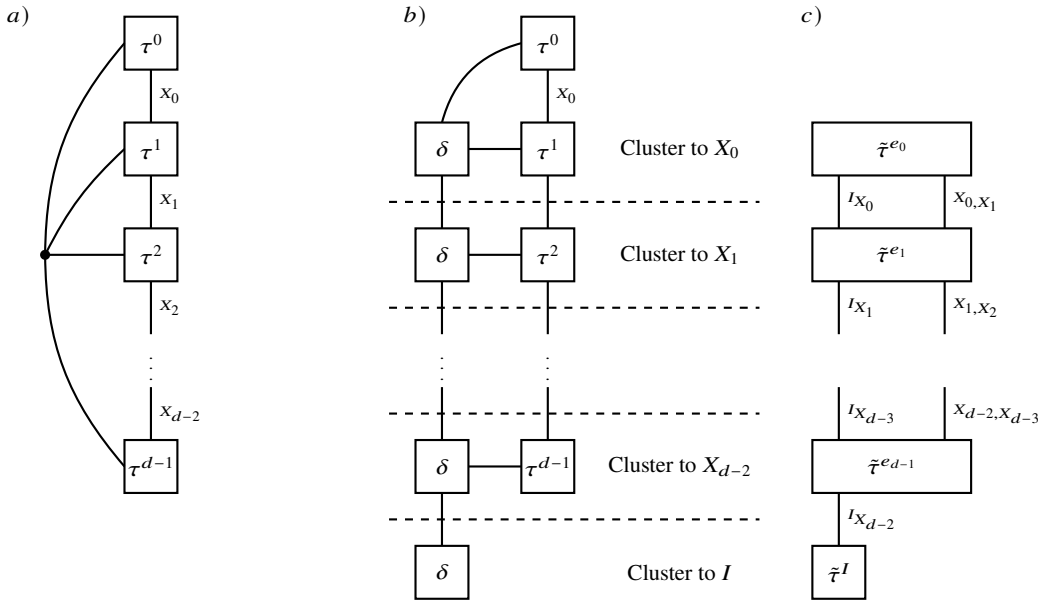



**Fig. 8.2** Variable Elimination on a tensor network in the order $(X_0, \ldots, X_{d-1}, I)$. a) Original contraction on a non-tree tensor network, b) Coarse grained Bethe hypergraph and c) Coarse grained contraction.

# Chapter 9
# Coordinate calculus

**Abstract** We review techniques to do computations by tensors. Coordinate calculus means that we encode information in the coordinates of the tensors. As a major example we investigate generic Fourier transforms.
Fourier transform as an example.

## 9.1 Group fields

A group field is a vector

$$\tau\,[X]\ :\ G \to \mathbb{C}$$

where $X$ enumerated the $|G| = m$ elements of the group. The $x$-th element of the group is then mapped to $\tau\,[X = x]$.

*Example 9.1* Unfolding is not an isomorphism Let us note that the unfolding of $[\prod_{k\in[d]} m_k]$ to $\bigtimes_{k\in[d]}[m_k]$ is not a group isomorphism. The group operation in $\prod_{k\in[d]} m_k$ is the integer summation

$$x \circ y = x + y \mod \left(\prod_{k\in[d]} m_k\right).$$

In the unfolding the group operation is

$$(x_0,\ldots,x_{d-1}) \circ (y_0,\ldots,y_{d-1}) = (x_0 + y_0 \mod m_0,\ldots,x_{d-1} + y_{d-1} \mod m_{d-1}).$$

Intuitively, the difference of these groups is that in the unfolded group summation carry bits are set to 0.

The group field perspective on tensors further applied in quantum gravity. Here symmetries are modeled based on the group structure [14, 13].

## 9.2 Fourier transform

### 9.2.1 Unfolding

The Fourier transform of vectors in $\mathbb{C}^m$ is represented by a matrix

$$F[X = x, Y = y] = \frac{1}{\sqrt{m}}\exp\left[2\pi i \cdot \frac{x \cdot y}{m}\right]$$

where $X, Y$ are variables of dimension $m$. The Fourier transform of a vector $V[X]$ is then the contraction

$$\langle V[X], F[X,Y]\rangle_{[Y]} \ .$$

We assume $m = 2^d$ and represent each index $x \in [2^d]$ by a tuple $(x_0, \ldots, x_{d-1}) \in [2]^d$ such that

$$x = \sum_{k\in[d]} x_k \cdot 2^{d-k-1} \ .$$

Note that in this convention, $x_{d-1}$ is the least significant bit. We follow [60] and denote $x$ by $x_0 \ldots x_{d-1}$. By $x_0 \ldots x_{k-1}.x_k \ldots x_{d-1}$ we denote $\frac{1}{2^{d-k-1}} \cdot x$.

The unfolded vector $V[X]$ is a tensor $V[X_{[d]}]$ with indices

$$V[X_{[d]} = x_{[d]}] = V[X = \left(\sum_{k\in[d]} x_k \cdot 2^{d-k-1}\right)] \ .$$

In the unfolded representation the fourier transform is described as the contraction with a tensor $F^d[X_{[d]}, Y_{[d]}]$ with coordinates

$$F^d[X_{[d]} = x_{[d]}, Y_{[d]} = y_{[d]}] = \frac{1}{\sqrt{2^d}} \exp\left[2\pi i \cdot \left(\sum_{k\in[d]} x_k \cdot 2^{d-k-1}\right) \cdot \left(\sum_{k\in[d]} y_k \cdot 2^{d-k-1}\right)\right] \ ,$$

that is the Fourier transform of $V[X_{[d]}]$ is

$$\left\langle V[X_{[d]}], F[X_{[d]}, Y_{[d]}]\right\rangle_{[Y_{[d]}]} \ .$$

### 9.2.2 Tensor network decomposition

We now derive a tensor network decomposition of the tensor $F^d[X_{[d]}, Y_{[d]}]$.

**Lemma 9.1** *The Fourier transform of basis vectors $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$ is the elementary tensor*

$$\frac{1}{\sqrt{2^d}} \bigotimes_{k\in[d]} \left(\epsilon_0\left[Y_k\right] + \exp\left[2\pi i \cdot (0.x_{d-k-1} \cdots x_{d-1})\right] \cdot \epsilon_1\left[Y_k\right]\right)$$

*Proof.* For each $y_{[d]}$ we have

$$\begin{aligned}
&\left\langle \{(\epsilon_0\left[Y_k\right] + \exp\left[2\pi i \cdot 0.x_{d-k-1} \cdots x_{d-1}\right] \cdot \epsilon_1\left[Y_k\right]) \ : \ k \in [d]\} \cup \{\epsilon_{y_{[d]}}\left[Y_{[d]}\right]\}\right\rangle_{[\varnothing]} \\
&= \prod_{k\in[d]} \exp\left[2\pi i \cdot (0.x_{d-k-1} \cdots x_{d-1}) \cdot y_k\right] \\
&= \prod_{k\in[d]} \exp\left[2\pi i \cdot (x_0 \cdots x_{d-k-2}.x_{d-k-1} \cdots x_{d-1}) \cdot y_k\right] \\
&= \exp\left[2\pi i \cdot x \sum_{k\in[d]} y_k \cdot 2^{d-k-1}\right] \\
&= \exp\left[2\pi i \cdot \frac{x \cdot y}{2^d}\right] \ .
\end{aligned}$$

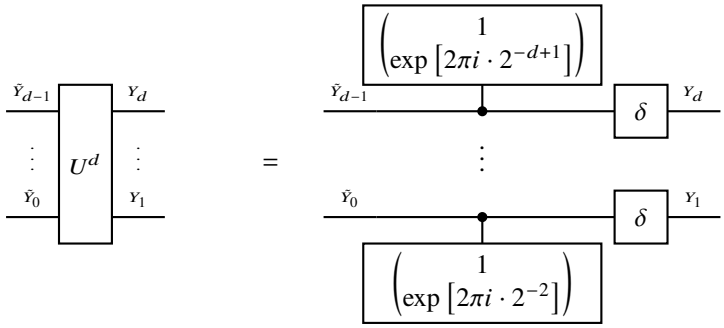
Since the basis coefficients of the elementary tensor coincide with those of the Fourier transform of $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$, both tensors are equal. □

We introduce the diagonal unitary

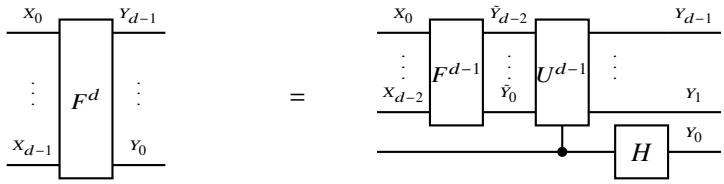
$$U^d[X_{[d]} = x_{[d]}, Y_{[d]} = y_{[d]}] = \begin{cases} \exp\left[2\pi i \cdot \left(\sum_{k\in[d]} x_k \cdot 2^{-k-2}\right)\right] & \text{if} \quad x_{[d]} = y_{[d]} \\ 0 & \text{else} \end{cases}$$

Note that $U^d$ is diagonal and the tensor product of matrices with variables $X_k, Y_k$. Diagonality can be used to contract $U^d$ to an arbitrary tensor with computational demand in $\mathcal{O}(2^d)$.

Using diagonality and elementarity, we have




**Lemma 9.2** *The Fourier transform on* $[2^d]$ *can be decomposed into a Fourier transform on* $[2^{(d-1)}]$ *as:*




*Proof.* It suffices to show that the Fourier transform of arbitrary basis tensors $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$ follows the decomposition. The Fourier transform of $\epsilon_{[d-1]}\left[X_{[d-1]}\right]$ is by the Lemma above

$$\frac{1}{\sqrt{2^{d-1}}} \bigotimes_{k\in[d-1]} \left(\epsilon_0\left[\tilde{Y}_k\right] + \exp\left[2\pi i \cdot (0.x_{d-k-2}\cdots x_{d-2})\right] \cdot \epsilon_1\left[\tilde{Y}_k\right]\right)$$

In case of $x_{d-1} = 0$, the Fourier transform of $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$ is the tensor product of this state with $\epsilon^+[X_0]$. This is reproduced by the decomposition, since the controlled unitary $U^{d-1}$ acts trivially and the Hadamard gate transforms $\epsilon_0[d-1]$ to $\epsilon^+[X_0]$.

In case of $x_{d-1} = 1$, the leg vectors $\left(\epsilon_0\left[\tilde{Y}_k\right] + \exp\left[2\pi i \cdot (0.x_{d-k-2}\cdots x_{d-2})\right] \cdot \epsilon_1\left[\tilde{Y}_k\right]\right)$ are further transformed into

$$\left(\epsilon_0[Y_{k+1}] + \exp\left[2\pi i \cdot (0.x_{d-k-2}\cdots x_{d-2}x_{d-1})\right] \cdot \epsilon_1[Y_{k+1}]\right) .$$

In both cases we arrive at the Fourier transform of $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$. □

By iteratively decomposing $F^{d-1}$ until we arrive at $F^1$ (which is the $H$ matrix), we get a decomposition of $F^d$ into $d$ $H$ matrices and controlled $U$ unitaries.

### 9.2.3 Fast Fourier Transform

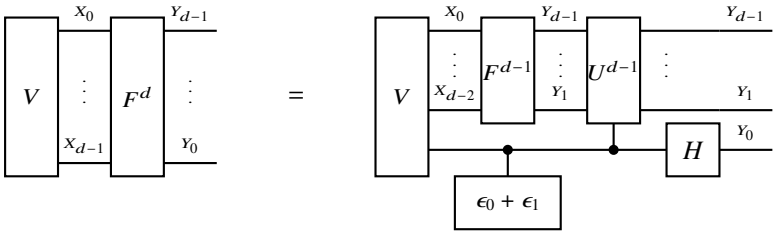
The FFT has been studied in the tensor formalism, see [24] and Chapter 14 in [36].

#### 9.2.3.1 Layerwise contraction

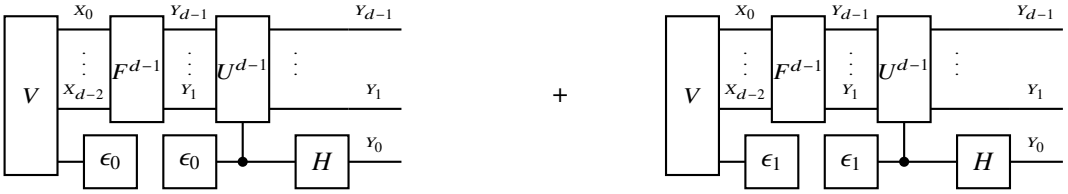
We contract the FFT tensor network layerwise. There are $2 \cdot d$ layers, each has a contraction demand in $\mathcal{O}(2^d)$ (exploiting diagonality of $U$). The algorithm has a total computational demand in $\mathcal{O}(d2^d)$.

#### 9.2.3.2 Divide and conquer

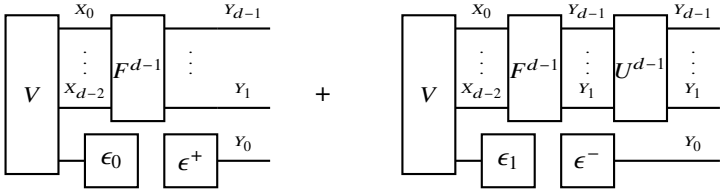
Called the radix-2 FFT [17]. Can be seen as an iterative simplification by adding a $\mathbb{I}[X_{d-1}]$ and splitting it into the basis vectors.




Now by linearity of contractions, this splits into two terms




We simplify them to




The contraction of $V$ with $\epsilon_1[X_{d-1}]$ respectively $\epsilon_1[X_{d-1}]$ are the even and the odd coordinates of $V$. Both are Fourier transformed with $d-1$ bits, which are further divided into Fourier transforms of lower order. In the conquer step of the FFT, the odd fourier transform gets a coordinatewise transform by $U$, which is efficient since $U$ is diagonal.

### 9.2.4 Generic groups

So far we have treated the Fourier transform on the cyclic group $[\prod_{k\in[d]} m_k]$ using unfoldings. More naturally, a tensor is defined as a complex field on the cartesian product group $\bigtimes_{k\in[d]}[m_k]$. Let us now generalize to generic groups.

#### 9.2.4.1 Representations

Recall that tensors are group fields, see Sect. 9.1.

A representation is a map of $G$ to a matrix group. Irreducible representations (irrep) $\rho$ are those, which are not equivalent to matrix groups in block form. We represent them by tensors

$$\rho[X, Y^{\text{in}}, Y^{\text{out}}]\,.$$

*Example 9.2* Cyclic group For the cyclic group $G = [m]$ the irreps are one-dimensional, that is $n = 1$, and enumerated by $\ell \in [m]$ as

$$\rho^{[m_\ell]}[X = x, L = \ell, Y^{\text{in}} = 0, Y^{\text{out}} = 0] = \exp\left[2\pi i \cdot \frac{x \cdot \ell}{m}\right]\,.$$

Note that for $m = 2$ this is the Hadamard matrix

$$\rho^{[2]}[X, L, Y^{\text{in}} = 0, Y^{\text{out}} = 0] = \sqrt{2} \cdot H[X, L] \ .$$

#### 9.2.4.2 Finite Abelian groups

Finite Abelian groups are equivalent to cartesian products of cyclic groups, that is

$$G \sim \bigtimes_{k \in [d]} [m_k] \, .$$

The irreps of finite Abelian groups are thus disconnected tensor networks

$$\rho^{\times_{k\in[d]}[m_k]}[X_{[d]}, L_{[d]}, Y_{\text{in}}^{[d]}, Y_{\text{out}}^{[d]}] = \bigotimes_{k\in[d]} \rho^{[m_k]}[X_k, L_k, Y_{\text{in}}^k, Y_{\text{out}}^k]$$

where each $\rho^{[m_k]}[X_k, L_k, Y_{\text{in}}^k, Y_{\text{out}}^k]$ is the irrep tensor to $[m_k]$, see Example 9.2.

#### 9.2.4.3 Fourier transform

The Fourier transform of a group field $\tau[X]$ maps each irrep $\rho$ to

$$\sqrt{\frac{n}{|G|}} \cdot \left\langle \tau[X], \rho[X, Y^{\text{in}}, Y^{\text{out}}] \right\rangle_{[Y^{\text{in}}, Y^{\text{out}}]} \ .$$

*Example 9.3 (Walsh-Hadamard transform)* Consider the group $[2]^d$, and a complex field on it by a quantum state $\psi\left[X_{[d]}\right]$. The Fourier transform is then the application of Hadamard gates on each qubit, that is

$$\left\langle \{\psi\left[X_{[d]}\right]\} \cup \bigcup_{k\in[d]} \{H[X_k, L_k]\} \right\rangle_{[L_{[d]}]} \ .$$

This is the Walsh-Hadamard transform of the quantum state.

# Chapter 10
# Sparse Calculus

**Abstract** We here investigate schemes to efficiently represent tensors based on adapted CP formats. Optimization problems of them are related to known formats like HUBO and QUBO. We in this chapter develop sparse tensor representation formats based on constrained CP formats. Our motivation for these formats result from the connection to encoding mechanisms, which we have applied in previous chapters.

Merge chapters 15 and 16 from the report.

## 10.1 CP Decomposition

The CP decomposition is one way to generalize the ranks of matrices to tensors. It is oriented on the Singular Value Decomposition of matrices, providing a representation of the matrix as a weighed sum of the tensor product of singular vectors. Given a matrix $M[X_0, X_1]$, we enumerate its singular values by $I$ taking values in $[n]$ and store them in a vector $\lambda[I]$. With the corresponding singular vectors by $\rho^0[X_0, I]$ and $\rho^1[X_1, I]$, the singular value decomposition of $M$ is

$$M[X_0, X_1] = \sum_{i \in [n]} \lambda[I = i] \cdot \rho^0[X_0, I = i] \otimes \rho^1[X_1, I = i] \ .$$

Here the smallest $n$ such that this decomposition exists, is the matrix rank $\operatorname{rank}(M)$. In contraction notation we abbreviate this to

$$M[X_0, X_1] = \left\langle \lambda[I], \rho^0[X_0, I], \rho^1[X_1, I] \right\rangle_{[X_0, X_1]} \ .$$

Given a tensor of higher order, a generalization of this decomposition is a tensor product over multiple vectors, as we define next.

**Definition 10.1** A CP decomposition of size $n$ of a tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k \in [d]} \mathbb{R}^{m_k}$ is a collections of a scalar core $\lambda[I]$ and leg cores $\rho^k[I, X_k]$ for $k \in [d]$, where $I$ is an enumeration variable taking values in $[n]$, such that

$$\tau\left[X_{[d]}\right] = \left\langle \{\lambda[I]\} \cup \{\rho^k[X_k, I] \ : \ k \in [d]\} \right\rangle_{\left[X_{[d]}\right]} \ .$$

We say that the CP Decomposition is

- directed, when for each $k$ the core $\rho^k$ is directed with $I$ incoming and $X_k$ outgoing.
- boolean, when for each $k$ the core $\rho^k$ is boolean.
- basis, where we demand both properties, that is for each $k \in [d]$ and $i \in [n]$

$$\rho^k[X_k, I = i] \in \{\epsilon_{x_k}[X_k] \ : \ x_k \in [m_k]\} \, .$$

- basis+, when for each $k \in [d]$ and $i \in [n]$

$$\rho^k[X_k, I = i] \in \{\epsilon_{x_k}[X_k] \ : \ x_k \in [m_k]\} \cup \{\mathbb{I}[X_k]\} \, .$$

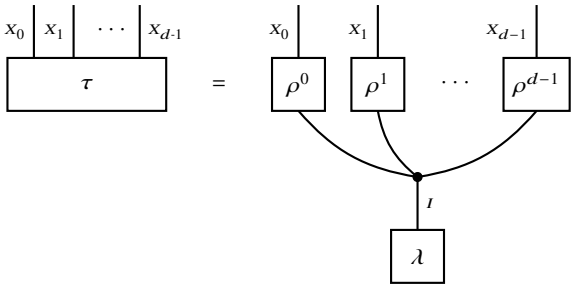



**Fig. 10.1** Tensor Network diagram of a generic CP decomposition (see Def. 10.1)

We denote by $\operatorname{rank}(\tau)$, respectively $\operatorname{rank}^{\text{bin}}(\tau)$, $\operatorname{rank}^{\text{bas}}(\tau)$ and $\operatorname{rank}^{\text{bas+}}(\tau)$ the minimal cardinality such that $\tau$ has a CP Decomposition, respectively with directed cores, boolean cores, basis cores and basis+ cores.

All ranks have a naive bound by the space dimension, which is obvious from the coordinate decomposition (see Chapter **??**)

$$\tau\left[X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \tau\left[X_{[d]} = x_{[d]}\right] \cdot \bigotimes_{k\in[d]} \epsilon_k\left[X_k\right] .$$

If we construct $i$ as an enumeration of the coordinates in $\bigtimes_{k\in[d]}[m_k]$, that is $n = \prod_{k\in[d]} m_k$, this is a CP decomposition, which is basis and therefore also directed, boolean and basis+.

CP decomposition as a tensor network format come with some drawbacks. The set of tensors with a fixed rank are not closed [6] and approximation problems are often ill posed [21]. Since as a consequence their numerical treatment comes with many problems [25], alternative formats have gained popularity. Common formats are the TUCKER-format originally introduced in [39], and often referred to as higher-order singular value decomposition, and the more recently developed TT and HT decomposition formats (see Chapter **??**). Given a HT the best approximation of a tensor always exists (Theorem 11.58 in [36]).

### 10.1.1 Directed Leg Cores

The contraint of directionality of the leg cores does not influence decomposablitiy of a tensor, as we show next.

**Lemma 10.1** *For any tensor $\tau\left[X_{[d]}\right]$ we have*

$$\operatorname{rank}(\tau) = \operatorname{rank}^{\text{dir}}(\tau) .$$

*Proof.* Let there be a CP decomposition of $\tau$ by

$$\tau\left[X_{[d]}\right] = \left\langle \{\lambda[I]\} \cup \{\rho^k\left[X_k, I\right] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}\right]} .$$

We then transform the scalar core to another core $\tilde{\lambda}[I]$ with coordinates to $i \in [n]$ by

$$\tilde{\lambda}[I = i] = \lambda[I = i] \cdot \prod_{k\in[d]} \left\langle \rho^k\left[X_k, I = i\right] \right\rangle_{[\varnothing]} .$$

It follows for any $i \in [n]$, that

$$\lambda[I = i] \cdot \bigotimes_{k\in[d]} \rho^k\left[X_k, I = i\right] = \tilde{\lambda}[I = i] \cdot \bigotimes_{k\in[d]} \left\langle \rho^k\left[X_k, I = i\right] \right\rangle_{[X_k|\varnothing]}$$

and thus

$$\tau\left[X_{[d]}\right] = \left\langle \{\tilde{\lambda}[I] \cup \{\left\langle \rho^k\left[X_k, I\right]\right\rangle_{[X_k, I|\varnothing]} \,:\, k \in [d]\}\right\rangle_{\left[X_{[d]}\right]} .$$

We have thus constructed a directed CP decomposition of same size $n$ to an arbitrary CP decomposition and conclude that $\mathrm{rank}\,(\tau) = \mathrm{rank}^{\mathrm{dir}}\,(\tau)$. □

### 10.1.2 Basis CP Decompositions and the $\ell_0$-norm

The slices of directed and boolean tensors with respect to incoming variables are basis tensors. We have thus called CP decomposition with the restiction of directed an boolean leg vectors basis CP decomposition. Based on this intuition, we can interpret basis CP decomposition by mappings to non-vanishing coordinates of the decomposed tensor. To start, let us define the number of nonzero coordinates of tensors by the $\ell_0$-norm.

**Definition 10.2** The $\ell_0$-norm counts the nonzero coordinates of a tensor by

$$\ell_0\,(\tau) = \#\left\{x_0, \ldots, x_{d-1} \,:\, \tau_{x_0,\ldots,x_{d-1}} \neq 0\right\} .$$

The $\ell_0$-norm is not a norm, but at each tensor the limit of $\ell_p$-norms (which are norms for $p \geq 1$) for $p \to 0$.

The $\ell_0$ norm is the number of non-vanishing coordinates of a tensor. We understand the leg cores as the basis encoding of functions mapping to the slices of these coordinates given an enumeration. This is consistent with the previous analysis of Chapter 7, where we characterized boolean and directed cores by the encoding of associated functions. Based on this idea, we can proof, that any tensor has a directed and boolean CP decomposition with rand $\ell_0\,(\tau)$.

**Theorem 10.1** *For any tensor $\tau\left[X_{[d]}\right]$ we have*

$$\mathrm{rank}^{\mathrm{bas}}\,(\tau) = \ell_0\,(\tau) .$$

*Proof.* Let us first show, that $\mathrm{rank}^{\mathrm{bas}}\,(\tau) \leq \ell_0\,(\tau)$. We find a map

$$D : [\ell_0\,(\tau)] \to \bigtimes_{k\in[d]} [m_k]$$

which image is the set of non-vanishing coordinates of $\tau\left[X_{[d]}\right]$. Denoting its image coordinate maps by $D_k$ we have

$$\tau\left[X_{[d]}\right] = \sum_{j\in[m]} \lambda[D(()j)]\left(\bigotimes_{k\in[d]} \epsilon_{D_k(j)}\left[X_k\right]\right).$$

This is a basis CP decomposition of size $\ell_0\,(\tau)$ and we thus have $\mathrm{rank}^{\mathrm{bas}}\,(\tau) \leq \ell_0\,(\tau)$.

Conversely, let us show $\mathrm{rank}^{\mathrm{bas}}\,(\tau) \geq \ell_0\,(\tau)$. Any basis CP decomposition of $\tau$ with size $r$ would has at most $r$ coordinates different from zero and thus $\ell_0\,(\tau) \leq r$. Thus, there cannot be a CP decomposition with a dimension $r \leq \ell_0\,(\tau)$. □

The next theorem relates the basis CP decomposition with encodings of $d$-ary relations (see Def. 7.4).

**Theorem 10.2** *Any boolean tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k}$ is the encoding of a $d$-ary relation $\mathcal{R} \subset \bigtimes_{k\in[d]} [m_k]$ with cardinality*

$$|\mathcal{R}| = \mathrm{rank}^{\mathrm{bas}}\,(\tau) .$$

*Proof.* We fine a basis CP decomposition of $\tau\left[X_{[d]}\right]$ with $n = \mathrm{rank}^{\mathrm{bas}}\,(\tau)$. Since $\tau\left[X_{[d]}\right]$ is boolean, and since each $i$ labels a disjoint non-vanishing coordinate (see proof of Thm. 10.1), the decomposition has a trivial scalar core $\lambda[I] = \mathbb{I}\,[I]$. It follows, that

$$\tau\left[X_{[d]}\right] = \sum_{i\in[n]}\left(\bigotimes_{k\in[d]} \rho^k\left[X_k, I = i\right]\right)$$

Since the CP decomposition is basis, the slice $\rho^k\left[X_k, I = i\right]$ is for any $k \in [d]$ and $i \in [n]$ a basis vector. We then define

$$x_k^i = \epsilon^{-1}(\rho^k\left[X_k, I = i\right])$$

and notice, that for the relation

$$\mathcal{R} = \{x_{[d]}^i \,:\, i \in [n]\} \subset \bigtimes_{k\in[d]} [m_k]$$

we have

$$\epsilon_{\mathcal{R}}\left[X_{[d]}\right] = \sum_{i\in[n]}\left(\bigotimes_{k\in[d]} \rho^k\left[X_k, I = i\right]\right).$$

This coincides with the above CP decomposition of $\tau\left[X_{[d]}\right]$ and the claim is established. □

*Remark 10.1 (Matrix Storage of basis* CP *decompositions)* The storage demand of any CP decomposition is at most linear in the size and the sum of its leg dimension. When we have a basis CP decomposition, this demand can be further improved. The basis vectors can be stored by its preimage of the one hot encoding $\epsilon_\cdot$, that is the number of the basis vector in $[m]$. This reduces the storage demand of each basis vector to the logarithms of the space dimension without the need of storing the full vector. More precisely, we can define a leg selecting variable $L$ taking values in $[d+1]$ and store a basis CP decomposition of size $n$ by the matrix

$$M[I, L] \in \mathbb{R}^{m\times(d+1)}$$

defined for $i \in [n]$ and $k \in [d]$ by

$$M[I = i, L = k] = \begin{cases} \lambda[I = i] & \text{if} \quad k = d \\ \epsilon^{-1}(\rho^k\left[X_k, I = i\right]) & \text{else} \end{cases}.$$

This is a common trick to store relational databases.

### 10.1.3 Basis+ CP Decompositions and Polynomials

The basis+ CP decompositions are closely related to monomial decompositions of a tensor, which we will define next.

**Definition 10.3** A monomial decomposition of a tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k}$ is a set $\mathcal{M}$ of tuples $(\lambda, A, x_A)$ where $\lambda \in \mathbb{R}$, $A \subset [d]$ and $x_A \in \bigtimes_{k\in A}[m_k]$ such that

$$\tau\left[X_{[d]}\right] = \sum_{(\lambda,A,x_A)\in\mathcal{M}} \lambda \cdot \left\langle \epsilon_{x_A}\left[X_A\right]\right\rangle_{\left[X_{[d]}\right]}. \tag{10.1}$$

For any tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k}$ we define its polynomial sparsity of order $r$ as

$$\operatorname{rank}^{\text{bas+},r}(\tau) = \min\left\{|\mathcal{M}| \,:\, \tau\left[X_{[d]}\right] = \sum_{(\lambda,A,x_A)\in\mathcal{M}} \lambda \cdot \left\langle \epsilon_{x_A}\left[X_A\right]\right\rangle_{\left[X_{[d]}\right]}, \forall_{(\lambda,A,x_A)\in\mathcal{M}} |A| \le r\,.\right\}$$

We refer to the terms in a decomposition (10.1) in Def. 10.3 as monomials of boolean features, which are enumerated by pairs $(k, x_k)$ and indicate whether the variable $X_k$ is in state $x_k \in [m_k]$. Each such boolean features is represented

by the indicator

$$\mathbb{I}_{X_k=x_k}\left[X_k\right] = \epsilon_{x_k}\left[X_k\right] .$$

The monomial of multiple such boolean features indicates, whether all variables labelled by a set $A$ are in the state $X_A$. We have

$$\mathbb{I}_{\forall k\in A:\, X_k=x_k}\left[X_A\right] = \epsilon_{x_A}\left[X_A\right] = \bigotimes_{k\in A} \epsilon_{x_k}\left[X_k\right] .$$

The states of the variables labeled by $k \in [d]/A$ are not specified in the monomial and the indicators are trivially extended to

$$\left\langle \epsilon_{x_A}\left[X_A\right]\right\rangle_{\left[X_{[d]}\right]} = \epsilon_{x_A}\left[X_A\right] \otimes \mathbb{I}\left[X_{[d]/A}\right] .$$

Since we are working with boolean features, there is no need to consider higher-order powers of individual features, since for any $n \in \mathbb{N}$, $n \geq 1$ and any boolean value $z \in \{0, 1\}$ we have $z^n = z$.

For some monomial orders $r < d$ there are tensors $\tau\left[X_{[d]}\right]$, which do not have a monomial decomposition of order $r$. In that case the minimum is over an empty set and we define $\mathrm{rank}^{\mathrm{bas+},r}\left(\tau\right) = \infty$. We characterize in the next theorem the set of tensors with monomial decompositions of order $r$.

**Theorem 10.3** *For any $d, r$, the set of tensors of $d$ variables with leg dimension $m$, which have a monomial decomposition of order $r$, is a linear subspace $V^{d,r}$ with dimension*

$$\dim(V^{d,r}) \leq \sum_{s\in[r]} m^s \binom{d}{s} .$$

*Proof.* The set of tensors admitting a monomial decomposition of order $r$ is closed under addition and scalar multiplication. Specifically, the sum of two such tensors retains a monomial decomposition, formed by concatenating their respective decompositions. Scalar multiplication can be performed by a rescaling of each scalar $\lambda$ and therefore preserves the decomposition structure. Hence, these tensors form a linear subspace.

To bound the dimension of this subspace, we consider tensors of the form $\left\langle \epsilon_{x_A}\right\rangle_{\left[X_{[d]}\right]}$. The number of such tensors is given by

$$\sum_{s\in[r]} m^s \binom{d}{s} .$$

Since any tensor with a monomial decomposition is a weighted sum of those, this provides an upper bound on the dimension.

We notice that the set of slices is in general not linear independent, and therefore forms a frame instead of a linear basis [11]. The number of elements in the frame is therefore in general a loose upper bound on the dimension. □

Thm. 10.3 states, that the tensors admitting a monomial decomposition of a small order build a low-dimensional subspace in the $m^d$ dimensional space of tensors, since for $r << d$ we have

$$\dim(V^{d,r}) << m^d .$$

If $r \geq d$, we always find a monomial decomposition by an enumeration of nonzero coordinates. In the next theorem, we show that in that case the $\mathrm{rank}^{\mathrm{bas+},r}\left(\tau\right)$ furthermore coincides with the basis+ CP rank $\mathrm{rank}^{\mathrm{bas+}}\left(\tau\right)$.

**Theorem 10.4** *For any tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k}$ we have*

$$\mathrm{rank}^{\mathrm{bas+},d}\left(\tau\right) = \mathrm{rank}^{\mathrm{bas+}}\left(\tau\right) .$$

*In case of two-dimensional legs (i.e. $m_k = 2$ for all $k \in [d]$) we also have*

$$\mathrm{rank}^{\mathrm{bin}}\left(\tau\right) = \mathrm{rank}^{\mathrm{bas+},d}\left(\tau\right) .$$

*Proof.* To prove the first claim, we construct a basis+ CP decomposition given a monomial decomposition and vice versa. To show $\text{rank}^{\text{bas+},d}(\tau) \geq \text{rank}^{\text{bas+}}(\tau)$, let there be an arbitrary tensor $\tau\left[X_{[d]}\right]$ with a monomial decomposition by $\mathcal{M}$ with $|\mathcal{M}| = m$ and let us enumerate the elements in $\mathcal{M}$ by $(\lambda^i, A^i, x^i_{A^i})$ for $i \in [n]$. We define for each $k \in [d]$ the tensors

$$\rho^k[I, X_k] = \left(\sum_{i\in[n]\,:\,k\in A} \epsilon_i[I] \otimes \epsilon_{x^i_k}[X_k]\right) + \left(\sum_{i\in[n]\,:\,k\notin A} \epsilon_i[I] \otimes \mathbb{I}[X_k]\right)$$

and

$$\lambda[I] = \sum_{i\in[n]} \lambda^i \cdot \epsilon_i[I]$$

and notice that

$$\begin{aligned}\tau\left[X_{[d]}\right] &= \sum_{i\in[n]} \lambda^i \cdot \left\langle \epsilon_{x^i_A} \right\rangle_{[X_{[d]}]} \\ &= \sum_{i\in[n]} \left(\lambda[I=i] \cdot \bigotimes_{k\in[d]} \rho^k[I=i, X_k]\right) \\ &= \left\langle \{\lambda[I]\} \cup \{\rho^k[I, X_k] \,:\, k \in [d]\}\right\rangle_{[X_{[d]}]} .\end{aligned}$$

By construction this is a basis+ CP decomposition with rank $n$. Since any monomial decomposition can be transformed into a basis+ CP decomposition with same rank we have

$$\text{rank}^{\text{bas+},d}(\tau) \geq \text{rank}^{\text{bas+}}(\tau) .$$

To show $\text{rank}^{\text{bas+},d}(\tau) \leq \text{rank}^{\text{bas+}}(\tau)$, let there now be a basis+ CP decomposition of an arbitrary $\tau\left[X_{[d]}\right]$. We define for each $i \in [n]$

$$A^i = \{k \in [d] : \rho^k[I=i, X_k] \neq \mathbb{I}[X_k]\} \quad \text{and} \quad x^i_A = \{\epsilon^{-1}(\rho^k[I=i, X_k]) \,:\, k \in A\}$$

where by $\epsilon^{-1}(\cdot)$ we denote the inverse of the one-hot encoding.

We notice that this is a monomial decomposition of $\tau\left[X_{[d]}\right]$ to the tuple set

$$\mathcal{M} = \{(\lambda[I=i], A^i, x^i_{A^i}) \,:\, i \in [n]\} .$$

It follows from this that

$$\text{rank}^{\text{bas+},d}(\tau) \leq \text{rank}^{\text{bas+}}(\tau)$$

and the first claim is shown.

The second claim follows from the observation, that the set of non-vanishing boolean vectors coincides with the set of one-hot encodings extended by the trivial vector. Thus, a CP decomposition with non-vanishing slices is boolean if and only if it is basis+. This establishes, that both ranks are equal, since a CP decomposition of minimal rank cannot contain non-vanishing slices. □

*Remark 10.2 (Sparse representation of propositional formulas)* When all leg dimensions of a boolean tensor $\tau$ are 2, we can further interpret $\tau$ as a logical formula. We can use the boolean CP decomposition of any tensor $\tilde{\tau}$ with $\mathbb{I}_{\neq 0}(\tilde{\tau}) = \tau$ as a CNF of $\tau$. Finding the sparsest CNF thus amounts to finding the $\tilde{\tau}$ with minimal $\text{rank}^{\text{bas+},d}(\tilde{\tau})$ such that $\mathbb{I}_{\neq 0}(\tilde{\tau}) = \tau$.

*Remark 10.3 (Matrix Storage of basis+ CP decompositions)* We can adapt the storage format of Remark 10.1 from basis to basis+ CP decompositions. To this end, let there be a basis+ CP decomposition of a tensor with scalar core $\lambda[I]$ and leg cores $\{\rho^k[X_k, I] \,:\, k \in [d]\}$. We use a value $z \in \mathbb{R}/\text{im}(\lambda)$ distinguished from the coordinates of the scalar core and define a matrix

$$M^z\,[I, L]$$

where $L$ takes values in $[d]$, coordinatewise as

$$M^z\,[I = i, L = k] = \begin{cases} \lambda[I = i] & \text{if} \quad k = d \\ z & \text{if} \quad \rho^k\,[X_k, I = i] = \mathbb{I}\,[X_k] \\ \epsilon^{-1}(\rho^k\,[X_k, I = i]) & \text{else} \end{cases} .$$

## 10.2 Constructive Bounds on CP Ranks

After having defined different CP decompositions, let us investigate bounds on their ranks, which proofs rely on explicit core constructions.

### 10.2.1 Cascade of Ranks

We start by showing a cascade of bounds of CP ranks, when demanding different leg restrictions as in Def. 10.1.

**Theorem 10.5** *For any tensor* $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k}$ *we have*

$$\operatorname{rank}(\tau) = \operatorname{rank}^{\mathrm{dir}}(\tau) \leq \operatorname{rank}^{\mathrm{bin}}(\tau) \leq \operatorname{rank}^{\mathrm{bas+}}(\tau) \leq \operatorname{rank}^{\mathrm{bas}}(\tau) .$$

*Proof.* The equality $\operatorname{rank}(\tau) = \operatorname{rank}^{\mathrm{dir}}(\tau)$ has been established in Lem. 10.1. The further inequalities follow by consecutive subset relations of the set of allowed leg slices in the respective CP decompositions. These imply, that any basis CP decomposition of a tensor $\tau\left[X_{[d]}\right]$ is also a basis+ CP decomposition, further that any basis+ CP decomposition is also a boolean CP decomposition and that any boolean CP decomposition is trivially an unrestricted CP decomposition. Thus, the ranks are minima of enlarging sets and the claimed rank cascade is established. □

Let us notice, that the stated bounds are not tight in general. To give an example, let us consider the tensor $\tau\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right]$, for which we have

$$\operatorname{rank}^{\mathrm{bas}}(\tau) = \ell_0(\tau) = \prod_{k\in[d]} m_k .$$

Since in the other restricted CP formats we can choose trivial slices to the leg cores, we have

$$\operatorname{rank}^{\mathrm{bas+}}(\tau) = 1 = \operatorname{rank}^{\mathrm{bin}}(\tau) = \operatorname{rank}^{\mathrm{dir}}(\tau) = \operatorname{rank}(\tau) .$$

The trivial tensor serves thus as an example, where the demand of the the storage format in Remark 10.1 has an exponential overhead compared to the storage format in Remark 10.3.

### 10.2.2 Operations on CP Decompositions

When using CP decompositons of tensors in practicle applications, such as those investigated in Part **??** and Part **??**, we have to perform numerical manipulations in the form of summations, contractions and normalizations of the represented tensors. Let us here investigate, how these operations influence the decomposition.

#### 10.2.2.1 Summation

We start with the sum of tensors in a CP decomposition, which can be captured by a concatenation of the slices.

**Theorem 10.6** *For any collections of tensors* $\{\tau^\ell\ [X_{\mathcal{V}}] : \ell \in [p]\}$ *with identical variables and scalars* $\lambda^\ell \in \mathbb{R}$ *for* $\ell \in [p]$ *we have*

$$\operatorname{rank}\left(\sum_{\ell\in[p]} \lambda^\ell \cdot \tau^\ell\right) \leq \sum_{\ell\in[p]} \operatorname{rank}\left(\tau^\ell\right) .$$

*The bound still holds, when we replace on both sides* rank $(\cdot)$ *by* $\operatorname{rank}^{\text{bin}}(\cdot)$*, by* $\operatorname{rank}^{\text{bas}}(\cdot)$ *or by* $\operatorname{rank}^{\text{bas+}}(\cdot)$.

*Proof.* Products with scalars do not change the rank, since they just rescale the core $\lambda$. The sum of CP decomposition is just the combination of all slices, thus the rank is at most additive. □

Let us notice, that the upper bound is loose in many applications. For example, if two slice tuples of two decomposed tensors agree on $x_A, A$, then their sum can be performed by a sum of the corresponding scalar.

#### 10.2.2.2 Contraction

We continue to show rank bounds for arbitrary contractions by the product of the ranks of contracted tensors.

**Theorem 10.7** *For any tensor network* $\tau^{\mathcal{G}}\ [X_{\mathcal{V}}]$ *on a graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$*, we have for any subset* $\mathcal{U} \subset \mathcal{V}$

$$\operatorname{rank}\left(\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]}\right) \leq \prod_{e\in\mathcal{E}\,:\,\mathcal{U}\cap e\neq\emptyset} \operatorname{rank}\left(\tau^e\right) .$$

*The bound still holds, when we replace on both sides* rank $(\cdot)$ *by* $\operatorname{rank}^{\text{bin}}(\cdot)$*, by* $\operatorname{rank}^{\text{bas}}(\cdot)$ *or by* $\operatorname{rank}^{\text{bas+}}(\cdot)$.

Remarkably, in Thm. 10.7 the upper bound on the CP rank is build only by the ranks of the tensor cores, which have remaining open edges. We prepare for its proof by first showing the following lemmata.

**Lemma 10.2** *For any tensors* $\tau^1\left[X_{\mathcal{V}^1}\right]$ *and* $\tau^2\left[X_{\mathcal{V}^2}\right]$ *and any set of variables* $\mathcal{U} \subset \mathcal{V}^1 \cup \mathcal{V}^2$ *we have*

$$\operatorname{rank}\left(\left\langle \tau^1, \tau^2 \right\rangle_{[X_{\mathcal{U}}]}\right) \leq \operatorname{rank}\left(\tau^1\right) \cdot \operatorname{rank}\left(\tau^2\right) .$$

*The bound still holds, when we replace on both sides* rank $(\cdot)$ *by* $\operatorname{rank}^{\text{bin}}(\cdot)$*, by* $\operatorname{rank}^{\text{bas}}(\cdot)$ *or by* $\operatorname{rank}^{\text{bas+}}(\cdot)$.

*Proof.* Let there be CP decompositions of $\tau^1\left[X_{\mathcal{V}^1}\right]$ and $\tau^2\left[X_{\mathcal{V}^2}\right]$ by

$$\tau^1\left[X_{\mathcal{V}^1}\right] = \left\langle \{\lambda^1[I_1]\} \cup \{\rho^{1,k}\ [X_k, I_1]\ :\ k \in \mathcal{V}^1\} \right\rangle_{[X_{\mathcal{V}^1}]}$$

and

$$\tau^2\left[X_{\mathcal{V}^1}\right] = \left\langle \{\lambda^2[I_2]\} \cup \{\rho^{2,l}\ [X_l, I_2]\ :\ l \in \mathcal{V}^2\} \right\rangle_{[X_{\mathcal{V}^2}]} .$$

By linearity of contractions we have

$$\begin{aligned}
\left\langle \tau^1, \tau^2 \right\rangle_{[X_{\mathcal{U}}]} &= \sum_{i_1\in n_1} \sum_{i_2\in n_2} \lambda^1[I_1 = i_1] \cdot \lambda^2[I_2 = i_2] \\
&\qquad \cdot \left\langle \{\rho^{1,k}\ [X_k, I_1 = i_1]\ :\ k \in \mathcal{V}^1\} \cup \{\rho^{2,l}\ [X_l, I_2 = i_2]\ :\ l \in \mathcal{V}^2\} \right\rangle_{[X_{\mathcal{U}}]} \\
&= \sum_{i_1\in n_1} \sum_{i_2\in n_2} \lambda^1[I_1 = i_1] \cdot \lambda^2[I_2 = i_2] \cdot \\
&\qquad \left\langle \{\rho^{1,k}\ [X_k, I_1 = i_1]\ :\ k \in \mathcal{V}^1/\mathcal{U}\} \cup \{\rho^{2,l}\ [X_l, I_2 = i_2]\ :\ l \in \mathcal{V}^2/\mathcal{U}\} \right\rangle_{[\emptyset]} \cdot \\
&\qquad \bigotimes_{k\in\mathcal{U}} \rho^k\ [X_k, I_1 = i_1, I_2 = i_2] \ ,
\end{aligned}$$

where we denote

$$\rho^k\left[X_k, I_1 = i_1, I_2 = i_2\right] = \begin{cases} \rho^{1,k}\left[X_k, I_1 = i_1\right] & \text{if} \quad k \notin \mathcal{V}^2 \\ \rho^{2,k}\left[X_k, I_2 = i_2\right] & \text{if} \quad k \notin \mathcal{V}^1 \\ \left\langle \rho^{1,k}\left[X_k, I_1 = i_1\right], \rho^{2,k}\left[X_k, I_2 = i_2\right]\right\rangle_{[X_k]} & \text{else} \end{cases} .$$

Note, that since $k \in \mathcal{U} \subset \mathcal{V}^1 \cup \mathcal{V}^2$, these slices are well-defined. We build a new decomposition variable $I$ enumerating the summands to indices $[n_1] \times [n_2]$ and have thus found a CP decomposition of $\left\langle \tau^1, \tau^2 \right\rangle_{[X_\mathcal{U}]}$ of size $n = n_1 \cdot n_2$. This shows the claim in the case of rank $(\cdot)$.

When the CP decompositions of $\tau^1$ and $\tau^2$ are boolean, basis or basis+, then the property is preserved in the constructed CP decomposition, since the constructed slices $\rho^k\left[X_k, I_1 = i_1, I_2 = i_2\right]$ are either copies of the leg cores or their contractions and the respective property is preserved in both cases. Thus, the constructive rank bounds hold also for $\operatorname{rank}^{\text{bin}}(\cdot)$, $\operatorname{rank}^{\text{bas}}(\cdot)$ and $\operatorname{rank}^{\text{bas+}}(\cdot)$. □

When one core of the contracted tensor network does not contain variables which are left open, we can drastically sharpen the bound provided by Lem. 10.2 as we show next.

**Lemma 10.3** *For any two tensors $\tau^1\left[X_{\mathcal{V}^1}\right]$, $\tau^2\left[X_{\mathcal{V}^2}\right]$ and any set $\mathcal{U}$ with $\mathcal{U} \cap \mathcal{V}^2 = \emptyset$ we have*

$$\operatorname{rank}\left(\left\langle \tau^1, \tau^2 \right\rangle_{[X_\mathcal{U}]}\right) \leq \operatorname{rank}\left(\tau^1\right) .$$

*The bound still holds, when we replace on both sides* $\operatorname{rank}(\cdot)$ *by* $\operatorname{rank}^{\text{bin}}(\cdot)$*,* $\operatorname{rank}^{\text{bas}}(\cdot)$ *or by* $\operatorname{rank}^{\text{bas+}}(\cdot)$*.*

*Proof.* As in the proof of Lem. 10.2 we assume a CP decomposition of $\tau^1\left[X_{\mathcal{V}^1}\right]$ and $\tau^2\left[X_{\mathcal{V}^2}\right]$ and use the linearity of contractions to get

$$\begin{aligned} \left\langle \tau^1, \tau^2 \right\rangle_{[X_\mathcal{U}]} = \sum_{i_1 \in n_1} \sum_{i_2 \in n_2} & \lambda^1[I_1 = i_1] \cdot \lambda^2[I_2 = i_2] \cdot \\ & \left\langle \{\rho^{1,k}\left[X_k, I_1 = i_1\right] \,:\, k \in \mathcal{V}^1/\mathcal{U}\} \cup \{\rho^{2,l}\left[X_l, I_2 = i_2\right] \,:\, l \in \mathcal{V}^2/\mathcal{U}\}\right\rangle_{[\emptyset]} \cdot \\ & \bigotimes_{k \in \mathcal{U}} \rho^{1,k}\left[X_k, I_1 = i_1\right] , \end{aligned}$$

where we used that $\mathcal{U} \cup \mathcal{V}^2 = \emptyset$. By rearranging the sum of $i_2$, we have a CP decomposition with decomposition variable $I_1$ and slices

$$\begin{aligned} \lambda[I_1 = i_1] = \sum_{i_2 \in n_2} & \lambda^1[I_1 = i_1] \cdot \lambda^2[I_2 = i_2] \cdot \\ & \left\langle \{\rho^{1,k}\left[X_k, I_1 = i_1\right] \,:\, k \in \mathcal{V}^1/\mathcal{U}\} \cup \{\rho^{2,l}\left[X_l, I_2 = i_2\right] \,:\, l \in \mathcal{V}^2/\mathcal{U}\}\right\rangle_{[\emptyset]} . \end{aligned}$$

This shows the rank bound for rank $(\cdot)$. The properties of the CP decomposition are trivially inherited by the constructed decomposition, since the leg cores of the decomposition of $\tau^1\left[X_{\mathcal{V}^1}\right]$ are chosen. Thus, the rank bounds hold also for any other rank in the claim. □

*Proof of Thm. 10.7.* We partition the edges into the set $\mathcal{E}^1 = \{e \in \mathcal{E} \,:\, e \cup \mathcal{U} \neq \emptyset\}$ and $\mathcal{E}^2 = \{e \in \mathcal{E} \,:\, e \cup \mathcal{U} = \emptyset\}$. We then have

$$\left\langle \tau^\mathcal{G} \right\rangle_{[X_\mathcal{U}]} = \left\langle \left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}^1\}\right\rangle_{\left[X_{\bigcup_{e \in \mathcal{E}^1} e}\right]}, \left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}^2\}\right\rangle_{\left[X_{\bigcup_{e \in \mathcal{E}^2} e}\right]} \right\rangle_{[X_\mathcal{U}]} \tag{10.2}$$

By an iterative application of Lem. 10.2 when including the cores to $e \in \mathcal{E}^1$ after each other to the contraction, we get the bound

$$\operatorname{rank}\left(\left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}^1\}\right\rangle_{\left[X_{\bigcup_{e \in \mathcal{E}^1} e}\right]}\right) \leq \prod_{e \in \mathcal{E}^1} \operatorname{rank}\left(\tau^e\right) .$$

With the decomposition (10.2) and Lem. 10.3 we then arrive at the claim

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]} \leq \prod_{e \in \mathcal{E}^1} \operatorname{rank}\left(\tau^e\right) .$$

Since the applied lemmata hold also for the restricted CP ranks in the claim, the derived bound is also for those valid. □

*Example 10.1 (Composition of formulas with connectives)* For any formula $f$ we have $1 - f = \neg f$. The CP rank bound brings an increase by at most factor 2 when taking the contraction with $\beta^{\neg}$ which has polynomial sparsity of 2. This is not optimal, since $\neg f$ has at most an absolute polynomial sparsity increase of 1.

For any formulas $f$ and $h$ we have $f \cdot h = f \wedge h$. Here the CP rank bounds on contractions can also be further tightened.

*Example 10.2 (Distributions of independent variables)* Independence means factorization, conditional independence means sum over factorizations. Again, the $\ell_0$ norm is bounded by the product of the $\ell_0$ norm of the factors.

## 10.3 Sparse Encoding of Functions

We now state that the basis CP rank of basis encodings is equal to the cardinality of the domain. The basis CP format can therefore not provide a sparse representation when the factored system contains many categorical variables.

**Theorem 10.8** *For any function*

$$q : \bigtimes_{k \in [d]} [m_k] \to \bigtimes_{l \in [r]} [m_l]$$

*between factored systems we have*

$$\operatorname{rank}^{\mathrm{bas}}\left(\beta^q\right) = \prod_{k \in [d]} m_k .$$

*Proof.* The bound follows from Thm. 10.1, using that $\ell_0\left(\beta^q\right) = \prod_{k \in [d]} m_k$. □

Let us further provide a construction scheme to find a basis CP decomposition of $\beta^q$ of size $\prod_{k \in [d]} m_k$. We notice that

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{q(X_{[d]} = x_{[d]})}\left[Y_{[p]}\right] .$$

We build for $k \in [d]$ decomposition variables $I_{[d]}$ with $n_k = m_k$ and define leg cores

$$\rho^k\left[X_k, I_{[d]}\right] = \delta\left[X_k, I_k\right]$$

and for $\tilde{k} \in [r]$ and $i_{[d]}$

$$\rho^{\tilde{k}}\left[Y_{\tilde{k}}, I_{[d]} = i_{[d]}\right] = \epsilon_{q_{\tilde{k}}[X_{[d]} = i_{[d]}]} Y_{\tilde{k}} .$$

We then have with a trivial scalar core

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \left\langle \{\mathbb{I}\left[I_{[d]}\right]\} \cup \{\rho^k\left[X_k, I_{[d]}\right] \,:\, k \in [d]\} \cup \{\rho^{\tilde{k}}\left[Y_{\tilde{k}}, I_{[d]}\right] \,:\, \tilde{k} \in [r]\} \right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} .$$

This is a basis CP decomposition of size $\prod_{k \in [d]} m_k$.

In combination with Thm. 10.5, Thm. 10.8 also provides bounds on all other CP ranks defined in Def. 10.1. This is obvious, since basis leg slices are the most restrictive properties compared with boolean, directed or basis+.

We restate Thm. **??** as a basis CP decomposition bound.

**Theorem 10.9** *Let $q$ and be a function between factored systems*

$$q : [m] \to \bigtimes_{k\in[d]} [m_k]$$

*and $q_k$ as in Thm.* **??**. *Then $\beta^q\left[X_{[d]}, X\right]$ has a basis* CP *decomposition with decomposition index $X$, trivial slices $\mathbb{I}[X]$ leg vectors $\beta^{q_k}[X_k, X]$, that is*

$$\beta^q\left[X, X_{[d]}\right] = \langle \{\mathbb{I}[X]\} \cup \{\beta^{q_k}[X_k, X] \,:\, k \in [d]\}\rangle_{[X_{[d]}]}$$

*Proof.* The claimed decomposition directly follows from Thm. **??**, since the trivial scalar core $\lambda[X] = \mathbb{I}[X]$ does not influence the contraction and can be omitted. □

Basis CP decompositions can be constructed by understanding the variable $O_{\text{in}}$ of the basis encoding of a function $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ as the slice selection variable.

*Example 10.3 (Empirical distributions, see Thm. 5.16)* Let there be a data map

$$D : [m] \to \bigtimes_{k\in[d]} [m_k]\,.$$

We can use Thm. 10.9 to find a tensor network representation fo $\beta^D$ as

$$\beta^D\left[X_{[d]}, J\right] = \left\langle \{\beta^{D_k}[X_k, J] : k \in [d]\}\right\rangle_{[X_{[d]}, J]}\,.$$

This representation is a basis CP decomposition, when adding trivial scalar core. This provides also a basis CP decomposition for the empirical distribution, since normalization can be done by setting a slice core to $\frac{1}{m}\mathbb{I}[J]$.

*Example 10.4* Exponential families The statistic has a CP decomposition with rank by the cardinality of states, that is

$$\beta^t\left[Y_{[p]}, X_{[d]}\right] = \left\langle \{\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_{[p]}, X_{[d]}]}\,.$$

While Thm. 10.8 and Thm. 10.9 provide CP rank bounds based on the domain factored system, we can also show in the next theorem a bound using the structure of the image.

**Theorem 10.10** *Let $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ be an arbitrary function and let us consider for each $y \in \text{im}(q)$ the indicator*

$$\mathbb{I}^{q=y}[O_{\text{in}}] = \begin{cases} 1 & \text{if} \quad q(O_{\text{in}} = o_{\text{in}}) = y \\ 0 & \text{else}\,. \end{cases}$$

*The basis+ rank of the basis encoding of $q$ then obeys the bound*

$$\text{rank}\,(\beta^q) \leq \sum_{y\in\text{im}(q)} \text{rank}\,(\mathbb{I}^{q=y})\,.$$

*The bound still holds, when we replace on both sides* $\text{rank}\,(\cdot)$ *by* $\text{rank}^{\text{bin}}(\cdot)$, *by* $\text{rank}^{\text{bas}}(\cdot)$ *or by* $\text{rank}^{\text{bas+}}(\cdot)$.

*Proof.* We have

$$\beta^q[O_{\text{out}}, O_{\text{in}}] = \sum_{y\in\text{im}(q)} \epsilon_{I[y]}[O_{\text{out}}] \otimes \mathbb{I}^{q=y}[O_{\text{in}}]\,.$$

For any $y \in \text{im}(q)$ it is obvious that

$$\text{rank}\left(\epsilon_{I[y]}[O_{\text{out}}] \otimes \mathbb{I}^{q=y}[O_{\text{in}}]\right) = \text{rank}\left(\mathbb{I}^{q=y}[O_{\text{in}}]\right)\,,$$

which also holds true for the other bounds in the claim. We then use the summation bound of Thm. 10.6 to get

$$\begin{aligned}\operatorname{rank}\left(\beta^{q}\left[O_{\text{out}}, O_{\text{in}}\right]\right) &\leq \sum_{y \in \operatorname{im}(q)} \operatorname{rank}\left(\epsilon_{I[y]}\left[O_{\text{out}}\right] \otimes \mathbb{I}^{q=y}\left[O_{\text{in}}\right]\right) \\ &\leq \sum_{y \in \operatorname{im}(q)} \operatorname{rank}\left(\mathbb{I}^{q=y}\left[O_{\text{in}}\right]\right) .\end{aligned}$$

Again, the bound still hold for the other ranks in the claim. □

The above claim still holds when replacing $\operatorname{rank}^{\text{bas+}}(\cdot)$ with the ranks $\operatorname{rank}^{\text{bas}}(\cdot)$ or $\operatorname{rank}^{\text{bin}}(\cdot)$. For the rank $\operatorname{rank}^{\text{bas}}(\cdot)$ it leads to the bound of Thm. 10.8, since summing the number of non zero coordinators of the indicators is the cardinality of the domain.

*Example 10.5 (Propositional formulas)* Let us now illustrate how the above representation scheme can be leveraged for the sparse representation of propositional formulas. For an arbitrary propositional formula $f$ we have $\operatorname{im}(f) \subset \{0, 1\}$ and the indicators

$$\mathbb{I}^{f=1}\left[X_{[d]}\right] = f\left[X_{[d]}\right] \quad \text{and} \quad \mathbb{I}^{f=0}\left[X_{[d]}\right] = \neg f\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right] - f\left[X_{[d]}\right] .$$

For the conjunction $\wedge[X_0, X_1] = X_0 \wedge X_1$ we have

$$\beta^{\wedge}\left[X_0, X_1\right] = \epsilon_1\left[Y_{\wedge}\right] \otimes \epsilon_{1,1}\left[X_0, X_1\right] + \epsilon_0\left[Y_{\wedge}\right] \otimes \left(\mathbb{I}\left[X_0, X_1\right] - \epsilon_{1,1}\left[X_0, X_1\right]\right)$$

and thus

$$\operatorname{rank}^{\text{bas+}}\left(\beta^{\wedge}\right) \leq 3$$

while $\operatorname{rank}^{\text{bas}}(\beta^{\wedge}) = 4$.

We can even generalize this observation to $d$-ary conjunctions $\wedge\left[X_{[d]}\right] = X_0 \wedge \ldots \wedge X_{d-1}$ (see Remark 2.1)

$$\wedge[X_{[d]}] = \bigotimes_{k \in [d]} \epsilon_1\left[X_k\right] \quad \text{and} \quad \neg\wedge[X_{[d]}] = \mathbb{I}\left[X_{[d]}\right] - \bigotimes_{k \in [d]} \epsilon_1\left[X_k\right]$$

and thus

$$\beta^{\wedge}\left[X_{[d]}\right] = \epsilon_1\left[Y_{\wedge}\right] \otimes \left(\bigotimes_{k \in [d]} \epsilon_1\left[X_k\right]\right) + \epsilon_0\left[Y_{\wedge}\right] \otimes \left(\bigotimes_{k \in [d]} \epsilon_1\left[X_k\right]\right)$$

Thus, while the basis CP rank is $\operatorname{rank}^{\text{bas}}(\beta^{\wedge}) = 2^d$, the basis+ rank is bounded by 3, independently of $d$.

## 10.4 Reed-Muller expansions

So far we have investigated sparse representations based on CP decompositions with respect to the default sum-product semiring. The Reed-Muller expansion of Boolean functions [54, 68] can be regarded as a CP decomposition with respect to the Galois field $\mathbb{F}_2$ (see Sect. 8.1.2.5).

Since the multiplication operation coincided in the semiring $\mathbb{F}_2$ with the standard sum-product semiring, also the elementary tensor decomposition coincides in both semirings. As a consequence elementary tensors correspond also with respect to $\mathbb{F}_2$ to terms (see Sect. 2.3.6), which are parametrized by a subset $\mathcal{U} \subset [d]$ of affected atoms and the polarity index $y_{\mathcal{U}}$ of the literals. Each term is a tensor

$$f_{(\mathcal{U}, y_{\mathcal{U}})}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if} \quad \forall k \in \mathcal{U} : x_k = y_k \\ 0 & \text{else} \end{cases}$$

and has therefore an elementary decomposition by

$$f_{(\mathcal{U},y_\mathcal{U})}\left[X_{[d]}=x_{[d]}\right]=\langle\{\epsilon_{y_k}\left[X_k\right]\,:\,k\in\mathcal{U}\}\cup\{\mathbb{I}\left[X_k\right]\,:\,k\notin\mathcal{U}\}\rangle^{\mathbb{F}_2}_{[X_{[d]}]}\,.$$

Given a mixed-polarity Reed-Muller expansion

$$f\left[X_{[d]}=x_{[d]}\right]=\bigoplus_{i\in[n]}f_{(\mathcal{U}^i,y^i_{\mathcal{U}^i})}\left[X_{[d]}=x_{[d]}\right]$$

we build leg cores for $k\in[d]$ with coordinates for $i\in[n]$ by

$$\rho^k\left[X_k,I=i\right]=\begin{cases}\mathbb{I}\left[X_k\right] & \text{if}\quad k\notin\mathcal{U}^i\\ \epsilon_{y^i_k}\left[X_k\right] & \text{else}\,.\end{cases}$$

The three possible leg vectors $\mathbb{I}\left[X_k\right]$, $\epsilon_0\left[X_k\right]$ and $\epsilon_1\left[X_k\right]$ correspond thus an absent literal, $\neg X_k$ and $X_k$. Using these cores we have the CP decomposition of $f\left[X_{[d]}\right]$ in the $\mathbb{F}_2$ semiring by

$$f\left[X_{[d]}\right]=\langle\{\rho^k\left[X_k,I\right]\,:\,k\in[d]\}\rangle^{\mathbb{F}_2}_{[X_{[d]}]}\,.$$

The sum over the states $i$ of the hidden variable $I$ is the $\mod 2$ summation over terms.

Variants are:

- Mixed polarity reed-muller expansion: General CP decomposition
- Algebraic normal form: Allowing only $\mathbb{I}\left[X\right]$ and $\epsilon_1\left[X\right]$ for the leg vectors (also called positive polarity)

*Example 10.6 (Reed-Muller expansion for the implication)* The propositional formula $f\left[X_0,X_1\right]=X_0\longrightarrow X_1$ has the mixed polarity expansion

$$1\oplus(X_0\wedge\neg X_1)\,,$$

where we denote by 1 the formula which is always true. Which corresponds with the CP decomposition of rank 2 and leg cores

$$\rho^0\left[X_0,I\right]=\begin{pmatrix}1&0\\1&1\end{pmatrix}\left[X_0,I\right]\quad,\quad\rho^1\left[X_1,I\right]=\begin{pmatrix}1&1\\1&0\end{pmatrix}\left[X_0,I\right]\,.$$

When allowing only for positive polarity (i.e. building the algebraic normal form), we further decompose to

$$\begin{aligned}1\oplus(X_0\wedge\neg X_1)&=1\oplus\big(X_0\wedge(1\oplus X_1)\big)\\&=1\oplus X_0\oplus(X_0\wedge X_1)\,.\end{aligned}$$

The positive polarity expansion is thus the CP decomposition of rank 3 and leg cores

$$\rho^0\left[X_0,I\right]=\begin{pmatrix}1&0&0\\1&1&1\end{pmatrix}\left[X_0,I\right]\quad,\quad\rho^1\left[X_1,I\right]=\begin{pmatrix}1&1&0\\1&1&1\end{pmatrix}\left[X_0,I\right]\,.$$

### 10.4.1 Rank bound

Using that basis CP decompositions are a special case of basis+ CP decompositions, we get the following rank bound.

**Lemma 10.4** *The* $\mathrm{CP}^{\mathbb{F}_2}$ *rank is bounded by the basis* CP *rank.*

*Proof.* Use $A=[d]$, and $x_A$ to each supported state. Then the mod2-sum is a usual sum and the basis CP decomposition is also a mod2-basis+ CP decomposition. □

# Chapter 11
# Languages and Automata

**Abstract** We here present tensor network representation of standard models of computation. In particular, we derive TT representations of finite automata and represent generic logical circuits. Include in basis calculus chapter.

## 11.1 Languages

Languages over finite alphabets $\Sigma$ are collection of strings in $\Sigma^*$. The symbols in $\Sigma$ are enumerated by an index interpretation map $I$.

Collections $\mathcal{U}$ of strings of length $n$ are represented by a boolean tensor

$$\tau^{\mathcal{U},n}\left[O_{[n]}\right] = \sum_{v\in\mathcal{U}} \bigotimes_{l\in[n]} \epsilon_{I[v|_l]}\left[O_l\right] \ .$$

A language $L$ is encoded by the infinite collection of tensors

$$\{\tau^{L,n}\left[O_{[n]}\right] \ : \ n\in\mathbb{N}\}\,,$$

each representing the set of strings of length $n$.

We want to study how to decompose such tensors and relate tensor network schemes to standard models of computation.

## 11.2 Deterministic Finite Automata

Literature:

- [20] Basic Representation by TT/MPS.
- [5] Usage of canonical formats (orthogonalization) to find quantum circuit representations.

Idea:

- Construct chain graphs decorated with basis encodings of state transition functions, as in [20, 5]
- Contract with basis encoding of halting states and get the boolean tensor indicating the accepted states of the automaton

**Definition 11.1 (Deterministic finite automaton (DFA), see Def 2 in [5])** A tuple of

- set of internal states $Q$
- alphabet $\Sigma$
- initial state $r_0 \in Q$
- accepted states $F \subset Q$

- transition function $\delta : Q \otimes \Sigma \to Q \cup \{\varnothing\}$

is called a deterministic finite automaton (DFA).

We enumerate the set $Q$ by an map $I_Q$ with enumeration variable $O_Q$, and the alphabet $\Sigma$ by a map $I_\Sigma$ with an enumeration variable $O_\Sigma$. Both indicator variables will be copied for $k \in [d]$ (respectively $k \in [d+1]$) and the copies denoted by $O_{\Sigma,k}$ and $O_{Q,k}$. The acceptance tensor of order $d \in \mathbb{N}$ is the boolean tensor $\tau\left[O_{\Sigma,[d]}\right]$ indicating whether the automaton accepts the input sequence of length at most $d$. Its coordinates are

$$\tau\left[O_{\Sigma,[d]} = o_{\Sigma,[d]}\right] = \begin{cases} 1 & \text{if} \quad \text{The automaton accepts the input sequence } \left(I_\Sigma\left[O_k = o_k\right] \,:\, k \in [d]\right) \\ 0 & \text{else} \end{cases}.$$

**Lemma 11.1 (Tensor representation)** *Let $(Q, \Sigma, \delta, r_0, F)$ be a deterministic finite automaton. Then the acceptance tensor of order $d \in \mathbb{N}$ is the contraction*

$$\tau\left[O_{\Sigma,[d]}\right] = \left\langle \left\{\beta^\delta\left[O_{Q,k}, O_{\Sigma,k}, O_{\Sigma,k}\right] \,:\, k \in [d]\right\} \cup \left\{\epsilon_{I_Q^{-1}(r_0)}\left[O_{Q,0}\right], \epsilon_{I_Q^{-1}(F)}\left[O_{Q,d}\right]\right\}\right\rangle_{\left[O_{\Sigma,[d]}\right]}.$$

*Proof.* By basis calculus: Given the one-hot encoding of the input word and contracting with the tensor network (expect the basis encoding tensor of $F$) leaving $O_{Q,d}$ open, we get the one hot encoding of the finial state of the automaton. Contracting with the one-hot encoding of $F$ is then 1, if and only if the final state is in the accepted states. □

Note that the acceptance tensor of the complement DFA is prepared when replacing the basis encoding $\epsilon_{I_Q^{-1}(F)}\left[O_{Q,d}\right]$ by the basis encoding $\epsilon_{I_Q^{-1}(\bar{F})}\left[O_{Q,d}\right]$ of the complement of the accepting states.

### 11.2.1 Outlook

We can use this theory to:

- Criterion that a language is not regular: Unbounded matrification ranks in the word tensors. For example, this shows that $\{0^n 1^n \,:\, n \in \mathbb{N}\}$ is not regular.
- Proof that regular languages are closed under regular operations: Use parallel TT cores, modifications in the transition functions.

## 11.3 Nondeterministic Finite Automata

Instead of transition function we have in NFAs a transition relation. Instead of directed boolean TT cores, we thus modify to generic boolean TT cores.

Usage:

- Proof that any NFA is a DFA: By using basis encodings of the cores understood as linear maps.

### 11.3.1 Stochastic Finite Automata

Ideas:

- The transition function is replaced by a cpd, which is encoded by a more general directed tensor and copied to all cores of a TT.
- The result is a Markov Chain, i.e. a Bayesian Network on a chain graph.
- The contraction above then gives the probability of acceptance for a given input word.
- One can add a distribution over the input words, and contract to get a joint distribution of input words with an acceptance variable.

# Chapter 12
# Quantum circuits

**Abstract** Quantum mechanics is formulated in tensor spaces and quantum circuits are naturally tensor networks. Quantum Circuits are naturally tensor networks. This motivates the investigation, which contractions can be efficiently performed on a Quantum Computer.

## 12.1 Foundations

By its central axioms, quantum mechanics of multiple qubits is formulated by tensors capturing states and discrete time evolutions. Motivated by the structural similarity, we investigate how quantum circuits can be utilized for the tensor-network based approach towards efficient and explainable AI in the tnreason framework.

Quantum circuits are contractions of gate tensors and therefore tensor networks, and measurement probabilities are given by contractions.

### 12.1.1 Quantum and Tensor Parallelism

Potential Advantage: *Quantum Parallelism* (see [78, Section 3.2.5]).

- Evaluation of multiple function values by single circuit evaluation, we will relate it with the contraction of $\beta^f$ here.
- Contraction perspective: Loop-tolerant efficient contractions.
- However: Need a generic encoding scheme to exploit this advantage, which is not known yet.
- We here only provide a scheme based on post-selection, which provides a quantum advantage only through amplitude amplification. Without amplitude amplification and post-selection of samples, the encoded distribution is always uniform.
- Further challenge: Application in NISQ devises [67], instead of fault-tolerant quantum computers.

Comparison with classical side, which we can call *Tensor Network Parallelism*: Message-passing schemes for efficient contractions, but exact in limited cases.

We present the basic notation of quantum circuits using the tensor network notation used in the tnreason formalism.

### 12.1.2 Quantum states and quantum circuits

Quantum states are complex tensors

$$\psi\left[X_{[d]}\right] : \bigtimes_{k\in[d]} [2] \to \mathbb{C}$$

such that

$$\left\langle (\psi\left[X_{[d]}\right])^*, \psi\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[2]} (\psi\left[X_{[d]} = x_{[d]}\right])^* \psi\left[X_{[d]} = x_{[d]}\right] = 1\,.$$

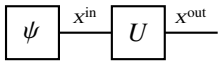
Quantum gates are unitary tensors

$$U\left[X^{\text{in}}, X^{\text{out}}\right] : \bigtimes_{k\in[d]}[2] \times \bigtimes_{k\in[d]}[2] \to \mathbb{C}$$

that is

$$\left\langle (U\left[X^{\text{mid}}, X^{\text{out}}\right])^\dagger, U\left[X^{\text{mid}}, X^{\text{in}}\right]\right\rangle_{[=]} \delta\left[X^{\text{in}}, X^{\text{out}}\right]\,.$$

Application of quantum gates on quantum states is their contraction with respect to the complex sum-product semiring (see Sect. 8.1.2.1). A quantum state $\psi\left[X^{\text{in}}\right]$ transforms under the gate $U\left[X^{\text{in}}, X^{\text{out}}\right]$ to the state




.

### 12.1.3 Controlled single qubit gates

We define the rotation gate around the Y-axis by an angle $\alpha$ as

$$R_Y(\alpha)\left[A^{\text{in}}, A^{\text{out}}\right] := \begin{pmatrix} \cos\left(\frac{\alpha}{2}\right) & -\sin\left(\frac{\alpha}{2}\right) \\ \sin\left(\frac{\alpha}{2}\right) & \cos\left(\frac{\alpha}{2}\right) \end{pmatrix}\left[A^{\text{in}}, A^{\text{out}}\right]\,.$$

Further we define the Pauli-X:

$$\sigma_1\left[A^{\text{in}}, A^{\text{out}}\right] := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\left[A^{\text{in}}, A^{\text{out}}\right]\,.$$

Controlled single qubit gates are defined using control qubits, where the gate is applied to the target qubit if the control qubits are in a specific state and the identity is applied otherwise. In the tensor network diagrams, we do not distinguish between incoming and outgoing control qubit variables, since the control acts as a Dirac tensor. Thus, controlled unitary with target qubit $X_t$ and control qubits $X_c$ are represented by tensors

$$\mathcal{U}\left[X^t_{\text{in}}, X^t_{\text{out}}, X_c\right]$$

where for each state $x_c$ to the control variables we have that

$$\mathcal{U}\left[X^t_{\text{in}}, X^t_{\text{out}}, X_c = x_c\right]$$

is a unitary matrix acting on the leg space of the target variable.

### 12.1.4 Measurements and Contractions

The main difficulty of using quantum circuits as contraction providers is that we can only extract information through measurements. Therefore measurement is the only way to execute contractions of the circuit, which come with restrictions when interested in contraction with open variables.

#### 12.1.4.1 Computational basis measurement

The computational basis measurement of the qubits $X_A$ of a Quantum State $\psi\left[X_{[d]}\right]$ is equal to drawing samples from a distribution

$$\mathbb{P}\left[X_A\right] = \left\langle \psi\left[X_{[d]}\right], \psi^*\left[X_{[d]}\right]\right\rangle_{[X_A]} .$$

Here $\psi^*\left[X_{[d]}\right]$ is the complex conjugate of $\psi\left[X_{[d]}\right]$. When $\psi$ is prepared by a quantum circuit acting on a initial state, the complex conjugate is the hermitean conjugate of the circuit acting on the complex conjgate of the initial state.

We abbreviate these contractions by extending the contraction diagrams with measurement symbols (see Figure 12.1).

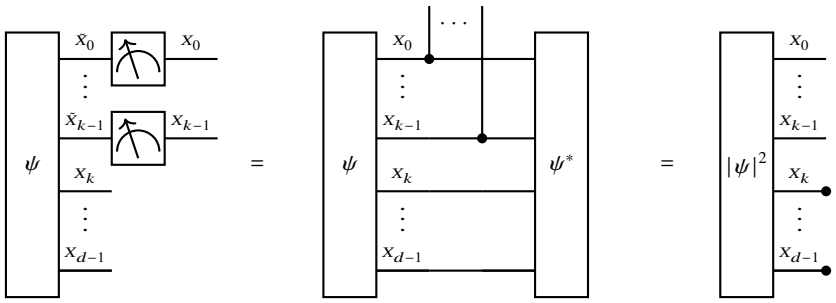



**Fig. 12.1** Computational Basis Measurement of a quantum state $\psi$. The measurement symbols on the left side indicate the measured qubits and the first equation is understood as a definition. In the second equation we sketch, that the measurement distribution is equal to the contraction of the square absolute transform of $\psi$ to the measured variables.

Each complex-valued tensor $\psi\left[X_{[d]}\right]$ has a decomposition into a phase tensor $\phi\left[X_{[d]}\right]$ and an absolute tensor $|\psi|\left[X_{[d]}\right]$ defined by

$$\psi\left[X_{[d]}\right] = \left\langle \exp\left[i \cdot \phi\left[X_{[d]}\right]\right], |\psi\left[X_{[d]}\right]|\right\rangle_{\left[X_{[d]}\right]} .$$

The measurement distribution depends only on $|\phi|$, that is

$$\mathbb{P}\left[X_{[d]}\right] = \left|\psi\left[X_{[d]}\right]\right|^2 .$$

Note, that this measurement distribution is not sensitive to the phases of the amplitudes.

When only a subset of variables is measured, the distribution is the contraction of the absolute square transform (these operations do not commute)

$$\mathbb{P}\left[X_{\mathcal{U}}\right] = \left\langle \left|\psi\left[X_{[d]}\right]\right|^2 \right\rangle_{[X_{\mathcal{U}}]} .$$

When we are interested in the preparation of quantum states with a specific computational basis measurement distribution, we can restrict to states with vanishing phase cores, that is

$$\psi\left[X_{[d]}\right] = \left\langle \exp\left[i \cdot 0\left[X_{[d]}\right]\right], |\psi|\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = |\psi|\left[X_{[d]}\right] .$$

#### 12.1.4.2 Phase sensitive measurement

When applying a Walsh-Hadamard transform on the qubits $\mathcal{V}\backslash\mathcal{U}$ to be closed (see Figure 12.2), the probability of measuring the closed qubits in the ground state is

$$\mathbb{P}\left[X_{\mathcal{V}\setminus\mathcal{U}} = 0_{\mathcal{V}\setminus\mathcal{U}}, X_{\mathcal{U}}\right] = \frac{1}{2^{|\mathcal{U}|}} \cdot \left|\left\langle \psi\left[X_{[d]}\right]\right\rangle_{[X_{\mathcal{U}}]}\right|^2 .$$

We thus sample from the contracted quantum state. Note that in this way, the relative phases of the amplitudes are contracted before the absolutes are taken. In this way, the measurement distribution is sensitive to the relative phases. This property is exploited in the Deutsch-Jozsa algorithm, which utilizes signs of the amplitudes.

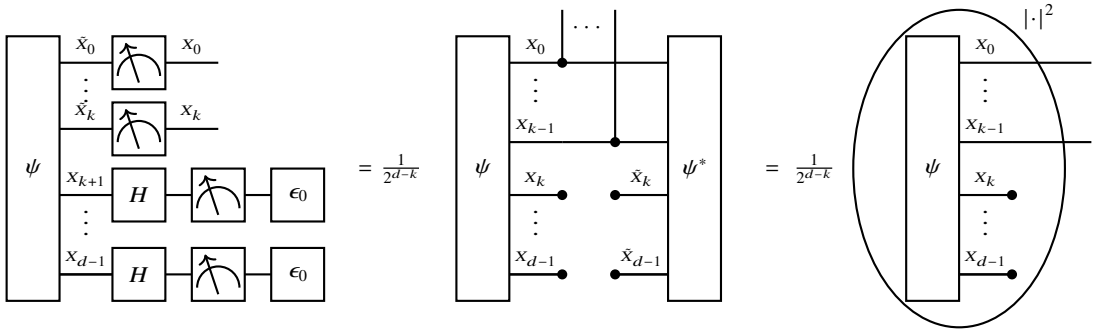



**Fig. 12.2** Computational Basis Measurement of a quantum state $\psi$, after the Walsh-Hadamard Transform on the closed qubits. By the ellipse we indicate a coordinatewise transform to the square of the absolute, after the contraction.

#### 12.1.4.3 POVM measurements

The most general measurement formalism is through a POVM, a set

$$\{E_y[X^{\text{in}}, X^{\text{out}}] \,:\, y \in [r]\}$$

of positive operators with

$$\sum_{y\in[r]} E_y[X^{\text{in}}, X^{\text{out}}] = \delta\left[X^{\text{in}}, X^{\text{out}}\right]$$

Measuring a pure state $|\psi\rangle$ we get the outcome $y$ with probability

$$\left\langle \psi\left[X^{\text{out}}\right], E_y[X^{\text{in}}, X^{\text{out}}], \psi\left[X^{\text{in}}\right]\right\rangle_{[\varnothing]}$$

Introducing $Y$ as an enumeration we define the tensor

$$E[Y, X^{\text{in}}, X^{\text{out}}] = \sum_{y\in[r]} \epsilon_y[Y] \otimes E_y[X^{\text{in}}, X^{\text{out}}] .$$

The condition on $E$ is then

$$\left\langle E[Y, X^{\text{in}}, X^{\text{out}}]\right\rangle_{\left[X^{\text{in}}, X^{\text{out}}\right]} = \delta\left[X^{\text{in}}, X^{\text{out}}\right] .$$

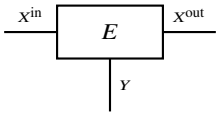
The measurement distribution of a pure state is then

$$\left\langle \psi\left[X^{\text{out}}\right], E[Y, X^{\text{in}}, X^{\text{out}}], \psi\left[X^{\text{in}}\right]\right\rangle_{[Y]} .$$

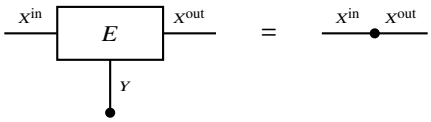
When having a mixed state defined by a density matrix $\rho[X^{\text{in}}, X^{\text{out}}]$, the distribution is

$$\left\langle E[Y, X^{\text{in}}, X^{\text{out}}], \rho[X^{\text{in}}, X^{\text{out}}]\right\rangle_{[Y]} .$$

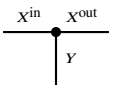
In our diagramatic picture, POVMs are tensors




such that




The standard computational basis measurement is




and satisfies these criteria.

## 12.2 Directed Circuits

We introduce a class of quantum circuits, which we frame directed circuits. We show that their measurement distribution is equivalent with that of Bayesian networks.

### 12.2.1 Definition

We build on the definition of uniformly controlled unitaries (see [58]), and study their alignment along directed acyclic hypergraphs. We define directed circuits using hypergraphs, which nodes are associated with the qubits of the circuit. The hyperedges are tuples of two subsets of the qubits, namely the incoming qubits representing control and the outgoing qubit representing the target of a unitary. To each hyperedge we then associate a uniformly controlled unitary.

**Definition 12.1 (Directed Circuit)** Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed acyclic hypergraph, where each hyperedge has exactly one outgoing node and all nodes appear exactly once as outgoing nodes of an hyperedge. Then a directed circuit on $\mathcal{G}$ is a decoration of the edges $e = (e^{\text{in}}, \{v\}) \in \mathcal{E}$ by uniformly controlled unitaries

$$\mathcal{U}^e \left[ X^v_{\text{in}}, X^v_{\text{out}}, X^{e^{\text{in}}}_{\text{out}} \right] .$$

Since the hypergraph $\mathcal{G}$ to a directed circuit is acyclic, we find an order of the qubits, such that to each qubit appearing in the outgoing qubits of a hyperedge is ordered after the incoming qubits. In such way, we have a unique definition of the directed circuit as a concatenation of the controlled unitaries respecting a directionality order. When there are multiple orders respecting the directionality, then the concatenation respecting the orders all lead to an equivalent circuit.

*Example 12.1 (Generic State Representation)* In [58] a preparation scheme for arbitrary states with real and positive amplitudes by directed circuits is presented. Therein, the hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is constructed as follows:

- Nodes $\mathcal{V} = [d]$
- Edges $\mathcal{E} = \{([k], \{k\}) \,:\, k \in [d]\}$

It is possible to realize each controlled unitary by a sequence of single qubit rotations and C-NOTs (see [58]), where the angles are computed using gray codes.

### 12.2.2 Preparation of q-samples to Bayesian Networks

Here: Bayesian networks with boolean variables

#### 12.2.2.1 Activation circuits

We now study circuits preparing q-samples of Bayesian Networks.

Activation circuits are uniformly controlled unitaries (see [58]), where the rotation axis is chosen as the $y$-axes of the Bloch sphere, and the angles are computed by the function $h(\cdot)$ acting on a function value to be encoded.

We define the angle preparing function on $p \in [0,1]$ by

$$h(p) = 2 \cdot \cos^{-1}\left(\sqrt{1-p}\right) .$$

For any $p \in [0,1]$ we then have

$$\left\langle \epsilon_0 \left[A^{\text{in}}\right], R_Y\left(h(p)\right)\left[A^{\text{in}}, A^{\text{out}}\right]\right\rangle_{\left[A^{\text{out}}\right]} = \begin{pmatrix}\sqrt{1-p}\\ \sqrt{p}\end{pmatrix}\left[A^{\text{out}}\right] .$$

**Definition 12.2 (Activation circuit)** Given a function

$$f : \bigtimes_{k\in[d]} [m_k] \to [0,1]$$

its activation circuit is the uniformly controlled unitary $\mathcal{V}^f\left[A^{\text{in}}, A^{\text{out}}, X_{[d]}\right]$ defined as

$$\mathcal{V}^\tau\left[A^{\text{in}}, A^{\text{out}}, X_{[d]}\right] = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes R_Y\left(h(f\left[x_{[d]}\right])\right)\left[A^{\text{in}}, A^{\text{out}}\right] .$$

When we have a probability tensor, its activation circuit be prepared, since all values are in $[0,1]$. Note that for rejection sampling, only the quotients of the values are important, we can therefore scale the value by a scalar such that the mode is 1.

Tensors $\tau\left[X_{[d]}\right]$ with non-negative coordinates can be encoded after dividing them by their maximum, that is the activation circuit of the function

$$f\left[x_{[d]}\right] = \frac{\tau\left[X_{[d]} = x_{[d]}\right]}{\max_{\tilde{y}_{[p]}} \tau\left[X_{[d]} = \tilde{y}_{[p]}\right]}$$

When the maximum of the tensor is not known, it can be replaced by an upper bound (reducing the acceptance rate of the rejection sampling).

#### 12.2.2.2 Q-samples of Bayesian Networks

Q-samples of Bayesian Networks can be prepared by quantum circuits, where each conditional probability distribution is prepared by an activation circuit.

**Theorem 12.1 ([50])** *Let there be a Bayesian Network $\mathbb{P}\left[X_{[d]}\right]$ on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ with dimensions $m_k = 2$ and decorations of the edges $\mathcal{E} = \{(\text{Pa}(k), \{k\}) : k \in [d]\}$ by conditional probabilities*

$$\mathbb{P}\left[X_k \middle| X_{\text{Pa}(k)}\right] .$$

*To each $k \in [d]$ we construct the function*

$$q_k : \bigtimes_{\tilde{k}\in \mathrm{Pa}(k)} [2] \to [2] \quad , \quad q_k(x_{\mathrm{Pa}(k)}) = \mathbb{P}\left[X_k = 1 \middle| X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right] .$$

*Then the concatenation of the activation circuits (see Figure 12.3)*

$$\mathcal{V}^{q_k}\left[X_k^{\mathrm{in}}, X_k, X_{\mathrm{Pa}(k)}\right]$$

*respecting the order* $[d]$ *prepares the q-sample to the Bayesian Network* $\mathbb{P}\left[X_{[d]}\right]$ *when acting on the initial state* $\bigotimes_{k\in[d]} \epsilon_0\left[X_k^{\mathrm{in}}\right]$.

*Proof.* Let $\psi\left[X_{[d]}\right]$ be the state prepared by the activation circuits to the conditional probability distributions. We need to show that the measurement distribution of $\psi$ is the Bayesian Network. For any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have

$$\begin{aligned}\left|\psi\left[X_{[d]} = x_{[d]}\right]\right|^2 &= \prod_{k\in[d]} \left(\mathcal{V}^{q_k}\left[X_k^{\mathrm{in}} = 0, X_k = x_k, X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right]\right)^2 \\ &= \prod_{k\in[d]} \mathbb{P}\left[X_k = x_k \middle| X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right] \\ &= \mathbb{P}\left[X_{[d]} = x_{[d]}\right] .\end{aligned}$$

□

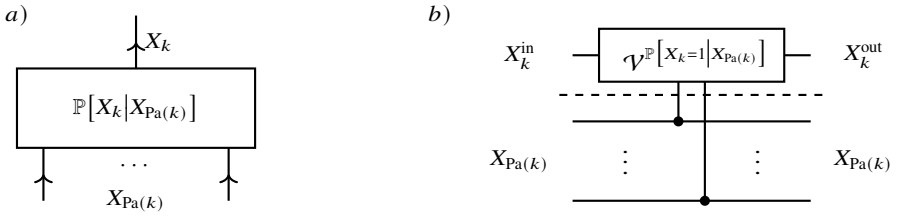



**Fig. 12.3** Representation of directed and positive tensor by a controlled rotation. a) Conditional probability tensor $\mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right]$ being a tensor in a Bayesian Network. b) Circuit Encoding as a controlled rotation, which is the Activation circuit of the tensor $\mathbb{P}\left[X_k = 1 \middle| X_{\mathrm{Pa}(k)}\right]$.

*Example 12.2 (Encoding of discretized distributions)* We here review the preparation of a discretized distribution from [34] as an application of activation circuits. Let $\mathbb{Q}[X]$ be a continuous probability distribution of a variable $X$ taking values in $[0, 1]$. For any $d \in \mathbb{N}$ we define its discretization of order $d$ as a discrete probability distribution $\mathbb{P}\left[X_{[d]}\right]$ with coordinates

$$\mathbb{P}^d\left[X_{[d]} = x_{[d]}\right] = \mathbb{Q}\left[X \in [\sum_{k\in[d]} 2^{-(k+1)} x_k, 2^{-d} + \sum_{k\in[d]} 2^{-(k+1)} x_k]\right] .$$

Then we have for any $p \in \mathbb{N}$ with $p \leq d$

$$\mathbb{P}^p\left[X_{[p]}\right] = \left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{\left[X_{[p]}\right]} .$$

The discretization order is thus increased by the contraction

$$\mathbb{P}^{d+1}\left[X_{[d+1]}\right] = \left\langle \mathbb{P}^d\left[X_{[d]}\right], \mathbb{P}^{d+1}\left[X_d | X_{[d]}\right]\right\rangle_{\left[X_{[d+1]}\right]} .$$

Having an activation circuit preparing $\mathbb{P}^d$ we can therefore prepare $\mathbb{P}^{d+1}$ by adding the activation circuit for the function

$$x_{[d]} \to \mathbb{P}^{d+1}\left[X_d = 1 | X_{[d]} = x_{[d]}\right] .$$

Note that the function evaluation amounts to the integration of $\mathbb{Q}$ as

$$\mathbb{P}^{d+1}\left[X_d = 1 | X_{[d]} = x_{[d]}\right] = \frac{\mathbb{Q}\left[X \in [2^{-(d+1)} + \sum_{k\in[d]} 2^{-(k+1)} x_k, 2^{-d} + \sum_{k\in[d]} 2^{-(k+1)} x_k]\right]}{\mathbb{Q}\left[X \in [\sum_{k\in[d]} 2^{-(k+1)} x_k, 2^{-d} + \sum_{k\in[d]} 2^{-(k+1)} x_k]\right]} .$$

### 12.2.3 Equivalence with Bayesian Networks

Above we have shown, how Bayesian Networks can be prepared by activation circuits, which are a class of directed circuits. We now show, that the measurement distribution of any directed circuit is a Bayesian Network. It then follows that the set of distributions prepared by directed circuits is exactly the set of Bayesian Networks.

This is closely connected to the chain decomposition of probability distributions, and their construction. In [50] Bayesian Networks are prepared by directed circuits. When in the graph $\mathcal{G}$ each node appears at most once as outgoing node, it is also the hypergraph to a family of Bayesian Networks. The measurement distributions of a state prepared by a $\mathcal{G}$-controlled Circuit acting on disentangled initial states are exactly the Bayesian Networks with respect to $\mathcal{G}$.

**Theorem 12.2** *Let $\mathcal{G}$ be a directed acyclic hypergraph, such that node appears at most once as an outgoing node. The measurement distributions of the by $\mathcal{G}$ controlled circuits acting on disentangled initial states are equal to the Bayesian Networks on $\mathcal{G}$.*

**Lemma 12.1** *Let $\mathcal{G}$ be a directed acyclic hypergraph, such that node appears at most once as an outgoing node. Then any Bayesian network on $\mathcal{G}$ can be prepared by a $\mathcal{G}$-controlled circuit with activation circuits of the conditional probability tensors.*

*Proof.* Let $\mathbb{P}\left[X_{[d]}\right]$ be a Bayesian network on the graph $\mathcal{G}$. Enumerate the nodes $\mathcal{V}$ of the $\mathcal{G}$ by $[d]$, such that for each $k \in [d]$ we have $\mathrm{Pa}(k) \subset [k]$. Then define a $\mathcal{G}$-controlled circuit, by choosing for each $k \in [d]$ controlled unitaries which satisfy

$$\mathcal{U}^k\left[X_k^{\mathrm{in}} = 0, X_{v,\mathrm{out}}, X_{\mathrm{Pa}(k),\mathrm{out}}\right] = \sqrt{\mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right]} .$$

Here we specified only the action of the controlled unitary on the basis vector $\epsilon_0[X_k]$, the action on $\epsilon_1[X_k]$ can be chosen by an arbitrary orthogonal unit vector. For more explicit construction, see the activation circuits. Any such defined $\mathcal{G}$-controlled circuit acting on the initial state $\bigotimes_{k\in[d]} \epsilon_0[X_k]$ prepares a quantum state $\psi\left[X_{[d]}\right]$ with measurement distribution

$$|\psi|^2\left[X_{[d]}\right] .$$

Given arbitrary $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have

$$|\psi|^2\left[X_{[d]} = x_{[d]}\right] = \prod_{k\in[d]} \mathbb{P}\left[X_k = x_k \middle| X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right] = \mathbb{P}\left[X_{[d]} = x_{[d]}\right] .$$

Here we used in the last equation, that $\mathbb{P}\left[X_{[d]}\right]$ is a Bayesian network. Since the equivalence holds for any coordinate, this establishes the equivalence of the measurement distribution of $\psi\left[X_{[d]}\right]$ and $\mathbb{P}\left[X_{[d]}\right]$. □

While we have already shown by Lem. 12.1 that arbitrary Bayesian Networks can be prepared by directed circuits, we now show the converse, namely that any distribution prepared by directed circuits is a Bayesian Network. To this end, we use the characterization of Bayesian Networks by the conditional independencies they encode (see [45]). To be more precise, we need to show that any node variable is conditionally independent of its non-descendants given its parents.

**Lemma 12.2** *Let $(\mathcal{G}, \mathcal{U})$ be a $\mathcal{G}$-controlled circuit acting on a disentangled initial state and $\mathbb{P}[X_\mathcal{V}]$ the corresponding measurement distribution. Then we have for each $v \in \mathcal{V}$ the conditional independence*

$$\left(X_v \perp X_{\mathrm{NonDes}(v)}\right) \middle| X_{\mathrm{Pa}(v)} .$$

*Proof.* We choose to a given $v \in \mathcal{V}$ an enumeration $[d]$ of the nodes, such that for each $k \in [d]$ we have $\mathrm{Pa}(k) \subset [k]$ and for the enumerator $\tilde{k}$ of $v$ we further have $\mathrm{NonDes}(\tilde{k}) \subset [\tilde{k}]$. Let $\mathbb{P}\left[X_{[d]}\right]$ be the measurement distribution of the $\mathcal{G}$-controlled circuit acting on a disentangled initial state $\bigotimes_{k\in[d]} \psi^k\left[X_k\right]$ and choose arbitrary $x_{[k]}$. We then have

$$\begin{aligned}
\mathbb{P}\left[X_{\tilde{k}}, X_{[k]} = x_{[k]}\right] &= \left\langle \left( \bigcup_{k\in[d]} \{\mathcal{U}^k, \mathcal{U}^{k,\dagger}, \psi^k, \psi^{k,*}\} \right) \cup \left( \bigcup_{k\in[\tilde{k}]} \epsilon_{x_k}\left[X_k^{\mathrm{out}}\right] \right) \right\rangle_{[X_{\tilde{k}}]} \\
&= \left\langle \bigcup_{k\in[\tilde{k}]} \{\mathcal{U}^k, \mathcal{U}^{k,\dagger}, \psi^k, \psi^{k,*}, \epsilon_{x_k}\left[X_k^{\mathrm{out}}\right]\} \right\rangle_{[X_{\tilde{k}}]} \\
&= \left| \left\langle \mathcal{U}^{\tilde{k}}\left[X_{\tilde{k},\mathrm{in}}, X_{\tilde{k},\mathrm{out}}\right], X_{\mathrm{Pa}(\tilde{k}),\mathrm{out}} = x_{\mathrm{Pa}(\tilde{k}),\mathrm{out}} \right\rangle_{[X_{\tilde{k},\mathrm{out}}]} \right|^2 \\
&\quad \cdot \prod_{k\in[\tilde{k}]} \left( \left\langle \psi^k\left[X_k^{\mathrm{in}}\right], \mathcal{U}^k\left[X_k^{\mathrm{in}}, X_k^{\mathrm{out}}, X_{\mathrm{Pa}(k),\mathrm{out}} = x_{\mathrm{Pa}(k),\mathrm{out}}\right] \right\rangle_{[\varnothing]} \right)^2
\end{aligned}$$

Here we used in the second equation the unitarity of the controlled unitaries to $k \notin [\tilde{k}]$. Since the indices $x_{[\tilde{k}]/\mathrm{Pa}(\tilde{k})} = x_{\mathrm{NonDes}(\tilde{k})}$ appear only in the constant term, we conclude

$$\mathbb{P}\left[X_{\tilde{k}} \middle| X_{[k]}\right] = \mathbb{P}\left[X_{\tilde{k}} \middle| X_{\mathrm{Pa}(\tilde{k})}\right] \otimes \mathbb{I}\left[X_{\mathrm{NonDes}(\tilde{k})}\right] ,$$

which establishes the conditional independence $\left(X_v \perp X_{\mathrm{NonDes}(v)}\right) \middle| X_{\mathrm{Pa}(v)}$. □

*Proof of Thm. 12.2.* The theorem follows directly from Lem. 12.1 and Lem. 12.2, using that Bayesian Networks are characterized by the conditional independence of each variable to its non-descendants given its parents. □

*Example 12.3 (Chain decomposition)* The hypergraph in Example 12.1 corresponds with a family of Bayesian Networks subsuming any probability distribution. This can be verified based on the chain decomposition of a generic distribution.

Another question is, whether each quantum state, which measurement distribution is a Bayesian Network can be prepared by a $\mathcal{G}$-controlled circuit. This is not always the case, since the phase tensor does not influence the measurement distribution. Any phase tensor, of a by $\mathcal{G}$-controlled circuit prepared state has however a decomposition

$$\phi\left[X_{[d]}\right] = \sum_{k\in[d]} \phi^k\left[X_k, X_{\mathrm{Pa}(k)}\right] \otimes \mathbb{I}\left[X_{[d]/\{\{k\}\cup\mathrm{Pa}(k)\}}\right] ,$$

where the phase cores $\phi^k$ can be read of the controlled unitaries. When there are phase cores which do not have such a decomposition, the corresponding states are not representable. [58] shows, that when the graph is chosen as in Example 12.1, then also the phases can be represented.

### 12.2.4 Ancilla Augmentation

We provide here some further theory underlying the ancilla augmentation of Computation-Activation Networks.

Graph controlled circuits prepare Bayesian Networks, and do not apply to undirected graphical models such as Markov Networks and Computation-Activation Networks. We therefore introduce in the following ancilla augmentation, which is a method to represent Markov Networks as conditioned Bayesian Network. This enables usage of the previously used preparation schemes for Bayesian Networks.

#### 12.2.4.1 Post-selection by Ancilla Variables

First of all we define ancilla augmentation and then show how they can be prepared as measurement distributions using activation circuits.

**Definition 12.3 (Ancilla Augmentation of a Distribution)** Let $\mathbb{P}\left[X_{[d]}\right]$ be a probability distribution of variables $X_{[d]}$. Another distribution $\tilde{\mathbb{P}}[X_{[d]}, A_{[p]}]$ of variables $X_{[d]}$ and ancilla variables $A_{[p]}$ is called an ancilla augmentation of $\mathbb{P}\left[X_{[d]}\right]$, if there is $\lambda > 0$ with

$$\lambda \cdot \mathbb{P}\left[X_{[d]}\right] = \tilde{\mathbb{P}}[X_{[d]}, A_{[p]} = 1_{[p]}] \,.$$

We refer to $\lambda$ as the corresponding acceptance probability.

Note that $\lambda \leq 1$ since

$$\lambda = \left\langle \lambda \cdot \mathbb{P}\left[X_{[d]}\right] \right\rangle_{[\varnothing]} = \left\langle \tilde{\mathbb{P}}[X_{[d]}, A_{[p]} = 1_{[p]}] \right\rangle_{[\varnothing]} \leq \left\langle \tilde{\mathbb{P}}[X_{[d]}, A_{[p]}] \right\rangle_{[\varnothing]} = 1 \,.$$

Sampling from the distribution can be done by rejection sampling, also called post selection, on the ancilla augmented distribution in the following way. When drawing a sample $(x_{[d]}, a)$ from $\tilde{\mathbb{P}}[X_{[d]}, A]$ and rejecting it whenever $a = 0$, the accepted samples are distributed as $\mathbb{P}\left[X_{[d]}\right]$. We then have $\lambda$ as the probability of accepting a sampling in this rejection sampling method. The acceptance probability is critical in determining the efficiency of this approach, since we need to draw $\sim \frac{1}{\lambda}$ samples from $\tilde{\mathbb{P}}[X_{[d]}, A]$ to get an accepted sample.

Since

$$\tilde{\mathbb{P}}[X_{[d]}, A] = \left\langle \tilde{\mathbb{P}}[X_{[d]}], \tilde{\mathbb{P}}[A|X_{[d]}] \right\rangle_{[X_{[d]}, A]} \tag{12.1}$$

the described sampling procedure is equal to drawing $x_{[d]}$ from the marginal

$$\tilde{\mathbb{P}}[X_{[d]}]$$

and accepting the sample with probability

$$\tilde{\mathbb{P}}[A = 1|X_{[d]}] \,.$$

#### 12.2.4.2 Construction based on Uniform Distributions

We construct ancilla augmentations based on the decomposition (12.1) where the marginal of $X_{[d]}$ is the uniform distribution, that is

$$\tilde{\mathbb{P}}[X_{[d]}] = \frac{1}{\prod_{k \in [d]} m_k} \mathbb{I}\left[X_{[d]}\right]$$

and we construct the conditional probability of the ancilla variable such that

$$\tilde{\mathbb{P}}[A = 1|X_{[d]}] = \lambda \cdot \left( \prod_{k \in [d]} m_k \right) \mathbb{P}\left[X_{[d]}\right] \,.$$

For the latter we can apply the preparation schemes of conditioned distributions, which have been studied above for Bayesian Networks.

We denote for any non-negative tensor $\tau\left[X_{[d]}\right]$ we construct

$$\begin{aligned} \mathcal{A}^{\tau,\lambda}\left[A, X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \Big( & \lambda \cdot \big( \prod_{k \in [d]} m_k \big) \cdot \tau\left[X_{[d]} = x_{[d]}\right] \cdot \epsilon_1\left[A\right] \\ & + \big(1 - \lambda \cdot \big( \prod_{k \in [d]} m_k \big) \cdot \tau\left[X_{[d]} = x_{[d]}\right] \big) \cdot \epsilon_0\left[A\right] \Big) \,. \end{aligned}$$

For this to be a conditional distribution we need to demand that

$$\max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \lambda \cdot \Big( \prod_{k\in[d]} m_k \Big) \cdot \tau\left[X_{[d]} = x_{[d]}\right] \leq 1\,,$$

which is equivalent to

$$\lambda \leq \left( \prod_{k\in[d]} m_k \right)^{-1} \cdot \left( \max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \tau\left[X_{[d]} = x_{[d]}\right] \right)^{-1} .$$

Note that if $\tau\left[X_{[d]}\right]$ is the uniform distribution $\mathbb{I}\left[X_{[d]}|\varnothing\right]$, then we can achieve the best acceptance rate $\lambda = 1$.

Given a distribution $\mathbb{P}\left[X_{[d]}\right]$ we add an ancilla variable $A$ and construct the augmented distribution (see Figure 12.4)

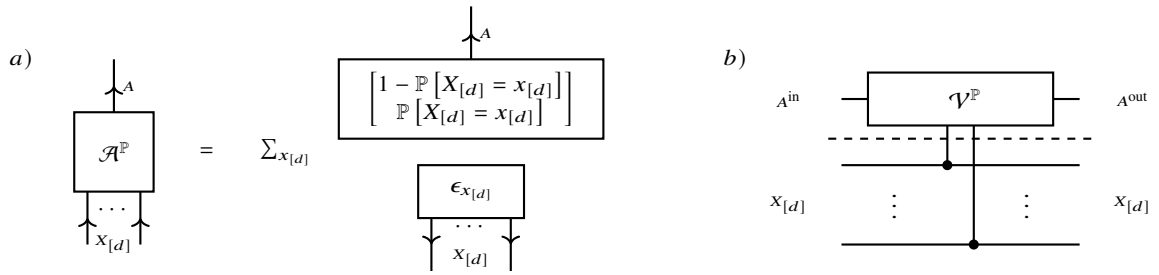



**Fig. 12.4** Ancilla augmentation of a distribution $\mathbb{P}\left[X_{[d]}\right]$. a) Augmented distribution $\mathcal{A}^{\mathbb{P}}\left[A, X_{[d]}\right]$ with the property that $\mathbb{P}\left[X_{[d]}\right] = \mathcal{A}^{\mathbb{P}}\left[X_{[d]}|A = 1\right]$. b) Preparation of the augmented distribution by the activation circuit of $\mathbb{P}\left[X_{[d]}\right]$.

**Lemma 12.3** *The activation circuit to a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ applied on the initial state*

$$\sqrt{\frac{1}{\prod_{k\in[d]} m_k}}\,\mathbb{I}\left[X_{[d]}\right] \otimes \epsilon_0\left[A\right]$$

*prepares a quantum state $\psi$, which is the Q-sample of the ancilla augmentation*

$$\left\langle \mathcal{A}^{\mathbb{P},\frac{1}{m}}\left[A, X_{[d]}\right] \right\rangle_{\left[A, X_{[d]}|\varnothing\right]}$$

*with the acceptance probability $m^{-1} := \left(\prod_{k\in[d]} m_k\right)^{-1}$.*

*Proof.* We first show, that $\left\langle \mathcal{A}^{\mathbb{P},\frac{1}{m}}\left[A, X_{[d]}\right] \right\rangle_{\left[A, X_{[d]}|\varnothing\right]}$ is indeed an ancilla augmentation. We have

$$\lambda = \left\langle \mathcal{A}^{\mathbb{P}}\left[A, X_{[d]}\right] \right\rangle_{[A=1]} = \frac{1}{m} \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] = \frac{1}{m}\,.$$

and for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ it holds that

$$\left\langle \mathcal{A}^{\mathbb{P},\frac{1}{m}}\left[A, X_{[d]}\right] \right\rangle_{\left[A=1, X_{[d]}=x_{[d]}|\varnothing\right]} = \frac{1}{\lambda} \frac{1}{\prod_{k\in[d]} m_k} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] = \mathbb{P}\left[X_{[d]} = x_{[d]}\right]\,,$$

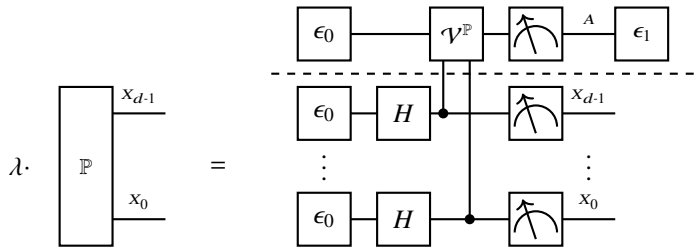
The tensor $\left\langle \mathcal{A}^{\mathbb{P},\frac{1}{m}}\left[A, X_{[d]}\right] \right\rangle_{\left[A, X_{[d]}|\varnothing\right]}$ is thus an ancilla augmentation for $\mathbb{P}$.

We continue to show that the quantum state $\psi$ prepared by the activation circuit is the Q-sample of $\left\langle \mathcal{A}^{\mathbb{P},\frac{1}{m}}\left[A, X_{[d]}\right] \right\rangle_{\left[A, X_{[d]}|\varnothing\right]}$. Then we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$

$$\psi\left[A, X_{[d]} = x_{[d]}\right] = \sqrt{\frac{1}{\prod_{k\in[d]} m_k}}\left(\sqrt{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}\cdot \epsilon_1\left[A\right] + \sqrt{(1-\mathbb{P}\left[X_{[d]} = x_{[d]}\right])}\cdot \epsilon_0\left[A\right]\right),$$

which is indeed the Q-sample. □

In our tensor network circuit notation, Lem. 12.3 states the equation




for the acceptance probability (see proof of Lem. 12.3)

$$\lambda = \frac{1}{\prod_{k\in[d]} m_k}\,.$$

Note that the acceptance probability is exponentially small in the number of variables. If $\mathbb{P}\left[X_{[d]}\right]$ is not uniform, we can improve by building the activation circuit to the function

$$\frac{1}{\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}\cdot\mathbb{P}\left[X_{[d]}\right]\,.$$

#### 12.2.4.3 Best Acceptance Probability by ∞-Renyi Divergence

Let us now show, how the best acceptance probability in sets of ancilla augmented distributions is connected with ∞-Renyi divergences. For $0 < \alpha < \infty$ and $\alpha \neq 1$ the Renyi Divergence is defined as

$$D_\alpha\left[\mathbb{P}||\mathbb{Q}\right] = \frac{1}{1-\alpha}\ln\left[\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]^\alpha}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]^{\alpha-1}}\right].$$

The ∞ Renyi Divergence is the limit $\alpha\to\infty$

$$D_\infty\left[\mathbb{P}||\mathbb{Q}\right] = \ln\left[\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right].$$

**Theorem 12.3** *Let $\mathbb{P}\left[X_{[d]}\right]$ and $\mathbb{Q}[X_{[d]}]$ be distributions, such that for each $x_{[d]}\in\bigtimes_{k\in[d]}[m_k]$ with $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] > 0$ we also have $\mathbb{Q}[X_{[d]} = x_{[d]}] > 0$. Let us consider all ancilla augmentations $\tilde{\mathbb{P}}[X_{[d]}, A]$ which marginal is $\mathbb{Q}[X_{[d]}]$, that is*

$$\mathbb{Q}[X_{[d]}] = \left\langle\tilde{\mathbb{P}}[X_{[d]}, A]\right\rangle_{[X_{[d]}]}\,.$$

*The best acceptance probability among these ancilla augmentations is then*

$$\lambda = \exp\left[-D_\infty\left[\mathbb{P}||\mathbb{Q}\right]\right]\,,$$

*and achieved when for any $x_{[d]}\in\bigtimes_{k\in[d]}[m_k]$*

$$\tilde{\mathbb{P}}[A=1|X_{[d]} = x_{[d]}] = \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\cdot\left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right)^{-1}\,.$$

*Proof.* Any $\tilde{\mathbb{P}}[X_{[d]}, A]$ with marginal $\mathbb{Q}\left[X_{[d]}\right]$ has a decomposition

$$\tilde{\mathbb{P}}[X_{[d]}, A] = \left\langle \mathbb{Q}\left[X_{[d]}\right], \tilde{\mathbb{P}}[A|X_{[d]}] \right\rangle_{[X_{[d]}, A]}$$

and in order to be an ancilla augmentation for $\mathbb{P}\left[X_{[d]}\right]$ we find $\lambda > 0$ with

$$\tilde{\mathbb{P}}[X_{[d]}, A = 1] = \lambda \cdot \mathbb{P}\left[X_{[d]}\right] .$$

It follows that for each $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$

$$1 \geq \tilde{\mathbb{P}}[A = 1|X_{[d]} = x_{[d]}] = \lambda \cdot \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} .$$

and thus

$$\lambda \leq \left( \max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \right)^{-1} . \tag{12.2}$$

For the in the claim constructed ancilla augmentation, we have the acceptance probability

$$\begin{aligned}
\tilde{\mathbb{P}}[A = 1] &= \mathbb{E}_{x_{[d]}\sim\mathbb{Q}} \left[ \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \cdot \left( \max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \right)^{-1} \right] \\
&= \left\langle \mathbb{Q}\left[X_{[d]}\right], \left(\mathbb{Q}\left[X_{[d]}\right]\right)^{-1}, \mathbb{P}\left[X_{[d]}\right] \right\rangle_{[\varnothing]} \left( \max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \right)^{-1} \\
&= \left( \max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \right)^{-1} \\
&= \exp\left[-D_\infty\left[\mathbb{P}||\mathbb{Q}\right]\right] .
\end{aligned}$$

The constructed ancilla augmented distribution satisfies the bound (12.2) on the acceptance probability holds straight, and it is thus the one with largest acceptance rate. □

Note, that in the section above we have constructed ancilla augmentations with uniform $\mathbb{Q}$ by activation circuits. For these the optimal acceptance rate is

$$\lambda = \left( \prod_{k\in[d]} m_k \right)^{-1} \cdot \left( \max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \right)^{-1} .$$

Thus, $\lambda = 1$ is only achievable if $\mathbb{P}\left[X_{[d]}\right]$ is itself the uniform distribution.

**Extension:** We could increase the acceptance probability, when we sample from a proposal distribution $\mathbb{Q}$ with smaller $\infty$ Renyi divergence to $\mathbb{P}$. When sampling with Quantum Circuits, this could be implemented by a state preparation for the distributed variables, before the Computation Activation Circuit is applied. It would be interesting to train variational quantum circuits for this task. However, when we want to apply the same scheme as above, one needs to encode $\mathbb{Q}$ into the ancilla preparing rotations, so $\mathbb{Q}$ would need to be an elementary Computation-Activation Network as well.

#### 12.2.4.4 Augmentation of Markov Networks

Let us now show how multiple ancilla variables can be designed to sample from arbitrary Markov Networks.

*Example 12.4 (Elementary Tensors)* Let there be an elementary distribution

$$\mathbb{P}\left[X_{[d]}\right] = \bigotimes_{k\in[d]} \mathbb{P}^k\left[X_k\right] ,$$

that is, all variables are pairwise independent. Then the ancilla augmentation of the distribution is given by the Bayesian Network with the hypergraph shown in Figure 12.5a). Compared with the ancilla augmentation of a generic distribution this augmentation is sparser in the circuit depth (see Figure 12.5b), but introduces multiple ancilla variables. The sparsity originates since each ancilla variable is controlled by a single qubit. The total amount of controlled rotations is thus $\sum_{k\in[d]}[m_k]$ instead of $\prod_{k\in[d]}[m_k]$ for the generic implementation.

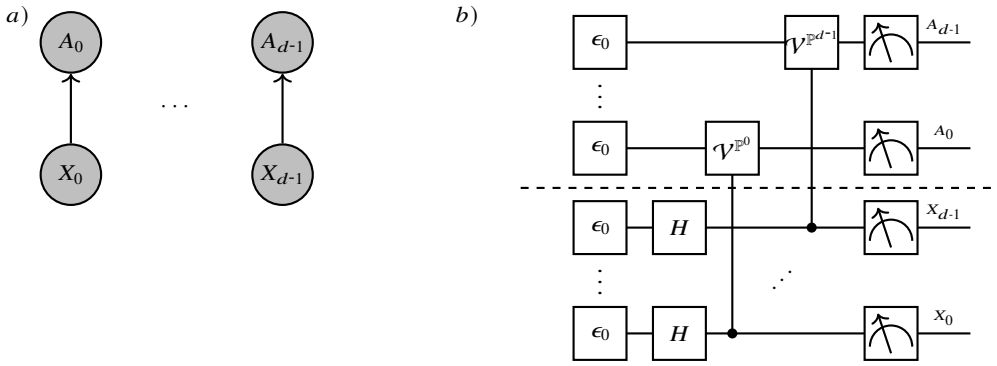



**Fig. 12.5** Ancilla augmentation of an elementary distribution (which variables are pairwise independent). a) Hypergraph to the Bayesian Network representing the ancilla augmentation. b) Circuit preparing the ancilla augmented distribution by activation circuits acting on the disentangled initial state.

Let there be a Markov Network, i.e. any non-vanishing tensor network of non-negative tensors, which is understood as a probability disribution after normalization (see [31]). For an example of a Markov Network see Figure 12.6a.

We apply a similar ancilla augmentation on any hypercore of a Markov Network, as has been done above for probability distributions. To prepare, we need to scale each core by dividing by its maximum (or any upper bound on its maximum), such that any coordinate is smaller than 1. We introduce for each core an boolean ancilla variable $A_e$ (while $X_e$ is the tuple of distributed variables collected in the hyperedge $e$) then build directed tensors

$$\mathcal{A}^{\tau^e,\lambda^e}\left[A_e, X_e\right] = \sum_{x_e} \epsilon_{x_e}\left[X_e\right] \otimes \Big( \lambda^e \cdot \big(\prod_{k\in e} m_k\big) \cdot \tau^e\left[X_e = x_e\right] \cdot \epsilon_1\left[A\right] + \left(1 - \lambda^e \cdot \big(\prod_{k\in e} m_k\big) \cdot \tau^e\left[X_e = x_e\right]\big) \cdot \epsilon_0\left[A\right] \Big) .$$

where we choose the maximal values

$$\lambda^e = \left(\prod_{k\in e} m_k\right)^{-1} \cdot \left(\max_{x_e} \tau^e\left[X_e = x_e\right]\right)^{-1} ,$$

such that $\mathcal{A}^e$ stays non-negative and can be interpreted as a conditional probability distribution. Note, that $\lambda^e$ is not an acceptance rate itself, since $\tau^e$ is not always a distribution.

**Lemma 12.4** *When augmenting all cores of a Markov Network with distinct ancilla variables in this way (see Figure 12.6b), we get an ancilla augmentation of the Markov Network.*

$$\tilde{\mathbb{P}}[A_{\mathcal{E}}, X_{[d]}] = \left\langle \{\mathcal{A}^{\tau^e,\lambda^e}\left[A_e, X_e\right] \,:\, e \in \mathcal{E}\} \cup \{\mathbb{I}\left[X_{[d]}|\varnothing\right]\}\right\rangle_{\left[A_{\mathcal{E}}, X_{[d]}\right]} .$$

*Proof.* $\tilde{\mathbb{P}}[A_{\mathcal{E}}, X_{[d]}]$ is a Bayesian Network (see Figure 12.6d) and thus a distribution. We further have

$$\begin{aligned}\tilde{\mathbb{P}}[A_{\mathcal{E}} = 1_{\mathcal{E}}, X_{[d]}] &= \left\langle \{\mathcal{A}^{\tau^e,\lambda^e}[A_e = 1, X_e] \,:\, e \in \mathcal{E}\} \cup \{\mathbb{I}\left[X_{[d]}|\varnothing\right]\}\right\rangle_{[A_{\mathcal{E}}, X_{[d]}]} \\ &= \frac{\prod_{e\in\mathcal{E}} \lambda^e \left(\prod_{k\in e} m_k\right)}{\prod_{k\in[d]} m_k} \langle\{\tau^e[X_e] \,:\, e \in \mathcal{E}\}\rangle_{[X_{[d]}]} \,.\end{aligned}$$

Thus, $\tilde{\mathbb{P}}[A_{\mathcal{E}}, X_{[d]}]$ is an ancilla augmentation of the Markov Network with acceptance probability

$$\frac{\prod_{e\in\mathcal{E}} \lambda^e \left(\prod_{k\in e} m_k\right)}{\langle\{\tau^e[X_e] \,:\, e \in \mathcal{E}\}\rangle_{[\varnothing]} \cdot \left(\prod_{k\in[d]} m_k\right)} = \left(\prod_{k\in[d]} m_k\right)^{-1} \cdot \frac{\prod_{e\in\mathcal{E}} \left(\max_{x_e} \tau^e[X_e = x_e]\right)}{\langle\{\tau^e[X_e] \,:\, e \in \mathcal{E}\}\rangle_{[\varnothing]}} \,.$$

□

The main advantage of this augmentation is, that all cores are directed (see Figure 12.6b, and build therefore a Bayesian Network. Using the activation circuits studied above, we can now determine a rejection sampling procedure for arbitrary Markov Networks.

**Lemma 12.5** *Let there be a Markov Network $\tau^{\mathcal{G}} = \{\tau^e[X_e] \,:\, e \in \mathcal{E}\}$ on a hypergraph $\mathcal{G}$. To each $e \in \mathcal{E}$ we build a function*

$$q_e = \frac{\tau^e[X_e = y_e]}{\max_{y_e} \tau^e[X_e = y_e]} \,.$$

*We then build a concatenation of activation circuits to each function $q_e$, where each has a distinct ancilla qubit $A_e$. Then the constructed directed circuit acting on the state*

$$\sqrt{\frac{1}{\prod_{v\in\mathcal{V}} m_v}} \mathbb{I}[X_{\mathcal{V}}] \otimes \left(\bigotimes_{e\in\mathcal{E}} \epsilon_0[A_e]\right)$$

*prepares an ancilla augmented distribution of the Markov Network.*

*Proof.* We show, that the measurement distribution is the ancilla augmented Markov Network. □

We use this result to prepare samples from Markov Networks by rejection sampling on the ancilla variables, in qcreason (see Sect. **??**).

When the maximum of the hypercores are not known, and only an upper bound is given, it suffice to rescale them such that the maximum is lower than 1.

When the maximal coordinate of the Markov Network is, restricted to each edge, the maximal coordinate of the hypercore of that edge, we say that the Markov Network is not frustrated, otherwise frustrated.

**Lemma 12.6** *If the Markov Network is not frustrated, then the acceptance probability $\lambda$ of the rejection sampling scheme using the ancilla augmented Markov Network (with respect to the uniform distribution) is the $\infty$-Renyi divergence between the Markov Network and the uniform distribution.*

*Proof.* If and only if the Markov Network is not frustrated we have

$$\max_{x_{[d]}} \left(\prod_{e\in\mathcal{E}} \tau^e[X_e = x_e]\right) = \prod_{e\in\mathcal{E}} \left(\max_{x_e} \tau^e[X_e = x_e]\right) \,.$$

In this case, the ancilla augmentation has the best acceptance probability, since for the state $x_{[d]}$ maximizing all local cores we have

$$\tilde{\mathbb{P}}[A_{[p]} = 1_{[p]}|X_{[d]} = x_{[d]}] = 1 \,.$$

Alternatively: Show this with the acceptance rate in proof of Lem. 12.4, comparing with best naive augmentation. □

If the Markov Network is frustrated, we cannot achieve the best acceptance probability by the distributed ancilla preparation approach described above. In this case, one faces a tradeoff between the size of the circuit to prepare ancilla augmentations and the acceptance probability.

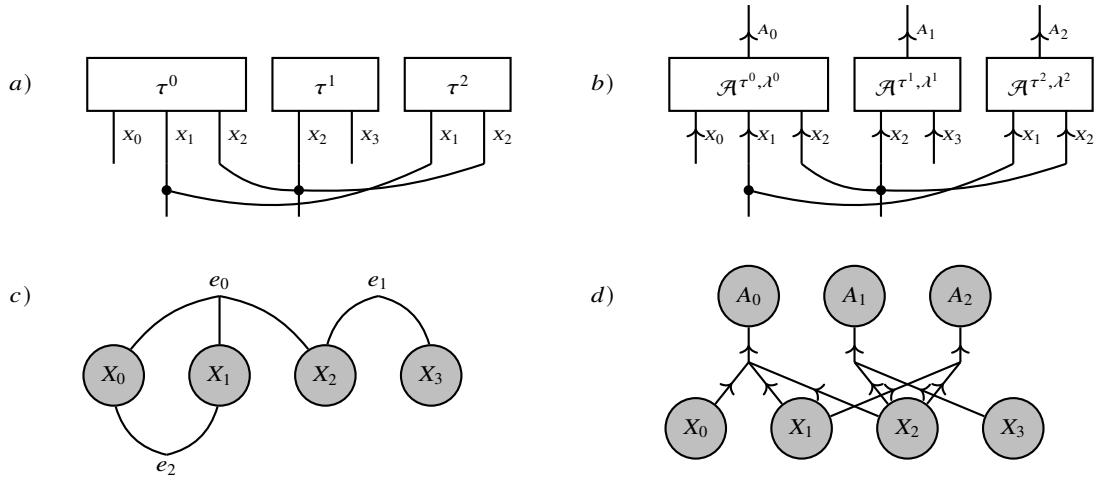



**Fig. 12.6** Ancilla augmentation of a Markov Network a) onto a Bayesian Network b). To this end the arbitrary non-negative cores $\tau^e$ are replaced by directed ancilla augmented cores $\tilde{\tau}^e$, with ancilly variables outgoing.

## 12.3 Computation circuits

In the previous sections we have studied the schemes to encode arbitrary conditional probability distributions, which concatenation prepares q-samples of Bayesian networks [50]. While these schemes do not assume any further structure in the conditional probability distributions, we now investigate how to exploit the structure to achieve sparser circuits. To be more precise, we focus on deterministic conditional probability distributions, which are basis encodings to functions. To this end, let us first introduce a quantum pendant to encode functions, which has the decomposition by contraction property as we show later. We will also relate them to basis encodings.

**Definition 12.4 (Computation circuit)** Given a boolean function $f : \bigtimes_{k\in[d]}[2] \to [2]$ the computation circuit is the unitary tensor

$$\begin{aligned} C^f\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right] \quad = & \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]\,:\,f\left[x_{[d]}\right]=1} \sigma_1\left[Y_f^{\text{in}}, Y_f^{\text{out}}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \\ & + \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]\,:\,f\left[x_{[d]}\right]=0} \delta\left[Y_f^{\text{in}}, Y_f^{\text{out}}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] . \end{aligned}$$

*Example 12.5 (CNOT gate)* The CNOT gate is the controlled Pauli-X, that is

$$\text{CNOT}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] = \sigma_1\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \epsilon_1\left[X\right] + \delta\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \epsilon_0\left[X\right] .$$

For the identity function

$$\text{Id}(x) = x .$$

we have

$$C^{\text{Id}}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] = \text{CNOT}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] .$$

The CNOT is therefore the computation circuit to the identity function.

Functions with multiple output variables, i.e. $f : \bigtimes_{k\in[d]}[2] \to \bigtimes_{\ell\in[p]}[2]$, can be encoded image coordinate wise as a concatenation of the respective circuits.

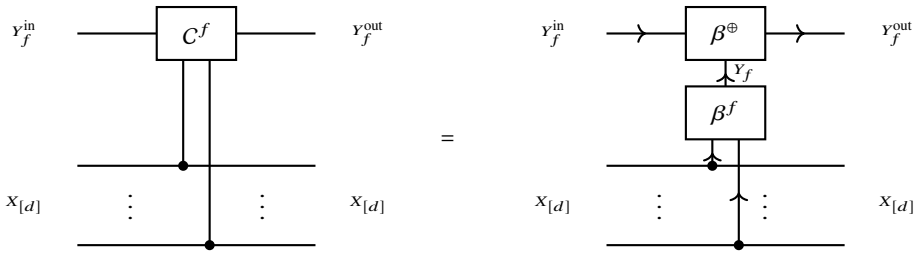



**Fig. 12.7** Relation of computation circuit and basis encodings. The computation circuit of a function is the contraction of its basis encoding with the basis encoding of the mod2 sum $\oplus$.

### 12.3.1 Relation with basis encodings

Basis encodings are schemes to encode functions into directed boolean tensors, which enable reasoning about functions by tensor network contractions (see [31]). To connect with this formalism, let us now show the relation to the computation circuits.

**Lemma 12.7** *For any boolean function we have (see Figure 12.7)*

$$C^f\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \beta^{\oplus}\left[Y_f^{\text{out}}, Y_f^{\text{in}}, Y_f\right]\right\rangle_{\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right]} .$$

The computation circuit applied on the initial state

$$\sqrt{\frac{1}{2^d}}\mathbb{I}\left[X_{[d]}^{\text{out}}\right] \otimes \epsilon_0\left[Y^{\text{in}}\right]$$

prepares a q-sample to the normalized basis encoding.

In [31] (and more detailed in [30]) sparse decompositions of basis encodings have been studied. Based on the equivalence of computation circuits and contracted basis encodings, stated by Lem. 12.7 we transfer these decomposition schemes in the following.

*Example 12.6 (Multiple-controlled NOT)* Let us consider the $d$-ary controlled NOT gate

$$\text{MCNOT}^d\left[Y^{\text{in}}, Y^{\text{out}}, X_{[d]}\right] = \sigma_1\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \epsilon_{1_{[d]}}\left[X_{[d]}\right] + \delta\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \left(\mathbb{I}\left[X_{[d]}\right] - \epsilon_{1_{[d]}}\left[X_{[d]}\right]\right) .$$

This is the computation circuit of the $d$-ary logical conjunction

$$\wedge^d : [2]^d \to [2] \quad , \quad \wedge^d(x_{[d]}) = \begin{cases} 1 & \text{if} \quad x_{[d]} = 1_{[d]} \\ 0 & \text{else} \end{cases} .$$

Special instances are the CNOT for $d = 1$ (see Example 12.5) and the Toffoli gate for $d = 2$.

### 12.3.2 Circuit decomposition sparsity

We develop now a quantum equivalent to the circuit decompositions of basis encodings (see Sect. 7.3) sis now a composition of circuits property, as stated in the next lemma. Similar to the introduction of hidden variables, we here utilize further working qubits to realize decompositions. To highlight the similarity, we call these sparsity principle circuit decomposition sparsity.

**Lemma 12.8** *We have for functions* $q : \bigtimes_{k\in[d]}[2] \to \bigtimes_{\ell\in[p]}[2]$, $g : \bigtimes_{\ell\in[p]}[2] \to \bigtimes_{s\in[r]}[2]$ *(see Figure* **??***)*

$$C^{g\circ q}\left[Y^{\text{in}}_{g\circ q}, Y^{\text{out}}_{g\circ q}, X_{[d]}\right] \otimes \epsilon_0\left[Y^{\text{out}}_q\right]$$
$$= \left\langle \epsilon_0\left[Y^{\text{in}}_q\right], C^q\left[Y^{\text{in}}_q, Y^{\text{mid}}_q, X_{[d]}\right], C^g\left[Y_{\text{in},g\circ q}, Y^{\text{out}}_{g\circ q}, Y^{\text{mid}}_q\right], C^q\left[Y^{\text{mid}}_q, Y^{\text{out}}_q, X_{[d]}\right]\right\rangle_{\left[Y^{\text{in}}_{g\circ q}, Y^{\text{out}}_{g\circ q}, X_{[d]}, Y^{\text{out}}_q\right]}.$$

*Here each copy of $Y_{g\circ q}$ denotes a tuple of $r$ boolean variables and $Y_q$ of $p$ boolean variables.*

*Proof.* We show the claim based on basis calculus and sketch the steps in Figure 12.8. In the first equation we use Lem. 12.7 and represent each computation circuit by a contraction of two basis encodings. In the second equation we use the identity

$$\left\langle \beta^{\oplus}\left[Y^{\text{out}}, Y^{\text{in}}, Y_q\right], \epsilon_0\left[Y^{\text{in}}\right]\right\rangle_{\left[Y^{\text{out}}, Y_q\right]} = \delta\left[Y^{\text{out}}, Y_q\right].$$

Now in the third equation we use that the contraction of basis encodings is the basis encoding of the composition function, and that $q \oplus q$ is the vanishing function, which basis encoding is a tensor product with $\epsilon_0\left[Y^{\text{out}}_q\right]$. We use we Lem. 12.7 again in the fourth equation to arrive at the claim. □

When having a syntactical decomposition of a propositional formula, we can iteratively apply the computation circuit decomposition theorem and prepare each connective by a circuit. We can decompose any propositional formula into logical connectives and prepare to each a computation circuit (exploiting further sparsity mechanisms). This works, when the target qubit of one connective is used as a value qubit of another.

This entailment of the working qubit with the distributed qubits can be resolved by an un-computation circuit, which is the adjoint of the computation circuit for the working qubits (see Figure 12.8). Note that while this is necessary in some applications, it is not necessary for the quantum rejection sampling presented here.

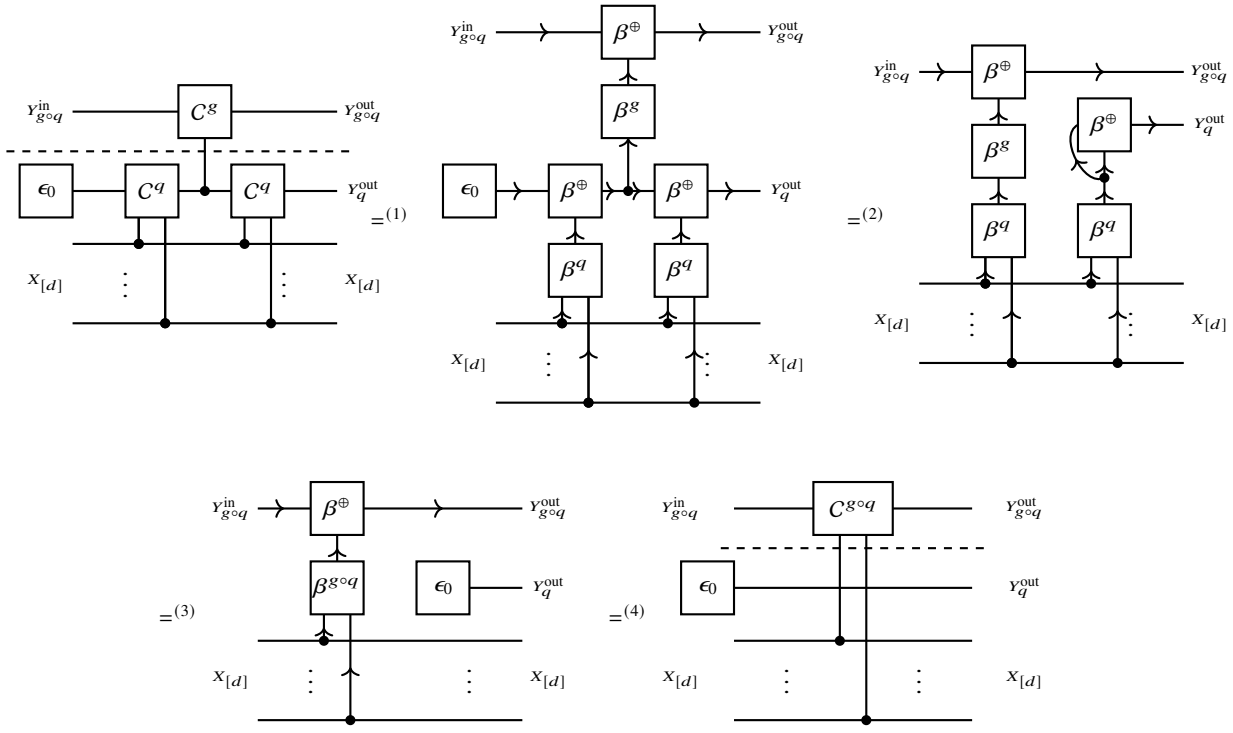



**Fig. 12.8** Proof of Lem. 12.8 based on basis calculus.

*Example 12.7* Let us reconsider the $d$-ary controlled NOT gate from Example 12.6. The $d$-ary logical conjunction is decomposed as a sequence of 2-ary logical conjunctions. The computation circuit of each 2-ary conjunction is a Toffoli gate applied to an input qubit and a worker qubit. Using Decomposition sparsity we therefore find the standard decomposition of $d$-ary controlled NOT gates into Toffoli gates.

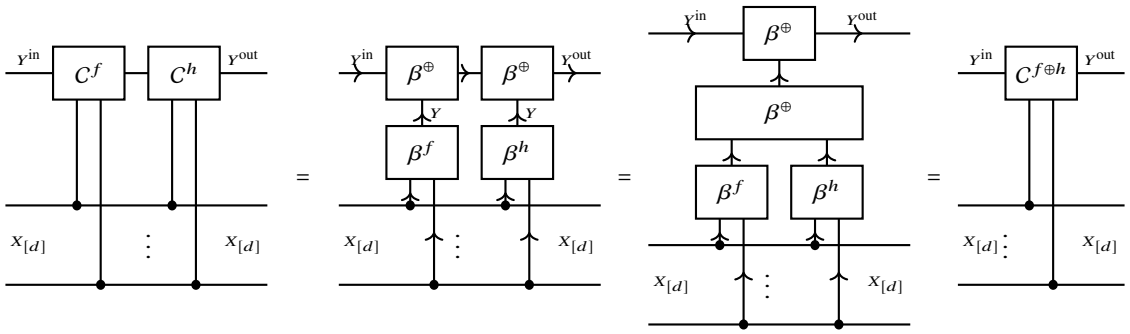



**Fig. 12.9** Equivalence of the composition of computation circuits with the computation circuit of their mod2 sum (Lem. 12.9).

### 12.3.3 Circuit polynomial sparsity

While we have introduced additional auxiliary qubits to represent decomposable functions, we now investigate concatenations of computation circuits, which are sharing the same target qubit. As we will show, these are related to summations mod 2 (which are contractions in the semiring $\mathbb{F}_2$, see Sect. 8.1.2.5), and can be constructed using Reed-Muller expansions (see Sect. 10.4). To highlight the similarity to polynomial sparsity, which has been developed in Chapter 10, we call this sparsity principle circuit polynomial sparsity.

**Lemma 12.9** *Let* $\{f_\ell\left[X_{[d]}\right] : \ell \in [p]\}$ *be a set of propositional formulas, and let*

$$h\left[X_{[d]}\right] = \langle \sigma^f\left[X_{[d]}, L\right]\rangle^{\mathbb{F}_2}_{[X_{[d]}]}$$

*be their summation* mod 2*, which can be expressed using the selection encoding* $\sigma^f\left[X_{[d]}, L\right]$ *of the formula set. Then we have that*

$$\left\langle \{C^{f_\ell}\left[Y^\ell, Y^{\ell+1}, X_{[d]}\right] : \ell \in [p]\}\right\rangle_{\left[X_{[d]}\right]} = C^h\left[Y^0, Y^p, X_{[d]}\right] .$$

*Proof.* We first show the claim for $p = 2$, the sum of two formulas $f_0$ and $f_1$. We decompose in Figure 12.9 each computation circuit into a contraction of the functions basis encoding with the basis encoding of $\oplus$, the summation mod 2. By associativity of $\oplus$, we can rewrite the contraction using the basis encoding of $f_0 \oplus f_1$.

We then show the generic statement by induction over $p$. The statement is trivial for $p = 1$. Let us assume that the claim holds for $p - 1$ formulas, and let there be $p$ formulas. By assumption, the concatenation of the first $p - 1$ computation circuits is the computation circuit of

$$\bigoplus_{k\in[p]} f_k$$

Applying the statement for $p = 2$, we conclude that the concatenation of all $p$ computation circuits is the computation circuit of

$$h = \bigoplus_{k\in[p]} f_{\bar{k}} = \langle \sigma^f\left[X_{[d]}, L\right]\rangle^{\mathbb{F}_2}_{[X_{[d]}]} .$$

□

Recall that while the quantum circuit itself is defined in the complex sum-product semiring, we use the semiring $\mathbb{F}_2$ for function decomposition. The $\mathrm{CP}^{\mathbb{F}_2}$ decomposition is closely related to the Algebraic Normal Form of a Boolean polynomial. In the latter, we need to restrict to leg vectors $\mathbb{I}\left[X\right]$ and $\epsilon_1\left[X\right]$, that is forbid the usage of $\epsilon_0\left[X\right]$ vectors. The Algebraic Normal Form is also called positive-polarity Reed-Muller expansion. Allowing for the $\epsilon_0\left[X\right]$ vector in the $\mathrm{CP}^{\mathbb{F}_2}$ decomposition corresponds with mixed-polarity Reed-Muller (MPRM) expansions.

Let us now determine computation circuits for MPRM terms to get an encoding for arbitrary formulas in MPRM expansions.

**Lemma 12.10** *A MPRM term $f_{(\mathcal{U},y_\mathcal{U})}$ has the computation circuit (see Figure 12.10)*

$$\begin{aligned}
&\left\langle \mathcal{V}^{f_{(\mathcal{U},y_\mathcal{U})}}\left[Y^{\text{in}}, Y^{\text{out}}, X_{[d]}\right], \delta\left[X_{[d]}, X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]\right\rangle_{\left[Y^{\text{in}},Y^{\text{out}},X^{\text{in}}_{[d]},X^{\text{out}}_{[d]}\right]} \\
&\quad = \left\langle \{\text{MCNOT}^{|\mathcal{U}|}[Y^{\text{in}}, Y^{\text{out}}, X_\mathcal{U}]\} \cup \left( \bigcup_{k\in\mathcal{U},\, y_k=0} \{\sigma_1\left[X^{\text{in}}_k, X_k\right], \sigma_1\left[X_k, X^{\text{out}}_k\right]\}\right)\right. \\
&\qquad \left. \cup \left( \bigcup_{k\in\mathcal{U},\, y_k=1} \{\delta\left[X_k, X^{\text{in}}_k, X^{\text{out}}_k\right]\}\right)\right\rangle_{\left[Y^{\text{in}},Y^{\text{out}},X^{\text{in}}_{[d]},X^{\text{out}}_{[d]}\right]}.
\end{aligned}$$

*The computation circuit of a formula $f$ with MPRM expansion $\mathcal{M}$ is a concatenation of the computation circuits to $\mathcal{U} \in \mathcal{M}$.*

*Proof.* For any $x^{\text{in}}_{[d]}, x^{\text{out}}_{[d]}$ the slices on both sides are vanishing, if $x^{\text{in}}_{[d]} \neq x^{\text{out}}_{[d]}$. If $x^{\text{in}}_{[d]} = x^{\text{out}}_{[d]}$ we have

$$\begin{aligned}
&\left\langle \{\text{MCNOT}^{|\mathcal{U}|}[Y^{\text{in}}, Y^{\text{out}}, X_\mathcal{U}]\} \cup \left( \bigcup_{k\in\mathcal{U},\, y_k=0} \{\sigma_1\left[X^{\text{in}}_k, X_k\right], \sigma_1\left[X_k, X^{\text{out}}_k\right]\}\right)\right. \\
&\qquad \left. \cup \left( \bigcup_{k\in\mathcal{U},\, y_k=1} \{\delta\left[X_k, X^{\text{in}}_k, X^{\text{out}}_k\right]\}\right)\right\rangle_{\left[Y^{\text{in}},Y^{\text{out}},X^{\text{in}}_{[d]}=x^{\text{in}}_{[d]},X^{\text{out}}_{[d]}=x^{\text{out}}_{[d]}\right]} \\
&= \left\langle \{\text{MCNOT}^{|\mathcal{U}|}[Y^{\text{in}}, Y^{\text{out}}, X_\mathcal{U}]\} \cup \{\epsilon_{(1-x^{\text{in}}_k)}\left[X_k\right] : k \in \mathcal{U},\, y_k = 0\} \cup \{\epsilon_{x^{\text{in}}_k}\left[X_k\right] : k \in \mathcal{U},\, y_k = 1\}\right\rangle_{\left[Y^{\text{in}},Y^{\text{out}}\right]} \\
&= \begin{cases} \text{NOT}[Y^{\text{in}}, Y^{\text{out}}] & \text{if} \quad f_{(\mathcal{U},y_\mathcal{U})}(x^{\text{in}}_{[d]}) = 1 \\ \delta\left[Y^{\text{in}}, Y^{\text{out}}\right] & \text{if} \quad f_{(\mathcal{U},y_\mathcal{U})}(x^{\text{in}}_{[d]}) = 0 \end{cases}.
\end{aligned}$$

Thus, the right contraction coincides for all slices on the variables $X^{\text{in}}_{[d]}$ and $X^{\text{out}}_{[d]}$ with the oracle of $f_{(\mathcal{U},y_\mathcal{U})}$. □

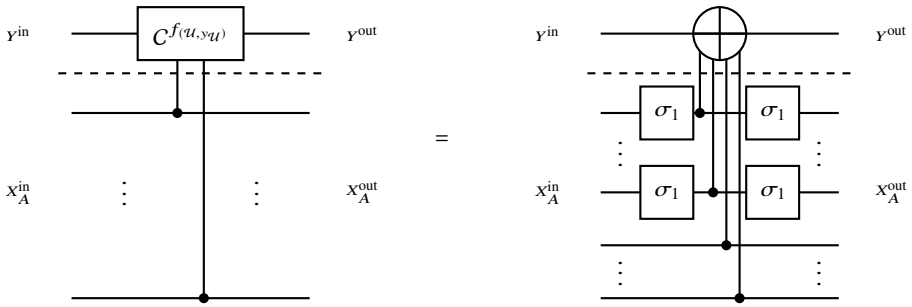



**Fig. 12.10** Computation circuit to a MPRM term $f_{(\mathcal{U},y_\mathcal{U})}$, consisting in a multiple controlled NOT gate and Pauli-X gates for legs with $y_k = 0$.

Combining Lem. 12.10 and Lem. 12.9 we can create the computation circuit of any function based on the $\text{CP}^{\mathbb{F}_2}$ decomposition, which is equivalent to Reed-Muller expansions (see Sect. 10.4).

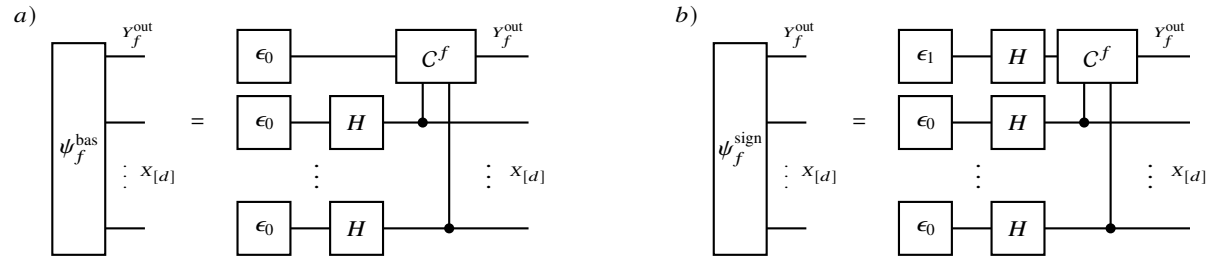



**Fig. 12.11** Construction of the basis encoding state (a) and the sign encoding state (b) using the computation circuit of a function.

### 12.3.4 Function Encoding

We investigate how Boolean functions can be encoded in quantum states using computation circuits. Depending on the start state, we introduce two function encoding states.

*Remark 12.1 (Basis and Amplitude Encodings in Quantum Computation [78] and* tnreason*)* Basis Encoding in Quantum Computation refers to the representation of classical $n$ bit strings by $n$ qubit basis states, and is called one-hot encoding in tnreason. The Basis Encoding scheme in tnreason goes beyond this scheme and also encodes subsets by sums of one-hot encodings to the members of the set. In this way, relations and functions are represented by boolean tensors and contraction of them is refered to as Basis calculus.

Amplitude Encoding in Quantum Computation refers to the storage of complex numbers in the amplitudes of quantum states. The pendant in tnreason is the Coordinate Encoding scheme, where real numbers are stored in the coordinates of real-valued tensors. Compared to Amplitude Encoding, Coordinate Encoding does not have the normalization constraint of quantum states. The Amplitude Encoding of the square root of a probability distribution is sometimes called q-sample.

#### 12.3.4.1 Basis encoding

**Definition 12.5** The basis encoding of a boolean function $f$ is the state

$$\psi_f^{\text{bas}}\left[X_{[d]}, Y\right] = \sqrt{\frac{1}{2^d}} \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[2]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{f(x_{[d]})}\left[Y\right] .$$

**Lemma 12.11** *When applying the computation circuit on the initial state*

$$\sqrt{\frac{1}{2^d}} \mathbb{I}\left[X_{[d]}\right] \otimes \epsilon_0\left[Y\right]$$

*we get the basis encoding state (see Figure 12.11a).*

*Proof.* Since the Pauli-X turns $\epsilon_0$ into $\epsilon_1$, and is applied exactly when $f(x_{[d]}) = 1$. □

**Corollary 12.1** *Using Lem. 12.7 we have*

$$\psi_f^{\text{bas}}\left[X_{[d]}, Y\right] = \sqrt{\frac{1}{2^d}} \beta^f\left[X_{[d]}, Y\right]$$

The initial state can be prepared by applying Hadamard gates on all distributed variable qubits and preparing the head variable qubit in the ground state. The overlap of two basis encoding states (which share the same head variable) is proportional to the contraction of the corresponding formulas.

**Lemma 12.12** *Let $f, \tilde{f}$ be two Boolean formulas on the states of $X_{[d]}$. Then we have*

$$\left\langle \psi_f^{\text{bas}}\left[X_{[d]}, Y\right], \psi_h^{\text{bas}}\left[X_{[d]}, Y\right] \right\rangle_{[\varnothing]} = \frac{1}{2^d} \left\langle f\left[X_{[d]}\right], \tilde{f}\left[X_{[d]}\right] \right\rangle_{[\varnothing]} .$$

*Proof.* We have

$$\begin{aligned}\left\langle \psi_f^{\text{bas}}\left[X_{[d]},Y\right],\psi_h^{\text{bas}}\left[X_{[d]},Y\right]\right\rangle_{[\varnothing]} &= \frac{1}{2^d}\sum_{x_{[d]}\in[2]^d}\left\langle \epsilon_{f\left[X_{[d]}=x_{[d]}\right]}\left[Y\right],\epsilon_{\tilde{f}\left[X_{[d]}=x_{[d]}\right]}\left[Y\right]\right\rangle_{[\varnothing]} \\ &= \frac{1}{2^d}\left\langle f\left[X_{[d]}\right],\tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .\end{aligned}$$

□

#### 12.3.4.2 Sign encoding

**Definition 12.6** The sign encoding of a boolean function $f$ is the state

$$\psi_f^{\text{sign}}\left[X_{[d]},Y\right]=\left(\sqrt{\frac{1}{2^d}}\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[2]}(-1)^{f(x_{[d]})}\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]\right)\otimes\sqrt{\frac{1}{2}}\left(\epsilon_0\left[Y\right]-\epsilon_1\left[Y\right]\right) .$$

Sign encodings can be prepared by the phase kickback trick, as we show next.

**Lemma 12.13 (Phase kickback trick)** *Applying the computation circuit* $C^f\left[Y^{\text{in}},Y^{\text{out}},X_{[d]}\right]$ *on a state*

$$\psi\left[X_{[d]}\right]\otimes\epsilon^-\left[A^{\text{in}}\right]$$

*we get*

$$\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}(-1)^{f(x_{[d]})}\psi\left[X_{[d]}=x_{[d]}\right]\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]\otimes\epsilon^-\left[A^{\text{out}}\right] .$$

*Proof.* We get a disentangled ancilla qubit, because $\epsilon^-\left[A^{\text{in}}\right]$ is an eigenvector of the identity and the Pauli-X. For the identity, which is applied when $f(x_{[d]})=0$, the eigenvalue is 1. For the Pauli-X, which is applied when $f(x_{[d]})=0$, the eigenvalue is $(-1)$. □

We get the sign encoding with the phase kickback trick, when we initialize with the q-sample of the uniform distribution.

**Corollary 12.2** *When applying the computation circuit on the initial state*

$$\sqrt{\frac{1}{2^d}}\mathbb{I}\left[X_{[d]}\right]\otimes\sqrt{\frac{1}{2}}\left(\epsilon_0\left[Y\right]-\epsilon_1\left[Y\right]\right)$$

*we get the sign encoding state (see Figure 12.11b). Note that the initial state is a Hadamard gate acting on the state* $\epsilon_1\left[Y\right]$.

The sign encoding is also called the phase encoding. One can ask, whether we can use other ancilla states and get different phase kickbacks while keeping the ancilla disentangled. However, since we construct here computation circuit we are restricted to the eigenvectors of the Pauli-X gate, which is the Hadamard basis

$$\epsilon^-\left[Y\right]=\sqrt{\frac{1}{2}}\left(\epsilon_0\left[Y\right]-\epsilon_1\left[Y\right]\right)\quad,\quad\epsilon^+\left[Y\right]=\sqrt{\frac{1}{2}}\left(\epsilon_0\left[Y\right]+\epsilon_1\left[Y\right]\right)$$

with eigenvalues $(-1)$ and 0.

The initial state can be prepared by applying Hadamard gates on all distributed variable qubits and preparing the head variable qubit by applying a Hadamard gate on the first basis state, i.e.

$$\frac{1}{\sqrt{2}}\left(\epsilon_1\left[Y_{\text{out}}\right]-\epsilon_0\left[Y_{\text{out}}\right]\right)=H\left[Y_{\text{out}},Y_{\text{in}}\right]\epsilon_0\left[Y_{\text{in}}\right] ,$$

**Lemma 12.14** *Let $f, \tilde{f}$ be two Boolean formulas on the states of $X_{[d]}$. Then we have*

$$\left\langle \psi_f^{\text{sign}}\left[X_{[d]}, Y\right], \psi_h^{\text{sign}}\left[X_{[d]}, Y\right]\right\rangle_{[\varnothing]} = \frac{1}{2^d}\left(\left\langle f\left[X_{[d]}\right], \tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle f\left[X_{[d]}\right], \neg\tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\right).$$

*Proof.* We have

$$\begin{aligned}\left\langle \psi_f^{\text{sign}}\left[X_{[d]}, Y\right], \psi_h^{\text{sign}}\left[X_{[d]}, Y\right]\right\rangle_{[\varnothing]} &= \frac{1}{2^d}\sum_{x_{[d]}\in[2]^d}(-1)^{f\left[X_{[d]}=x_{[d]}\right]+\tilde{f}\left[X_{[d]}=x_{[d]}\right]} \\ &= \frac{1}{2^d}\sum_{x_{[d]}\in[2]^d : f\left[X_{[d]}=x_{[d]}\right]=\tilde{f}\left[X_{[d]}=x_{[d]}\right]} 1 + \sum_{x_{[d]}\in[2]^d : f\left[X_{[d]}=x_{[d]}\right]\neq\tilde{f}\left[X_{[d]}=x_{[d]}\right]} (-1) \\ &= \frac{1}{2^d}\left(\left\langle f\left[X_{[d]}\right], \tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle f\left[X_{[d]}\right], \neg\tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\right).\end{aligned}$$

□

Limitations of sign encodings:

- The prepared state $\psi_f^{\text{sign}}$ is distinguished from the state $\psi_{\neg f}^{\text{sign}}$ only by a non-observable phase factor

$$\psi_f^{\text{sign}} = (-1)\cdot\psi_{\neg f}^{\text{sign}}$$

- With this construction the statistic qubit is disentangled with the distributed qubits (in contrast to basis encoding states). Further manipulation of the statistic qubits alone (as we do with activation circuits) will not retrieve any information about the encoded function.
- When doing the sign encoding procedure for multiple formulas, we get a disentangled state of the head variables with the distributed variables encoded in the sign encoding state of $\bigoplus_{f\in\mathcal{F}} f$ (where by $\bigoplus$ we denote the mod2 sum of boolean functions).

## 12.4 Quantum algorithms

We now review standard Quantum algorithms in the tensor network formalism.

### 12.4.1 Quantum Fourier transform

Recall the tensor network decomposition of the Fourier transform developed in Sect. 9.2.

When $V[X_{[d]}]$ is a quantum state, the Fourier transform can be understood as a unitary evolving the state. Since both the controlled $U$ tensor and the Hadamard $H$ are unitary, each of them can be implemented by a sequence of gates. For sparse implementation one can exploit that $U$ is a tensor product of matrices (i.e. single qubit gates).

We introduce the gates

$$R^k[X^{\text{in}}, X^{\text{out}}] = \begin{pmatrix}1 & 0\\ 0 & \exp\left[2\pi\mathrm{i}\frac{1}{2^k}\right]\end{pmatrix}\left[X^{\text{in}}, X^{\text{out}}\right].$$

The unitary $U^d$ in the tensor network decomposition of the fourier transform (see Sect. 9.2.2) is then the tensor product

$$U^d[X_{\text{in}}^{[d]}, X_{\text{out}}^{[d]}] = \bigotimes_{k\in[d]} R^{k+2}[X_{\text{in}}^k, X_{\text{out}}^k].$$

Controlling $U^d$ by $X_d$ is equivalent to controlling each $R^k$ gate.

To realize the QFT we need $d$ single-qubit Hadamard gates and $\sum_{k\in[d]} k = \frac{d\cdot(d-1)}{2}$ controlled diagonal single-qubit gates (to realize the $U^k$ transforms). The gate complexity is therefore $O(d^2)$.

#### 12.4.1.1 Walsh-Hadamard transform

In case of $m_k = 2$, the Fourier transform is known to be the Walsh-Hadamard tranform. The transform is performed by applying Hadamard gates on each variable. This is equivalent with contraction of the tensor

$$H^d\left[X_{[d],\mathrm{in}}, X_{[d],\mathrm{out}}\right] = \bigotimes_{k\in[d]} H\left[X_k^{\mathrm{in}}, X_k^{\mathrm{out}}\right] ,$$

which coordinates are

$$H^d\left[X_{[d],\mathrm{in}} = x_{[d],\mathrm{in}}, X_{[d],\mathrm{out}} = x_{[d],\mathrm{out}}\right] = \sqrt{\frac{1}{2^d}}\,(-1)^{\sum_{k\in[d]} x_{k,\mathrm{in}}\cdot x_{k,\mathrm{out}}} .$$

We have

$$\left\langle \epsilon_0\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} = \sqrt{\frac{1}{2}}\mathbb{I}\left[X^{\mathrm{out}}\right]$$
$$\left\langle \epsilon_1\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} = \sqrt{\frac{1}{2}}\begin{pmatrix}1\\-1\end{pmatrix}\left[X^{\mathrm{out}}\right]$$

and conversely

$$\left\langle \sqrt{\frac{1}{2}}\mathbb{I}\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} = \epsilon_0\left[X^{\mathrm{out}}\right]$$
$$\left\langle \sqrt{\frac{1}{2}}\begin{pmatrix}1\\-1\end{pmatrix}\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} = \epsilon_1\left[X^{\mathrm{out}}\right] .$$

If and only if the function is constant, then the transformed state is the ground state.

Let us present two known algorithms as the measurement of states prepared by computation circuits.

*Example 12.8 (Deutsch-Jozsa Test)* We measure the probability of the ground state of the sign encoding after a Walsh-Hadamard transform. The probability of measuring the ground state after a Walsh-Hadamard transform of the distributed qubits is then

$$\frac{1}{2^d}\cdot\left\langle\left|\left\langle\psi_f^{\mathrm{sign}}\left[X_{[d]}, Y\right]\right\rangle_{[Y]}\right|^2\right\rangle_{[\varnothing]} = \frac{1}{2^{2\cdot d}}\left(\langle f\rangle_{[\varnothing]} - \langle\neg f\rangle_{[\varnothing]}\right)^2 .$$

Thus, if and only if the probability of the ground state is 1, we have that the formula is constant, that is $f \in \{\top, \bot\}$. If and only if the probability of the ground state is 0, we have that the formula is balanced, that is $\langle f\rangle_{[\varnothing]} = \langle\neg f\rangle_{[\varnothing]}$. When its known, that the formula is either constant or balanced, we can decide which one it is by measuring the probability of the ground state. This is the Deutsch-Josza algorithm [23].

Note that when allowing for an exponentially small error the decision can be efficiently performed by a classical algorithm. When drawing $n$ samples and deciding on $\mu = 0.5$ if and only if in the samples a 0 and a 1 is observed, we have an error probability of 0 when $\mu \in \{0, 1\}$ and of

$$\mathbb{P}^{E|\mu=0.5} = 2\cdot\frac{1}{2^n} .$$

The quantum advantage is thus bound to the demand of an error-free decision, for which a generic classical algorithm would iterate through all states and is therefore exponentially slower than the quantum alternative.

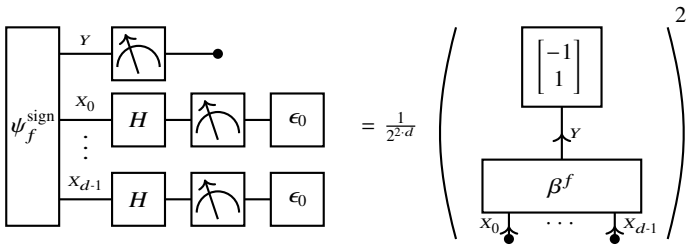



**Fig. 12.12** Measurement setup for the Deutsch-Jozsa algorithm on the formula $f$.

Another generalizations would be period detection. Deutsch-Jozsa can be generalized to estimating the moments of a distribution given its q-sample. Note, that we have to demand a q-sample (i.e. vanishing phase tensor), otherwise the measurement is affected by the phase tensor and the Deutsch-Jozsa test is effectively done wrt another function. If for example the coordinates of the phase tensor are multiples of $\pi$, and the phases are thus $+1/-1$, then the test is done wrt the function $f \oplus (\frac{1}{\pi}\phi)$ instead of $f$.

*Example 12.9 (Inversion Test)* Concatenating the computation circuits to two formulas $f$ and $h$ prepares the state

$$\psi^{\text{bas}}_{f\oplus h}\left[X_{[d]}, Y\right]$$

The probability of measuring the ground state after a Walsh-Hadamard transform of the distributed qubits (see Figure 12.13) is then

$$\begin{aligned}\frac{1}{2^d}\left|\left\langle\psi^{\text{bas}}_{f\oplus h}\left[X_{[d]}, Y\right]\right\rangle_{[Y]}\right|^2 &= \frac{1}{2^{2\cdot d}}\left[\langle f\oplus h\rangle_{[\varnothing]}\left\langle\neg(f\oplus h)\right\rangle_{[\varnothing]}\right] \\ &= \frac{1}{2^{2\cdot d}}\begin{bmatrix}\#\left\{x_{[d]} \in \bigtimes_{k\in[d]}[2] \,:\, f\left[X_{[d]} = x_{[d]}\right] = h\left[X_{[d]} = x_{[d]}\right]\right\} \\ \#\left\{x_{[d]} \in \bigtimes_{k\in[d]}[2] \,:\, f\left[X_{[d]} = x_{[d]}\right] \neq h\left[X_{[d]} = x_{[d]}\right]\right\}\end{bmatrix}.\end{aligned}$$

The second equation holds since $\neg(f \oplus h)$ is equivalent with the biconditional $f \Leftrightarrow h$.

This is in more generality the inversion test of measuring the overlap of $\psi^{\text{bas}}_f$ with $\psi^{\text{bas}}_h$ (see [78]). Here we used that the computation circuit is self-adjoint and understand the concatenation as the unitaries preparing $\psi^{\text{bas}}_f$ and the adjoint (together with the Walsh-Hadamard transform) of $\psi^{\text{bas}}_h$.

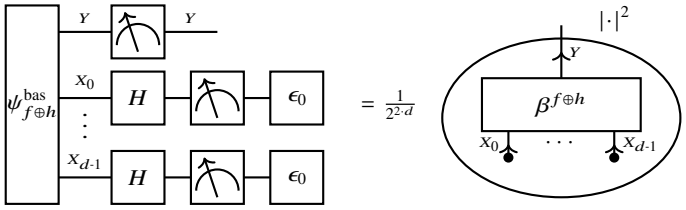



**Fig. 12.13** Measurement setup for the inversion test on the formulas $f$ and $h$. We measure the probability of the statistic qubit $Y$, when the distributed qubits $X_{[d]}$ are in the ground state after a Walsh-Hadamard transform. This is equal to a scaled and squared contraction of the basis encoding to the formula $f \oplus h$.

Quantum Circuits such as the SWAP test and the Hadamard test ([78, Section 3.6.1]) can be used to measure overlaps of quantum states, which are the squared absolutes of contractions of two state tensors.

- When we have preparation schemes for two tensors, we can control them with a common ancilla qubit and measure their contraction by a Hadamard test (alternatively, using the SWAP test and state preparation in two registers).
- Can we extend these schemes to contractions of more general tensor networks?

While the preparation of contracted basis encodings is done efficiently, accessing the prepared state by measurements is a bottleneck. This is a typical *quantum state tomography* problem [70]. Sparsity of the contracted state can be exploited to probabilistically guarantee precise estimates [33]. However, when deciding whether $f$ is satisfiable, based on the prepared contraction $\sqrt{\left\langle \beta^f \left[Y, X_{[d]}\right]\right\rangle_{[Y|\varnothing]}}$, we have an exponentially small error tolerance. Moreover, the applied sample mean is the UMVUE (Uniformly Minimum Variance Unbiased Estimator, see [12]), and can thus not be improved by other unbiased estimators.

#### 12.4.1.2 Period detection

QFT is a typical component in Quantum algorithms: QFT can be done efficiently with $\wr(d^2)$ controlled single-qubit gates. If the Fourier transform is a basis vector, a single measurement is sufficient to determine it. We now investigate how this can be used in the detection of periods.

Based on the support of the Fourier transform, one can derive the periods of a function. Periods are indices $(Z_0, \ldots, Z_{d-1})$ such that

$$\tau\left[X_{[d]} = x_{[d]}\right] = \tau\left[X_{[d]} = x_{[d]} \oplus z_{[d]}\right]$$

where $\oplus$ is the position-wise sum mod $m_k$.

Each supported index of the Fourier transform corresponds with periods in the function. The periods of the functions are thus at least the intersection of those (when assuming more structure one might have more periods).

In case of the Walsh-Hadamard transform (i.e. $m_k = 2$), we can easily determine the periods corresponding to each basis vector. The inverse Fourier transform of $\epsilon_{\ell_{[d]}}\left[L_{[d]}\right]$ is the tensor

$$\left(\bigotimes_{k\in[d]\,:\,\ell_k=0} \epsilon^+\left[X_k\right]\right) \otimes \left(\bigotimes_{k\in[d]\,:\,\ell_k=1} \epsilon^-\left[X_k\right]\right).$$

For each $\epsilon^+\left[X_k\right]$ we have a period generator of $\epsilon_k$. For each pair of $\epsilon^-\left[X_k\right] \otimes \epsilon^-\left[X_{\tilde{k}}\right]$ we have a period generator of $\epsilon_k + \epsilon_{\tilde{k}}$.

One example is the Deutsch-Jozsa test, where the transformed is the $\epsilon_0\left[L_{[d]}\right]$ state, if and only if the function is constant. In this case, any vector is a period, since the group itself is generated by the period generators.

### 12.4.2 Quantum Phase Estimation

When having a circuit implementing a unitary, the phase of the eigenvalues can be estimated using the Quantum Phase Estimation routine [43, 15].

Let $U\left[X_{\text{in}}^{[d]}, X_{\text{out}}^{[d]}\right]$ be a unitary then the absolute of any eigenvalue is 1 and the eigenvalue is expressible by

$$\exp\left[2\pi\mathrm{i}\cdot\phi\right]$$

for some $\phi \in [0, 1]$. Let $\phi = 0.\phi_0 \cdots \phi_{n-1}$ be a 2-adic representation of $\phi$, that is

$$\phi = \sum_{k\in[n]} 2^{-k-1}\phi_k\,.$$

In the Quantum Phase Estimation routine the Fourier transform of $\epsilon_{\phi_0,\ldots,\phi_{n-1}}\left[L_{[n]}\right]$

$$\sqrt{\frac{1}{2^n}} \cdot \bigotimes_{k\in[n]} \left(\epsilon_0\left[L_0\right] + \exp\left[2\pi\mathrm{i}\cdot 0.\phi_{n-k-1}\cdots\phi_{n-1}\right]\right)$$

is prepared as explained below. The inverse Fourier transform on the register $L_{[n]}$ results then in the state

$$\epsilon_{\phi_0,\ldots,\phi_{n-1}}\left[L_{[n]}\right]\otimes\psi\left[X_{[d]}\right]$$

and $\phi_k$ for $k\in[n]$ can be retrieved by a single computational basis measurement of the register $L_{[n]}$.

To prepare the Fourier transform of $\epsilon_{\phi_0,\ldots,\phi_{n-1}}\left[X_{[d]}\right]$, we start with the state

$$\epsilon^{+}\left[L\right]\otimes\psi\left[X_{[d]}\right].$$

When applying $U\left[X_{\text{in}}^{[d]},X_{\text{out}}^{[d]}\right]$ $2^k$ times controlled on $L$ we get the state

$$\sqrt{\frac{1}{2}}\left(\epsilon_0\left[L_0\right]+\exp\left[2\pi\mathrm{i}\cdot 0.\phi_{n-k-1}\cdots\phi_{n-1}\right]\right)\otimes\psi\left[X_{[d]}\right]$$

Now we apply for each $k\in[n]$ the unitary $U\left[X_{\text{in}}^{[d]},X_{\text{out}}^{[d]}\right]$ $2^k$ times controlled on $L_k$ and get

$$\sqrt{\frac{1}{2^n}}\mathbb{I}\left[L_{[n]}\right]\otimes\psi\left[X_{[d]}\right].$$

### 12.4.3 Grover operator

We derive quantum inference schemes based on the Grover operator, which we want to first introduce in full generality.

#### 12.4.3.1 Definition

Let us assume we have two orthogonal quantum states $\psi^{\parallel}\left[X_{[d]}\right],\psi^{\perp}\left[X_{[d]}\right]$ spanning a subspace $V=\text{span}\left(\psi^{\parallel}\left[X_{[d]}\right],\psi^{\parallel}\left[X_{[d]}\right]\right)$ and let us choose another state

$$\psi^{0}\left[X_{[d]}\right]\in V.$$

Then there is $\alpha\in[0,4\pi)$ with

$$\psi^{0}\left[X_{[d]}\right]=\sin\left(\frac{\alpha}{2}\right)\psi^{\parallel}\left[X_{[d]}\right]+\cos\left(\frac{\alpha}{2}\right)\psi^{\perp}\left[X_{[d]}\right].$$

Consider the reflection about the states $\psi^{\perp}$ and $\psi^{0}$ by

$$\begin{aligned}\mathcal{S}^{\perp}\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{out}}\right]&=2\cdot\psi^{\perp}\left[X_{[d]}^{\text{in}}\right]\otimes\psi^{\perp}\left[X_{[d]}^{\text{out}}\right]-\delta\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{out}}\right]\\ \mathcal{S}^{0}\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{out}}\right]&=2\cdot\psi^{0}\left[X_{[d]}^{\text{in}}\right]\otimes\psi^{0}\left[X_{[d]}^{\text{out}}\right]-\delta\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{out}}\right],\end{aligned}$$

and their concatenation

$$G\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{out}}\right]=\left\langle\mathcal{S}^{\perp}\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{mid}}\right],\mathcal{S}^{0}\left[X_{[d]}^{\text{mid}},X_{[d]}^{\text{out}}\right]\right\rangle_{\left[X_{[d]}^{\text{in}},X_{[d]}^{\text{out}}\right]}$$

which is called the *Grover operator*.

#### 12.4.3.2 Action on projected quantum states

The Grover operator leaves any quantum state $\psi$ invariant, which is orthogonal to $\psi^{\parallel}$ and $\psi^{\perp}$ (such states get a factor $(-1)$ by both reflection operations). The projection of the quantum state onto $V$ gets rotated by the Grover operator (see

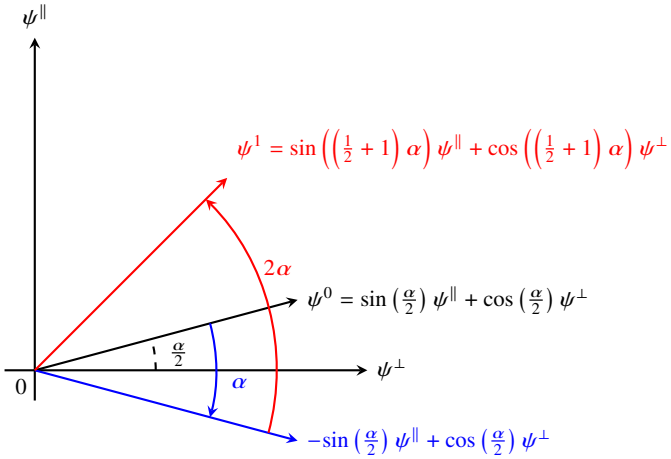



**Fig. 12.14** Grover operator as a rotation in span $(\psi^{\parallel}, \psi^{\perp})$.

Figure 12.14). To be more precise, we denote the projection by $(\mathcal{P}\psi)[L]$ which has the coordinates

$$(\mathcal{P}_V\psi)\,[L=0] = \left\langle \left(\psi\left[X_{[d]}\right]\right)^{\dagger}, \psi^{\parallel}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$
$$(\mathcal{P}_V\psi)\,[L=1] = \left\langle \left(\psi\left[X_{[d]}\right]\right)^{\dagger}, \psi^{\parallel}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .$$

We define a matrix representation of the Grover operator as

$$G^{\mathrm{red}}\left[L^{\mathrm{out}}, L^{\mathrm{in}}\right] = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix}\left[L^{\mathrm{out}}, L^{\mathrm{in}}\right]$$

and it holds for any quantum state $\psi\left[X_{[d]}\right]$ that

$$(\mathcal{P}_V G\psi)\left[L^{\mathrm{out}}\right] = \left\langle G^{\mathrm{red}}\left[L^{\mathrm{out}}, L^{\mathrm{in}}\right], (\mathcal{P}_V\psi)\left[L^{\mathrm{in}}\right]\right\rangle_{[\mathrm{out}]} .$$

The Grover operator therefore has the eigenvalues:

- $\exp[\pm i\alpha]$ with the eigen spaces spanned by $\psi^{\parallel} \pm i\psi^{\perp}$
- 1 with the eigen space by the orthogonal complement of $V$

Application the Grover operator $j \in \mathbb{N}$ times on $\psi^0$ results in the state

$$\psi^j\left[X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\parallel}\left[X_{[d]}\right] + \cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\perp}\left[X_{[d]}\right] .$$

#### 12.4.3.3 Angle measurement

Using Quantum Phase Estimation we estimate the angle of the Grover rotation.

The eigenvalues of the Grover operator are (see Sect. 12.4.3) $\exp[\pm i\alpha]$ and 1, and have therefore phases $\frac{\pm\alpha}{2\pi i}$ and 0. Note that span $(\psi^{\parallel}, \psi^{\perp})$ is also spanned by the first two eigenvectors. When the state of the register is in span $(\psi^{\parallel}, \psi^{\perp})$ we therefore measure either $\frac{\alpha}{2\pi i}$ or $\frac{2\pi i - \alpha}{2\pi i}$.

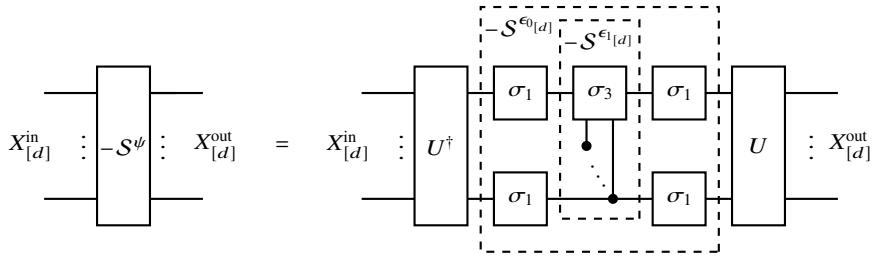



**Fig. 12.15** Preparation of the negated reflection operator $-\mathcal{S}^{\psi}$ about a state $\psi$, which is prepared by a unitary $U$ acting on the ground state $\epsilon_{0_{[d]}}\left[X_{[d]}\right]$. The multiply controlled Pauli-Z is the negated reflection $-\mathcal{S}^{1_{[d]}}$, and extended to the negated reflection $-\mathcal{S}^{1_{[d]}}$ about the ground state by adding layers of single qubit Pauli-X. When concatenating with $U^\dagger$ and $U$, the $-\mathcal{S}^{\psi}$ results.

#### 12.4.3.4 Circuit Realization of the reflection operators

Let us now investigate how the reflection operations in the Grover operator can be realized by circuits.

First, the reflection about the state $\epsilon_{1_{[d]}}\left[X_{[d]}\right]$ is realized by a multiple controlled Pauli-Z (see Figure 12.15)

$$-\mathcal{S}^{\epsilon_{1_{[d]}}}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] = \left\langle \text{MC}^{\text{d}-1}\sigma_3\left[X^{\text{in}}_{d-1}, X^{\text{out}}_{d-1}, X_{[d-1]}\right], \delta\left[X'_{[d-1]} X^{\text{in}}_{[d-1]}, X^{\text{out}}_{[d-1]}\right]\right\rangle_{\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]}$$

Note that a global phase of $-1$ is introduced, which leaves the measurement distribution invariant. Adding layers of Pauli-X gates to each $k \in [d]$ before and after $-\mathcal{S}^{\epsilon_{1_{[d]}}}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]$ (see Figure 12.15) prepares the negated reflection $-\mathcal{S}^{\epsilon_{0_{[d]}}}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]$, since

$$\epsilon_1\left[X^{\text{out}}_k\right] = \left\langle \sigma_3\left[X^{\text{out}}_k, X^{\text{in}}_k\right], \epsilon_0\left[X^{\text{in}}_k\right]\right\rangle_{\left[X^{\text{out}}_k\right]} .$$

Having a unitary $U$ preparing $\psi$ by action on the ground state, we realize the reflection (see Figure 12.15)

$$\mathcal{S}^{\psi}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] = \left\langle U^\dagger\left[X^{\text{in}}_{[d]}, X^{\text{mid},0}_{[d]}\right], \mathcal{S}^{\epsilon_{0_{[d]}}}\left[X^{\text{mid},0}_{[d]}, X^{\text{mid},1}_{[d]}\right], U\left[X^{\text{mid},1}_{[d]}, X^{\text{out}}_{[d]}\right]\right\rangle_{\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]} .$$

Alternatively we can use the phasekick technique (Lem. 12.13) when acting on an disentangled ancilla qubit prepared in the Hadamard basis state

$$\epsilon^-\left[Y\right] = \frac{1}{\sqrt{2}} \cdot \begin{pmatrix} 1 \\ -1 \end{pmatrix}\left[Y\right] .$$

We notice, that

$$\left\langle C^f\left[Y^{\text{out}}, Y^{\text{in}}, X_{[d]} = x_{[d]}\right], \epsilon^-\left[Y^{\text{in}}\right]\right\rangle_{[Y^{\text{out}}]} = \begin{cases} \epsilon^-\left[Y^{\text{in}}\right] & \text{if} \quad f(x_{[d]}) = 0 \\ -\epsilon^-\left[Y^{\text{in}}\right] & \text{if} \quad f(x_{[d]}) = 1 \end{cases}$$

Let us consider the span of one-hot encoded models of $\neg f$, namely

$$V^{\neg f} := \text{span}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \;:\; f(x_{[d]}) = 0\right)$$

and the span of one-hot encoded models

$$V^{f} := \text{span}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \;:\; f(x_{[d]}) = 1\right) .$$

The reflection across $V^{\neg f}$ is then performed by (see Figure 12.16)

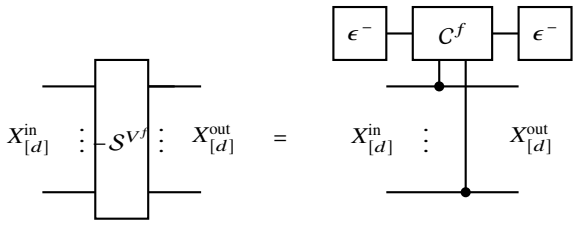



**Fig. 12.16** Realization of the reflection across the subspace of one-hot encoded models to a Boolean function $f$. An ancilla qubit is prepared in the Hadamard basis state $\epsilon^-$ and serves as the head qubit of the oracle $C^f$. By the phase kickback effect, the qubit stays disentangled and the reflection $-\mathcal{S}^{V^f}$ is effectively performed on the qubits $X_{[d]}$.

$$\mathcal{S}^{V^{\neg f}}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] = \left\langle \epsilon^-\left[Y^{\text{out}}\right], C^f\left[Y^{\text{out}}, Y^{\text{in}}, X_{[d]}\right], \epsilon^-\left[Y^{\text{in}}\right], \delta\left[X_{[d]}, X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right]\right\rangle_{\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right]}$$

We further have

$$\mathcal{S}^{V^f}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] = -\mathcal{S}^{(V^f)^\perp}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] = -\mathcal{S}^{V^{\neg f}}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] .$$

When preparing a disentangled ancilla qubit $Y$ in the basis state $\epsilon^-[Y]$, the oracle $C^f$ is effectively a negated reflection across the span of one-hot encoded models of $f$.

### 12.4.4 Moment Estimation

As an example, we here investigate how to use the introduced quantum routines to measure the mean of a function given a probability distribution. Given a distribution $\mathbb{P}\left[X_{[d]}\right]$, the mean of a propositional formula $f\left[X_{[d]}\right]$ is the contraction

$$\mu = \left\langle \mathbb{P}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .$$

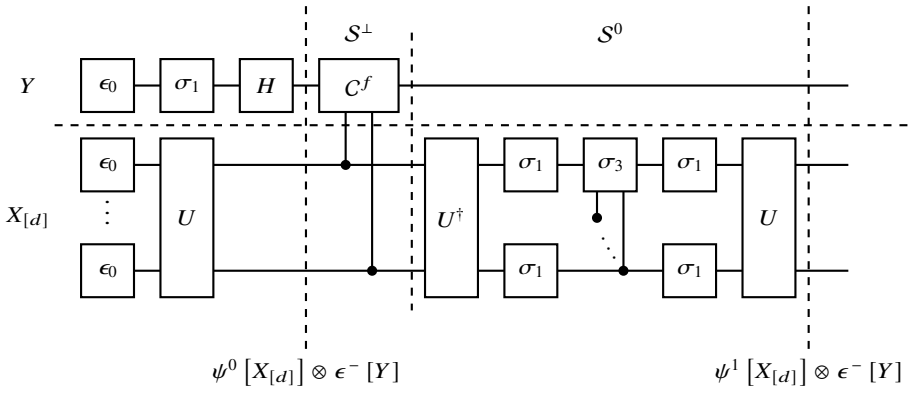



**Fig. 12.17** Amplification scheme of the at the models of a Boolean function $f$ supported coordinates of a quantum state. The ancilla variable $Y$ is prepared as an disentagled Hadamard basis vector and stays disentangled during the following reflections. The oracle $C^f$ effectively performs the reflection $\mathcal{S}^\perp$ across the bad state supported at the models of $f$.

We assume that we have

- a circuit $U$ preparing a state with measurement distribution $\mathbb{P}\left[X_{[d]}\right]$, i.e.

$$\psi^0\left[X^{\text{out}}_{[d]}\right] = \left\langle \epsilon_{0_{[d]}}\left[X^{\text{in}}_{[d]}\right], U\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]\right\rangle_{\left[X^{\text{out}}_{[d]}\right]}$$

and

$$\left|\psi^0\left[X_{[d]}\right]\right|^2 = \mathbb{P}\left[X_{[d]}\right]$$

- a computation circuit for $f\left[X_{[d]}\right]$

#### 12.4.4.1 Moments as rotation angles

We construct a Grover operator, which rotation angle determines the moment. The moment can therefore be estimated based on the measurement of the operators eigenvalue phases.

We have

$$\sum_{x_{[d]}\in[2]^d : f(x_{[d]})=1} \left|\psi^0\left[X_{[d]} = x_{[d]}\right]\right|^2 = \left\langle \mathbb{P}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \mu\,.$$

This insight motivates us to define the orthonormal quantum states

$$\psi^{\parallel}\left[X_{[d]}\right] = \frac{1}{\sqrt{\mu}} \sum_{x_{[d]}\in[2]^d : f(x_{[d]})=1} \psi^0\left[X_{[d]} = x_{[d]}\right]\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

$$\psi^{\perp}\left[X_{[d]}\right] = \frac{1}{\sqrt{1-\mu}} \sum_{x_{[d]}\in[2]^d : f(x_{[d]})=0} \psi^0\left[X_{[d]} = x_{[d]}\right]\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

and split the prepared state into a weighted sum of them as

$$\psi^0\left[X_{[d]}\right] = \sqrt{\mu}\cdot\psi^{\parallel}\left[X_{[d]}\right] + \sqrt{1-\mu}\cdot\psi^{\perp}\left[X_{[d]}\right]\,.$$

Note that here the mean parameter $\mu$ is the analogon to the acceptance probability $\lambda$ in quantum rejection sampling. To be more precise, when aiming to sample from the models of $f$ using a rejection sampling scheme starting with the uniform distribution, the acceptance probability would be $\mu$. Instead of amplifying this acceptance probability we here want to measure it to retrieve the mean parameter.

We prepare the reflection $-\mathcal{S}^0$ in the Pauli-Z scheme and the reflection $\mathcal{S}^{\perp}$ in the oracle scheme (see Sect. 12.4.3). Their concatenation results in the negated Grover operator, which causes a phase offset by $\pi$.

#### 12.4.4.2 Retrieving the moment from the phase

We now estimate the eigenvalue phases of the Grover algorithm (see Sect. 12.4.3.3) to read of the moment.

Note that the projection of the state $\psi^0$ onto the eigen space to the eigen value 1 vanishes, and the corresponding trivial phase is not measured by the Quantum Phase Estimation procedure.

The phase is related to the rotation angle by (see Sect. 12.4.3.3)

$$\alpha \approx \pm\frac{1}{2\pi\mathrm{i}} \sum_{k\in[n]} \phi_k 2^{d-k-1}\,.$$

The rotation angle is further related to the mean $\mu$ as

$$\alpha = 2\cdot\sin^{-1}\left(\sqrt{\mu}\right)\,.$$

Through measuring the phase $\phi \in \{\pm\alpha\}$ of one of these eigenvalues we can therefore retrieve the mean by

$$\mu = \left(\sin\left(\frac{\phi}{2}\right)\right)^2 .$$

#### 12.4.4.3 Measuring the phase

We apply the Quantum Phase Estimation (Sect. 12.4.2) to measure the moment.

We prepare auxiliary qubits $A_{[p]}$ and perform for each $\ell \in [p]$ $2^\ell$ repetitions of $G^{\mathbb{P},f}$ controlled on $A_\ell$. The inverse Fourier transform on the auxiliary qubits is supported on the (2-adic representation of the) phases on the eigenvalues, i.e. $\pm\alpha$, and can be detected by a computational basis measure of the ancilla variables.

When measuring the phase register as $a_{[p]}$ we compute a phase

$$\phi = 2\pi \cdot \sum_{\ell \in [p]} 2^{-(\ell+1)} a_\ell$$

which is either $\alpha$ or $-\alpha$. We retrieve the mean in both cases by

$$\mu = \sin^2\left(\pi \cdot \sum_{\ell \in [p]} 2^{-(\ell+1)} a_\ell\right) .$$

Computational resources: Square root improvement?

## 12.5 Conclusion

Update.

We have seen that preparing some classically hard to compute contraction can be done efficiently by a quantum circuit. We showed in particular that contracted basis encodings can be prepared by circuits, which are classically hard to contract in the presence of undirected loops in the decomposition hypergraph. However, when assessing the prepared contraction state by computational basis measurements, we effectively draw samples from the corresponding distribution, which can also be achieved classically. A quantum advantage could arise from different measurement schemes studied in the quantum state tomography community (see [33, 19, 70]). To this end, future research could show, how hard to achieve expectation queries can efficiently be prepared by customized measurement schemes.

# Part III
# Computation-Activation Networks

We now study a more specific tensor network architecture, namely Computation-Activation Networks, which represent both logical as well as probabilistic models. We investigate their properties and inference schemes on them.

# Chapter 13
# Elementary Computation-Activation Networks

**Abstract** Dicuss here the case of elementary activation, Hard, Markov and Hybrid Logic Networks.
Topics

- Exponential Families (Report I.2.3)

## 13.1 Exponential Families

### 13.1.1 Definition

**Definition 13.1 (Exponential Family)** Given a statistic function

$$t : \bigtimes_{k\in[d]} [m_k] \to \mathbb{R}^p$$

and a boolean base measure

$$\nu : \bigtimes_{k\in[d]} [m_k] \to \{0,1\}$$

with $\langle \nu \rangle_{[\varnothing]} \neq 0$, the set $\Gamma^{t,\nu} = \{\mathbb{P}^{(t,\theta,\nu)} : \theta [L] \in \mathbb{R}^p\}$ of probability distributions

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta[L]\right\rangle_{\left[X_{[d]}\right]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$$

is called the exponential family to $t$. We further define for each member with parameters $\theta$ the associated energy tensor

$$\phi^{(t,\theta,\nu)}[X_{[d]}] = \left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}$$

and the cumulant function

$$A^{(t,\nu)}(\theta) = \ln\left[\left\langle \nu, \exp\left[\left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}\right]\right\rangle_{[\varnothing]}\right].$$

*Example 13.1 (Exponential family of coin tosses)* Recall the family of distributions of boolean $X_{[d]}$ from Example 5.5, which has the order statistic $t^+$ as a sufficient statistic. We now in addition assume that the variables $X_{[d]}$ are i.i.d. with respect to any member of the family (see Example 5.5). For the variables to be i.i.d., we need $\nu\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right]$ and can thus choose a representation such that for $z \in \mathrm{val}(Z)$

$$\mathbb{P}\left[X_{[d]}\middle|Z = z\right] = \left\langle \beta^{t^+}\left[Y_+, X_{[d]}\right], \xi\left[Y_+, Z = z\right]\right\rangle_{\left[X_{[d]}\right]}$$

where for each $k \in [d+1]$

$$\xi\left[Y_+ = k, Z = z\right] = (1-z)^{d-k} \cdot z^k \,.$$

The marginal distribution $\mathbb{P}\big[Y_+ \big| Z = z\big]$ is then the binominal distribution $B(d, z)$. When excluding the case of $z \in \{0, 1\}$, the family is a subset of the exponential family with the head count statistic, where each member is reparametrized by

$$\theta := \ln\left[\frac{z}{1-z}\right] .$$

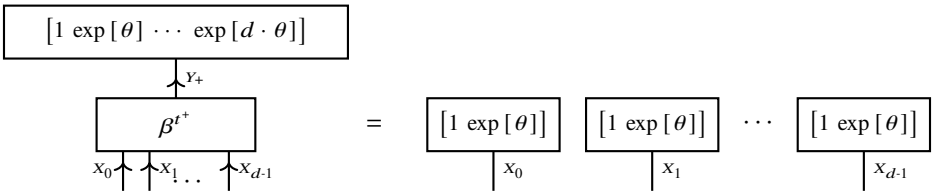
To see that this is true, we observe that the coordinate $y_+ \in [d+1]$ of the activation tensor of $\mathbb{P}^{(t^+,\theta,\mathbb{I})}$ is

$$\alpha^\theta\left[Y_+ = y_+\right] = \exp\left[y_+ \cdot \theta\right] = \frac{z^{y_+}}{(1-z)^{y_+}} \,.$$

Now, with $\mathcal{Z}(\theta) = \frac{1}{(1-z)^d}$ we have for any $x_{[d]}$ with $\sum_{k\in[d]} x_k = y_+$ that

$$\frac{1}{\mathcal{Z}(\theta)} \cdot \left\langle \beta^{t^+}\left[Y_+, X_{[d]} = x_{[d]}\right], \alpha^\theta\left[Y_+\right]\right\rangle_{[\varnothing]} = (1-z)^d \cdot \frac{z^{y_+}}{(1-z)^{y_+}} = z^{y_+} \cdot (1-z)^{d-y_+} \,.$$

Comparing with the activation tensor $\xi\left[Y_+\right]$ above, we note that $\mathcal{Z}(\theta)$ is the partition function of the exponential family and $\mathbb{P}^{(t^+,\theta,\mathbb{I})}\left[X_{[d]}\right]$ coincides with the member $\mathbb{P}\big[X_{[d]}\big| Z = z\big]$. We further observe that since the statistic $t^+$ decomposes as a sum of terms depending on single variables only, we have a decomposition of the corresponding CompActNet by




This reproduces the fact that distributions of independent variables are representable by elementary tensors (see Example 5.1).

We used the selection encoding to represent the weighted summation over the statistics, that is the tensor (see Def. **??**)

$$\sigma^t\left[X_{[d]}, L\right] : \bigtimes_{k\in[d]} [m_k] \times [p] \to \mathbb{R}$$

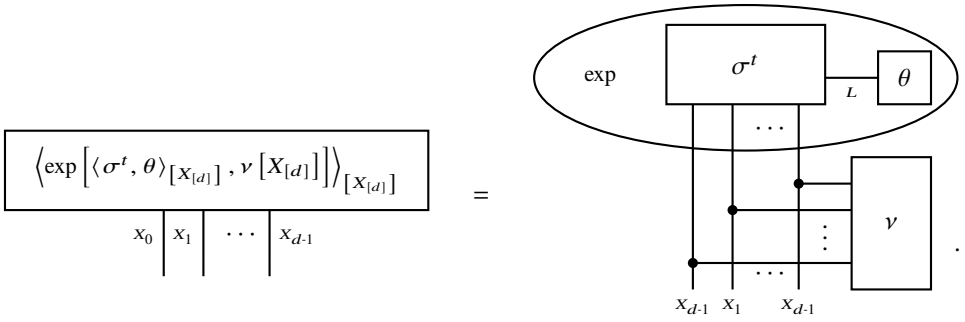
defined for $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $\ell \in [p]$ as

$$\sigma^t\left[X_{[d]} = x_{[d]}, L = \ell\right] = t_\ell\left[X_{[d]} = x_{[d]}\right] .$$

The selection encoding represent the weighted sum of the statistic coordinates by the canonical parameter vector $\theta\left[L\right]$ as a contraction

$$\sum_{\ell\in[p]} \theta\left[L = \ell\right] \cdot t_\ell\left[X_{[d]}\right] = \left\langle \sigma^t\left[X_{[d]}, L\right], \theta\left[L\right]\right\rangle_{[X_{[d]}]} .$$

For more details on this representation scheme, we refer to Thm. **??** in Chapter **??**. Up to normalization, we sketch the probability distribution of any member by the tensor network diagram




We here denote by an ellipsis the coordinatewise transformation by the exponential function (see Sect. **??**). Since such coordinatewise transformation are nonlinear, they are a caveat for efficient contraction of the diagram.

Since we restrict the discussion to finite state spaces, the distribution $\mathbb{P}^{(t,\theta,\nu)}$ is well-defined for any $\theta\,[L] \in \mathbb{R}^p$. For infinite state space there are sufficient statistics and parameters, such that the partition function $\left\langle \nu, \exp\left[\langle \sigma^t, \theta\rangle_{[X_{[d]}]}\right]\right\rangle_{[\varnothing]}$ diverges and the normalization $\mathbb{P}^{(t,\theta,\nu)}$ is not well-defined. In that cases, the canonical parameters need to be chosen from a subset where the partition function is finite [82].

As before, we restrict to boolean base measures, which have to satisfy $\langle \nu\rangle_{[\varnothing]} \neq 0$ for respective distributions to exist. We notice that by positivity of the exponential function, any distribution in an exponential family $\Gamma^{t,\nu}$ is positive with respect to $\nu$ (see Def. 5.3). In Chapter **??** we will investigate distributions, where the base measures and the sufficient statistics share a common decomposition framework.

**Lemma 13.1** *For any member of an exponential family $\Gamma^{t,\nu}$ we have*

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \exp\left[\phi^{(t,\theta,\nu)}[X_{[d]}] - A^{(t,\nu)}(\theta)\cdot \mathbb{I}\left[X_{[d]}\right]\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} .$$

*Proof.* By definition we have

$$\begin{aligned}
\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] &= \left\langle \exp\left[\left\langle \sigma^t, \theta\right\rangle_{[X_{[d]}]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}|\varnothing]} \\
&= \frac{\left\langle \exp\left[\langle \sigma^t, \theta\rangle_{[X_{[d]}]}, \nu\left[X_{[d]}\right]\right]\right\rangle_{[X_{[d]}]}}{\left\langle \exp\left[\left\langle \sigma^t, \theta\right\rangle_{[X_{[d]}]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}} \\
&= \frac{\left\langle \exp\left[\phi^{(t,\theta,\nu)}[X_{[d]}]\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]}}{\exp\left[A^{(t,\nu)}(\theta)\right]} \\
&= \left\langle \exp\left[\phi^{(t,\theta,\nu)}[X_{[d]}] - A^{(t,\nu)}(\theta)\cdot \mathbb{I}\left[X_{[d]}\right]\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} .\square
\end{aligned}$$

A further useful criterion is that of minimality of an exponential family, as we define next.

**Definition 13.2 (Minimal Statistics)** We say that a statistic $t$ is minimal with respect to a boolean base measure $\nu$, if there is no pair of a non-vanishing vector $V[L]$ and a scalar $\lambda \in \mathbb{R}$ with

$$\left\langle \sigma^t\left[X_{[d]}, L\right], V[L], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} = \lambda\cdot \nu\left[X_{[d]}\right] .$$

If a statistic is not minimal, we can omit coordinates of it without affecting the expressivity $\Gamma^{t,\nu}$. As long as we find a non-vanishing vector $V[L]$ and $\lambda \in \mathbb{R}$ as in Def. 13.2, we can choose a coordinate $t_\ell$ such that $V[L = \ell] \neq 0$, conclude that the coordinate is linear dependent on the others and drop it as redundant.

### 13.1.2 Tensor Network Representation

As we have observed, the selection encoding formalism can efficiently represent the energy tensor to a member of an exponential family, but through coordinatewise transform by the exponential does not provide an efficient decomposition scheme of the probability distribution itself. We now overcome this problem with usage of the basis encoding formalism to represent members of exponential families by a single contraction without nonlinear transforms.

**Theorem 13.1 (Tensor Network Representation of Exponential Families)** *Given any base measure $\nu$ and a sufficient statistic $t$ we enumerate for each coordinate $\ell \in [p]$ the image* $\mathrm{im}\,(t_\ell)$ *by a variable $Y_\ell$ taking values in* $[|\mathrm{im}\,(t_\ell)|]$ *(see for more details on this scheme Chapter 7), given an interpretation map*

$$I_\ell \,:\, [|\mathrm{im}\,(t_\ell)|] \to \mathrm{im}\,(t_\ell)\ .$$

*For any canonical parameter vector $\theta\,[L] \in \mathbb{R}^p$ we build the activation cores $\alpha^{\ell,\theta}\,[Y_\ell]$ for each coordinate $y_\ell \in [|\mathrm{im}\,(t_\ell)|]$ by*

$$\alpha^{\ell,\theta}\,[Y_\ell = y_\ell] = \exp\left[\theta\,[L = \ell] \cdot I_\ell\,[y_\ell]\right]$$

*and have*

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \{\nu\left[X_{[d]}\right]\} \cup \{\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\} \cup \{\alpha^{\ell,\theta}\,[Y_\ell] \,:\, \ell \in [p]\}\right\rangle_{\left[X_{[d]}|\varnothing\right]}\ .$$

*Proof.* We embed the image of $t$ in the cartesian product of the coordinate images

$$\mathrm{im}\,(t) \subset \bigtimes_{\ell\in[p]} \mathrm{im}\,(t_\ell)$$

and design enumerate the embedded image of $t$ by the variables $Y_{[p]}$. Thm. **??**, to be shown in Chapter 7, implies

$$\exp\left[\left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \exp\left[\langle\cdot, \theta\,[L]\rangle\right]|_{\bigtimes_{\ell\in[p]}\mathrm{im}(t_\ell)}\right\rangle_{\left[X_{[d]}\right]}\ .$$

Here we denote by $\langle\cdot, \theta\,[L]\rangle$ the dual function to $\theta\,[L]$, which assigns to vectors their contraction with $\theta\,[L]$. Its restriction onto the vectors in $\bigtimes_{\ell\in[p]}\mathrm{im}\,(t_\ell)$ is the tensor satisfying

$$\exp\left[\langle\cdot, \theta\rangle\right]|_{\mathrm{im}(t)}\left[Y_{[p]}\right] = \bigotimes_{\ell\in[p]} \exp\left[\cdot\theta\,[L = \ell]\right]|_{\mathrm{im}(t_\ell)}\,[Y_\ell] = \bigotimes_{\ell\in[p]} \alpha^{\ell,\theta}\,[Y_\ell]\ .$$

We further have (see Thm. **??** in Chapter 7)

$$\beta^t\left[Y_{[p]}, X_{[d]}\right] = \left\langle \{\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]}\ .$$

Refining the above decomposition of $\exp\left[\left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}\right]$ by these further decompositions we arrive at the claim. □

In the proof of Thm. 14.2 we have observed, that the basis encoding $\beta^t\left[Y_{[p]}, X_{[d]}\right]$ of the statistics decomposed into a tensor network of basis encodings $\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right]$ to the coordinate of the statistic. We can exploit further decomposition mechanisms, which will be discussed in full detail in Chapter 7, to find even sparser decompositions. This is for example the case, when the coordinates of the statistic are compositions of functions depending on small numbers of variables. When the coordindates of the statistic furthermore share similar parts in their compositions, these parts can be shared in the decomposition. We will investigate such sparsification mechanisms in more detail in Chapter **??**, where the coordinates of the statistic are propositional formulas with a natural decomposition by their syntactical description.

The tensor network representation of an exponential family by Thm. 14.2 is split into computation and activation cores (see Sect. **??**):

- **Computation Cores:** The basis encodings $\beta^{t_\ell}$ of the coordinates of a statistic form the computation cores. Our intuition is that they compute the hidden variable $Y_\ell$, based on Basis Calculus (see Chapter 7), which encode the

value of the coordinate with respect to the image interpretation map $I_\ell$. We notice that since they are directed with $Y_\ell$ being the only outgoing variable, they do not influence any contraction with open variables $X_{[d]}$, unless further tensors sharing the variable $Y_\ell$ are present in the contraction.

- **Activation Cores:** The influence of the contraction is performed by the activation cores $\alpha^{\ell,\theta}\left[Y_\ell\right]$, which exploit the computed statistic variables $Y_{[p]}$ and provide in combination with the basis encoding a factor

$$\left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]}\right], \alpha^{\ell,\theta}\left[Y_\ell\right]\right\rangle_{\left[X_{[d]}\right]}$$

to the tensor network representing the distribution of the observed variables $X_{[d]}$.

For the implementation of an exponential family as a Computation-Activation Network, see Example **??**.

When the canonical parameter is vanishing at a coordinate, that is $\theta\left[L=\ell\right]=0$, then the corresponding factor is trivial, since $\alpha^{\ell,0}\left[Y_\ell\right]=\mathbb{I}\left[Y_\ell\right]$ and as a consequence of the directionality of basis encodings we have

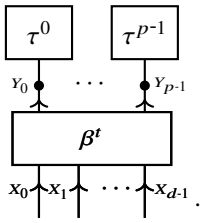
$$\left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]}\right], \alpha^{\ell,\theta}\left[Y_\ell\right]\right\rangle_{\left[X_{[d]}\right]} = \left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]}\right], \mathbb{I}\left[Y_\ell\right]\right\rangle_{\left[X_{[d]}\right]} = \mathbb{I}\left[X_{[d]}\right] .$$

In that case both the activation core and the corresponding computation core can be dropped from the network without changing its distribution.

By Thm. 14.2 any member of an exponential family is represented by the normalized contraction of a collection of unary activation cores contracted with the computation cores $\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right]$. The unary activation cores form a tensor network on the elementary hypergraph

$$\mathrm{EL} = \left([p], \{\{\ell\} : \ell \in [p]\}\right),$$

which hyperedges contain single variables. We call this graph elementary, since it corresponds with elementary tensor network formats consistent of tensor products of vectors. For unary activation cores, that is for the elementary graph EL, any member of $\Lambda^{t,\mathrm{EL}}$ has up to a normalization factor a tensor network decomposition by the diagram

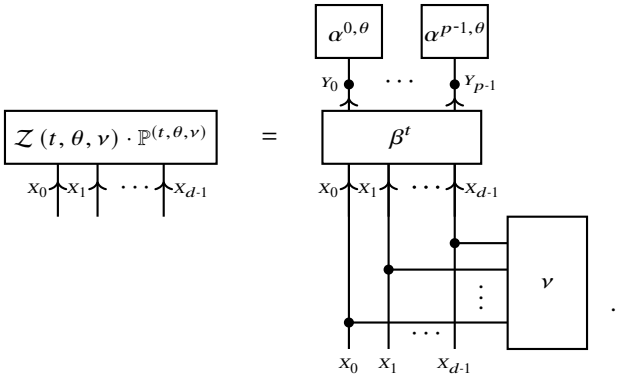



Comparing this representation scheme with Thm. 14.2, we conclude as the next corollary, that any member of an exponential family with trivial base measure can be represented by an elementary activation tensors.

**Corollary 13.1 (of Thm. 14.2)** *For any statistic* $t : \bigtimes_{k\in[d]}[m_k] \to [p]$ *and trivial base measure* $\nu\left[X_{[d]}\right]=\mathbb{I}\left[X_{[d]}\right]$ *we have*

$$\Gamma^{t,\mathbb{I}} \subset \Lambda^{t,\mathrm{EL}} .$$

For elements of the exponential family with general boolean base measure we have with the activation cores constructed in Thm. 14.2




where the partition function represents the normalizing contraction of the tensor network. Let us note, that when choosing activation cores with nontrivial support, we can also prepare boolean base measures and in principle extend Cor. 13.1 to families of nontrivial base measures. We will investigate such schemes later in Chapter **??**, where we call them Hybrid Logic Networks.

In comparison with the selection encoding representation of energy tensors, we have prepared a contraction without non-linear transforms, which represents the probability distributions being members of an exponential family. However, relation encoding come with the expense of introducing more auxiliary variables compared with selection encodings. To be more precise, while selection encodings bundle the coordinates of the statistic in single selection variables, relation encodings create for each state $\ell \in [p]$ of these selection variable an own auxiliary variable $Y_\ell$, which enumerated the image of the coordinate and can therefore be of high dimension. Thus, selection encodings offer in general a more efficient storage format coming at the expense of nonlinear operations in the computation of probabilities. We later will encounter situations, where selection encodings are feasible while relation encodings are not, when applying the formalism of formula selecting networks (see Chapter **??**) in neuro-symbolic reasoning (see Chapter **??**).

Based on Cor. 13.1 a further natural question is, whether $\Gamma^{t,\mathbb{I}}$ is a proper subset of $\Lambda^{t,\mathrm{EL}}$. This is the case for most statistics $t$, since members of exponential families are positive with respect to their base measure, which is in the corollaries setting trivial, while in $\Lambda^{t,\mathrm{EL}}$ we allow also for activation cores with vanishing coordinates, which in general do not produce positive distributions. The only statistics where $\Gamma^{t,\mathbb{I}}$ is not a proper subset of $\Lambda^{t,\mathcal{G}}$ are along this argumentation constant, since then the activation cores are one-dimensional vectors and vanishing coordinates are prohibited by the need for normalizability. We will follow these intuitions in the discussion of logical reasoning, starting with Chapter **??**, and will use the formats $\Lambda^{t,\mathcal{G}}$ as hybrid formats storing probability distributions and logical knowledge bases.

While we have restricted our discussion on the elementary decomposition of the activation tensor, further decomposition schemes have interesting interpretations as well. Given a CP decomposition of the activation tensor (see for more details Chapter **??**), the corresponding distributions are weighted mixture distributions built from the elementary decompositions. In general, the expressivity increases monotonously with the introduction of additional auxiliary variables and hyperedges in the representation format of activation tensors.

### 13.1.3 Markov Networks as Exponential Families

As we have claimed before, exponential families can be regarded as a generalization of graphical models. We here show this claim by a construction of an exponential family to any hypergraph, such that any positive Markov Networks on this hypergraph is in the exponential family.

**Theorem 13.2 (Exponential Representation of Markov Networks)** *Let there be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with a coloring of the nodes by dimensions $m_v$, we define a sufficient statistics*

$$t^{\mathcal{G}} : \bigtimes_{v\in\mathcal{V}} [m_v] \to \bigtimes_{e\in\mathcal{E}} \left( \bigtimes_{v\in e} [m_v] \right)$$

*by a cartesian product $t^{\mathcal{G}} = \bigtimes_{e\in\mathcal{E}} t_e$ of statistics*

$$t_e : \bigtimes_{v \in \mathcal{V}} [m_v] \to \bigtimes_{v \in e} [m_v]$$

*defined by the restriction of indices to the respective edge, that is for* $x_{\mathcal{V}} \in \bigtimes_{v \in \mathcal{V}} [m_v]$

$$t_e(x_{\mathcal{V}}) = x_e \, .$$

*Given any Markov Network with positive tensors* $\{\tau^e : e \in \mathcal{E}\}$ *decorating the hyperedges of* $\mathcal{G}$ *we define*

$$\theta\,[L] = \bigtimes_{e \in \mathcal{E}} \theta_e\,[X_e]$$

*where*

$$\theta_e\,[X_e] = \ln\left[\tau^e\,[X_e]\right]$$

*and L enumerates a concatenation of the states of* $X_e$*. Then, the Markov Network is the member with canonical parameter* $\theta$ *of the exponential family with trivial base measure, statistic* $t^{\mathcal{G}}$*, which we denote by* $\Gamma^{t^{\mathcal{G}},\mathbb{I}}$*.*

*Proof.* We have for any $x_{\mathcal{V}}$

$$\begin{aligned}
\prod_{e \in \mathcal{E}} \tau^e\,[X_e = x_e] &= \exp\left[\sum_{e \in \mathcal{E}} \ln\left[\tau^e\,[X_e = x_e]\right]\right] \\
&= \exp\left[\sum_{e \in \mathcal{E}} \theta_e\,[X_e = x_e]\right] \\
&= \exp\left[\sum_{e \in \mathcal{E}} \left\langle \theta_e\,[X_e]\,, t_e(x_{\mathcal{V}})\right\rangle_{[\varnothing]}\right] .
\end{aligned}$$

By contraction, we further have

$$\left\langle t\,(X_{\mathcal{V}}, L)\,, \theta\,[L]\right\rangle_{[X_{\mathcal{V}}]} = \sum_{e \in \mathcal{E}} \left\langle t_e\,[X_{\mathcal{V}}, X_e]\,, \theta_e\,[X_e]\right\rangle_{[X_{\mathcal{V}}]}$$

we thus get with the above

$$\left\langle \{\tau^e : e \in \mathcal{E}\}\right\rangle_{[X_{\mathcal{V}}]} = \exp\left[\left\langle t\,(X_{\mathcal{V}}, L)\,, \theta\,[L]\right\rangle_{[X_{\mathcal{V}}]}\right] . \tag{13.1}$$

This implies, that the contraction of the tensors in the Markov Network coincides with the exponential of the energy tensor of the constructed member of the exponential family. It follows for the normalization, that

$$\left\langle \{\tau^e : e \in \mathcal{E}\}\right\rangle_{[X_{\mathcal{V}}|\varnothing]} = \left\langle \exp\left[\left\langle \theta, t\right\rangle_{[X_{\mathcal{V}}]}\right]\right\rangle_{[X_{\mathcal{V}}|\varnothing]} . \tag{13.2}$$

We thus conclude, that the Markov Network coincides with the constructed member of the exponential family. □

The mean parameter of the Markov Network exponential family is the cartesian product of the marginals $\mu_e\,[X_e]$. They are often referred to as beliefs in the literature, as introduced by [62]. For Markov Networks on tree hypergraphs, and their embedding into junction tree formats, the corresponding mean parameter polytope can be characterized by local consistency constraints. More precisely, it can be shown, that for the statistic constructed in Thm. 13.2, in case of tree hypergraphs, the

$$\begin{aligned}
\mathcal{M}_{t^{\mathcal{G}}} = \Big\{ \mu\,[L] = \left(\mu_e\,[X_e]\right)_{e \in \mathcal{E}} : &\forall e, \tilde{e} \in \mathcal{E} : \left\langle \mu_e\,[X_e]\right\rangle_{[X_{e \cap \tilde{e}}]} = \left\langle \mu_e\,[X_e]\right\rangle_{[X_{e \cap \tilde{e}}]}, \\
&\forall e : 0\,[X_e] \prec \mu_e\,[X_e] \wedge \left\langle \mu_e\,[X_e]\right\rangle_{[\varnothing]} = 1 \Big\} .
\end{aligned}$$

That is, the polytope of realizable mean parameters consists of those non-negative and normalized beliefs, which are coinciding on the contraction of shared variables. Capturing these constraints by Lagrange parameters and performing

optimization of certain objectives then results in message-passing schemes, as we will discuss in Chapter 8. If the hypergraph is not minimally connected, this constructed polytope is only an outer bound of the true mean parameter polytope, but still serves as a motivation of loopy belief propagation schemes (see Chapter 4 in [82]).

### 13.1.4 Representation of Generic Distributions

We now present a universal exponential family, which contains all positive distributions with respect to a base measure. This implies that the formalism of exponential families can capture any probability distribution, when applying statistic functions of large expressivity. Taking for the statistic the identity function $\delta\left[X_{[d]}, L_{[d]}\right]$ defined as

$$\delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right] = \begin{cases} 1 & \text{if} \quad x_{[d]} = \ell_{[d]} \\ 0 & \text{else} \end{cases} ,$$

we can represent any positive probability distribution $\mathbb{P}\left[X_{[d]}\right]$ as a member of the exponential family $\Gamma^{\delta,\mathbb{I}}$. To see this, it is enough to choose

$$\theta\left[L_{[d]}\right] = \left\langle \ln\left[\mathbb{P}\left[X_{[d]}\right]\right], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[L_{[d]}\right]} ,$$

where the contraction with $\delta$ copies the variables $X_{[d]}$ to $L_{[d]}$. The energy tensor of this member of $\Gamma^{\delta,\mathbb{I}}$ is then

$$\left\langle \theta\left[L_{[d]}\right], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = \ln\left[\mathbb{P}\left[X_{[d]}\right]\right]$$

and thus

$$\mathbb{P}^{\delta,\theta,\mathbb{I}}[X_{[d]}] = \left\langle \exp\left[\ln\left[\mathbb{P}\left[X_{[d]}\right]\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} = \mathbb{P}\left[X_{[d]}\right] .$$

We further note, that the mean parameter of this constructed element of $\Gamma^{\delta,\mathbb{I}}$ is

$$\begin{aligned} \mu\left[L_{[d]}\right] &= \left\langle \mathbb{P}^{\delta,\theta,\mathbb{I}}[X_{[d]}], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[L_{[d]}\right]} \\ &= \left\langle \mathbb{P}\left[X_{[d]}\right], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[L_{[d]}\right]} , \end{aligned}$$

and thus coincides with the distribution itself, after a relabelling of the distributed variables. Let us notice, that this family also correspond with the Markov Network on the maximal hypergraph $\mathrm{MAX} = (\mathcal{V}, \{\mathcal{V}\})$. We will further revisit this family in Chapter **??**, where we will refer to it by the minterm family in order to connect with terminology developed for logical reasoning in Chapter **??**.

# Chapter 14
# Mean Polytopes and Maximum Entropy Distributions

**Abstract** Motivated by the search for maximum entropy distributions we investigate the convex polytope of mean parameters to a statistic function. We then show that exponential families are maximum entropy distributions to mean parameters in the relative interior of the mean polytope and generalize the result to the faces of the mean polytope. In the case of Boolean statistics we provide a characterization in terms of propositional logic. Further, we investigate the mean parameter as a sufficient statistic and investigate schemes to compute the activation tensor to faces.

## 14.1 Maximum entropy problem

Given a non-negative and non-vanishing base measure $\nu\left[X_{[d]}\right]$, a probability distribution is a non-negative tensor $\mathbb{P}\left[X_{[d]}\right]$ such that

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 1\,.$$

We denote the set of such distributions by

$$\begin{aligned}\Lambda^{\nu} = \big\{\mathbb{P}\left[X_{[d]}\right] \;:\; & \mathbb{P}\left[X_{[d]}\right] \geq 0\left[X_{[d]}\right], \left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 1, \\ & \forall_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \nu\left[X_{[d]} = x_{[d]}\right] = 0 \Rightarrow \mathbb{P}\left[X_{[d]} = x_{[d]}\right] = 0\big\}\,.\end{aligned}$$

Note that we here restrict the support of the tensor $\mathbb{P}\left[X_{[d]}\right]$ to be included in the support of the base measure, to avoid ambiguity in the representation.

The entropy of a distribution relative to the base measure $\nu$ is

$$\mathbb{H}_{\nu}\left[\mathbb{P}\left[X_{[d]}\right]\right] = -\left\langle \mathbb{P}\left[X_{[d]}\right], \ln\left[\mathbb{P}\left[X_{[d]}\right]\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\,.$$

Note that this quantity depends on the chosen base measure.

A statistic is a function $t : \bigtimes_{k\in[d]}[m_k] \to \mathbb{R}^p$ assigning a real vector of dimension $p$ to each state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$. We denote by $t_\ell$ the restriction to the $\ell$-th coordinate of $t$. The mean parameter of a distribution $\mathbb{P}\left[X_{[d]}\right]$ to a statistic $t$ is the vector $\mu\left[L\right] \in \mathbb{R}^p$ with the coordinates

$$\mu\left[L = \ell\right] = \mathbb{E}\left[f_\ell\right] = \left\langle t_\ell\left[X_{[d]}\right], \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\,.$$

To express the computation of the mean parameter as a single contraction, we introduce a coordinate selection variable $L$ of dimension $p$ and the selection encoding $\sigma^t\left[X_{[d]}, L\right]$ with coordinates by

$$\sigma^t\left[X_{[d]} = x_{[d]}, L = \ell\right] = t_\ell\left[X_{[d]} = x_{[d]}\right]\,.$$

The mean parameter is then

$$\mu\left[L\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]} .$$

The maximum entropy problem given a mean parameter $\mu^*[L]$ is stated by

$$\operatorname{argmax}_{\mathbb{P}[X_{[d]}]\in\Lambda^\nu} \mathbb{H}_\nu\left[\mathbb{P}\left[X_{[d]}\right]\right] \quad \text{subject to} \quad \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], \nu\left[X_{[d]}\right]\right\rangle_{[L]} = \mu^*[L] \tag{$\mathrm{P}_{t,\mu,\nu}$}$$

## 14.2 The mean polytope

Given a statistic function

$$t : \bigtimes_{k\in[d]} [m_k] \to \mathbb{R}^p$$

on the states $\bigtimes_{k\in[d]}[m_k]$ and a base measure $\nu\left[X_{[d]}\right]$ the mean polytope is the set

$$\mathcal{M}_{t,\nu} := \left\{\left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], \nu\left[X_{[d]}\right]\right\rangle_{[L]} : \mathbb{P}\left[X_{[d]}\right] \in \Lambda^\nu\right\}$$

containing all mean parameters for the distributions in $\Lambda^\nu$. By $\sigma^t\left[X_{[d]}, L\right]$ we denote the selection encoding of the statistic (see introduction). Since $\Lambda^\nu$ is the convex hull of the vertices $\frac{1}{\nu\left[X_{[d]}=x_{[d]}\right]}\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]$, and any mean parameter is a linear transformation of the distribution, the mean polytope is the convex hull of the vectors

$$\mathcal{N}_{t,\nu} := \{\sigma^t\left[X_{[d]} = x_{[d]}, L\right] : x_{[d]} \in \bigtimes_{k\in[d]} [m_k],\ \nu\left[X_{[d]} = x_{[d]}\right] \neq 0\} .$$

### 14.2.1 Faces

When studying maximum entropy distributions, we will pay special attention to the faces of a polytope, which form the boundary. Here, we define them based on convex hulls and introduce tensor representations by refined base measures.

**Definition 14.1** We say that the convex hull of a subset $\mathcal{N}^{\mathcal{F}}_{t,\nu} \subset \mathcal{N}_{t,\nu}$ is a face of a polytope $\mathcal{M}_{t,\nu}$, if and only if there is a vector $\theta_{\mathcal{F}}[L] \in \mathbb{R}^p$ such that

$$\mathcal{N}^{\mathcal{F}}_{t,\nu} = \operatorname{argmax}_{\mu[L]\in\mathcal{N}_{t,\nu}} \left\langle \mu\left[L\right], \theta_{\mathcal{F}}\left[L\right]\right\rangle_{[\varnothing]} .$$

We denote the face as $\mathcal{F} = \operatorname{conv}\left(\mathcal{N}^{\mathcal{F}}_{t,\nu}\right)$. The set of all faces of the mean polytope $\mathcal{M}_{t,\nu}$ is denoted by $L\left(\mathcal{M}_{t,\nu}\right)$.

The set $L\left(\mathcal{M}_{t,\nu}\right)$, partially ordered by the inclusion operation, is a lattice (see [84]). We now show that each face itself is a convex polytope corresponding to a refined base measure.

**Definition 14.2** The refined base measure with respect to a face $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ is the tensor $\nu^{t,\mathcal{F}}\left[X_{[d]}\right]$ with coordinates to $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ by

$$\nu^{t,\mathcal{F}}\left[X_{[d]}\right] = \begin{cases} \nu\left[X_{[d]} = x_{[d]}\right] & \text{if} \quad t\left(x_{[d]}\right) \in \mathcal{N}^{\mathcal{F}}_{t,\nu} \\ 0 & \text{else} \end{cases} .$$

The refined base measure has by definition a smaller support than the base measure. We now show that restricting the distributions in $\Lambda^\nu$ to those in $\Lambda^{\nu^{t,\mathcal{F}}}$ amounts to a restriction of the polytope $\mathcal{M}_{t,\nu}$ to the corresponding face.

**Lemma 14.1** *For any face $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ we have*

$$\mathcal{F} = \mathcal{M}_{t,\nu^{t,\mathcal{F}}} .$$

*Proof.* For any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have $\nu^{t,\mathcal{F}}\left[X_{[d]} = x_{[d]}\right] \neq 0$ if and only if $t\left(x_{[d]}\right) \in \mathcal{N}^{\mathcal{F}}_{t,\nu}$ and $\nu\left[X_{[d]} = x_{[d]}\right] \neq 0$. Thus we have

$$\begin{aligned}\mathcal{F} &= \operatorname{conv}\left(\mathcal{N}^{\mathcal{F}}_{t,\nu}\right)\\ &= \operatorname{conv}\left(\sigma^t\left[X_{[d]} = x_{[d]}, L\right] \,:\, t\left(x_{[d]}\right) \in \mathcal{N}^{\mathcal{F}}_{t,\nu},\ \nu\left[X_{[d]} = x_{[d]}\right] \neq 0\right)\\ &= \operatorname{conv}\left(\sigma^t\left[X_{[d]} = x_{[d]}, L\right] \,:\, \nu^{t,\mathcal{F}}\left[X_{[d]} = x_{[d]}\right] \neq 0\right)\\ &= \mathcal{M}_{t,\nu^{t,\mathcal{F}}}.\square\end{aligned}$$

Representability of a distribution with respect to face measures is an equivalent condition for the mean parameter of a distribution to be on a face, as we show next.

**Lemma 14.2** *If and only if for a distribution $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu}$ and a face $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ we have*

$$\left\langle \sigma^t\left[X_{[d]}, L\right], \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[L]} \in \mathcal{F},$$

*then*

$$\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu^{t,\mathcal{F}}}.$$

*Proof.* For a distribution $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu}$ to be in $\Lambda^{\nu^{t,\mathcal{F}}}$, it is enough to show that $\mathbb{P}\left[X_{[d]}\right]$ is only supported at $\nu^{t,\mathcal{F}}\left[X_{[d]}\right]$. We have

$$\mu[L] = \sum_{x_{[d]}} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \nu\left[X_{[d]} = x_{[d]}\right] \cdot \gamma^t\left[X_{[d]} = x_{[d]}, L\right].$$

Now, let $\theta_{\mathcal{F}}[L]$ be a face normal to the face $\mathcal{F}$. We then have

$$\left\langle \mu[L], \theta_{\mathcal{F}}[L]\right\rangle_{[\varnothing]} = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \nu\left[X_{[d]} = x_{[d]}\right] \cdot \left\langle \gamma^t\left[X_{[d]} = x_{[d]}, L\right]\theta_{\mathcal{F}}[L]\right\rangle_{[\varnothing]}.$$

Now if and only if $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \nu\left[X_{[d]} = x_{[d]}\right]$ is supported only for $x_{[d]}$ with $t\left(x_{[d]}\right) \in \mathcal{N}^{\mathcal{F}}_{t,\nu}$ we have that

$$\left\langle \mu[L], \theta_{\mathcal{F}}[L]\right\rangle_{[\varnothing]} = \max_{\mu[L]\in\mathcal{M}_{t,\nu}} \left\langle \mu[L], \theta_{\mathcal{F}}[L]\right\rangle_{[\varnothing]}$$

which is equal to $\mu[L] \in \mathcal{F}$. Thus, if and only if $\mu[L] \in \mathcal{F}$ then $\mathbb{P}\left[X_{[d]}\right]$ is only supported at $x_{[d]}$ in the support of $\nu^{t,\mathcal{F}}\left[X_{[d]}\right]$. $\square$

Let us now investigate tensor network representations of refined base measures, based on the basis encoding $\beta^t$ of a statistic.

**Theorem 14.1** *For any face $\mathcal{F}$ of $\mathcal{M}$ we have*

$$\nu^{t,\mathcal{F}}\left[X_{[d]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \kappa^{\mathcal{F}}\left[Y_{[p]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

*where*

$$\kappa^{\mathcal{F}}\left[Y_{[p]}\right] = \sum_{v[L]\in\mathcal{N}^{\mathcal{F}}_{t,\nu}} \epsilon_{v[L]}\left[Y_{[p]}\right].$$

*Proof.* For any $v[L] \in \mathcal{N}^{\mathcal{F}}_{t,\nu}$ the tensor

$$\tau^v\left[X_{[d]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \epsilon_{v[L]}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

is the indicator of the preimage of $\mu$ under $\sigma^t$. Since preimages of the elements in $\mathcal{N}_{t,\nu}^{\mathcal{F}}$ are disjoint, the support of $\tau^v\left[X_{[d]}\right]$ is disjoint and their sum

$$\sum_{v[L]\in\mathcal{N}_{t,\nu}^{\mathcal{F}}} \tau^v\left[X_{[d]}\right]$$

is the indicator of the preimage of $\mathcal{F}$ under $\sigma^t$. The face measure obeys thus

$$\begin{aligned}\nu^{t,\mathcal{F}}\left[X_{[d]}\right] &= \left\langle\left(\sum_{v[L]\in\mathcal{N}_{t,\nu}^{\mathcal{F}}} \tau^v\left[X_{[d]}\right]\right), \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} \\ &= \sum_{v[L]\in\mathcal{N}_{t,\nu}^{\mathcal{F}}} \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \epsilon_{v[L]}\left[Y_{[p]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} \\ &= \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \kappa^{\mathcal{F}}\left[Y_{[p]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} \square\end{aligned}$$

We now investigate the representation of refined base measures by Computation-Activation Networks.

**Definition 14.3** Let $\mathcal{G}$ be a hypergraph whose nodes include $[p]$. We say that a face $\mathcal{F}$ is representable by $\mathcal{G}$ if and only if there is a set $\mathcal{U}$ of basis vectors with $\mathcal{U}\cap\mathcal{N}_{t,\nu} = \mathcal{N}_{t,\nu}^{\mathcal{F}}$ and there is a tensor network $\kappa^{\mathcal{G}}$ with respect to the hypergraph $\mathcal{G}$ such that

$$\left\langle\kappa^{\mathcal{G}}\right\rangle_{\left[Y_{[p]}\right]} = \sum_{v\in\mathcal{U}} \epsilon_v\left[Y_{[p]}\right] .$$

We call any such tensor network a face activating tensor network.

**Lemma 14.3** *If and only if a face is representable by a hypergraph $\mathcal{G}$, we have*

$$\frac{1}{\left\langle\nu^{t,\mathcal{F}}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}} \cdot \nu^{t,\mathcal{F}}\left[X_{[d]}\right] \in \Lambda^{t,\mathcal{G},\nu} .$$

*Proof.* For any $\mathcal{U}$ with $\mathcal{U}$ : $\mathcal{U}\cap\mathcal{N}_{t,\nu} = \mathcal{N}_{t,\nu}^{\mathcal{F}}$ and tensor network $\kappa^{\mathcal{G}}$ respecting the assumptions of Def. 14.3 we have

$$\begin{aligned}\left\langle\left\langle\kappa^{\mathcal{G}}\right\rangle_{\left[Y_{[p]}\right]}, \beta^t\left[Y_{[p]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} &= \left\langle\sum_{v\in\mathcal{U}} \epsilon_v\left[Y_{[p]}\right], \beta^t\left[Y_{[p]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} \\ &= \left\langle\sum_{v\in\mathcal{N}_{t,\nu}^{\mathcal{F}}} \epsilon_v\left[Y_{[p]}\right], \beta^t\left[Y_{[p]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} .\end{aligned}$$

Here we used in the second equation that only the vertices in $\mathcal{N}_{t,\nu}$ are in the image of $t$. It follows, that

$$\frac{1}{\left\langle\nu^{t,\mathcal{F}}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}} \cdot \nu^{t,\mathcal{F}}\left[X_{[d]}\right] = \frac{\left\langle\left\langle\kappa^{\mathcal{G}}\right\rangle_{\left[Y_{[p]}\right]}, \beta^t\left[Y_{[p]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}}{\left\langle\left\langle\kappa^{\mathcal{G}}\right\rangle_{\left[Y_{[p]}\right]}, \beta^t\left[Y_{[p]}, X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}}$$

and thus $\nu^{t,\mathcal{F}}\left[X_{[d]}|\varnothing\right] \in \Lambda^{t,\mathcal{G},\nu}$. □

*Example 14.1 (Vertices)* Vertices $\mathcal{F} = \{v\,[L]\}$ are proper faces of affine dimension 0, that is they consist of single vectors $v\,[L]$. Since each vertex is in the image $\mathcal{N}_{t,\nu}$, there exists an index tuple $x_{[d]} \in \mathcal{X}$ such that $\nu\left[X_{[d]} = x_{[d]}\right] = 1$ and

$$v\,[L] = \sigma^t\left[X_{[d]} = x_{[d]}, L\right] .$$

Then $\kappa^{\mathcal{F}}\left[Y_{[p]}\right]$ is the one-hot encoding of the vector $\sigma^t\left[X_{[d]} = x_{[d]}, L\right]$, that is

$$\kappa^{\mathcal{F}}\left[Y_{[p]}\right] = \epsilon_{\sigma^t\left[X_{[d]}=x_{[d]},L\right]}\left[Y_{[p]}\right] = \left\langle \{\epsilon_{\sigma^t\left[X_{[d]}=x_{[d]},L=\ell\right]}\left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{\left[Y_{[p]}\right]} .$$

Such tensors are examples of elementary tensors corresponding to the elementary hypergraph (see Figure 14.1)

$$\mathrm{EL} = \left([p], \{\{\ell\} \,:\, \ell \in [p]\}\right).$$

In particular, the activation core is elementary and the corresponding refined base measure is in $\Lambda^{t,\mathrm{EL},\nu}$.

While vertices are the minimal non-vanishing faces in the face-lattice (see [84]), we now show that the maximal face, namely the polytope itself, is also representable with respect to the elementary hypergraph EL (see Figure 14.1).

*Example 14.2 (Maximal face)* The maximal face $\mathcal{F}^{\varnothing} = \mathcal{M}_{t,\nu}$ coincides with the mean polytope itself and is given by the choice $\theta_{\varnothing}[L] = 0[L]$. In this case the corresponding activation tensor to the face measure is

$$\kappa^{\varnothing}\left[Y_{[p]}\right] = \mathbb{I}\left[Y_{[p]}\right] = \bigotimes_{\ell\in[p]} \mathbb{I}\left[Y_\ell\right] ,$$

where by $\mathbb{I}\left[Y_{[p]}\right]$ we denote tensors with all coordinates 1. $\kappa^{\varnothing}Y_{[p]}$ is elementary and the maximal face is always representable by an elementary activation.

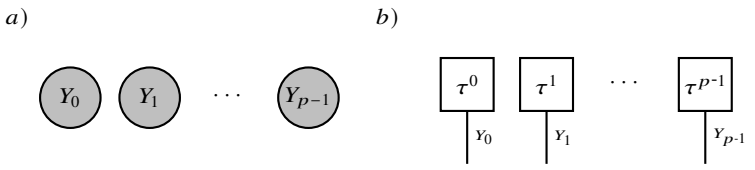



**Fig. 14.1** Elementary tensor format consisting of the elementary hypergraph EL $a$), and corresponding tensor networks $b$) being the tensor products of vectors.

### 14.2.2 Representation in the CP format

Extending Example 14.1 and Example 14.2, we can provide a coarse estimation of the hypergraph $\mathcal{G}$ required to decompose $\kappa^{\mathcal{F}}$ for generic faces $\mathcal{F}$. To this end, we use the CP format describing tensor networks on the CP hypergraph (see Figure 14.2)

$$\mathrm{CP} = \left([p] \cup \{I\}, \{\{\ell, I\} \,:\, \ell \in [p]\}\right).$$

The dimension of the variable $I$ is called the hidden rank of the format.

**Lemma 14.4** *Any face $\mathcal{F}$ is representable by a* CP *graph with hidden rank*

$$r \leq \min\left(\left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right|, \left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right| + 1\right) .$$

*Proof.* We show the claim by constructing two face activating tensor networks to $\mathcal{F}$ in a CP hypergraph with hidden rank $\left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right|$ and in a CP hypergraph with hidden rank $\left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right| + 1$. To show the first representation we enumerate the vertices by a variable $I$ with dimension $r = \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right|$, i.e. $\mathcal{N}_{t,\nu}^{\mathcal{F}} = \{v^i[L] \,:\, i \in [n]\}$. Then we define for $\ell \in [p]$ core tensors $\tau^\ell\left[Y_\ell, I\right]$

$$\tau^\ell\left[Y_\ell, I = i\right] = \epsilon_{v^i[L=\ell]}\left[Y_\ell\right] .$$

Then we have

$$\left\langle \{\tau^{\ell}\left[Y_{\ell}, I\right] \,:\, \ell \in [p]\}\right\rangle_{\left[Y_{[p]}\right]} = \sum_{v \in \mathcal{N}_{t,\nu}^{\mathcal{F}}} \epsilon_v\left[Y_{[p]}\right]$$

and thus have found an activation tensor network in a CP graph with hidden rank $\left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right|$ representing the face $\mathcal{F}$.

We continue with the second representation, for which we enumerate the set $\mathcal{N}_{t,\nu}/\mathcal{N}_{t,\nu}^{\mathcal{F}}$ by $v^i\left[L\right]$ where $i \in [\left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right|]$. We define variable $I$ with dimension $r = \left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right| + 1$ and define for $\ell \in [p]$ core tensors

$$\tau^{\ell}\left[Y_{\ell}, I = i\right] = \begin{cases} -\epsilon_{v^i[L=\ell]}\left[Y_{\ell}\right] & \text{if} \quad i < \left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right| \\ \mathbb{I}\left[Y_{\ell}\right] & \text{if} \quad i = \left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right| \end{cases} .$$

We then have

$$\begin{aligned} \left\langle \{\tau^{\ell}\left[Y_{\ell}, I\right] \,:\, \ell \in [p]\}\right\rangle_{\left[Y_{[p]}\right]} &= \mathbb{I}\left[Y_{[p]}\right] - \sum_{v \in \mathcal{N}_{t,\nu}/\mathcal{N}_{t,\nu}^{\mathcal{F}}} \epsilon_v\left[Y_{[p]}\right] \\ &= \sum_{v \in \mathcal{N}_{t,\nu}^{\mathcal{F}}} \epsilon_v\left[Y_{[p]}\right] + \sum_{v \in \left(\bigtimes_{\ell \in [p]}[p_{\ell}]\right)/\mathcal{N}_{t,\nu}} \epsilon_v\left[Y_{[p]}\right] . \end{aligned}$$

We have thus found a face activating tensor network for $\mathcal{F}$ in a CP format with hidden rank $r = \left|\mathcal{N}_{t,\nu}\right| - \left|\mathcal{N}_{t,\nu}^{\mathcal{F}}\right| + 1$. □

We notice that the CP rank bound of Lem. 14.4 is tight in Example 14.1 and Example 14.2, since the elementary format is a restriction of the CP format to rank 1.

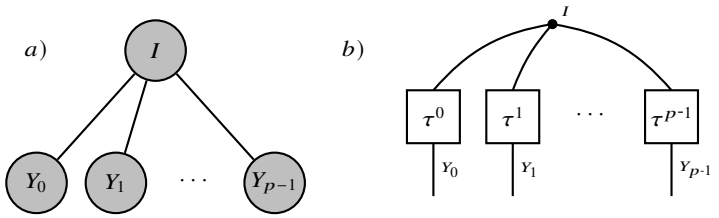



**Fig. 14.2** The CP tensor format consist of the CP hypergraph a) and decorating tensor networks b). Tensors in this format constructed as in Lem. 14.4 represent arbitrary faces.

### 14.2.3 Examples of elementarily representable faces

We show in the following example that all faces of a hypercube and a simplex are representable by elementary activation tensors.

*Example 14.3 (Hypercube)* In cases where $X_{[d]}$ are Boolean and we have $\ell \in [p]$ features

$$f_{\ell}\left[X_{[d]} = x_{[d]}\right] = x_{\ell}$$

the mean polytope is the hypercube

$$\mathcal{M}_{\{f_{\ell} \,:\, \ell \in [p]\},\mathbb{I}} = [0,1]^p .$$

The non-empty faces in the face lattice $L\left([0,1]^p\right)$ can be enumerated by subsets $\mathcal{U} \subset [p]$ and indices $y_{\mathcal{U}} \in \bigtimes_{\ell \in \mathcal{U}}[2]$ and represented by the cartesian products

$$\mathcal{F}^{(\mathcal{U},y_{\mathcal{U}})} = \bigtimes_{\ell\in[p]} \mathcal{I}^{l,(\mathcal{U},y_{\mathcal{U}})}$$

where

$$\mathcal{I}^{l,(\mathcal{U},y_{\mathcal{U}})} = \begin{cases} [0,1] & \text{if} \quad \ell \notin \mathcal{U} \\ \{y_\ell\} & \text{if} \quad \ell \in \mathcal{U} \end{cases}.$$

Each of these faces can be represented with respect to the elementary graph EL, namely by the tensor product of leg vectors

$$\kappa^{\ell,(\mathcal{U},y_{\mathcal{U}})}\left[Y_\ell\right] = \begin{cases} \mathbb{I}\left[Y_\ell\right] & \text{if} \quad \ell \notin \mathcal{U} \\ \epsilon_{y_\ell}\left[Y_\ell\right] & \text{if} \quad \ell \in \mathcal{U} \end{cases}.$$

*Example 14.4 (Simplices)* The $(p-1)$-dimensional standard simplex $\mathcal{M}_{\triangle,p-1}$ in $[0,1]^p$ is the convex hull of the vertex sets

$$\mathcal{N} = \{\epsilon_\ell\left[L\right] \,:\, \ell \in [p]\}\,.$$

The simplex is the mean polytope of the $p$-dimensional statistic with

$$t_\ell \,:\, [p] \to [2] \quad , \quad t_\ell\left[\tilde{\ell}\right] = \begin{cases} 1 & \text{if} \quad \tilde{\ell} = \ell \\ 0 & \text{if} \quad \tilde{\ell} \neq \ell \end{cases}.$$

The simplex represents all distributions with respect to the trivial base measure $\mathbb{I}$ which coordinates are all 1. When flattened to a vector, any tensor in $\Lambda^{\mathbb{I}}$ is an element of $\mathcal{M}_{\triangle,p-1}$ where $p = \prod_{k\in[d]} m_k$.

The face lattice of the simplex is enumerated by subsets of $[p]$ as

$$L\left(\mathcal{M}_{\triangle,p-1}\right) = \{\mathcal{F}^{\mathcal{U}} \,:\, \mathcal{U} \subset [p]\}$$

where the faces are

$$\mathcal{F}^{\mathcal{U}} = \text{conv}\left(\epsilon_\ell\left[L\right] \,:\, \ell \in \mathcal{U}\right)$$

and are therefore themselves $(|\mathcal{U}|-1)$-dimensional standard simplices. The partial order of the faces coincides with the inclusion order of subsets $\mathcal{U}$.

### 14.2.4 Partition into relative interiors of faces

Let us now introduce relative interiors, which enable us to find disjoint partitions of the mean polytope.

**Definition 14.4 (Relative Interior)** The relative interior $(\mathcal{U})^\circ$ of an arbitrary set $\mathcal{U} \subset \mathbb{R}^p$ is the interior of $\mathcal{U}$ in the affine subspace conv $(\mathcal{U})$.

**Lemma 14.5** *To each $\mu \in \mathcal{M}_{t,\nu}$ there is a unique face $\mathcal{F}$, which we denote by $\mathcal{F}(\mu)$, such that $\mu \in (\mathcal{F})^\circ$.*

*Proof.* For any mean vector $\mu$ on a face $\mathcal{F}$ we either have $\mu \in (\mathcal{F})^\circ$ or there is a face $\tilde{\mathcal{F}}$ of lower affine dimension with $\tilde{\mathcal{F}} \subset \mathcal{F}$ and $\mu \in \tilde{\mathcal{F}}$. Starting from the face $\mathcal{M}_{t,\nu}$ (for which by assumption $\mu \in \mathcal{M}_{t,\nu}$) we iteratively choose a face $\mathcal{F}$ of lower dimension containing $\mu$ until we reach a face with $\mu \in (\mathcal{F})^\circ$. This procedure is guaranteed to terminate, since the affine dimension is decreasing and any face of affine dimension 0 coincides with its relative interior. □

## 14.3 Exponential families

It is well known that exponential families are specific maximum entropy distributions. However, not every maximum entropy distribution is an exponential family member. To prepare for the characterization of generic maximum entropy distributions, we will in this section represent exponential family members as Computation-Activation Networks. We further show that a maximum entropy distribution is an exponential family member if and only if its mean parameter is in the relative interior of the mean polytope.

### 14.3.1 Representation as elementary Computation-Activation Networks

Fusion with the definition in previous section: Resolve the base measure difference.

Let us first define exponential families as classes of probability distributions and then derive their representation by elementary Computation-Activation Networks.

**Definition 14.5 (Exponential Family)** Given a statistic function

$$t : \bigtimes_{k \in [d]} [m_k] \to \mathbb{R}^p$$

and a base measure $\nu\left[X_{[d]}\right]$ with $\langle \nu \rangle_{[\varnothing]} \neq 0$, the set $\Gamma^{t,\nu} = \{\mathbb{P}^{(t,\theta,\nu)} : \theta\,[L] \in \mathbb{R}^p\} \subset \Lambda^{\nu}$ of probability distributions

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \frac{\exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta\,[L]\right\rangle_{\left[X_{[d]}\right]}\right]}{\left\langle \exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta\,[L]\right\rangle_{\left[X_{[d]}\right]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}}$$

is called the exponential family with respect to $t$.

To connect with standard notation of exponential families we notice that the coordinate of any distribution $\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right]$ at some state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ is

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]} = x_{[d]}\right] = \exp\left[\sum_{\ell \in [p]} \theta\,[L = \ell] \cdot t_\ell\left[x_{[d]}\right] - \psi_{t,\nu}(\theta)\right]$$

where

$$\psi_{t,\nu}(\theta) = \ln\left[\left\langle \exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta\,[L]\right\rangle_{\left[X_{[d]}\right]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\right]$$

is the cumulant generating function. The function $\psi_{t,\nu}(\theta)$ and its Legendre transform $\psi^*_{t,\nu}(\mu)$ define the dually flat Riemannian structure on the exponential family manifold [1]. We now continue with the notation of Def. 14.5 to find representations of exponential families by elementary Computation-Activation Networks.

**Theorem 14.2 (Exponential Families are elementary Computation-Activation Networks)** *Given any base measure $\nu$ and a sufficient statistic $t$ we enumerate for each coordinate $\ell \in [p]$ the image* $\mathrm{im}\,(t_\ell)$ *by a variable $Y_\ell$ taking values in* $[|\mathrm{im}\,(t_\ell)|]$*, given an interpretation map*

$$I_\ell : [|\mathrm{im}\,(t_\ell)|] \to \mathrm{im}\,(t_\ell)\ .$$

*For any canonical parameter vector $\theta\,[L] \in \mathbb{R}^p$ we build the activation cores $\alpha^{\ell,\theta}\,[Y_\ell]$ for each coordinate $y_\ell \in [|\mathrm{im}\,(t_\ell)|]$ by*

$$\alpha^{\ell,\theta}\,[Y_\ell = y_\ell] = \exp\left[\theta\,[L = \ell] \cdot I_\ell\,[y_\ell]\right]$$

*and have (see Figure 14.3)*

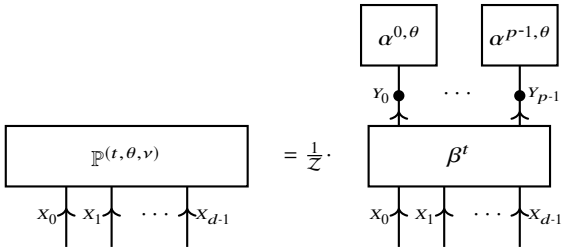



**Fig. 14.3** Representation of a member of an exponential family by a Computation-Activation Network with elementary activation. Here $\mathcal{Z} \in \mathbb{R}$ is a normalization constant.

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \frac{\left\langle \{\beta^t\left[Y_{[p]}, X_{[d]}\right]\} \cup \{\alpha^{\ell,\theta}\left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{[X_{[d]}]}}{\left\langle \{\beta^t\left[Y_{[p]}, X_{[d]}\right]\} \cup \{\alpha^{\ell,\theta}\left[Y_\ell\right] \,:\, \ell \in [p]\} \cup \{\nu\left[X_{[d]}\right]\}\right\rangle_{[\varnothing]}} .$$

*Proof.* For each $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have

$$\begin{aligned}
&\left\langle \{\beta^t\left[Y_{[p]}, X_{[d]}\right]\} \cup \{\alpha^{\ell,\theta}\left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{[X_{[d]}=x_{[d]}]} \\
&= \prod_{\ell\in[p]} \left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]} = x_{[d]}\right], \alpha^{\ell,\theta}\left[Y_\ell\right]\right\rangle_{[\varnothing]} \\
&= \prod_{\ell\in[p]} \exp\left[\theta\left[L=\ell\right] \cdot I_\ell\left[t_\ell\left[x_{[d]}\right]\right]\right] \\
&= \exp\left[\sum_{\ell\in[p]} \theta\left[L=\ell\right] \cdot I_\ell\left[t_\ell\left[x_{[d]}\right]\right]\right] \\
&= \exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta\left[L\right]\right\rangle_{[X_{[d]}=x_{[d]}]}\right] .
\end{aligned}$$

Therefore we have

$$\left\langle \{\beta^t\left[Y_{[p]}, X_{[d]}\right]\} \cup \{\alpha^{\ell,\theta}\left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{[X_{[d]}]} = \exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta\left[L\right]\right\rangle_{[X_{[d]}]}\right] .$$

The claim follows, since this implies that also the contraction of both sides with $\nu\left[X_{[d]}\right]$ is equivalent. □

We will use the following well known property (see e.g. [10]), that the mean parameters of the members of an exponential family are exactly the relative interior of the mean polytope.

**Lemma 14.6** *The set of mean parameters of the members of an exponential family is the relative interior of the mean polytope.*

*Proof.* See The 3.3 in [82]. □

Based on this property we define the forward and backward mappings to an exponential family.

**Definition 14.6** The forward map of an exponential family is the map

$$F^{(t,\nu)} \,:\, \mathbb{R}^p \to \left(\mathcal{M}_{t,\nu}\right)^\circ$$

defined as

$$F^{(t,\nu)}(\theta) = \left\langle \mathbb{P}^{t,\theta,\nu}[X_{[d]}], \sigma^t\left[X_{[d]}, L\right], \nu\left[X_{[d]}\right]\right\rangle_{[L]}$$

Any map $B^{(t,\nu)} : \left(\mathcal{M}_{t,\nu}\right)^\circ \to \mathbb{R}^p$ with $F^{(t,\nu)}(B^{(t,\nu)}(\mu)) = \mu$ for all $\mu \in \left(\mathcal{M}_{t,\nu}\right)^\circ$ is called a backward map.

### 14.3.2 Maximum entropy distributions

One universal property of exponential families amongst others (see [56]) is that they maximize the entropy among those with the same mean vector.

**Lemma 14.7** *Any member of an exponential family $\Gamma^{t,\nu}$ is a maximum entropy distribution with respect to the moment constraint on $t$ and the base measure $\nu$.*

*Proof.* Let $\mathbb{P}^{t,\theta,\nu}$ be an exponential family member with canonical parameter $\theta$ and let $\mu[L]$ be the corresponding mean parameter. Let $\tilde{\mathbb{P}}[X_{[d]}]$ be any feasible distribution for the maximum entropy problem with respect to the moment constraint on $t$ by $\mu[L]$ and the base measure $\nu$. We also have $\left\langle \tilde{\mathbb{P}}[X_{[d]}], \sigma^t\left[X_{[d]}, L\right], \nu\left[X_{[d]}\right]\right\rangle_{[L]} = \mu[L]$ and thus

$$\begin{aligned}
\mathbb{H}_\nu\left[\tilde{\mathbb{P}}, \mathbb{P}^{t,\theta,\nu}\right] &= -\left\langle \tilde{\mathbb{P}}[X_{[d]}], \ln\left[\mathbb{P}^{t,\theta,\nu}[X_{[d]}]\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\
&= -\left\langle \tilde{\mathbb{P}}[X_{[d]}], \sigma^t\left[X_{[d]}, L\right], \theta[L], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} + A^{(t,\nu)}(\theta) \\
&= -\langle \theta[L], \mu[L]\rangle_{[\varnothing]} + A^{(t,\nu)}(\theta) \\
&= \mathbb{H}_\nu\left[\mathbb{P}^{t,\theta,\nu}\right] .
\end{aligned}$$

With the Gibbs inequality we have if $\tilde{\mathbb{P}} \neq \mathbb{P}^{t,\theta,\nu}$

$$\mathbb{H}_\nu\left[\mathbb{P}^{t,\theta,\nu}\right] - \mathbb{H}_\nu\left[\tilde{\mathbb{P}}\right] = \mathbb{H}_\nu\left[\tilde{\mathbb{P}}, \mathbb{P}^{t,\theta,\nu}\right] - \mathbb{H}_\nu\left[\tilde{\mathbb{P}}\right] > 0$$

and thus $\mathbb{H}_\nu\left[\tilde{\mathbb{P}}\right] < \mathbb{H}_\nu\left[\mathbb{P}^{t,\theta,\nu}\right]$. Therefore, any feasible $\tilde{\mathbb{P}}$ different from $\mathbb{P}^{t,\theta,\nu}$ has less entropy than $\mathbb{P}^{t,\theta,\nu}$ and $\mathbb{P}^{t,\theta,\nu}$ is a maximum entropy distribution. □

Together with Lem. 14.6 stating the existence of an exponential family member for any mean parameter in the relative interior of the mean polytope we arrive at the following theorem.

**Theorem 14.3** *If and only if $\mu[L]$ is in the relative interior of $\mathcal{M}_{t,\nu}$, then the unique solution of the maximum entropy problem is the distribution*

$$\mathbb{P}^{t,\theta,\nu}[X_{[d]}] \in \Gamma^{t,\nu}$$

*where for some backward map $B^{t,\nu}$*

$$\theta = B^{t,\nu}(\mu) .$$

*Proof.* By Lem. 14.6 if

$$\mu[L] \in \left(\mathcal{M}_{t,\nu}\right)^\circ ,$$

there is at least one weight parameter $\theta[L] \in \mathbb{R}^p$ with

$$\left\langle \mathbb{P}^{t,\theta,\nu}[X_{[d]}], \sigma^t\left[X_{[d]}, L\right], \nu\left[X_{[d]}\right]\right\rangle_{[L]} = \mu[L]$$

and $B^{t,\nu}(\mu)$ with respect to any backward map $B^{t,\nu}$ is such one. With Lem. 14.7 we conclude that $\mathbb{P}^{(t,B^{t,\nu}(\mu),\nu)}$ is the maximum entropy distribution with respect to the moment constraint on $t$ by $\mu[L]$ and the base measure $\nu$. □

While the maximum entropy distribution is unique, the weight vector $\theta$ representing the maximum entropy distribution is not always unique. This is reflected by multiple possible choices of backward mappings. Only if the exponential family is minimal (see [82]) then the weight vector is unique. However, we did not restrict to such situations, since exponential families on refined base measures are not minimal.

## 14.4 Generic maximum entropy distributions

In the previous section we have shown that the members of an exponential family are maximum entropy distributions. However, while their mean parameters constitute the relative interior of the mean polytope, they never lie on faces of lower dimension. We now exploit the tensor representation study of such faces (see Sect. 14.2) to characterize the maximum entropy distributions in the general case.

### 14.4.1 Main result

Our main result generalizes the maximum entropy characterization of Thm. 14.3 to arbitrary mean parameters.

**Theorem 14.4 (Generic characterization of Maximum Entropy Solutions)** *Let $t$ be a statistic and $\nu$ a base measure. For any $\theta\left[L\right]$ and $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ the distribution*

$$\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}}\left[X_{[d]}\right]$$

*is a maximum entropy distribution.*
*For any $\mu\left[L\right]$ the maximum entropy problem has a feasible distribution, if and only if $\mu\left[L\right] \in \mathcal{M}_{t,\nu}$. If $\mu\left[L\right] \in \mathcal{M}_{t,\nu}$ the unique maximum entropy distribution is $\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}}$ where $\mathcal{F} = \mathcal{F}(\mu)$ (see Lem. 14.5) and*

$$\theta = B^{t,\nu^{t,\mathcal{F}}}(\mu)$$

*where $\nu^{t,\mathcal{F}}$ is the refined base measure (see Def. 14.2) and $B^{t,\nu^{t,\mathcal{F}}}$ is any backward map to the exponential family $\Gamma^{t,\nu^{t,\mathcal{F}}}$ (see Def. 14.6).*
*For any hypergraph $\mathcal{G}$ and any face $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ we have that all maximum entropy distributions to mean parameters on $\mathcal{F}^\circ$ are in $\Lambda^{t,\mathcal{G},\nu}$ if $\mathcal{F}$ is representable with respect to $\mathcal{G}$ (see Def. 14.3).*

To prepare for the proof of this theorem we first show in an auxiliary lemma that we can reduce the set of feasible distributions in Problem $\mathrm{P}_{t,\mu,\nu}$.

**Lemma 14.8** *For any $\mu\left[L\right] \in \mathcal{M}_{t,\nu}$ and a face $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ with $\mu\left[L\right] \in (\mathcal{F})^\circ$ the solutions of $\mathrm{P}_{t,\mu,\nu}$ and $\mathrm{P}_{t,\mu,\nu^{t,\mathcal{F}}}$ coincide.*

*Proof.* By Lem. 14.2 all feasible distributions are representable by the base measure refined to the face $\mathcal{F}$. We have that any for $\mathrm{P}_{t,\mu,\nu}$ feasible distribution $\mathbb{P}\left[X_{[d]}\right]$ satisfies

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \nu^{t,\mathcal{F}} \right\rangle_{[\varnothing]} = 1$$

and thus $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu^{t,\mathcal{F}}}$. Conversely, any $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu^{t,\mathcal{F}}}$ satisfies

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \nu^{t,\mathcal{F}} \right\rangle_{[\varnothing]} = \left\langle \mathbb{P}\left[X_{[d]}\right], \nu \right\rangle_{[\varnothing]} = 1$$

and thus $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu}$. Problem $\mathrm{P}_{t,\mu,\nu}$ is thus equal to

$$\operatorname{argmax}_{\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu^{t,\mathcal{F}}}} \mathbb{H}_\nu\left[\mathbb{P}\left[X_{[d]}\right]\right] \quad \text{subject to} \quad \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], \nu\left[X_{[d]}\right] \right\rangle_{[L]} = \mu^*[L]$$

We further have that any $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\nu^{t,\mathcal{F}}}$

$$\mathbb{H}_\nu\left[\mathbb{P}\left[X_{[d]}\right]\right] = \mathbb{H}_{\nu^{t,\mathcal{F}}}\left[\mathbb{P}\left[X_{[d]}\right]\right]$$

and arrive together with the above equivalence at the claim. □

*Proof of Thm. 14.4.* To show that for any $\theta\left[L\right]$ and $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ the distribution $\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}}$ is a maximum entropy distribution, we apply Lem. 14.8 and Thm. 14.3.

If $\mu[L] \notin \mathcal{M}_{t,\nu}$ then Problem $\mathrm{P}_{t,\mu,\nu}$ is not feasible by definition. In case $\mu[L] \in \mathcal{M}_{t,\nu}$ we find $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ with $\mathcal{F} = \mathcal{F}(\mu)$ and by Lem. 14.8 the solution of maximum entropy problem Problem $\mathrm{P}_{t,\mu,\nu}$ coincides with $\mathrm{P}_{t,\mu,\nu^{t,\mathcal{F}}}$. Since by Lem. 14.1 the face is the polytope

$$\mathcal{F} = \mathcal{M}_{t,\nu^{t,\mathcal{F}}}$$

we find a weight parameter

$$\theta = B^{t,\nu^{t,\mathcal{F}}}(\mu)\,.$$

and by Thm. 14.3 $\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}}$ is the corresponding maximum entropy distribution.
To show the last claim, it is enough to show that for any face $\mathcal{F}$ representable by $\mathcal{G}$ and for any $\theta \in \mathbb{R}^p$ we have

$$\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}} \in \Lambda^{t,\mathcal{G},\nu}\,.$$

To this end, we construct activation cores as follows. By Def. 14.3 and Lem. 14.3 there is a tensor network $\kappa^{\mathcal{G}}$ such that

$$\nu^{t,\mathcal{F}}\left[X_{[d]}\right] = \left\langle \kappa^{\mathcal{G}} \cup \{\nu\left[X_{[d]}\right]\}\right\rangle_{\left[X_{[d]}\right]}\,.$$

For each $\ell \in [p]$ we find an edge $e(\ell) \in \mathcal{E}$ with $\ell \in e(\ell)$. Now we define a tensor network $\tau^{\mathcal{G}}$ by the cores to $e \in \mathcal{E}$ as

$$\tau^{e}\left[X_e\right] = \left\langle \{\kappa^{e}\left[X_e\right]\} \cup \{\alpha^{\ell,\theta}\left[Y_\ell\right] \,:\, e = e(\ell)\}\right\rangle_{\left[X_e\right]}\,.$$

Then, we have

$$\left\langle \beta^{t}\left[Y_{[p]}, X_{[d]}\right], \left\langle \tau^{\mathcal{G}}\right\rangle_{\left[Y_{[p]}\right]}\right\rangle_{\left[X_{[d]}\right]} = \left\langle \beta^{t}\left[Y_{[p]}, X_{[d]}\right], \alpha^{\theta}\left[Y_{[p]}\right], \left\langle \kappa^{\mathcal{G}}\right\rangle_{\left[Y_{[p]}\right]}\right\rangle_{\left[X_{[d]}\right]}$$

and therefore

$$\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}} = \frac{\left\langle \{\beta^{t}\left[Y_{[p]}, X_{[d]}\right]\} \cup \tau^{\mathcal{G}}\right\rangle_{\left[X_{[d]}\right]}}{\left\langle \{\beta^{t}\left[Y_{[p]}, X_{[d]}\right], \nu\left[X_{[d]}\right]\} \cup \tau^{\mathcal{G}}\right\rangle_{[\varnothing]}}\,.$$

We conclude $\mathbb{P}^{t,\theta,\nu^{t,\mathcal{F}}} \in \Lambda^{t,\mathcal{G},\nu}$. □

### 14.4.2 Family of maximum entropy distributions

The family of maximum entropy distributions is the set

$$\Gamma^{t,\nu} = \{\mathbb{P}\left[X_{[d]}\right] \,:\, \exists \mu \in \mathcal{M}_{t,\nu} : \mathbb{P}\left[X_{[d]}\right] \quad \text{is a solution of Problem } \mathrm{P}_{t,\mu,\nu}\}\,.$$

Thm. 14.4 suggests a parametrization of this family as tuples of corresponding faces $\mathcal{F}$ and weight vectors $\theta[L]$. As sketched in Figure 14.4 the activation tensor of each maximum entropy distribution is decomposed into a possibly non-elementary Boolean tensor corresponding to the base measure refinement on a face $\mathcal{F}$, and an elementary tensor dependent on the weight vector $\theta \in \mathbb{R}^p$. We thus define the canonical parameter space

$$\mathcal{P}_{t,\nu} := L\left(\mathcal{M}_{t,\nu}\right) \times \mathbb{R}^p$$

and have

$$\Gamma^{t,\nu} = \left\{ \frac{\left\langle \alpha^{\theta}\left[Y_{[p]}\right], \kappa^{\mathcal{F}}\left[Y_{[p]}\right], \beta^{t}\left[Y_{[p]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}}{\left\langle \alpha^{\theta}\left[Y_{[p]}\right], \kappa^{\mathcal{F}}\left[Y_{[p]}\right], \beta^{t}\left[Y_{[p]}, X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}} \,:\, \theta[L] \in \mathbb{R}^p, \mathcal{F} \in \mathcal{P}_{t,\nu} \right\}\,.$$

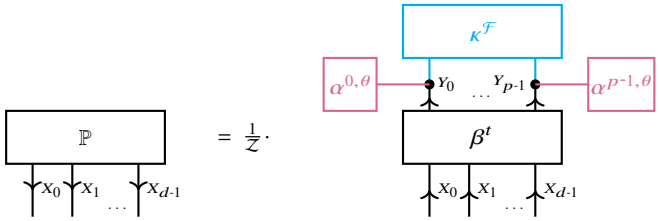



**Fig. 14.4** Tensor network decomposition of maximum entropy distributions to the constraint $\mu[L] = \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]}$. Blue: Boolean tensor $\kappa^{\mathcal{F}}\left[Y_{[p]}\right]$ dependent on the face $\mathcal{F}(\mu)$, which represents the refinement of the base measure to the face, and is possibly non-elementary. Red: Elementary activation tensor $\alpha^{\theta}\left[Y_{[p]}\right] = \bigotimes_{s\in[n]} \alpha^{\ell,\theta}[Y_\ell]$ to the weight $\theta[L] = B^{t,\nu^{t,\mathcal{F}}}(\mu)$.

Note that these families are disjoint, since each member of an exponential family has support by the support of the face measure.

## 14.5 Boolean statistics

We say that a statistic is Boolean, if $\mathrm{im}(t_\ell) \subset \{0,1\}$ for all $\ell \in [p]$. The mean polytope is then a 0/1-polytope [83], which vertices are exactly the contained Boolean vectors. The special case of Boolean statistics is especially interesting for two reasons. First, we can provide additional intuition on those faces representable by elementary face activating tensors, namely based on intersections with cube faces. Based on these properties we also show that any elementary Computation-Activation Network to a Boolean statistic is a maximum entropy distribution. Second, the statistic itself can be interpreted in terms of logical formulas. We exploit this property to provide examples and to characterize each 0/1-polytope as a mean polytope to a collection of propositional formulas.

### 14.5.1 Characterization of elementary representable faces

Orienting on Example 14.3 we now introduce cube-likeness of faces and polytopes.

**Definition 14.7** We say that a face $\mathcal{F}$ of $\mathcal{M}_{t,\nu}$ is cube-like, if it is empty or there is $\mathcal{U} \subset [p]$ and $y_{\mathcal{U}} \in \bigtimes_{\ell\in\mathcal{U}}[2]$ such that

$$\mathcal{F} = \mathcal{M}_{t,\nu} \cap \mathcal{F}^{(\mathcal{U},y_{\mathcal{U}})}\,.$$

Here we denote by $\mathcal{F}^{(\mathcal{U},y_{\mathcal{U}})}$ the faces of the hypercube $[0,1]^p$ (see Example 14.3). We further say that a polytope $\mathcal{M}_{t,\nu}$ is cube-like, if all faces $\mathcal{F} \in L\left(\mathcal{M}_{t,\nu}\right)$ are cube-like.

We now show that a face is cube-like if and only if it is representable by an elementary tensor.

**Theorem 14.5** *Let $t$ be a Boolean statistic, $\nu$ a base measure and $\mathcal{F}$ be a face of $\mathcal{M}_{t,\nu}$. Then the following are equivalent:*

*(i) $\mathcal{F}$ is cube-like (see Def. 14.7).*
*(ii)$\mathcal{F}$ is representable by an elementary tensor (see Def. 14.3).*

*Proof.* If the face is empty, i.e. $\mathcal{F} = \emptyset$, it is by definition cube-like and has $0\left[Y_{[p]}\right]$ as an elementary activation tensor. We therefore assume in the following $\mathcal{F} \neq \emptyset$.

(i)⇒(ii): Let us assume that $\mathcal{F}$ is cube-like, that is there is $\mathcal{U} \subset [p]$ and $y_{\mathcal{U}} \in \bigtimes_{\ell\in\mathcal{U}}[2]$ such that $\mathcal{F} = \mathcal{M}_{t,\nu} \cap \mathcal{F}^{(\mathcal{U},y_{\mathcal{U}})}$. We use that $\mathcal{F}$, $\mathcal{M}_{t,\nu}$ and $\mathcal{F}^{(\mathcal{U},y_{\mathcal{U}})}$ are the convex hulls of the cube vertex sets $\mathcal{N}^{\mathcal{F}}_{t,\mathcal{M}}$, $\mathcal{N}_{t,\mathcal{M}}$ and that for the set

$$\mathcal{N}^{\mathcal{U},y_{\mathcal{U}}} = \{v[L] \,:\, \forall \ell \in \mathcal{U} : v[L=\ell] = y_{\mathcal{U}}\}\,, \tag{14.1}$$

we have

$$\mathcal{N}_{t,\mathcal{M}}^{\mathcal{F}} = \mathcal{N}^{\mathcal{U},y_\mathcal{U}} \cap \mathcal{N}_{t,\mathcal{M}} \,.$$

Thus, we can choose $\mathcal{N}^{\mathcal{U},y_\mathcal{U}}$ for the representation of the face $\mathcal{F}$ (see Def. 14.3) and further have that

$$\sum_{v[L] \in \mathcal{N}^{(\mathcal{U},y_\mathcal{U})}} \epsilon_{v[L]} \left[Y_{[p]}\right] = \bigotimes_{\ell \in [p]} \kappa^{\ell,(\mathcal{U},y_\mathcal{U})} \left[Y_\ell\right] \,,$$

where

$$\kappa^{\ell,(\mathcal{U},y_\mathcal{U})} \left[Y_\ell\right] = \begin{cases} \epsilon_{y_\ell} \left[Y_\ell\right] & \text{if} \quad \ell \in \mathcal{U} \\ \mathbb{I} \left[Y_\ell\right] & \text{if} \quad \ell \notin \mathcal{U} \,. \end{cases}$$

We have thus found an elementary activation tensor for the face $\mathcal{F}$.

(ii)→(i) Conversely, let $\xi \left[Y_{[p]}\right]$ be an elementary activation tensor of the face $\mathcal{F}$ in $\mathcal{M}_{t,\nu}$. Since $\xi \left[Y_{[p]}\right]$ is the sum of different one-hot encodings it is Boolean and we find an elementary decomposition $\xi \left[Y_{[p]}\right] = \bigotimes_{\ell \in [p]} \xi^\ell \left[Y_\ell\right]$ such that the leg vectors $\xi \left[Y_{[p]}\right]$ are Boolean. Since $\mathcal{F} \neq \emptyset$ we further have for $\ell \in [p]$ that $\xi^\ell \left[Y_\ell\right] \neq 0 \left[Y_\ell\right]$, and thus $\xi^\ell \left[Y_\ell\right] \in \{\epsilon_0 \left[Y_\ell\right], \epsilon_1 \left[Y_\ell\right], \mathbb{I} \left[Y_\ell\right]\}$ We construct a set $\mathcal{U} \subset [p]$

$$\mathcal{U} = \left\{\ell \,:\, \xi^\ell \left[Y_\ell\right] \neq \mathbb{I} \left[Y_\ell\right]\right\}$$

and a tuple

$$y_\ell = \begin{cases} 1 & \text{if} \quad \xi^\ell \left[Y_\ell\right] = \epsilon_1 \left[Y_\ell\right] \\ 0 & \text{if} \quad \xi^\ell \left[Y_\ell\right] = \epsilon_0 \left[Y_\ell\right] \end{cases} .$$

By construction $\xi^\ell \left[Y_\ell\right] = \kappa^{\ell,(\mathcal{U},y_\mathcal{U})} \left[Y_\ell\right]$ (see Example 14.3) follows for all $\ell \in [p]$ and therefore $\xi \left[Y_{[p]}\right] = \kappa^{(\mathcal{U},y_\mathcal{U})} \left[Y_{[p]}\right]$. We therefore have for the set (14.1)

$$\xi \left[Y_{[p]}\right] = \sum_{v \in \mathcal{N}^{\mathcal{U},y_\mathcal{U}}} \epsilon_v \left[Y_{[p]}\right]$$

and since $\xi \left[Y_{[p]}\right]$ is an activation tensor for $\mathcal{F}$, that

$$\mathcal{N}_{t,\mathcal{M}}^{\mathcal{F}} = \mathcal{N}^{\mathcal{U},y_\mathcal{U}} \cap \mathcal{N}_{t,\mathcal{M}} \,.$$

Taking convex hulls on both sides, this is equivalent to

$$\mathcal{F} = \mathcal{F}^{(\mathcal{U},y_\mathcal{U})} \cap \mathcal{M}_{t,\nu} \,.$$

We conclude that $\mathcal{F}$ is cube-like. □

Thm. 14.5 implies that the maximum entropy distributions to hypercubes (see Example 14.3) and simplices (see Example 14.4) are in $\Lambda^{t,\mathrm{EL},\nu}$, since these polytopes are cube-like.

We can further show, that in case of Boolean statistics any distribution in $\Lambda^{t,\mathrm{EL},\nu}$ is a maximum entropy distribution.

**Theorem 14.6** *Let $t$ be a Boolean statistic. Any distribution in $\Lambda^{t,\mathrm{EL},\nu}$ is a maximum entropy distribution with respect to $(t, \mu, \nu)$ where $\mu$ is its mean parameter.*

*Proof.* Let $\mathbb{P} \in \Lambda^{t,\mathrm{EL},\nu}$ be arbitrary. We then find an elementary tensor $\bigotimes_{s \in [n]} \tau^\ell \left[Y_\ell\right]$ and $\mathcal{Z} \in \mathbb{R}$ such that

$$\mathbb{P} \left[X_{[d]}\right] = \frac{1}{\mathcal{Z}} \left\langle \beta^t \left[Y_{[p]}, X_{[d]}\right], \bigotimes_{s \in [n]} \tau^\ell \left[Y_\ell\right] \right\rangle_{\left[X_{[d]}\right]} .$$

We then define for each $\ell \in [p]$ the vector $\kappa^{\ell}[Y_{\ell}]$ as the support of $\tau^{\ell}[Y_{\ell}]$ and

$$\theta[L=\ell] = \begin{cases} \ln\left[\frac{\tau^{\ell}[Y_{\ell}=1]}{\tau^{\ell}[Y_{\ell}=0]}\right] & \text{if} \quad \tau^{\ell}[Y_{\ell}=0], \tau^{\ell}[Y_{\ell}=1] \neq 0 \\ 0 \text{else} \end{cases}.$$

Since $t$ is a Boolean statistic the $\bigotimes_{\ell\in[p]} \kappa^{\ell}[Y_{\ell}]$ is a face activation tensor to the face

$$\mathcal{F} := \mathcal{M}_{t,\mathbb{I}} \cap \mathcal{F}^{(\mathcal{U},y_{\mathcal{U}})}$$

where $\mathcal{U} = \{\ell : \kappa^{\ell}[Y_{\ell}] \neq \mathbb{I}[Y_{\ell}]\}$ and for $\ell \in \mathcal{U}$

$$y_{\ell} = \begin{cases} 0 & \text{if} \quad \kappa^{\ell}[Y_{\ell}=1] = 0 \\ 1 & \text{else} \end{cases}.$$

Therefore, $\mathbb{P}$ is the maximum entropy distribution to the canonical parameters $\mathcal{F}$ and $\theta$. □

### 14.5.2 Interpretation by propositional formulas

Let us now assume that $m_k = 2$ for any $k \in [d]$. Following the formalism in [31] we understand each coordinate $t_{\ell}$ of the statistic as a propositional formula and the variables $X_{[d]}$ as atoms of a propositional theory. We use propositional connectives for syntactical representations of the formulas and denote by $\neg^1$ the logical negation and by $\neg^0$ the identity. A Boolean tensor $f\left[X_{[d]}\right]$ is called satisfiable, if there is at least one $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ with $f\left[X_{[d]} = x_{[d]}\right] = 1$.

**Theorem 14.7** *Let $t$ be a Boolean statistic. Any Boolean vector $v[L] \in \{0,1\}^p$ is a vertex of $\mathcal{M}_{t,\mathbb{I}}$ if and only if*

$$f_v\left[X_{[d]}\right] := \bigwedge_{\ell\in[p]} \neg^{1-v[L=\ell]} f_{\ell}\left[X_{[d]}\right]$$

*is satisfiable.*

*Proof.* Let $v[L] \in \{0,1\}^p$ be arbitrary. If and only if $v[L]$ is a vertex of $\mathcal{M}_{t,\mathbb{I}}$ we find $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ with $v[L] = \sigma^t X_{[d]} = x_{[d]}, L$. This is further equivalent to $f_{\ell}\left[X_{[d]} = x_{[d]}\right] = v[L=\ell]$ for any $\ell \in [p]$, $\neg^{1-v[L=\ell]} f_{\ell}\left[X_{[d]} = x_{[d]}\right] = 1$ for any $\ell \in [p]$, and $f_v\left[X_{[d]} = x_{[d]}\right] = 1$. □

Since the presence of Boolean vectors in $\mathcal{M}_{t,\mathbb{I}}$ is related by Thm. 14.7 to the satisfiability of the conjunction $f_v\left[X_{[d]}\right]$, the absence is related to entailment statements. To be more precise, for any $v[L] \in \{0,1\}^p$ and arbitrary $\mathcal{U} \subset [p]$ we have $v[L] \notin \mathcal{M}_{t,\mathbb{I}}$, if and only if

$$\left(\bigwedge_{\ell\in\mathcal{U}} \neg^{1-v[L=\ell]} f_{\ell}\left[X_{[d]}\right]\right) \models \left(\bigvee_{\ell\notin\mathcal{U}} \neg^{v[L=\ell]} f_{\ell}\left[X_{[d]}\right]\right).$$

By definition (see [74]) the latter holds if and only if

$$\left(\bigwedge_{\ell\in\mathcal{U}} \neg^{1-v[L=\ell]} f_{\ell}\left[X_{[d]}\right]\right) \wedge \neg\left(\bigvee_{\ell\notin\mathcal{U}} \neg^{v[L=\ell]} f_{\ell}\left[X_{[d]}\right]\right) = f_v\left[X_{[d]}\right]$$

is not satisfiable.

Let us now relate two well-studied sets of formulas, namely atomic formulas and minterm formulas, with two well-studied polytopes, namely hypercubes and simplices.

*Example 14.5 (Atomic formulas)* Let us consider the case of atomic formulas $f_{\ell}\left[X_{[d]}\right]$ defined with the coordinates at $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ by

$$f_\ell \left[X_{[d]} = x_{[d]}\right] = x_\ell \,.$$

The mean polytope in this case is the $d$-dimensional hypercube (see Example 14.3)

$$\mathcal{M}_{\mathcal{F}_{[d]},\mathbb{I}} = [0,1]^p$$

which is called a simple polytope, since each vertex is contained in the minimal number of $d$ facets.

Each face is characterized by the projections onto each variable, which is either $\{0\}$, $\{1\}$ or $[0,1]$. The projections are represented by the tuple $(\mathcal{U}, y_\mathcal{U})$ defined in the following way:

- We define the set $\mathcal{U} \subset [d]$ of variables, such that the projection onto the variable is $\{0\}$ or $\{1\}$
- We define to each $\ell \in \mathcal{U}$ an index $y_\ell = 0$ if the projection is $\{0\}$ and $y_\ell = 1$ if the projection is $\{1\}$.

Trivially, each face of the hypercube is a cube face.

More generally, the mean polytope to a collection $f_\ell \left[X_{[d]}\right]$ of propositional formulas is the hypercube, if and only if for any Boolean vector $v\,[L]$ we have

$$\left\langle \bigwedge_{\ell \in [p]} \neg^{1-v[L=\ell]} f_\ell \left[X_{[d]}\right] \right\rangle_{[\varnothing]} \neq 0 \,.$$

This means that no conjunction of possibly negated formulas from this statistic entails nor contradicts another formula.

*Example 14.6 (Minterm formulas build simplices)* The set of minterm formulas is indexed by $v\,[L] \in \{0,1\}^p$ and given by

$$f_v \left[X_{[d]}\right] = \bigwedge_{\ell \in [d]} \neg^{1-v[L=\ell]} X_\ell = \epsilon_{v[L]} \left[X_{[d]}\right]$$

where by $X_\ell$ we denote the $\ell$-th atomic formula (see Example 14.5). The mean polytope to the minterm statistic and base measure $\nu$ is the standard simplex of dimension

$$\left|\left\{v\,[L] \,:\, v \in \{0,1\}^p,\, \nu\left[X_{[d]} = v\,[L]\right] \neq 0\right\}\right| - 1 \,,$$

that is the set

$$\mathcal{M}_{\mathcal{F}_\wedge,\nu} = \mathrm{conv}\left(\epsilon_{v[L]}\left[Y_{[p]}\right] \,:\, v \in \{0,1\}^p,\, \nu\left[X_{[d]} = v\,[L]\right] \neq 0\right) \,.$$

We notice that for Boolean base measures $\mathcal{M}_{\mathcal{F}_\wedge,\nu} = \Lambda^\nu$. The simplex is further cube-like, since each face is indexed by a subset $\mathcal{U} \subset \{0,1\}^p$ of vertices and the intersection of the simplex with the cube face $\mathcal{F}^{(\{0,1\}^p \setminus \mathcal{U}, 1_{\{0,1\}^p \setminus \mathcal{U}})}$.

More generally, the mean polytope to a Boolean statistic is a standard simplex, if and only if each target coordinate restriction $f_{\tilde{\ell}}$ contradicts all others, that is

$$\forall \ell \neq \tilde{\ell} \in [p] : f_\ell \models \neg f_{\tilde{\ell}} \,.$$

While hypercubes and simplices are cube-like, we now provide examples of statistics, which polytopes are not cube-like.

*Example 14.7 (Maximum entropy distribution with non-elementary activation cores)*

Consider two atomic variables $X_0$ and $X_1$ and a statistic $\mathcal{F}$ consisting of the formulas

$$f_0\,[X_0, X_1] = (X_0 \wedge X_1) \quad , \quad f_1\,[X_0, X_1] = (X_0 \Rightarrow X_1)$$

with the coordinatewise expressions

$$f_0\,[X_0, X_1] = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} [X_0, X_1] \quad , \quad f_1\,[X_0, X_1] = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} [X_0, X_1] \,.$$

We can think of $X_0$ as a feature on an invoice, and $X_1$ as a feature on the accounting proposal.

From this we have

$$\beta^{(f_0,f_1)}\left[Y_0=0,Y_1=0,X_0,X_1\right]=\begin{pmatrix}0&0\\1&0\end{pmatrix}\left[X_0,X_1\right] \quad ,$$

$$\beta^{(f_0,f_1)}\left[Y_0=0,Y_1=1,X_0,X_1\right]=\begin{pmatrix}1&1\\0&0\end{pmatrix}\left[X_0,X_1\right] \quad ,$$

$$\beta^{(f_0,f_1)}\left[Y_0=1,Y_1=0,X_0,X_1\right]=\begin{pmatrix}0&0\\0&0\end{pmatrix}\left[X_0,X_1\right] \quad \text{and}$$

$$\beta^{(f_0,f_1)}\left[Y_0=1,Y_1=1,X_0,X_1\right]=\begin{pmatrix}0&0\\0&1\end{pmatrix}\left[X_0,X_1\right] \; .$$

Since the only vanishing slice of $\beta^{\mathcal{F}}$ with respect to the head variables is that to $y_{0,1} = (1,0)$, the vertices of the mean polytope are the vectors to the other head indices. The mean polytope is the convex hull of these vertices

$$\mathcal{M}_{(f_0,f_1)}=\mathrm{conv}\left(\begin{pmatrix}0\\0\end{pmatrix}\left[L\right],\begin{pmatrix}0\\1\end{pmatrix}\left[L\right],\begin{pmatrix}1\\1\end{pmatrix}\left[L\right]\right) \; .$$

This polytope has a non cube-like face (sketched blue in Figure 14.5), which is the convex hull of the vertices $(0\,0)^T,\ (1\,1)^T$. This face is parametrized by the (CP-rank 2) hard activation core

$$\kappa^{(0,0),(1,1)}\left[Y_0,Y_1\right]=\epsilon_{(0,0)}\left[Y_0,Y_1\right]+\epsilon_{(1,1)}\left[Y_0,Y_1\right]=\begin{pmatrix}1&0\\0&1\end{pmatrix}\left[Y_0,Y_1\right]$$

and has the refined base measure

$$\left\langle \kappa^{(0,0),(1,1)}\left[Y_0,Y_1\right],\beta^{\mathcal{F}}\left[Y_0,Y_1,X_0,X_1\right]\right\rangle_{[X_0,X_1]}=\begin{pmatrix}0&0\\1&1\end{pmatrix}\left[X_0,X_1\right] \; .$$

Any mean parameter $\mu$ on the interior of that face can be parametrized by a scalar $\lambda \in (0,1)$

$$\mu_\lambda\left[L\right]=\begin{pmatrix}\lambda&\lambda\end{pmatrix}\left[L\right] \; .$$

With the canonical parameters $\theta\left[L\right] \in \mathbb{R}^2$ of the maximum entropy distributions on this face by

$$\mathbb{P}\left[X_0,X_1\right]=\frac{1}{1+\exp\left[\theta\left[L=0\right]+\theta\left[L=1\right]\right]}\begin{pmatrix}0&0\\1&\exp\left[\theta\left[L=0\right]+\theta\left[L=1\right]\right]\end{pmatrix}\left[X_0,X_1\right]$$

we get the correspondence by the sigmoid

$$\lambda=\frac{1}{1+\exp\left[-\left(\theta\left[L=0\right]+\theta\left[L=1\right]\right)\right]} \; .$$

Note, that the hard activation core $\kappa^{(0,0),(1,1)}\left[Y_0,Y_1\right]$ to the blue face is the only non-elementary activation core. While the vertices have always elementary cores, the other non-vertex faces have elementary activation cores

$$\kappa^{(0,0),(1,0),(1,1)}\left[Y_0,Y_1\right]=\begin{pmatrix}1&1\\1&1\end{pmatrix}\left[Y_0,Y_1\right]=\mathbb{I}\left[Y_0\right]\otimes\mathbb{I}\left[Y_1\right],$$

$$\kappa^{(0,0),(1,0)}\left[Y_0,Y_1\right]=\begin{pmatrix}1&0\\1&0\end{pmatrix}\left[Y_0,Y_1\right]=\mathbb{I}\left[Y_0\right]\otimes\epsilon_0\left[Y_1\right],$$

$$\kappa^{(1,0),(1,1)}\left[Y_0,Y_1\right]=\begin{pmatrix}1&1\\0&0\end{pmatrix}\left[Y_0,Y_1\right]=\epsilon_0\left[Y_0\right]\otimes\mathbb{I}\left[Y_1\right] \; .$$

The maximum entropy distributions to mean parameters on the interior of all other faces than the blue face are represented by Computation-Activation Networks with only elementary activation cores. The corresponding lattice is sketched in Figure 14.6.

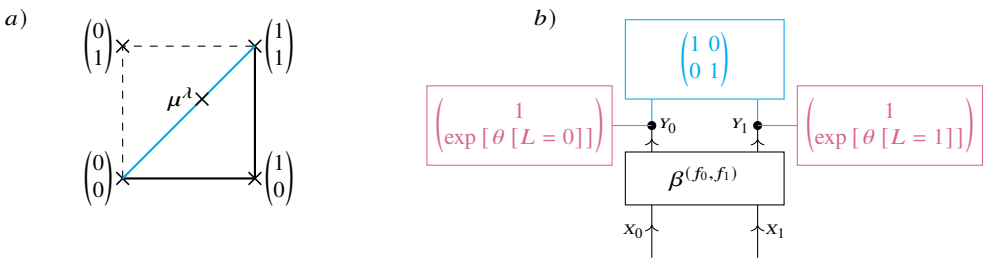



**Fig. 14.5** a) Mean polytope of the statistic $t = (X_0 \wedge X_1, X_0 \Rightarrow X_1)$ (thick), as a subset of the cube $[0,1]^2$ (dashed). The blue line is the face of the polytope, which is not cube-like, that is not an intersection of the polytope with the faces of the hypercube. We further define for $\lambda \in (0,1)$ a mean parameter $\mu_\lambda[L] = [\lambda\,\lambda]^T$ which is on the interior of the blue face. b) Corresponding Computation-Activation Network being the maximum entropy distribution reproducing $\mu_\lambda[L]$, when $\lambda$ is the sigmoid of $\theta[L=0] + \theta[L=1]$.

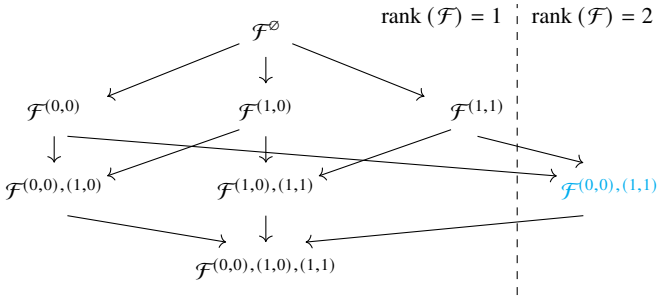



**Fig. 14.6** Face lattice $L((X_0 \wedge X_1, X_0 \Rightarrow X_1))$ to Example 14.7. The directed arrows represent inclusion of the faces, which is a partial order of the faces. The face $\mathcal{F}^{(0,0),(1,1)}$ is the only face, which is not representable by an elementary hypergraph. It is representable in a CP with hidden rank 2

### 14.5.3 Faces activated in the TT format

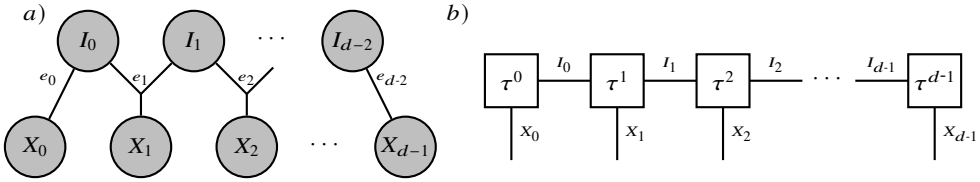



**Fig. 14.7** TT format consisting of the TT hypergraph a) and decorating tensor networks b).

The TT format is a well-studied tensor network format with favorable numerical properties [41] and applied across several scientific areas [76]. This format describes tensor networks on a hypergraph (see Figure 14.7)

$$\mathrm{TT} = \left(X_{[d]} \cup I_{[d-1]}, \{\{I_{k-1}, X_k, I_k\} : k \in \{1,\dots,d-2\}\} \cup \{\{X_0, I_0\}, \{I_{d-2}, X_{d-1}\}\}\right),$$

where $I_{[d-1]}$ are auxiliary hidden variables. We now construct an example of a polytope with a face activatable with TT, in which all hidden variables are of dimension 2.

*Example 14.8 (*TT *representation of diagonal faces)* Consider the vertex set

$$\mathcal{N} := \{0,1\}^p / v[L]$$

for some Boolean vector $v[L]$. We are interested in the face $\mathcal{F}^{\triangleleft,1}$ with the normal $\theta[L] = \frac{1}{2}(2v[L] - \mathbb{I}[L])$. Its vertices are the $p$ elements of $\{0,1\}^p$, which differ from $v[L]$ in exactly one coordinate (i.e. those with Hamming distance of 1

from $v\,[L]$). We denote these vertices by $v^\ell$ for $\ell \in [p]$, which coordinates to $\tilde{\ell} \in [p]$ are

$$v^\ell\left[L=\tilde{\ell}\right] = \begin{cases} v\left[L=\tilde{\ell}\right] & \text{if} \quad \tilde{\ell} \neq \ell \\ 1 - v\left[L=\tilde{\ell}\right] & \text{if} \quad \tilde{\ell} = \ell \end{cases} .$$

We now represent their sum in an TT format. The Boolean hidden variables are denoted by $I_{[p-1]}$ and are interpreted as indicators, whether the coordinate flip has happened in $[\ell]$ coordinates. We construct a TT core for $\ell = 0$ by

$$\tau^0\left[Y_0, I_0\right] = \epsilon_{v[L=0]}\left[Y_0\right] \otimes \epsilon_0\left[I_0\right] + \epsilon_{1-v[L=0]}\left[Y_0\right] \otimes \epsilon_1\left[I_0\right] ,$$

for $\ell \notin \{0, p-1\}$ the cores

$$\begin{aligned} \tau^\ell\left[I_{\ell-1}, Y_\ell, I_\ell\right] = {} & \epsilon_0\left[I_{\ell-1}\right] \otimes \epsilon_{1-v[L=\ell]}\left[Y_\ell\right] \otimes \epsilon_1\left[I_\ell\right] + \epsilon_0\left[I_{\ell-1}\right] \otimes \epsilon_{v[L=\ell]}\left[Y_\ell\right] \otimes \epsilon_0\left[I_\ell\right] \\ & + \epsilon_1\left[I_{\ell-1}\right] \otimes \epsilon_{v[L=\ell]}\left[Y_\ell\right] \otimes \epsilon_1\left[I_\ell\right] \end{aligned}$$

and for $\ell = p-1$

$$\tau^{p-1}\left[I_{p-2}, Y_{p-1}\right] = \epsilon_0\left[I_{p-2}\right] \otimes \epsilon_{1-v[L=p-1]}\left[Y_{p-1}\right] + \epsilon_1\left[I_{p-2}\right] \otimes \epsilon_{v[L=p-1]}\left[Y_{p-1}\right] .$$

For this tensor network in the TT format we then have

$$\sum_{\ell\in[p]} \epsilon_{v^\ell}\left[Y_{[p]}\right] = \left\langle \{\tau^0\left[Y_0, I_0\right], \tau^{p-1}\left[I_{p-2}, Y_{p-1}\right]\} \cup \{\tau^\ell\left[I_{\ell-1}, Y_\ell, I_\ell\right] : \ell \in \{1, \ldots, p-2\}\} \right\rangle_{\left[Y_{[p]}\right]} .$$

The dimension 2 to the hidden variables is furthermore minimal, since each matrification based on a partition of $[p]$ into non-empty sets has a matrix rank of 2 [41].

### 14.5.4 Construction of Boolean statistics to polytopes

We now show that any convex polytope with Boolean vertices in $\mathbb{R}^p$ is the mean polytope to a Boolean statistic. To construct the corresponding formulas we denote by $\mathcal{M}|_{\mathcal{U}}$ the restriction of the vectors in the set $\mathcal{M}$ to the coordinates in $\mathcal{U}$.

**Theorem 14.8** *Let $\mathcal{M}$ be an arbitrary polytope with Boolean vertices in $\mathbb{R}^p$. Then we construct propositional formulas on atoms $X_{[p]}$ by*

$$f_0\left[X_0\right] = \begin{cases} \top & \textit{if} \quad \mathcal{M}|_{\{0\}} = \{1\} \\ \bot & \textit{if} \quad \mathcal{M}|_{\{0\}} = \{0\} \\ X_0 & \textit{if} \quad \mathcal{M}|_{\{0\}} = [0,1] \end{cases}$$

*and iteratively for $\ell \in [p]$ with $\ell \geq 1$ by*

$$\begin{aligned} f_\ell\left[X_{[\ell+1]}\right] = \bigwedge_{v[L]\in(\mathcal{M}\cap\{0,1\}^p)|_{[\ell]}} \Bigg( \Big( \bigwedge_{\tilde{\ell}\in[\ell]} \neg^{1-v\left[L=\tilde{\ell}\right]} f_{\tilde{\ell}}\left[X_{[\tilde{\ell}+1]}\right] \Big) \Rightarrow \\ \begin{cases} \top & \textit{if} \quad (\mathcal{M}\cap v\times\mathbb{R}^{p-\ell})|_{\{\ell\}} = \{1\} \\ \bot & \textit{if} \quad (\mathcal{M}\cap v\times\mathbb{R}^{p-\ell})|_{\{\ell\}} = \{0\} \\ X_\ell & \textit{if} \quad (\mathcal{M}\cap v\times\mathbb{R}^{p-\ell})|_{\{\ell\}} = [0,1] \end{cases} \Bigg). \end{aligned}$$

*Here we denote by $\mathcal{M}|_V$ the projections of the vertices in $\mathcal{M}$ onto the subspaces $V$, and by $0_p$ the zero vector in $\mathbb{R}^p$.*

*Proof.* We show per induction, that for any $\ell \in [p]$ the family of Hybrid Logic Networks with the statistic $f_{[\ell+1]}$ by the first $\ell+1$ formulas has the mean polytope

$$\mathcal{M}_{f_{[\ell+1]},\mathbb{I}} = \mathcal{M}|_{[\ell]} \,. \tag{14.2}$$

"$\ell = 0$": The polytope $\mathcal{M}|_{[\ell]} = \{1\}$ (respectively $\mathcal{M}|_{[\ell]} = \{0\}$) is reproduced by $f_0$ being a tautology (respectively a contradiction). In the case $\mathcal{M}|_{[\ell]} = [0,1]$ the polytope is reproduced by the any formula, which is neither a tautology nor a contradiction, and the atomic formula $X_0$ is an example of such a contingency. Since the projection of a 0-1 polytope onto the first coordinates is itself a 0-1 polytope, these are the only possible cases and we conclude that in all

$$\mathcal{M}_{f_{[\ell]},\mathbb{I}} = \mathcal{M}|_{[\ell]} \,.$$

"$\ell \to \ell + 1$": We define for any $v \in \mathcal{M}_{f_{[\ell]},\mathbb{I}} \cap \{0,1\}^\ell$ the formula

$$f_{\ell,v}\left[X_{[\ell+1]}\right] := \bigwedge_{\tilde{\ell}\in[\ell]} \neg^{1-v\left[L=\tilde{\ell}\right]} f_{\tilde{\ell}}\left[X_{[\tilde{\ell}+1]}\right] \,.$$

and notice that all formulas are satisfiable (since the corresponding vertices are in the $\mathcal{M}_{f_{[\ell]},\mathbb{I}}$) and have disjoint models. We have for any Boolean vector $v \in \{0,1\}^\ell$ and $\lambda \in \{0,1\}$ that $v \times \{\lambda\} \in \mathcal{M}|_{[\ell+1]}$, if and only if

$$v \in \mathcal{M}|_{[\ell]} \quad \text{and} \quad \lambda \in \mathcal{M} \cap v \times \mathbb{R}^{p-\ell}|_{[\ell]} \,.$$

With modus ponens on $f_{\ell+1}\left[X_{[\ell+2]}\right]$ this is further equivalent to the satisfaction of

$$f_{\ell,v}\left[X_{[\ell+1]}\right] \wedge \neg^{1-\lambda} f_{\ell+1}\left[X_{[\ell+2]}\right]$$

and thus to $v\times\{\lambda\} \in \mathcal{M}_{f_{[\ell+1]},\mathbb{I}}$. The vertices of the 0/1 polytopes $\mathcal{M}_{f_{[\ell+1]},\mathbb{I}}$ and $\mathcal{M}|_{[\ell+1]}$ thus coincide and both polytopes are therefore equivalent.

By induction the equation (14.2) holds for arbitrary $\ell \in [p]$. For $\ell = p - 1$ this is the claim. □

### 14.5.5 Examples in statistical physics and coding theory

Let us now investigate cube-like faces to standard examples in statistical physics and coding theory.

*Example 14.9 (Ising model on* 2 *nodes)* Consider two Boolean variables $X_0, X_1$ and the Ising statistic $t = (f_0, f_1, f_2)$ by three propositional formulas

$$f_0\left[X_{[2]}\right] = X_0 \quad , \quad f_1\left[X_{[2]}\right] = X_1 \quad \text{and} \quad f_2\left[X_{[2]}\right] = X_0 \wedge X_1 \,.$$

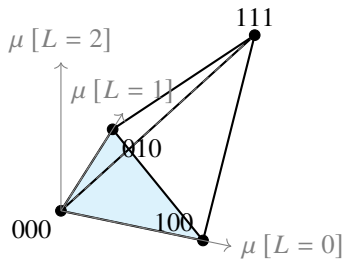
In the Ising interpretation, the Boolean variables represent interacting spins at two locations. Their value is then measured by the first two formulas and their interaction by the third.

The vertices of the mean polytope to this statistic are

$$\sigma^t\left[X_{[2]} = (0,0), L\right] = \begin{pmatrix}0\\0\\0\end{pmatrix}[L] \quad , \quad \sigma^t\left[X_{[2]} = (0,1), L\right] = \begin{pmatrix}0\\1\\0\end{pmatrix}[L] \quad ,$$

$$\sigma^t\left[X_{[2]} = (1,0), L\right] = \begin{pmatrix}1\\0\\0\end{pmatrix}[L] \quad \text{and} \quad \sigma^t\left[X_{[2]} = (1,1), L\right] = \begin{pmatrix}1\\1\\1\end{pmatrix}[L] \quad .$$

The mean polytope is the convex hull of these, sketched as:




We notice, that the convex hull of any three out of the four vertices builds a facet. The only cube-like facet of these four (sketched in cyan in the above plot) is the convex hull of

$$\operatorname{conv}\left(\begin{pmatrix}0\\0\\0\end{pmatrix}[L], \begin{pmatrix}1\\0\\0\end{pmatrix}[L], \begin{pmatrix}0\\1\\0\end{pmatrix}[L]\right)$$

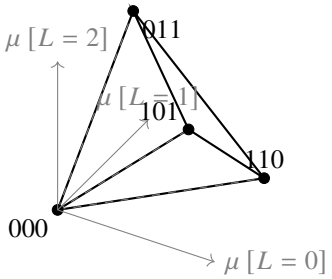
to which we have the parametrization by $(\mathcal{U}, y_{\mathcal{U}}) = (\{2\}, 0)$.

As a consequence, there are maximum entropy distributions of mean parameters in the Ising statistics, which do not have a representation by an elementary CompActNet in the Ising statistic.

*Example 14.10 (Codeword Polytopes)* Consider the codeword polytope with the vertices

$$\mathcal{N} = \left\{ y_{[3]} \,:\, \sum_{k\in[3]} y_k \mod 2 = 0 \right\} = \left\{ \begin{pmatrix}0\\0\\0\end{pmatrix}[L], \begin{pmatrix}0\\1\\1\end{pmatrix}[L], \begin{pmatrix}1\\0\\1\end{pmatrix}[L], \begin{pmatrix}1\\1\\0\end{pmatrix}[L] \right\}$$

We interpret the vertices by the strings of length 3, which parity vanishes. Such constraints are common in the construction of codewords using parity-check codes (see [27]).




All facets are non-cube-like, whereas all other faces are cube-like (the edges are labeled by the 6 facets of $[0,1]^3$).

Each facet is characterized by the Hamming distance of exactly 1 to a single of the 4 cube vertices, which are not codewords (see Example 3.9 in [82]). We can construct TT activation tensors with rank 2, which calculate the Hamming distance to these vertices via hidden ranks. Each core tensor in that format adds the contribution of a mode to the Hamming distance.

In more generality, Low-Density-Parity-Checking Codes (see [27]) construct codewords by a collection of parity constraints on subsets of variables. Such polytopes are also known as cycle polytopes, and studied e.g. in [35].

## 14.6 Mean as a Statistic

We in this section take the perspective of estimating a maximum entropy distribution given observed data.

- $\mu_D$ is an unbiased estimator of $\mu$, i.e. $\mathbb{E}\left[\mu_D\left[L\right]\right] = \mu$
- $\mu_D$ is a consistent estimator of $\mu$, i.e. $\mu_D\left[L\right] \to \mu$ by the law of large numbers coordinatewise almost everywhere.
- $\mu_D$ is the minimal variance unbiased estimator of $\mu$.

The mean parameter $\mu_D$ given a dataset can be understood as a statistic of the dataset. We here show that for the family of maximum entropy distributions this statistic is a minimal sufficient statistic.

The family of maximum entropy distributions is the set

$$\left\{\mathbb{P}^{\mu}\left[X_{[d]}\right] \,:\, \mu \in \mathcal{M}_{t,\nu}\right\}$$

which has been characterized above by a union of exponential families with respect to face measures.

Taking a frequentist perspective we now understand datasets by random variables $X_{[d]\times[m]}$, where for $j \in [m]$ the variables $X_{[d],j}$ are drawn i.i.d. from a maximum entropy distribution. The mean statistic is then a tensor

$$\mu_D\left[X_{[d]\times[m]}, L\right]$$

whith coordinates

$$\mu_D\left[X_{[d]\times[m]} = x_{[d]\times[m]}, L\right] = \frac{1}{m}\sum_{j\in[m]} \sigma^t\left[X_{[d],j} = x_{[d],j}, L\right] .$$

**Theorem 14.9** *The mean statistic is a sufficient statistic for the family of maximum entropy distributions* $(t, \nu)$*. If the base measure has maximal support, the mean statistic is in addition minimal.*

*Proof.* It sufficies to show that the likelihood is a function of $\mu_D$. Let us choose a face $\mathcal{F}$ of $\mathcal{M}_{t,\nu}$, then the likelihood is different from 0 if and only if the empirical distribution is representable with respect to the face measure. This is the case if and only if $\mu_D$ is on the face. In case that $\mu_D$ is on the face, then the likelihood of any distribution on that face exponential family is

$$\exp\left[m \cdot \left(\left\langle \mu_D\left[X_{[d]\times[m]}, L\right], \theta\left[L\right]\right\rangle_{[\varnothing]} - A^{(t,\nu)}(\theta)\right)\right]$$

We have thus shown that the likelihood is always a function of $\mu_D$.

The minimality claim can be shown by the constant quotient criterion (see Thm. 6.2.13 in [12]). □

## 14.7 Face Computation

We take the perspective of computation whether a vertex is in a face. Based on this perspective, we derive activation tensors to faces. Note, that we construct face activation tensors based on DFA computations here. Thus the decomposition studies of the circuits and automata chapter are relevant.

### 14.7.1 Examples

*Example 14.11 (Generalization of Example 14.8 to larger* TT *ranks)*

We understand this as the tensor representation of an automaton counting the cutted Hamming distance between two states.

We now generalize the construction of Example 14.8 by using the Hadamard distance $d(\cdot,\cdot)$ in $\{0,1\}^p$, which counts the number of coordinates two vertices differ in. For $s \in \{1,\ldots,p-1\}$ we define for a fixed $v\left[L\right] \in \{0,1\}^p$ a polytope as the convex hull

$$\left\{\tilde{v}[L] \,:\, d(\tilde{v}, v) \geq s\right\} .$$

The face to the normal $\frac{1}{2}(2v\left[L\right] - \mathbb{I}\left[L\right])$ is the convex hull

$$\mathcal{F}^{\triangleleft,s} := \mathrm{conv}\left(\left\{\tilde{v}[L] \,:\, d(\tilde{v}, v) = s\right\}\right)$$

containing $\binom{p}{s}$ vertices. We label these vertices by subsets $\mathcal{U} \subset [p]$ of cardinality $s$ and define for $\ell \in [p]$

$$v^{\mathcal{U}}\left[L=\ell\right]=\begin{cases} v\left[\ell\in[p]\right] & \text{if} \quad \ell\notin\mathcal{U} \\ 1-v\left[\ell\in[p]\right] & \text{if} \quad \ell\notin\mathcal{U} \end{cases}$$

We now construct a TT with hidden variables $I_\ell$ and ranks $r_\ell = \min(\ell+1, p-\ell+1, s+1)$ to represent the sum of their one-hot encodings. To this end, let there be TT cores for $\ell = 0$ by

$$\tau^0\left[Y_0, I_0\right] = \epsilon_{v[L=0]}\left[Y_0\right]\otimes\epsilon_0\left[I_0\right] + \epsilon_{1-v[L=0]}\left[Y_0\right]\otimes\epsilon_1\left[I_0\right]$$

further for $\ell\notin\{0, p-1\}$ the cores

$$\tau^\ell\left[I_{\ell-1}, Y_\ell, I_\ell\right] = \sum_{\ell\in\{\max(s-p+\ell,0),\ldots,\max(\ell,s)\}} \epsilon_\ell\left[I_\ell\right]\otimes\left(\epsilon_{v[L=\ell]}\left[Y_\ell\right]\otimes\epsilon_\ell\left[I_{\ell+1}\right] + \epsilon_{1-v[L=\ell]}\left[Y_\ell\right]\otimes\epsilon_{\ell+1}\left[I_{\ell+1}\right]\right)$$

and for $\ell = p-1$

$$\tau^{p-1}\left[I_{p-1}, Y_{p-1}\right] = \epsilon_0\left[I_{p-1}\right]\otimes\epsilon_{1-v[L=p-1]}\left[Y_{p-1}\right] + \epsilon_1\left[I_{p-1}\right]\otimes\epsilon_{v[L=p-1]}\left[Y_{p-1}\right] .$$

Based on the interpretation, that the hidden variables $I_\ell$ count the Hamming distance of the vectors $v\left[L\right]|_{\mathbb{R}^\ell\times 0_{p-\ell}}$ and the respective $v^{\mathcal{U}}\left[L\right]|_{\mathbb{R}^\ell\times 0_{p-\ell}}$ one can show that

$$\sum_{\mathcal{U}\subset[p]:|\mathcal{U}|=s}\epsilon_{v^{\mathcal{U}}}\left[Y_{[p]}\right] = \left\langle\left\{\tau^0\left[Y_0, I_0\right], \tau^{p-1}\left[I_{p-1}, Y_{p-1}\right]\right\}\cup\left\{\tau^\ell\left[I_{\ell-1}, Y_\ell, I_\ell\right] : \ell\notin\{0, p-1\}\right\}\right\rangle_{\left[Y_{[p]}\right]} .$$

*Example 14.12 (Generalization of Example 14.11 to arbitrary* HT *formats)* Instead of aligning the Hamming count variables linearly, one can find a representation of the activation tensor

$$\sum_{\mathcal{U}\subset[p]:|\mathcal{U}|=s}\epsilon_{v^{\mathcal{U}}}\left[Y_{[p]}\right]$$

in an arbitrary directed acyclic tree hypergraph format $\mathcal{G}$, which hidden ranks are bounded by the number of leafs in the subtree and $s+1$.

At any leaf of the tree we define a $2x2$ matrix

$$\tau^\ell\left[Y_\ell, I_\ell\right] = \epsilon_{v[L=\ell]}\left[Y_\ell\right]\otimes\epsilon_0\left[I_\ell\right] + \epsilon_{1-v[L=\ell]}\left[Y_\ell\right]\otimes\epsilon_1\left[I_\ell\right] .$$

At each intermediate non-root hyperedge we choose an outgoing counting variable $I_{e^{\text{out}}}$ with dimension by $n_{e^{\text{out}}} = \min(\sum_{v\in e^{\text{in}}} n_{e^{\text{in}}}, s+1)$ and define a tensor with the slices

$$\tau^e\left[I_{e^{\text{out}}}, I_{e^{\text{in}}}\right] = \beta^+\left[Y_+ = i_{e^{\text{out}}}, I_{e^{\text{in}}}\right] .$$

We further build at the root hyperedge $e$ a tensor

$$\tau^e\left[I_e\right] = \sum_{i:\langle i\rangle_{[\varnothing]}=s}\epsilon_{i_e}\left[I_e\right] .$$

Using the counting variable interpretation one can now show that

$$\sum_{\mathcal{U}\subset[p]:|\mathcal{U}|=s}\epsilon_{v^{\mathcal{U}}}\left[Y_{[p]}\right] = \left\langle\left\{\tau^e\left[I_e\right] : e\in\mathcal{E}\right\}\cup\left\{\tau^\ell\left[Y_\ell, I_\ell\right] : v\in\mathcal{V}\right\}\right\rangle_{\left[Y_{[p]}\right]} .$$

### 14.7.2 Generic Construction

When the vertices are characterized by the sum of local terms, we can construct the face activation tensor in TT format, by summation cores. This is for example the case for the examples constructed based on Hamming distances.

### 14.7.3 Relation with Facet Complexity

Facet complexity [35, Definition 6.2.2] bounds the encoding length of the inequalities describing a polytope.

For any face $\mathcal{F}$ we find a normal $a \in \mathbb{R}^p$ and $b \in \mathbb{R}$ such that $x$

$$\mathcal{F} = \{\mu\left[L\right] \in \mathcal{M} : \left\langle a\left[L\right], \mu\left[L\right]\right\rangle_{[\varnothing]} = b\}\,.$$

We can thus represent a face activating tensor by computing the contraction $\left\langle a\left[L\right], y[L]\right\rangle_{[\varnothing]}$.

To this end, let $\mathcal{U} \subset \mathbb{R}$ be a set with index interpretation function

$$I : [r] \to \mathcal{U}\,.$$

We demand $0, b \in \mathcal{U}$. We can for example choose $\mathcal{U}$ as the $d$-bit binary integer representation.

We then define for $\ell \in [p]$ the summation operations

$$+^{\ell, a[L=\ell]} : \mathcal{U} \times [n] \to \mathcal{U}$$

which satisfies for any $u \in \mathcal{U}$ and $y_\ell$ for which $u + a\left[L = \ell\right] \cdot y_\ell \in \mathcal{U}$ the equation

$$+^{\ell, a[L=\ell]}(u, y_\ell) = u + a\left[L = \ell\right] \cdot y_\ell\,.$$

We further assume, that for any tuple $y_{[p]} \in \bigtimes_{\ell \in [p]} [n_\ell]$ and any $\tilde{\ell} \in [p]$

$$\sum_{\ell \leq \tilde{\ell}} a\left[L = \ell\right] \cdot y_{\tilde{\ell}} \in \mathcal{U}\,.$$

Then we have

$$\sum_{v[L] \in \mathcal{N}^{\mathcal{F}}_{r,\nu}} \epsilon_{v[L]}\left[Y_{[p]}\right] = \left\langle \{\beta^{+^{\ell, a[L=\ell]}}\left[Y_{+^{\ell, a[L=\ell]}}, Y_\ell\right] : \ell \in [p]\} \cup \{\epsilon_{I^{-1}(0)}\left[Y_{+^{-1, a[L=-1]}}\right], \epsilon_{I^{-1}(b)}\left[Y_{+^{p-1, a[L=p-1]}}\right]\}\right\rangle_{\left[Y_{[p]}\right]}\,.$$

The variable $Y_{+^{-1,a[L=-1]}}$ is auxiliary to initialize the sum with 0.

We can weaken the assumptions on $\mathcal{U}$: When $a\left[L\right]$ is positive, then any partial sum which exceeds $b$ can be represented by any member of $\mathcal{U}$ which is larger than $b$. This brings further sparsity, since no values larger than $b$ are needed to represent the face (add for example the $\infty$ element to $\mathcal{U}$ and represent any partial sum exceeding $b$ by it).

**Definition 14.8** We say that a face $\mathcal{F}$ has *TT face complexity* $r$ if there exists a normal $a \in \mathbb{R}^p$ and $b \in \mathbb{R}$ with

$$\mathcal{F} = \{\mu\left[L\right] \in \mathcal{M} : \left\langle a\left[L\right], \mu\left[L\right]\right\rangle_{[\varnothing]} = b\}\,.$$

and there is a subset $\mathcal{U}$ of cardinality at most $r$, for which $0, b \in \mathcal{U}$ and for any tuple $y_{[p]} \in \bigtimes_{\ell \in [p]} [n_\ell]$ and any $\tilde{\ell} \in [p]$

$$\sum_{\ell \leq \tilde{\ell}} a\left[L = \ell\right] \cdot y_{\tilde{\ell}} \in \mathcal{U}\,.$$

Comparison with facet complexity [35, Definition 6.2.2]:

- We define complexity here without the log (which would get the string length), to make a direct connection with TT ranks.
- Facet complexity considers only the representation of the equation, not the demand of computing it.
- Facet complexity considers just facets, not general faces, since it makes a statement about encoding the halfspace representation of the whole polytope.

**Theorem 14.10** *Let $\mathcal{F}$ be a face with TT face complexity $r$. Then the face activation tensor has a* TT *rank of at most $r$.*

*Proof.* See above construction. □

# Chapter 15
# Variational Inference

**Abstract** We present here variational inference methods to reason on Computation-Activation Networks.
Topics:

- I.3.4 (Exponential family) Variational inference
- I.3.7 Mean field methods

## 15.1 Foundational inference problems

We have two parametrizations of maximum entropy distributions:

- **Mean parametrization:** The mean parameter $\mu\left[L\right] \in \mathcal{M}_{t,\nu}$
- **Canonical parametrization:** The face $\mathcal{F} \in Q^{\mathcal{F}}_{t,\nu}$ together with the weight vector $\theta\left[L\right]$

The fundamental inference problem are finding one parametrization given the other.

- **Forward mapping:** Given the canonical parametrization $(\mathcal{F}, \theta\left[L\right])$, find the mean parameter $\mu\left[L\right]$.
- **Backward mapping:** Given the mean parameter $\mu\left[L\right]$, find the face $\mathcal{F}$ such that $\mu\left[L\right]$ is in the relative interior of the face, and the corresponding canonical parameter $\theta\left[L\right]$.

### 15.1.1 Forward mapping

The forward mapping is the contraction of the distribution with the selection encoding of the statistic, that is the map

$$F^{(t,\nu)}(t) \,:\, \mathcal{P}_t \to \mathcal{M}_t$$

with

$$F^t(\mathcal{F}, \theta) = \frac{\left\langle \alpha^\theta\left[Y_{[p]}\right], \kappa^{\mathcal{F}}\left[Y_{[p]}\right], \beta^t\left[Y_{[p]}, X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]}}{\left\langle \alpha^\theta\left[Y_{[p]}\right], \kappa^{\mathcal{F}}\left[Y_{[p]}\right], \beta^t\left[Y_{[p]}, X_{[d]}\right]\right\rangle_{[\varnothing]}}\,.$$

### 15.1.2 Backward mapping

Determining the face such that $\mu\left[L\right]$ in the relative interior, and the corresponding canonical parameter.

#### 15.1.2.1 Cross entropy minimization

The cross entropy between a distribution $\mathbb{P}^* \left[X_{[d]}\right]$ and a maximum entropy distribution $\mathbb{P}^{\mathcal{F},\theta} \left[X_{[d]}\right]$ is given by

$$\begin{aligned}\mathbb{H}\left[\mathbb{P}^* \left[X_{[d]}\right], \mathbb{P}^{\mathcal{F},\theta} \left[X_{[d]}\right]\right] = &- \left\langle \mathbb{P}^* \left[X_{[d]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right], \ln\left[\alpha^\theta \left[Y_{[p]}\right]\right]\right\rangle_{[\varnothing]} \\ &- \left\langle \mathbb{P}^* \left[X_{[d]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right], \ln\left[\kappa^{\mathcal{F}} \left[Y_{[p]}\right]\right]\right\rangle_{[\varnothing]} \\ &+ \ln\left[\left\langle \alpha^\theta \left[Y_{[p]}\right], \kappa^{\mathcal{F}} \left[Y_{[p]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right]\right\rangle_{[\varnothing]}\right]\end{aligned}$$

The first term can be simplified as

$$\begin{aligned}-\left\langle \mathbb{P}^* \left[X_{[d]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right], \ln\left[\alpha^\theta \left[Y_{[p]}\right]\right]\right\rangle_{[\varnothing]} &= -\sum_{\ell \in [p]} \theta\left[L = \ell\right] \cdot \left\langle \mathbb{P}^* \left[X_{[d]}\right], t_\ell \left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\ &= -\left\langle \mu^*[L], \theta\left[L\right]\right\rangle_{[\varnothing]}\end{aligned}$$

To investigate the second term, we notice that

$$\left\langle \beta^t \left[Y_{[p]}, X_{[d]}\right], \ln\left[\kappa^{\mathcal{F}} \left[Y_{[p]}\right]\right]\right\rangle_{\left[X_{[d]} = x_{[d]}\right]} = \begin{cases} 0 & \text{if} \quad \gamma^{x_{[d]}} \in \mathcal{F} \\ -\infty & \text{else} \end{cases}$$

Therefore the cross entropy is finite, if and only if $\mathbb{P}^* \left[X_{[d]}\right]$ is supported at the face measure. This is only the case if $\mu^*[L] \in \mathcal{F}$. If the cross entropy is finite, the second term vanishes.

Among the faces with $\mu^*[L] \in \mathcal{F}$ the third term is minimal for each $\theta \in \mathbb{R}^p$.

$$\begin{aligned}&\operatorname{argmin}_{\mathcal{F} \in L(\mathcal{M}_{t,\nu})} \operatorname{argmin}_{\theta \in \mathbb{R}^p} \mathbb{H}\left[\mathbb{P}\left[X_{[d]}\right], \mathbb{P}^{\mathcal{F},\theta}\left[X_{[d]}\right]\right] \\ &\quad = \operatorname{argmin}_{\theta \in \mathbb{R}^p} -\left\langle \mu^*[L], \theta\left[L\right]\right\rangle_{[\varnothing]} + \ln\left[\left\langle \alpha^\theta \left[Y_{[p]}\right], \kappa^{\mathcal{F}^{\mu^*}} \left[Y_{[p]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right]\right\rangle_{[\varnothing]}\right]\end{aligned}$$

#### 15.1.2.2 Definition of backward mapping

## 15.2 Face estimation

Generic situation: Find the face in $L\left(\mathcal{M}_t\right)$ which contains $\mu^*$ in its relative interior.

**Theorem 15.1** *Let $\mathbb{P}^* \left[X_{[d]}\right]$ be a distribution with mean parameter $\mu^*[L]$. Then the face $\mathcal{F}(\mu^*)$ is the minimal face in the lattice $L\left(\mathcal{M}_t\right)$ such that*

$$\left\langle \mathbb{P}^* \left[X_{[d]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right], \kappa^{\mathcal{F}} \left[Y_{[p]}\right]\right\rangle_{[\varnothing]} = 1\,.$$

*Proof.* • $\mu^*[L] \in \mathcal{F}$ is equivalent to $\left\langle \mathbb{P}^* \left[X_{[d]}\right], \beta^t \left[Y_{[p]}, X_{[d]}\right], \kappa^{\mathcal{F}} \left[Y_{[p]}\right]\right\rangle_{[\varnothing]} = 1$.

• If $\mu^*[L]$ is not in the relative interior of $\mathcal{F}$, then it is contained in a smaller face.

□

### 15.2.1 Cube-like faces

Cube-faces are parametrized by $\mathcal{U}$ and a boolean tuple $y_{\mathcal{U}}$. For cube-like faces, it is enough to decide entailment and contradiction of each feature.

**Theorem 15.2** *If the mean parameter $\mu^*[L]$ to a distribution $\mathbb{P}^* \left[X_{[d]}\right]$ is in the relative interior of a cube-like face, then the face $\mathcal{F}(\mu^*)$*

$$\mathcal{U} = \{\ell \,:\, \ell \in [p],\, \left\langle \mathbb{P}^* \left[X_{[d]}\right], t_\ell \left[X_{[d]}\right]\right\rangle_{[\varnothing]} \in \{0,1\}\}$$

*and*

$$y_\ell = \left\langle \mathbb{P}^* \left[X_{[d]}\right], t_\ell \left[X_{[d]}\right]\right\rangle_{[\varnothing]} .$$

### 15.2.2 Face refinement

We present Algorithm 10 to search the $\mathcal{F}(\mu^*)$ in the face lattice. In each iteration of the while loop it either finds a facet of the current with $\mu^*[L] \in \tilde{\mathcal{F}}$ or it terminates. Since the face lattice has a finite depth $n$, it always terminates after at most $n$ iterations of the while loop.

**Algorithm 10** Face refinement

**Require:** Distribution $\mathbb{P}^*\left[X_{[d]}\right]$, statistic $t$
**Ensure:** Face $\mathcal{F}(\mu^*[L])$

$\mathcal{F} \leftarrow \mathcal{M}_{t,\nu}$
**while** True **do**
  refined $\leftarrow$ False
  **for all** facets $\tilde{\mathcal{F}}$ of $\mathcal{F}$ **do**
    **if** not refined **then**
      Compute

$$\lambda = \left\langle \mathbb{P}^* \left[X_{[d]}\right], \nu^{\tilde{\mathcal{F}}} \left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$

      **if** $\lambda = 1$ **then**
        refined $\leftarrow$ True
        $\mathcal{F} \leftarrow \tilde{\mathcal{F}}$
      **end if**
    **end if**
  **end for**
  **if** not refined **then**
    **return** $\mathcal{F}$
  **end if**
**end while**

## 15.3 Parameter estimation

We now present iterative algorithms to train the weight vector, one the face to a mean parameter has been determined. They are coordinate descent methods on the cross-entropy loss function.

### 15.3.1 Moment matching

We differentiate the cross-entropy with respect to the weight vector

$$\frac{\partial \mathbb{H}\left[\mathbb{P}^*, \mathbb{P}^{\mathcal{F},\theta}\right]}{\partial \theta\,[L=\ell]} = \mu\,[L=\ell] - \mu^*[L=\ell]\,.$$

The derivative vanishes when both mean parameters coincide, which is the moment matching condition.

### 15.3.2 Issues with erroneous faces

If applied to the wrong face, they fail:

- If $\mu\,[L]$ not on the face, then the cross-entropy is infinite.
- If $\mu\,[L]$ on the face, but not in the relative interior, then the norm of the weight vector diverges.

# Chapter 16
# Inductive Reasoning

**Abstract** In this chapter, we investigate the problem of selecting statistics.
Topics:

- II.7 Formula Selecting Networks
- II.9.5 Structure Learning in Hybrid Logic Networks

# Chapter 17
# Probabilistic Guarantees

**Abstract** Here we show probabilistic guarantees for the success of the reasoning mechanisms on Computation-Activation Networks.

Topics: Chapter II.10

## 17.1 Fluctuations

### 17.1.1 Widths of the noise vector

### 17.1.2 Recovery guarantees given width bounds

## 17.2 Face recovery

Faces to the mean parameter represent the hard constraints in a model. Recovering the right face therefore corresponds with a correct identification of the hard constraints.

**Lemma 17.1** *When generating data by a distribution* $\mathbb{P}^* \left[X_{[d]}\right]$ *with mean* $\mu^*$ *we always have*

$$\mu_D \left[L\right] \in \mathcal{F}(\mu^*) \,.$$

*Proof.* We always have $\mathbb{I}_{\neq 0}\left(\mathbb{P}^* \left[X_{[d]}\right]\right) \prec \mathbb{I}_{\neq 0}\left(\mathbb{P}^D \left[X_{[d]}\right]\right)$ when drawing data from $\mathbb{P}^* \left[X_{[d]}\right]$. Since the distribution $\mathbb{P}^* \left[X_{[d]}\right]$ has its mean on the face $\mathcal{F}(\mu^*)$, the image of its support is included in $\mathcal{N}_{t,\nu}^{\mathcal{F}}$. Also the image of the support of $\mathbb{P}^D \left[X_{[d]}\right]$ is included therein and thus $\mu_D \left[L\right] \in \mathcal{F}(\mu^*)$. □

Estimating the maximum entropy distribution with the data mean, we always estimate the face parameter as $\mathcal{F}(\mu_D)$. Since $\mu_D \left[L\right] \in \mathcal{F}(\mu^*)$ we have $\mathcal{F}(\mu_D) \subset \mathcal{F}(\mu^*)$. The constraints are therefore never underestimated.

*Example 17.1 (Face recovery of bernoulli)*

The mean parameter is parameter of Bernoulli distribution, the distribution of $\mu_D$ is the binomial distribution

$$\mu_D \sim \frac{1}{m} B(m, \mu^*) \,.$$

Face recovery:

- When $\mu^* \in \{0, 1\}$, then we always have face recovery and perfect recovery.
- When $\mu^* \in (0, 1)$, then we have not always face recovery. This case is investigated in the following.

Expectation of squared norm of the error: $\frac{1}{m}\mu^* \cdot (1 - \mu^*)$ We have for any $\delta > 0$ the chebyshev bound

$$\mathbb{P}\left[|\mu_D - \mu^*| > \delta\right] = \mathbb{P}\left[|\mu_D - \mu^*| > \delta^2\right] \leq \frac{\mu^* \cdot (1-\mu^*)}{m\delta^2}\,.$$

Thus, choosing $\delta = \mu^*$, we have for any $\eta$ with probability $1-\eta$ face recovery provided that

$$m \geq \frac{1}{\eta} \cdot \frac{1-\mu^*}{\mu^*}\,.$$

To show that the bound is tight we compute the exact probability of face recovery. The face recovery event is the complement of all datapoints being 0 or 1. The probability is therefore

$$1 - (1-\mu^*)^m - (\mu^*)^m\,.$$

If $\mu^*$ is very small, then this is approximately $m \cdot \mu^* + O(\mu^{*2})$. Therefore we need $m \sim \frac{1}{\mu^*}$ samples for face recovery.

# Chapter 18
# Quantum inference

**Abstract** We here investigate, how inference on Computation-Activation Networks can be executed by quantum algorithms. While we find circuits corresponding to the computation networks, we exploit Grover operators for the activation of the computed qubits. We propose two inference algorithms:

- **Quantum rejection sampling:** When the canonical parameters of an CompActNet are available, we prepare q-samples of the ancilla augmented CompActNet and apply Grover rotations to achieve an square root advantage in sampling.
- **Quantum moment matching:** When the mean parameters to an CompActNet are available, we iteratively construct the q-sample of an approximation.

## 18.1 Introduction

Tensor network contractions have been identified as central inference mechanism of neuro-symbolic AI. However, their exact execution is NP-hard, and one either has to restrict itself to feasible situations, or apply approximations.

We in this chapter investigate, how specific contractions (especially forward inference on Computation-Activation Networks) can be performed more efficiently on a Quantum Computer.

Tensor Network contractions can be performed by quantum computing as follows:

- **Rejection Sampling based:** Can augment any tensor network by an Bayesian network, which is efficiently inferred by forward sampling. We prepare quantum states, which measurement statistics can be utilized to prepare samples from interesting distributions such as Computation-Activation Networks. The square root quantum advantage is obtained by amplitude amplification.
- **Phase Estimation based:** Measure the eigenvalues of a Grover operator to a formula and a q-sample.
- **Fourier Transform based:** Limited contractions (given promises on their possible values) can be formulated as Hidden Subgroup Problems and solved by Fourier transforms. We are inspired by Deutsch-Jozsa algorithm, which we generalize here to a general moment computation problem.

Note that the rejection sampling scheme and the phase estimation are ignorant under the phase of the prepared state, and only the Fourier transform based scheme depend on the phase.

Generic tensor network contraction is NP-hard, reflecting the hardness of logical and probabilistic inference. Quantum Computers can prepare states, which measurement distribution are specific network contractions, which can be hard to contract classically. We study schemes to retrieve information about these states, for example by sampling schemes, and use the retrieved information in inference.

### 18.1.1 Circuit Preparation schemes

Circuit preparing schemes based on approximation:

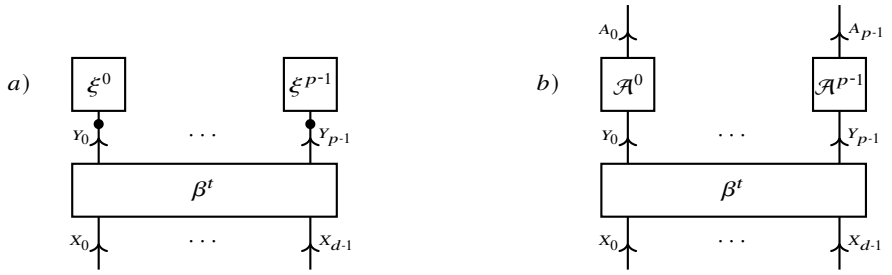



**Fig. 18.1** Ancilla augmentation of an elementary Computation-Activation Network a) by replacing each leg vector of the elementary activation tensor by an directed ancilla augmentation.

- Q-Alchemy [3], Q-Tucker [8]
- Tensor-Network Optimization based (alternating schemes) [72, 71]
- Preparing a quantum circuit given an MPS requires logarithmic depth in the tensor order [53]

Exact circuit preparing schemes for distributions:

- Grover-Rudolph [34], but no quantum advantage [38]
- Uniform controlled rotations [58]
- Quantum Shannon decomposition [80]

Circuit simulation: Since qcreason can prepare quantum circuits to arbitrary tensor networks, it can also be used to simulate quantum circuits (with an overhead!).

- [76, 75]

### 18.1.2 Related Works

Quantum algorithms for linear algebra:

- Contraction by SWAP and Hadamard test (see Appendix)
- Deutsch-Jozsa Algorithm
- HHL algorithm to solve linear equations [37]

## 18.2 Preparation of q-samples to CompActNets

We investigate, how the above circuit encoding schemes can be applied in the preparation of q-samples to Computation-Activation Networks. In particular, we build compositions of computation circuits and activation circuits, for which conditional distributions coincide with Computation-Activation Networks. Note that the resulting circuits are graph-controlled circuits, equivalent to Bayesian networks.

The computation network consists already of directed cores and therefore is already a Bayesian network. The activation network however needs ancilla augmentation.

Elementary Computation-Activation Networks can be augmented by ancilla augmentation of each leg vector in the activation tensor network (see Figure 18.1). We understand ancilla variables as variables in a Bayesian Network having single parents, corresponding with the ancilla augmentation of an elementary tensor. For an example, see Figure 18.4.

To a generic activation hypergraph $\mathcal{G}$, we would do ancilla augmentation for the activation tensor network, where each hidden variable is treated as a distributed qubit. To treat it as a distributed qubit, each hidden activation qubit is prepared in uniform state (i.e. a Hadamard gate acting on a ground state) and will be omitted in the measurement scheme (either not measured, or its measurement ignored). This can be understood from a tensor network perspective, where marginalization means contracting, which is exactly how hidden activation variables are used. For an example in the TT format, see Figure 18.2.

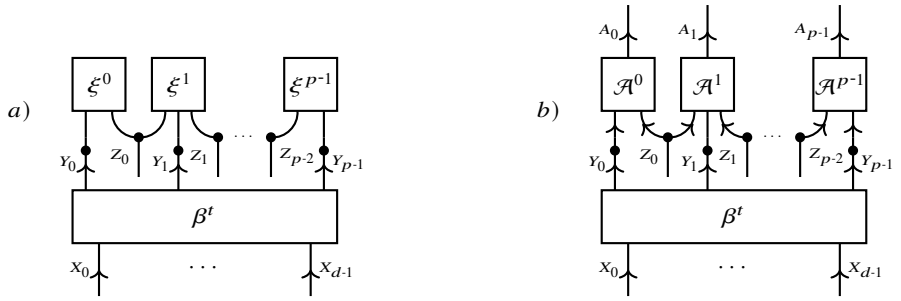



**Fig. 18.2** Ancilla augmentation of an TT Computation-Activation Network a) by a Bayesian Network b). The hidden variables in the TT activation tensor are treated as distributed variables. Marginalization over the hidden variables and conditioning on the activation variables in state 1, we reproduce the Computation-Activation Network.

### 18.2.1 Sampling from Computation-Activation Networks

For Computation-Activation Networks we find a q-sample by concatenating computation circuits and activation circuits (see Figure 18.3).

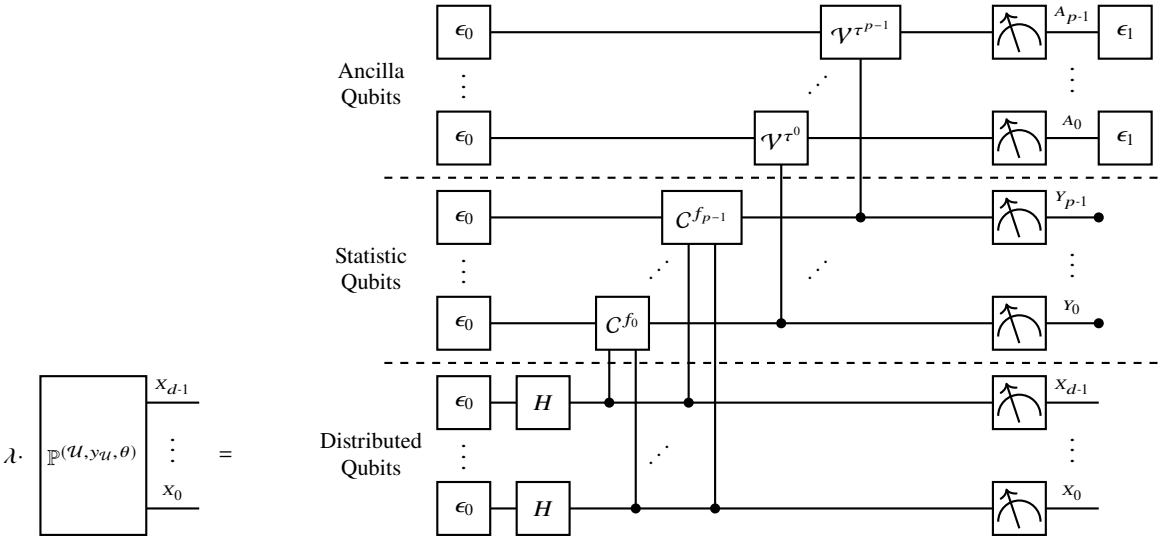



**Fig. 18.3** Quantum Circuit to reproduce a q-sample of an augmented Computation-Activation Network. We measure the distributed qubits $X_{[d]}$ and the ancilla qubits $A_{[p]}$ and reject all samples, where an ancilla qubit is measured as 0.

*Example 18.1 (Toy Accounting Example)* For a more detailed example, let us consider a system of three variables $A1$ Account 1 is booked, $A2$ Account 2 is booked, $F$ a feature on an invoice. Assume the following two rules have to be respected:

- Exactly one account must be booked.
- If feature F is present on the invoice, the account A1 is typically booked.

We formalize this with the statistic

$$t = (X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}) .$$

Any elementary Computation-Activation Network with the statistic $t$ can be realized by the circuit shown in Figure 18.4. For the preparation of the first statistic qubit $Y_0$ to $t_0 = X_{A1} \oplus X_{A2}$ by a computation circuit we exploit that

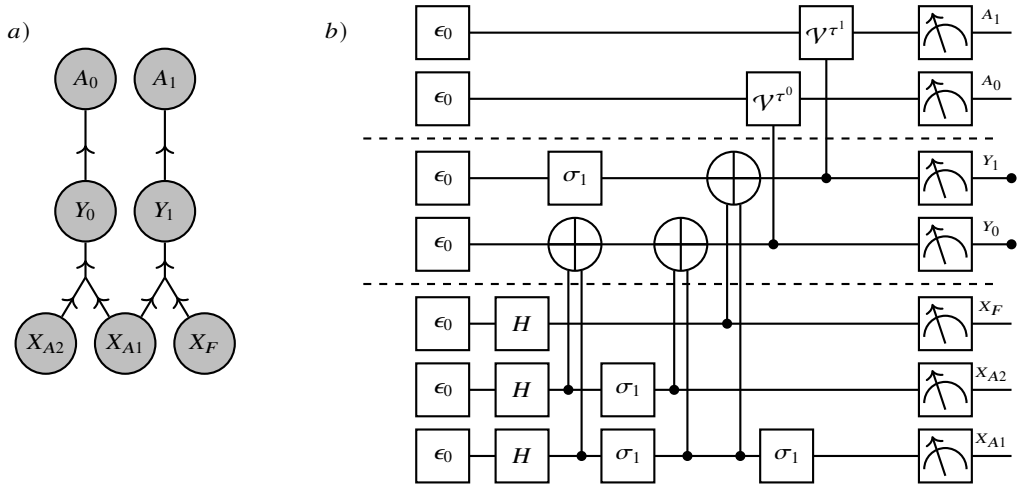



**Fig. 18.4** Circuit to sample from the Computation-Activation Network in the toy accounting Example 18.1.

$$\begin{aligned} X_{A1} \oplus X_{A2} &= \epsilon_1 [X_{A1}] \otimes \epsilon_0 [X_{A2}] + \epsilon_0 [X_{A1}] \otimes \epsilon_1 [X_{A2}] \\ &= (\epsilon_1 [X_{A1}] \otimes \epsilon_0 [X_{A2}]) \oplus (\epsilon_0 [X_{A1}] \otimes \epsilon_1 [X_{A2}]) \end{aligned}$$

where by $\oplus$ we denote coordinatewise summation mod 2. The statistic qubit $Y_0$ is therefore prepared by two controlled NOT gates (see Figure 18.4b).

For the preparation of the second statistic qubit $Y_1$ to $t_1 = X_F \Rightarrow X_{A1}$ we exploit that the implication is True except for the case where the premise $X_F$ is True and the head $X_{A1}$ is False. In out mod 2 calculus, this amounts to

$$X_F \Rightarrow X_{A1} = \mathbb{I} [X_F, X_{A1}] \oplus \epsilon_1 [X_F] \otimes \epsilon_0 [X_{A1}] \ .$$

It follows that the statistic qubit $Y_1$ is prepared by a NOT gate (that is a Pauli-X gate $\sigma_1$) and a second controlled NOT gate (see Figure 18.4b).

For both cases, the preparation of the statistic qubit can be done by two controlled NOT gates exploiting the polynomial sparsity of the connectives.

Any non-elementary Computation-Activation Network with the statistic $t$ can be prepared by the circuit, when choosing a single ancilla qubit, which is uniformly controlled by the qubits $Y_{[2]}$.

## 18.3 Quantum Rejection Sampling

We present the Quantum Rejection Sampling method in two steps:

- **Ancilla augmentation:** Find an ancilla augmentation of a distribution, such that there is a unitary $U$ preparing a state with the augmented distribution as measurement distribution
- **Acceptance amplification:** Amplify the amplitude of the ancilla qubits in state 1 by repeated application of the Grover operator

Compared with the rejection sampling approach on the start augmentation of the distribution, we achieve a square-root speed-up in the expected number of gates to be applied per sample.

We in particular apply this scheme to sample from Computation-Activation Networks, where we apply the q-sample preparation scheme of Sect. 18.2.

### 18.3.1 Acceptance Amplification

Let there be a quantum state, which measurement distribution is an ancilla augmentation $\tilde{\mathbb{P}}[A_{[p]}, X_{[d]}]$ of a target distribution $\mathbb{P}\left[X_{[d]}\right]$. We now show how Grover operators can be used to amplify the acceptance state and improve the efficiency of the rejection sampling method through an increase of the acceptance probability. This is an instance of amplitude amplification.

#### 18.3.1.1 Grover construction

We relate the state $\psi^{\parallel}$ with the event $A_{[p]} = 1_{[p]}$, by constructing it as

$$\psi^{\parallel}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^{t}\left[Y_{[p]}, X_{[d]}\right], \sqrt{\mathbb{P}\left[X_{[d]}\right]} \right\rangle_{[Y_{[p]}, X_{[d]}]}$$

The complement state $\psi^{\perp}$ is constructed if $\lambda \neq 1$ (in which case the acceptance amplilification scheme would not be necessary)

$$\mathbb{P}^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \frac{1}{1-\lambda} \cdot \left(\tilde{\mathbb{P}}[A_{[p]}, Y_{[p]}, X_{[d]}] - \lambda \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \mathbb{P}\left[Y_{[p]}, X_{[d]}\right]\right)$$

and

$$\psi^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sqrt{\mathbb{P}^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right]}\,.$$

We thus have

$$\psi^{0}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sin\left(\frac{\alpha}{2}\right)\psi^{0}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] + \cos\left(\frac{\alpha}{2}\right)\psi^{1}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right]$$

where the $\alpha$ is related to the acceptance probability as

$$\sin\left(\frac{\alpha}{2}\right) = \sqrt{\lambda}\,.$$

#### 18.3.1.2 Circuit Realization

Both reflection operators are realized as their negations using the Pauli-Z scheme (see Sect. 12.4.3). The negations chancel in their concatenation, which is therefore the Grover operator.

#### 18.3.1.3 Ancilla augmentation preserved

The measurement distribution to any quantum state in $V \backslash \mathrm{span}\,(\psi^{\perp})$ is an ancilla augmentation of $\mathbb{P}\left[X_{[d]}\right]$. When amplifying the acceptance probability we stay in the subspace $V$. When we do not overrotate and since we start with $\lambda > 0$ the amplified state is never in span $(\psi^{\perp})$.

### 18.3.2 Optimal rotation numbers

Now the application of $j$ grover operators $G$ on $\psi^{0}$ prepares a state

$$\psi^{j}\left[A_{[p]}, X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\parallel}\left[A_{[p]}, X_{[d]}\right] + \cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\perp}\left[A_{[p]}, X_{[d]}\right]$$

The state is closest to $\psi^{\parallel}$ when

$$\left(\frac{1}{2}+j\right)\alpha \approx \frac{\pi}{2}\,.$$

Estimating for small $\lambda$ (tight for small $\lambda$)

$$\sqrt{\lambda} = \sin\left(\frac{\alpha}{2}\right) \leq \frac{\alpha}{2}$$

we get a bound

$$j^{\text{opt}} \leq \frac{\pi}{4\sqrt{\lambda}}\,.$$

This shows that we have a square root improvement on classical rejection sampling, which would require $\frac{1}{\lambda}$ repetitions to produce a sample. Note however that we compare quantum circuit length with the number of classical repetitions in sampling.

### 18.3.3 Applications on CompActNets

The above scheme prepares the ancilla augmented Computation-Activation Network

$$\tilde{\mathbb{P}}[A_{[p]}, Y_{[p]}, X_{[d]}] = \lambda \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \mathbb{P}\left[X_{[d]}\right]\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} + (1-\lambda)\mathbb{P}^{\perp}\left[X_{[d]}, Y_{[p]}, A_{[p]}\right]$$

and the prepared state is the combination of q-samples

$$\psi^0\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sqrt{\lambda} \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \sqrt{\mathbb{P}\left[X_{[d]}\right]}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} + \sqrt{(1-\lambda)}\sqrt{\mathbb{P}^{\perp}\left[X_{[d]}, Y_{[p]}, A_{[p]}\right]}\,.$$

Sampling from the augmented Computation-Activation Network can be also done classically by forward sampling: Just draw the variables from uniform, calculate the value qubit by a logical circuit inference and accept with probability by the computed value. To achieve a quantum advantage we manipulate the circuit first by the amplitude amplification scheme and increase the acceptance probability.

### 18.3.4 Examples

## 18.4 Moment Amplification

Instead of measuring the phases of the effective Grover operator, we now investigate the effect of uncontrolled Grover operations on the distributions. We find an analogon in discrete increases of the canonical parameters when the measurement distribution of the start state is in an exponential family.

### 18.4.1 Interpretation of Grover rotations

Rotating $j$ times across the initial state then gives the tensor product of the antisymmetric ancilla state with

$$\psi^j\left[X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\mathbb{P}^0, f}\left[X_{[d]}\right] + \cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\mathbb{P}^0, \neg f}\left[X_{[d]}\right]\,.$$

This reflection can be performed by the operator on the distributed qubits

$$U^t \left( \delta \left[ X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]} \right] - 2\epsilon_0 \left[ X^{\text{in}}_{[d]} \right] \otimes \epsilon_0 \left[ X^{\text{out}}_{[d]} \right] \right) U \, .$$

#### 18.4.1.1 Mean parameter interpretation

The mean parameter of $f$ with respect to the measurement distribution is

$$\mu^j = \left\langle \mathbb{P}^j \left[ X_{[d]} \right], f \left[ X_{[d]} \right] \right\rangle_{[\varnothing]} = \sin\left( \left( \frac{1}{2} + j \right) \alpha \right)^2 = \sin\left( (1+2j) \sin^{-1}\left( \sqrt{\mu^0} \right) \right)^2$$

Here we used that $\alpha = 2 \cdot \sin^{-1}\left( \sqrt{\mu^0} \right)$.

#### 18.4.1.2 Canonical parameter interpretation

**Lemma 18.1** *Works also for more generic measurement distributions! Let $\psi^0$ be the q-sample of the elementary Computation-Activation Network with canonical parameters $(\mathcal{U}, y_{\mathcal{U}}, \theta)$, and let us amplify the formula $f_\ell$. Then $\psi^j$ is the q-sample of the elementary Computation-Activation Network with canonical parameters*

$$\mathcal{U}^j = \begin{cases} \mathcal{U} \cup \{\ell\}, y^\ell_{\mathcal{U}} = (y_{\mathcal{U}}, 1) & \text{if} \quad \sin\left( (1 + 2 \cdot j) \sin^{-1}\left( \sqrt{\mu_0 \left[ L = \ell \right]} \right) \right) = 1 \\ \mathcal{U}, y_{\mathcal{U}} & \text{else} \end{cases}$$

*and*

$$\theta_j \left[ L = \ell \right] = \begin{cases} 0 & \text{if} \quad \sin\left( (1 + 2 \cdot j) \sin^{-1}\left( \sqrt{\mu_0 \left[ L = \ell \right]} \right) \right) = 1 \\ \theta \left[ L = \ell \right] + \frac{\cos^2\left(\frac{\alpha}{2}\right)\left(1 - \cos^2\left(\left(\frac{1}{2}+j\right)\alpha\right)\right)}{\sin^2\left(\frac{\alpha}{2}\right) \cdot \cos^2\left(\left(\frac{1}{2}+j\right)\alpha\right)} & \text{else} \end{cases} .$$

*Proof.* Notice, that

$$\left\langle \mathbb{P}^0 \left[ X_{[d]} \right], \beta^{f_\ell} \left[ Y_\ell, X_{[d]} \right] \right\rangle_{[Y_\ell]} = \begin{pmatrix} \left(\cos\left(\frac{\alpha}{2}\right)\right)^2 \\ \left(\sin\left(\frac{\alpha}{2}\right)\right)^2 \end{pmatrix} [Y_\ell] = \begin{pmatrix} 1 - \mu_0 \left[ L = \ell \right] \\ \mu_0 \left[ L = \ell \right] \end{pmatrix} [Y_\ell]$$

and

$$\left\langle \mathbb{P}^j \left[ X_{[d]} \right], \beta^{f_\ell} \left[ Y_\ell, X_{[d]} \right] \right\rangle_{[Y_\ell]} = \begin{pmatrix} \left(\cos\left(\left(\frac{1}{2} + j\right)\alpha\right)\right)^2 \\ \left(\sin\left(\left(\frac{1}{2} + j\right)\alpha\right)\right)^2 \end{pmatrix} [Y_\ell] \begin{pmatrix} 1 - \mu_j \left[ L = \ell \right] \\ \mu_j \left[ L = \ell \right] \end{pmatrix} [Y_\ell]$$

If $\mu_j \left[ L = \ell \right] = 1$ (that is $\sin\left(\left(\frac{1}{2} + j\right)\alpha\right) = 1$), the amplitude amplification amounts to adding $f_\ell$ as a hard constraint. This is equal to $\mathcal{U}^j = \mathcal{U} \cup \{\ell\}$ and $y^j_{\mathcal{U}} = (y_{\mathcal{U}}, 1)$.

If $\mu_j \left[ L = \ell \right] \neq 1$ we choose $\theta_\Delta \left[ L = \ell \right] \geq 0$ as in the claim and have after some algebra

$$\frac{\left(\cos\left(\frac{\alpha}{2}\right)\right)^2}{\left(\cos\left(\frac{\alpha}{2}\right)\right)^2 + \theta_\Delta \left[ L = \ell \right] \cdot \left(\sin\left(\frac{\alpha}{2}\right)\right)^2} = \left(\cos\left(\left(\frac{1}{2} + j\right)\alpha\right)\right)^2$$

and therefore

$$\left\langle \mathbb{P}^j \left[ X_{[d]} \right], \beta^{f_\ell} \left[ Y_\ell, X_{[d]} \right] \right\rangle_{[Y_\ell]} = \left\langle \exp\left[ \theta_\Delta \left[ L = \ell \right] \cdot f_\ell \left[ \theta \right] \right] \mathbb{P}^0 \left[ X_{[d]} \right], \beta^{f_\ell} \left[ Y_\ell, X_{[d]} \right] \right\rangle_{[Y_\ell | \varnothing]} .$$

Thus, the rotation of the q-sample corresponds with an increase of the canonical parameter $\theta_0 \left[ L = \ell \right]$ by $\theta_\Delta \left[ L = \ell \right]$. □

#### 18.4.1.3 Maximum Entropy

When $\left(\frac{1}{2}+j\right)\alpha \leq \frac{\pi}{2}$ then all amplitudes remain positive, and $\psi^j\left[X_{[d]}\right]$ is a q-sample. What is more, we can show that the property of maximum entropy with respect to a statistic containing the amplified formula.

**Theorem 18.1** *Let $\psi^0$ be the q-sample of a maximum entropy distribution with respect to the uniform base measure and the statistic $\mathcal{F}$ containing the amplified formula, and let $j \in \mathbb{N}$ be a rotation number such that $\left(\frac{1}{2}+j\right)\alpha \leq \frac{\pi}{2}$. Then also $\psi^j$ is the q-sample of a maximum entropy distribution.*

*Proof.* By Lem. 18.1 the amplitude amplified state is an elementary Computation-Activation Network and thus a maximum entropy distribution. □

Notice, that we can only increase the canonical parameter by amplitude amplification. Decreasing could be done by amplitude amplification on $\neg f_\ell$ instead.

### 18.4.2 Moment Matching

We now investigate which Computation-Activation Networks can be prepared directly with amplitude amplification. While we already have studied sparse representations of the computation network parts, we here apply amplitude amplification by Grover rotations as an activation mechanism.

We orient on the iterative proportional fitting algorithm, which is used to estimate the canonical parameters in exponential families [82] and more generally of Computation-Activation Networks [30].

The algorithm loops over the moment estimation and moment amplification subroutines, which have been introduced in the sections above. The idea is to first compute the current moments of a distribution and second amplify one.

We understand this rotation as a moment matching operation to the formula $f$, see Algorithm 11.

**Algorithm 11** Quantum Circuit Moment Matcher

**Require:** Formulas $f_p\left[X_{[d]}\right]$ for $\ell \in [p]$, vector $\mu_D[L]$ of mean parameters
**Ensure:** Circuit $U$ preparing a state (when applied on the ground state) which measurement distribution matched $\mu_D[L]$

$U \leftarrow (\sigma_1 \circ H)[A_{\text{in}}, A_{\text{out}}] \otimes \left(\bigotimes_{k\in[d]} H\left[X_{k,\text{in}}, X_{k,\text{out}}\right]\right)$
**while** Convergence criterion not met **do**
Estimate $\mu[L]$,

- either by particle bases inference
- or by quantum counting (measuring the eigenvalue of the Grover operator), see Sect. 12.4.4

Select an $\ell \in [p]$, which moment has to be updated (e.g. by comparison of $\mu[L]$ with $\mu_D[L]$)
Choose whether to amplify $f_\ell$ or $\neg f_\ell$ based on whether the estimated mean is smaller than the target mean
Compute optimal $j$ to match $\mu_D[L]$
Extend Circuit $U$ by $j$ alternations of

- $C^{f_\ell}$ (effectively an reflection across the subspace spanned by one-hot encoded models)
- reflections across the current state (by $U(I - 2e_0e_0)U$)

**end while**

We can use Algorithm 11 as a preparation algorithm of a q-sample before doing ancilla augmentation as in the main section. Note that is important to keep track of the canonical parameters in the preparation. When activating the statistic qubits, only the difference of the already prepared canonical parameters and the target canonical parameters has to be used (importance sampling interpretation).

## 18.5 Extension: Sampling from proposal distributions

We can prepare basis circuit encodings to selection augmented formulas, in this way introducing formula selecting networks.

**Idea for an inductive reasoning scheme:** Prepare a q-sample from the empirical distribution and the current distribution. Then prepare the basis circuit encodings, where the selection variables are shared and the distributed variables assigned to the prepared samples. Now, the ancilla qubits can be designed to $\epsilon_1$ and $\epsilon_0$ accordingly. The rejection sampling scheme on both ancillas being 1 and the measurement of $L$ prepares then the distribution

$$\left\langle \left\langle \mathbb{P}^D \left[ X_{[d]} \right], \sigma^t \left[ X_{[d]}, L \right] \right\rangle_{[L]}, \left\langle \tilde{\mathbb{P}} \left[ X_{[d]} \right] \right\rangle_{[L]} \right\rangle_{[L|\varnothing]}$$

That is, the probability of selecting $\ell$ is proportional to

$$\mu_D \left[ L = \ell \right] \cdot \left( 1 - \tilde{\mu} \left[ L = \ell \right] \right)$$

and thus prefers formulas, which have a large empirical mean, but a small current mean.

**Open Question:** Since the distribution is "similar" to $\exp\left[ \mu_D \left[ L = \ell \right] - \tilde{\mu} \left[ L = \ell \right] \right)]$ (terms appear in Taylor of first order), can we tune the distribution with an inverse temperature parameter $\beta$?

# Chapter 19
# Probabilistic Relational Models

**Abstract** Computation networks in first-order logic contain incoming axes to the substitution and the semantic structure. It is often not possible to contract them and therefore to perform reasoning schemes on them. We in this chapter investigate methods to reduce reasoning to the activation network, a strategy which we call *lifting*. First of all, we review inference schemes developed in first-order logic. We further investigate, how Computation-Activation Networks can be trained given data in the form of a first-order logic world.

## 19.1 Ideas

We here understand lifting as the reduction of inference in structured representations to factored representations. In the perspective of Computation-Activation Networks, we find lifting schemes as reductions to inference in the activation network. We draw on the representations investigated in Chapter 4, where the computation network contains the FOL atoms, and the activation the connectives acting on them.

We investigate two lifting schemes:

- **FOL lifted inference:** Restrict the computation networks implicitly by contraction equations and restrict to the activation network parts of these equations. Simplify the activation network based on substitutions and propositional inference.
- **Probabilistic Relational Models (PRM):** Represent the likelihood of a single observed FOL world by the likelihood of a dataset in a factored representation.

## 19.2 Inference in FOL

Substitutions are used to lift inference to propositional logics:

- Represent the formulas to be infered with the same computation network of atomic formulas.
- Represent the formulas with an entangled activation network (whereas before they were disentangled, since refering to different atomic sentences).

### 19.2.1 Manipulation of contraction equations

Universally quantified FOL formulas are equations of FOL Computation-Activation Networks to the trivial tensor.

We understand inference as manipulations in the activation network.

- **Substituion**: Substitution of variables to represent multiple sentences by same activation network. The task of unification is to find a substitution, such that two computation networks can be merged with same term variables.

- **Propositional inference**: Manipulations in activation network, as for constraint networks
- **Restrictions to parts**: Any sub-network of the activation network in a contraction equation poses an implied contraction equation.

#### 19.2.1.1 Substitution

Substitution is the contraction with one-hot encodings to the term variables. It is easy to see, that when a contraction equation holds, it also holds for any substitution of the term variables. Note that substitution changes the computation network, but not the activation network.

#### 19.2.1.2 Propositional inference

We understand propositional inference as the simplication of the activation network. Whenever a propositional formula is infered by the activation network, we can add it to the activation network as a new decorated edge.

#### 19.2.1.3 Extraction of entailed formulas

We can extract entailed formulas from a contraction equation by restricting the activation network to a sub-network, as we show as the next theorem. Therefore any result of the propositional inference, in the shape of a Boolean tensor added to the activation network, can be used to extract entailed formulas.

**Theorem 19.1** *Let $\tau^{\mathcal{G}}$ be an activation network of boolean tensors on a hypergraph $\mathcal{G}$ and let $\tilde{\mathcal{E}} \subset \mathcal{E}$. Let $g_{[d]}$ be an arbitrary set of first order predicates with the term variables If*

$$\left\langle \tau^{\mathcal{G}} \cup \{\beta^k \left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \right\rangle_{\left[O_{[n]}\right]} = \mathbb{I}\left[X_{[n]}\right]$$

*then also*

$$\left\langle \{\tau^e \left[X_e\right] \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\beta^k \left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \right\rangle_{\left[O_{[n]}\right]} = \mathbb{I}\left[X_{[n]}\right]$$

*Proof.* Notice, that the contraction with the smaller set of edges $\tilde{\mathcal{E}} \subset \mathcal{E}$ is a Boolean tensor. By the monotonicity of the contraction, we have that the support of the contraction is smaller (in the partial order of tensors) to that of all edges. Since by assumption the latter is the ones tensors, the former must also be the ones tensor. □

### 19.2.2 Unification

Unification of atomic sentences: Find a substitution to the variables, such that both sentences are represented by the same computation network.

Compressing of complex sentences:

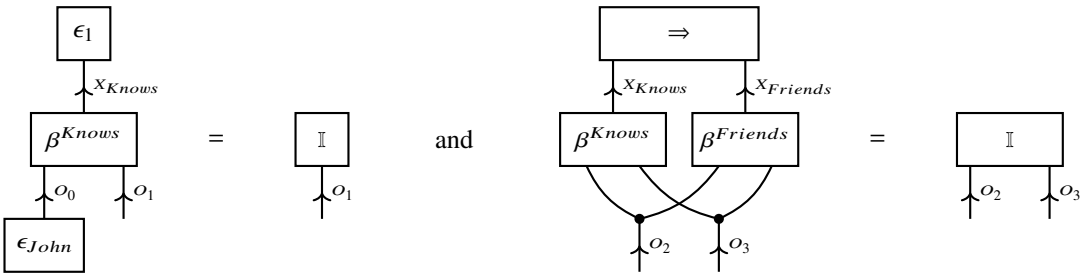
- Find a unification of the atomic sentences in both complex sentences
- Represent the substituted complex sentences by a constraction of the activation networks

### 19.2.3 Modus ponens

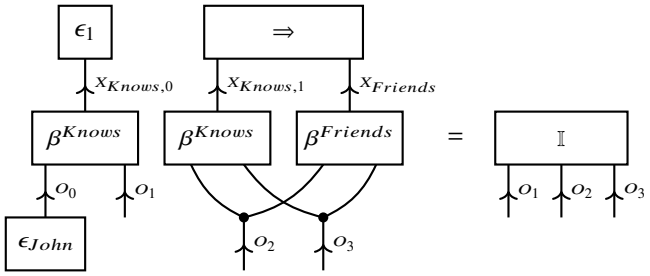
*Example 19.1 (Lifted modus ponens)* For example we have

- John knows everyone: Knows(John,z)
- People who know each other are friends: Knows(x,y) ⇒ Friends(x,y)

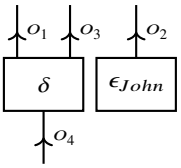
We illustrate the derivation of the entailed formula Friends(John,z) (i.e. John is a friend of everyone) by modus ponens. The formulas are represented by the contraction equation



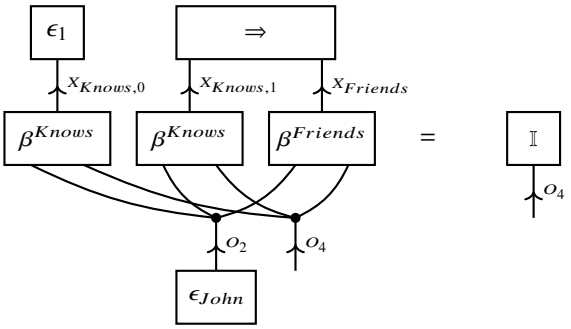

We combine them by the tensor product of both sides as



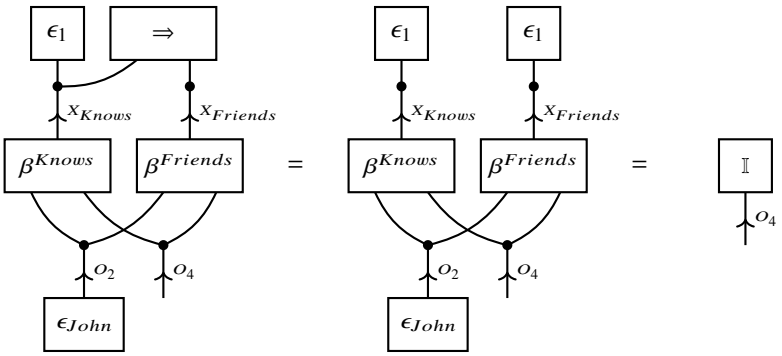

The necessary substitution for the unification of both is $O_2 \mapsto$ John and $O_1, O_3 \mapsto O_4$. This amounts to contraction of both sides with the tensor



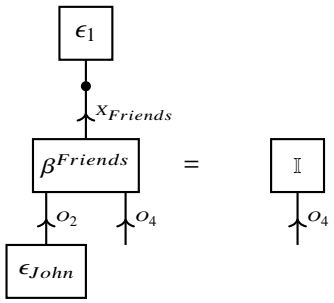

We contract this tensor to both sides of the combined contraction equation and get




Now we notice that the computation network of the variables $X_{Knows,0}$ and $X_{Knows,1}$ are equivalent (what was the goal of the unification). We simplify the equation to



Here we applied modus ponens on the activation network in the first equation. Restricting the activation network to the variable $X_{Friends}$ is then the derived formula Friends(John,z) (i.e. John is a friend of everyone):




### 19.2.4 Contraction equations

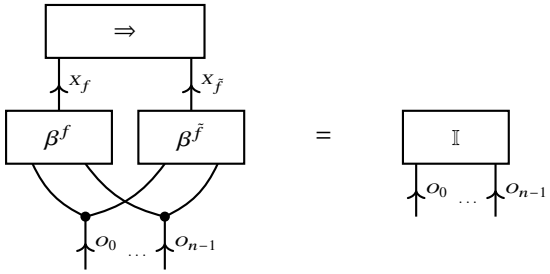
Quantified formulas understood are contraction equations.

Understanding them as hard constraints, one can infer parts of the substitution tensors, which holds for all supported worlds.

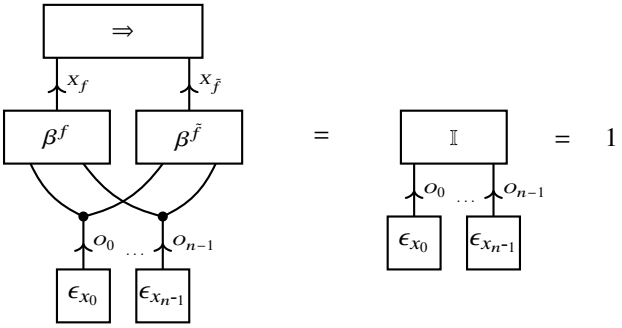
*Example 19.2 (Modus ponens on the substitution structure)* Repetition? We here examplify an application of unification to apply the modus ponens inference rule in the substitution structure. Let us consider the formula

$$\forall x_{[d]} : f\left[X_{[d]} = x_{[d]}\right] \Rightarrow \tilde{f}\left[X_{[d]} = x_{[d]}\right],$$

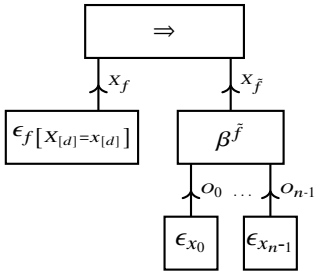
which corresponds to the contraction equation




When knowing for some $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ that $f\left[X_{[d]} = x_{[d]}\right] = 1$, this is the contraction equation we conclude that $\tilde{f}\left[X_{[d]} = x_{[d]}\right] = 1$.

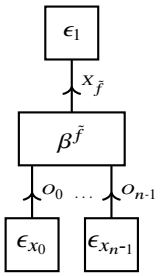



We simplify the left hand side using basis calculus to




$= \quad 1$

When known that $f\left[X_{[d]} = x_{[d]}\right] = 1$, this further simplifies to




$= \quad 1$

which is equal to $\tilde{f}\left[X_{[d]} = x_{[d]}\right] = 1$.

More sophisticated reasoning schemes based on contraction equations are applied in [77, 81], where recursive linear equations are derived.

## 19.3 Probabilistic Relational Models

So far we have studied Markov Logic Networks in propositional logic as probability distributions over worlds. In FOL they define probability distributions over relations in worlds with a fixed set of objects. More generally, such models are probabilistic relational models (see for an overview [28].

### 19.3.1 Maximum entropy distributions on counted formulas

We investigate maximum entropy distributions to the statistics

$$\left\langle q_\ell|_{x_W}\left[O_{q_\ell}\right]\right\rangle_{[\varnothing]}$$

for $\ell \in [p]$.

We assume, that all formulas have the same term variables. This is not restricting since we can always trivially extend a formula, which would just scale the count by a factor.

We in this section treat random worlds in first-order logic with fixed domains $\mathcal{D}$. This is often called unique name assumptions and domain closure assumption [74].

Then we are in finite dimensional cases, where we can apply the results of Chapter 14.

#### 19.3.1.1 Mean polytope

A scaling of the $|\mathcal{D}| = 1$ case, faces also get scaled.

### 19.3.2 Hybrid First-Order Logic Networks

Following [69] Markov Logic Networks in first-order logic are templates for distributions, which instantiate random worlds when choosing a set of objects $\mathcal{D}$. Given a fixed set of constants, they then define a distribution over the worlds, which objects correspond with the constants. This applies database semantics, where only those worlds are considered, where the unique name and domain closure assumptions given a set of constants are satisfied.

We count the number of true groundings to a formula by

$$\#q_\ell|_{x_W} = \left\langle q_\ell|_{x_W}\left[O_{q_\ell}\right]\right\rangle_{[\varnothing]} .$$

Using these counts as statistics, we now define Hybrid First-Order Logic Networks as a Computation Activation Network.

**Definition 19.1 (Hybrid First-Order Logic Networks (HFLN))** Let there be a set $\mathcal{D}$ of objects and a set $Q$ of first-order logic formulas. Further let $(\mathcal{U}, y_\mathcal{U}, \theta)$ as in Def. **??** be a tuple of a subset $\mathcal{U} \subset [p]$, a tuple $y_\mathcal{U} \in \bigtimes_{\ell\in\mathcal{U}}[2]$ and $\theta\,[L] \in \mathbb{R}^p$. We then define the Hybrid First-Order Logic Network as the distribution

$$\begin{aligned}\mathbb{P}^{\#Q,(\mathcal{U},y_\mathcal{U},\theta)}\,[X_W] = \Big\langle &\{\beta^{\#q_\ell}\,[Y_\ell, X_W] \,:\, \ell \in [p]\} \\ &\cup \{\alpha^{\ell,\theta}\,[Y_\ell] \,:\, \ell \in [p]\} \cup \{\kappa^{\ell,(\mathcal{U},y_\mathcal{U})}\,[Y_\ell] \,:\, \ell \in [p]\}\Big\rangle_{\left[X_W|\varnothing\right]}\end{aligned}$$

where

$$\kappa^{\ell,(\mathcal{U},y_\mathcal{U})}\,[Y_\ell] = \begin{cases}\epsilon_0\,[Y_\ell] & \text{if} \quad \ell \in \mathcal{U},\ y_\ell = 0 \\ \epsilon_{n_\ell-1}\,[Y_\ell] & \text{if} \quad \ell \in \mathcal{U},\ y_\ell = 1 \\ \mathbb{I}\,[Y_\ell] & \text{else}\,.\end{cases}$$

The mean parameter polytope is the convex hull of the vectors

$$\sigma^Q\,[X_W = x_W, L]$$

to the worlds $x_W$ with $\nu\,[X_W = x_W] = 1$. These vectors store are the counts of satisfied groundings to each formula, that is

$$\sigma^Q\,[X_W = x_W, L] = \left|\{o_{q_\ell} \,:\, q_\ell|_{x_W}\left[O_{q_\ell} = o_{q_\ell}\right] = 1\}\right| .$$

### 19.3.3 Propositionalization

Let us notice, that different to the case of propositional Hybrid Logic Networks treated in Chapter **??**, the statistic does not consist of boolean features, when formulas contain variables and we have multiple objects. One could, however, replace each $q_\ell$ by the set of the possible groundings, i.e. substitutions of the formulas variables by any tuple of objects in $\mathcal{D}$. The resulting distribution would be an Hybrid Logic Network with boolean statistic, which coincides with the Hybrid First-Order Logic Network when posing certain weight sharing conditions on $\theta$. The downside of this construction is the increase in the number of features from $p$ to $\sum_{\ell\in[p]} |\mathcal{D}|^{|O_{q_\ell}|}$. This polynomial in the cardinality of the domain set increase poses significant computational challenges, see [69].

We now show that the Hybrid First-Order Logic Network can be propositionalized to a Hybrid Logic Network in propositional logic.

**Lemma 19.1** *We have*

$$\begin{aligned}&\left\langle \beta^{\#q_\ell}\,[Y_\ell, X_W]\,, \alpha^{\ell,\theta}\,[Y_\ell]\right\rangle_{[X_W]} \\ &\quad = \left\langle \{\beta^{q_\ell|_{x_W}[x_W,o_{q_\ell}]}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell} \in [r_{q_\ell}]\} \cup \{\alpha^{\ell,\theta}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell} \in [r_{q_\ell}]\}\right\rangle_{[X_W]}\end{aligned}$$

*and*

$$\left\langle \beta^{\#q_\ell}\left[Y_\ell, X_W\right], \kappa^{\ell,(\mathcal{U},y_\mathcal{U})}\left[Y_\ell\right]\right\rangle_{[X_W]} = \left\langle \{\beta^{q_\ell|_{x_W}[x_W,o_{q_\ell}]}\left[Y_{o_{q_\ell}}\right] : o_{q_\ell} \in [r_{q_\ell}]\} \cup \{\kappa^{\ell,(\mathcal{U},y_\mathcal{U})}\left[Y_{o_{q_\ell}}\right] : o_{q_\ell} \in [r_{q_\ell}]\}\right\rangle_{[X_W]}.$$

*Proof.* The first claim holds, since for any world $x_W$ we have

$$\begin{aligned}\left\langle \beta^{\#q_\ell}\left[Y_\ell, X_W\right], \alpha^{\ell,\theta}\left[Y_\ell\right]\right\rangle_{[X_W=x_W]} &= \prod_{o_{q_\ell}\,:\,q_\ell|_{x_W}[x_W,o_{q_\ell}]=1} \exp\left[\theta\left[L=\ell\right]\right] \\ &= \left\langle \{\beta^{q_\ell|_{x_W}[x_W,o_{q_\ell}]}\left[Y_{o_{q_\ell}}\right] : o_{q_\ell} \in [r_{q_\ell}]\} \cup \{\alpha^{\ell,\theta}\left[Y_{o_{q_\ell}}\right] : o_{q_\ell} \in [r_{q_\ell}]\}\right\rangle_{[X_W=x_W]}.\end{aligned}$$

For $\ell \notin \mathcal{U}$ the second claim is trivial, since any contraction of a basis encoding with a trivial head is trivial. If $\ell \in \mathcal{U}$ and $y_\ell = 0$ then

$$\begin{aligned}\left\langle \beta^{\#q_\ell}\left[Y_\ell, X_W\right], \kappa^{\ell,(\mathcal{U},y_\mathcal{U})}\left[Y_\ell\right]\right\rangle_{[X_W=x_W]} &= \begin{cases}1 & \text{if} \quad \forall o_{q_\ell} : q_\ell|_{x_W}\left[x_W, o_{q_\ell}\right] = 0 \\ 0 & \text{else}\end{cases} \\ &= \left\langle \{\beta^{q_\ell|_{x_W}[x_W,o_{q_\ell}]}\left[Y_{o_{q_\ell}}\right] : o_{q_\ell} \in [r_{q_\ell}]\} \cup \{\kappa^{\ell,(\mathcal{U},y_\mathcal{U})}\left[Y_{o_{q_\ell}}\right] : o_{q_\ell} \in [r_{q_\ell}]\}\right\rangle_{[X_W=x_W]}.\end{aligned}$$

For $y_\ell = 1$ this holds analogously. □

Each substitution of the variables in $q_\ell$ by objects in $\mathcal{D}$, which satisfies the formula in the world $x_W$, therefore provides a factor of $\exp\left[\theta\left[L=\ell\right]\right]$ to the probability of $x_W$.

### 19.3.4 Conditioning on an Importance Formula

To reduce the number of object tuples influencing the probability distribution, we now condition Hybrid First-Order Logic Networks on situations where a formula, called the importance formula, has a fixed grounding tensor.

To this end, we mark pairs of term indices relevant to the distributions by an auxiliary index $j \in [m]$. Given a set $\{o^j_{[n]} : j \in [m]\}$ of indices to the important tuples we build a set encoding (see Def. 7.1)

$$\underline{p} = \sum_{j\in[m]} \left( \bigotimes_{l\in[n]} \epsilon_{o^j_l}\left[O_l\right] \right).$$

We interpret the tensor $\underline{p}$ as the grounding of a formula, which we call the importance formula.

To have a constant importance formula we define a syntactic representation and restrict the support of the Hybrid First-Order Logic Network to those world coinciding with groundings of the importance formula coinciding with $\underline{p}$ by designing a base measure

$$\nu^{\underline{p}}\left[X_W\right] = \begin{cases}1 & \text{if} \quad p|_{x_W}\left[O_p\right] = \underline{p} \\ 0 & \text{else}\end{cases}.$$

The base measure restricts the Hybrid First-Order Logic Network to be those worlds, where $p|_{x_W}$ is coincides with the fixed tensor $\underline{p}$. Intuitively, $p|_{x_W}$ represents certain evidence about a first-order logic world, whereas other formulas are uncertain.

### 19.3.5 Decomposition of the Log Likelihood

To reduce the likelihood of a world to we make the assumption that all formulas in a Hybrid First-Order Logic Network are of the form

$$q_\ell \left[O_{q_\ell}\right] = \left(p \left[O_{[n]}\right] \Rightarrow h_\ell \left[O_{q_\ell}\right]\right) \tag{19.1}$$

that is a rule with the importance formula being the premise. In particular, we assume, that they depend on all term variables $O_{[n]}$. If this is not the case, we extend the formula trivially on the missing term variables. When this assumption holds, we can think of the importance formula as a conditions on individuals to satisfy a statistical relation given by $h$.

Towards connecting with propositional logic, we further make the assumption, that we can decompose the formula $h_\ell$ in what we will call extraction formulas.

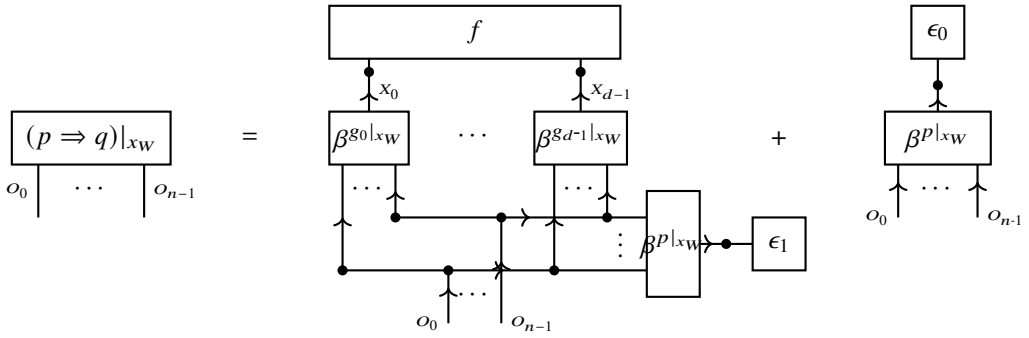
**assumption 1.** *We assume that there exist formulas $\{g_k \left[X_W, O_{[n]}\right] : k \in [d]\}$, which we refer to as atom extraction formulas, and an importance formula $p \left[O_{[n]}\right]$ such that the following holds. To each first-order logic formula $q_\ell$ there is another first-order logic formula $h_\ell \left[O_{q_\ell}\right]$ and a propositional formula $f_\ell \left[X_{[d]}\right]$ such that*

$$q_\ell \left[X_W, O_{[n]}\right] = \left(p \left[X_W, O_{[n]}\right] \Rightarrow h_\ell \left[X_W, O_{[n]}\right]\right)$$

*and*

$$h_\ell \left[O_{[n]}\right] = \left\langle \{f_\ell \left[X_{[d]}\right]\} \cup \{\beta^{g_k} \left[X_k, X_W, O_{[n]}\right] : k \in [d]\}\right\rangle_{\left[O_{[n]}\right]} .$$

We depict the assumption, that any formula is of the form (19.1) in the diagram




where the second summand depends only on the query $p$ and therefore does not appear in the likelihood.

### 19.3.6 Reduction to Propositional Logic

We now make additional assumptions to decompose the partition function of an Hybrid First-Order Logic Network as a product of Hybrid Logic Network partition functions.

**assumption 2.** *Given an importance formula $p$ and a tensor $\underline{p}$ we consider the base measure*

$$\nu^{\underline{p}} \left[X_W\right] = \begin{cases} 1 & \text{if} \quad p|_{x_W} \left[O_p\right] = \underline{p} \left[O_p\right] \\ 0 & \text{else} \end{cases}$$

*of worlds with grounding of $p$ by $\underline{p}$. Let further enumerate by $j \in [m]$ the support of $\underline{p}$, that is $o^j_{[n]}$ are the indices of $O_{[n]}$ with $\underline{p} \left[O_{[n]} = o^j_{[n]}\right] = 1$. We assume that with respect to $\underline{p}$ the variables*

$$\left(g_k|_{x_W} \left[O_{[n]} = o^j_{[n]}\right]\right) \tag{19.2}$$

*are for $k \in [d]$ and $j \in [m]$ independent and uniformly distributed.*

When the above assumption holds, we now show that the probability of a first-order logic world with respect to a Hybrid First-Order Logic Network coincides with the likelihood of a dataset in a propositional Hybrid Logic Network.

**Theorem 19.2** *Let there be a set of formulas $\mathcal{Q}$ such that Assumption 1 and Assumption 2 hold with an importance formula $p$ and a tensor $\underline{p}$. Then for any $(\mathcal{U}, y_{\mathcal{U}}, \theta)$ and any world $x_W$ with $p|_{x_W} = \underline{p}$ we have*

$$\frac{1}{m} \ln \left[ \mathbb{P}^{\#\mathcal{Q},(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ X_W = x_W \middle| p|_{x_W} = \underline{p} \right] \right] = \mathbb{H} \left[ \mathbb{P}^{D}, \mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)} \right] - \frac{\ln \left[ \left\langle \nu^{\underline{p}} \left[ X_W \right] \right\rangle_{[\varnothing]} \right]}{m}$$

*where $\mathcal{F}$ is the set of propositional equivalents to $\mathcal{Q}$ (see Assumption 1) and $D$ the data map with evaluation at $j \in [m]$ by the enumerated non-vanishing coordinates of $\underline{p}$ $\left[ O_p \right]$*

$$D(j) = \left( g_0|_{x_W} \left[ O_{[n]} = o^j_{[n]} \right], \ldots, g_{d-1}|_{x_W} \left[ O_{[n]} = o^j_{[n]} \right] \right).$$

To show the theorem, we show first in the following lemma the factorization of the partition function of the Hybrid First-Order Logic Network.

**Lemma 19.2** *Under the assumptions of Thm. 19.2, we have*

$$\begin{aligned}
&\left\langle \beta^{\mathcal{Q}|_{\mathcal{D}}} \left[ Y_{[p]}, X_W \right], \tau^{(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y_{[p]} \right], \nu^{\underline{p}} \left[ X_W \right] \right\rangle_{[\varnothing]} \\
&\quad = \left\langle \nu^{\underline{p}} \left[ X_W \right] \right\rangle_{[\varnothing]} \cdot \left( \prod_{o_{[n]} : \underline{p}\left[ O_{[n]} = o_{[n]} \right] = 0} \tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y = 1 \right] \right) \cdot \\
&\qquad \left( \frac{1}{2^d} \cdot \left\langle \beta^{t} \left[ Y_{[p]}, X_{[d]} \right], \tau^{(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y_{[p]} \right] \right\rangle_{[\varnothing]} \right)^m .
\end{aligned}$$

*Proof.* Using the assumption on the structure of the formulas we have

$$\#q_\ell|_{x_W} = \#\neg p|_{x_W} + \#(p \wedge h_\ell)|_{x_W}$$

and for any $x_W$

$$\begin{aligned}
&\left\langle \beta^{\#q_\ell} \left[ Y_\ell \right], \tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y \right], \nu^{\underline{p}} \left[ X_W \right] \right\rangle_{[X_W = x_W]} \\
&\quad = \left( \prod_{o_{[n]} : \underline{p}\left[ O_{[n]} = o_{[n]} \right] = 0} \tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y = 1 \right] \right) \cdot \left\langle \beta^{\#(p \wedge h_\ell)} \left[ Y_\ell \right], \tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y_\ell \right], \nu^{\underline{p}} \left[ X_W \right] \right\rangle_{[X_W = x_W]}
\end{aligned}$$

Here by $\tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} [Y]$ we denote the two-dimensional realization of the activation core. We use that the grounding tensor of $p$ is constant among the by $\nu^{\underline{p}} [X_W]$ supported worlds and thus

$$\begin{aligned}
&\left\langle \beta^{\#(p \wedge h_\ell)} \left[ Y_\ell \right], \tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y_\ell \right], \nu^{\underline{p}} \left[ X_W \right] \right\rangle_{[X_W]} \\
&\quad = \left\langle \bigcup_{j \in [m]} \left\{ \beta^{h_\ell|_{o^j_{[n]}}} \left[ Y_{\ell,j} \right], \tau^{\ell,(\mathcal{U},y_{\mathcal{U}},\theta)} \left[ Y_{\ell,j} \right] \right\} \cup \left\{ \nu^{\underline{p}} \left[ X_W \right] \right\} \right\rangle_{[X_W]} .
\end{aligned}$$

For each $j \in [m]$ we have by Assumption 1

$$\left\langle \beta^{h_\ell|_{o^j_{[n]}}}\left[Y_{\ell,j}\right], \tau^{\ell,(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{\ell,j}\right]\right\rangle_{[X_W]}$$
$$= \left\langle \{\beta^{g_k|_{o^j_{[n]}}}\left[Y_{k,j}\right] : k \in [d]\} \cup \{\beta^{f_\ell}\left[Y_f, Y_{k,j}\right], \tau^{\ell,(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_f\right]\}\right\rangle_{[X_W]}.$$

From Assumption 2 we know

$$\left\langle \{\beta^{g_k|_{o^j_{[n]}}}\left[Y_{k,j}\right] : k \in [d],\ j \in [m]\} \cup \{\nu^{\underline{p}}\left[X_W\right]\}\right\rangle_{\left[Y_{[d]\times[m]}\right]} = \frac{\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}}{2^{d\cdot m}} \cdot \mathbb{I}\left[Y_{[d]\times[m]}\right].$$

Combining the above we get

$$\begin{aligned}
&\left\langle \beta^{Q|_\mathcal{D}}\left[Y_{[p]}, X_W\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]} \\
&= \frac{\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}}{2^{d\cdot m}} \cdot \\
&\left(\prod_{o_{[n]} : \underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(\mathcal{U},y_\mathcal{U},\theta)}\left[Y=1\right]\right) \prod_{j\in[m]} \left\langle \beta^t\left[Y_{[p]}, Y_{[d]\times\{j\}}\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{[p]}\right]\right\rangle_{[\varnothing]} \\
&= \left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]} \cdot \left(\prod_{o_{[n]} : \underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(\mathcal{U},y_\mathcal{U},\theta)}\left[Y=1\right]\right) \\
&\left(\frac{1}{2^d} \cdot \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{[p]}\right]\right\rangle_{[\varnothing]}\right)^m .\square
\end{aligned}$$

We notice that for the event $p|_{x_W} = \underline{p}$ to be of non-vanishing probability, we need to have $y_\mathcal{U} = \mathbb{I}_\mathcal{U}$ or $\underline{p}\left[O_p\right] = \mathbb{I}\left[O_p\right]$. With this lemma, we are now show Thm. 19.2.

*Proof of Thm. 19.2.* We have

$$\begin{aligned}
\ln\left[\mathbb{P}^{\#Q,(\mathcal{U},y_\mathcal{U},\theta)}\left[X_W = x_W \middle| p|_{x_W} = \underline{p}\right]\right] &= \ln\left[\left\langle \beta^{Q|_\mathcal{D}}\left[Y_{[p]}, X_W\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W=x_W]}\right] \\
&\quad - \ln\left[\left\langle \beta^{Q|_\mathcal{D}}\left[Y_{[p]}, X_W\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]
\end{aligned}$$

While the second term is decomposed by Lem. 19.2 we now derive a decomposition of the first term. By Assumption 1 and $p|_{x_W} = \underline{p}$ we have

$$\begin{aligned}
&\left\langle \beta^{Q|_\mathcal{D}}\left[Y_{[p]}, X_W\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W=x_W]} \\
&= \left(\prod_{o_{[n]} : \underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(\mathcal{U},y_\mathcal{U},\theta)}\left[Y=1\right]\right) \cdot \\
&\quad \prod_{j\in[m]} \left\langle \{\beta^{g_k|_{o^j_{[n]}}}\left[Y_k, X_W = x_W\right] : k \in [d]\} \cup \{\beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(\mathcal{U},y_\mathcal{U},\theta)} Y_{[p]}\}\right\rangle_{[\varnothing]}.
\end{aligned}$$

With Lem. 19.2 we then have

$$\begin{aligned}
&\frac{1}{m}\ln\left[\mathbb{P}^{\#Q,(\mathcal{U},y_{\mathcal{U}},\theta)}\left[X_W = x_W \middle| p|_{x_W} = \underline{p}\right]\right] \\
&= \frac{1}{m}\sum_{j\in[m]}\ln\left[\left\langle\{\beta^{g_k|_{o^j_{[n]}}}\left[Y_k, X_W = x_W\right] : k\in[d]\}\cup\{\beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(\mathcal{U},y_{\mathcal{U}},\theta)}Y_{[p]}\}\right\rangle_{[\varnothing]}\right] \\
&\quad - \ln\left[\left\langle\{\beta^{g_k|_{o^j_{[n]}}}\left[Y_k, X_W\right] : k\in[d]\}\cup\{\beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(\mathcal{U},y_{\mathcal{U}},\theta)}Y_{[p]}\}\right\rangle_{[\varnothing]}\right] - \frac{\ln\left[\left\langle\nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]}{m} \\
&= \mathbb{H}\left[\mathbb{P}^D, \mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)}\right] - \frac{\ln\left[\left\langle\nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]}{m}.\square
\end{aligned}$$

Let us now investigate, in which cases the Assumption 2 of independent data can be matched.

*Example 19.3* If the $p$ and $g_0, \ldots, g_{d-1}$ are predicates applied on variables, and the index tuples $o^j_{[n]}$ are pairwise different, then Assumption 2 is met. This is the case, since the values $g_k\left[O_{[n]} = o^j_{[n]}\right]|_{x_W}$ are determined by different variables in $X_W$.

There are situations, where Assumption 2 is violated.

- extraction formula being a) conjunctions of predicates: Probability that they are satisfied decreases b) disjunctions of predicates: Probability that they are satisfied increases
- extraction formula coinciding with importance formula: Always satisfied, in this case still boolean
- extraction formulas contradicting each other, more general not independent from each other

*Remark 19.1 (Approximation by Independent Samples)* As argued above, we do not have independent samples in general. As a consequence, we cannot apply Lem. 19.2 to decompose the partition function term of the log-probability into factors to each solution map of $p$. In this case, it might be still benefitial to use the reduction to the likelihood of a HLN, but needs to understand it as a approximation to the true world probability.

If the expectations of each sample with respect to the marginalized distributions coincide, the average of empirical distribution also coincides with these (by linearity). When the creation of samples has sufficient mixing properties, the empirical distribution converges to this expectation in the asymptotic case of large numbers of samples.

### 19.3.7 Sample Extraction from First-Order Logic Worlds

We have observed that in certain situations the log-likelihood of a first-order logic world with respect to a Hybrid First-Order Logic Network coincides with the likelihood of a data set in a propositional Hybrid Logic Network. Let us now investigate the extraction process of these data set. We model the extraction process as a relation between a tuple of individuals and the extracted world in the factored system of atoms $X_k$.

**Definition 19.2** Given a first-order logic world $x_W$, an importance formula $p$ and extraction formulas $g_k$ for $k \in [d]$, we define the extraction relation

$$\mathcal{R} \subset \left(\bigtimes_{l\in[n]}[r]\right)\otimes\left(\bigtimes_{k\in[d]}[2]\right)$$

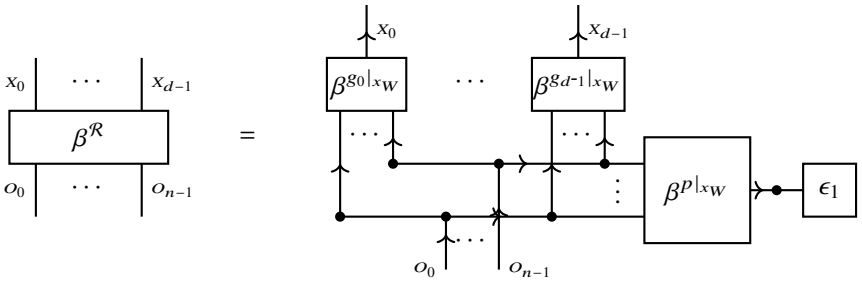
by

$$\mathcal{R} = \left\{(o_{[n]}, x_{[d]}) : p|_{x_W}\left[O_{[n]} = o_{[n]}\right] = 1, \forall k\in[d] : x_k = g_k\left[O_{[n]} = o_{[n]}\right]\right\}.$$

The encoding of an extraction relation is the tensor

$$\beta^{\mathcal{R}}\left[O_{[n]}, X_{[d]}\right] \subset \left(\bigotimes_{l\in[n]}\mathbb{R}^r\right)\otimes\left(\bigotimes_{k\in[d]}\mathbb{R}^2\right)$$

and drawn in a contraction diagram by




Here the contraction of $\beta^p$ with the truth vector $\epsilon_1$ represents the matching condition posed by $p$ when extracting pairs of individuals.

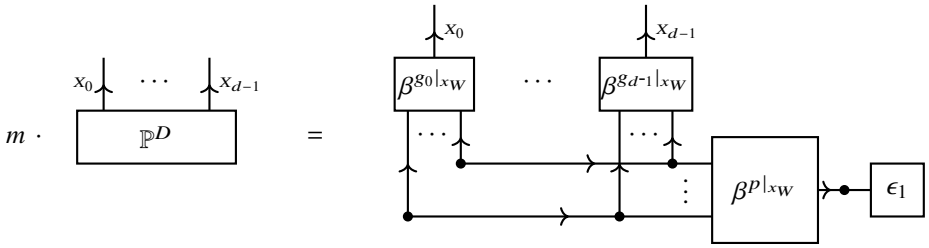
The empirical distribution of the extracted data is then the normalized contraction leaving only the legs to the extracted atomic formulas open, that is

$$\mathbb{P}^D = \frac{\left\langle \beta^{\mathcal{R}} \right\rangle_{[X_{[d]}]}}{\left\langle \beta^{\mathcal{R}} \right\rangle_{[\varnothing]}} .$$

Here the number of extracted data is the denominator

$$m = \left\langle \beta^{\mathcal{R}} \right\rangle_{[\varnothing]} = \left\langle \beta^p \left[ Y_p, O_{[n]} \right], \epsilon_1 \left[ Y_p \right] \right\rangle_{[\varnothing]} .$$

We depict this by




To connect with the empirical distribution introduced in Sect. 5.8.1 we now show how the empirical distribution extracted from the interpretations of the formulas $p, g_0, \ldots, g_{d-1}$ on a first-order logic world $x_W$ can be represented by tensor networks.

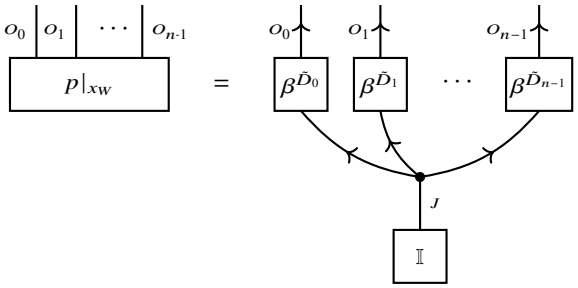
First of all, we decompose the importance formula into a basis CP format (see Chapter **??**), that is a decomposition

$$p|_{x_W} \left[ O_{[n]} \right] = \left\langle \{ \rho^l \left[ O_l, J \right] \, : \, l \in [n] \} \right\rangle_{[O_{[n]}]}$$

such that all $\rho^l \left[ O_l, J \right]$ are directed and boolean tensors. Here an auxiliary variables $J$ taking values in $[m]$ is introduced, which we call the data variable, which enumerates the non-vanishing coordinates of $p|_{x_W}$. With this decomposition, we can understand the decomposition of $p|_{x_W} \left[ O_{[n]} \right]$ as a basis encoding of an term selection map $\tilde{D}$ with coordinate maps defined such that

$$\beta^{\tilde{D}_l} \left[ O_l, J \right] = \rho^l \left[ O_l, J \right] .$$

We depict this decomposition by:




Based on these construction, we now provide a tensor network decomposition of the extracted empirical distribution.

**Theorem 19.3** *Given a first-order logic world $x_W$, an importance formula $p$ and extraction formulas $g_k$ for $k \in [d]$, we have*

$$\beta^{\mathcal{R}}\left[O_{[n]}, X_{[d]}\right] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\} \right\rangle_{\left[O_{[n]}, X_{[d]}\right]}$$

*and thus*

$$\mathbb{P}^D\left[X_{[d]}\right] = \frac{1}{m}\left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\} \right\rangle_{\left[X_{[d]}\right]} .$$

*Proof.* To show the first claim, let us choose arbitrary state tuples $o_{[n]}$ and $x_{[d]}$. We then have

$$\begin{aligned}
&\left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\} \right\rangle_{\left[O_{[n]}=o_{[n]}, X_{[d]}=x_{[d]}\right]} \\
&\quad = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k = x_k, O_{[n]} = o_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l = o_l, J\right] \,:\, l \in [n]\} \right\rangle_{[\varnothing]} .
\end{aligned}$$

This contraction evaluates to 1, if and only if for all $k \in [d]$ we have $\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] = 1$ and

$$\left\langle \{\beta^{\tilde{D}_l}\left[O_l = o_l, J\right] \,:\, l \in [n]\} \right\rangle_{[\varnothing]} = 1 .$$

The first condition is equal to $x_k = g_k\left[O_{[n]} = o_{[n]}\right]$ for all $k \in [d]$ and the second to

$$p|_{x_W}\left[O_{[n]} = o_{[n]}\right] = 1 .$$

Comparing with the definition of the extraction relation (see Def. 19.2), we notice that these conditions are equal to $(o_{[n]}, x_{[d]}) \in \mathcal{R}$ and therefore to

$$\beta^{\mathcal{R}}\left[O_{[n]} = o_{[n]}, X_{[d]} = x_{[d]}\right] .$$

The first claim follows, since $\beta^{\mathcal{R}}$ is boolean, as is the contraction of the cores $\beta^{g_k|_{x_W}}$ with the cores $\beta^{\tilde{D}_l}$, which leaves the outgoing variables $X_{[d]}$ open. The second claim follows from the first using that $\mathbb{P}^D\left[X_{[d]}\right] = \frac{1}{m}\left\langle \beta^{\mathcal{R}} \right\rangle_{\left[X_{[d]}\right]}$. □

To connect with the representation of empirical distributions based on data cores (see Sect. 5.8.1), we now form data cores by contractions with the grounding of extraction formulas with the cores $\beta^{\tilde{D}_l}$ (see Figure 19.1),

$$\beta^{D_k}[X_k, J] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right]\} \cup \{\rho^l\left[O_l, J\right] \,:\, l \in [n]\} \right\rangle_{[X_k, J]} .$$

The empirical distribution is then a tensor network of these tensors, as we show next.

**Theorem 19.4** *We have*

$$\left\langle \beta^{\mathcal{R}} \right\rangle_{\left[X_{[d]}\right]} = \left\langle \{\beta^{D_k}[J, X_k] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}\right]}$$

*and thus*

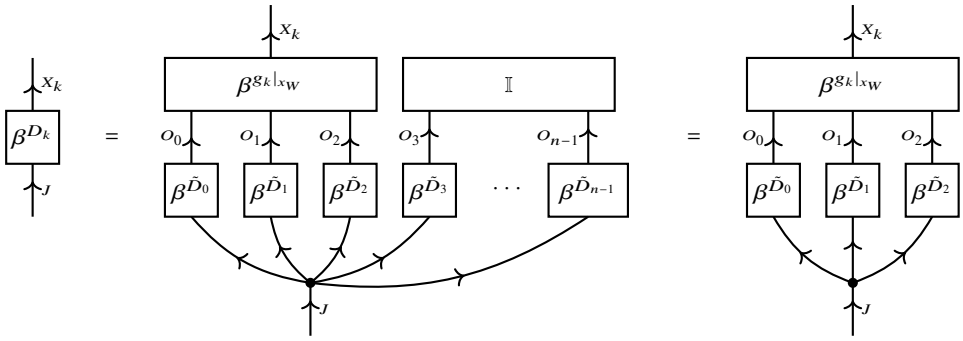



**Fig. 19.1** Generation of a data core for the variable $X_k$ given an extraction formula $g_k$ and an importance formula, which grounding is decomposed into a basis CP format with leg vectors $\beta^{\tilde{D}_l}[O_l, J]$. Term variables, which are appearing in the importance formula, but not in the extraction formula $g_k$ can be treated trivally by contraction with the trivial tensor (here $O_4, \ldots, O_{n-1}$).

$$\mathbb{P}^D\left[X_{[d]}\right] = \frac{1}{m}\left\langle \{\beta^{D_k}[J, X_k] \,:\, k \in [d]\}\right\rangle_{[X_{[d]}]} .$$

*Proof.* By Thm. 19.3 we have

$$\beta^{\mathcal{R}}\left[O_{[n]}, X_{[d]}\right] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\}\right\rangle_{[O_{[n]}, X_{[d]}]} .$$

Since $\beta^{\tilde{D}_l}[O_l, J]$ are directed and boolean, they can be copied and separately contracted with each $g_k|_{x_W}$, without changing the contraction. We arrive at

$$\begin{aligned}
&\beta^{\mathcal{R}}\left[O_{[n]}, X_{[d]}\right] \\
&= \left\langle \left\{ \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\}\right\rangle_{[X_k, J]} \,:\, k \in [d]\right\}\right\rangle_{[O_{[n]}, X_{[d]}]} \\
&= \left\langle \{\beta^{D_k}[J, X_k] \,:\, k \in [d]\}\right\rangle_{[X_{[d]}]} ,
\end{aligned}$$

which established the claim. □

When many atom extraction formulas differ only by a constant, we can replace the constant by an auxiliary term variable. The atoms are then the atomizations of this variable (see Sect. **??**), treated as a categorical variable, with respect to the constant in the extraction query. The advantages are that we can avoid the $\beta$-formalism and directly model the categorical distributions.

This also enables a batchwise computation of multiple SPARQL queries, which differ only in one constant.

## 19.4 Generation of First-Order Logic Worlds

So far we have discussed, how Probabilistic Relational Models for first-order logic Knowledge Bases such as Knowledge Graphs can be built by extracting data. Conversely, any binary tensor can be interpreted as a Knowledge Graph. To be more precise, we follow the intuition that the ones coordinates mark possible worlds compatible with the knowledge about a factored system. Each possible world can then be encoded in a subgraph of the Knowledge Graph representing the world. This amounts to an ”inversion” of the data generation process described in the subsectionion above.

In the previous section we have described a way to extract an effective empirical distribution for the likelihood of a first-order logic world given a Hybrid First-Order Logic Network. We now want to investigate methods to reproduce an empirical distribution based on a constructed first-order logic world.

**Definition 19.3 (Reproduction of Empirical Distributions)** Given an empirical distribution $\mathbb{P}^D \in \bigotimes_{k\in[d]} \mathbb{R}^2$, we say that a triple $(x_W, p, g_{[d]})$ of a first-order logic world $x_W$ an importance formula $p$ and extraction formulas

$g_{[d]} = \{g_k \,:\, k \in [d]\}$ reproduces $\mathbb{P}^D$, when

$$\mathbb{P}^D = \left\langle \{p|_{x_W}\left[O_{[n]}\right]\} \cup \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\}\right\rangle_{[X_{[d]}|\varnothing]} .$$

Note that for distribution $\mathbb{P}$ to be reproducible, it needs to have rational coordinates. If any only if all coordinates are rational, we find a $m \in \mathbb{N}$ such that im $(m \cdot \mathbb{P}) \subset \mathbb{N}$. We can then interpret $m$ as the number of samples, and construct a sample selector map by understanding each coordinate of $m \cdot \mathbb{P}$ as the number of appearances of the respective world in the samples.

We show different schemes and give examples on Knowledge Graphs, where we provide examples for importance and extraction formulas by SPARQL queries.

### 19.4.1 Samples by Single Objects

In the first reproduction scheme we construct datapoints by dedicated objects, which represent a sample, that is we choose a domain $\mathcal{D} = [m]$.

**Theorem 19.5** *Let there be an empirical distribution $\mathbb{P}^D$ to a sample selector map $D$ (see Def. 5.16), we construct a world $x_W[L, O]$ with $d$ unary predicates by*

$$x_W[L, O] = \sum_{k\in[d]} \sum_{j\in[m]\,:\,D_k(j)=1} \epsilon_k[L] \otimes \epsilon_j[O] .$$

*We further choose a trivial importance query, that is*

$$p|_{x_W}[O] = \mathbb{I}[O] ,$$

*and extraction queries coinciding with the unary predicates, that is for $k \in [d]$*

$$g_k = g_k .$$

*Then, the triple $(x_W, p, g_{[d]})$ reproduces $\mathbb{P}^D$.*

*Proof.* By Thm. 19.4 it is enough to show, that the data cores constructed from the data extraction process coincide with those of $\mathbb{P}^D$. We enumerate to this end the non-vanishing coordinates of $p|_{x_W}$ by the data variable $J$ taking values $j \in [m]$, as

$$p|_{x_W}[O = j] = 1$$

and choose

$$\tilde{D} = \delta .$$

For arbitrary $k \in [d]$ and $j \in [m]$ we now have

$$\begin{aligned}\beta^{D_k}[X_k, J = j] &= \left\langle \beta^{g_k|_{x_W}}[X_k, O], \rho^0[O, J = j]\right\rangle_{[X_k,J]} \\ &= \left\langle \beta^{g_k|_{x_W}}[X_k, O], \epsilon_{\tilde{D}(j)}[O]\right\rangle_{[X_k,J=j]} \\ &= \epsilon_{D_k(j)}[X_k] .\end{aligned}$$

This coincides with the slice of the data core of the CP representation of empirical distributions used in Thm. 5.16. Since the slice and the core was arbitrary, the tensor network representations in Thm. 5.16 and Thm. 19.4 are equal and thus the triple $(x_W, p, g_{[d]})$ reproduces $\mathbb{P}^D$. □

We now give by the next theorem an example of a Knowledge Graph with SPARQL queries reproducing and arbitrary empirical distribution.

**Theorem 19.6** *Let $\mathbb{P}^D$ be an empirical distribution to the sample selector $D$. We construct a Knowledge Graph of the resources $\mathcal{D} = \{s_j \,:\, j \in [m]\} \cup \{C\} \cup \{C_k \,:\, k \in [d]\}$, where $s_j$ represent samples and $C_k$ unary predicates, by*

$$\mathrm{RDF}|_{x_W} = \sum_{j\in[m]} \epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\mathrm{rdftype}}} O_p \otimes \epsilon_{I_C} O_o + \sum_{j\in[m]} \sum_{k\in[d]\,:\, D_k(j)=1} \epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\mathrm{rdftype}}} O_p \otimes \epsilon_{I_{C_k}} O_o \,.$$

*We further define an importance formula by the* SPARQL *query*

$$p = \textit{SELECT}\ \{\ ?x\ \}\ \textit{WHERE}\ \{\ ?x \quad \mathrm{rdf:type} \quad C\ .\}$$

*and for each $k \in [d]$ an extraction formula by the query*

$$g_k = \textit{SELECT}\ \{\ ?x\ \}\ \textit{WHERE}\ \{\ ?x \quad \mathrm{rdf:type} \quad C_k\ .\}\ .$$

*Then the triple $(\mathrm{KG}|_{x_W}, p, g_{[d]})$ reproduces $\mathbb{P}^D$.*

*Proof.* We show the theorem analogously to Thm. 19.5, with the slide difference in the importance formula. We have for the grounding of $p$ on $\mathrm{KG}|_{x_W}$ that

$$p|_{x_W}\,[O] = \sum_{j\in[m]} \epsilon_{I_{s_j}} O$$

and enumerate the non-vanishing coordinates by $J$.

For each extraction formula we have

$$g_k|_{x_W}\,[O] = \sum_{j\in[m]\,:\, D_k(j)=1} \epsilon_{I_{s_j}} O\,.$$

It follows that the data cores used in Thm. 19.4 are

$$\beta^{D_k}\,[X_k, j] = \epsilon_0\,[X_k] \otimes \left( \sum_{j\in[m]\,:\, D_k(j)=0} \epsilon_j\,[J] \right) + \epsilon_1\,[X_k] \otimes \left( \sum_{j\in[m]\,:\, D_k(j)=1} \epsilon_j\,[J] \right)$$

and they thus coincide with those in the decomposition in Thm. 5.16. The claim follows therefore with the same argumentation as in the proof of Thm. 19.5. □

Let us provide some more insights on the construction of the reproducing Knowledge Graph in Thm. 19.6. By the insertions to the one-hot encodings $\epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\mathrm{rdftype}}} O_p \otimes \epsilon_{I_C} O_o$ we mark each sample representing resource by a class and ensure its appearance as a owl : NamedIndividual in the graph. The insertions $\epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\mathrm{rdftype}}} O_p \otimes \epsilon_{I_{C_k}} O_o$ on the other side encode the sample selecting map, by inserting exactly the assertions corresponding with the respective sample. In this simple Knowledge Graph, Description Logic is expressive enough to represent any formula $q$ composed of the formulas $g_0, \ldots, g_{d-1}$.

### 19.4.2 Samples by Tuples of Objects

We now instantiate multiple objects for each datapoint, one for each variable of the importance formula, i.e. $\mathcal{D} = [m] \times [n]$ Label individuals $s_{j,l}$ by data index and variable index.

**Lemma 19.3** *Let there a data map $D$, queries $p, g_{[d]}$ and a first-order logic world containing objects $s_{j,l}$ for $j \in [m]$ and $l \in [n]$ If*

$$p|_{x_W} = \sum_{j\in[m]} \bigotimes_{l\in[n]} \epsilon_{I_{s_{j,l}}}\,[O_l]$$

*and for any $k \in [d]$*

$$g_k|_{x_W} = \sum_{j:D_k(j)=1} \bigotimes_{O_l \in O_{g_k}} \epsilon_{I_{s_{j,l}}} [O_l] \ .$$

*Then the tuple* $(\mathrm{KG}|_{x_W}, p, \{g_0, \ldots, g_{d-1}\})$ *reproduces* $\mathbb{P}^D$.

*Proof.* We notice that the grounding of the importance formula is in a basis CP format, since by assumption

$$p|_{x_W} = \sum_{j \in [m]} \bigotimes_{l \in [n]} \epsilon_{I_{s_{j,l}}} [O_l] \ .$$

We choose $J$ to enumerate the non-vanishing entries and get a term selecting map

$$\tilde{D}_l(j) = I_{s_{j,l}} \ .$$

From this we have

$$\left\langle \{\beta^{g_k|_{x_W}} \left[X_k, O_{g_k}\right]\} \cup \{\beta^{\tilde{D}_l} [O_l, J] \ : \ l \in [n]\} \right\rangle_{[X_k, J]} = \beta^{D_k} [X_k, J]$$

and the claim follows with the same argumentation as in the proof of Thm. 19.5. □

## 19.5 Discussion

Statistical Models are called Probabilistic Relational Models. Extensions are models that also handle structural uncertainty, i.e. distributions of worlds with varying $\mathcal{D}$.

In the emerging area of network science [4, 29], statistical models for random graphs are investigated. Statistical Models of first-order logic go beyond the typical single edge type perspective of network science.

*Remark 19.2 (Alternative Representation of empirical distributions)* So far, we have motivated the representation of empirical distributions based on basis CP decompositions based on data maps. In this section, based on the extraction queries, we have observed that empirical distributions might have more efficient representation formats. In many applications such as the computation of log-likelihoods we can use any representation of the empirical distribution by tensor networks. It is thus not necessary to compute the data cores as above, unless one requires a list of the extracted samples.

# Glossary

**Bethe Hypergraph** 117

**Circuit** 100

**Dirac Delta** xiii

**Fourier transform**
  **Quantum Fourier transform** 165

**Quantum Phase Estimation** 168, 170
  **for Grover algorithm** 170

**Reed-Muller Expansion** 138
  **for quantum circuit construction** 161

**Tensor Network** xiv

**Variable Elimination** 118

**Walsh-Hadamard transform** 166

## References

1. Shun-ichi Amari. *Information Geometry and Its Applications*, volume 194 of *Applied Mathematical Sciences*. Springer Japan, Tokyo, 2016.
2. Grigoris Antoniou, Paul Groth, Frank Van Harmelen, and Rinke Hoekstra. *A Semantic Web Primer, third edition*. The MIT Press, Cambridge (Mass.), third edition edition, August 2012.
3. Israel F. Araujo, Carsten Blank, Ismael CS Araújo, and Adenilton J. da Silva. Low-rank quantum state preparation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(1):161–170, 2023.
4. Albert-László Barabási. *Network Science*. Cambridge University Press, Cambridge, illustrated edition edition, July 2016.
5. Armando Bellante, Reinis Irmejs, Marta Florido-Llinàs, María Cea Fernández, Marianna Crupi, Matthew Kiser, and J. Ignacio Cirac. Compiling Quantum Regular Language States, February 2026. arXiv:2602.02698 [quant-ph].
6. Gregory Beylkin and Martin J. Mohlenkamp. Algorithms for Numerical Analysis in High Dimensions. *SIAM Journal on Scientific Computing*, 26(6):2133–2159, January 2005.
7. Alexander Bigerl, Felix Conrads, Charlotte Behning, Mohamed Ahmed Sherif, Muhammad Saleem, and Axel-Cyrille Ngonga Ngomo. Tentris – A Tensor-Based Triple Store. In Jeff Z. Pan, Valentina Tamma, Claudia d'Amato, Krzysztof Janowicz, Bo Fu, Axel Polleres, Oshani Seneviratne, and Lalana Kagal, editors, *The Semantic Web – ISWC 2020*, volume 12506, pages 56–73. Springer International Publishing, Cham, 2020.
8. Carsten Blank and Israel F. Araujo. Tucker iterative quantum state preparation. volume 2744, pages 274–283. 2026. arXiv:2602.09909 [quant-ph].
9. Ronald Brachman and Dr Hector Levesque. *Knowledge Representation and Reasoning*. Morgan Kaufmann, Amsterdam Boston, December 2014.
10. Lawrence D. Brown. *Fundamentals of Statistical Exponential Families*. Institute of mathematical Statistics, Hayward, Calif, June 1987.

11. Peter G. Casazza, Gitta Kutyniok, and Friedrich Philipp. Introduction to Finite Frame Theory. In Peter G. Casazza and Gitta Kutyniok, editors, *Finite Frames: Theory and Applications*, pages 1–53. Birkhäuser, Boston, 2013.
12. George Casella and Roger Berger. *Statistical Inference*. Cengage Learning, Pacific Grove, Calif, June 2001.
13. Goffredo Chirco, Alex Goeßmann, Daniele Oriti, and Mingyi Zhang. Group field theory and holographic tensor networks: dynamical corrections to the Ryu–Takayanagi formula. *Classical and Quantum Gravity*, 37(9):095011, April 2020.
14. Goffredo Chirco, Daniele Oriti, and Mingyi Zhang. Group field theory and tensor networks: towards a Ryu–Takayanagi formula in full quantum gravity. *Classical and Quantum Gravity*, 35(11):115011, May 2018.
15. R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca. Quantum algorithms revisited. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 454(1969):339–354, January 1998.
16. P. Clifford and J. M. Hammersley. Markov fields on finite graphs and lattices. *Unpublished*, 1971.
17. James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
18. Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, N.J, 2nd edition edition, September 2006.
19. Marcus Cramer, Martin B. Plenio, Steven T. Flammia, Rolando Somma, David Gross, Stephen D. Bartlett, Olivier Landon-Cardinal, David Poulin, and Yi-Kai Liu. Efficient quantum state tomography. *Nature Communications*, 1(1):149, December 2010.
20. Gregory M. Crosswhite and Dave Bacon. Finite automata for caching in matrix product algorithms. *Physical Review A*, 78(1):012356, July 2008.
21. Vin de Silva and Lek-Heng Lim. Tensor Rank and the Ill-Posedness of the Best Low-Rank Approximation Problem. *SIAM Journal on Matrix Analysis and Applications*, 30(3):1084–1127, January 2008.
22. Morris H. DeGroot. *Probability and Statistics*. PEARSON INDIA, January 2016.
23. David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, 439(1907):553–558, December 1992.
24. Sergey Dolgov, Boris Khoromskij, and Dmitry Savostyanov. Superfast Fourier Transform Using QTT Approximation. *Journal of Fourier Analysis and Applications*, 18(5):915–953, October 2012.
25. Mike Espig, Wolfgang Hackbusch, Thorsten Rohwedder, and Reinhold Schneider. Variational calculus with sums of elementary tensors of fixed rank. *Numerische Mathematik*, 122(3):469–488, November 2012.
26. Python Software Foundation. Python Language Reference, version 3.13.2, April 2025.
27. Robert G. Gallager. *Low-Density Parity-Check Codes*. The MIT Press, September 1963.
28. Lise Getoor and Ben Taskar. *Introduction to Statistical Relational Learning*. MIT Press, September 2019.
29. Vincenzo Nicosia Giovanni Russo Vito Latora. *Complex Networks: Principles, Methods and Applications. With 58 exercises*. Cambridge University Press, Cambridge, United Kingdom ; New York, NY, September 2017.
30. Alex Goessmann. The Tensor-Network Approach to Efficient and Explainable AI, 2025.
31. Alex Goessmann, Janina Schütte, Maximilian Fröhlich, and Martin Eigel. A tensor network formalism for neuro-symbolic AI, January 2026. arXiv:2601.15442 [cs].
32. Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and Algorithms*. Springer, New York, May 2008.
33. David Gross, Yi-Kai Liu, Steven T. Flammia, Stephen Becker, and Jens Eisert. Quantum State Tomography via Compressed Sensing. *Physical Review Letters*, 105(15):150401, October 2010.
34. Lov Grover and Terry Rudolph. Creating superpositions that correspond to efficiently integrable probability distributions, August 2002. arXiv:quant-ph/0208112.
35. Martin Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*, volume 2 of *Algorithms and Combinatorics*. Springer, Berlin, Heidelberg, 1993.
36. Wolfgang Hackbusch. *Tensor Spaces and Numerical Tensor Calculus*. Springer Series in Computational Mathematics. Springer-Verlag, Berlin Heidelberg, 2012.
37. Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum Algorithm for Linear Systems of Equations. *Physical Review Letters*, 103(15):150502, October 2009.
38. Steven Herbert. No quantum speedup with Grover-Rudolph state preparation for quantum Monte Carlo integration. *Physical Review E*, 103(6):063302, June 2021.
39. Frank L. Hitchcock. The Expression of a Tensor or a Polyadic as a Sum of Products. *Journal of Mathematics and Physics*, 6(1-4):164–189, 1927.
40. Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, and Sebastian Neumaier. *Knowledge Graphs*. Springer, Cham, 1st edition edition, November 2021.
41. Sebastian Holtz, Thorsten Rohwedder, and Reinhold Schneider. On manifolds of tensors of fixed TT-rank. *Numerische Mathematik*, 120(4):701–731, April 2012.
42. Mengyao Hu and Jordi Tura. Tropical Contraction of Tensor Networks as a Bell Inequality Optimization Toolset. *Physical Review Letters*, 136(10):100202, March 2026.
43. A. Yu Kitaev. Quantum measurements and the Abelian Stabilizer Problem, November 1995. arXiv:quant-ph/9511026.
44. Donald E. Knuth. *The Art of Computer Programming, Volume 4B: Combinatorial Algorithms*. Addison-Wesley Professional, October 2022. Google-Books-ID: 6tnPEAAAQBAJ.
45. Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. The MIT Press, Cambridge, Mass., 1. edition edition, July 2009.
46. S. L. Lauritzen and D. J. Spiegelhalter. Local Computations with Probabilities on Graphical Structures and Their Application to Expert Systems. *Journal of the Royal Statistical Society: Series B (Methodological)*, 50(2):157–194, January 1988.
47. Steffen L. Lauritzen. *Graphical Models*. Clarendon Press, Oxford, July 1996.

48. Jin-Guo Liu, Xun Gao, Madelyn Cain, Mikhail D. Lukin, and Sheng-Tao Wang. Computing solution space properties of combinatorial optimization problems via generic tensor networks. *SIAM Journal on Scientific Computing*, 45(3):A1239–A1270, June 2023. arXiv:2205.03718 [cond-mat.stat-mech].
49. Jin-Guo Liu, Lei Wang, and Pan Zhang. Tropical Tensor Network for Ground States of Spin Glasses. *Physical Review Letters*, 126(9):090506, March 2021.
50. Guang Hao Low, Theodore J. Yoder, and Isaac L. Chuang. Quantum inference on Bayesian networks. *Physical Review A*, 89(6):062315, June 2014.
51. David J. C. MacKay. *Information Theory, Inference and Learning Algorithms*. Cambridge University Press, Cambridge, illustrated edition edition, September 2003.
52. Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, February 1977.
53. Daniel Malz, Georgios Styliaris, Zhi-Yuan Wei, and J. Ignacio Cirac. Preparation of Matrix Product States with Log-Depth Quantum Circuits. *Physical Review Letters*, 132(4):040404, January 2024.
54. D.E. Muller. Application of Boolean algebra to switching circuit design and to error detection. *Journal of Symbolic Logic*, 20(2), 1954.
55. Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. The MIT Press, Cambridge, Massachusetts London, England, March 2022.
56. Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. The MIT Press, Cambridge, August 2023.
57. Marc Mézard. *Information, Physics, and Computation*. ACADEMIC, Oxford ; New York, March 2009.
58. Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. Transformation of quantum states using uniformly controlled rotations. *Quantum Info. Comput.*, 5(6):467–473, September 2005.
59. Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. A Review of Relational Machine Learning for Knowledge Graphs. *Proceedings of the IEEE*, 104(1):11–33, January 2016.
60. Isaac L. Chuang Michael A. Nielsen. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, Cambridge ; New York, December 2010.
61. I. V. Oseledets. Tensor-Train Decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, January 2011.
62. Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, s.l., September 1988.
63. Judea Pearl. *Causality: Models, Reasoning and Inference. Ausgezeichnet: ACM Turing Award for Transforming Artificial Intelligence 2011*. Cambridge University Press, Cambridge New York, NY Port Melbourne New Delhi Singapore, 2 edition, November 2009.
64. Judea Pearl, Madelyn Glymour, and Nicholas P. Jewell. *Causal Inference in Statistics: A Primer*. Wiley, Chichester, West Sussex, February 2016.
65. Judea Pearl and Dana Mackenzie. *The Book of Why: The New Science of Cause and Effect*. Basic Books, New York, May 2018.
66. Roger Penrose. *Spinors and Space-Time: Volume 1, Two-Spinor Calculus and Relativistic Fields*. Cambridge University Press, Cambridge, February 1987.
67. John Preskill. Quantum Computing in the NISQ era and beyond. *Quantum*, 2:79, August 2018.
68. I. Reed. A class of multiple-error-correcting codes and the decoding scheme. *Transactions of the IRE Professional Group on Information Theory*, 4(4):38–49, 1954.
69. Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1-2):107–136, February 2006.
70. Ingo Roth, Jadwiga Wilkens, Dominik Hangleiter, and Jens Eisert. Semi-device-dependent blind quantum tomography. *Quantum*, 7:1053, July 2023.
71. Manuel S. Rudolph, Jing Chen, Jacob Miller, Atithi Acharya, and Alejandro Perdomo-Ortiz. Decomposition of matrix product states into shallow quantum circuits. *Quantum Science and Technology*, 9(1):015012, 2023.
72. Manuel S. Rudolph, Jacob Miller, Danial Motlagh, Jing Chen, Atithi Acharya, and Alejandro Perdomo-Ortiz. Synergistic pretraining of parametrized quantum circuits via tensor networks. *Nature Communications*, 14(1):8367, 2023.
73. Sebastian Rudolph. Foundations of Description Logics. In Axel Polleres, Claudia d'Amato, Marcelo Arenas, Siegfried Handschuh, Paula Kroner, Sascha Ossowski, and Peter Patel-Schneider, editors, *Reasoning Web. Semantic Technologies for the Web of Data: 7th International Summer School 2011, Galway, Ireland, August 23-27, 2011, Tutorial Lectures*, pages 76–136. Springer, Berlin, Heidelberg, 2011.
74. Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach, Global Edition: A Modern Approach, Global Edition*. Pearson, Boston, 4 edition, May 2021.
75. Aaron Sander, Maximilian Fröhlich, Mazen Ali, Martin Eigel, Jens Eisert, Michael Hintermüller, Christian B. Mendl, Richard M. Milbradt, and Robert Wille. Quantum circuit simulation with a local time-dependent variational principle, August 2025. arXiv:2508.10096 [quant-ph].
76. Aaron Sander, Maximilian Fröhlich, Martin Eigel, Jens Eisert, Patrick Gelß, Michael Hintermüller, Richard M. Milbradt, Robert Wille, and Christian B. Mendl. Large-scale stochastic simulation of open quantum systems. *Nature Communications*, 16(1):11074, December 2025. arXiv:2501.17913 [quant-ph].
77. Taisuke Sato. A linear algebraic approach to datalog evaluation. *Theory and Practice of Logic Programming*, 17(3):244–265, May 2017.
78. Maria Schuld and Francesco Petruccione. *Machine Learning with Quantum Computers*. Springer, Cham, October 2021.
79. C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
80. Vivek V. Shende, Stephen S. Bullock, and Igor L. Markov. Synthesis of Quantum Logic Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 25(6):1000–1010, June 2006. arXiv:quant-ph/0406176.
81. Efthimis Tsilionis, Alexander Artikis, and Georgios Paliouras. A Tensor-Based Formalization of the Event Calculus. In *Proceedings of the Thirty-ThirdInternational Joint Conference on Artificial Intelligence*, pages 3584–3592, Jeju, South Korea, August 2024. International Joint Conferences on Artificial Intelligence Organization.
82. Martin J. Wainwright and Michael Irwin Jordan. *Graphical Models, Exponential Families, and Variational Inference*. Now Publishers Inc, 2008.
83. Günter M. Ziegler. Lectures on 0/1-Polytopes. In Gil Kalai and Günter M. Ziegler, editors, *Polytopes — Combinatorics and Computation*, pages 1–41. Birkhäuser, Basel, 2000.
84. Günter M. Ziegler. *Lectures on Polytopes*. Springer, New York, 1995th edition edition, October 2013.